

O'REILLY®

Третье
издание

Git: контроль версий

Мощные инструменты и методы для совместной разработки программного обеспечения



SPRINT
book

Прем Кумар Понуторай
Джон Лолигер

THIRD EDITION

Version Control with Git

*Powerful Tools and Techniques for
Collaborative Software Development*

Prem Kumar Ponuthorai and Jon Loeliger

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

ТРЕТЬЕ ИЗДАНИЕ

Git: контроль версий

Мощные инструменты и методы для совместной разработки программного обеспечения

Прем Кумар Понуторай, Джон Лолигер

SPRINT
book 2025

Прем Кумар Понуторай, Джон Лолигер

Git: контроль версий

3-е издание

Перевел с английского Д. Брайт

ББК 32.973.2-018

УДК 004.42

Прем Кумар Понуторай, Джон Лолигер

П56 Git: контроль версий. 3-е изд. — Астана: «Спринт Бук», 2025. — 512 с.: ил.

ISBN 978-601-08-4358-5

Ускорьте свою работу, задействовав Git для отслеживания, ветвления, слияния и управления ревизиями кода. Организованное в виде серии пошаговых уроков, это практическое руководство плавно проводит вас от знакомства с основами Git к использованию продвинутых техник, попутно предоставив дружеские, но при этом серьезные советы по работе со множеством функций этой открытой системы контроля версий.

Авторы Прем Кумар Понуторай и Джон Лолигер детально описывают принципы работы Git, удобно разделяя их по уровням профессионализма. Это тщательно проработанное издание также включает рекомендации по управлению деревьями, развернуто раскрывает тему журнала ссылок и локального хранилища, а в завершение подробно знакомит читателя с платформой GitHub. Как только вы научитесь оперировать гибкими возможностями Git, эта система предоставит вам бесчисленное множество способов управления разработкой кода.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ISBN 978-1492091196 англ.

Authorized Russian translation of the English edition of Version Control with Git, 3rd Edition, ISBN 9781492091196 © 2023 Prem Kumar Ponuthurai.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

ISBN 978-601-08-4358-5

© Перевод на русский язык ТОО «Спринт Бук», 2024

© Издание на русском языке, оформление ТОО «Спринт Бук», 2024

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

Изготовлено в России. Изготовитель: ТОО «Спринт Бук». Место нахождения и фактический адрес:
010000, Казахстан, г. Астана, район Алматы, Проспект Рахымжан Кошкарбаев, д. 10/1, н. п. 18.

Дата изготовления: 10.2024. Наименование: книжная продукция. Срок годности: не ограничен.

Подписано в печать 23.08.24. Формат 70×100/16. Бумага офсетная. Усл. п. л. 41,280. Тираж 1000. Заказ 0000.

Краткое содержание

Вступление	17
------------------	----

Часть I ФИЛОСОФИЯ GIT

Глава 1. Знакомство с Git.....	24
Глава 2. Фундаментальные понятия	46

Часть II ОСНОВЫ GIT

Глава 3. Ветки	75
Глава 4. Коммиты	100
Глава 5. Управление файлами и индекс	123
Глава 6. Слияния	143
Глава 7. Команда diff	177

Часть III БАЗОВЫЕ НАВЫКИ

Глава 8. Поиск коммитов.....	196
Глава 9. Изменение коммитов.....	205
Глава 10. Локальное хранилище и журнал ссылок.....	238
Глава 11. Удаленные репозитории.....	256
Глава 12. Управление репозиторием.....	303

Часть IV
ПРОДВИНУТЫЕ НАВЫКИ

Глава 13. Патчи	330
Глава 14. Хуки.....	354
Глава 15. Подмодули.....	366
Глава 16. Продвинутое управление.....	386

Часть V
СОВЕТЫ И ПРИЕМЫ

Глава 17. Советы, приемы и техники.....	418
Глава 18. Git и GitHub.....	453
Приложение А. История Git.....	499
Приложение Б. Установка Git.....	506
Об авторах	510
Иллюстрация на обложке.....	511

Оглавление

Вступление.....	17
Для кого эта книга	17
Требования.....	17
Новое в третьем издании	18
Навигация по книге	18
Установка Git.....	19
Что не вошло в книгу	19
Условные обозначения.....	19
Благодарности	21
Владельцы торговых марок.....	22
От издательства.....	22

Часть I ФИЛОСОФИЯ GIT

Глава 1. Знакомство с Git	24
Компоненты Git	24
Характеристики Git.....	26
Командная строка Git	28
Использование Git: быстрый старт	30
Подготовка к работе с Git.....	31
Работа с локальным репозиторием	31
Работа с общим репозиторием	41
Файлы конфигурации	42
Итоги	45

Глава 2. Фундаментальные понятия	46
Репозитории	46
Хранилище объектов Git.....	47
Индекс.....	48
Контентно-адресуемая база данных.....	50
Отслеживание содержимого в Git	51
Имя пути или содержимое	52
Файлы packfile	53
Визуализация хранилища Git.....	54
Внутренние механики Git: принципы в действии	57
Внутри каталога .git	57
BLOB-объекты и хеши	59
Объект дерева и файлы	60
Особенности использования SHA-1 в Git.....	61
Иерархия деревьев	63
Объекты коммита	64
Объекты тегов	67
Итоги	68

Часть II ОСНОВЫ GIT

Глава 3. Ветки.....	75
Зачем в Git нужны ветки	76
Руководство по ветвлению	76
Имена веток.....	77
Правила именования веток	78
Управление ветками	78
Работа в ветках.....	79
Создание веток.....	81
Вывод списка имен веток.....	83
Просмотр веток и их коммитов.....	83
Переключение веток.....	86
Внесение изменений в другую ветку.....	90

Создание новой ветки и переключение на нее.....	93
Отсоединенная HEAD	94
Удаление веток	96
Итоги	99
Глава 4. Коммиты	100
Коммиты: зафиксированные единицы изменений.....	100
Атомарные наборы изменений	101
Идентификация коммитов	102
Абсолютные имена коммитов.....	103
Ссылки и символические ссылки	104
Относительные имена коммитов	106
История коммитов	108
Просмотр старых коммитов	109
Графы коммитов	112
Диапазоны коммитов.....	117
Итоги	122
Глава 5. Управление файлами и индекс	123
Важность индекса.....	123
Классификация файлов в Git	125
Использование git add.....	127
Заметки по использованию git commit	131
Использование git commit --all.....	131
Запись сообщений коммитов.....	133
Использование git rm.....	133
Использование git mv	136
Заметки по отслеживанию переименования	138
Файл .gitignore	139
Итоги	142
Глава 6. Слияния	143
Слияние: техническая сторона	143
Примеры слияния	144
Подготовка к слиянию.....	144

Слияние двух веток.....	144
Когда слияние ведет к конфликту.....	147
Работа с конфликтами слияния.....	150
Обнаружение конфликтующих файлов	151
Проверка конфликтов.....	152
Как Git отслеживает конфликты.....	158
Последний этап разрешения конфликта.....	160
Отмена или перезапуск слияния.....	161
Стратегии слияния.....	162
Вырожденные слияния	164
Нормальные слияния	168
Специализированные слияния.....	170
Применение стратегий слияния.....	171
Драйверы слияния.....	172
Как Git анализирует слияния	173
Слияния и объектная модель Git	173
Сжатое слияние (squash merge).....	174
Почему бы просто не вливать изменения одно за другим?	175
Итоги	176
Глава 7. Команда diff	177
Формы команды git diff.....	179
Простой пример git diff	183
Интерпретация вывода git diff	186
git diff и диапазоны коммитов.....	188
git diff с ограничением пути	191
Как Git выводит отличия	193
Итоги	194

Часть III

БАЗОВЫЕ НАВЫКИ

Глава 8. Поиск коммитов	196
Использование git bisect.....	196
Использование git blame	203

Использование инструмента Pickaxe	204
Итоги	204
Глава 9. Изменение коммитов	205
Изменение истории коммитов. Общие принципы	206
Изменения истории: будьте осторожны	207
Использование git revert	209
Изменение HEAD-коммита	210
Использование git reset	213
Использование git cherry-pick	221
reset, revert и checkout	223
Перебазирование коммитов	224
Использование git rebase -i	227
Сравнение rebase и merge	231
Итоги	237
Глава 10. Локальное хранилище и журнал ссылок	238
Локальное хранилище	238
Пример: прерывание рабочего процесса	238
Пример: обновление локальной работы изменениями из вышестоящего репозитория	244
Пример: преобразование изменений из локального хранилища в ветку	247
Журнал ссылок (reflog)	250
Итоги	255
Глава 11. Удаленные репозитории	256
Блок I: основные концепции	257
Пустые репозитории и репозитории разработки	257
Клоны репозитория	259
Удаленные репозитории	260
Ветки отслеживания	261
Ссылки на другие репозитории	263
Обращение к удаленным репозиториям	263
Refspec	265

Блок II: пример использования удаленных репозиториях	268
Создание достоверного репозитория	269
Создание собственного удаленного подключения origin	271
Разработка в вашем репозитории	273
Передача изменений	274
Добавление нового разработчика	275
Получение обновлений репозитория	277
Блок III: цикл разработки в удаленном репозитории в картинках	283
Клонирование репозитория	284
Альтернативные истории	285
Передача изменений без перемотки	286
Получение альтернативной истории	287
Слияние историй	288
Конфликты слияния	288
Передача объединенной истории	289
Блок IV: конфигурация удаленного репозитория	290
Использование git remote	291
Использование git config	292
Ручное редактирование	293
Блок V: работа с ветками отслеживания	293
Создание веток отслеживания	294
Опережение и отставание	297
Добавление и удаление удаленных веток	298
Голые репозитории и git push	300
Итоги	301
Глава 12. Управление репозиторием	303
Публикация репозиториях	304
Репозитории с управляемым доступом	304
Репозитории с анонимным доступом к чтению	306
Репозитории с анонимным доступом к записи	310
Совет по публикации репозиториях	310
Структура репозиториях	311
Структура общего репозитория	311
Распределенная структура репозиториях	312

Практическая сторона распределенной разработки	313
Изменение публичной истории.....	313
Разделение коммита и публикации	314
Не существует единственной истинной истории.....	315
Знать свое место	316
Вышестоящий и нижестоящий репозитории	316
Роли разработчика и мейнтейнера.....	317
Взаимодействие между мейнтейнером и разработчиком.....	318
Двойные роли	319
Работа с несколькими репозиториями	320
Собственное рабочее пространство.....	320
С чего начать свой репозиторий.....	321
Преобразование в другой вышестоящий репозиторий	322
Использование нескольких вышестоящих репозиторияев.....	324
Форки проектов	325
Итоги	328

ЧАСТЬ IV. ПРОДВИНУТЫЕ НАВЫКИ

Глава 13. Патчи	330
Зачем использовать патчи?	331
Генерация патчей	332
Патчи и топологическая сортировка	340
Отправка патчей по электронной почте	341
Применение патчей	344
Плохие патчи	352
Патчи и слияния	353
Итоги	353
Глава 14. Хуки	354
Типы хуков	354
Об использовании хуков	355
Установка хуков.....	357
Шаблоны хуков.....	357

Создание первого хука	359
Доступные хуки	361
Хуки для коммитов	361
Хуки для патчей	362
Хуки для передачи	363
Другие хуки локального репозитория	364
Создавать хук или нет?	365
Итоги	365
Глава 15. Подмодули	366
Gitlink	366
Подмодули	369
Зачем нужны подмодули	370
Работа с подмодулями	370
Подмодули и переиспользование учетных данных	379
Поддеревья Git	380
Добавление подпроекта	380
Получение обновлений подпроекта	382
Изменение подпроекта из суперпроекта	383
Визуальное сравнение подмодулей и поддеревьев Git	384
Итоги	385
Глава 16. Продвинутое управление	386
Интерактивное индексирование фрагментов	386
Полезная команда git rev-list	397
Переключение на основе даты	398
Предостережения при переключении на основе даты	399
Извлечение старой версии файла	401
Восстановление утраченного коммита	403
Команда git fsck	403
Восстановление связи с потерянным коммитом	407
Использование git filter-repo	408
Примеры использования git filter-repo	409
Итоги	416

Часть V

СОВЕТЫ И ПРИЕМЫ

Глава 17. Советы, приемы и техники.....	418
Интерактивное перебазирование при грязном рабочем каталоге.....	418
Сборка мусора	419
Советы по восстановлению коммитов	422
Восстановление после перебазирования вышестоящего репозитория	422
Быстрый обзор изменений.....	424
Очистка	425
Использование git-grep для поиска по репозиторию	426
Обновление и удаление ссылок	428
Отслеживание перемещаемых файлов.....	429
Автоматизация повторений	430
Миграция в Git	431
Перенос из системы Git	431
Перенос из другой системы контроля версий.....	435
Примечание по работе с крупными репозиториями	440
Git LFS.....	441
Репозиторий до и после Git LFS	442
Установка Git LFS.....	444
Отслеживание с помощью Git LFS крупных объектов.....	445
Полезные техники в работе с Git LFS.....	449
Преобразование репозитория для использования Git LFS	451
Итоги	452
Глава 18. Git и GitHub.....	453
О GitHub	453
Виды учетных записей GitHub	454
GitHub в экосистеме Git.....	458
Размещение репозитория на GitHub.....	461
Представление репозитория	464
Code	467

Тикеты в GitHub	470
Пул-реквесты	472
GitHub Flow	482
Разрешение конфликтов слияния в GitHub	486
Потоки разработки	493
Интеграция с GitHub	496
Итоги	498
Приложение А. История Git.....	499
Рождение Git.....	500
Эволюция систем контроля версий.....	503
Хронология	504
Что означает название Git	505
Приложение Б. Установка Git.....	506
Использование бинарных дистрибутивов Linux	506
Debian/Ubuntu	506
Другие бинарные дистрибутивы.....	506
Установка Git в macOS	507
Установка Git в Windows.....	508
Получение исходного кода релиза	508
Сборка и установка на основе исходного релиза	508
Об авторах	510
Иллюстрация на обложке.....	511

Вступление

Git — это бесплатная открытая распределенная система контроля версий, созданная Линусом Торвалдсом. Она не требует больших операционных затрат, но при этом гибкая и достаточно мощная, так что способна поддерживать требования сложных, масштабных и распределенных проектов.

Цель этой книги — показать, как максимально эффективно использовать Git и с легкостью управлять репозиторием внутри этой системы. Вы освоите философию и фундаментальные принципы работы Git, а также получите продвинутые навыки по отслеживанию содержимого, совместной работе и управлению проектами в командах.

Для кого эта книга

Эта книга написана в первую очередь для инженеров в области разработки программного обеспечения (разработчиков, инженеров инфраструктуры, DevOps и т. д.). В связи с этим значительная часть понятий и используемых примеров относится к повседневным задачам специалистов в индустрии ПО. Тем не менее Git — достаточно надежный инструмент для отслеживания содержимого в совершенно различных областях, включая data science, графический дизайн и написание книг. (Кстати, при работе над этой книгой мы использовали Git для отслеживания ревью и правок.) Если вы используете Git для контроля версий, то вне зависимости от вашей должности и уровня знаний на страницах книги найдете для себя много ценного.

Требования

Наличие опыта работы с любой системой версионирования и понимание ее целей помогут освоить принципы Git. Хорошо, если вы умеете пользоваться каким-либо инструментом командной строки, например оболочки Unix, а также знаете базовые команды оболочки, поскольку в примерах и обсуждениях мы будем активно их использовать. Кроме того, плюсом станет общее понимание принципов программирования.

Примеры создавались в средах macOS и Ubuntu. Они должны работать и на других платформах — Debian, Solaris и Windows (через установленные с помощью Git инструменты командной строки, например Git для Windows), но при этом могут быть некоторые отличия.

Для выполнения некоторых упражнений могут потребоваться операции системного уровня, требующие root-доступа на машинах. В подобных ситуациях вы должны четко понимать последствия таких операций.

Новое в третьем издании

Перед вами третье издание книги, где мы применили новый модульный подход к знакомству с принципами Git. Мы начнем с основ и фундаментальной философии этой системы, затем изучим общие команды, которые помогут повысить эффективность потока работы. Завершается книга продвинутыми командами `git` и принципами, которые позволят лучше понять внутреннюю механику Git.

В новое издание мы добавили рисунки, объясняющие сложные принципы Git. Рисунки помогут выстроить ментальную модель для более глубокого понимания сути. Кроме того, мы особо остановились на новой функциональности, появившейся в последнем релизе Git, а также предоставили примеры и советы, которые улучшат ваш текущий поток распределенной разработки.

Навигация по книге

Книга разбита на разделы (части) по категориям, соответствующим разному уровню знакомства читателя с Git. Внутри этих разделов главы выстроены так, чтобы вы могли читать их как по отдельности, так и последовательно, когда темы очередной главы основаны на предыдущих.

В каждой главе мы стремились создать согласованную структуру и применить последовательный подход к объяснению принципов, поэтому рекомендуем внимательно ознакомиться с этой структурой. В будущем это поможет вам в любой момент использовать книгу в качестве удобного руководства.

Если вы не знаете, с чего лучше начать чтение, не волнуйтесь. Таблица В.1 поможет выбрать те главы, которые дадут вам максимум полезных знаний в кратчайший срок.

Таблица В.1. Матрица категорий

	Философия Git	Основы Git	Общие навыки	Продвинутые навыки	Советы и приемы
Разработчики ПО	x	x	x	x	x
Дата-сайентисты	x	x	x		x
Графические дизайнеры	x	x			x

	Философия Git	Основы Git	Общие навыки	Продвинутые навыки	Советы и приемы
Преподаватели	x	x			x
Авторы	x	x			x

Установка Git

Для закрепления знаний настоятельно рекомендуем вам прорабатывать примеры кода. Для этого установите Git на любую платформу, с которой вы обычно работаете. Поскольку этапы установки Git отличаются в зависимости от версии ОС, все соответствующие инструкции мы привели в приложении Б.

Что не вошло в книгу

Благодаря активному сообществу Git постоянно развивается. Например, пока мы писали эту книгу, вышла очередная версия для коммерческого использования: 2.37.1¹. Мы бы не хотели упустить какую-либо информацию в этом издании, но такова неизбежная реальность при написании руководства по постоянно развивающейся технологии.

Мы намеренно решили не освещать все внутренние команды и опции Git, чтобы вы могли сосредоточиться на наиболее распространенных. Мы также не стали описывать все доступные инструменты, связанные с Git, просто потому, что их слишком много.

Но, несмотря на это, мы уверены, что книга даст необходимую базу и подготовит вас к более глубокому погружению в Git. Подробный список изменений последнего релиза можно найти в документации Release Notes (<https://oreil.ly/R2nn4>).

Условные обозначения

В книге используются следующие условные обозначения.

Курсив

Курсивом выделены новые термины или важные понятия.

Моноширинный шрифт

Используется для листингов программ, а также внутри абзацев для обозначения таких элементов, как переменные и функции, базы данных, типы

¹ На момент подготовки русскоязычного издания доступна версия 2.4.5.2. — *Примеч. ред.*

данных, переменные среды, операторы и ключевые слова, имена файлов и их расширения.

Моноширинный полужирный шрифт

Показывает команды или другой текст, который пользователь должен ввести самостоятельно.

Моноширинный курсив

Показывает текст, который должен быть заменен значениями, введенными пользователем, или значениями, определяемыми контекстом.

Шрифт без засечек

Используется для обозначения URL, адресов электронной почты, названий кнопок и других элементов интерфейса, каталогов.



Этот рисунок указывает на совет или предложение.



Этот рисунок указывает на предупреждение.



Этот рисунок указывает на общее примечание.

Вы также должны быть знакомы с базовыми командами оболочки для управления файлами и директориями. Многие примеры будут содержать команды, которые позволяют добавлять/удалять директории, копировать или создавать файлы. Например, такие:

```
# Создание новой директории
$ mkdir newdirectory
```

```
# Запись содержимого в файл
$ echo "Test line" > file.txt
```

```
# Добавление содержимого в конец файла
$ echo "Another line" >> file.txt
```

```
# Создание копии файла
$ cp file.txt copy-of-file.txt
```



```
# Удаление файла
$ rm newdirectory/file
```

```
# Удаление директории
$ rmdir newdirectory
```

Команды, требующие прав `root`, или те, которые должны выполняться с `root`-правами, записываются с помощью операции `sudo`. Например:

```
# Установка основного пакета Git
$ sudo apt-get install git-core
```

Редактировать файлы или вносить изменения в рабочий каталог вы можете любым удобным способом. Также вы должны быть знакомы с текстовым редактором. В этой книге мы обозначаем процесс редактирования файла либо с помощью прямого комментария, либо псевдокомандой:

```
# внесение в file.c нового текста
$ edit index.html
```

Благодарности

Сумма знаний многих перевешивает сумму знаний одного. Этот проект оказался возможен благодаря усилиям множества людей. Я, Прем Кумар Понуторай, лично хочу выразить глубокую признательность Мэттью Маккалоху (Matthew McCullough), который дал мне возможность продолжать знакомить мир с Git и GitHub в новом издании. Также благодарю Джона Лолигера, основного автора книги, чья работа являлась превосходным источником более глубокого понимания тонкостей Git в предыдущих изданиях.

Хочу поблагодарить научных редакторов Джеффа Кинга (Jeff King), Джесс Мэйлс (Jess Males), Аарона Самнера (Aaron Sumner), Дональда Эллиса (Donald Ellis) и Мислава Маронича (Mislav Marohnić), которые вычитывали сырой текст и давали ценнейшую обратную связь для придания главам той структуры, которая была бы понятна всем читателям.

Также хочу поблагодарить Тэйлора Блау (Taylor Blau), который давал советы и полезную обратную связь по первым главам. Это помогло мне сформировать общую структуру последующих частей книги. Ларс Шнайдер (Lars Schneider) разработал идею и концепцию раздела «Git LFS» на с. 441, в основу которой легли его лекции. За это выражаю ему свою глубокую благодарность. Ссылки на работы Элайджи Ньюрен (Elijah Newren), Деррика Столи (Derrick Stolee) и Винсента Дриссена (Vincent Driessen) даны с их разрешения, и я признателен им за этот вклад. Хочу сказать спасибо и Питеру Мюррею (Peter Murray). Наше обсуждение структуры книги помогло мне убедиться, что я иду правильным путем, внося изменения в третье издание.

Благодарю редакторов и сотрудников издательства O'Reilly — в частности, Ширу Эванс, Вирджинию Уилсон, Бет Келли, Одри Дойл, Ким Сандовал, Сью Клефстад, Дэвида Футато, Карен Монт-Гомери, Кейт Дуллеа, Сюзанну Хьюстон и Мелиссу Даффилд (Shira Evans, Virginia Wilson, Beth Kelly, Audrey Doyle, Kim Sandoval, Sue Klefstad, David Futato, Karen Montgomery, Kate Dullea, Suzanne Huston, and Melissa Duffield). Сердечно признателен вам за ваше терпение, мотивацию и доброжелательность, благодаря которым мы смогли довести дело до конца.

В завершение хочу поблагодарить свою жену Теджирл и дочь Темиксчиру за их безусловную моральную поддержку и терпение, за принесенные в жертву работе семейные ужины, романтические вечера и праздники, а самое главное — за веру в меня. Также благодарю своих родителей, Понутория и Джаялетчми, которые привили мне упорство. И отдельная благодарность Ракше — моей белой швейцарской овчарке, которая постоянно находилась рядом, пока я писал эту книгу.

Владельцы торговых марок

Политика торговой марки Git (<https://oreil.ly/YU0BU>) доступна онлайн.

Linux® — зарегистрированная торговая марка Линуса Торвальдса в США и других странах.

PowerPC® — торговая марка корпорации International Business Machines в США и других странах.

Unix® — зарегистрированная торговая марка The Open Group в США и других странах.

От издательства

В книге приведено большое количество фрагментов кода. Все комментарии в коде переведены на русский язык, но сообщения коммитов оставлены на английском, как принято (хотя и не является обязательным) среди разработчиков. Кроме того, так вы сможете воспроизвести приведенные в книге примеры и получить тот же результат, что у авторов.

Ваши замечания, предложения, вопросы отправляйте по адресу

`comp@sprintbook.kz`

(издательство «SprintBook», компьютерная редакция).

Мы будем рады узнать ваше мнение!

ЧАСТЬ I

Философия Git

Первая часть книги знакомит с Git и ее основами. Мы начнем с общего представления экосистемы Git, затем разберем ее характеристики, чтобы вы поняли, в чем заключаются уникальность этого инструмента и его принцип работы. Далее мы рассмотрим примеры работы с Git через командную строку, а также примеры кода.

Во второй части книги углубимся в технические особенности Git. Сначала разберем концепцию репозитория, после чего поговорим о хранилище объектов Git, закрепив понимание того, что, по сути, Git — это контентно-адресуемая база данных. Чтобы подготовить читателя к последующим главам, я рассматриваю внутреннее устройство Git именно в начале книги.

Первые главы части I позволят сформировать правильное мышление в рамках этой системы. Считается, что освоить Git нелегко. Это связано с тем, что понять принципы контроля версий файлов и проектов удастся не сразу, особенно если вы переходите на распределенную Git с традиционной централизованной системы версионирования. Такой переход на первых порах может затруднить понимание Git. Но будьте уверены: научившись мыслить в рамках Git, вы без труда освоите все остальные технические навыки.

ГЛАВА 1

Знакомство с Git

Говоря простыми словами, Git — это система отслеживания содержимого, в ней используются те же принципы, что и в большинстве систем контроля версий. Тем не менее уникальной Git делает ее распределенная природа. Благодаря этому система быстрая и масштабируемая, имеет множество наборов команд, дающих доступ к высокоуровневым и низкоуровневым операциям, а также оптимизирована для выполнения локальных операций.

В этой главе вы узнаете фундаментальные принципы Git, ее характеристики и базовые команды. Вы также получите общее руководство по созданию репозитория и внесению в него изменений.

Настоятельно рекомендуем ознакомиться с приведенными здесь ключевыми принципами. Эти темы — строительные блоки Git, которые помогут понять общие и продвинутые техники управления репозиторием в рамках вашей ежедневной работы. Эти базовые принципы также ускорят ваше обучение, когда мы будем разбирать внутреннее устройство Git в части II «Основы Git», части III «Базовые навыки» и части IV «Продвинутые навыки».

Компоненты Git

Прежде чем углубиться в мир команд `git`, давайте поговорим о компонентах, составляющих экосистему Git. На рис. 1.1 показана взаимосвязь этих компонентов.

Инструменты графического интерфейса (GUI) Git выступают в качестве фронтов для командной строки этой системы. Некоторые из них имеют расширения, которые интегрируются с популярными платформами хостинга Git. Клиентские инструменты Git в основном работают с локальной копией вашего репозитория.

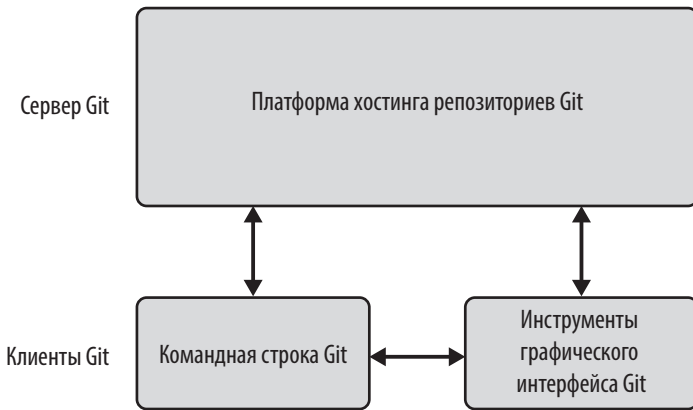


Рис. 1.1. Обзор компонентов Git

При работе с Git типичная конфигурация включает сервер и клиентов Git. Вы можете исключить сервер, но это усложнит поддержку и управление репозиториями при обмене вносимыми в новые ревизии изменениями в совместных проектах и затруднит согласованность (об этом поговорим в главе 11). Сервер и клиенты Git работают так:

Сервер Git

Сервер Git упрощает совместную работу, поскольку обеспечивает доступность центрального и достоверного источника истины для репозитория, в которых вы будете работать. На этом сервере также хранятся ваши удаленные репозитории Git. Как это принято, в репозитории хранятся наиболее актуальные и стабильные исходники ваших проектов. Вы можете установить и настроить собственный сервер Git либо исключить эту дополнительную нагрузку и разместить свои репозитории в надежной сторонней системе — GitHub, GitLab или BitBucket.

Клиенты Git

Клиенты Git взаимодействуют с вашими локальными репозиториями, а вы можете взаимодействовать с клиентами через командную строку Git или инструменты GUI. Установив и настроив клиент Git, вы сможете получать доступ к удаленным репозиториям, работать над локальной копией репозитория, а также передавать изменения обратно на сервер Git. Если вы только знакомитесь с Git, то советую начать с командной строки, изучить наиболее распространенный набор команд `git`, необходимый для выполнения повседневных операций, а затем уже переходить к предпочтительному инструменту GUI.

Причина такого подхода в том, что инструменты GUI могут использовать собственные термины или абстракции, которые отличаются от стандартных команд Git. Например, инструмент с опцией `sync`, которая маскирует внутреннее связывание в цепочку двух или более команд `git` для получения нужного результата. Если по какой-то причине вы введете подкоманду `sync` в командную строку, то можете получить такой непонятный вывод:

```
$ git sync
```

```
git: 'sync' is not a git command. See 'git --help'.
```

```
The most similar command is
    svn
```



`git sync` не является допустимой подкомандой `git`. Чтобы обеспечить синхронизацию локальной копии репозитория с изменениями из удаленного репозитория Git, нужно выполнить комбинацию следующих команд: `git fetch`, `git merge`, `git pull` или `git push`.

Вам доступно огромное число инструментов GUI. Некоторые из них достаточно сложны и могут расширяться через подключаемую модель, позволяющую использовать функциональность сторонних сервисов хостинга Git. Несмотря на все удобство освоения Git через GUI, мы будем применять инструмент командной строки, чтобы заложить прочную основу знаний и дать вам большую гибкость при работе с этой системой.

Характеристики Git

Теперь, когда у нас есть общее представление о компонентах Git, перейдем к знакомству с характеристиками системы. Их понимание позволит вам перестроить ментальную модель с централизованного управления версиями на распределенное. Я называю это «философией Git».

Git сохраняет изменения ревизий в виде снимков

Первый принцип: Git сохраняет множество ревизий файла, над которым вы работаете. В отличие от других систем контроля версий, Git не отслеживает изменение ревизий в виде серии модификаций, которые обычно называют *дельтами*. Git делает моментальные снимки (снапшоты) внесенных в репозиторий изменений в конкретные моменты времени. В терминологии Git это называется *фиксацией* или *коммитом*. Это можно сравнить с фиксацией момента во времени при фотосъемке.

Git расширена под локальную разработку

В Git вы работаете с копией репозитория на своей локальной машине для разработки. Эта копия называется *локальным репозиторием* или клоном удаленного

репозитория на сервере Git. Ваш локальный репозиторий будет иметь ресурсы и снимки изменений ревизии, внесенные в эти ресурсы, все в одном месте. В Git эти коллекции связанных снимков называются *историей коммитов репозитория* или просто *историей репозитория*. Это позволяет вам работать в автономной среде, поскольку Git не нуждается в постоянном подключении к серверу для контроля версий ваших изменений. В результате вы получаете возможность работать над крупными сложными проектами распределенными командами без снижения эффективности и качества операций контроля версий.

Git действует в соответствии с инструкциями

Команды `git` — явные. Git ожидает, пока вы предоставите инструкции о том, что и когда нужно сделать. Например, Git не будет автоматически синхронизировать изменения из локального репозитория с удаленным репозиторием, равно как и сохранять снимок ревизии в историю локального репозитория. Каждое действие ожидает от вас явной команды или инструкции, которая сообщит системе, что от нее требуется, включая создание новых коммитов, исправление существующих, передачу изменений с локального репозитория на удаленный и даже извлечение последних изменений с удаленного репозитория. Вам нужно проявлять свои намерения в действиях. Сюда же относится указание Git, какие файлы вы хотите отслеживать, поскольку система не будет автоматически добавлять для контроля версий новые файлы.

Git предназначена для поддержки нелинейной разработки

С Git вы можете создавать различные реализации функций и экспериментировать с ними, искать оптимальные решения для вашего проекта, отклоняться в сторону от линии основной, стабильной кодовой базы и работать параллельно с ней. Эта методология называется *ветвлением* (branching). Это очень распространенная практика, которая обеспечивает целостность основной линии разработки и исключает любые случайные изменения, которые могли бы ее нарушить.

В Git принцип ветвления считается легковесным и незатратным, поскольку ветка — это всего лишь указатель на последний коммит в серии связанных коммитов. Git отслеживает серию коммитов в каждой создаваемой вами ветке. Вы можете переключаться между ветками локально. После этого система сохраняет состояние проекта в последний момент, в который был создан снимок указанной ветки. Когда вы решите внести изменения из какой-либо ветки в основную линию разработки, Git совместит эту серию коммитов, применив методики, о которых пойдет речь в главе 6.



Поскольку Git предлагает множество новшеств, нужно помнить, что принципы и практики других систем контроля версий могут работать здесь иначе или вообще не подходить.

Командная строка Git

Интерфейс командной строки Git очень прост и дает пользователю полный контроль. В связи с этим есть много способов реализовать одно и то же действие. Сосредоточьтесь на тех командах, которые важны в вашей повседневной работе, тем самым вы сможете упростить их изучение.

Для начала выполните **git version** или **git --version**, чтобы определить, установлена ли у вас Git. Вывод должен быть примерно таким:

```
$ git --version
git version 2.37.0
```

Если в вашей системе нет Git, загляните в приложение Б, где описано, как установить ее на вашу ОС, и уже потом переходите к следующему разделу.

Перед установкой выполните **git** без аргументов. На это Git выдаст список доступных опций и наиболее актуальных подкоманд:

```
$ git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate] [-P | --no-pager] [--no-replace-objects] [--bare]
          [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
          [--super-prefix=<path>] [--config-env=<name>=<envvar>]
          <command> [<args>]
```

Вот список распространенных команд Git для различных ситуаций:

Запуск рабочей области (также см.: `git help tutorial`)

<code>clone</code>	Копирует репозиторий в новый каталог
<code>init</code>	Создает пустой репозиторий Git либо повторно инициализирует текущий

Работа с текущими изменениями (также см.: `git help everyday`)

<code>add</code>	Добавляет в индекс содержимое файла
<code>mv</code>	Перемещает или переименовывает файл, каталог или символическую ссылку
<code>restore</code>	Восстанавливает файлы рабочего дерева
<code>rm</code>	Удаляет файлы из рабочего дерева и индекса

Анализ истории и состояния (также см.: `git help revisions`)

<code>bisect</code>	С помощью двоичного поиска ищет коммит, который внес ошибку
<code>diff</code>	Показывает изменения между коммитами, коммитом и рабочим деревом и т. д.
<code>grep</code>	Выводит строки, соответствующие паттерну
<code>log</code>	Показывает логи коммитов
<code>show</code>	Показывает различные типы объектов
<code>status</code>	Показывает статус рабочего дерева

Расширение, разметка и корректировка истории

<code>branch</code>	Вывод списка, создание или удаление ветвей
<code>commit</code>	Регистрирует изменения в репозитории

merge	Объединяет две или более истории разработки
rebase	Повторно применяет коммиты поверх другого базового коммита
reset	Сбрасывает текущую HEAD до указанного состояния
switch	Переключает ветви
tag	Создание, вывод списка, удаление или проверка объекта тега, подписанного с помощью GPG

Совместная работа (также см.: `git help workflows`)

fetch	Скачивает объекты и ссылки из другого репозитория
pull	Получение из/интеграция с другим репозиторием или локальной веткой
push	Обновляет удаленные ссылки вместе со связанными объектами

'`git help -a`' и '`git help -g`' выводят доступные подкоманды и некоторые руководства по принципам. Для ознакомления

с конкретной подкомандой или принципом введите '`git help <command>`'

или '`git help <concept>`'

Для общего обзора системы выполните '`git help git`'.



Для вывода всего списка подкоманд `git` введите **`git help --all`**.

Как видно из подсказки, к `git` применимо не так много опций. Большинство из них, представленные как `[ARGS]`, применимы к конкретным подкомандам.

Например, опция `--version` влияет на команду `git` и выводит номер версии:

```
$ git --version
git version 2.37.0
```

В противоположность этому `--amend` является примером опции, специфичной для подкоманды `git.commit`:

```
$ git commit --amend
```

Некоторые вызовы требуют использования обеих опций (здесь дополнительные пробелы в командной строке просто служат для визуального отделения подкоманды от базовой команды и необязательны):

```
$ git --git-dir=project.git    repack -d
```

При этом можно получить документацию для каждой подкоманды `git`, используя `git help subcommand`, `git --help subcommand`, `git subcommand --help` или `man git- subcommand`.



Полная документация по Git доступна онлайн (<https://oreil.ly/7njp8>).

Исторически Git представляла собой набор из множества простых, отдельных и самостоятельных команд, разработанных в соответствии с философией Unix, согласно которой нужно создавать небольшие, функционально совместимые инструменты. Каждая команда имела название с дефисом, например `git-commit` и `git-log`. Современные версии Git больше не поддерживают такие виды команд и используют единый исполняемый элемент `git` с подкомандой.

Команды `git` понимают как «короткие», так и «длинные» опции. Например, для команды `git commit` следующие примеры будут равнозначны:

```
$ git commit -m "Fix a typo."
$ git commit --message="Fix a typo."
```

В короткой форме, `-m`, используется один дефис, в то время как в длинной, `--message`, их два. (Это соответствует стилю длинных опций GNU.) Некоторые опции могут быть доступны только в одной из форм.



Вы можете создать сводку по коммиту и подробное сообщение для этой сводки, используя опцию `-m` дважды:

```
$ git commit -m "Summary" -m "Detail of Summary"
```

Наконец, вы можете отделять опции от списка аргументов с помощью *двойного тире*. Например, используйте двойное тире для отделения управляющей части командной строки от списка операндов, таких как имена файлов:

```
$ git diff -w main origin -- tools/Makefile
```

Вам может потребоваться использовать двойное тире для отделения и явного обозначения имен файлов, чтобы не спутать их с остальной частью команды. К примеру, если у вас будут и файл, и тег под именем `main.c`, то потребуется выполнять операции по-отдельности:

```
# Переключиться на тег "main.c"
$ git checkout main.c

# Переключиться на файл "main.c"
$ git checkout -- main.c
```

Использование Git: быстрый старт

Чтобы посмотреть Git в действии, создайте новый репозиторий, внесите в него содержимое и отслеживайте несколько ревизий. Создать репозиторий можно двумя путями: либо с нуля, заполнив его содержимым, либо работать с существующим, *клонировав* его с удаленного сервера Git.

Подготовка к работе с Git

Независимо от того, создаете вы новый репозиторий или же работаете с существующим, есть базовые конфигурации, которые нужно настроить после установки Git на локальную машину. Подобным образом вы настроили бы новую фотокамеру: сначала установили корректную дату и язык и только потом начали фотографировать.

Как минимум Git потребует от вас ввести имя и адрес электронной почты, прежде чем создать первый коммит в репозитории. Предоставленный вами идентификатор отображается в качестве автора коммита вместе с другими метаданными снимка. Вы можете сохранить свой идентификатор в конфигурационном файле с помощью команды `git config`:

```
$ git config user.name "Jon Loeliger"
$ git config user.email "jdl@example.com"
```

Если вы решите не вносить данные в файл конфигурации, то придется указывать их при выполнении каждой подкоманды `git commit`, дополняя ее аргументом `--author` в конце:

```
$ git commit -m "log message" --author="Jon Loeliger <jdl@example.com>"
```

Имейте в виду, что это трудоемкий вариант, который быстро вам надоест.

Вы также можете указать свой идентификатор, передав имя и e-mail в переменных `GIT_AUTHOR_NAME` и `GIT_AUTHOR_EMAIL` соответственно. Если эти переменные установлены, то они переопределяют все настройки конфигурации. Для спецификаций, устанавливаемых в командной строке, Git будет переопределять значения, передаваемые в файле конфигурации и переменной среды.

Работа с локальным репозиторием

Теперь, когда вы настроили идентификатор, можно начать работу с репозиторием. Первым делом создайте новый репозиторий на локальной машине. Мы начнем с простого и будем постепенно продвигаться к изучению методик работы с общим репозиторием на сервере Git.

Создание начального репозитория

Смоделируем типичную ситуацию, создав репозиторий для личного сайта. Предположим, что вы начинаете с нуля и собираетесь добавить содержимое для вашего проекта в локальный каталог `~/my_website`, который поместите в репозиторий Git.

Введите следующие команды для создания этого каталога и поместите в файл `index.html` какое-нибудь содержимое:

```
$ mkdir ~/my_website
$ cd ~/my_website
$ echo 'My awesome website!' > index.html
```

Чтобы преобразовать `~/my_website` в репозиторий Git, выполните `git init`. Здесь мы сопровождаем эту команду опцией `-b` и предустановленным именем ветки `main`:

```
$ git init -b main
```

Инициализирован пустой репозиторий Git в `../my_website/.git/`

Если вы хотите сначала инициализировать пустой репозиторий, а затем добавить в него файлы, то можете сделать это с помощью следующих команд:

```
$ git init -b main ~/my_website
```

Инициализирован пустой репозиторий Git в `../my_website/.git/`

```
$ cd ~/my_website
$ echo 'My awesome website!' > index.html
```



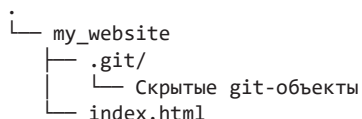
Вы можете инициализировать как абсолютно пустой каталог, так и существующий каталог с файлами. В любом случае процесс преобразования каталога в репозиторий Git будет одинаковым.

Команда `git init` создает скрытый каталог `.git` на корневом уровне проекта. Вся информация о ревизии вместе с метаданными и расширениями Git хранится на верхнем уровне, в скрытом каталоге `.git`.

Git воспринимает `~/my_website` как *рабочий каталог*. В нем содержится текущая версия файлов для вашего сайта. Когда вы вносите изменения в текущие файлы или добавляете в проект новые, Git регистрирует эти изменения в скрытом каталоге `.git`.

Чтобы проиллюстрировать принцип инициализации нового репозитория Git, мы будем ссылаться на два виртуальных каталога, которые назовем *Local History* и *Index*. Каталоги *Local History* и *Index* будем разбирать в главах 4 и 5 соответственно.

Изложенное выше показано на рис. 1.2.



Пунктирные линии вокруг *Local History* и *Index* представляют скрытые каталоги в директории `.git`.

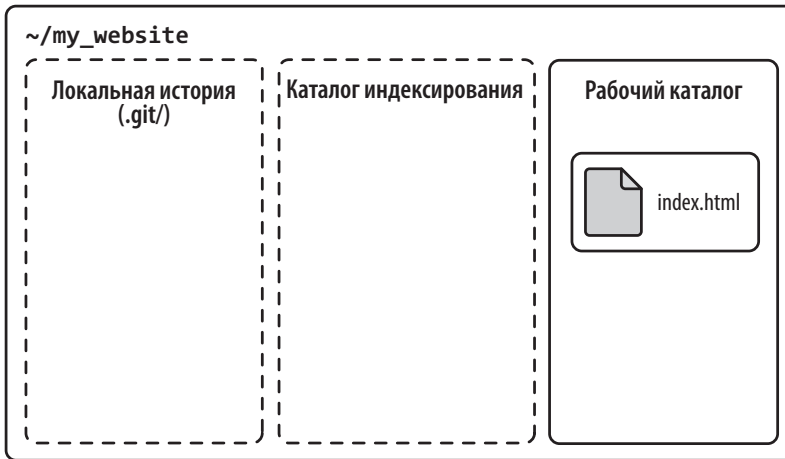


Рис. 1.2. Начальный репозиторий

Добавление файла в репозиторий

К этому моменту вы лишь *создали* новый репозиторий. Иными словами, этот репозиторий пуст. И хотя в каталоге `~/my_website` есть файл `index.html`, для Git это *рабочий каталог*, представляющий собой своего рода блокнот или каталог, где вы часто изменяете свои файлы.

Когда вы закончили с изменением файлов и хотите внести эти изменения в репозиторий, сделайте это явно с помощью команды `git add file`:

```
$ git add index.html
```



Хотя вы и можете позволить Git добавить все файлы в каталоге и его подкаталогах, используя команду `git add .`, это проиндексирует все содержимое. Советуем избирательно подходить к тому, что вы хотите проиндексировать: главным образом для того, чтобы исключить передачу в коммитах чувствительной информации или нежелательных файлов. Для этого вы можете использовать файл `.gitignore`, который будет описан в главе 5.

Аргумент `.` (точка) — это сокращение для текущего каталога.

С помощью команды `git add` вы даете Git понять, что хотите включить в репозиторий в качестве ревизии последнюю итерацию изменений файла `index.html`. До сих пор Git просто размещал файл, что было промежуточным шагом перед созданием снимка с помощью коммита.

Git разделяет этапы `add` и `commit`, чтобы избежать нестабильности и в то же время обеспечить гибкость и детальность в фиксировании изменений. Представьте, насколько раздражающим, запутанным и долгим было бы обновление репозитория каждый раз, когда вы добавляете, удаляете или изменяете файл. Вместо этого

несколько предварительных и связанных с этим процессом шагов, например добавление, можно *сгруппировать*, сохранив репозиторий в стабильном и согласованном состоянии. Этот метод также позволит создать описание причин изменений в коде. В главе 4 мы разберем этот принцип подробно.

Старайтесь группировать пакеты логических изменений, прежде чем делать коммит. Это называется *атомарным* коммитом, он поможет в ситуациях, требующих выполнения продвинутых операций, о которых речь пойдет в следующих главах.

Запуск команды `git status` показывает промежуточный статус файла `index.html`:

```
$ git status
On branch main
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   index.html
```

Эта команда сообщает, что во время следующего коммита в репозиторий будет добавлен новый файл `index.html`.

После индексирования этого файла следующим шагом будет его отправка в репозиторий. После коммита этот файл станет частью *истории репозитория*. При каждом выполнении коммита Git записывает вместе с ним другие элементы данных, в частности сообщение коммита и автора вносимого изменения.

При выполнении команды `git commit` разработчику, вносящему коммит, нужно предоставить ясное и информативное сообщение в журнале коммитов, в котором указано, что именно было изменено или добавлено. Это очень помогает, когда вам нужно перебрать историю репозитория для прослеживания конкретного изменения или быстро определить изменения коммита, не углубляясь в их детали. Эту тему мы более подробно разберем в главах 4 и 8.

Зафиксируем проиндексированный файл `index.html` для вашего сайта:

```
$ git commit -m "Initial contents of my_website"
[main (root-commit) c149e12] Initial contents of my_website
 1 file changed, 1 insertion(+)
 create mode 100644 index.html
```



Подробности об авторе, сделавшем коммит, извлекаются из конфигурации Git, которую мы настроили ранее.

В этом примере мы передали аргумент `-m`, чтобы можно было предоставить сообщение журнала напрямую в командной строке. Если вы предпочитаете

предоставлять подробное сообщение журнала через интерактивный сеанс редактора, то это тоже вполне допустимо. Для этого настройте Git для запуска редактора во время `git commit` (опустите аргумент `-m`); если он еще не настроен, то можете установить переменную среды `$GIT_EDITOR` так:

```
# В bash или zsh
$ export GIT_EDITOR=vim

# В tcsh
$ setenv GIT_EDITOR emacs
```



Git будет ориентироваться на базовый текстовый редактор, установленный в переменных среды оболочки `VISUAL` и `EDITOR`. Если ни один из них не установлен, Git вернется к редактору `vi`.

После коммита файла `index.html` в репозиторий выполните `git status` для получения обновления по текущему состоянию этого репозитория. В нашем примере выполнение `git status` должно показывать, что для коммита нет ожидающих изменений:

```
$ git status

On branch main
nothing to commit, working tree clean
```

Git также сообщает, что ваш *рабочий каталог* пуст, то есть не содержит новых или измененных файлов, отличающихся от находящихся в репозитории.

На рис. 1.3 изображены все описанные выше шаги.

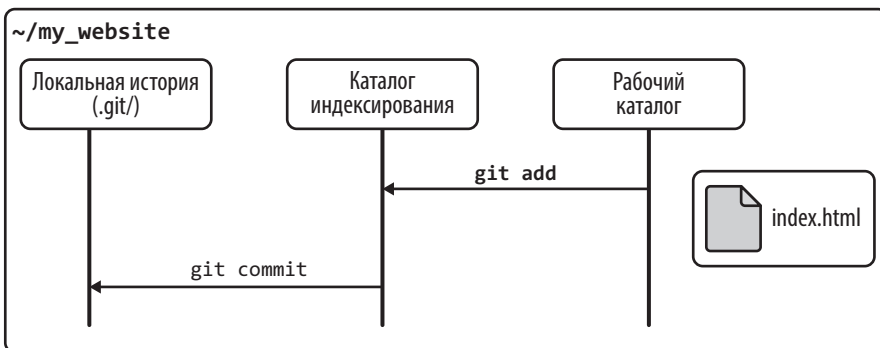


Рис. 1.3. Индексирование и добавление файла в репозиторий

Разницу между `git add` и `git commit` можно сравнить с тем, как вы организуете группу школьников в нужном порядке для создания идеальной общей фотографии: `git add` выполняет организацию, а `git commit` делает снимок.

Делаем еще один коммит

Теперь внесем в `index.html` некоторые изменения и создадим историю внутри репозитория.

Преобразуйте `index.html` в подходящий HTML-файл и зафиксируйте следующие изменения:

```
$ cd ~/my_website
# Редактирует файл index.html.
$ cat index.html
<html>
<body>
My website is awesome!
</body>
</html>
$ git commit index.html -m 'Convert to HTML'
[main 521edbe] Convert to HTML
1 file changed, 5 insertions(+), 1 deletion(-)
```

Если вы уже знакомы с Git, то у вас может возникнуть вопрос: почему мы пропустили этап `git add index.html`, прежде чем делать коммит файла? Дело в том, что передаваемое содержимое в Git может быть определено разными способами.

Введите `git commit --help`, чтобы узнать об этих способах подробнее:

```
git commit --help
NAME
    git-commit - Record changes to the repository
SYNOPSIS
    git commit [-a | --interactive | --patch] [-s] [-v] [-u<mode>] [--amend]
               [--dry-run] [(-c | -C | --squash) <commit> | --fixup
    [(amend|reword):]<commit>)]
               [-F <file> | -m <msg>] [--reset-author] [--allow-empty]
               [--allow-empty-message] [--no-verify] [-e] [--author=<author>]
               [--date=<date>] [--cleanup=<mode>] [--no-]status]
               [-i | -o] [--pathspec-from-file=<file>] [--pathspec-file-nul]]
               [--trailer <token>[(=|:)<value>]]... [-S[<keyid>]]
               [--] [<pathspec>...]
```



Подробное описание различных способов коммита также приводится в мануале `git commit --help`.

В этом примере мы решили зафиксировать файл `index.html` с дополнительным аргументом, переключателем (switch) `-m`, который передал сообщение, поясняющее вносимые в коммите изменения: `'Convert to HTML'` (преобразован в HTML).

На рис. 1.4 показан описанный метод.

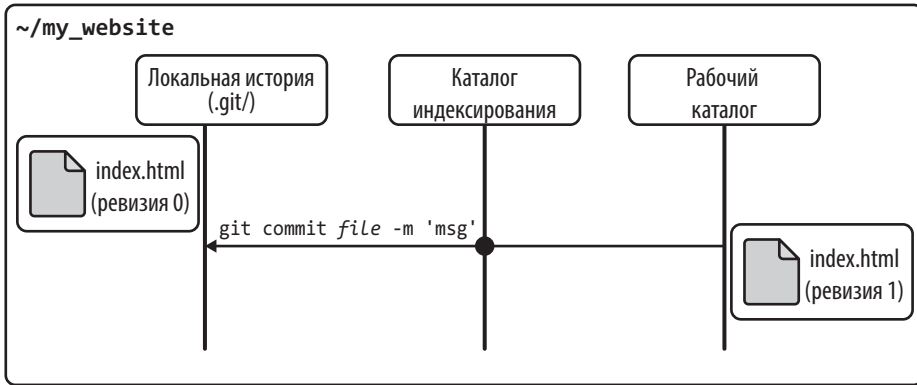


Рис. 1.4. Индексирование и добавление изменений в отслеживаемый файл в репозитории

Заметьте, что наше использование `git commit index.html -m 'Convert to HTML'` не пропускает индексирование файла. Git обрабатывает его автоматически в рамках выполнения коммита.

Просмотр коммитов

Теперь, когда в истории репозитория есть уже не один коммит, вы можете просмотреть их разными способами. Некоторые команды `git` выведут последовательность отдельных коммитов, другие покажут сводку отдельного коммита, а третьи — все подробности любого коммита репозитория, который вы укажете.

Команда `git log` выводит последовательную историю отдельных коммитов в репозитории:

```
$ git log

commit 521edbe1dd2ec9c6f959c504d12615a751b5218f (HEAD -> main)
Author: Jon Loeliger <jdl@example.com>
Date:   Mon Jul 4 12:01:54 2022 +0200

    Convert to HTML

commit c149e12e89a9c035b9240e057b592ebfc9c88ea4
Author: Jon Loeliger <jdl@example.com>
Date:   Mon Jul 4 11:58:36 2022 +0200

    Initial contents of my_website
```

В этом выводе мы получили подробную информацию журнала для каждого коммита репозитория. Сейчас в истории репозитория всего два коммита, поэтому читать такой вывод легко. В случае репозитория с большим числом коммитов такой вывод не даст с тем же удобством просмотреть длинный список подробностей.

В таких ситуациях можно передать переключатель `--oneline`, чтобы вывести обобщенный ID коммита вместе с его сообщением:

```
$ git log --oneline
521edbe (HEAD -> main) Convert to HTML
c149e12 Initial contents of my_website
```

Записи коммитов из журнала выводятся в порядке убывания от последней к первой¹ (к исходному файлу); каждая запись показывает имя автора коммита и его e-mail, дату коммита, сообщение об изменении и внутренний ID коммита. ID коммита будет описан в разделе «Контентно-адресуемая база данных» на с. 50. Сами же коммиты мы разберем подробнее в главе 4.

Если вы хотите увидеть дополнительные сведения о конкретном коммите, используйте команду `git show` с его ID:

```
$ git show c149e12e89a9c035b9240e057b592ebfc9c88ea4
commit c149e12e89a9c035b9240e057b592ebfc9c88ea4
Author: Jon Loeliger <jdl@example.com>
Date:   Mon Jul 4 11:58:36 2022 +0200

Initial contents of my_website

diff --git a/index.html b/index.html
new file mode 100644
index 0000000..6331c71
--- /dev/null
+++ b/index.html
@@ -0,0 +1 @@
+My awesome website!
```



Если вы выполните `git show` без указания номера коммита, команда просто выдаст подробности коммита HEAD, который в нашем случае идет последним.

Команда `git log` отображает журнал коммитов, показывая, как изменения из каждого коммита включены в историю репозитория. Если вы хотите увидеть сжатые однострочные сводки по текущей ветке разработки, не расширяя команду `git log --oneline` дополнительными фильтрами, то альтернативным подходом станет команда `git show-branch`:

```
$ git show-branch --more=10
[main] Convert to HTML
[main^] Initial contents of my_website
```

¹ Строго говоря, эти записи находятся скорее не в хронологическом, а в топологическом порядке (каждый коммит строится на основе предыдущего, и топологическая сортировка учитывает эти отношения).

Выражение `--more=10` раскрывает до 10 дополнительных версий, но у нас их пока всего две, поэтому показаны обе. (По умолчанию в этом случае будет выводиться только последний коммит.) `main` — это предустановленное имя ветки.

О ветках речь пойдет в главе 3, где мы также более подробно поговорим о команде `git show-branch`.

Просмотр отличий коммитов

Добавив коммиты, мы создали историю репозитория и теперь можем посмотреть отличия между двумя ревизиями `index.html`. Для этого потребуется вспомнить ID коммитов и выполнить команду `git diff`:

```
$ git diff c149e12e89a9c035b9240e057b592ebfc9c88ea4 \
          521edbe1dd2ec9c6f959c504d12615a751b5218f

diff --git a/index.html b/index.html
index 6331c71..8cfcb90 100644
--- a/index.html
+++ b/index.html
@@ -1,5 @@
-My awesome website!
+<html>
+<body>
+  My website is awesome!
+</body>
+</html>
```

Полученный вывод соответствует тому, что должна выдавать команда `git diff`. Согласно принятой структуре, коммит первой ревизии, `9da581d910c9c4ac93557ca4859e767f5caf5169`, представляет начальное содержимое `index.html`, а коммит второй ревизии, `ec232cddfb94e0dfd5b5855af8ded7f5eb5c90d6`, конечное. Следовательно, знак «плюс» (+) предворяет каждую строку нового содержимого после знака «минус», который указывает на удаленное содержимое.



Не пугайтесь длинных шестнадцатеричных чисел. Git предоставляет множество более коротких и простых способов выполнения подобных команд, позволяя избегать больших и сложных ID коммитов. Как правило, достаточно первых семи символов шестнадцатеричных чисел, как было показано ранее в примере `git log --oneline`. Подробно об этом поговорим в разделе «Конфигурно-адресуемая база данных» на с. 50 в главе 2.

Удаление и переименование файлов в репозитории

Вы узнали, как добавлять файлы в Git-репозиторий, теперь рассмотрим, как удалить файл из него. Операция удаления аналогична добавлению, только использует команду `git rm`. Предположим, в содержимом сайта есть файл `adverts.html`, который вы хотите удалить. Сделать это можно так:

```
$ cd ~/my_website
$ ls
index.html  adverts.html

$ git rm adverts.html
rm 'adverts.html'

$ git commit -m "Remove adverts html"
[main 97ff70a] Remove adverts html
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 adverts.html
```

Как и добавление, удаление происходит в два этапа: вы выражаете свое намерение удалить файл с помощью `git rm`, которая также индексирует этот файл. Затем реализуете это изменение в репозитории командой `git commit`.



Как и в случае с `git add`, когда мы используем `git rm`, то не удаляем файл напрямую, а изменяем то, что отслеживается, — удаление или добавление.

Вы можете переименовать файл косвенно, используя комбинацию команд `git rm` и `git add`, либо сделать это быстрее напрямую с помощью `git mv`. Вот пример первого варианта:

```
$ mv foo.html bar.html
$ git rm foo.html
rm 'foo.html'

$ git add bar.html
```

В этой последовательности вначале выполняется `mv foo.html bar.html`, чтобы `git rm` не удалила `foo.html` из системы безвозвратно. Вот та же операция, выполняемая с помощью `git mv`:

```
$ git mv foo.html bar.html
```

В любом случае индексированные изменения впоследствии нужно зафиксировать:

```
$ git commit -m "Moved foo to bar"
[main d1e37c8] Moved foo to bar
 1 file changed, 0 insertions(+), 0 deletions(-)
 rename foo.html => bar.html (100%)
```

Git обрабатывает операции перемещения не так, как большинство аналогичных систем, — здесь используется механизм, основанный на сходстве содержимого двух версий файла. Подробнее об этом в главе 5.

Работа с общим репозиторием

Итак, вы уже инициализировали новый репозиторий и внесли в него изменения. Все эти изменения доступны только для вашей локальной машины. Это хороший пример управления проектом, который доступен только вам. Но как работать над репозиторием совместно, когда он размещен на сервере Git? Давайте разбираться.

Создание локальной копии репозитория

Вы можете создать полную копию, или *клон*, репозитория с помощью команды `git clone`. Таким образом реализуются совместная работа, внесение изменений в файлы и синхронизация изменений из других версий в одном репозитории.

Начнем с простого и создадим копию существующего репозитория. Затем рассмотрим тот же пример, но для случая, как если бы репозиторий находился на удаленном сервере Git:

```
$ cd ~
$ git clone my_website new_website
```

Несмотря на то что эти два репозитория теперь содержат одинаковые объекты, файлы и каталоги, между ними есть некоторые различия, которые можно посмотреть с помощью следующих команд:

```
$ ls -lsa my_website new_website
...
$ diff -r my_website new_website
...
```

В локальной файловой системе применение `git clone` для создания копии репозитория очень похоже на использование `cp -a` или `rsync`. В противоположность этому для *клонирования* репозитория с сервера Git требуется такой синтаксис:

```
$ cd ~
$ git clone https://git-hosted-server.com/some-dir/my_website.git new_website
Cloning into 'new_website'...
remote: Enumerating objects: 2, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (103/103), done.
remote: Total 125 (delta 45), reused 65 (delta 18), pack-reused 0
Receiving objects: 100% (125/125), 1.67 MiB | 4.03 MiB/s, done.
Resolving deltas: 100% (45/45), done.
```

Клонировав репозиторий, вы можете изменять сделанную копию, делать новые коммиты, изучать логи и т. д. Это полноценный репозиторий со всей историей. Помните, что вносимые в клонированную версию изменения не будут автоматически передаваться в исходный репозиторий.

Указанный принцип отражен на рис. 1.5.

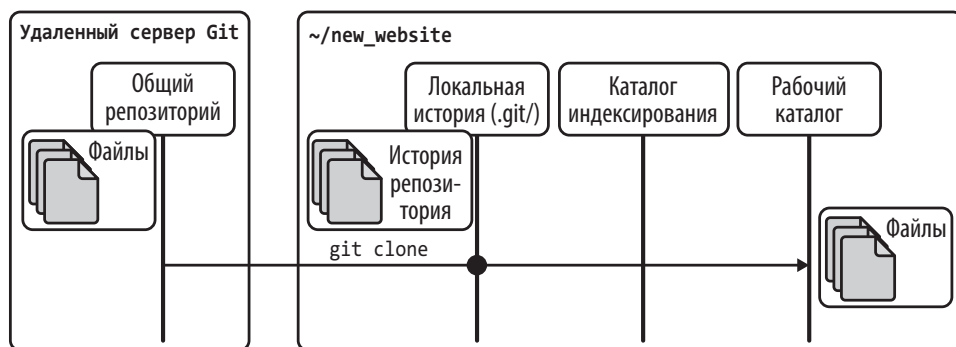


Рис. 1.5. Клонирование общего репозитория

Пусть вас не смущают некоторые термины, которые вы видите в выводе. В Git для именования клонируемого репозитория может использоваться богатый набор источников, включая сетевые имена. Об этих формах и их использовании мы поговорим в главе 11.

Файлы конфигурации

Конфигурационные файлы Git — это простые текстовые файлы в стиле `.ini`. Они нужны для хранения настроек и установок, используемых множеством команд `git`. Некоторые из этих настроек представляют личные установки (например, нужно ли использовать `color.pager?`), другие важны для правильного функционирования репозитория (например, `core repositoryformatversion`), а третьи слегка корректируют поведение команд `git` (например, `gc.auto`). Как и прочие инструменты, Git поддерживает иерархию файлов конфигурации.

Иерархия файлов конфигурации

На рис. 1.6 показана иерархия файлов конфигурации Git в порядке убывания приоритета.

`.git/config`

Относящиеся к репозиторию настройки, управляемые через опцию `--file` или установленные по умолчанию. Запись в этот файл также можно выполнять с помощью опции `--local`. Содержащиеся в нем настройки имеют наивысший приоритет.

`~/.gitconfig`

Пользовательские настройки, управляемые через опцию `--global`.

/etc/gitconfig

Системные настройки, управляемые через опцию `--system`, доступны, если у вас есть соответствующие разрешения Unix для записи в файл `gitconfig`. Эти установки имеют наименьший приоритет. В зависимости от вашей установки их файл может находиться в другом месте (возможно, в `/usr/local/etc/gitconfig/`) или вовсе отсутствовать.

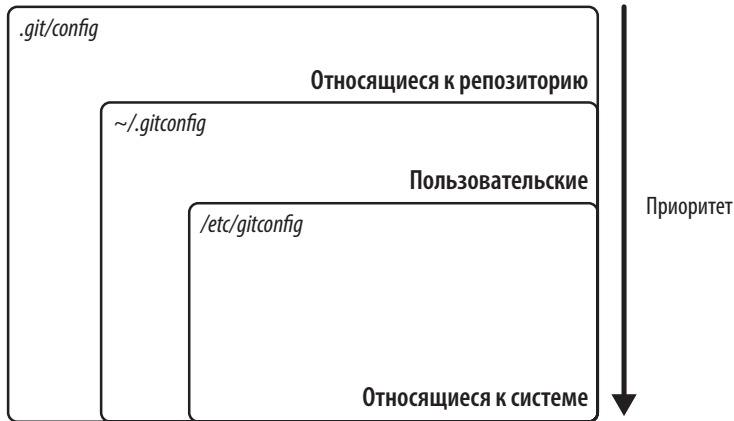


Рис. 1.6. Иерархия файлов конфигурации Git

Например, для сохранения имени или почты автора, которые будут использоваться во всех ваших коммитах для всех ваших репозиториях, настройте значения `user.name` и `user.email` в файле `$HOME/.gitconfig` с помощью `git config --global`:

```
$ git config --global user.name «Jon Loeliger»
$ git config --global user.email «jdl@example.com»
```

Если нужно установить относящиеся к репозиторию имя и почту, которые переопределят установку `--global`, опустите флаг `--global` либо используйте флаг `--local`, чтобы сделать это явно:

```
$ git config user.name "Jon Loeliger"
$ git config user.email "jdl@special-project.example.org"
```

Вы можете использовать `git config -l` (или ее длинную форму `--list`) для вывода списка настроек всех переменных, находящихся во всех файлах конфигурации:

```
# Создание нового, пустого репозитория
$ mkdir /tmp/new
$ cd /tmp/new
$ git init

# Установка некоторых значений в конфигурации
$ git config --global user.name «Jon Loeliger»
$ git config --global user.email «jdl@example.com»
```

```
$ git config user.email "jdl@special-project.example.org"
```

```
$ git config -l
user.name=Jon Loeliger
user.email=jdl@example.com
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
user.email=jdl@special-project.example.org
```



При указании команды `git config -l` добавление опций `--show-scope` и `--show-origin` позволит вывести различные источники для конфигурации. Попробуйте выполнить в своем терминале `git config -l --show-scope --show-origin`.

Поскольку файлы конфигурации — это, по сути, просто текстовые файлы, их содержимое можно просматривать с помощью `cat` и редактировать через любой текстовый редактор:

```
# Просмотр только настроек репозитория

$ cat .git/config
[core]
  repositoryformatversion = 0
  filemode = true
  bare = false
  logallrefupdates = true
  ignorecase = true
  precomposeunicode = true

[user]
  email = jdl@special-project.example.org
```



Содержимое файла конфигурации может быть представлено с некоторыми отличиями в зависимости от типа вашей операционной системы. Многие из этих отличий связаны с разными характеристиками файловых систем.

Если требуется удалить настройку из файла конфигурации, используйте опцию `--unset` вместе с правильным флагом:

```
$ git config --unset --global user.email
```

В Git есть множество опций конфигурации и переменных среды, которые зачастую служат одной и той же цели. Например, можно указать значение для редактора, который будет использоваться при составлении сообщения коммита. В зависимости от конфигурации вызов будет происходить по следующим этапам:

- 1) переменная среды `GIT_EDITOR`;
- 2) опция конфигурации `core.editor`;
- 3) переменная среды `VISUAL`;
- 4) переменная среды `EDITOR`;
- 5) команда `vi`.

Есть несколько сотен параметров конфигурации. Не будем утомлять вас длинным перечислением и укажем лишь самые важные. Более обширный (хотя и не полный) список ищите на странице руководства `git config`:



Полный список всех команд `git` доступен онлайн (<https://oreil.ly/Gj2zg>).

Настройка псевдонима

Псевдонимы Git позволяют замещать типичные и часто используемые, но сложные команды `git`, простыми и легко запоминающимися псевдонимами. Это избавляет вас от необходимости запоминать или заново вводить длинные команды и исключает возможные опечатки:

```
$ git config --global alias.show-graph \
    'log --graph --abbrev-commit --pretty=oneline'
```

В этом примере мы создали псевдоним `show-graph` и сделали его доступным для использования в любом создаваемом нами репозитории. Команда `git show-graph` будет давать тот же вывод, что и длинная команда `git log` со всеми указанными опциями.

Итоги

Наверняка после прочтения этой главы у вас осталось множество вопросов о том, как работает Git. Например, как Git сохраняет каждую версию файла? Из чего на самом деле состоит коммит? Откуда берутся все эти странные имена коммитов? Почему мы используем название ветки `main`? И вообще, что *означает* «ветка»?

Это хорошие вопросы. Рассмотренный здесь материал познакомил вас с операциями, которые вы будете чаще всего использовать в своих проектах, а подробные объяснения будут даны в части II.

В следующей главе будут определены некоторые термины, представлена часть принципов Git и заложена основа для изучения уроков, которые мы разберем в остальной части книги.

ГЛАВА 2

Фундаментальные понятия

В предыдущей главе вы познакомились с основами Git, ее характеристиками и типичными вариантами применения контроля версий. Все это наверняка вызвало у вас немало вопросов. Например, как Git отслеживает ревизии файла при каждом коммите на вашей локальной машине? Что содержится в скрытом каталоге `.git` и зачем это нужно? Как генерируется ID коммита, почему он выглядит именно так и нужно ли обращать на него внимание?

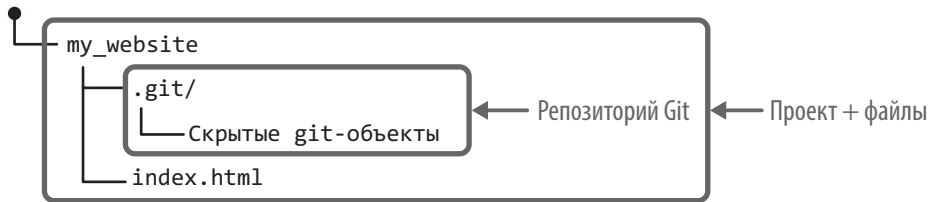
Если вы уже использовали систему контроля версий, например SVN или CVS, то некоторые из описанных в предыдущей главе команд могли показаться вам знакомыми. Несмотря на то что Git выполняет те же функции и предоставляет все операции, которые можно ожидать от современной системы контроля версий, ее внутренние принципы фундаментальным и удивительным образом отличаются.

В этой главе мы изучим отличия Git, рассмотрим ключевые компоненты ее архитектуры и некоторые важные концепции. Сосредоточимся на основах, общей терминологии и связи между объектами Git, а также их использованием на примере одного репозитория. Сведения из этой главы также применимы к случаям, когда вы работаете с несколькими взаимосвязанными репозиториями.

Репозитории

Репозиторий Git — это, по сути, база данных «ключ — значение», в которой находится вся нужная информация для хранения и управления ревизиями и историей файлов проекта. Репозиторий Git содержит полную копию всего проекта за всю историю проекта. Но, в отличие от большинства других систем контроля версий, он предоставляет не только полную рабочую копию всех хранящихся в проекте файлов, но и копию самого репозитория (базу данных пар «ключ — значение») (рис. 2.1).

```
.
├── my_website
│   ├── .git
│   │   └── Скрытые git-объекты
└── index.html
```

**Рис. 2.1.** Репозиторий Git

Мы используем термин *репозиторий* для описания всего проекта и его базы данных, содержащей пары «ключ — значение», как одного объекта.

Помимо данных файлов и метаданных репозитория, в каждом репозитории Git содержится набор значений конфигурации. В прошлой главе мы поработали с некоторыми из них, в частности с именем и почтой пользователя репозитория. При клонировании или дублировании эти настройки не передаются из одного репозитория в другой. Вместо этого Git управляет собственной конфигурацией и настройками для каждого окружения, пользователя и репозитория.

Внутри репозитория Git поддерживает две основные структуры данных: хранилище объектов (object store) и индекс. Все данные репозитория хранятся в корне рабочего каталога в скрытом подкаталоге `.git`.

Хранилище объектов предназначено для того, чтобы его можно было эффективно копировать во время операции клонирования. Это часть механизма, поддерживающего полностью распределенную систему контроля версий. Индекс — это временная информация, которая относится конкретно к репозиторию и при необходимости может создаваться или изменяться.

В следующих разделах подробно рассмотрим хранилище и индекс.

Хранилище объектов Git

В основе реализации репозитория лежит хранилище объектов. Оно содержит исходные файлы данных и все сообщения журнала, сведения об авторах, датах и прочую информацию, необходимую для воссоздания или восстановления любой версии или ветки проекта на конкретный момент времени.

Git помещает в хранилище всего четыре типа объектов: большие двоичные объекты (BLOB, Binary Large Object), деревья, коммиты и теги. Эти четыре атомарных вида объектов формируют основу более высокоуровневых структур данных в Git:

BLOB

Каждая версия файла представлена в виде BLOB. Этот термин обычно используется для обозначения переменной или файла, который может содержать

любые данные и внутренняя структура которого игнорируется программой. BLOB считается непрозрачным. Он содержит данные файла, но в нем нет никаких метаданных об этом файле и даже его имени.

Деревья

Объект дерева представляет один уровень информации каталога. Он регистрирует идентификаторы BLOB-объектов, пути и некоторые метаданные для всех файлов в одном каталоге. Он также может рекурсивно ссылаться на другие поддеревья, тем самым создавая полную иерархию файлов и подкаталогов. Говоря простым языком, дерево регистрирует содержимое одного уровня в иерархии каталогов. В нем перечислены файлы и поддеревья, то есть их имена и идентификаторы для объекта Git, который они представляют (либо OID BLOB-объекта, либо другого дерева для подкаталогов).

Коммиты

Объект коммита содержит метаданные для каждого внесенного в репозиторий изменения, включая его автора, коммиттера, дату коммита и сообщение журнала. Каждый коммит указывает на свой объект дерева в одном полном снимке, отражая состояние репозитория на фиксации коммита. Изначальный, или корневой, коммит родителя не имеет. У большинства коммитов есть один родительский, хотя позже мы объясним, как коммит может ссылаться более чем на одного родителя.

Теги

Объект тега присваивает конкретному объекту, обычно коммиту, человеко-читаемое имя. `9da581d910c9c4ac93557ca4859e767f5caf5169` относится к конкретному, правильно определенному коммиту, но привычное имя тега вроде `Ver-1.0-Alpha` может оказаться более осмысленным.



Эти четыре объекта в хранилище являются неизменяемыми. Заметьте, что только *аннотированный тег* может быть изменен. *Легковесный тег* — это неизменяемый объект, поскольку его можно использовать в качестве ссылки, обновляемой напрямую.

Со временем вся информация в объектном хранилище изменяется и множится, отражая отслеживание и моделирование ваших правок проекта, дополнения и удаления. Для эффективного использования пространства диска и пропускной способности сети Git сжимает и хранит объекты в файлах формата *packfile*, которые помещаются в хранилище объектов.

Индекс

Индекс, или каталог индексирования, сохраняет двоичные данные и является приватным для вашего репозитория. Содержимое индекса временное и описывает

структуру всего репозитория в заданный момент времени. Он предоставляет кэшированное представление всех BLOB-объектов, которое отражает текущее состояние вашего проекта.

Информация в индексе временная, то есть относится к промежуточной стадии между двумя состояниями: рабочим каталогом проекта (файловой системой) и объектным хранилищем репозитория (историей репозитория). Индекс также называют *промежуточным каталогом* (staging directory).

На рис. 2.2 визуально показан этот принцип:

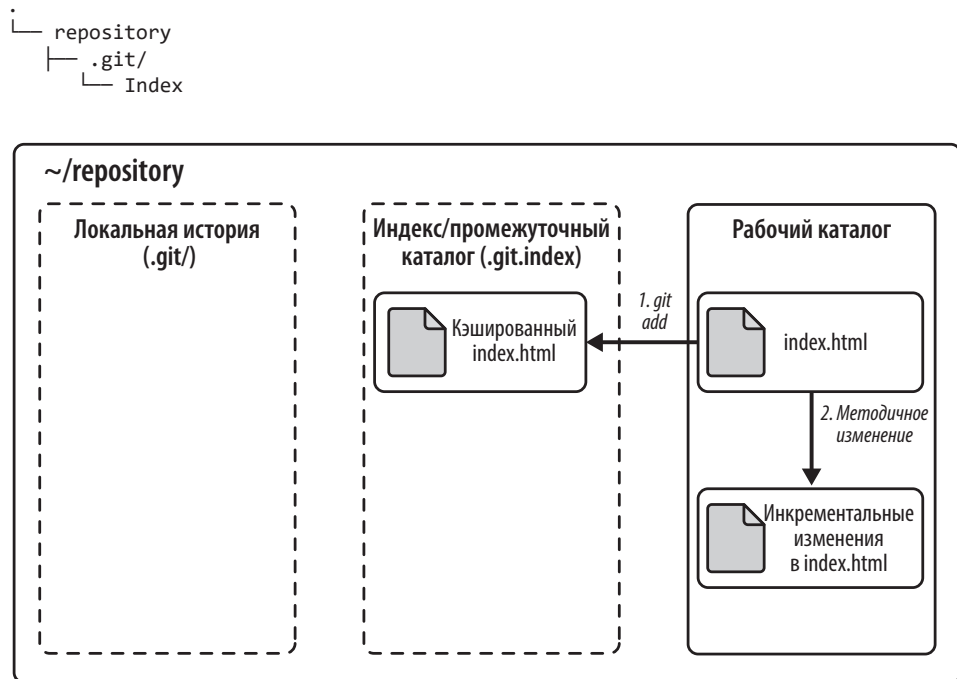


Рис. 2.2. Индекс/промежуточный каталог

Индекс — это одна из ключевых отличительных особенностей Git. Это связано с тем, что вы можете последовательно изменять содержимое индекса и таким образом более четко контролировать содержимое, которое будет сохранено в следующем коммите. Индекс позволяет разделить инкрементальные шаги разработки и коммиты вносимых ими изменений.

Посмотрим на процесс подробнее. Как правило, разработчик добавляет, удаляет или редактирует файл либо набор файлов. Эти изменения влияют на текущее состояние репозитория. Далее требуется выполнить команду `git add` для внесения этих изменений в индекс. Затем индекс отслеживает записи об этих изменениях

и хранит их, пока вы не будете готовы сделать коммит. Git также позволяет удалять внесенные в индекс изменения. Таким образом, с помощью индекса можно поэтапно переносить ваш репозиторий со старой версии на новую.

В главе 6 вы увидите, что индекс играет важную роль в операции слияния, позволяя одновременно просматривать несколько версий одного файла и управлять ими.

Контентно-адресуемая база данных

Git также описывается как контентно-адресуемая система хранения. Это происходит потому, что хранилище объектов организовано и реализовано для хранения пар «ключ — значение» каждого объекта, который создается под капотом при контроле версий проекта. Каждый объект в этом хранилище связан с уникальным именем, которое создается путем применения к содержимому этого объекта алгоритма SHA-1, генерирующего хеш-значение.

Для генерации хеш-значения Git использует все содержимое объекта. Само хеш-значение считается уникальным для конкретного содержимого в конкретный момент времени, в связи с чем используется в качестве индекса или имени для этого объекта в объектном хранилище. Малейшее изменение файла ведет к изменению хеша SHA-1, в результате чего новая версия файла индексируется отдельно.

Значения SHA-1 являются 160-битными и обычно представлены в виде шестнадцатеричного числа, состоящего из 40 цифр, например `9da581d910c9c4ac93557ca4859e767f5caf5169`. Иногда во время отображения значения SHA-1 сокращаются до более короткого и удобного для обращения префикса. Пользователи Git используют термины SHA-1, хеш и ID объекта как взаимозаменяемые.

ГЛОБАЛЬНО УНИКАЛЬНЫЕ ИДЕНТИФИКАТОРЫ

Важной характеристикой вычисления хеша SHA-1 является то, что алгоритм для идентичного содержимого всегда вычисляет одинаковый ID, независимо от того, где оно находится. Один и тот же файл в разных каталогах и даже на разных машинах будет давать одинаковый хеш. Таким образом, хеш файла — это, по сути, глобально-уникальный идентификатор (Globally Unique Identifier, GUID).

Как следствие, файлы или BLOB-объекты произвольного размера вы можете проверить на равенство по всему интернету, просто сопоставляя их идентификаторы SHA-1.

В следующем разделе изучим это свойство более подробно.

Отслеживание содержимого в Git

Git — это гораздо больше, чем просто система контроля версий. Для лучшего понимания внутренней механики Git ее нужно рассматривать как *систему отслеживания содержимого*.

Такое отличие, хотя и очень тонкое, определяет многие принципы дизайна Git. Это ключевая причина, по которой Git может относительно легко выполнять внутренние манипуляции с данными без ущерба быстродействию (при правильном выполнении). Для новичков в Git этот принцип становится одним из самых трудных в освоении, поэтому заслуживает объяснения.

Отслеживание содержимого в Git происходит двумя способами, которые фундаментально отличаются практически от всех других систем контроля версий¹:

1. Объектное хранилище Git основано на вычислении хеша содержимого объектов, а не на именах файлов или каталогов из их исходного расположения на машине пользователя.
2. Внутренняя база данных Git хранит каждую версию каждого файла, а не их отличия по мере перехода от одной ревизии к другой.

Разберем этот момент подробнее. Когда Git помещает файл в хранилище, то делает это на основе хеша его данных (содержимого файла), а не имени файла (метаданных файла). В действительности Git не отслеживает имена файлов или каталогов, которые связаны с файлами косвенно. Данные хранятся в виде BLOB-объектов в объектном хранилище. Git отслеживает содержимое, а не файлы.

Если два отдельных файла имеют одинаковое содержимое, независимо от того, находятся они в одном или разных каталогах, Git сохраняет в хранилище только одну копию этого содержимого в виде BLOB. Git вычисляет хеш-код каждого файла согласно его содержимому, затем определяет, что файлы имеют одинаковые значения SHA-1, а значит, и одинаковое содержимое, и помещает BLOB в хранилище, индексированное этим значением SHA-1. Оба файла в проекте, независимо от того, где они расположены в структуре каталогов пользователя, используют для содержимого один BLOB.

Поскольку Git берет в качестве имени файла хеш всего содержимого этого файла, то она работает с каждой полной копией файла. Система не может основываться на его записях в хранилище объектов, на части содержимого или на отличиях между двумя ревизиями файла. Возьмем приведенный выше пример, когда два файла имеют одинаковое содержимое. При изменении одного из них Git вычисляет для него новый SHA-1, определяет, что теперь он является другим BLOB, и добавляет этот новый BLOB в хранилище. Исходный BLOB остается в хранилище для использования неизмененным файлом.

¹ Яркими исключениями будут Monotone, Mercurial, OpenCMS и Venti.

По этой причине типичное представление файла, у которого есть ревизии и который переходит от одной ревизии к другой, — это, по сути, артефакт. Git вычисляет эту историю в виде набора изменений между разными BLOB с варьирующимися хешами, а не хранит имя файла и набор отличий непосредственно. Это может казаться странным, но такая специфика позволяет Git с легкостью выполнять определенные задачи.

Этот принцип представлен на рис. 2.3.

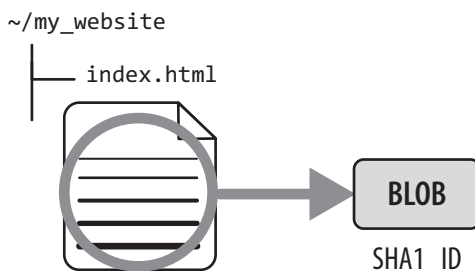


Рис. 2.3. BLOB-объект

Имя пути или содержимое

Как и во многих других системах контроля версий, Git нужно поддерживать явный список файлов, формирующих содержимое репозитория. Но это не обязательно должно происходить на основе имен файлов. Git рассматривает имя файла как элемент данных, отличный от содержимого этого файла. Таким образом, в стиле традиционных баз данных Git отделяет индекс от данных. В табл. 2.1 Git сравнивается с другими известными системами.

Таблица 2.1. Сравнение баз данных

Система	Механизм индексации	Хранилище данных
Реляционная база данных	Индексно-последовательный метод доступа (ISAM)	Записи данных
Файловая система Unix	Каталоги (/path/to/file)	Блоки данных
Git	.git/objects/`hash`, содержимое объекта дерева	BLOB-объекты и объекты деревьев

Имена файлов и каталогов берутся из внутренней файловой системы, но Git они неинтересны. Система просто регистрирует каждый путь к файлу и убеждается, что может точно воссоздать файлы и каталоги из его содержимого, которое

индексируется по хеш-значению. Этот набор информации находится в хранилище в виде объекта *дерева*.

В системе Git не моделируется физическая раскладка данных в соответствии со структурой каталогов на машине пользователя. В ней используется совершенно другая структура, которая может воссоздать исходную структуру размещения файлов и каталогов в проекте. Внутренняя структура Git более эффективна с точки зрения собственных операций и задач хранилища.

Когда Git нужно создать рабочий каталог, она обращается к файловой системе: «У меня есть большой BLOB данных, который нужно поместить по пути `path/to/directory/file`. Сможешь сделать?» На это файловая система отвечает: «Да, я распознаю эту строку как набор имен подкаталогов и понимаю, куда поместить BLOB с данными. Спасибо!»

Этот принцип показан на рис. 2.4.

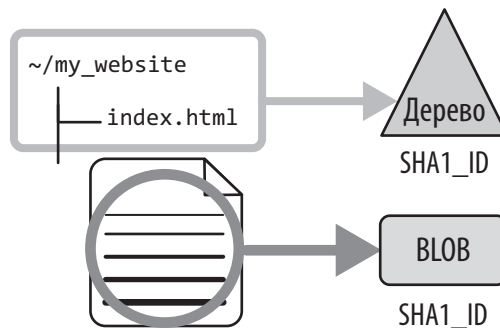


Рис. 2.4. Объект дерева

Файлы packfile

Теперь рассмотрим, как Git хранит BLOB-объекты и объекты деревьев в хранилище. Внимательный читатель может подумать, что Git реализует неэффективный метод хранения всего содержимого каждой версии каждого файла прямо в своем хранилище объектов. Даже если Git сжимает файлы, то хранить полное содержимое разных версий одного и того же файла все равно неэффективно. А если мы добавим в него всего одну строку? Git все равно сохранит полное содержимое обеих версий.

К счастью, Git не хранит объекты в своей базе данных таким образом. Система применяет более эффективный механизм под названием *pack-файлы* (*packfiles*). Git использует zlib (<https://zlib.net>), открытую библиотеку, реализующую алгоритм DEFLATE (<https://oreil.ly/GD3OL>) для сжатия каждого объекта перед его сохранением в хранилище. Подробно мы разберем pack-файлы в главе 11.



Для повышения эффективности алгоритм Git специально создает для крупных объектов дельты, чтобы учитывать пространство, необходимое для сохранения сжатого файла. Эта оптимизация размера также есть во многих других дельта-алгоритмах, поскольку удаление данных считается менее затратным, чем их добавление в дельта-объект.

Имейте в виду, что `pack`-файлы содержатся в хранилище вместе с другими объектами. Они также используются для эффективной передачи данных репозитория по сети.

Визуализация хранилища Git

Теперь, когда мы узнали об эффективном механизме сохранения объектов в Git, выясним, как эти объекты сочетаются и взаимодействуют, формируя цельную файловую систему.

- BLOB-объект находится «внизу» структуры данных. Он не ссылается ни на какие другие объекты Git, а на него ссылаются только объекты дерева. В отношении к объекту дерева его можно считать листовым узлом. На рисунках ниже каждый BLOB представлен в виде прямоугольника.
- Объекты дерева указывают на BLOB-объекты и, возможно, на другие деревья. На любой отдельный объект дерева могут указывать многие разные объекты коммитов. Деревья представлены в виде треугольников. В главе 15 мы узнаем, что объект дерева также может указывать на объект коммита, но сейчас будем придерживаться простой схемы.
- Коммиты обозначены кругами. Коммит указывает на одно конкретное дерево, которое создается в репозитории за счет коммита.
- Каждый тег представлен в виде ромба и может указывать максимум на один коммит.

Ветка — это не фундаментальный объект Git, хотя он и играет важнейшую роль в именовании коммитов. Каждая ветка на рисунке представлена в виде прямоугольника.

На рис. 2.5 показано, как сочетаются все элементы. На схеме представлено состояние репозитория после единственного коммита, когда были добавлены два файла. Оба файла находятся в каталоге верхнего уровня. И ветка `main`, и тег `V1.0` указывают на коммит с ID 1492.

```

├── ~/project
│   ├── .git
│   │   └── .git/objects/*
│   ├── file dead23
│   └── file feeb1e

```

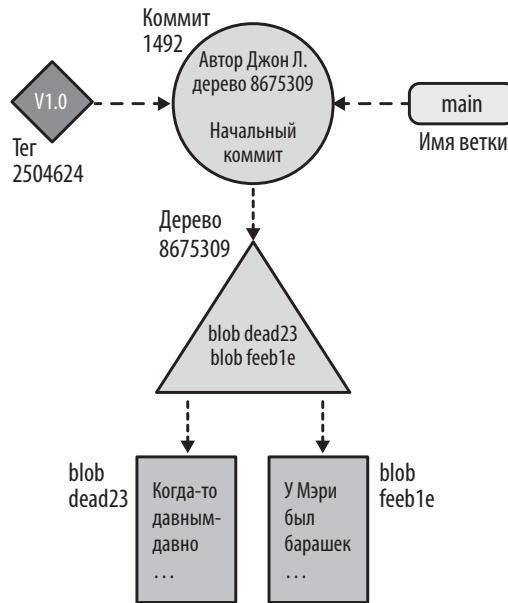


Рис. 2.5. Объекты Git

Теперь немного усложним пример. Оставим два исходных файла без изменений и добавим новый подкаталог с одним файлом. Итоговое хранилище будет выглядеть так, как показано на рис. 2.6:

```

├── ~/project
│   ├── .git
│   │   └── .git/objects/*
│   ├── file dead23
│   ├── file feeb1e
│   ├── newsubdir
│   └── file 1010b

```

Как и на рис. 2.5, новый коммит добавил один связанный с ним объект дерева, представляющий общее состояние каталога и файловой структуры. Поскольку каталог верхнего уровня изменяется добавлением нового подкаталога, *содержимое* объекта дерева верхнего уровня тоже изменилось, поэтому Git создает новое дерево, `cafed00d`.

BLOB-объекты `dead23` и `feeb1e` между первым и вторым коммитами не изменились. Git понимает, что их ID не изменились, а значит, новое дерево `cafed00d` может ссылаться на них и использовать их.

Обратите внимание на направление стрелок между коммитами. Родительский коммит или коммиты находятся в структуре выше. Представьте себе это в виде

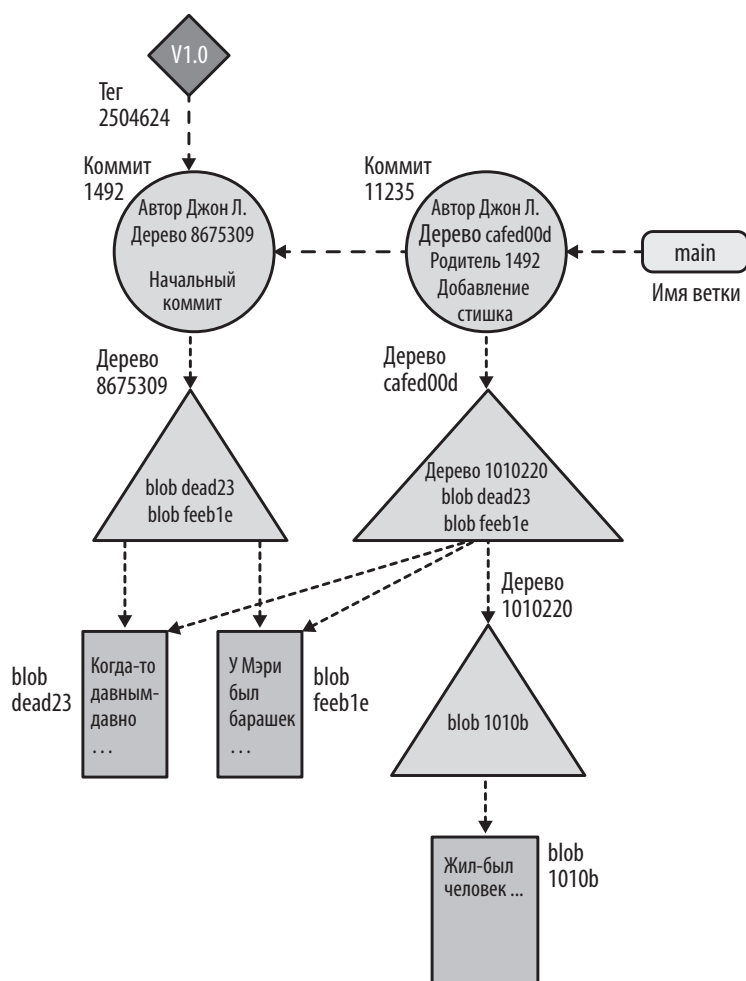


Рис. 2.6. Объекты Git после второго коммита

диаграммы DAG: направленный ациклический граф (directed acyclic graph), в котором каждый узел ориентирован на предыдущий в одном направлении, формируя топологический порядок.

Следовательно, в реализации Git каждый коммит указывает на своего родителя или родителей. Многих это путает, потому что состояние репозитория традиционно изображается в обратном направлении: в виде потока данных *от* родительского коммита к потомкам. Иными словами, при упорядочивании слева направо крайний правый коммит в диаграмме DAG представляет последнее состояние репозитория.

В главе 4 я подробнее покажу, как с помощью различных команд создается и управляется история репозитория.

Внутренние механики Git: принципы в действии

Теперь заглянем под капот и посмотрим, как все эти концепции сочетаются в репозитории. Начнем с создания нового репозитория и более детального рассмотрения внутренних файлов и объектного хранилища Git. Продвигаться будем от нижней части структуры данных Git вверх, постепенно добираясь до хранилища.

В Git есть несколько категорий команд для реализации внутренних механик. Подразделяются они так:

- основные (высокоуровневые команды для выполнения рутинных операций Git);
- вспомогательные (помогают запрашивать внутреннее хранилище данных Git);
- низкоуровневые команды (связующие команды для внутренних операций Git);
- внешние команды (расширяют стандартные операции Git);
- команды, которые обеспечивают связь с выбранным инструментом контроля версий (взаимодействуют с другими командами);
- псевдонимы команд (пользовательские псевдонимы для маскировки сложных команд Git).

Для повседневного использования и взаимодействия с Git чаще всего мы будем применять основные команды. Для лучшего понимания внутренней механики Git в этом разделе также используем некоторые низкоуровневые команды.



Вы можете вывести полный список команд и их категорий, выполнив в терминале `git help -a`.

Внутри каталога .git

Для начала инициализируем пустой репозиторий с помощью `git init`, а затем через `tree .git` посмотрим полученный результат:

```
$ mkdir /tmp/hello
$ cd /tmp/hello
$ git init -b main
Initialized empty Git repository in /tmp/hello/.git/

# Вывод всех файлов в текущем каталоге
$ tree .git
.git
├── HEAD
└── config
```

```

├── description
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── prepare-commit-msg.sample
│   ├── push-to-checkout.sample
│   └── update.sample
├── info
│   └── exclude
├── objects
│   ├── info
│   └── pack
├── refs
│   ├── heads
│   └── tags

```

8 directories, 17 files

Как видите, в `.git` много всего. Файлы отображаются на основе каталога-шаблона, который вы можете настроить по своему усмотрению, передав параметр `--template=каталог-шаблон`. Например, если вы решите создать новый репозиторий, реализующий пользовательские хуки Git, то можете указать на шаблон, предварительно настроенный на структуру пользовательского каталога и файлы хуков Git. Хуки более подробно мы разберем в главе 14.



В зависимости от установленной версии Git, ваш вывод может выглядеть иначе. Например, в более старых версиях в файлах `.git/hooks` не используется суффикс `.sample`. Подробности об этой команде вы можете узнать, выполнив команду `man git-init` в командной строке.

Обычно не нужно просматривать или управлять файлами в каталоге `.git`. Эти «скрытые» файлы считаются частью связующих команд или команд конфигурации.

Изначально каталог `.git/objects` (каталог для всех объектов Git) пуст, за исключением нескольких местозаполнителей:

```
$ find .git/objects
```

```

.git/objects
.git/objects/pack
.git/objects/info

```

Теперь создадим простой объект:

```
$ echo "hello world" > hello.txt
$ git add hello.txt
```

Если вы ввели "hello world" в точности как показано здесь (без изменений в пробелах и регистре букв), то ваш каталог **objects** будет выглядеть так:

```
$ find .git/objects
.git/objects
.git/objects/3b
.git/objects/3b/18e512dba79e4c8300dd08aeb37f8e728b8dad
.git/objects/pack
.git/objects/info
```

Сейчас в нем есть всего один объект: BLOB с ID SHA-1, сгенерированным на основе содержимого файла **hello.txt**. Выглядит загадочно. В следующем разделе вам станет понятно, что здесь происходит.

BLOB-объекты и хеши

Когда мы создали файл **hello.txt** и индексировали его в каталог **index** с помощью **git add**, Git создал BLOB-объект. Здесь для Git неважно, что файл называется **hello.txt**. Систему интересует только его содержимое: последовательность из 12 байтов, представляющих "hello world", и завершающий символ перевода строки (тот же BLOB, созданный ранее). Git выполняет над этим BLOB-объектом несколько операций, вычисляет его хеш и вносит в объектное хранилище как файл, названный в соответствии с шестнадцатеричным представлением хеша. В данном случае хеш-значение равно **3b18e512dba79e4c8300dd08aeb37f8e728b8dad**. 160 битов этого хеша соответствуют 20 байтам, что требует 40 байт шестнадцатеричного числа для отображения, поэтому содержимое хранится как **.git/objects/3b/18e512dba79e4c8300dd08aeb37f8e728b8dad**.

Git вставляет после первых двух цифр символ **/** для повышения эффективности работы файловой системы. (Некоторые системы замедляются, если поместить в каталог слишком много файлов; превращение первого байта хеша в каталог — это простой способ создать фиксированное 256-стороннее пространство имен с равным распределением для всех возможных объектов.) Чтобы убедиться, что Git не изменил содержимого файла (это по-прежнему тот самый "hello world"), вы можете использовать сгенерированный хеш для извлечения содержимого из хранилища объектов с помощью низкоуровневой команды:

```
# Использование команды git cat-file
$ git cat-file -p 3b18e512dba79e4c8300dd08aeb37f8e728b8dad hello world
```

или

```
# Использование команды git hash-object
$ echo "hello world" | git hash-object --stdin
3b18e512dba79e4c8300dd08aeb37f8e728b8dad
```



Git «знает», что 40 символов неудобно набирать вручную, поэтому предоставляет команду для поиска объектов по уникальному префиксу хеша объекта:

```
$ git rev-parse 3b18e512d
3b18e512dba79e4c8300dd08aeb37f8e728b8dad
```

ОТКУДА МЫ ЗНАЕМ, ЧТО ХЕШ SHA-1 УНИКАЛЕН?

Есть крайне малая вероятность, что два разных BLOB-объекта дадут одинаковый хеш. Подобная ситуация называется *коллизией*. В случае SHA-1 коллизия настолько маловероятна, что вы можете не беспокоиться о ней при работе с Git. Но может ли такая коллизия произойти случайным образом? Посмотрим.

При 160 битах у вас есть 2^{160} или около 10^{48} (1 с 48 нулями) возможных хешей. Это невероятно огромное число. Даже если вы наймете триллион людей для создания триллиона уникальных BLOB-объектов в секунду в течение триллиона лет, то у вас все равно получится всего около 10^{43} таких объектов.

Если вы хешируете 2^{80} случайных BLOB-объектов, то можете столкнуться с такой коллизией. Не верьте нам на слово. Прочтите статью Брюса Шнайера (Bruce Schneier) «Cryptanalysis of SHA-1» (<https://oreil.ly/BtN34>).

SHA-1 традиционно считается криптографически безопасным алгоритмом. Так было до недавних пор, когда исследователи безопасности смогли выявить несколько новых уязвимостей в целостности хеш-функции SHA-1. Об этом они написали на сайте Shattered (<https://shattered.io>).

Начиная с версии 2.13.0, в Git появилась логика обнаружения и отклонения атак, описанных в работе SHattered. Вероятность повтора такого вектора атаки нельзя исключить в будущем, поэтому в Git ввели новое расширение формата репозитория, которое позволяет использовать вместо SHA-1 алгоритм SHA256. Подробно это описано в технической документации (<https://oreil.ly/RW7eA>).

Теперь перейдем к структуре данных, чтобы понять, как Git хранит имена путей и файлов.

Объект дерева и файлы

Теперь, когда BLOB "hello world" надежно размещен в объектном хранилище, мы разберем, как он связан с именем файла. Система Git была бы не особо полезна, если не могла бы находить файлы по имени.

Как я говорил, Git отслеживает пути файлов с помощью другого вида объектов — *деревьев*. Когда вы используете `git add`, Git создает объект для содержимого каждого

добавляемого файла, но не объект для вашего дерева. Вместо этого система обновляет индекс. Индекс находится в `.git/index` и отслеживает пути файлов вместе с соответствующими BLOB-объектами. При каждом выполнении команд вроде `git add`, `git rm` или `git mv` Git обновляет индекс новой информацией о пути и BLOB-объекте.

При желании вы можете создать объект дерева из текущего индекса, сделав снимок его текущей информации с помощью низкоуровневой команды `git write-tree` (в реальной жизни это действие требуется редко).

Сейчас индекс содержит всего один файл, `hello.txt`:

```
$ git ls-files -s
100644 3b18e512dba79e4c8300dd08aeb37f8e728b8dad 0      hello.txt
```

Здесь вы можете видеть связь файла с BLOB `3b18e5...` Теперь сделаем снимок состояния индекса и сохраним его в объект дерева:

```
$ git write-tree
68aba62e560c0ebc3396e8ae9335232cd93a3f60

$ find .git/objects
.git/objects
.git/objects/68
.git/objects/68/aba62e560c0ebc3396e8ae9335232cd93a3f60
.git/objects/pack
.git/objects/3b
.git/objects/3b/18e512dba79e4c8300dd08aeb37f8e728b8dad
.git/objects/info
```

Теперь здесь два объекта: BLOB `"hello world"` в `3b18e5` и только появившийся объект дерева в `68aba6`. Как видите, имя объекта в виде хеша SHA-1 в точности соответствует подкаталогу и имени файла в `.git/objects`.

Но как выглядит дерево? Поскольку это такой же объект, как и BLOB, для его просмотра можно использовать ту же низкоуровневую команду:

```
$ git cat-file -p 68aba6
100644 blob 3b18e512dba79e4c8300dd08aeb37f8e728b8dad      hello.txt
```

Интерпретировать содержимое этого объекта несложно. Первое число, `100644`, представляет атрибуты объекта в восьмеричной системе, которая должна быть знакома всем, кто использовал команду Unix `chmod`. Здесь `3b18e5` — это имя BLOB-объекта `hello world`, а `hello.txt` — имя, связанное с этим BLOB.

Теперь легко увидеть, что объект *дерева* отразил информацию, которая находилась в индексе при выполнении `git ls-files -s`.

Особенности использования SHA-1 в Git

Прежде чем более подробно рассматривать содержимое дерева, давайте повторим важную особенность хешей SHA-1:

```
$ git write-tree  
68aba62e560c0ebc3396e8ae9335232cd93a3f60
```

```
$ git write-tree  
68aba62e560c0ebc3396e8ae9335232cd93a3f60
```

```
$ git write-tree  
68aba62e560c0ebc3396e8ae9335232cd93a3f60
```

В предыдущем примере при каждом вычислении очередного объекта дерева для одного и того же индекса (без добавления или удаления файлов) хеш остается одинаковым. Git не нужно заново создавать новый объект дерева. Если вы выполняете все эти шаги на своем компьютере, то должны видеть *точно такие же хеши*, что и приведенные здесь.

В математическом смысле хеш-функция является истинной функцией: для заданного ввода она всегда выдает один и тот же вывод. Подобную хеш-функцию иногда называют хеш-суммой, или дайджестом, чтобы подчеркнуть, что она служит своего рода обобщением для хешируемого объекта. Это также верно для любой хеш-функции. Даже простой бит четности обладает этим свойством.

Например, если вы создадите точно такое же содержимое, что и другой разработчик, независимо от того, где и как вы оба работаете, идентичный хеш будет достаточным подтверждением, что полное содержимое также идентично. Git рассматривает эти два содержимых как одинаковые, и это очень важно понимать.

Но подождите секунду: разве хеши SHA-1 не уникальны? Что случилось с триллионом людей, создающих триллионы BLOB-объектов в секунду, которые никогда не порождают коллизий? Это типичная причина путаницы, с которой сталкиваются новые пользователи Git. Читайте внимательно: если вы сможете понять это различие, то все остальное содержание текущей главы дастся вам легко.

В описанном выше случае идентичные хеши SHA-1 *не считаются коллизией*. Мы бы сочли их коллизией, если бы два *разных* объекта выдали одинаковый хеш. Здесь же мы создали два отдельных экземпляра идентичного содержимого, а идентичное содержимое всегда будет давать одинаковый хеш.

Git зависит от еще одного следствия хеш-функции SHA-1: неважно, *как* вы получили дерево с названием 68aba62e560c0ebc3396e8ae9335232cd93a3f60. Если оно у вас есть, можете быть уверены, что это тот же объект дерева, который, скажем, есть у другого читателя книги. Рассмотрим примеры.

Сценарий 1

Боб мог создать дерево, совместив в общем репозитории коммиты А и В от Дженни и коммит С от Сергея.

Сценарий 2

Работая в одном общем репозитории, вы могли создать такое же дерево, но по другому пути. Возможно, вы получили коммит А от Сью и апдейт от Лакшми, совмещающий коммиты В и С.

В обоих сценариях результаты для сгенерированного объекта дерева будут одинаковыми. Этот принцип способствует распределенной разработке с помощью Git.

Если вас попросят найти объект `68aba62e560c0ebc3396e8ae9335232cd93a3f60` и вы его найдете, то можете быть уверены, что перед вами в точности те же данные, из которых этот хеш был создан, поскольку SHA-1 является криптографическим хешем.

Будет верно и обратное: если вы не найдете объект с конкретным хешем в своем хранилище, то можете быть уверены, что у вас нет копии такого объекта. Короче говоря, вы можете определить, содержит ли ваше хранилище конкретный объект, даже ничего не зная о его (возможно, очень объемном) содержимом. Так что хеш служит надежной меткой или именем для объекта.

Иерархия деревьев

В примерах из предыдущего раздела у нас была информация для одного файла. Но реальные проекты содержат сложные, глубоко вложенные каталоги, которые со временем реорганизовываются и перемещаются. В этом разделе создадим новый подкаталог, содержащий идентичную копию файла `hello.txt`, чтобы увидеть, как Git обрабатывает такой сценарий:

```
$ pwd
/tmp/hello
$ mkdir subdir
$ cp hello.txt subdir/
$ git add subdir/hello.txt
$ git write-tree
492413269336d21fac079d4a4672e55d5d2147ac

$ git cat-file -p 4924132693
100644 blob 3b18e512dba79e4c8300dd08aeb37f8e728b8dad    hello.txt
040000 tree 68aba62e560c0ebc3396e8ae9335232cd93a3f60    subdir
```

Это новое дерево верхнего уровня содержит два элемента: исходный файл `hello.txt`, а также новый подкаталог `subdir`, имеющий тип `tree`, а не `blob`.

Взгляните внимательно на имя объекта `subdir`. Замечаете что-нибудь необычное? Это наш старый друг, хеш `68aba62e560c0ebc3396e8ae9335232cd93a3f60`!

Как так вышло? Новое дерево для `subdir` содержит только один файл, `hello.txt`, в котором находится то же старое содержимое `"hello world"`. Так что дерево `subdir` в точности совпадает с более старым деревом верхнего уровня. И вы правы, если

решили, что по одной этой причине он имеет то же самое имя SHA-1, что и раньше, в математическом смысле отражая специфику *истинной функции*.

Теперь заглянем в каталог `.git/objects` и посмотрим, на что повлияло это последнее изменение:

```
$ find .git/objects
.git/objects
.git/objects/49
.git/objects/49/2413269336d21fac079d4a4672e55d5d2147ac
.git/objects/68
.git/objects/68/aba62e560c0ebc3396e8ae9335232cd93a3f60
.git/objects/pack
.git/objects/3b
.git/objects/3b/18e512dba79e4c8300dd08aeb37f8e728b8dad
.git/objects/info
```

Здесь по-прежнему всего три *уникальных* объекта: BLOB с "hello world"; одно дерево с файлом `hello.txt`, который содержит текст "hello world", и символ переноса строки; а также второе дерево, которое содержит *еще одну* ссылку на `hello.txt` вместе с первым деревом (рис. 2.7).

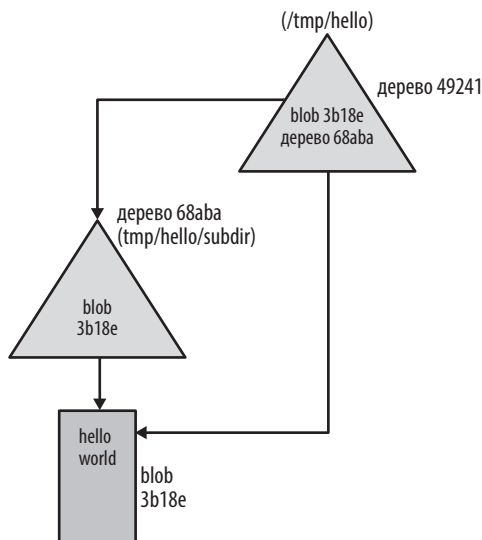


Рис. 2.7. Иерархия деревьев

Объекты коммита

Теперь обсудим объект коммита. После того как мы добавили `hello.txt` командой `git add` и создали объект дерева командой `git write-tree`, можно создать объект коммита с помощью низкоуровневых команд:

```
$ echo -n "Commit a file that says hello\n"
| git commit-tree 492413269336d21fac079d4a4672e55d5d2147ac
3ede4622cc241bcb09683af36360e7413b9ddf6c
```

Результат:

```
$ git cat-file -p 3ede462
tree 492413269336d21fac079d4a4672e55d5d2147ac
author Jon Loeliger <jdl@example.com> 1656932750 +0200
committer Jon Loeliger <jdl@example.com> 1656932750 +0200
```

Commit a file that says hello

Смотрите рис. 2.8.

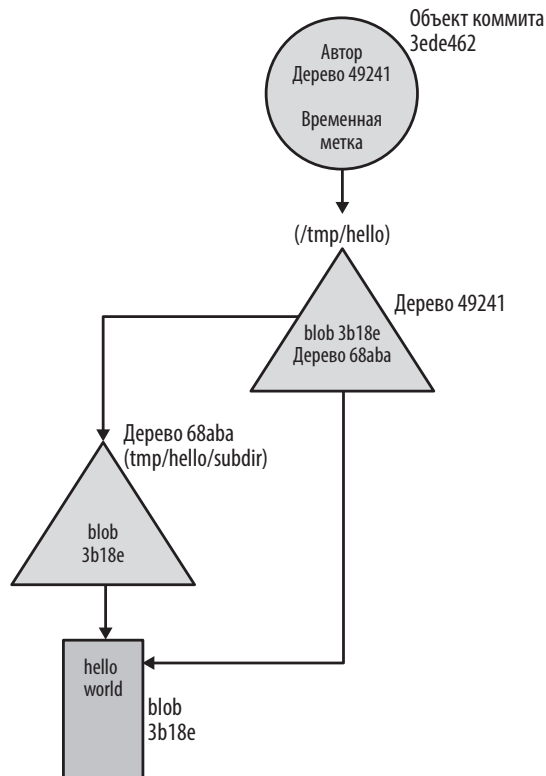


Рис. 2.8. Объект коммита

Если вы повторите этот пример на своем компьютере, то заметите, что у вас значение сгенерированного объекта коммита будет отличаться от приведенного здесь. Если вы поняли все предыдущие разъяснения, то причина должна быть для вас очевидна: наш объект коммита отличается от вашего.

Ваш содержит ваше имя и время создания коммита, а наш — другую временную метку и другое имя автора. Естественно, они отличаются.

Вместе с тем ваш коммит содержит то же самое дерево. Именно поэтому объекты коммитов хранятся отдельно от объектов деревьев: разные коммиты зачастую относятся к одному и тому же дереву. Когда такое происходит, система Git оказывается достаточно «сообразительной», чтобы перенести только новый объект коммита, который очень мал, а не все BLOB-объекты и объекты деревьев, которые наверняка намного больше.

В реальности вы можете (и должны) избегать используемых в примерах низкоуровневых команд `git write-tree` и `git commit-tree`. Чтобы эффективно и с легкостью использовать Git, просто применяйте команду `git commit`, не запоминая все эти низкоуровневые альтернативы.

Базовый объект коммита весьма прост, и это последний компонент для реальной системы контроля версий. Показанный выше объект коммита содержит:

- имя объекта дерева, которое фактически идентифицирует связанные с ним файлы;
- имя человека, создавшего новую ревизию (автора), и время ее создания;
- имя человека, который поместил новую ревизию в репозиторий (коммиттера), и время ее фиксации;
- описание причины создания этой ревизии (сообщение коммита).

По умолчанию автор и коммиттер — это один и тот же человек, хотя есть сценарии, в которых это могут быть разные люди.



Для просмотра дополнительных деталей заданного коммита используйте команду `git show --pretty=fuller`.

Более типична ситуация, когда проект содержит множество коммитов. В таком случае, когда вы создаете в проекте новый коммит, то можете присвоить ему один или больше родительских коммитов. С учетом этого рассмотрите последний коммит (или связанный с ним объект дерева) в проекте. Он содержит в себе хеш родительских коммитов и его дерева, которое, в свою очередь, содержит хеши всех его поддеревьев и BLOB-объектов рекурсивно по всей структуре данных. Следовательно, хеш исходного коммита однозначно определяет состояние всей структуры данных, корнем которой является этот коммит.

Проходя обратно по цепочке родителей, вы можете проследить историю своего проекта — отсюда и происходит термин *история коммитов*. Объекты коммитов также хранятся в структуре графа, хотя и полностью отличной от структур,

используемых объектами деревьев. Более подробно о коммитах и графе коммитов поговорим в главе 4.

Объекты тегов

Последний объект, которым управляет Git, — это тег. Несмотря на то что система Git реализует в своем хранилище всего один вид тегов, она поддерживает два их базовых типа: легковесный и аннотированный теги.

Легковесные теги (lightweight tags) — это ссылки на объект коммита, которые обычно считаются принадлежащими репозиторию. Они не хранятся как постоянные объекты в хранилище.

Аннотированный тег (annotated tag) создает объект. Он содержит сообщение, которое передаете вы, и его можно подписать цифровым ключом GnuPG.

Git равноценно воспринимает как легковесные, так и аннотированные имена тегов для связывания коммита с осмысленным человекочитаемым именем.



Типичный случай использования аннотированного тега — создание конкретной версии релиза проекта. Для легковесного тега типичный случай применения — когда требуется временная метка, прикрепленная к объекту коммита.

Аннотированный, неподписанный тег с сообщением коммита создается с помощью команды `git tag`:

```
$ git tag -a V1.0 3ede462
```

После выполнения этой команды Git запустит ваш предустановленный редактор, и вы сможете передать сообщение тега для завершения операции.

Содержимое созданного объекта тега просматривается через команду `git cat-file -p`, но какой у него хеш SHA-1? Чтобы это выяснить, воспользуйтесь советом из раздела «BLOB-объекты и хеши» на с. 59:

```
$ git rev-parse V1.0
6b608c1093943939ae78348117dd18b1ba151c6a

$ git cat-file -p 6b608c
object 3ede462cc241bcb09683af36360e7413b9ddf6c
type commit
tag V1.0
tagger Jon Loeliger <jdl@example.com> 1656932858 +0200

Tag version 1.0
```

Помимо сообщения журнала и информации об авторе, этот тег ссылается на объект коммита `3ede462`.

Git обычно отмечает тегом объект коммита, указывающий на объект дерева, который включает общее состояние всей иерархии файлов и каталогов в вашем репозитории.

Напомню (см. рис. 2.5), что тег `v1.0` указывает на коммит с именем `1492`, который, в свою очередь, указывает на дерево (`8675309`), охватывающее несколько файлов. Таким образом, тег одновременно применяется ко всем файлам этого дерева.

Это расходится, например, с CVS, который применяет тег к отдельному файлу, а затем использует коллекцию всех этих размеченных тегами файлов для воссоздания всей сопровождаемой тегами ревизии. И если CVS позволяет перемещать тег на отдельный файл, то Git требует нового коммита, включающего изменение состояния файла, к которому перемещается тег.

Итоги

Мы подробно разобрали внутренние механизмы Git. Давайте подытожим ключевые моменты.

Глава началась с общего знакомства с репозиторием. Мы узнали о различных рабочих каталогах в Git, главным образом об `index` (каталоге индексирования), `working directory` (рабочем каталоге) и `local history` (локальной истории). Затем изучили объектное хранилище Git и проанализировали каждый из его неизменяемых объектов. Мы также научились взаимодействовать с этими внутренними объектами напрямую с помощью низкоуровневых команд `git`, которые в реальной работе используются редко. Понимание этого принципа должно подчеркнуть тот факт, что Git по своей сути — это контентно-адресуемая база данных, чьи внутренние механизмы хоть и достаточно однозначны, но порой могут оказаться сложны для понимания. Мы также наглядно описали связь между объектами в хранилище Git. Теперь можем перейти ко второй части книги.

Часть II

ОСНОВЫ Git

В последующих главах я рассмотрю процесс управления файлами в Git, объясню важность индекса и его связь с этим процессом, а также подробно разберу объект коммита и важную роль веток в репозитории.

Для начала расскажу, почему эти темы будут идти именно в таком порядке.

В разделе «Внутренние механики Git: принципы в действии» на с. 57 в главе 2 мы изучили объекты, доступные в хранилище Git. Теперь разберем, как эти объекты создаются и когда устанавливают связь друг с другом.

Создадим пустой каталог и посмотрим его содержимое с помощью команды `tree`:

```
$ mkdir myrepo && cd myrepo

$ git status
fatal: not a git repository (or any of the parent directories): .git

$ tree .git
.git [error opening dir]

0 directories, 0 files
```

Поскольку это не репозиторий Git, в нем нет каталога `.git`. Далее инициализируем пустой репозиторий Git и еще раз посмотрим содержимое:

```
$ git init -b main
Initialized empty Git repository in /myrepo/.git/

$ tree .git
.git
├── HEAD
├── config
├── description
├── hooks
│   ├── applypatch-msg.sample
│   └── commit-msg.sample
```

```
|   | fsmonitor-watchman.sample
|   | post-update.sample
|   | pre-applypatch.sample
|   | pre-commit.sample
|   | pre-merge-commit.sample
|   | pre-push.sample
|   | pre-rebase.sample
|   | pre-receive.sample
|   | prepare-commit-msg.sample
|   | push-to-checkout.sample
|   | update.sample
|   |
|   | info
|   |   | exclude
|   |
|   | objects
|   |   | info
|   |   | pack
|   |
|   | refs
|   |   | heads
|   |   | tags
```

8 directories, 17 files

Тут мы видим, что каталог `.git` был создан. В соответствующих подкаталогах будут храниться объекты `git`.

При инициализации мы создали ветку `main`. В главе 3 мы узнаем, что эти ветки — просто указатели на объекты коммитов, в частности на `HEAD` или верхушку коммита текущей ветки.

Изучим содержимое каталога `HEAD` на основе вывода дерева:

```
$ cat .git/HEAD
ref: refs/heads/main
```

Оно указывает на имя ветки, которую мы создали ранее. Заглянем в содержимое поглубже:

```
$ cat .git/refs/heads/main
cat: .git/refs/heads/main: No such file or directory
```

Интересно, что у нас есть ветка, но она пуста. Вернемся к этому позднее.

Создадим файл и снова выполним команду `tree .git`:

```
$ echo "Hello Git" > file
$ tree .git
.git
|   HEAD
|   config
|   description
|   hooks
|   |   applypatch-msg.sample
|   |   commit-msg.sample
```

```

├── fsmonitor-watchman.sample
├── post-update.sample
├── pre-applypatch.sample
├── pre-commit.sample
├── pre-merge-commit.sample
├── pre-push.sample
├── pre-rebase.sample
├── pre-receive.sample
├── prepare-commit-msg.sample
├── push-to-checkout.sample
├── update.sample
├── info
│   └── exclude
├── objects
│   ├── info
│   └── pack
├── refs
│   ├── heads
│   └── tags

```

8 directories, 17 files

Заметьте, что каталог `index` пока не был создан. Отметим файл для отслеживания Git:

```

$ git add file
$ tree .git
.git
├── HEAD
├── config
├── description
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── prepare-commit-msg.sample
│   ├── push-to-checkout.sample
│   └── update.sample
├── index
├── info
│   └── exclude
├── objects
│   ├── 9f
│   │   └── 4d96d5b00d98959ea9960f069585ce42b1349a
│   └── info

```

```

├── pack
└── refs
    ├── heads
    └── tags

```

9 directories, 19 files

Когда этот файл начнет отслеживаться, выполнение команды `tree .git` раскроет два изменения. Первое: появился каталог `index` с типом `binary`. Второе: в каталоге `object` есть новое содержимое. Более подробно разберем это в главе 5:

```

$ cat .git/index
DIRCbfX
bFXX

I
BBBBB4file([o+fu\%

```

Теперь разберем содержимое каталога `object` с помощью низкоуровневых команд `git`:

```

# Проверка типа объекта
$ git cat-file -t 9f4d96
blob

# Проверка содержимого объекта
$ git cat-file -p 9f4d96
Hello Git

```

Вы видите, что содержимое имеет тип `blob` и соответствует тому, что мы передали. Сделаем первый коммит:

```

$ git commit -m "Initial commit"
[main (root-commit) 75f501d] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 file

```

```

# выполняем tree .git, чтобы просмотреть изменения в объектном хранилище
$ tree .git
.git
├── COMMIT_EDITMSG
├── HEAD
├── config
├── description
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   └── pre-applypatch.sample

```

```

├── pre-commit.sample
├── pre-merge-commit.sample
├── pre-push.sample
├── pre-rebase.sample
├── pre-receive.sample
├── prepare-commit-msg.sample
├── push-to-checkout.sample
├── update.sample
├── index
├── info
│   └── exclude
├── logs
│   ├── HEAD
│   └── refs
│       └── heads
│           └── main
├── objects
│   ├── 16
│   │   └── b2356b798a3bfa903c9a54dc608282f4394e2b
│   ├── 9f
│   │   └── 4d96d5b00d98959ea9960f069585ce42b1349a
│   ├── 75
│   │   └── f501da811525b594f2240e015eebeed388042
│   ├── info
│   └── pack
├── refs
│   ├── heads
│   │   └── main
│   └── tags

```

14 directories, 25 files

Посмотрим, что изменилось. При создании первого коммита мы получили SHA с меткой (корневой коммит). Тот же сокращенный SHA встречается в своей полной форме в каталоге **object**. Проверить это можно так:

```

# Проверка содержимого объекта коммита
$ git cat-file -p 75f501
tree 16b2356b798a3bfa903c9a54dc608282f4394e2b
author Prem Kumar Ponuthorai <ppremk@gmail.com> 1658662758 +0200
committer Prem Kumar Ponuthorai <ppremk@gmail.com> 1658662758 +0200

```

Initial commit

Здесь также видно, что объект коммита содержит объект дерева. Более углубленное изучение объекта дерева показывает, что он связан с ранее созданным BLOB-объектом. Подробнее об этом поговорим в главе 4.

```

# Проверка содержимого объекта tree
$ git cat-file -p 16b23
100644 blob 9f4d96d5b00d98959ea9960f069585ce42b1349a readme.md

```

Поскольку у нас есть объект коммита, вернемся и заглянем в каталог `.git/refs/heads/main`:

```
$ cat .git/refs/heads/main
75f501da811525b594f2240e015eeeebeed388042
```

На этот раз есть содержимое, и оно является хешем объекта коммита. Мы сделали полный круг. Эта связь наглядно показана на рис. II.1.

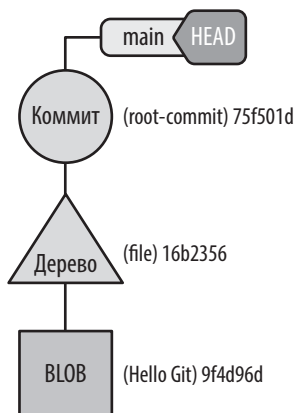


Рис. II.1. Связь объектов Git

Эти примеры показывают внутренние механизмы Git и их тесную взаимосвязь. Как видите, не так-то просто решить, какой порядок глав будет наиболее логичен. В конце концов мы решили придерживаться подхода «снаружи вовнутрь» и «сверху вниз», когда сначала объясняем, что такое ветки, затем говорим о коммитах и, наконец, рассказываем, как управлять файлами и как они соотносятся с индексом внутри репозитория Git.

Идея состоит в том, чтобы преподнести вам эти темы в той последовательности, в какой вы будете реализовывать их в работе. Иными словами, начинаем с поверхности и уже затем заглядываем внутрь, разбираясь с внутренними механизмами Git.

В процессе чтения вы будете встречать некоторые концепции или формулировки, которые будут разъяснены немного позже. Это может запутать или вызвать вопросы. Если такое случится, просто сделайте для себя пометку и продолжайте чтение. В конце этой части все сложится в осмысленную картину.

Ветки

Ветка позволяет запускать отдельную линию разработки в рамках проекта. Создавая ветку (branch), вы создаете ответвление, или форк (fork), от конкретного состояния проекта. Это позволяет работать над проектом одновременно в нескольких направлениях.

Можно представить это как путешествие во времени, где вы можете создавать параллельные временные линии, исходящие из одной временной точки. Ветка также позволяет создавать разные версии проекта. Зачастую ветку можно синхронизировать и объединять с другими, совмещая усилия разных команд.

Создание веток в Git считается легковесной и недорогой операцией. Причина в том, что ветка — это просто указатель на конкретный объект коммита в репозитории. Git позволяет создавать множество веток, а значит, в любой момент времени в репозитории может существовать множество линий разработки. Более того, в Git есть отличная поддержка слияния веток. В итоге пользователи регулярно используют ветки по указанным выше причинам.

В этой главе рассмотрим функционирование веток в Git сверху вниз, изучив, как разработчики поддерживают несколько линий развития в рамках проекта. Концепция, обсуждаемая в этой главе, дополнит основные выводы, с которыми мы познакомились в главе 2.

Мы покажем, как выводить список, выбирать, создавать и удалять ветки. Мы также приведем некоторые лучшие практики работы, чтобы ваши ветки не нарушали структуру времени и существование параллельных временных линий¹.

¹ Мы знаем о парадоксе путешествий во времени и сложностях, которые могут возникнуть. Поэтому будем придерживаться только воображаемых временных линий, не затрагивая тему реального путешествия во времени.

Зачем в Git нужны ветки

В репозитории Git ветки могут создаваться по техническим, философским, управленческим и даже социальным причинам. Распределенная природа Git совместно с продуманной стратегией ветвления позволяет сосуществовать нескольким потокам разработки, которые способствуют структурированной работе над проектами среди нескольких команд. Вот наиболее типичные причины:

- Ветка может отражать этапы разработки проекта — например, стадии *stable* (стабильная), *development* (разработка), *release candidate* (кандидат на релиз) и *production* (продакшен). Это позволяет скоординировать и упростить рабочий поток с вовлечением нескольких команд.
- Если вы планируете запустить новую версию своего проекта, но знаете, что часть клиентов захотят остаться на старой версии, то вы можете сохранить старую версию в виде отдельной ветки для обеспечения обратной совместимости.
- Ветка позволяет вносить корректировки и работать над конкретной функциональностью или исправлением бага в проекте в изолированной среде разработки. Вы можете создавать несколько веток функциональности, которые легко объединить путем слияния перед релизом. Система ветвления в Git надежная и незатратная, так что при работе над небольшими изменениями в каждой создаваемой ветке такой подход приветствуется и не считается избыточным. Слово *функциональность* (feature) говорит о том, что каждая ветка в репозитории служит конкретной цели.
- В большом и комплексном проекте отдельная ветка может представлять труд отдельного разработчика. Такой метод работы позволяет командам со сложными требованиями к проекту разделять реализацию различных частей проекта, прежде чем создавать новую ветку для интеграции и объединения этих частей в финальный продукт перед релизом.

В Git также есть понятие *ветки отслеживания* (tracking branch), или ветки для сохранения синхронности между клонами репозитория. В главе 11 объясняется, как использовать эту ветку.

Руководство по ветвлению

Если в своих проектах вы будете опираться на лучшие практики, а также иметь под рукой понятные практические руководства, то успех и комфортная работа

всех участников обеспечены. То же самое справедливо и для работы с ветками в репозитории. В этом разделе мы поговорим о некоторых типичных приемах и советах по использованию веток в Git.

Имена веток

Имя, присваиваемое ветке, по сути произвольно, хотя здесь есть некоторые ограничения. При инициализации репозитория Git присваивает ветке имя по умолчанию. В версии Git 2.37.1 это имя `master`, но мы рекомендуем поменять его на `main`. Предусловленное имя ветки используется разработчиками для ведения надежной и стабильной кодовой базы. Git позволяет переименовывать и даже удалять предусмотренную ветку. Если вы инициализируете новый репозиторий, то можете изменить имя ветки по умолчанию:

```
$ git init --initial-branch=branch-name
```

или

```
$ git init -b branch-name
```

В качестве альтернативы можете подправить файл конфигурации, включив в него атрибут `init.defaultBranch`, который обеспечит постоянное использование указанного вами имени:

```
$ git config --global init.defaultBranch branch-name
```

Если вы планируете переименовать ветку репозитория, сохранив его историю коммитов, то это можно сделать так:

```
$ git branch -m old-branch-name new-branch-name
```

Для поддержания масштабируемости и организации по категориям можете создать иерархическое имя ветки, напоминающее пути в Unix. Например, вы хотите с легкостью определять через имя ветки различные реализации функциональности и исправления багов либо представлять внесенные вами изменения (если ваша работа будет включена в основной проект менеджером по интеграции). Тогда вы можете создать иерархическую структуру на отдельных ветках с названиями вроде *features/feature-A*, *bug/ticket-no-x* и *ppremk/feature-B*.

Преимущество использования иерархических имен веток в том, что Git, подобно оболочке Unix, поддерживает символы подстановки. Например, при наличии схемы именования *bug/ticket-no-x* и *bug/ticket-no-y* вы можете выбрать все ветки *bug* сразу, используя знакомый шаблон:

```
git show-branch 'bug/*'
```

Правила именования веток

Имена веток должны соответствовать простым правилам:

- Для создания иерархической схемы именования можно использовать прямой слеш (/). Однако заканчиваться слешем имя не может.
- Никакой разделенный слешем компонент не может начинаться с точки (.). Имя ветки вроде `feature/.new` недопустимо.
- Имя не может начинаться со знака «минус» (-).
- Имя не может содержать две последовательные точки (.).
- Имя не может включать пробелы или другие символы пустого пространства.
- Имя не может включать символы, имеющие для Git особое значение, в том числе тильду (~), каретку (^), двоеточие (:), вопросительный знак (?), звездочку (*) и открывающую квадратную скобку ([).
- Имя не может включать управляющий символ ASCII, который является любым байтом со значением меньше восьмеричного \040, или символ DEL (восьмеричное \177).

Соблюдение этих правил проверяется низкоуровневой командой `git check-ref-format`.

Правила гарантируют, что имя каждой ветки легко напечатать и оно пригодно в качестве имени файла в каталоге `.git` и скриптах.

Управление ветками

Помимо базовой ветки, в репозитории в течение жизни проекта может появляться множество других. Тем не менее одновременно вы будете работать только в одной текущей, или активной, ветке. Эта ветка помогает определить, какие файлы находятся в вашем *рабочем каталоге*. Иными словами, она отражает состояние файла, находящегося в разработке.

Текущая ветка — это зачастую неявный операнд в командах Git, например цель (`target`) в операции `merge` (слияние). По умолчанию при каждой инициализации репозитория Git активной выступает ветка `main`, но вы можете создать дополнительные и сделать текущей любую из них.

На рис. 3.1 показан граф коммитов, содержащий две ветки. Заметьте, что указатель `HEAD` нацелен на имя ветки и постоянно остается обновленным, указывая на последний коммит в ветке. Держите эту структуру графа в уме, когда будете управлять ветками — так вы закрепите понимание элегантной и простой объектной модели, лежащей в основе ветвления в Git. Подробно мы разберем коммиты в главе 4.

в ветке, который представляет стабильную точку в проекте, вы можете явно присвоить легковесный либо аннотированный тег. Все это показано на рис. 3.3.

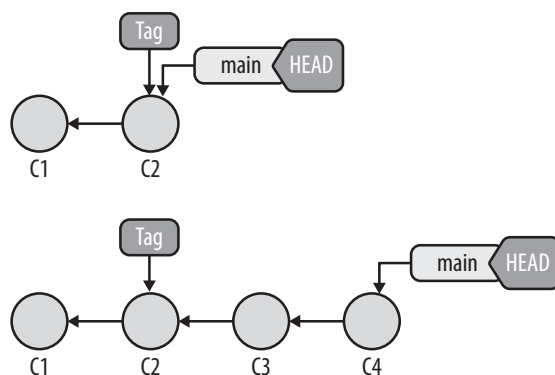


Рис. 3.3. Имя ветки и имя тега

Любая ветка включает достаточное число коммитов для воссоздания истории проекта вдоль ветки, от которой она ответвилась, и вплоть до начала проекта.

ВЕТКИ И ТЕГИ

Ветка и тег могут казаться похожими, но они служат разным целям.

Тег можно использовать как временную закладку (легковесный) или как постоянный указатель (аннотированный) для обозначения момента релиза конкретной версии продукта. Таким образом, теги являются статичными и всегда указывают на конкретный объект коммита в репозитории.

Ветка же указывает на каждый коммит, сделанный во время разработки, и перемещается вместе с ним. Каждая ссылка на ветку (имя ветки) будет следовать каждой отходящей линии разработки в репозитории. То есть ветки динамичны.

Когда использовать тег, а когда ветку? Решение полностью за вами и вашей политикой разработки. Учитывайте их ключевые отличительные особенности. Ссылка статичная и неизменяемая или же динамическая? В первом случае нужно использовать тег, во втором — ветку.



Вы можете называть ветку и тег одним именем. Но тогда, чтобы различать их, вам придется использовать полные ссылочные имена, например `refs/tags/v1.0` и `refs/heads/v1.0`. Рекомендуем избегать одинаковых имен для ветки и тега, если у вас нет на это веских причин. Для определения того, какую ссылку использовать, Git применяет последовательность, которая основана на правиле из документации (<https://oreil.ly/Mvm6S>).

На рис. 3.4 ветка **development** указывает на верхний, или HEAD, коммит в этой ветке. Если вы хотите воссоздать состояние репозитория, когда он находился в точке Z, вам потребуются все коммиты, достижимые от позиции Z обратно к исходному коммиту A. Эта достижимая часть графа выделена толстыми линиями и охватывает каждый коммит, за исключением S, G, H, J, K и L. Заметьте, что стрелки от коммита W к F и от коммита X к R представляют операцию слияния между соответствующими ветками. Операцию слияния мы разберем в главе 6.

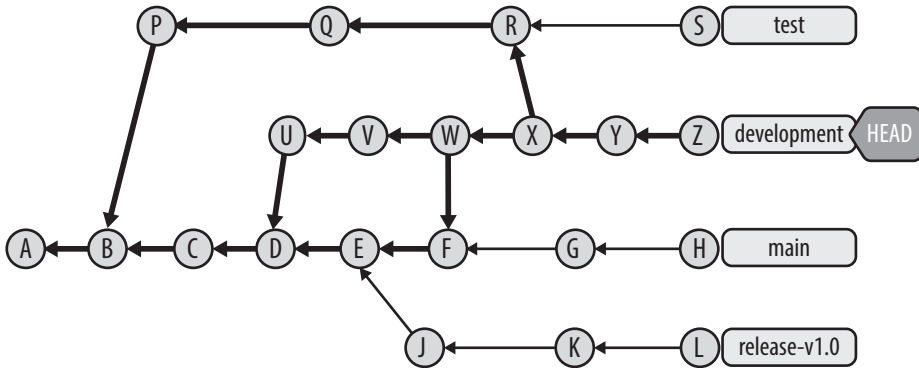


Рис. 3.4. Коммиты, достижимые из ветки разработки (development)

Каждое из имен веток вместе с зафиксированным содержимым локально для вашего репозитория. Когда вы делаете свой репозиторий доступным для других, то можете опубликовать или сделать доступными также одну либо множество веток и связанные с ними коммиты. Публикация ветки должна выполняться явно. Подробнее поговорим об этом в главе 11. Кроме того, если ваш репозиторий клонирован, то имена и разработка веток станут его частью.

Создание веток

Git спроектирована так, чтобы поддерживать сложную структуру ветвления. Сюда относится создание новых веток из уже существующих, а также создание форков, расходящихся на несколько веток от одного коммита.

Когда вы создаете новую ветку, она всегда основывается на существующем в репозитории коммите. Это может быть текущий HEAD коммит или другой, на который вы ссылаетесь явно с помощью его SHA (ID объекта коммита).

Ветка может иметь короткий или длинный срок жизни, что будет зависеть от вас. За весь период существования ветки ее имя можно добавлять и удалять множество раз в зависимости от ваших потребностей при разработке.

Базовая форма команды для создания ветки выглядит так:

```
$ git branch branchname start-point
```

Если не указать *start-point*, то по умолчанию стартовой точкой будет верхний коммит текущей активной ветки. Другими словами, новая ветка по умолчанию начинается там, где вы сейчас работаете.

Заметьте, что команда `git branch` просто регистрирует имя в репозитории. Она не переключает рабочий каталог на *использование* этой новой ветки. Команда просто создает новую именованную ветку у заданного коммита. Это также означает, что изменений в файлах рабочего каталога не будет, как и неявных изменений в контексте ветки. И, естественно, при выполнении команды никакие новые коммиты совершены не будут.

Чтобы начать работать в новой ветке, на нее нужно переключиться (поговорим об этом в разделе «Переключение веток» на с. 86). Причина этого в том, что **HEAD** после создания новой ветки продолжает указывать на активную, как показано на рис. 3.5.

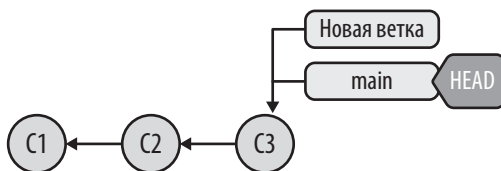


Рис. 3.5. HEAD, активная ветка и новая ветка

Предположим, что стратегия ветвления в вашем проекте отражает его релизные продакшен-версии, и перед вами стоит задача исправить баг в определенных релизах. В этом случае удобно использовать параметр *start-point* для создания ветки исправления бага, исходящей от последнего коммита указанной ветки релиза. Это позволит вам переключить рабочий каталог для отражения состояния проекта в точке, представляющей этот релиз.

Например, вы можете указать ветку с именем `rel-2.3`, чтобы представить Production Version 2.3 вашего проекта. Внести исправление багов в отдельной ветке для указанной версии можно так:

```
$ git branch bugs/fix-1311 rel-2.3
```



Параметр *start-point* получает имя ветки, SHA коммита или имя тега. Использование SHA коммита позволяет вам указывать на любые коммиты внутри линии разработки любой ветки. Вы можете ответвляться от одного коммита в нескольких направлениях, чтобы изучить альтернативные решения относительно общей стартовой точки:

```
$ git branch bugs/fix-1311 e69de29bb2d1d6434b8b29ae775ad8c2e48c5391
```

Вывод списка имен веток

Команда `git branch` выводит имена веток, находящихся в репозитории:

```
$ git branch
bugs/pr-1311
development
* main
```

В этом примере показаны три ветки. Активная ветка, связанная с рабочим каталогом, обозначена звездочкой. В этом примере также показаны две другие ветки: `bugs/pr-1311` и `development`.

Без дополнительных параметров выводятся только ветки в локальном репозитории. Как вы увидите в главе 11, в репозитории могут быть дополнительные ветки удаленного отслеживания. Их можно выводить, дополнив команду `branch` параметром `-r`. Используя параметр `-a`, можно выводить *локальные* и *удаленные* ветки одновременно.

Просмотр веток и их коммитов

Команда `git show-branch` предоставляет более детальный вывод, чем `git branch`. Она выводит коммиты, относящиеся к одной или более веток в обратном хронологическом порядке. Как и в случае с командой `git branch`, без дополнительных параметров `git show-branch` выведет *локальные* ветки. При использовании с параметром `-r` команда `git show-branch` показывает ветки удаленного отслеживания. С параметром `-a` она показывает все *локальные* и *удаленные* ветки.

Вывод команды `git show-branch` делится на две части, разделенные линией из тире (---).

В части, идущей над этой линией, перечисляется следующее:

- имена веток, заключенные в квадратные скобки, по одной на строку;
- имя каждой ветки, предваренное специальными символами: звездочкой (*) для обозначения текущей ветки и восклицательным знаком (!) для ссылки на ветку *функциональности*;
- имя каждой ветки с ее однострочным сообщением из последнего коммита, что удобно использовать в качестве наглядной справки.

Под разделяющей линией перечисляется каждый коммит вместе с именем его ветки в соответствии со следующими соглашениями.

- Вывод в формате матрицы, указывающей, какие коммиты есть в соответствующих столбцах каждой перечисленной ветки.

- Префиксы со специальными символами. Знак «плюс» (+) указывает, что коммит есть в перечисленной ветке, звездочка (*) — что коммит есть в текущей ветке, а знак «минус» (−) означает, что коммит является *коммитом слияния* в этой ветке.
- С однострочным сообщением коммита. Как уже говорилось, Git присваивает имя ветки последнему коммиту. Значит, предыдущий коммит в той же ветке будет иметь то же имя со специальным символом вставки (^) на конце.

Рассмотрим вывод команды `git show-branch`:

```
$ git show-branch
! [bugs/pr-1] Fix Problem Report      ❶
* [dev] Improve the new development   ❷
! [main] Added Bob's fixes.          ❸
---
* [dev] Improve the new development.
* [dev^] Start some new development.
+ [bugs/pr-1] Fix Problem Report 1.
+*+ [main] Add Bob's fixes.
```

- ❶ Коммиты в ветке `bugs/pr-1` начинаются в первом столбце.
- ❷ Коммиты в текущей ветке `dev` начинаются во втором столбце.
- ❸ Коммиты в ветке `main` начинаются в третьем столбце.



В качестве столбцов используйте пунктирные линии, а в качестве их заголовков — имена веток. Это сделает вывод в виде матрицы удобочитаемым.

Например, оба показанных ниже коммита обозначены звездочками и присутствуют в ветке `dev`:

```
! ...
* [dev] Improve the new development
! ...
---
* [dev] Improve the new development
* [dev&#x2038;] Start some new development.
...
...
```

Ни в одной другой ветке этих коммитов нет. Они выведены в обратном хронологическом порядке: последний находится сверху, а самый старый — внизу. В главе 4 мы разберем, почему имена веток сопровождаются символами вставки для обозначения предпоследнего коммита. Два последних коммита в ветке `dev` также будут иметь имена `dev` и `dev^`.



Несмотря на то что коммиты внутри ветки упорядочены, сами ветки перечисляются в произвольном порядке. Причина в том, что все ветки имеют равный статус. Нет правила, которое бы указывало, что одна ветка важнее другой.

Если одинаковый коммит есть в нескольких ветках, он будет сопровожден знаком «плюс» или звездочкой для каждой. Таким образом, последний коммит, показанный в предыдущем выводе, имеется во всех трех ветках:

```
! [bugs/pr-1] Fix Problem Report 1
* [dev] Improve the new development
! [main] Added Bob's fixes.
---
...
...
+*+ [main] Added Bob's fixes.
```

Первый плюс означает, что коммит находится в `bugs/pr-1`, звездочка говорит о том, что тот же коммит находится в текущей ветке `dev`, а второй плюс — что коммит также есть в ветке `main`.

При выполнении команда `git show-branch` обходит все коммиты во всех показанных ветках, прекращая вывод на последнем общем коммите, присутствующем во всех них. В примере выше Git вывела четыре коммита, прежде чем нашла общий для всех трех веток (`Added Bob's fixes.`), где и остановилась.

Остановка у первого общего коммита считается стандартным эвристическим подходом. Предполагается, что достижение такой общей точки дает достаточный контекст для понимания взаимосвязанности веток. Если по какой-то причине вы хотите видеть более развернутую историю коммитов, используйте опцию `--more=num`, указывающую количество дополнительных коммитов, которые хотите вывести, двигаясь назад во времени вдоль общей ветки.



Вывод команды `git show-branch` показывает максимум 29 веток и коммитов одновременно.

Если хотите ограничить историю, отображаемую командой `git show-branch`, то можете передать в качестве параметров *имена веток*. Например, если новая ветка с именем `bugs/pr-2` была добавлена у коммита `main`, это будет выглядеть так:

```
$ git branch bugs/pr-2 main
$ git show-branch
! [bugs/pr-1] Fix Problem Report 1
! [bugs/pr-2] Added Bob's fixes.
* [dev] Improve the new development
```

```

! [main] Added Bob's fixes.
----
* [dev] Improve the new development
* [dev^] Start some new development.
+ [bugs/pr-1] Fix Problem Report 1
++*+ [bugs/pr-2] Added Bob's fixes.

```

Если вы хотите увидеть историю коммитов только для веток `bugs/pr-1` и `bugs/pr-2`, то сделайте так:

```

$ git show-branch bugs/pr-1 bugs/pr-2
! [bugs/pr-1] Fix Problem Report 1
! [bugs/pr-2] Added Bob's fixes.
--
+ [bugs/pr-1] Fix Problem Report 1
++ [bugs/pr-2] Added Bob's fixes.

```

Для удобства в этих примерах мы использовали хорошо структурированное соглашение именования. Это означает, что мы также можем использовать сопоставление имен веток с помощью символов подстановки. Те же результаты можно получить, взяв более простое подстановочное имя ветки `bugs/*`:

```

$ git show-branch bugs/*
! [bugs/pr-1] Fix Problem Report 1
! [bugs/pr-2] Added Bob's fixes.
--
+ [bugs/pr-1] Fix Problem Report 1
++ [bugs/pr-2] Added Bob's fixes.

```

Переключение веток

Как уже говорилось, одновременно вы можете работать только с одной веткой. Это отражает состояние файлов в рабочем каталоге. Если нужно переключиться на другую ветку, используйте команду `git checkout`:

```
$ git checkout branch
```

Когда вы указываете в этой команде имя ветки, она делает ее новой, текущей рабочей веткой, тем самым также изменяя файл рабочего каталога и структуру каталогов, чтобы они соответствовали заданной ветке. В Git также есть встроенный механизм защиты от внесения любых незафиксированных изменений в текущем рабочем каталоге при переключении на другую ветку, что помогает избежать потери данных в ходе этого процесса.



Команду `git checkout` можно использовать не только для получения состояния конкретной ветки, но и для получения доступа ко всем состояниям репозитория — от верхушки данной ветки и до начала проекта.

Простой пример переключения ветки

Возьмем пример из предыдущего раздела, чтобы лучше понять состояние рабочего каталога до и после смены ветки командой `git checkout`. Допустим, нужно переключиться с ветки `dev` на ветку `bugs/pr-1`, чтобы внести решение обнаруженной проблемы. Сделать это можно так:

```
$ git branch
  bugs/pr-1
  bugs/pr-2
* dev
  main

$ git checkout bugs/pr-1
Switched to branch "bugs/pr-1"

$ git branch
* bugs/pr-1
  bugs/pr-2
  dev
  main
```

Файлы и структура каталогов вашей рабочей директории будут обновлены, чтобы отразить состояние и содержимое новой ветки, `bugs/pr-1`. Степень этого изменения зависит от отличий между ставшей активной веткой, `bugs/pr-1`, и предыдущей, `dev`. Эффекты переключения веток можно обобщить следующим образом:

- Файлы и каталоги, находящиеся в ветке, на которую происходит переключение, но отсутствующие в предыдущей, из объектного хранилища помещаются в текущий рабочий каталог. Если в ветке `bugs/pr-1` есть файлы, которых не было в ветке `dev`, они появятся в рабочем каталоге.
- Файлы и каталоги, которые есть в текущей ветке, но отсутствуют в той, на которую происходит переключение, из рабочего каталога удаляются. Все файлы, которые находились в ветке `dev`, но не являются частью ветки `bugs/pr-1`, исчезнут из рабочего каталога.
- Файлы, находящиеся в обеих ветках, изменяются, чтобы отражать содержимое, присутствующее в новой ветке. Если в ветках `dev` и `bugs/pr-1` был одинаковый файл, его содержимое будет изменено для соответствия состоянию, в котором он находился на момент совершения коммита в ветке `bugs/pr-1`.



Переключение веток происходит почти мгновенно. Это одна из функциональностей Git, которая существенно отличает эту систему от многих других. Git также хорошо определяет минимальный набор файлов и каталогов, которые нужно изменить во время операции переключения.

Переключение при наличии незафиксированных изменений

В предыдущем примере мы показали переключение на ветку при пустом рабочем каталоге: изменили ветки, когда в индексе не было измененных или созданных файлов или каталогов. Теперь рассмотрим ситуации, когда при переключении веток в рабочем каталоге есть незафиксированные изменения.

По умолчанию Git исключает случайное удаление или изменение данных в локальном рабочем каталоге без явного на то запроса. Файлы и каталоги в рабочей директории, которые не отслеживаются (то есть не находятся в одном из объектных хранилищ Git), всегда оставляются. Git не будет удалять или изменять их.

Но если у вас есть локальные изменения в файле, отличающиеся от изменений в новой ветке, на которую вы переключаетесь, Git выдаст ошибку, как показано ниже, и откажется переключаться на целевую ветку:

```
$ git branch
  bugs/pr-1
  bugs/pr-2
  dev
* main
```

```
$ git checkout dev
```

```
error: Your local changes to the following files would be overwritten by
checkout:
```

```
    NewStuff
```

```
Please, commit your changes or stash them before you can switch branches.
```

```
Aborting
```

Сообщение предупреждает, что по какой-то причине Git была вынуждена остановить запрос на переключение. Но что это за причина? Выяснить это можно, просмотрев содержимое файла `NewStuff`, поскольку он локально изменяется в текущем рабочем каталоге и целевой ветке `dev`:

```
# Показывает, как выглядит NewStuff в рабочем каталоге
```

```
$ cat NewStuff
```

```
Something
```

```
Something else
```

```
# Показывает, что локальная версия файла содержит дополнительную строку,
```

```
# незакоммиченную в текущей ветке рабочего каталога (main)
```

```
$ git diff NewStuff
```

```
diff --git a/NewStuff b/NewStuff
```

```
index 0f2416e..5e79566 100644
```

```
--- a/NewStuff
```

```
+++ b/NewStuff
```

```
@@ -1 +1,2 @@
```

```
    Something
```

```
+Something else
```

```
# Показывает, как выглядят файлы в ветке dev
```

```
$ git show dev:NewStuff
Something
A text existing in the dev branch version
```

Если бы Git не глядя удовлетворила запрос на переключение с ветки `dev`, то локальные изменения в файле `NewStuff` рабочего каталога оказались бы переписаны версией из `dev`. По умолчанию Git обнаруживает эту потенциальную утрату и исключает подобный сценарий.



Если вас не беспокоит утрата изменений в рабочем каталоге, вы можете принудить Git выполнить переключение, используя опцию `-f`.

При появлении сообщения об ошибке, как в примере выше, вам может быть предложено обновить файл в каталоге `index/staging`, а затем уже выполнить переключение веток. Тем не менее этого окажется недостаточно, поскольку, используя `git add` для обновления нового содержимого файла `NewStuff` в каталоге `index/staging`, мы только поместим содержимое этого файла в каталог `index/staging`; оно не будет зафиксировано ни в какую ветку. В итоге Git так и не сможет переключиться на новую ветвь, не потеряв ваши изменения, — попытка сделать это снова провалится:

```
# Перемещение файла в каталог index/staging
$ git add NewStuff

$ git checkout dev
error: Your local changes to the following files would be overwritten by
checkout:
    NewStuff
Please, commit your changes or stash them before you can switch branches.
Aborting
```

Файл все равно будет перезаписан. Очевидно, что простого его добавления в индекс недостаточно.

На этом этапе можно просто выполнить команду `git commit`, чтобы зафиксировать изменение в текущую ветку (`main`). Но допустим, вы хотите внести это изменение в новую ветку `dev`. Ситуация выглядит тупиковой: вы не можете поместить изменение в ветку `dev`, пока не выполните переключение, а Git не позволяет его выполнить, потому что есть изменение.

К счастью, из этой ловушки есть выход. Первый вариант — использовать прием «откладывания изменений в архив» (`stash`), который будет описан в главе 10. Второй подход описывается в разделе «Внесение изменений в другую ветку», где мы будем выполнять внесение изменений файла одновременно с переключением на нужную ветку.

Git checkout: работа с файлами или ветками

Несмотря на то что команда `git checkout` используется для изменения рабочих веток, она также предоставляет надежные опции для восстановления состояний файлов. Посмотрите:

```
# изменение в ветке dev
$ git checkout dev
```

```
# восстановление состояния конкретного файла
# четыре коммита назад от верхнего, или HEAD, коммита
$ git checkout dev~4 index.js
```

Ее также можно использовать для восстановления удаленного файла в рабочем каталоге из каталога `index/ staging`:

```
# восстановление удаленного файла из каталога index/staging
$ rm -rf server.js
$ git checkout server.js
```

При использовании команды `git checkout` может возникнуть путаница, если у ветки и файла будут одинаковые имена. Если вы хотите работать с файлом, а не веткой, то потребуется указать символ `--` и затем имя этого файла. Git поймет, что не нужно интерпретировать последующие аргументы как опции:

```
# переключение на файл с тем же именем, что и у ветки
$ git checkout -- file-with-same-branch-name.js
```

НОВЫЕ ЭКСПЕРИМЕНТАЛЬНЫЕ КОМАНДЫ ДЛЯ РАБОТЫ С ВЕТКАМИ

В версии Git 2.23.0 (<https://oreil.ly/y3Cxy>) появилась новая команда `git switch`, которая позволяет переключаться на конкретную ветку в репозитории:

```
$ git switch branch
```

Кроме того, появилась команда `git restore`, позволяющая восстанавливать состояние файла:

```
$ git restore [options] file
```

В руководстве Git сказано, что `git switch` и `git restore` — это экспериментальные команды, поэтому в будущем их поведение может измениться.

Внесение изменений в другую ветку

Продолжим тему предыдущего раздела, где текущее состояние рабочего каталога конфликтовало с состоянием ветки, на которую мы хотели переключиться. Чтобы

сохранить изменения файла в рабочем каталоге и в то же время переключиться на новую ветку, нужно параллельно с переключением выполнить слияние.

Сделать это можно, сопроводив команду `git checkout` опцией `-m`; Git попытается перенести ваше локальное изменение в новый рабочий каталог, выполнив операцию слияния между локальными изменениями и состоянием файла в целевой ветке:

```
$ git checkout -m dev
M       NewStuff
Switched to branch "dev"
```

В этом примере Git успешно изменила файл `NewStuff` и переключилась на ветку `dev`. Эта операция слияния происходит полностью в рабочем каталоге. Она не вносит коммит слияния ни в какую ветку. Причина в том, что через опцию `-m` Git выполнила трехстороннее слияние между текущей веткой, текущим состоянием рабочего каталога и целевой веткой, на которую вы переключаетесь. Это неявное слияние между двух веток.

В случае конфликта слияния при переключении веток с помощью `-m` вам нужно будет разбираться с ним вручную. Git изменит соответствующий файл и поместит в него *маркеры разрешения конфликта*. Подробнее о слияниях и полезных методиках разрешения связанных с ними конфликтов будет сказано в главе 6.

Маркеры разрешения конфликтов (<<<,<=<,>>>) в файле NewStuff как результат
конфликта при слиянии в процессе переключения веток

```
$ cat NewStuff
Something
<<<<<< dev:NewStuff
A text existing in the dev branch version
=====
Something else
>>>>>> local:NewStuff
```

Если Git сможет переключить ветку и точно перенести ваши локальные изменения без конфликтов слияния, тогда запрос на смену веток завершится успехом.

Допустим, вы находитесь в ветке `main` репозитория разработки и внесли изменения в файл `NewStuff`. При этом вы вдруг понимаете, что внесенные изменения на самом деле должны были быть применены к другой ветке, поскольку исправляют Problem Report 1 и должны быть зафиксированы в ветку `bugs/pr-1`.

Ниже показана эта конфигурация. Начинаем с ветки `main`. Вносим в некоторые файлы изменения: здесь они представлены как добавление текста `Some bug fix` в файл `NewStuff`:

```
$ git show-branch
! [bugs/pr-1] Fix Problem Report 1
```

```
! [bugs/pr-2] Added Bob's fixes.
! [dev] Started developing NewStuff
* [main] Added Bob's fixes.
----
+ [dev] Started developing NewStuff
+ [dev^] Improve the new development
+ [dev~2] Start some new development.
+ [bugs/pr-1] Fix Problem Report 1
+++* [bugs/pr-2] Added Bob's fixes.
```

```
$ echo "Some bug fix" >> NewStuff
```

```
$ cat NewStuff
Something
Some bug fix
```

Здесь вы осознаете, что всю эту работу нужно зафиксировать в ветку `bugs/pr-1`, а не в `main`. Вот как выглядит файл `NewStuff` в ветке `bugs/pr-1` до переключения в следующем шаге:

```
$ git show bug/pr-1:NewStuff
Something
```

Для переноса изменений в нужную ветку надо просто попробовать на нее переключиться:

```
$ git checkout -m bug/pr-1
M       NewStuff
Switched to branch "bug/pr-1"
```

```
$ cat NewStuff
Something
Some bug fix
```

Здесь Git смогла корректно внести изменения из рабочих каталогов и целевой ветки, оставив их в структуре нового рабочего каталога. С помощью `git diff` вы можете убедиться, что слияние произошло в соответствии с вашими ожиданиями:

```
$ git diff
diff --git a/NewStuff b/NewStuff
index 0f2416e..b4d8596 100644
--- a/NewStuff
+++ b/NewStuff
@@ -1,2 @@
Something
+Some bug fix
```

Это однострочное дополнение (`+Some bug fix`) корректно.

ПОИСК ИНФОРМАЦИИ О БАЗОВОЙ ВЕТКЕ ДЛЯ СЛИЯНИЯ ВЕТОК

При слиянии веток не утрачивается имя ни исходной, ни целевой ветки. Слияние представляет собой дополнение. При его выполнении содержимое одной или более веток объединяется с целевой веткой.

Поскольку исходный коммит, из которого началась ветка, не определен явно, этот коммит (или его эквивалент) можно найти алгоритмически, используя имя исходной ветки, из которой ответвилась новая:

```
$ git merge-base original-branch new-branch
```

Подробнее о сложном процессе слияния веток мы узнаем в главе 6.

Создание новой ветки и переключение на нее

В разделе «Создание веток» мы научились создавать ветки, а в разделе «Переключение веток» узнали, как переключаться на конкретную ветку. Иногда нужно создать новую ветку и в то же время переключиться на нее. Для этого Git предоставляет опцию `-b new-branch`:

```
$ git checkout -b new-branch
```

Для примера начнем с прежней конфигурации, только теперь вместо переключения на существующую ветку создадим новую. Мы находимся в ветке `main`, редактируем файлы и хотим, чтобы все изменения были зафиксированы в новую ветку с именем `bugs/pr-3`. Последовательность действий будет такой:

```
$ git branch
  bugs/pr-1
  bugs/pr-2
  dev
* main

$ git checkout -b bugs/pr-3
M       NewStuff
Switched to a new branch "bugs/pr-3"

$ git show-branch
! [bugs/pr-1] Fix Problem Report 1
! [bugs/pr-2] Added Bob's fixes.
* [bugs/pr-3] Added Bob's fixes.
! [dev] Started developing NewStuff
! [main] Added Bob's fixes.
-----
+ [dev] Started developing NewStuff
+ [dev^] Improve the new development
+ [dev~2] Start some new development.
```

```
+ [bugs/pr-1] Fix Problem Report 1
++*++ [bugs/pr-2] Added Bob's fixes.
```

Напомню, что команда

```
$ git checkout -b new-branch start-point
```

в точности аналогична последовательному выполнению двух следующих команд при условии, что рабочий каталог у вас пуст:

```
$ git branch new-branch start-point
$ git checkout new-branch
```

Если при создании новой ветки и одновременном переключении на нее с помощью `-b new-branch` имя этой ветки конфликтует с именем существующей, то Git не позволит завершить эту команду:

```
$ git checkout -b shiny-branch-name
fatal: A branch named 'shiny-branch-name' already exists.
```

Это предустановленное поведение можно обойти, использовав опцию `-B new-branch`. Такой метод приведет к переключению и сбросу ветки, в результате чего вы потеряете всю предыдущую работу, выполненную в ветке с тем же именем:

```
$ git checkout -B shiny-branch-name
Switched to and reset branch shiny-branch-name
```

Равнозначно последовательному выполнению двух команд
опция `-f` используется для отмены всех локальных изменений в рабочем каталоге

```
$ git branch -f branch [start-point]
$ git checkout branch
```

Отсоединенная HEAD

По умолчанию при каждом переключении на другую ветку вы переходите на ее верхний коммит. Но Git также позволяет переключаться на конкретный коммит, ссылаясь на него по тегу или путем прямой адресации, используя команду `git checkout` с SHA коммита (ID объекта) в опции `start-point`.

В подобных случаях переключения на конкретный коммит будет возникать состояние, называемое режимом отсоединенной HEAD (detached HEAD), «отсоединенной головы». Этот принцип показан на рис. 3.6. Обратите внимание на указатель HEAD и именованную ветку `main`. Они указывают на разные коммиты.

Git помещает вас в режим отсоединенной HEAD, когда вы:

- переключаетесь на коммит, не являющийся верхушкой ветки, используя прямую ссылку или через именованный тег;
- переключаетесь на ветку отслеживания. Это можно делать, чтобы изучить изменения, внесенные недавно из удаленного репозитория;

- запускаете операцию `git bisect`, как описано в разделе «Использование `git bisect`»;
- используете команду `git submodule update`.

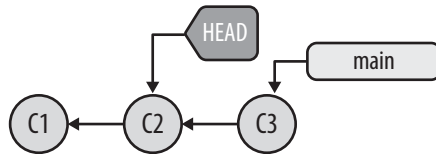


Рис. 3.6. Режим отсоединенной HEAD

Разберем режим отсоединенной HEAD на примере:

```
# Использование клонированного репозитория исходного кода Git!
$ cd git

$ git checkout v2.9.0-rc2
Note: switching to 'v2.9.0-rc2'.
```

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may do so (now or later) by using `-c` with the switch command. Example:

```
git switch -c <new-branch-name>
```

Or undo this operation with:

```
git switch -
```

Turn off this advice by setting config variable `advice.detachedHead` to `false`

HEAD is now at 49fa3dc761 Git 2.9-rc2

Как видно из вывода, находясь в режиме отсоединенной HEAD, вы можете свободно добавлять новые коммиты в виде экспериментальных изменений проекта. Это показано на рис. 3.7.

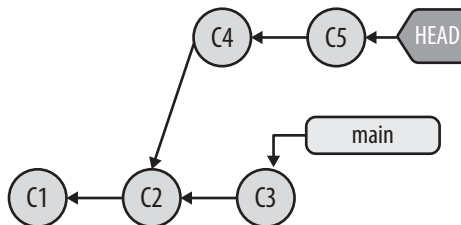


Рис. 3.7. Новые коммиты в режиме отсоединенной HEAD

Коммиты C4 и C5 на рис. 3.7 в этом контексте также называются *недостижимыми коммитами* (unreachable commits), так как при нахождении в режиме отсоединенной HEAD на них нет другой постоянной ссылки, кроме HEAD. Когда вы переключаетесь на другую именованную ветку, коммиты C4 и C5 в итоге удаляются стандартной операцией Git по сбору мусора.

Если вы решите сохранить эти коммиты, то сначала нужно будет создать новую ветку:

```
$ git checkout -b new_branch
```

Так вы получите новую ветку, основанную на том коммите, где находилась отсоединенная HEAD. Затем можете продолжить обычную разработку.



Можно создать именованный тег, указывающий на коммит C5, чтобы сохранить эти коммиты и получить возможность поработать с C5 позднее. Проще всего это делать, пока вы находитесь в режиме отсоединенной HEAD. Если вы переключитесь на другую ветку, то нужно будет восстановить коммит C5 с помощью команды `git reflog`, после чего добавить на него ссылку.

Команда `git branch` поможет определить, находитесь ли вы в режиме отсоединенной HEAD:

```
$ git branch
* (HEAD detached at 49fa3dc761)
  main
```

С другой стороны, если вы закончили работать с отсоединенной HEAD и просто хотите сбросить это состояние, то можете выполнить преобразование в именованную ветку командой `git checkout branch`:

```
$ git checkout main
Updating files: 100% (3013/3013), done.
Previous HEAD position was 49fa3dc761 Git 2.9-rc2
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

```
$ git branch
* main
```

Удаление веток

Удалить ветку можно с помощью команды `git branch -d branch name`. Это действие приведет к удалению именованной ветки из локального репозитория:

```
$ git branch -d branch
```

Если вы собираетесь также ликвидировать удаленные (то есть находящиеся не локально, а на сервере) ветки, то эту команду нужно будет дополнить опцией `-r`. Удаленные ветки отслеживания (remote-tracking branches) мы разберем в главе 11.

По умолчанию Git не позволяет удалить ветку, в которой вы работаете прямо сейчас, поскольку в этом случае система не сможет определять состояние рабочего каталога. Вместо этого всегда нужно указывать не текущую ветку.

Но есть и еще один нюанс. Git не позволяет удалять ветку, где есть коммиты, которые отсутствуют в текущей ветке, о чем говорит сообщение об ошибке в примере ниже. То есть Git не дает случайно удалить разработку, находящуюся в коммитах, которые при удалении ветки будут утрачены.

```
$ git checkout main
Switched to branch "main"

$ git branch -d bugs/pr-3
error: The branch 'bugs/pr-3' is not fully merged.
If you are sure you want to delete it, run 'git branch -D bugs/pr-3'.
```

Разберем это на примере команды `git show-branch`:

```
$ git show-branch
! [bugs/pr-1] Fix Problem Report 1
! [bugs/pr-2] Added Bob's fixes.
! [bugs/pr-3] Added a bug fix for pr-3.
! [dev] Started developing NewStuff
* [main] Added Bob's fixes..
-----
+ [bugs/pr-3] Added a bug fix for pr-3.
+ [dev] Started developing NewStuff
+ [dev^] Improve the new development
+ [dev~2] Start some new development.
+ [bugs/pr-1] Fix Problem Report 1
++++* [bugs/pr-2] Added Bob's fixes.
```

В этом выводе `git show-branch` коммит `Added a bug fix for pr-3` находится только в ветке `bugs/pr-3`. Если эту ветку удалить, получить доступ к этому коммиту будет невозможно.

Заявляя, что ветка `bugs/pr-3` *не полностью слита* (is not fully merged), Git сообщает, что линия разработки, находящаяся в ветке `bugs/pr-3`, не относится к разработке в текущей ветке, `main`.

Но это просто дружеское напоминание. Git не обязывает вас предварительно совмещать (сливать) удаляемую ветку с веткой `main`. Имейте в виду: ветка — это просто имя или указатель на коммит, в котором находится фактическое содержимое. Git предостерегает вас от случайной утраты содержимого в удаляемой ветке, которая не была совмещена с *текущей*.

Если содержимое удаляемой ветки уже есть в другой ветке, переключение на эту другую ветку и последующий запрос удаления из данного контекста сработает. Другой подход — объединить содержимое ветки, которую вы хотите удалить, с текущей веткой. Затем можно безопасно удалить первую ветку:

```
$ git merge bugs/pr-3
Updating 7933438..401b78d
Fast forward
 NewStuff | 1 +
 1 files changed, 1 insertions(+), 0 deletions(-)
```

```
$ git show-branch
! [bugs/pr-1] Fix Problem Report 1
! [bugs/pr-2] Added Bob's fixes.
! [bugs/pr-3] Added a bug fix for pr-3.
! [dev] Started developing NewStuff
* [main] Added a bug fix for pr-3.
-----
+ * [bugs/pr-3] Added a bug fix for pr-3.
+ [dev] Started developing NewStuff
+ [dev^] Improve the new development
+ [dev~2] Start some new development.
+ [bugs/pr-1] Fix Problem Report 1
+++* [bugs/pr-2] Added Bob's fixes.
```

```
$ git branch -d bugs/pr-3
Deleted branch bugs/pr-3.
```

```
$ git show-branch
! [bugs/pr-1] Fix Problem Report 1
! [bugs/pr-2] Added Bob's fixes.
! [dev] Started developing NewStuff
* [main] Added a bug fix for pr-3.
----
* [main] Added a bug fix for pr-3.
+ [dev] Started developing NewStuff
+ [dev^] Improve the new development
+ [dev~2] Start some new development.
+ [bugs/pr-1] Fix Problem Report 1
+++* [bugs/pr-2] Added Bob's fixes.
```

Наконец, как предлагает сообщение об ошибке, вы можете переопределить проверку Git, используя `-D` вместо `-d`. Сделайте это, если уверены, что вам не понадобится содержимое удаляемой ветки (If you are sure you want to delete it, run 'git branch -D bugs/pr-3').

Git не поддерживает какую-либо историю *имен* создаваемых, перемещаемых, изменяемых, совмещаемых или удаляемых веток. После удаления имени ветки оно исчезает навсегда.

А вот история коммитов удаляемой ветки — другое дело. Git удаляет коммиты, на которые больше не ссылаются никакие именованные ссылки: имя ветки или тега. Если вы хотите сохранить эти коммиты, то должны совместить их с другой веткой, создать для них отдельную ветку либо направить на них ссылочный тег. В противном случае, не имея ссылок, коммиты и BLOB-объекты оказываются недостижимыми и в конце концов удаляются инструментом `git gc`.



Если вы случайно удалите ветку или другую ссылку, то можете восстановить ее с помощью команды `git reflog`. Другие команды вроде `git fsck` и опции конфигурации — `gc.reflogExpire` и `gc.pruneExpire` — тоже позволяют восстановить утраченные коммиты, файлы и вершины веток.

Итоги

Глава началась с объяснения причин использования веток в Git. Затем мы разобрали некоторые рекомендуемые руководства по работе с ветками. В разделе «Управление ветками» познакомились со сценариями повседневной работы с ветками в Git. В частности, этот раздел осветил технические особенности управления ветками репозитория Git и работы с ними.

При работе с ветками главная часть состоит в планировании линии разработки. Вы следуете соглашениям об именовании и утверждаете процесс систематического внесения изменений в основную стабильную ветку, независимо от того, нацелены эти изменения на исправление багов или же внедрение в проект новой функциональности. После этого вы можете применять рассмотренные здесь техники для эффективной работы с несколькими наборами веток в своих репозиториях.

ГЛАВА 4

Коммиты

Коммит — это снимок, отражающий текущее состояние репозитория в определенный момент времени. Снимки коммитов связываются в цепочки, в которых каждый новый указывает на предшествующий. С течением времени последовательность изменений образует серию коммитов.

Git использует коммит как средство для записи изменений в репозитории. На первый взгляд коммит в Git похож на операцию check-in или фиксацию в других системах контроля версий. Но эти сходства поверхностны. Под капотом внутренние механизмы того, как Git создает и управляет коммитами, полностью уникальны.

Во время коммита Git делает снимок текущего состояния каталога индексирования и сохраняет его в хранилище объектов, о чем кратко говорилось в главе 2. Этот снимок не содержит копию каждого файла и каталога в индексе. Вместо этого при создании нового коммита Git сравнивает текущее состояние индекса с предыдущим снимком коммита, определяя список измененных файлов и каталогов. Исходя из этого списка, Git создает новые BLOB-объекты для всех изменившихся файлов и новые объекты деревьев для всех изменившихся каталогов. При этом все неизменившиеся BLOB-объекты и объекты деревьев система использует повторно.

Подготовку каталога индексирования к коммиту мы разберем в главе 5. В этой главе поговорим о том, что происходит при совершении коммита. Сначала изучим процесс внесения коммитов и поймем важность атомарных наборов изменений. Затем узнаем, как идентифицировать коммиты и просматривать их истории.

Git отлично подходит для частых коммитов и предоставляет богатый набор команд для управления ими. Мы покажем, как несколько коммитов, каждый из которых содержит небольшие, четко определенные изменения, могут улучшить организацию изменений и упростить работу с наборами исправлений.

Коммиты: зафиксированные единицы изменений

Коммит — единственный метод внесения изменений в репозиторий. Он обеспечивает возможность аудита и отчетности. Ни при каких обстоятельствах данные репозитория не должны изменяться без регистрации изменений через коммит.

Представьте, какой возникнет хаос, если содержимое репозитория изменится, а записи о том, как это произошло, кто это сделал и почему, не будет.

Коммиты явно вносятся разработчиками. Это самый типичный сценарий. Но бывают случаи, когда коммит может внести сама система Git. Например, операция слияния ведет к созданию в репозитории нового коммита в дополнение к другим, сделанным разработчиками ранее. Подробнее об этом вы узнаете в главе 6.

Частота создания коммитов зависит от вас. Желательно делать это тогда, когда ваша разработка находится в «состоянии покоя»: на момент создания коммита все процессы разработки временно приостановлены и проект находится в стабильном и работоспособном состоянии. При этом разработчик может быть уверен, что его изменения не приведут к нарушениям в работе приложения и все тесты успешно проходят.

Вы можете подумать, что для сравнения всего индекса с неким прежним состоянием требуется много времени, но на самом деле этот процесс происходит на удивление быстро. Как говорилось в главе 2, дело в том, что каждый объект Git имеет хеш SHA-1, и если два объекта, даже два поддерева, имеют одинаковый хеш, то при сравнении они будут считаться одинаковыми. Это позволяет Git избегать значительной части рекурсивных сравнений за счет исключения поддеревьев, имеющих одинаковое содержимое.

Однако это не должно означать, что нужно избегать регулярного внесения коммитов.

Атомарные наборы изменений

Каждый коммит в Git представляет собой один атомарный набор изменений (<https://oreil.ly/r1ThS>) по отношению к предыдущему состоянию. Вне зависимости от количества каталогов, файлов, строк или байтов, изменяющихся с этим коммитом¹, применяются либо все изменения, либо не применяется никаких.

Смысл такой атомарности становится яснее, если взглянуть на нее с точки зрения лежащей в основе Git объектной модели. Снимок коммита представляет состояние всего набора измененных файлов и каталогов. Это также означает, что он представляет полное преобразование от одного состояния дерева (структуры файлов и каталогов) к другому. Вы можете переключаться полностью только из одного состояния в другое, пошаговое переключение невозможно. В главе 7 мы обсудим, как находить различия между коммитами.

Это важный принцип, который разработчики должны учитывать. Рассмотрим рабочий поток перемещения функции из одного файла в другой. Если вы

¹ Git дополнительно записывает флаг режима, указывающий на исполняемость каждого файла. Изменения этого флага также являются частью набора изменений.

удалите эту функцию из первого файла с помощью одного коммита, а затем добавьте ее во второй файл с помощью другого, то в истории репозитория останется небольшой «семантический разрыв», отражающий время, в течение которого функция отсутствовала в обоих файлах. Выполнение двух коммитов в обратном порядке тоже проблематично. В каждом случае перед первым коммитом и после второго код семантически согласован, но после первого эта согласованность утрачивается.

Если же мы говорим об атомарном коммите, который одновременно удаляет функцию из первого файла и добавляет ее во второй, то здесь в истории не возникнет семантического разрыва. Понимание этого принципа позволит структурировать коммиты более правильно.

Git все равно, *почему* изменяются файлы, — содержание изменений системе неважно. Вы можете переместить функцию из одного файла в другой и ожидать, что это будет обработано как единое действие. Либо вы можете зафиксировать удаление, а позднее зафиксировать добавление. Для Git это не имеет значения, это никак не связано с семантикой содержимого файлов.

Идентификация коммитов

На каждый коммит в системе Git может вести явная или неявная ссылка. Возможность идентифицировать отдельные коммиты — важная задача для процесса разработки. Например, чтобы создать ветку, нужно выбрать коммит, от которого пойдет ответвление; чтобы сравнить изменения в коде, необходимо указать два коммита; а для редактирования истории коммитов нужно предоставить их коллекцию.

Когда нужно сослаться на коммит явно, то это делается с помощью *абсолютного имени коммита*. Когда это происходит неявно, используется *ссылка* (ref), *символическая ссылка* (symref) или *относительное имя коммита*.

Вы уже видели примеры явных и неявных ссылок на коммиты в примерах из глав 1–3. Уникальный шестнадцатеричный ID коммита — явная ссылка, в то время как HEAD, которая всегда указывает на последний коммит в ветке, — неявная ссылка (табл. 4.1).

Таблица 4.1. Различия между явными и неявными коммитами

	Явный	Неявный
Идентифицируется по	абсолютному имени коммита	ссылкам, символическим ссылкам, относительным именам коммитов
Пример	34043c95636aee319d606a7a380697cae4f1bfcc	HEAD, HEAD^2, etc.

Когда вы обсуждаете конкретный коммит с коллегой, работающим над теми же данными, но в распределенной среде, лучше использовать имя коммита, которое однозначно будет одинаковым в обоих репозиториях. Но если вы работаете внутри собственного репозитория и хотите обратиться к состоянию ветки несколько коммитов назад, то здесь подойдет простое относительное имя.

Git предоставляет множество механизмов для именования коммитов, каждый из которых имеет свои преимущества и оказывается более подходящим в определенной ситуации.

Абсолютные имена коммитов

Наиболее точным именем для коммита выступает ID его объекта, то есть хеш SHA-1. Этот ID — *абсолютное* имя, то есть он может относиться только к конкретному коммиту. Неважно, где этот коммит находится в истории репозитория. Хеш SHA-1 всегда указывает на один и тот же коммит, идентифицируя его.

Каждый ID коммита уникален *глобально*, то есть не только для одного репозитория, а для всех. Если вы сравните ссылку с конкретным ID коммита в вашем репозитории с репозиторием другого разработчика и найдете тот же ID, то будьте уверены, что у вас обоих есть один и тот же коммит с одинаковым содержимым.

Поскольку данные, определяющие ID коммита, содержат состояние всего дерева репозитория, а также состояние прежнего коммита, вы также можете быть уверены, что оба ссылаются на одну и ту же полную линию разработки, ведущую к этому коммиту и включающую его.

Поскольку 40-цифренное шестнадцатеричное число SHA-1 представляет собой громоздкую и уязвимую для ошибок запись, Git позволяет сократить его до уникального префикса внутри объектной базы данных репозитория. Рассмотрим пример из собственного репозитория Git:

```
$ git log -1 --pretty=oneline HEAD
30cc8d0f147546d4dd77bf497f4dec51e7265bd8 ... A regression fix for 2.37
```

```
$ git log -1 --pretty=oneline 30c
fatal: ambiguous argument '30c': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
```

```
$ git log -1 --pretty=oneline 30cc8d
a5828ae6b52137b913b978e16cd2334482eb4c1f ... A regression fix for 2.37
```



Несмотря на то что имя тега не является глобально уникальным, оно абсолютно в том смысле, что указывает на уникальный коммит и не изменяется со временем (только в случае явного изменения).

Ссылки и символические ссылки

Ссылка (ref) указывает на хеш SHA-1 внутри объектного хранилища Git. Теоретически простая ссылка может *указывать* (point) на любой объект Git, но обычно она *ссылается* (refers) на объект коммита. *Символическая ссылка*, или *symref*, — это имя, которое указывает на объект Git косвенно. Можно представить ее себе как ярлык, указывающий на фактический объект Git. Но это все же ссылка.

Каждая символическая ссылка имеет конкретное полное имя, начинающееся с `refs/`, и хранится иерархически внутри репозитория в каталоге `.git/refs/`. В `refs/` представлены три разных пространства имен:

- `refs/heads/ref` для локальных веток;
- `refs/remotes/ref` для удаленных веток отслеживания;
- `refs/tags/ref` для тегов.

Примеры ссылок — имена локальных веток, удаленных веток отслеживания и тегов. Имя локальной ветки функциональности `dev`, по сути, представляет краткую форму `refs/heads/dev`. При этом ветки удаленного отслеживания находятся в пространстве имен `refs/remotes/`, поэтому `origin/main` — это краткая форма `refs/remotes/origin/main`. А тег, например `v1.8.17`, — краткая форма `refs/tags/v1.8.17`.

При поиске ссылки или обращении к ней вы можете использовать полноценное имя (`refs/heads/main`) или его аббревиатуру (`main`). Когда вы ищете ветку и уже есть тег с таким же именем, Git применяет эвристику разрешения неоднозначности и использует первое совпадение, согласно приведенному ниже списку со страницы руководства `git rev-parse`:

```
.git/ref
.git/refs/ref
.git/refs/tags/ref
.git/refs/heads/ref
.git/refs/remotes/ref
.git/refs/remotes/ref/HEAD
```

Внутри Git обычно используется правило первого совпадения (`.git/ref`). Символические ссылки бывают следующими: `HEAD`, `ORIG_HEAD`, `FETCH_HEAD`, `CHERRY_PICK_HEAD` и `MERGE_HEAD`.



Теоретически имя каталога `.git` можно изменить. Поэтому в документации Git вместо литерала `.git` используется переменная `$GIT_DIR`.

Под капотом Git автоматически поддерживает следующие символические ссылки:

HEAD

HEAD всегда относится к последнему коммиту текущей ветки. Когда вы переключаете ветки, Git автоматически обновляет HEAD, чтобы она указывала на последний коммит новой ветки.

ORIG_HEAD

Определенные операции, например, слияние и сброс, записывают предыдущую версию HEAD в ORIG_HEAD перед тем, как корректировать ее на новое значение. Вы можете использовать ORIG_HEAD для восстановления предыдущего состояния, отката к предыдущему или для сравнения.

FETCH_HEAD

Когда используются удаленные репозитории, `git fetch` записывает вершины всех веток, указанных в файле `.git/FETCH_HEAD`. FETCH_HEAD — это сокращение для вершины последней извлекаемой ветки, оно валидно только непосредственно после операции извлечения. С помощью символической ссылки вы можете найти HEAD коммитов из `git fetch`, даже если используется анонимное извлечение, которое не называет конкретного имени ветки. Операция `fetch` будет подробно разобрана в главе 11.

MERGE_HEAD

Когда происходит слияние, вершина *другой* ветки временно записывается в символическую ссылку MERGE_HEAD. Иными словами, MERGE_HEAD — это коммит, который соединяется с HEAD.

CHERRY_PICK_HEAD

При использовании команды `git cherry-pick` символическая ссылка CHERRY_PICK_HEAD записывает коммиты, выбранные для предполагаемой операции. Подробно команда `git cherry-pick` описана в главе 8.

Все эти символические ссылки управляются низкоуровневой командой `git symbolic-ref`.



Хотя можно создать собственную ветку с одним из этих специальных символических имен, делать так не рекомендуется. В последних версиях Git реализованы защитные механизмы, не позволяющие использовать определенные символические имена (например, HEAD).

Для создания ссылочных имен есть множество специальных символов. Два наиболее распространенных — каретка (^) и тильда (~) — будут описаны в следующем разделе. Еще один вариант — двоеточие, которое можно использовать для ссылки на альтернативные версии общего файла, вовлеченного в конфликт слияния. Эта процедура описана в главе 6.

Относительные имена коммитов

Помимо того что Git использует абсолютные имена коммитов, ссылок и символических ссылок, система также предоставляет механизмы для идентификации коммита относительно другой ссылки, обычно вершины ветки. Это полезно, когда вы работаете в локальном репозитории и хотите быстро сослаться на изменения в прошлых коммитах.

Некоторые из этих имен вы уже видели, например `main` и `main^`, где `main^` всегда относится к предпоследнему коммиту ветки `main`. Есть и другие: вы можете использовать `main^^`, `main~2` и даже сложные имена вроде `main~10^2~2^2`.

За исключением первого, или корневого, коммита¹, каждый следующий происходит из как минимум одного раннего, а иногда и нескольких, где прямые предки называются *родительскими коммитами* (parent commits). Чтобы у коммита было несколько родительских коммитов, он должен быть результатом операции слияния. В результате для каждой ветви, участвующей в операции слияния, будет существовать родительский коммит.

В рамках одного поколения символ каретки используется для выбора другого родителя. При наличии коммита `C` `C^1` будет его первым родителем, `C^2` — вторым, а `C^n` — n -м родителем (рис. 4.1).

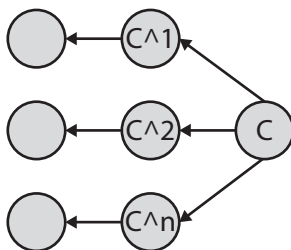


Рис. 4.1. Несколько имен родителей

Знак «тильда» используется для возвращения к точке, предшествовавшей родителю, и выбора предыдущего поколения. Для коммита `C` коммит `C~1` будет первым родителем, `C~2` — первым прародителем, а `C~3` — первым прапрародителем. Когда в одном поколении есть несколько родителей, цепочка продолжается за первым родителем. Вы также можете заметить, что `C^1` и `C~1` относятся к первому родителю. Оба этих имени будут корректны (рис. 4.2).

Git также поддерживает другие сокращения и комбинации. Сокращенные формы `C^` и `C~` равнозначны `C^1` и `C~1` соответственно. Кроме того, `C^^` будет равнозначно

¹ Да, вы и правда можете внести несколько корневых коммитов в один репозиторий. Это происходит, например, когда два проекта и их репозитории соединяются в один.

C^{1^1} и, поскольку это означает «первого родителя первого родителя коммита C », оно относится к тому же коммиту, что и C^2 .

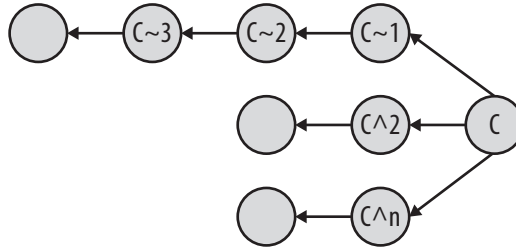


Рис. 4.2. Несколько имен родителей с их предками

Помните, что при операции слияния сокращенное выражение вроде $C^{\wedge}$ или $C^{\wedge\wedge}$ может не вернуть ожидаемого результата, как произошло бы при работе в ветке с линейной историей коммитов. Рисунок 4.3 показывает, что $C^{\wedge\wedge}$ неравнозначен C^2 .

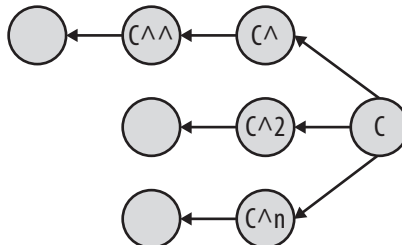


Рис. 4.3. $C^{\wedge\wedge}$ и C^2

Совмещая **ref** с символами «каретка» и «тильда», можно выбирать из графа коммитов **ref** произвольные коммиты. Но помните, что эти имена относительны для *текущего* значения **ref**. Если поверх **ref** произойдет новый коммит, граф будет дополнен новым поколением и каждое имя «родителя» сместится в истории и графе назад.

Вот пример из собственной истории Git, где ее ветка **main** находилась у коммита `a5828ae6b52137b913b978e16cd2334482eb4c1f`. Используя команду:

```
git show-branch --more=25
```

и ограничив вывод последними 25 строками, вы сможете изучить историю графа и оценить сложную структуру слияния веток:

```
$ git reset --hard a5828ae6b52137b913b978e16cd2334482eb4c1f
HEAD is now at a5828ae6b5 Git 2.31
```

```
$ git show-branch --more=25
[main] Git 2.31
[main^] Merge branch 'jn/mergetool-hideresolved-is-optional'
[main^^2] doc: describe mergetool configuration in git-mergetool(1)
[main^^2^] mergetool: do not enable hideResolved by default
[main^2~2] mergetool: add per-tool support and overrides for the hideResolved
flag
[main~2] Merge branch 'tb/pack-revindex-on-disk'
[main~2^2] pack-revindex.c: don't close unopened file descriptors
[main~3] Merge tag 'l10n-2.31.0-rnd2' of git://github.com/git-l10n/git-po
[main~3^2] l10n: zh_CN: for git v2.31.0 l10n round 1 and 2
[main~3^2^] Merge branch 'master' of github.com:vnwildman/git
[main~3^2^^2] l10n: vi.po(5104t): for git v2.31.0 l10n round 2
[main~3^2~2] Merge branch 'l10n/zh_TW/210301' of github.com:l10n-tw/git-po
[main~3^2~2^2] l10n: zh_TW.po: v2.31.0 round 2 (15 untranslated)
[main~3^2~3] Merge branch 'po-id' of github.com:bagasme/git-po
[main~3^2~3^2] l10n: Add translation team info
[main~3^2~4] Merge branch 'master' of github.com:Softcatala/git-po
[main~3^2~4^2] l10n: Update Catalan translation
[main~3^2~5] Merge branch 'russian-l10n' of github.com:DJM00n/git-po-ru
[main~3^2~5^2] l10n: ru.po: update Russian translation
[main~3^2~6] Merge branch 'pt-PT' of github.com:git-l10n-pt-PT/git-po
[main~3^2~6^2] l10n: pt_PT: add Portuguese translations part 1
[main~3^2~7] l10n: de.po: Update German translation for Git v2.31.0
[main~4] Git 2.31-rc2
[main~5] Sync with Git 2.30.2 for CVE-2021-21300
[main~5^2] Git 2.30.2
[main~6] Merge branch 'jt/transfer-fsck-across-packs-fix'
```

```
$ git rev-parse main~3^2~2^
8278f870221711c2116d3da2a0165ab00368f756
```

Между `main~3` и `main~4` произошло слияние, которое внесло несколько других слияний, а также простой коммит с именем `main~3^2~2^2`. Это коммит `8278f870221711c2116d3da2a0165ab00368f756`.

Команда `git rev-parse` — последняя инстанция при переводе любой формы имени коммита — тега, относительного, сокращенного или абсолютного — в фактический, абсолютный ID в объектной базе данных.

История коммитов

Основной командой для вывода истории коммитов является `git log`. Она содержит больше параметров, средств для раскраски кода, селекторов, форматировщиков и прочих возможностей, чем прославленная `ls`. Но не волнуйтесь.

Как и в случае `ls`, не нужно погружаться во все детали сразу. Далее мы перейдем к изучению нюансов истории коммитов репозитория.

Просмотр старых коммитов

При выполнении команды `git log` вывод будет включать все связанные коммиты и их сообщения журнала, находящиеся в истории коммитов, которая достижима из указанной стартовой точки.

Если не включить в команду `git log` дополнительные опции, то она будет равнозначна выполнению `git log HEAD`. Результатом станет вывод всех коммитов, начиная с коммита `HEAD`, и всех достижимых коммитов в обратном направлении вдоль графа коммитов (графы коммитов разберем в следующем разделе). По умолчанию результаты выводятся в обратном хронологическом порядке, но нужно помнить, что когда Git перебирает историю коммитов, то обратный порядок выстраивается в соответствии с графом коммитов, а не со временем получения снимка.

Для просмотра истории ветки рекомендуется явно указывать стартовую точку вывода с помощью команды `git log commit`. Используем эту команду для просмотра вывода из репозитория самой Git:

```
$ git log main -2
commit 30cc8d0f147546d4dd77bf497f4dec51e7265bd8 (HEAD -> main, ...)
Author: Junio C Hamano <gitster@pobox.com>
Date: Sat Jul 2 17:01:34 2022 -0700

    A regression fix for 2.37

Signed-off-by: Junio C Hamano <gitster@pobox.com>

commit 0f0bc2124b25476504e7215dc2af92d5748ad327
Merge: e4a4b31577 4788e8b256
Author: Junio C Hamano <gitster@pobox.com>
Date: Sat Jul 2 21:56:08 2022 -0700

    Merge branch 'js/add-i-delete'

    Rewrite of "git add -i" in C that appeared in Git 2.25 didn't
    correctly record a removed file to the index, which was fixed.

* js/add-i-delete:
    add --interactive: allow `update` to stage deleted files
```

Информация журнала — источник истины, заслуживающий доверия. Но пролистывание всей истории коммитов большого репозитория, пожалуй, не слишком практично. Более информативным и удобным в работе будет обращение к ограниченному диапазону истории.

Один из способов ограничения выводимой истории — указать диапазон коммитов. Эту технику мы разберем в следующей главе. Для ограничения диапазона истории используется форма *от..до*. При указании диапазона `git log` выводит все коммиты, следующие за *от* и заканчивающиеся *до*. Также в качестве стартовой точки можно указывать количество, например `git log -3`.

Пример:

```
$ git log --pretty=short --abbrev-commit main~9..main~7
```

```
commit be7935ed8b
Author: Junio C Hamano <gitster@pobox.com>
```

```
    Merged the open-eintr workaround for macOS
```

```
commit 58d581c344
Author: Elijah Newren <newren@gmail.com>
```

```
    Documentation/RelNotes: improve release note for rename detection work
```

Здесь `git log` показывает коммиты между `main~9` и `main~7`, то есть седьмой и восьмой предыдущие коммиты в основной ветке. О диапазонах подробнее поговорим в разделе «Диапазоны коммитов».

В этом примере мы ввели две опции форматирования, `--pretty=short` и `--abbrev-commit`. Первая корректирует количество информации о каждом коммите и имеет несколько вариаций, включая `oneline`, `short`, `medium` и `full`. Вторая же просто запрашивает сокращение хешей SHA-1.

Вы также можете использовать опцию `format:string`, чтобы указать, как должна настраиваться и отображаться информация журнала.

Пример:

```
$ git log --pretty=format:"%an was the author of commit %h, %ar with%nthe commit
titled: [%s]%" \
> --abbrev-commit main~9..main~7
```

```
Junio C Hamano was the author of commit be7935ed8b, 12 days ago with
the commit titled: [Merged the open-eintr workaround for macOS]
```

```
Elijah Newren was the author of commit 58d581c344, 12 days ago with
the commit titled: [Documentation/RelNotes: improve release note for rename
detection work]
```

Команду `git log` можно дополнить опцией `-n`, чтобы ограничить вывод до максимум `n` коммитов. Кроме того, добавление опции `-p` приведет к выводу *патча*, или изменений, внесенных коммитом, одновременно ограничив наборы результатов. Это поможет вам получить более подробную информацию о коммите, предоставив больше контекста.

Пример:

```
$ git log -1 -p 4fe86488
commit 4fe86488e1a550aa058c081c7e67644dd0f7c98e
Author: Jon Loeliger <jdl@freescale.com>
Date:   Wed Apr 23 16:14:30 2008 -0500

    Add otherwise missing --strict option to unpack-objects summary.

    Signed-off-by: Jon Loeliger <jdl@freescale.com>
    Signed-off-by: Junio C Hamano <gitster@pobox.com>
diff --git a/Documentation/git-unpack-objects.txt b/Documentation
        /git-unpack-objects.txt
index 3697896..50947c5 100644
--- a/Documentation/git-unpack-objects.txt
+++ b/Documentation/git-unpack-objects.txt
@@ -8,7 +8,7 @@ git-unpack-objects - Unpack objects from a packed archive

SYNOPSIS
-----
-'git-unpack-objects' [-n] [-q] [-r] <pack-file>
+'git-unpack-objects' [-n] [-q] [-r] [--strict] <pack-file>
```

Если вы захотите узнать, какие файлы изменились в коммите вместе с подсчетом количества измененных в каждом файле строк, то используйте опцию `--stat`.

Пример:

```
$ git log --pretty=short --stat main~9..main~7

commit be7935ed8bfff19f481b033d0d242c5d5f239ed50
Author: Junio C Hamano <gitster@pobox.com>

    Merged the open-eintr workaround for macOS

Documentation/RelNotes/2.31.0.txt | 5 +++++
1 file changed, 5 insertions(+)

commit 58d581c3446cb616b216307d6b47539bccd494cf
Author: Elijah Newren <newren@gmail.com>

    Documentation/RelNotes: improve release note for rename detection work

Documentation/RelNotes/2.31.0.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```



`git show` — это еще одна команда для вывода объектов из объектного хранилища Git. Ее же можно использовать для просмотра коммита:

```
$ git show HEAD~2
```

Вывод покажет сообщение коммита и текстовые различия в файлах, находящихся в этих коммитах.

Графы коммитов

До сих пор мы использовали команду `git log` с опциями, которые выводят результаты в линейном формате. И хотя это полезно для понимания истории коммитов, ведущей к определенной точке временной линии реализации проекта, в таком упрощенном виде не всегда очевидно, что два последовательных коммита могут принадлежать к одной ветке.

Опция `--graph`, используемая совместно с командой `git log`, выводит текстовое представление истории коммитов репозитория. В этом представлении вы можете визуализировать форки из коммита и точку, в которой ветки соединяются на временной линии репозитория.

Ниже показана упрощенная история ранних коммитов исходного кода Git:

```
$ git log 89d21f4b649..0a02ce72d9 --oneline --graph
* 0a02ce72d9 Clean up the Makefile a bit.
* 839a7a06f3 Add the simple scripts I used to do a merge with content conflicts.
* b51ad43140 Merge the new object model thing from Daniel Barkalow
|\
| * b5039db6d2 [PATCH] Switch implementations of merge-base, port to parsing
| * ff5ebe39b0 [PATCH] Port fsck-cache to use parsing functions
| * 5873b67eef [PATCH] Port rev-tree to parsing functions
| * 175785e5ff [PATCH] Implementations of parsing functions
| * 6eb8ae00d4 [PATCH] Header files for object parsing
* | a4b7dbef4e [PATCH] fix bug in read-cache.c which loses files when merging...
* | 1bc992acac [PATCH] Fix confusing behaviour of update-cache --refresh on...
* | 6ad6d3d36c Update README to reflect the hierarchical tree objects...
* | 64982f7510 [PATCH] (resend) show-diff.c off-by-one fix
* | 75118b13bc Pass a "merge-cache" helper program to execute a merge on...
* | 74b2428f55 [PATCH] fork optional branch point normazilation
* | d9f98eebcd Ignore any unmerged entries for "checkout-cache -a".
* | 5e5128ed1c Remove extraneous ', ' ';' and '.' characters from...
* | 08ca0b04ba Make the revision tracking track the object types too.
* | d0d7cbe730 Make "commit-tree" check the input objects more carefully.
* | 7d60ad7cc9 Make "parse_commit" return the "struct revision" for the commit.
|/
* 6683463ed6 Do a very simple "merge-base" that finds the most recent...
* 15000d7899 Make "rev-tree.c" use the new-and-improved "mark_reachable()"
* 01796b0e91 Make "revision.h" slightly better to use.
```



Опция `--oneline` — это сокращение для опций `--pretty=oneline --abbrev-commit`, используемых совместно.

История коммитов репозитория обычно визуализируется в виде графа¹: так, ее можно использовать для справки при обсуждении определенных команд Git.

¹ *Граф* — это коллекция узлов и набор ребер между ними. Обычно при объяснении истории коммитов Git используется направленный ациклический граф (DAG).

Чаще всего она связана с операциями, которые могут изменять историю коммитов репозитория. Переписывание истории коммитов обсудим в главе 8.

В разделе «Визуализация хранилища Git» на с. 54 в главе 2 был приведен рисунок со схемой, показывающей связь между объектами, хранящимися в объектном хранилище Git. Повторим его здесь (рис. 4.4).

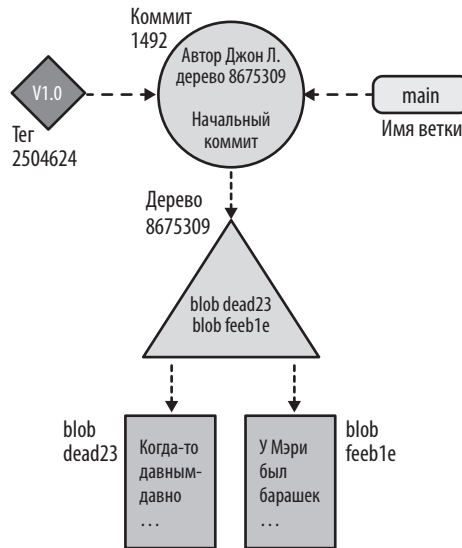


Рис. 4.4. Объекты Git

Если бы нам понадобилось изобразить историю коммитов с помощью рис. 4.4, то даже для небольшого репозитория с несколькими коммитами, слияниями и патчами этот рисунок стал бы огромным, когда мы захотели бы привести все детали. На рис. 4.5 показан более полный, но при этом упрощенный граф коммитов в том же формате. Представьте, как он будет выглядеть, если отобразить все коммиты и все структуры данных.

На рис. 4.6 показан тот же граф коммитов, что и на рис. 4.5, но без BLOB-объектов и объектов деревьев. В графах коммитов в качестве дополнительной справки также показываются имена веток.

Можно еще больше упростить рис. 4.5, если обратить внимание на одну важную деталь: каждый коммит вносит объект дерева, который ссылается на один или более BLOB-объектов и представляет все состояние репозитория на момент снимка. Следовательно, коммит можно изобразить просто как имя, значительно упростив схему истории коммитов в репозитории.

Рисунки 4.5 и 4.6 — это примеры направленного ациклического графа (DAG), который обладает важными свойствами:

- Все его ребра направлены от одного узла к другому.
- Если начать от любого узла графа, то нет пути вдоль направленных ребер, который вел бы обратно к этому начальному узлу.

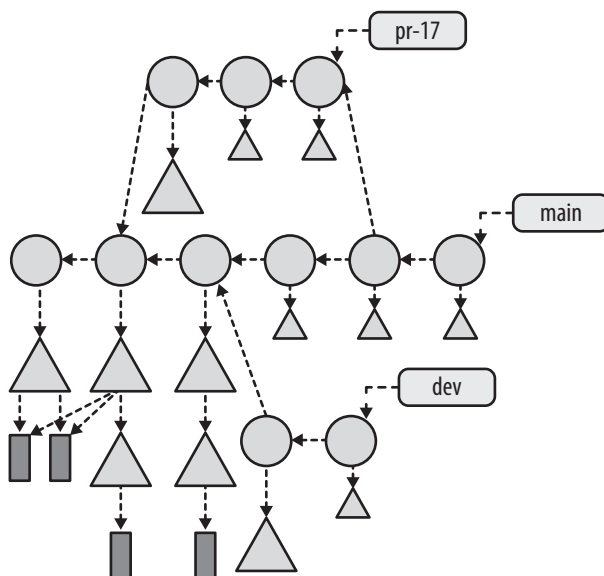


Рис. 4.5. Полный граф коммитов

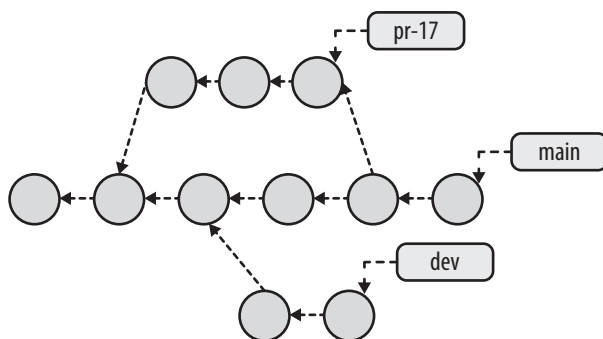


Рис. 4.6. Упрощенный граф коммитов

Git реализует историю коммитов в репозитории в виде DAG. В графе коммитов каждый узел — это один коммит, и все ребра направлены от узла-потомка к узлу-родителю, формируя наследственную связь. Отдельные узлы коммитов обычно обозначаются так, как показано на рис. 4.7, и используются для описания истории коммитов и связи между ними.

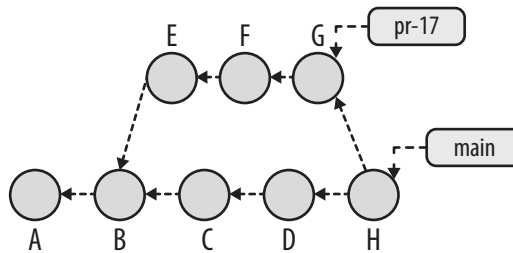


Рис. 4.7. Размеченный граф коммитов

Важной особенностью DAG является то, что для Git не имеет значения время создания коммитов либо их тайминг (абсолютный или относительный). Фактическая временная метка коммита может вести в заблуждение, поскольку часы компьютера могут быть настроены некорректно. Внутри распределенной среды разработки эта проблема обостряется еще больше. Точным же остается то, что если коммит Y указывает на родителя X, то X отражает состояние репозитория до состояния коммита Y, вне зависимости от того, какие в коммитах могут быть временные метки.

Исходя из этого, вот что можно увидеть на рис. 4.7.

- Время идет слева направо, но не обязательно точно.
- А — корневой коммит, поскольку не имеет родителя, а В произошел после А.
- И Е, и С были сделаны после В, но ничего нельзя утверждать об относительном тайминге между С и Е; любой из них мог быть сделан первым.
- Коммиты Е и С имеют общего родителя, В. Значит, В — это исходная точка ветки.
- Ветка `main` начинается с коммитов А, В, С и D.
- Последовательность коммитов А, В, Е, F и G формирует ветку `pr-17`, которая указывает на коммит G, как и объяснялось в главе 3.
- Коммит H — это коммит слияния, где ветка `pr-17` была соединена с веткой `main`. Об операции слияния подробно поговорим в главе 6.
- Поскольку это слияние, H содержит более одного родительского коммита — в данном случае D и G.
- После фиксации этого коммита `main` будет обновлена и начнет ссылаться на новый коммит H, но `pr-17` продолжит ссылаться на G.

Со временем, изучив множество направленных ациклических графов коммитов, вы заметите повторяющийся паттерн:

- Типичные коммиты имеют всего одного родителя, являющегося предыдущим коммитом в истории. Внесенное изменение — это отличие между новым коммитом и его родителем.

- Обычно есть всего один коммит без родителей — корневой.
- Коммит слияния имеет более одного родителя.
- Коммит, у которого более одного *потомка*, является точкой, где история начала расходиться и возникла новая ветка.

На практике нюансы промежуточных коммитов считаются неважными. Детали реализации коммита, указывающего на его родителя, зачастую опускаются (рис. 4.8).

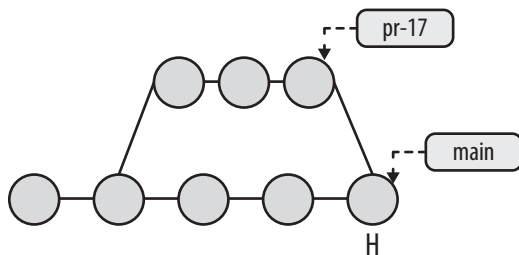


Рис. 4.8. Граф коммитов без стрелок

Время по-прежнему идет слева направо. Здесь показаны две ветки, в которых есть один обозначенный коммит слияния (H), но сами направленные ребра упрощены, поскольку их направление и так известно.

Графы коммитов — это очень абстрактное представление фактической истории коммитов, в противоположность инструментам, которые дают конкретные представления графов истории. С помощью таких инструментов время обычно выражается снизу вверх, от самого старого до самого свежего коммита. В принципе это то же самое. В следующем разделе мы кратко разберем свободно распространяемый инструмент `gitk`¹. Помимо него, есть много других платных и бесплатных инструментов.

Использование `gitk` для просмотра графа коммитов

Граф коммитов помогает визуализировать сложную структуру и связи. Команда `gitk` может вывести DAG репозитория, отражающий историю коммитов.

Разберем пример простого репозитория с двумя ветками и простыми коммитами для добавления файлов:

```
$ mkdir commit-graph-repo
# Операции добавления новых файлов, создания новой ветки и слияния ветки
...
...
$ gitk
```

¹ Команда `gitk` не является подкомандой `git`; это самостоятельная независимая команда и устанавливаемый пакет.

Программа `gitk` может очень многое, но пока мы сосредоточимся на построении DAG. На выходе получится примерно такой граф, как показан на рис. 4.9¹.

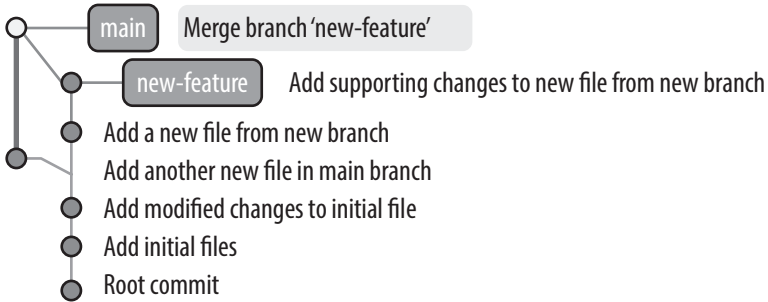


Рис. 4.9. Представление слияния, полученное с помощью `gitk`



Не существует постоянной записи стартовых точек ветки, но Git может алгоритмически определить их через команду `git merge-base`.

Диапазоны коммитов

Команда `git log` оперирует серией коммитов. Это одна из многих команд, позволяющих обходить историю коммитов вашего репозитория. Когда вы указываете коммит Y как стартовую точку для `git log`, это означает, что вы просите Git показать журнал всех коммитов, достижимых от коммита Y.

В графе коммитов Git набор достижимых коммитов — это набор коммитов, до которых можно добраться из заданного коммита путем обхода направленных родительских ссылок. С точки зрения потока данных набор достижимых коммитов — это набор коммитов-предков, которые вливаются в начальный коммит.



В теории графов узел X считается достижимым из узла A, если вы можете начать от A и, пройдя вдоль графа по установленным правилам, добраться до X. Набор достижимых узлов для узла A — это коллекция всех узлов, достижимых из A.

В Git есть несколько опций, позволяющих включать и исключать коммиты в заданном диапазоне. Эти опции не ограничиваются командой `git log`, они также

¹ Сверху вниз: Слить ветку 'new-feature'; Добавить нужные изменения в новый файл из новой ветки; Добавить новый файл из новой ветки; Добавить еще один новый файл в ветку main; Добавить модифицированные изменения в исходный файл; Добавить исходные файлы; Корневой коммит. — *Примеч. пер.*

применимы и к другим подкомандам Git. Например, вы можете исключить конкретный коммит X и все коммиты, достижимые от него, с помощью выражения `^X`. Как правило, диапазон используется для анализа ветки или ее части.

Обозначается диапазон нотацией двойной точки (`..`), в формате *начало..конец*.

Диапазон коммитов, *начало..конец*, определяется как набор коммитов, включающий *конец*, но исключающий *начало*. Это показано ниже на рис. 4.11.

В разделе «Просмотр старых коммитов» вы видели, как использовать диапазон коммитов с помощью `git log`. В том примере мы рассматривали диапазон `main~9..main~7` для выбора восьмого и седьмого коммитов в основной ветке. Чтобы визуальнo представить себе этот диапазон, рассмотрите граф коммитов на рис. 4.10. Здесь показана часть ветки M с линейной историей коммитов.

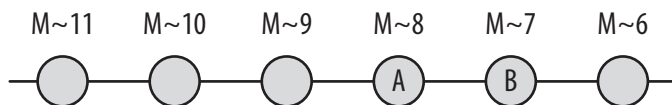


Рис. 4.10. Линейная история коммитов

Напомню, что поток времени идет слева направо, значит, `M~11` — самый старый коммит из показанных, `M~6` — последний, а A — восьмой из предыдущих коммитов.

В диапазон `M~9..M~7` входят два коммита, восьмой и седьмой, которые обозначены буквами A и B. В этот диапазон не входит `M~9` (вспомним правило: «включая `M~7`, но исключая `M~9`»).

ДИАПАЗОН КОММИТОВ КАК ОПЕРАЦИЯ НАД МНОЖЕСТВОМ

Для большей ясности можно рассмотреть диапазон коммитов как математическую операцию над множеством. Коммиты, достижимые из любого указанного коммита, формируют множество, а коммиты, достижимые из любого указанного коммита, имеющего в начале символ `^`, из этого множества удаляются. Например:

```
$ git log ^X Y
```

Это выражение можно прочесть так: «вывести все коммиты, достижимые из Y, но не выводите те, которые ведут к X, включая X».

Нотация двойной точки, `X..Y`, — это сокращение для `^X Y`. Таким образом, следующие команды можно использовать взаимозаменяемо:

```
$ git log X..Y
$ git log ^X Y
```

Соединяя эти две формы, диапазон коммитов $X..Y$ математически равнозначен $\wedge X Y$. Если представить это в виде вычитания множества, то получится так: вывести все ведущее к Y минус все ведущее к X , включая X .

Имеются еще две вариации. Если не обозначить в диапазоне *начальный* или *конечный* коммит, то на их месте будет предполагаться HEAD коммит. Таким образом:

- $..конец$ равнозначно $HEAD..конец$.
- $начало..$ равнозначно $начало..HEAD$.

Вернемся к примеру выше. Если рассматривать его как операции над множеством, то приходим к тому, что диапазон $M\sim9..M\sim7$ содержит всего два коммита, A и B.

- Начинаем со всего, что ведет к коммиту $M\sim7$. См. первую строку рис. 4.11.
- Находим все ведущее к коммиту $M\sim9$, включая $M\sim9$. См. вторую строку рисунка.
- Вычитаем $M\sim9$ из $M\sim7$, чтобы получить коммиты, показанные в третьей строке рисунка.

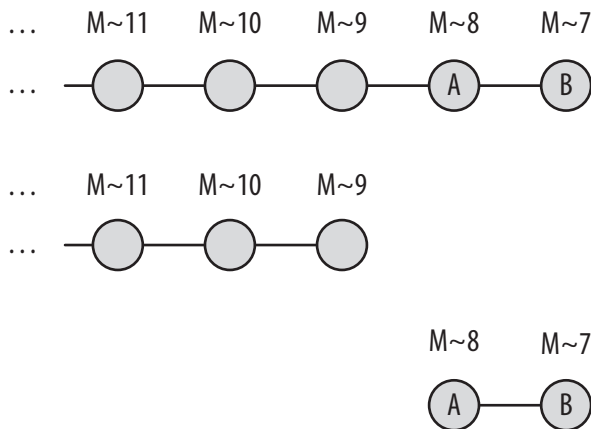


Рис. 4.11. Интерпретация диапазонов как вычитания множеств

Когда история вашего репозитория — это линейная серия коммитов, становится проще понять принцип работы диапазона. Но когда в графе есть ветки или слияния, все становится несколько сложнее. Поэтому важно понимать точные определения.

Рассмотрим другие примеры. Обращаю ваше внимание, что это абстрактные примеры; они созданы для того, чтобы вам было проще разобраться и понять суть. Для подкрепления изложенной выше концепции добавим операцию слияния *merge*, о которой будет подробно рассказано в главе 6.

Начнем со случая, когда ветка `main` имеет линейную историю, как показано на рис. 4.12. Множества `B..E`, `^B E` и `C, D, и E` равнозначны.

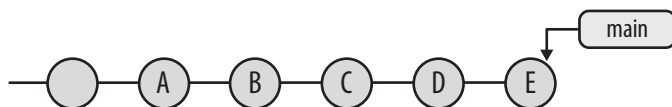


Рис. 4.12. Простая линейная история

На рис. 4.13 ветка `main` у коммита `V` была слита в ветку `feature` у коммита `B`.

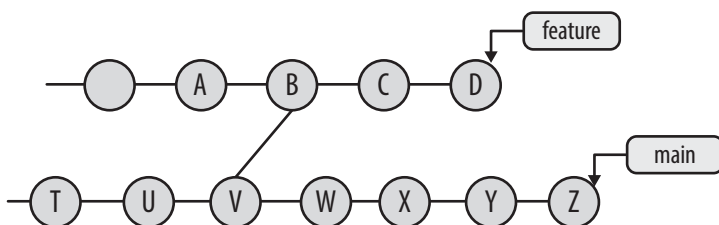


Рис. 4.13. Ветка `main` слита в ветку `feature`

Диапазон `feature..main` представляет эти коммиты в `main`, но не в `feature`. Поскольку каждый коммит в ветке `main` до коммита `V` включительно (то есть множество `{..., T, U, V}`) относится к `feature`, эти коммиты исключаются, оставляя `W, X, Y` и `Z`.

Обратный вариант этого же примера показан на рис. 4.14. Здесь `feature` слита в `main`.

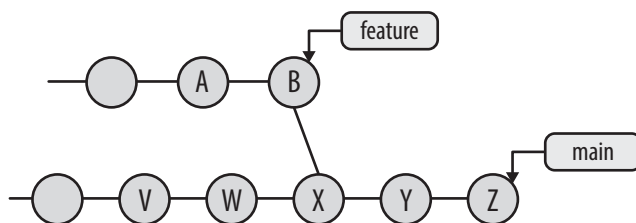


Рис. 4.14. Ветка `feature` слита в ветку `main`

В этом примере диапазон `feature..main`, снова представляющий эти коммиты в `main`, но не в `feature`, является множеством коммитов в ветке `main`, ведущих к `V, W, X, Y` и `Z` и включающих их.

Следует внимательно изучить всю историю ветки `feature`. Рассмотрим случай, в котором она началась как ветка, исходящая от `main`, и затем снова в нее вернулась. Это показано на рис. 4.15.

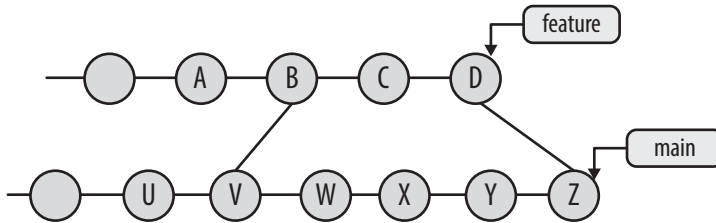


Рис. 4.15. Ветка и слияние

В этом случае `feature..main` содержит только коммиты W, X, Y и Z. Напомню, что из диапазона будут исключены *все* коммиты, достижимые (на пути назад или за пределами графа) от `feature` (то есть коммиты D, C, B, A и более ранние), а также V, U и более ранние от другого родителя B. В итоге останутся только коммиты от W до Z.

Наконец, по аналогии с тем, что *начало..конец* можно рассмотреть как операцию вычитания множества, нотация `A...B` (с тремя точками) представляет симметричные отличия между A и B либо множество коммитов, достижимых либо от A, либо от B, но не от обоих. Из-за симметрии функции ни один коммит не может по-настоящему считаться началом или концом. В этом смысле A и B равнозначны.

Множество ревизий в симметрической разности между A и B, `A...B` выводятся следующей командой:

```
$ git rev-list A B --not $(git merge-base --all A B)
```

Разберем пример, показанный на рис. 4.16.

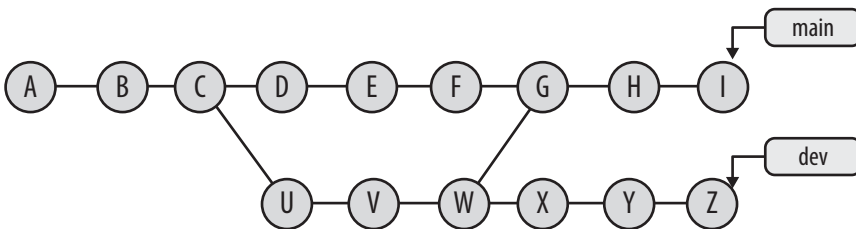


Рис. 4.16. Пример симметрической разности

К `main` относятся коммиты (I, H, ..., B, A, W, V, U), а к `dev` — коммиты (Z, Y, ..., U, C, B, A).

Объединением этих двух множеств будет (A, ..., I, U, ..., Z). База слияния между `main` и `dev` — коммит W. В более сложных случаях может быть несколько баз слияния, но здесь она всего одна. Коммиты, относящиеся к W, — это (W, V, U, C, B и A). Они же являются коммитами, общими для веток `main` и `dev`, поэтому их нужно удалить, чтобы сформировать симметрическую разность: (I, H, Z, Y, X, G, F, E, D).

Полезно представить симметрическую разность между двух веток, **A** и **B**, как «показать все в ветке **A** или в ветке **B**, но только до той точки, где эти две ветки разошлись».

Каждый элемент определения симметрической разности можно вычислить так:

```
A...B = (A OR B) AND NOT (merge-base --all A B)
```

Теперь, когда мы разобрали, что такое диапазоны коммитов, как их записывать и как они работают, важно отметить, что фактически в Git не поддерживается настоящий оператор диапазона. Это лишь удобное обозначение: **A...B** замещает форму **^A B**. Вообще говоря, Git предоставляет гораздо более мощные возможности управления множествами коммитов через командную строку. Команды, которые работают с диапазонами, могут принимать произвольную последовательность включенных и исключенных коммитов. Например, команда

```
$ git log ^dev ^feature ^bugfix main
```

приводит к выбору коммитов в ветке **main**, но не в ветках **dev**, **feature** или **bugfix**.

Эти примеры могут показаться абстрактными, но когда вы понимаете, что любое имя ветки можно использовать как часть диапазона, вы осознаете всю мощь использования диапазонов. Как будет рассказано в разделе «Ветки отслеживания» на с. 261 (глава 11), если одна из ваших веток представляет коммиты из другого репозитория, то вы можете быстро обнаружить множество коммитов, которые находятся в *вашем* репозитории, а не в другом.

Итоги

В этой главе мы познакомились с коммитами — единицами записи изменений. Затем мы рассмотрели важность записи коммитов как атомарных наборов изменений. Далее мы углубили понимание коммитов, познакомившись со способами их определения. Мы подчеркнули важность идентификации коммитов, поскольку она является строительным блоком для многих команд и концепций Git среднего и продвинутого уровня. Эти команды помогут вам уверенно справляться с любыми сложными требованиями, возникающими в ваших проектах при работе с репозиториями.

Мы также узнали, как просматривать историю коммитов репозитория, чтобы лучше понять, как он достиг своего текущего состояния. Тема работы с диапазонами коммитов может быть сложна для понимания, но вы лучше уясните ее принципы, когда начнете анализировать историю репозитория на конкретных сценариях, особенно тех, которые требуют перемещения по истории коммитов к определенному состоянию репозитория.

Управление файлами и индекс

Работая в любой системе контроля версий, вы редактируете файлы в своем рабочем каталоге и фиксируете изменения в репозиторий для безопасного хранения. Git работает так же, но вставляет между рабочим каталогом и репозиторием промежуточный слой — индекс. *Индекс*, иначе говоря *каталог индексирования* или промежуточный каталог (*staging directory*), используется для индексации, или сбора, изменений в любых файлах в качестве финального шага перед их фиксацией.

Индекс можно рассматривать как кэш текущего состояния рабочего каталога. Он важен при создании нового коммита, запросе статуса репозитория и слиянии веток. О слияниях поговорим в главе 6. Здесь же мы сосредоточимся на важности каталога индексирования по отношению к коммитам, о которых шла речь в главе 4.

Я расскажу, в чем заключается уникальность каталога индексирования в Git относительно других систем контроля версий. Объясню, как управлять индексом и коллекциями файлов, а также опишу, как добавлять и удалять файлы из репозитория. Вы узнаете, как переименовывать файл в Git и каталогизировать состояние индекса. В конце я расскажу, как заставить Git игнорировать временные и другие ненужные файлы, которые не требуются отслеживать для контроля версий.

Важность индекса

Когда вы управляете кодом с помощью Git, то редактируете рабочий каталог, собираете изменения в индексе и делаете коммит того, что собралось в индексе, в виде единого набора изменений. Этот принцип станет понятнее, если рассмотрим его на примере газетной статьи. Сначала вы готовите черновик статьи, затем редактор ее проверяет и предлагает правки. Затем вы вносите эти правки, и, наконец, статью публикуют.

Черновик статьи соответствует вашему рабочему каталогу в Git, редакторские правки — это то, что вы собираетесь вносить в каталог индексирования, а опубликованная статья — итоговый коммит.

Вы можете представить себе индекс Git как набор предполагаемых изменений. Вы добавляете, удаляете, перемещаете или повторно редактируете файлы вплоть до коммита, который актуализирует накопленные изменения в репозитории. Таким образом, основная часть важной работы фактически предшествует этапу коммита. Этот принцип показан на рис. 5.1.

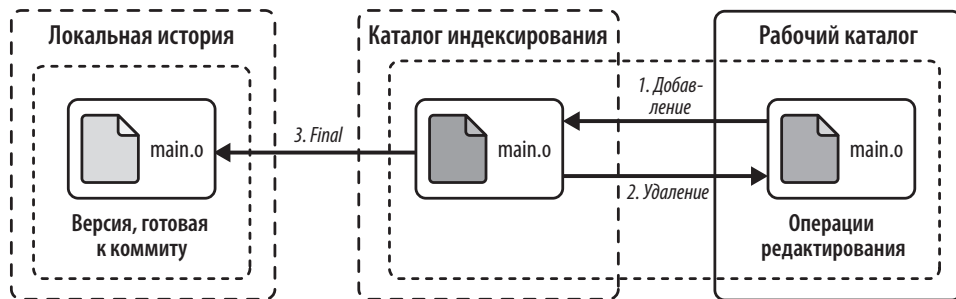


Рис. 5.1. Принцип работы с индексом и файлами

На рис. 5.1 версия `main.o` в блоке «Локальная история» представляет финальную версию файла из каталога индексирования после завершения всех операций редактирования.

Как описано в разделе «Индекс» на с. 48 в главе 2, этот каталог — одна из важнейших структур данных в Git (см. документацию Git (<https://oreil.ly/dBb90>)). Это двоичный файл, который содержит виртуальный упорядоченный список путей файлов вместе с их разрешениями и ссылкой на ID соответствующих BLOB-объектов. По сути, индекс содержит виртуальное состояние дерева, представляющее объект дерева, на который будет ссылаться будущий коммит (BLOB-объекты и объекты деревьев описаны в главе 2).

Операции добавления, удаления или редактирования файлов обычно используют высокоуровневые команды Git, которые скрывают детали индекса и упрощают вашу работу. В качестве примера вы можете запросить состояние индекса в любой момент времени командой `git status`. Эта команда явно сообщает, какие файлы Git считает индексированными. Для этого она сравнивает виртуальное состояние дерева с рабочим деревом и выводит различия.

Заглянуть во внутреннее состояние Git также можно с помощью низкоуровневых команд, например `git ls-files`. Вот пример вывода индекса:

```
$ git ls-files -s
100644 e69de29bb2d1d6434b8b29ae775ad8c2e48c5391 0    README.md
100644 6ff87c4664981e4397625791c8ea3bbb5f227e69 0    NewFile.md
```

В индексе Git нет содержимого файлов. Он просто отслеживает то, что вы хотите зафиксировать. Когда вы выполняете `git commit`, Git в поиске содержимого для коммита проверяет не рабочий каталог, а индекс.

При отправке файла в индекс пригодится команда `git diff`. Концепция `diff`, или визуального выделения отличий между текущей и предыдущей версиями файла, подробно раскрывается в главе 7.

Эта команда может выводить два набора изменений: `git diff` показывает изменения, которые остаются в вашем рабочем каталоге и не индексируются, а `git diff --cached` — изменения, которые индексированы, а значит, будут участвовать в следующем коммите.



`git diff --staged` — это синоним `git diff --cached`. Можно использовать любую из этих команд.

При индексировании изменений вы можете применять оба варианта `git diff`. Изначально `git diff` выводит большой набор всех предполагаемых изменений, а `--cached` не выводит ничего. В процессе индексирования первый набор будет уменьшаться, а последний расти. Если все ваши рабочие изменения будут индексированы и готовы к коммиту, `--cached` выдаст их полный список, а `git diff` не покажет ничего.

Классификация файлов в Git

Поскольку индекс — это переходное состояние, динамический слой между рабочим каталогом и объектным хранилищем Git, важно понимать, как Git отслеживает состояние ваших файлов.

Git классифицирует файлы на три группы: отслеживаемые, игнорируемые и не-отслеживаемые.

Отслеживаемые (tracked)

Отслеживаемый файл — это любой файл, уже находящийся в репозитории или внесенный в индекс. Для добавления в эту группу нового файла с именем `somefile` выполните команду `git add somefile`.

Игнорируемые (ignored)

Игнорируемый файл должен быть явно объявлен видимым или игнорируемым в репозитории, даже если он есть в рабочем каталоге. Проекты разработки ПО обычно содержат немало игнорируемых файлов. Как правило, к ним относятся временные и рабочие файлы, личные заметки, вывод компилятора и большинство файлов, генерируемых автоматически во время сборки. Git поддерживает предустановленный список файлов для игнорирования, и вы можете настроить репозиторий для распознавания дополнительных файлов. Игнорируемые файлы подробно разбираются в разделе «Файл .gitignore» на с. 139.

Неотслеживаемые (untracked)

Неотслеживаемый файл — это любой файл, который не принадлежит ни одной из предыдущих категорий. Git просматривает весь набор файлов в рабочем каталоге, исключая из него отслеживаемые и игнорируемые, чтобы вывести те, которые не отслеживаются.

Разберем эти категории файлов. Создадим новый рабочий каталог и репозиторий, а затем поработаем с некоторыми файлами:

```
# Инициализация пустого репозитория git
$ cd /tmp/my_stuff
$ git init --initial-branch=main

$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)

# Добавление нового файла с неким содержимым
$ echo "New data" > data

$ git status
On branch main

No commits yet

Untracked files:
(use "git add <file>..." to include in what will be committed)
  data

nothing added to commit but untracked files present (use "git add" to track)
```

В приведенном коде мы инициализировали репозиторий Git с предустановленным именем ветки `main`, используя команду `git init` с опцией `--initial-branch`.

В этом новом репозитории еще нет файлов, поэтому и наборы — отслеживаемые, игнорируемые и неотслеживаемые — пусты. После создания `data` и выполнения `git status` Git сообщил об одном неотслеживаемом файле.

Далее показано, как явно игнорировать файл в репозитории Git:

```
# Вручную создаем ненужный файл
$ touch main.o

$ git status
On branch main

No commits yet
```

```
Untracked files:
(use "git add <file>..." to include in what will be committed)
    data
    main.o

nothing added to commit but untracked files present (use "git add" to track)

# Явное добавление main.o как игнорируемого через файл .gitignore
$ echo main.o > .gitignore

$ git status
On branch main

No commits yet

Untracked files:
(use "git add <file>..." to include in what will be committed)
    .gitignore
    data

nothing added to commit but untracked files present (use "git add" to track)
```

Этот пример воссоздает то, что часто происходит в редакторах кода и средах сборки, которые оставляют в кодовой базе временные файлы. Такие файлы не должны отслеживаться в репозитории как файлы с исходным кодом. Чтобы Git игнорировала файл в каталоге, его имя нужно добавить в специальный файл `.gitignore`. О файле `.gitignore` мы еще поговорим в этой главе позднее.

В приведенном примере файл `main.o` игнорируется, но команда `git status` теперь показывает новый, неотслеживаемый файл `.gitignore`. И хотя `.gitignore` имеет для Git особое значение, он управляется точно так же, как любой файл в репозитории. Пока `.gitignore` не добавлен, Git считает его неотслеживаемым.

Далее рассмотрим разные способы изменения статуса файла (отслеживаемый, неотслеживаемый или игнорируемый), а также добавление и удаление его из индекса.

Использование `git add`

Коммит создается в два этапа: сначала вы индексируете изменения, а затем их фиксируете.

Файл индексируется командой `git add`. Если файл не отслеживается, команда `git add` меняет его статус на отслеживаемый. Когда `git add` используется в отношении имени каталога, все файлы и подкаталоги, находящиеся в нем, индексируются рекурсивно (то есть `git add` будет применена ко всем файлам и подкаталогам внутри указанного каталога, включая все уровни вложенности).

Продолжим разбирать наш пример:

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

        .gitignore
        data

# Отслеживание обоих новых файлов
$ git add data .gitignore

$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

        new file:   .gitignore
        new file:   data
```

Первая команда `git status` показывает, что не отслеживаются два файла. Отследить их можно с помощью `git add`. После выполнения команды `git add` файлы `data` и `.gitignore` помещаются в индекс и начинают отслеживаться. Теперь они готовы к добавлению в репозиторий при следующем коммите (рис. 5.2).

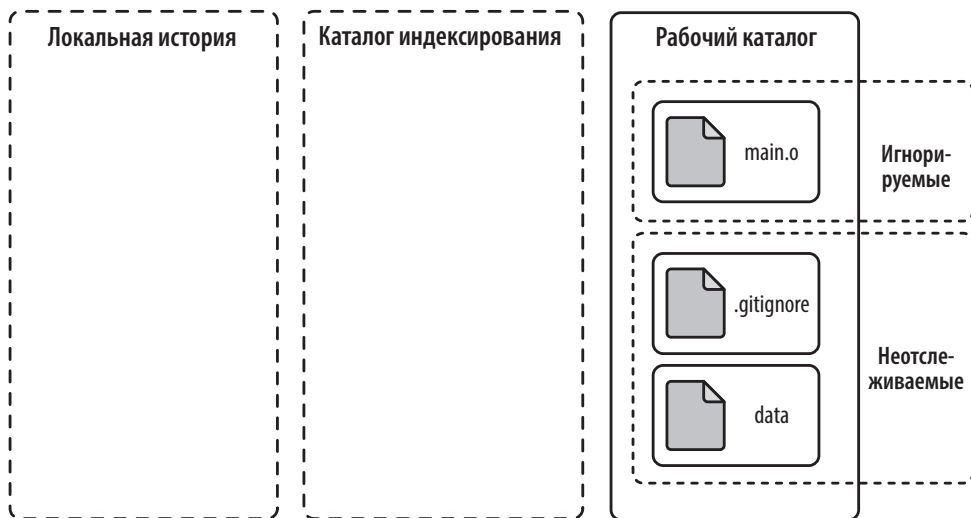


Рис. 5.2. Индекс и классификация файлов в Git

С позиции объектной модели Git в момент выполнения `git add` каждый файл целиком копируется в объектное хранилище и индексируется по своему SHA1-имени. Индексирование файла также называется *кэшированием файла*¹ или *помещением файла в индекс*.

С помощью `git ls-files` можно посмотреть внутрь объектной модели в поиске значений SHA-1 индексированных файлов:

```
$ git ls-files --stage
100644 0487f44090ad950f61955271cf0a2d6c6a83ad9a 0      .gitignore
100644 534469f67ae5ce72a7a274faf30dee3c2ea1746d 0      data
```

Это показано на рис. 5.3.

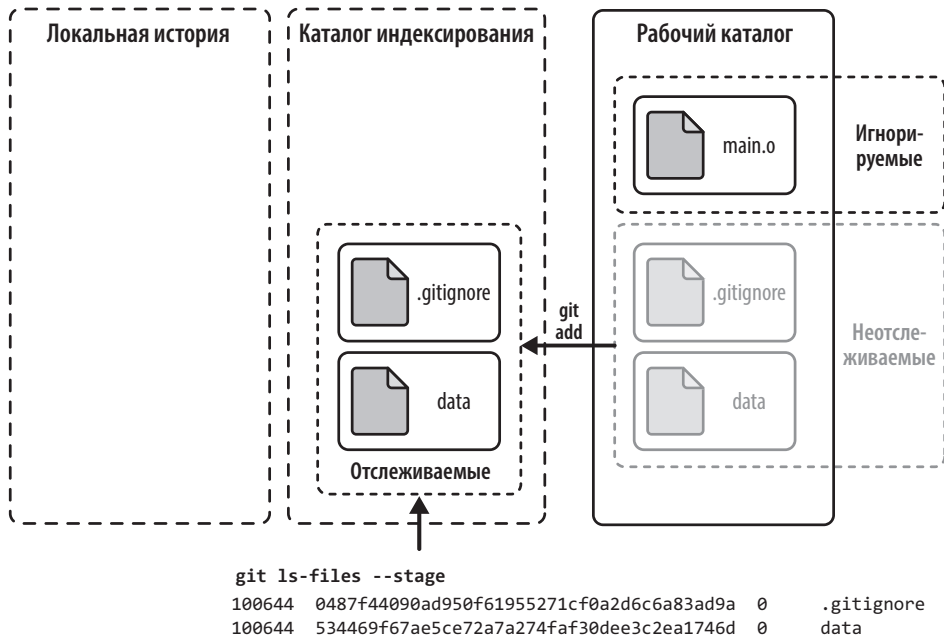


Рис. 5.3. Индекс и индексированные файлы

Большинство рутинных изменений в репозитории, скорее всего, будут простыми правками. После любой такой правки и до коммита выполните `git add` для обновления индекса окончательной версией файла. В противном случае у вас получатся две его версии: одна будет находиться в хранилище объектов и иметь ссылку из индекса, а вторая — в рабочем каталоге.

Продолжим рассматривать наш пример. Мы изменим файл `data`, чтобы он отличался от находящегося в индексе, и используем команду `git hash-object file`

¹ Вы же заметили `--cached` в выводе `git status`?

(которую вы вряд ли когда-нибудь вызовете напрямую). Она позволит непосредственно вычислить и вывести хеш SHA-1 для новой версии:

```
$ git ls-files --stage
100644 0487f44090ad950f61955271cf0a2d6c6a83ad9a 0      .gitignore
100644 534469f67ae5ce72a7a274faf30dee3c2ea1746d 0      data
```

редактирование "data" добавлением...

```
$ cat data
```

```
New data
```

```
And some more data now
```

```
$ git hash-object data
```

```
e476983f39f6e4f453f0fe4a859410f63b58b500
```

После изменения файла его прежняя версия в объектном хранилище и индексе имеет хеш 534469f67ae5ce72a7a274faf30dee3c2ea1746d. Но у обновленной версии этого файла хеш e476983f39f6e4f453f0fe4a859410f63b58b500.

Обновим индекс с помощью команды `git add`, чтобы он содержал новую версию файла:

```
$ git add data
```

```
$ git ls-files --stage
```

```
100644 0487f44090ad950f61955271cf0a2d6c6a83ad9a 0      .gitignore
100644 e476983f39f6e4f453f0fe4a859410f63b58b500 0      data
```

Это показано на рис. 5.4. Обратите внимание на новый ID SHA-1 для data.

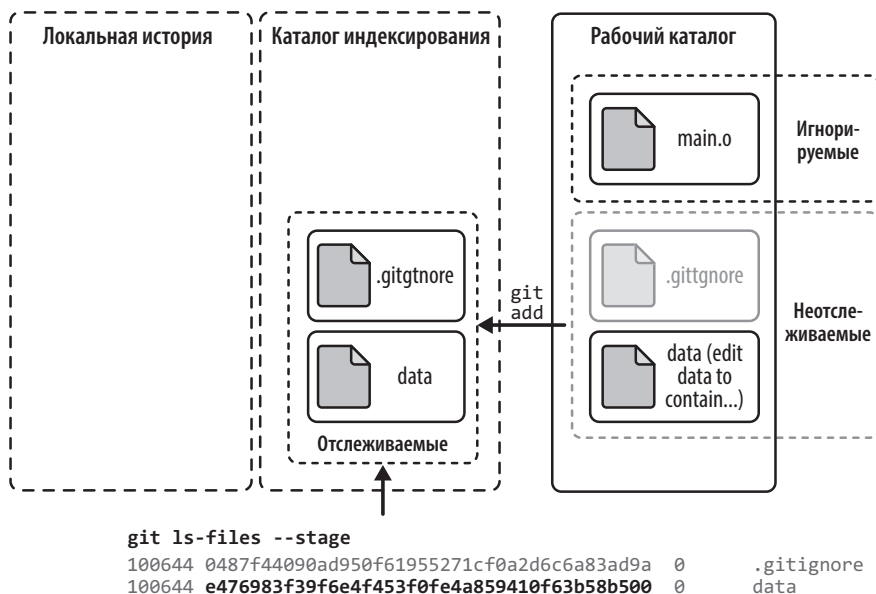


Рис. 5.4. Индекс и обновленные индексированные файлы

Теперь в индексе находится обновленная версия файла. Можно сказать, что «файл **data** был индексирован» или «файл **data** находится *в индексе*». Последняя фраза менее точна, поскольку на самом деле файл находится в объектном хранилище, а индекс просто ссылается на него виртуально. На первый взгляд бессмысленное манипулирование хешами SHA-1 и индексом раскрывает важный момент: рассматривайте команду `git add` не как «добавить этот файл», а как «добавить это содержимое в индекс».

Помните, что версия файла в рабочем каталоге может не быть синхронизирована с версией в индексе. Когда дело доходит до коммита, Git использует версию из индекса.



Опцию `--interactive`, используемую с `git add` или `git commit`, можно применять для просмотра файлов, которые вы собираетесь индексировать для последующего коммита.

Заметки по использованию `git commit`

Ниже приведены некоторые краткие пояснения, как индекс влияет на работу с коммитами.

Использование `git commit --all`

Опция `-a` или `--all` для `git commit` дает системе Git команду индексировать все неиндексированные, отслеживаемые изменения файлов, включая удаление файлов из рабочей копии до выполнения коммита.

Настроим несколько файлов с разными характеристиками индексирования:

```
# Настройка тестового репозитория
$ mkdir /tmp/commit-all-example
$ cd /tmp/commit-all-example
$ git init --initial-branch=main
Initialized empty Git repository in /tmp/commit-all-example/.git/

$ echo something >> ready
$ echo something else >> notyet
$ git add ready notyet
$ git commit -m "Setup"
[main (root-commit) 63ae368]] Setup
2 files changed, 2 insertions(+)
create mode 100644 notyet
create mode 100644 ready

# Изменение файла "ready" и его "git add" в индекс
# Редактирование ready
$ git add ready
```

```
# Изменение файла "notyet", делающее его неиндексированным
# редактирование notyet

# Добавление нового файла в подкаталог, но не в индекс
$ mkdir subdir
$ echo Nope >> subdir/new
```

С помощью `git status` посмотрим, что сделает стандартный коммит (без опций командной строки):

```
$ git status
On branch main
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
    modified:   ready

Changes not staged for commit:
(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)
    modified:   notyet

Untracked files:
(use "git add <file>..." to include in what will be committed)
    subdir/
```

Здесь индекс подготовлен к коммиту всего одного файла `ready`, поскольку только он был индексирован.

Если выполнить `git commit --all`, Git рекурсивно обойдет весь репозиторий, индексирует все известные измененные файлы и зафиксирует их. В редакторе в шаблоне сообщения коммита должно быть указано, что измененный и известный файл `notyet` тоже будет зафиксирован:

```
# Введите сообщение коммита для внесенных изменений.
# Строки, начинающиеся с '#', будут проигнорированы, а пустое
# сообщение приведет к отмене коммита.
#
# В ветке main
# Изменения для коммита:
#     изменен:   notyet
#     изменен:   ready
#
# Неотслеживаемые файлы:
#     subdir/
#
```

Но поскольку каталог `subdir/` новый и в нем не отслеживается никакое имя файла или путь, то даже опция `--all` не приведет к его коммиту. Завершение коммита подходящим сообщением даст такой вывод:

```
[main a6f75bb] Some --all thing.
2 files changed, 2 insertions(+)
```


Когда Git рекурсивно обходит репозиторий в поиске измененных и удаленных файлов, полностью новый каталог `subdir/` и все его файлы не становятся частью коммита — они остаются в рабочем каталоге.



Ради удобства при изменении существующего отслеживаемого файла в репозитории Git позволяет совмещать два шага, `git add` и `git commit`:

```
$ git commit -am trackedfile
```

Но если вы перемещаете или удаляете файл, у вас уже нет такой возможности. Эти два шага должны быть разделены:

```
$ git rm somefile  
$ git commit
```

Запись сообщений коммитов

Если вы не передадите напрямую сообщение журнала в командной строке, Git запустит редактор и предложит написать это сообщение. Git выбирает, какой редактор запускать, исходя из вашей конфигурации (см. раздел «Файлы конфигурации» на с. 42 в главе 1).

Если вы пишете сообщение в редакторе и по какой-то причине решаете отменить операцию, просто выйдите из редактора без сохранения. В итоге получится пустое сообщение, по которому Git поймет, что коммит нужно отменить:

```
$ git commit ready
```

```
# Запущен указанный в конфигурации редактор  
# Выход из редактора без написания сообщения коммита
```

```
Aborting commit due to empty commit message.
```

Если вы уже выполнили сохранение, просто удалите все сообщение журнала и повторите сохранение. Git не будет обрабатывать пустой (без текста) коммит.

Использование `git rm`

Команда `git rm` противоположна команде `git add`. Она удаляет файл из репозитория и рабочего каталога. Но поскольку удаление файла может вызвать больше проблем (если что-то пойдет не так), чем его добавление, то Git обрабатывает эту операцию более осторожно.

Git может удалить файл в двух ситуациях: когда вам нужно удалить его только из индекса и когда его нужно удалить из индекса и рабочего каталога одновременно. Поскольку Git не будет автоматически удалять файл из рабочего каталога, используйте стандартную команду `rm`.

Удаление файла из каталога и индекса не приведет к удалению существующей истории из репозитория. Любые версии файла, которые уже были зафиксированы, остаются в объектном хранилище и сохраняют эту историю.

Продолжая пример выше, давайте «случайно» внесем дополнительный файл, который не должен индексироваться, и посмотрим, как его удалить:

```
$ echo "random stuff" > oops
```

```
$ git rm oops
```

```
fatal: pathspec 'oops' did not match any files
```

```
# Вы не можете использовать "git rm" для файлов, которые не отслеживаются
```

```
# Для удаления файла из рабочего каталога используйте "rm oops"
```

Поскольку `git rm` — это тоже операция над индексом, команда не сработает для файла, который ранее не был добавлен в репозиторий или индекс. Сначала Git нужно о нем узнать, чтобы начать отслеживать. Поэтому случайно индексируем файл `oops`:

```
# Случайная индексация файла "oops"
```

```
$ git add oops
```

```
$ git status
```

```
On branch main
```

```
No commits yet
```

```
Changes to be committed:
```

```
(use "git rm --cached <file>..." to unstage)
```

```
new file:   .gitignore
```

```
new file:   data
```

```
new file:   oops
```

Для преобразования файла из индексированного в неиндексированный используем `git rm --cached`. Посмотрим, как это повлияет на индекс:

```
$ git ls-files --stage
```

```
100644 0487f44090ad950f61955271cf0a2d6c6a83ad9a 0      .gitignore
```

```
100644 e476983f39f6e4f453f0fe4a859410f63b58b500 0      data
```

```
100644 6bece861cc8fd34181e6d43264acfe1503486538 0      oops
```

```
$ git rm --cached oops
```

```
rm 'oops'
```

```
$ git ls-files --stage
```

```
100644 0487f44090ad950f61955271cf0a2d6c6a83ad9a 0      .gitignore
```

```
100644 e476983f39f6e4f453f0fe4a859410f63b58b500 0      data
```

Команда `git rm --cached` удаляет файл из индекса и оставляет его в рабочем каталоге:

```
$ git status
On branch main

No commits yet

Changes to be committed:
(use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   data

Untracked files:
(use "git add <file>..." to include in what will be committed)
    oops
```

В противоположность этому команда `git rm` удаляет файл из индекса и рабочего каталога.



Использовать `git rm --cached`, чтобы сделать файл неотслеживаемым и при этом оставить его копию в рабочем каталоге, опасно — так вы можете забыть, что он больше не отслеживается. Применение такого подхода также переопределяет проверку актуальности содержимого рабочего файла. Будьте внимательны, когда выбираете команду.

Чтобы удалить файл после его коммита, достаточно индексировать запрос с помощью простой команды `git rm filename`:

```
$ git commit -m "Add some files"
[main (root-commit) 57cbafa] Add some files
2 files changed, 2 insertions(+)
create mode 100644 .gitignore
create mode 100644 data

$ git rm data
rm 'data'

$ git status
On branch main
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
    deleted:   data
```

Прежде чем удалять файл, Git проверяет, что его версия в рабочем каталоге соответствует последней версии в текущей ветке (версии в последнем коммите, HEAD). Эта проверка исключает случайную потерю любых изменений (в результате редактирования), которые могли быть внесены в этот файл.



Для *принудительного* удаления файла используйте `git rm -f`. Эта опция представляет собой явное требование и удаляет файл, даже если с момента последнего коммита вы его изменили.

А в случае, если вы на самом-то деле хотели сохранить файл, который случайно удалили, просто добавьте его обратно:

```
$ git add data
fatal: pathspec 'data' did not match any files
```

О нет! Git удалила и рабочую копию тоже. Но волноваться не стоит — системы контроля версий хорошо справляются с восстановлением старых версий файлов:

```
$ git checkout HEAD -- data
$ cat data
New data
And some more data now

$ git status
On branch main
nothing to commit, working tree clean
```

Использование `git mv`

Допустим, вам нужно переместить или переименовать файл. Можно использовать комбинацию из команды `git rm` для старого файла и команды `git add` для нового либо напрямую применить `git mv`. Если у вас есть репозиторий с файлом `stuff`, который вы хотите переименовать в `newstuff`, то последовательность команд

```
$ mv stuff newstuff
$ git rm stuff
$ git add newstuff
```

и команда

```
$ git mv stuff newstuff
```

будут равнозначными операциями Git.

В обоих случаях Git удаляет имя файла `stuff` из индекса; добавляет новое имя, `newstuff`; сохраняет исходное содержимое `stuff` в объектном хранилище и повторно ассоциирует это содержимое с именем `newstuff`.

Продолжим наш пример. Поскольку файл `data` снова находится в репозитории, мы переименуем его и зафиксируем это изменение:

```
# Проверка индекса перед переименованием файла
$ git ls-files --stage
100644 0487f44090ad950f61955271cf0a2d6c6a83ad9a 0    .gitignore
100644 e476983f39f6e4f453f0fe4a859410f63b58b500 0    data
```

```
$ git mv data mydata

$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    data -> mydata

# Проверка индекса с переименованным файлом
# Заметьте, что ID SHA1 после изменения имени файла остается прежним
$ git ls-files --stage
100644 0487f44090ad950f61955271cf0a2d6c6a83ad9a 0    .gitignore
100644 e476983f39f6e4f453f0fe4a859410f63b58b500 0    mydata

$ git commit -m "Moved data to mydata"
[main 4b7a819] Moved data to mydata
 1 file changed, 0 insertions(+), 0 deletions(-)
 rename data => mydata (100%)
```

Если вы проверите историю файла, то можете забеспокоиться, что система как будто бы потеряла историю исходного файла **data** и помнит только, что переименовала в текущее имя:

```
$ git log mydata
commit 4b7a81923c636716d2b035574ca569018c723f21 (HEAD -> main)
Author: Prem Kumar Ponuthorai <ppremk@gmail.com>
Date:   Mon Apr 5 23:56:05 2021 +0200
```

Moved data to mydata

На самом деле Git по-прежнему помнит всю историю, но вывод ограничен конкретным именем файла, указанным в команде. Опция `--follow` просит Git пройти по журналу и найти всю историю с этим содержимым:

```
$ git log --follow mydata
commit 4b7a81923c636716d2b035574ca569018c723f21 (HEAD -> main)
Author: Prem Kumar Ponuthorai <ppremk@gmail.com>
Date:   Mon Apr 5 23:56:05 2021 +0200
```

Moved data to mydata

```
commit 57cbafaf8f20187395f6806aa7bc579fcbf7acde
Author: Prem Kumar Ponuthorai <ppremk@gmail.com>
Date:   Mon Apr 5 23:37:48 2021 +0200
```

Add some files

Одна из классических проблем систем контроля версий: переименование файла может приводить к тому, что система утрачивает отслеживание истории файла. Git сохраняет эту информацию даже после переименования.

Заметки по отслеживанию переименования

SVN — это традиционная система контроля ревизий. Сравним ее с Git и посмотрим, как в этих двух системах отслеживаются файлы, переименованные разными способами.

SVN проделывает большую работу, отслеживая переименование и перемещение файла, поскольку следит только за отличиями между файлами. Если вы перемещаете файл, то, по сути, удаляете все строки из старого и добавляете их в новый. Но будет неэффективно передавать и сохранять все содержимое файла каждый раз при простом переименовании. Представьте, что вы переименовываете подкаталог, содержащий тысячи файлов.

Чтобы исправить эту ситуацию, SVN отслеживает каждое переименование явно. Если вы хотите переименовать `hello.txt` в `subdir/hello.txt`, то вместо `svn rm` и `svn add` должны использовать для файлов `svn mv`. Иначе SVN не сможет увидеть, что это переименование, и будет вынуждена выполнить только что описанную неэффективную последовательность удаления/добавления.

Учитывая эту особенность отслеживания переименования, серверу SVN нужен специальный протокол, чтобы сообщить клиентам: «Переместите `hello.txt` в `subdir/hello.txt`». Более того, каждый клиент SVN должен обеспечить корректное выполнение этой операции.

ПРОБЛЕМЫ С ОТСЛЕЖИВАНИЕМ ПЕРЕИМЕНОВАНИЯ

Отслеживание переименования файла вызывает неутрахающие споры среди разработчиков систем управления версиями.

Простое переименование — достаточный повод для разногласий. Споры разгораются еще сильнее, когда изменяется имя файла, а затем его содержимое. Тогда сценарии перетекают из практической плоскости в философскую: «А этот “новый” файл — это на самом деле результат переименований или же он просто похож на старый?», «Насколько похожим должен быть новый файл, чтобы считаться тем же файлом?», «Если вы примените чей-то патч, который удалит файл и воссоздаст похожий где-то еще, как это должно обрабатываться?», «Что произойдет, если файл будет переименован двумя разными способами в двух разных ветках?», «Будет ли в таких ситуациях более безопасным автоматически обнаруживать переименование, как это делает Git, или же нужно требовать, чтобы пользователь определял переименование явно, как происходит в SVN?»

В реальных ситуациях система Git для обработки переименования файлов кажется более эффективной, чем те же запутанные методы в SVN. Но все же идеальной системы для обработки переименования не существует... пока.

Git же, напротив, не отслеживает переименование. Вы можете переместить или скопировать `hello.txt` в любое место, но это повлияет только на объекты деревьев. (Напомню, что объекты деревьев хранят связь между содержимым в то время, как само содержимое хранится в BLOB-объектах.) Если посмотреть на различия между двумя деревьями, станет очевидно, что один и тот же уникальный BLOB-объект переместился в новое место. И даже если вы не проверите отличия явно, каждая часть системы знает, что у нее уже есть этот BLOB, а значит, еще одна его копия ей не нужна.

В этой и многих других ситуациях система хранения Git, основанная на хешах, значительно упрощает задачи, которые могут вызывать трудности у других систем.

Файл `.gitignore`

Выше мы узнали, как использовать файл `.gitignore` для исключения нерелевантного файла `main.o`. По аналогии с тем примером вы можете пропустить любой файл, добавив его имя в файл `.gitignore`, находящийся в том же каталоге. Кроме того, вы можете проигнорировать этот файл везде, добавив его в `.gitignore`, расположенный в верхнем каталоге репозитория. Но Git поддерживает и более разносторонний механизм.

Файл `.gitignore` может содержать список шаблонов имен файлов, определяющий, какие файлы нужно игнорировать. Правила для шаблонов `.gitignore`:

- Пустые строки игнорируются. Строки, начинающиеся с символа `#`, могут использоваться для комментариев, но `#` не является комментарием, если идет за другим текстом в той же строке.
- Обычное имя файла соответствует файлу в любом каталоге с указанным именем.
- Имя может заканчиваться слешем (`/`). Это соответствует именованному каталогу и любому подкаталогу, но не файлу или символической ссылке.
- Шаблон, содержащий подстановочные символы, например символ «звездочка» `*`, разворачивается как шаблон подстановки символов. Как и в случае стандартной подстановки символов, это сопоставление не работает между каталогами, поэтому `*` может относиться только к одному имени файла или каталога. Но символ «звездочка» по-прежнему может быть частью шаблона, включающего слеш, для указания имен каталогов (например, `debug/*.o`).
- Стоящий в начале восклицательный знак (`!`) инвертирует смысл шаблона оставшейся части строки. Он также ставится для указания исключений из правил игнорирования. Инверсия имеет более высокий приоритет.

Рассмотрим пример.

```
$ cat .gitignore
# Пример формата шаблона файла .gitignore

# Ниже — не комментарий. Символ "#" — это часть выражения в качестве его первого
символа
\#*.tmp

# Имя файла
somefile.log

# Исключение каталога
my_package/

# Выражение с подстановкой
*.[oa]
*.log debug/*.o

# Инвертирование выражения с подстановкой (не игнорировать файл important.log)
!important.log
```

В табл. 5.1 показан пример вывода, соответствующий шаблону подстановки, который поддерживается файлом `.gitignore`.

Таблица 5.1. Пример вывода шаблона подстановки

Шаблон подстановки	Соответствия
*.log	.log important.log file.log dir/anotherfile.log
*.[oa]	file.o file.a
tmp/	tmp/files.log tmp/subdir/files.log parent/tmp/files.log grandparent/tmp/files.log
file.log	file.log dir/file.log
dir/**/file	dir/file dir/subdir/file dir/subdir/subsubdir/file

Шаблон подстановки	Соответствия
<code>**/file</code>	<code>dir/file</code> <code>anotherdir/file</code> <code>file</code>
<code>file?.log</code>	<code>file1.log</code> <code>file2.log</code>
<code>!important.log</code>	<code>important.log</code> <code>dir/important.log</code> <i>указанные выше файлы не будут игнорироваться</i>



Две смежные звездочки (******) в шаблонах подстановки, которые включают имя каталога, могут сопоставляться по-разному, как показано в примерах табл. 5.1.

Git позволяет разместить файлы `.gitignore` в любом каталоге вашего репозитория. Каждый такой файл влияет на свой каталог и находящиеся в нем подкаталоги. Правила `.gitignore` каскадные: вы можете переопределять их в каталоге более высокого уровня, включив инвертированный шаблон (с символом **!** в начале) в один из подкаталогов.

Поскольку файлы `.gitignore` могут присутствовать в нескольких каталогах, а кроме того, список игнорируемых файлов можно дополнять через командную строку, во избежание конфликтов в Git соблюдается следующая иерархия (сверху вниз; первый пункт в списке обладает наивысшим приоритетом):

- Шаблоны, заданные в командной строке.
- Шаблоны, считываемые из `.gitignore` в текущем каталоге.
- Шаблоны из родительских каталогов более высокого уровня. Шаблоны текущего каталога превосходят шаблоны родителей, а родители, находящиеся ближе всех к текущему каталогу, имеют приоритет над родителями более высокого уровня.
- Шаблоны из файла `.git/info/exclude`.
- Шаблоны из файла, указанного переменной конфигурации `core.excludesFile`.

Поскольку файл `.gitignore` рассматривается как обычный файл внутри репозитория, он копируется во время операций клонирования и шаблоны из него применяются ко всем копиям репозитория. Записи в файлы `.gitignore` нужно добавлять тогда, когда шаблоны применяются ко *всем* производным репозиториям единообразно.

Если же шаблоны исключения являются специфичными для одного конкретного репозитория и не должны применяться к чужим клонам этого репозитория, то нужно поместить шаблоны в файл `.git/info/exclude`. Этот файл не копируется во время клонирования. Формат и обработка его шаблонов такие же, как у файла `.gitignore`.

Рассмотрим еще один сценарий. Файлы `.o`, генерируемые компилятором из исходного кода, обычно следует исключать. Чтобы проигнорировать эти файлы, добавьте `*.o` в файл `.gitignore` верхнего уровня. Но что, если у вас есть конкретный файл `*.o`, который, например, был передан кем-то другим и для которого вы не могли сгенерировать замену сами? Вы наверняка захотите отслеживать этот конкретный файл явно. Тогда у вас может быть такая конфигурация:

```
├── my_package
│   ├── .gitignore
│   └── vendor_files
│       └── .gitignore
└──
```

```
$ cd my_package
$ cat .gitignore
*.o

$ cd my_package/vendor_files
$ cat .gitignore
!driver.o
```

Это значит, что Git будет игнорировать все файлы `.o` в репозитории, но при этом отслеживать одно исключение — файл `driver.o` в подкаталоге `vendor_files`.

Итоги

К этому моменту вы уже должны обладать базовыми навыками по управлению файлами. Отслеживание того, где и какой файл находится в рабочем каталоге, индексе и истории репозитория, может быть непростой задачей.

Чтобы во всем разобраться, нужно уяснить важность индекса и классификаций файлов в Git. Так вы поймете все значение каталога индексирования и разберетесь, как Git классифицирует и отслеживает файлы в репозиториях. Когда вы усвоите этот принцип, то вам станут ясны технические аспекты того, как добавлять, удалять и даже исключать файлы репозитория в соответствии с целями вашего проекта.

Слияния

Git — это распределенная система контроля версий, поэтому она позволяет разработчикам в разных геолокациях вносить и записывать изменения независимо. В любой момент времени несколько разработчиков могут комбинировать свои изменения. И все это без центрального репозитория. Эта операция, объединяющая две или более линий разработки, известна в Git как *слияние* (merge).

В этой главе на простых примерах вы узнаете, как совмещать несколько линий разработки, а также изучите техники разрешения конфликтов слияния. Мы разберем стратегии слияния в Git и рассмотрим, что происходит в объектном хранилище, когда происходит операция слияния.

Слияние: техническая сторона

В Git слияние объединяет две или более истории коммитов веток. Чаще всего сливаются всего две ветки, хотя Git поддерживает такую операцию для трех и более веток одновременно.

Слияние должно происходить в пределах одного репозитория, то есть в нем должны находиться все участвующие в слиянии ветки. Как они там окажутся, неважно. (В главе 11 вы увидите, как Git предоставляет механизмы для обращения к другим репозиториям и переноса удаленных веток в текущий рабочий репозиторий.)

Когда изменения в одной ветке не конфликтуют с изменениями в другой, Git вычисляет результат слияния и создает новый коммит, представляющий новое состояние, в которое включены все изменения. Но если в ветках есть конфликты, Git эту ситуацию не разрешает. Тогда система отмечает в индексе такие конфликтные изменения как «необъединенные» (unmerged), а разработчик должен сам решить, какое из конфликтующих изменений принять (в разделе «Характеристики Git» на с. 26 в главе 1 мы говорили, что Git ожидает от вас инструкций о том, что и когда делать). Если Git не может совместить ветки автоматически, то вам нужно делать итоговый коммит после разрешения всех конфликтов.

Примеры слияния

Операция `git merge` чувствительна к контексту. Ваша текущая ветка всегда является целевой, и изменения из других веток совмещаются именно с ней.

В этом примере переключимся на *main-branch*, чтобы установить ее как текущую ветку и совместить с ней *modified-branch*:

```
$ git checkout main-branch
$ git merge modified-branch
```



Более простой способ запомнить операцию слияния в Git — представить ее с позиции *исходной ветки* и *цели*. Есть некоторые изменения в другой ветке (*исходной*), и вы хотите слить эту ветку с текущей (*целевой*).

Разберем два примера: один без конфликтов, а другой со значительными наложениями. Для упрощения будем использовать несколько веток в соответствии с методиками из главы 3.

Подготовка к слиянию

Работа в Git будет значительно проще, если начинать каждое слияние с чистого рабочего каталога и индекса. При нормальном слиянии Git создает новые версии файлов и по завершении помещает их в рабочий каталог. Во время этой операции Git использует индекс для хранения временных и промежуточных версий файлов.

Если вы изменили файлы в рабочем каталоге или изменили индекс с помощью `git add` либо `git rm`, тогда ваш репозиторий содержит «грязный» рабочий каталог и/или индекс. Если вы начнете слияние в «грязном» состоянии, то Git не сможет за один проход совместить изменения из всех веток *и* из веток вашего рабочего каталога/индекса.



Необязательно начинать с чистого каталога. Git все равно выполнит слияние, например, если участвующие в нем файлы и грязные файлы в рабочем каталоге не будут иметь общих изменений. Тем не менее начинать с чистым каталогом проще, и мы рекомендуем делать именно так.

Слияние двух веток

Настроим репозиторий с одним файлом, создадим две ветки, а затем совместим их:

```
$ mkdir merge-conflict
# Инициализация репозитория
$ git init -b main
Initialized empty Git repository in /merge-conflict/.git/
```

```
# Создание файла
$ cat > file
Line 1 stuff
Line 2 stuff
Line 3 stuff
^D

# Добавление и коммит файла
$ git add file
$ git commit -m "Initial 3 line file"
[main (root-commit) 21a352e] Initial 3 line file
 1 file changed, 3 insertions(+)
 create mode 100644 file
```

Теперь создадим еще один коммит в основной ветке:

```
$ cat > other_file
Here is stuff on another file!
^D

$ git add other_file
$ git commit -m "Another file"
[main e5d26c6] Another file
 1 file changed, 1 insertion(+)
 create mode 100644 other_file
```

Сейчас репозиторий содержит одну ветку с двумя коммитами, где в каждом коммите добавлен новый файл. Теперь переключимся на другую ветку и изменим первый файл.

```
$ git checkout -b alternate main^
Switched to a new branch "alternate"

$ git show-branch
* [alternate] Initial 3 line file
! [main] Another file
--
+ [main] Another file
*+ [alternate] Initial 3 line file
```

Здесь ветка `alternate` изначально ответвляется от коммита `main^`, находящегося на один коммит позади текущей точки `HEAD`.

Мы внесем в этот файл еще одно изменение, а затем зафиксируем его. Помните, лучше фиксировать значительные изменения и начинать слияние с пустым рабочим каталогом:

```
$ cat >> file
Line 4 alternate stuff
^D

$ git commit -a -m "Add alternate's line 4"
[alternate 97e7ffe] Add alternate's line 4
 1 file changed, 1 insertion(+)
```

Теперь у нас есть две ветки, каждая со своей линией разработки. В ветку `main` был добавлен второй файл, а в ветку `alternate` внесено изменение. Поскольку эти два изменения не влияют на одни и те же части общего файла, слияние должно пройти без проблем.

Поскольку операция `git merge` чувствительна к контексту, нужно сделать так, чтобы ветка `main` была целевой и изменения в ветке `alternate` сливались в целевую ветку. Прежде чем продолжать, переключимся на `main`:

```
$ git checkout main
Switched to branch "main"

# Проверка чистоты рабочего каталога и индекса
$ git status
On branch main
nothing to commit, working tree clean

# Все хорошо, можно делать слияние

# Когда вы нажимаете ввод, сохранить и продолжить, Git выводит
# в редакторе предустановленное сообщение о слиянии
$ git merge alternate
Merge made by the 'ort' strategy.
  file | 1 +
  1 file changed, 1 insertion(+)
```

Чтобы увидеть результат операции, можно использовать другой инструмент, входящий в `git log`. Он предназначен для просмотра графа коммитов:

```
$ git log --graph --decorate --pretty=oneline --abbrev-commit

* 2dce921 (HEAD -> main) Merge branch 'alternate'
|\
| * 97e7ffe (alternate) Add alternate's line 4
* | e5d26c6 Another file
|/
* 21a352e Initial 3 line file
```

Это тот же граф коммитов, который описывался в разделе «Графы коммитов» на с. 112 в главе 4. Разница лишь в том, что он повернут на 90 градусов и последние коммиты находятся сверху, а не справа. Две его ветки разделились у первого коммита, `21a352e`; каждая показывает один коммит (`e5d26c6` и `97e7ffe`); позднее эти ветки снова сходятся в коммите `2dce921`.



Использование `git log --graph` — отличная альтернатива графическим инструментам вроде `gitk`. Визуализация, которую представляет `git log --graph`, хорошо подходит для простых терминалов, например, когда вы подключаетесь к серверу по SSH.

Git выполняет каждое слияние симметрично, чтобы создать один идентичный, объединенный коммит, который добавляется в вашу текущую ветку. Вторая ветка не затрагивается слиянием. Поскольку коммит слияния добавляется только в текущую ветку, то вы сливаете исходную ветку в эту целевую ветку.

Когда слияние ведет к конфликту

Операция слияния может продолжаться без запросов к пользователю, когда Git успешно разрешает различия между сторонами слияния. Но поскольку эта операция может получать любое количество независимых, различающихся и потенциально конфликтующих строк, иногда вмешательство пользователя все же требуется.

Рассмотрим сценарий, в котором слияние ведет к конфликту. Возьмем слияние из предыдущего раздела и внесем независимые конфликтующие изменения в ветки `main` и `alternate`. Затем выполним слияние ветки `alternate` с веткой `main`, столкнемся с конфликтом, разрешим его и зафиксируем итоговый результат.

В ветке `main` создадим новую версию файла `file` с несколькими дополнительными строками, после чего зафиксируем эти изменения:

```
$ git checkout main

$ cat >> file
Line 5 stuff
Line 6 stuff
^D

$ git commit -a -m "Add line 5 and 6"
[main 6c5634c] Add line 5 and 6
 1 file changed, 2 insertions(+)
```

В ветке `alternate` изменим тот же файл по-другому. В ветку `main` мы зафиксировали изменения, но ветка `alternate` еще не обновлялась:

```
$ git checkout alternate
Switched to branch "alternate"

$ git show-branch
* [alternate] Add alternate's line 4
! [main] Add line 5 and 6
--
+ [main] Add line 5 and 6
*+ [alternate] Add alternate's line 4

# В этой ветке "file" содержит коммит "Line 4 alternate stuff"

$ cat >> file
Line 5 alternate stuff
Line 6 alternate stuff
^D

$ cat file
```

```
Line 1 stuff
Line 2 stuff
Line 3 stuff
Line 4 alternate stuff
Line 5 alternate stuff
Line 6 alternate stuff

$ git diff
diff --git a/file b/file
index a29c52b..802acf8 100644
--- a/file
+++ b/file
@@ -2,3 +2,5 @@ Line 1 stuff
   Line 2 stuff
   Line 3 stuff
   Line 4 alternate stuff
+Line 5 alternate stuff
+Line 6 alternate stuff

$ git commit -a -m "Add alternate line 5 and 6"
[alternate 0ca87a4] Add alternate line 5 and 6
 1 file changed, 2 insertions(+)
```

Рассмотрим этот сценарий. История текущей ветки выглядит так:

```
$ git show-branch
* [alternate] Add alternate line 5 and 6
! [main] Add line 5 and 6
--
* [alternate] Add alternate line 5 and 6
+ [main] Add line 5 and 6
*+ [alternate^] Add alternate's line 4
```

Чтобы продолжить, переключимся на ветку `main` и попробуем выполнить слияние:

```
$ git checkout main
Switched to branch "main"

$ git merge alternate
Auto-merging file
CONFLICT (content): Merge conflict in file
Automatic merge failed; fix conflicts and then commit the result.
```

Когда возникает конфликт слияния, следует изучить место его возникновения с помощью команды `git diff`. Здесь у одного файла с именем `file` есть конфликт в его содержимом:

```
$ git diff
diff --cc file
index 4d77dd1,802acf8..0000000
--- a/file
+++ b/file
```



```

@@@ -2,5 -2,5 +2,10 @@@ Line 1 stuf
    Line 2 stuff
    Line 3 stuff
    Line 4 alternate stuff
++<<<<<< HEAD:file
+Line 5 stuff
+Line 6 stuff
+=====
+ Line 5 alternate stuff
+ Line 6 alternate stuff
++>>>>>> alternate:file

```

Команда `git diff` показывает различия файлов в рабочем каталоге и индексе. В традиционном стиле вывода команды `diff` измененное содержимое представлено между `<<<<<<` и `=====`, а альтернативное — между `=====` и `>>>>>>`. Однако в комбинированном формате `diff` используются дополнительные знаки «плюс» и «минус», чтобы обозначить изменения из нескольких источников относительно итоговой версии.

Предыдущий вывод показывает, что конфликт относится к строкам 5 и 6, где в двух ветках намеренно вносились разные изменения. Вам предстоит разрешить этот конфликт, и вы можете выбрать любой вариант. Например, взять строки только из одной или только из другой ветки, комбинацию из обеих веток или создать что-то совершенно новое. И хотя последний вариант не является истинным разрешением конфликта, он все равно допустим.

В нашем примере мы выберем по одной строке из каждой ветки для составления нашей исправленной версии. Отредактированный файл теперь выглядит так:

```

$ cat file
Line 1 stuff
Line 2 stuff
Line 3 stuff
Line 4 alternate stuff Line 5 stuff
Line 6 alternate stuff

```

Если вас устраивает такое разрешение конфликта, отметьте его как разрешенный. Для этого добавьте изменения в индекс с помощью команды `git add`, подготавливая файл к следующему коммиту:

```
$ git add file
```

Когда вы разрешили конфликты и проиндексировали итоговые версии каждого файла с помощью `git add`, нужно зафиксировать слияние через `git commit`. Git переместит вас в редактор по умолчанию с шаблонным сообщением:

```
Merge branch 'alternate'
```

```

# Конфликты:
#     file
#

```

```
# Похоже, вы фиксируете слияние.
# Если это не так, выполните
#   git update-ref -d MERGE_HEAD
# и повторите попытку.

# Введите сообщение коммита для изменений. Строки, начинающиеся с '#',
# будут проигнорированы; пустое сообщение отменит коммит.
#
# В ветке main
# Все конфликты исправлены, но вы все еще производите слияние
#
# Изменения для коммита:
#   modified:   file
#
```

Строки, начинающиеся с решетки (#), представляют собой комментарии и предназначены исключительно для справки. Все строки комментариев исключаются из итогового сообщения. Можете изменять или дополнять сообщение коммита на свое усмотрение, например, добавляя примечание о том, как конфликт был разрешен.

Когда вы выходите из редактора, Git укажет на успешное создание нового коммита слияния:

```
$ git commit
# Редактирование сообщения коммита слияния
[main f1fa82f] Merge branch 'alternate'

$ git show-branch
! [alternate] Add alternate line 5 and 6
* [main] Merge branch 'alternate'
--
- [main] Merge branch 'alternate'
+* [alternate] Add alternate line 5 and 6
```

Посмотреть итоговый коммит слияния (вершину, или HEAD коммит) можно с помощью команды:

```
$ git log
```

Работа с конфликтами слияния

В предыдущем примере мы рассмотрели ситуацию, когда конфликтующие изменения нельзя слить автоматически.

Создадим еще один сценарий с конфликтом слияния, чтобы изучить инструменты, которые Git предоставляет для разрешения конфликтов. Начнем с обычного файла `hello`, содержащего только слово `hello`, и создадим две ветки с двумя вариантами этого файла:

```
$ git init -b main
Initialized empty Git repository in /tmp/conflict/.git/

$ echo hello > hello
$ git add hello
$ git commit -m "Initial hello file"
[main (root-commit) e8a1845] Initial hello file
 1 file changed, 1 insertion(+)
 create mode 100644 hello

$ git checkout -b alt
Switched to a new branch "alt"
$ echo world >> hello
$ echo 'Yay!' >> hello
$ git commit -a -m "One world"
[alt 84e436c] One world
 1 file changed, 2 insertions(+)

$ git checkout main
$ echo worlds >> hello
$ echo 'Yay!' >> hello
$ git commit -a -m "All worlds"
[main 0d7dfb7] All worlds
 1 file changed, 2 insertions(+)
```

Теперь у нас есть одна ветка с файлом `hello` и содержимым “world” и вторая — с другой версией этого же файла, содержащей “worlds”.

Если переключиться на ветку `main` и попробовать слить в нее ветку `alt`, то возникнет конфликт:

```
$ git merge alt
Auto-merging hello
CONFLICT (content): Merge conflict in hello
Automatic merge failed; fix conflicts and then commit the result.
```

Как и ожидалось, Git предупреждает о конфликте в файле `hello`.

Обнаружение конфликтующих файлов

А что делать, если полезные указания Git выходят за пределы экрана? Или у вас есть множество файлов с конфликтами? К счастью, Git отслеживает проблемные файлы, отмечая каждый в индексе как конфликтующий (`conflicted`) или необъединенный (`unmerged`).

Можно использовать команду `git status` или `git ls-files -u` для вывода набора файлов, остающихся необъединенными в рабочем каталоге. Команда `git status` подсказывает, что делать для разрешения конфликта и завершения слияния:

```
$ git status
On branch main
```

```
You have unmerged paths.
(fix conflicts and run "git commit")
(use "git merge --abort" to abort the merge)
```

```
Unmerged paths:
(use "git add <file>..." to mark resolution)
   both modified:   hello
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

```
$ git ls-files -u
100644 ce013625030ba8dba906f756967f9e9ca394464a 1    hello
100644 e63164d9518b1e6caf28f455ac86c8246f78ab70 2    hello
100644 562080a4c6518e1bf67a9f58a32a67bff72d4f00 3    hello
```

Мы объясним, почему образовались три различные версии файла **hello**, в разделе «Как Git отслеживает конфликты» далее в этой главе, на с. 158.

Используйте `git diff`, чтобы увидеть, что еще не было слито. Эта команда также покажет все подробности конфликта. Далее узнаем, как изучать файл с конфликтами.

Проверка конфликтов

Когда возникает конфликт, копия рабочего каталога каждого конфликтующего файла дополняется *маркерами слияния* (или *маркерами разрешения конфликта*, когда разрешается конфликт слияния).

Вернемся к нашему примеру. Теперь конфликтующий файл выглядит так:

```
$ cat hello
hello
<<<<<<< HEAD
worlds
=====
world
>>>>>>> alt
Yay!
```

Маркеры слияния разделяют две возможные версии конфликтующей части файла. В одной версии содержится «worlds», а в другой — «world». Разделение с помощью маркеров слияния показано на рис. 6.1.



Линии маркеров слияния (<<<<<<<, ===== и >>>>>>>) генерируются автоматически, но они предназначены только для того, чтобы их прочитали вы, а не программа. После разрешения конфликтов маркеры следует удалять в редакторе. Для проверки запустите команду `git diff -check`: она подсветит все маркеры, которые еще не удалены.


```
+ world
++>>>>>> alt
  Yay!
```

Что это означает? Это простая комбинация двух операций `diff` (выявлений различий): для первого родителя, `HEAD`, и для второго, `alt`. (Не удивляйтесь, если второй родитель имеет абсолютное имя SHA-1, представляющее неименованный коммит из другого репозитория.) Для упрощения Git также присваивает второму родителю особое имя `MERGE_HEAD`.

Вы можете сравнить версии `HEAD` и `MERGE_HEAD` с «объединенной» (`merged`) версией из рабочего каталога:

```
$ git diff HEAD
diff --git a/hello b/hello index e63164d..1f2f61c 100644
--- a/hello
+++ b/hello
@@ -1,3 +1,7 @@
  hello
+<<<<<<< HEAD
+worlds
+=====
+world
+>>>>>> alt
+  Yay!

$ git diff MERGE_HEAD
diff --git a/hello b/hello
index 562080a..1f2f61c 100644
--- a/hello
+++ b/hello
@@ -1,3 +1,7 @@
  hello
+<<<<<<< HEAD
+worlds
+=====
+world
+>>>>>> alt
+  Yay!
```

Обратите внимание, что строка, начинающаяся с `index`, содержит различные сокращенные идентификаторы SHA1.

В качестве альтернативы можно использовать команду `git diff --ours`, являющуюся синонимом для `git diff HEAD`, поскольку показывает отличия между «нашей» (`ours`) версией и объединенной. Аналогичным образом, `git diff MERGE_HEAD` можно записать как `git diff --theirs` (отличия между «их» версией и объединенной).

Если сравнить строка за строкой результаты двух операций `diff`, то весь текст, за исключением столбцов `+`, будет одинаковым. Поэтому Git выводит основной текст всего один раз и располагает столбцы `+` рядом. Это показано на рис. 6.2.



Используйте `git diff --base`, чтобы увидеть объединенный набор изменений с момента базового слияния. Иначе команда будет записана весьма громоздко:

```
$ git diff $(git merge-base HEAD MERGE_HEAD)
```

<pre>\$ git diff HEAD diff --git a/hello b/hello index e63164d..1f2f61c 100644 --- a/hello +++ b/hello @@ -1,3 +1,7 @@ hello + <<<<<< HEAD + worlds + ===== + world + >>>>>> alt + Yay!</pre>	<pre>\$ git diff MERGE_HEAD diff --git a/hello b/hello index 562080a..1f2f61c 100644 --- a/hello +++ b/hello @@ -1,3 +1,7 @@ hello + <<<<< HEAD + worlds + ===== + world + >>>>>> alt + Yay!</pre>
---	---

```
$ git diff
diff --cc hello
index e63164d,562080a..0000000
--- a/hello
+++ b/hello
@@@ -1,3 -1,3 +1.7 @@@
hello
++ <<<<<< HEAD
++ worlds
++ =====
++ world
++ >>>>>> alt
++ Yay!
```

Рис. 6.2. Сопоставление совмещенных diff

В конфликте, обнаруженном `git diff`, есть два столбца информации, которые добавляются к каждой строке вывода. Знак «плюс» (+) в столбце указывает на добавление строки, знак «минус» (-) — на ее удаление, а пустое пространство обозначает строку без изменений. Первый столбец показывает, что конкретно изменяется относительно вашей версии, второй — что изменяется относительно другой. Линии маркеров конфликта новые в обеих версиях, поэтому отмечены ++. Строки `world` и `worlds` новые только в одной версии или в другой, поэтому содержат лишь один + в соответствующем столбце.

Допустим, вы отредактируете файл, чтобы выбрать третий вариант, вот таким образом:

```
$ cat hello
hello
worldly ones
Yay!
```

Тогда новый вывод `git diff` будет следующим:

```
$ git diff
diff --cc hello
index e63164d,562080a..0000000
--- a/hello
+++ b/hello
@@@ -1,3 -1,3 +1,3 @@@
     hello
-   worlds
-   world
++worldly ones
     Yay!
```

При желании вы можете выбрать одну или другую исходную версию:

```
$ cat hello
hello
world
Yay!
```

В таком случае вывод `git diff` будет таким:

```
$ git diff
diff --cc hello
index e63164d,562080a..0000000
--- a/hello
+++ b/hello
```

Подождите! Здесь произошло нечто странное. Где показано, что в базовую версию была добавлена строка `world`? А где показано, что строка `worlds` удалена из версии `HEAD`? Поскольку вы разрешили конфликт в пользу версии `MERGE_HEAD`, Git намеренно опускает эту `diff`, решая, что этот раздел вас больше не интересует.

Запуск `git diff` в конфликтующем файле показывает только те части, в которых действительно есть конфликт. В объемном файле со множеством разбросанных по нему правок большинство изменений не вступают в конфликт. Одна сторона слияния изменила одни части файла, вторая сторона — другие части. Когда вы работаете над разрешением конфликта, вас не интересуют эти части, поэтому `git diff` обрезает их с помощью простой эвристики: если раздел файла содержит изменения, сделанные только одной стороной, то его не показывают.

Эта оптимизация имеет немного сбивающий с толку побочный эффект: как только вы разрешите нечто, что *являлось* конфликтом, просто выбрав одну или другую сторону, часть файла, которая была конфликтной, перестает отображаться.

На самом деле это скорее побочный эффект реализации, чем намеренная функциональность, но это удобно: `git diff` показывает только те части файла, в которых еще остались конфликты, так что вы можете отслеживать те, что еще не разрешили.

git log с конфликтами

При разрешении конфликта вы можете использовать особые опции `git log`, чтобы выяснить, где именно возникли изменения и почему. Попробуйте такой вариант:

```
$ git log --merge --left-right -p
```

```
commit < 0d7dfb7f23cd5f4d6debedc27f7f85d5c72d5423 (HEAD -> main)
Author: Prem Kumar Ponuthurai <ppremk@demo.com>
Date: Thu Sep 2 23:10:35 2021 +0200
```

All worlds

```
diff --git a/hello b/hello
index ce01362..e63164d 100644
--- a/hello
+++ b/hello
@@ -1,3 @@
hello
+worlds
+Yay!
```

```
commit > 84e436cd3353739c67fd0fce36e18fc42b31d153 (alt)
Author: Prem Kumar Ponuthurai <ppremk@demo.com>
Date: Thu Sep 2 23:09:07 2021 +0200
```

One world

```
diff --git a/hello b/hello
index ce01362..562080a 100644
--- a/hello
+++ b/hello
@@ -1,3 @@
hello
+world
+Yay!
```

Эта команда показывает все коммиты в обеих частях истории, затрагивающие конфликтующие файлы, вместе с фактическими изменениями, внесенными каждым коммитом. Если вам интересно, когда, почему, как и кем в файл была добавлена строка `worlds`, используйте эту команду.

Опции `git log`:

- `--merge` показывает только коммиты, связанные с файлами, вызвавшими конфликт;
- `--left-right` выводит `<`, если коммит был с «левой» стороны слияния («наша» версия, с которой мы начали) или `>`, если коммит происходил из «правой» стороны («их» версия, в которую происходит слияние);
- `-p` показывает сообщение коммита и патч, связанный с каждым коммитом.

Если у вашего репозитория более сложная структура и в конфликте находятся несколько файлов, то вы также можете указать в виде опции командной строки имена интересующих вас файлов:

```
$ git log --merge --left-right -p hello
```

Для учебных целей приведенные здесь примеры были очень простыми. В реальных ситуациях все будет сложнее. Одна из техник разрешения обширных конфликтов при масштабных слияниях — это использование нескольких небольших коммитов с четко определенными эффектами.



Git отлично обрабатывает небольшие коммиты, поэтому не нужно ждать до последнего, чтобы зафиксировать обширные изменения. Небольшие коммиты и частые слияния уменьшают проблемы, связанные с разрешением сложных конфликтов.

Как Git отслеживает конфликты

Git отслеживает всю информацию о конфликтующем слиянии, записывая ее в следующих областях:

- `.git/MERGE_HEAD` содержит SHA-1 коммита, в который происходит слияние. Вам не нужно использовать этот SHA-1 самим — достаточно будет упоминания `MERGE_HEAD`, чтобы Git заглянула в этот файл.
- `.git/MERGE_MSG` содержит предустановленное сообщение о слиянии, используемое при выполнении `git commit` после разрешения конфликтов.
- Индекс Git содержит три копии каждого конфликтующего файла: базу слияния, «нашу» версию и «их» версию. Этим трем копиям присваиваются номера 1, 2 и 3 соответственно.
- Конфликтующая версия (маркеры слияния и прочее) не хранится в индексе. Она помещается в файл, находящийся в рабочем каталоге. Когда вы выполняете `git diff` без параметров, всегда происходит сравнение того, что находится в индексе и рабочем каталоге.

Чтобы посмотреть, как сохраняются записи индекса, используйте низкоуровневую команду `git ls-files`:

```
$ git ls-files -s
100644 ce013625030ba8dba906f756967f9e9ca394464a 1    hello
100644 e63164d9518b1e6caf28f455ac86c8246f78ab70 2    hello
100644 562080a4c6518e1bf67a9f58a32a67bff72d4f00 3    hello
```

Опция `-s` в `git ls-files` выводит *все* файлы со *всеми* стадиями. Если вы хотите видеть только конфликтующие файлы, используйте опцию `-u`.

Файл `hello` сохранен три раза, и каждая его версия имеет свой хеш. Конкретную версию вы можете просмотреть с помощью `git cat-file`:

```
$ git cat-file -p e63164d951
hello
worlds
Yay!
```

Вы можете дополнить `git diff` специальным синтаксисом для сравнения разных версий файла. Например, чтобы посмотреть, что изменилось между базой слияния и версией, в которую выполняется слияние, выполните команду:

```
$ git diff :1:hello :3:hello
diff --git a/:1:hello b/:3:hello
index ce01362..562080a 100644
--- a/:1:hello
+++ b/:3:hello
@@ -1,3 @@
 hello
+world
+Yay!
```

Использование номеров стадий для именования версий отличается от `git diff --theirs`, которая показывает отличия между их версией и итоговой объединенной (либо все еще конфликтующей) версией в рабочем каталоге после слияния. Объединенная версия пока не находится в индексе, поэтому у нее даже нет номера.



Если вы точно знаете, какую версию файла использовать для быстрого разрешения конфликта, то можете выполнить команду `git checkout` с опциями `--ours` («наша») или `--theirs` («их»). Это позволит просто переключиться на файл с одной или другой стороны конфликтующего сообщения. Обратите внимание, что эти опции можно использовать только во время разрешения конфликта.

Вернемся к примеру. Поскольку мы полностью отредактировали и разрешили версию в рабочем каталоге в пользу версии `theirs`, теперь отличий быть не должно:

```
$ cat hello
hello
world
Yay!

$ git diff --theirs
* Unmerged path hello
```

Остается только *напоминание о необъединенном пути файла*, который нужно добавить в индекс.

Последний этап разрешения конфликта

Внесем последнее изменение в файл `hello`, прежде чем объявить его объединенным:

```
$ cat hello
hello
everyone
Yay!
```

Теперь, когда файл прошел полноценное слияние, команда `git add` снова уменьшит индекс до одной копии `hello`:

```
$ git add hello
$ git ls-files -s
100644 ebc56522386c504db37db907882c9dbd0d05a0f0 0      hello
```

Обособленный `0` между SHA-1 и именем файла говорит о том, что номер стадии неконфликтующего файла — ноль.

Вам нужно обработать все конфликтующие файлы, отмеченные в индексе. Пока остаются неразрешенные конфликты, зафиксировать коммит не удастся. Когда конфликты будут устранены, выполните `git add` (или `git rm`, `git update-index` и т. д.) для файла, чтобы очистить его конфликтующий статус.



Не добавляйте файлы в индекс (через `git add`), если в них еще остались маркеры конфликта (выполнение `git diff --check` покажет оставшиеся маркеры слияния). Хотя это очистит конфликт в индексе и позволит выполнить коммит, сам файл останется некорректным.

Наконец, вы можете зафиксировать конечный результат с помощью `git commit` и использовать `git show` для просмотра коммита слияния:

```
$ cat .git/MERGE_MSG
Merge branch 'alt'

# Conflicts:
#      hello

$ git commit
[main d67ecec] Merge branch 'alt'

$ git show
commit d67ecec075281cebb32a9eedd76242b6e9ac76a8 (HEAD -> main)
Merge: 0d7dfb7 84e436c
Author: Prem Kumar Ponuthorai <ppremk@demo.com>
Date:   Sun Sep 5 21:49:53 2021 +0200

    Merge branch 'alt'

diff --cc hello
index e63164d,562080a..ebc5652
--- a/hello
```

```
+++ b/hello
@@@ -1,3 -1,3 +1,3 @@@
hello
- worlds
-world
++everyone Yay!
```

В этом коммите есть два интересных момента.

- В заголовке есть новая строка — `Merge:`. Обычно в `git log` или `git show` не нужно показывать родителя коммита, поскольку он всего один. Но коммиты слияния обычно имеют двух (а иногда более) родителей, которые важны для понимания слияния. Поэтому `git log` и `git show` всегда выводят SHA-1 каждого предка.
- Выводимые различия файлов в случае коммита слияния имеют не такой формат, как обычные `diff`. Они всегда выводятся в объединенном виде, или в виде «конфликтующего слияния». Успешное слияние в Git — это комбинация других изменений, которые уже произошли в истории. В связи с этим в выводе содержимого коммита слияния показывается не весь набор изменений, а только те части, которые отличаются от одной из объединенных веток. Здесь также указывается имя файла.

Отмена или перезапуск слияния

Если вам не удалось разрешить конфликт и вы хотите вернуться к исходному состоянию, используйте такую команду:

```
$ git checkout -m
```

Она восстановит индекс до версий файла, которые были отмечены как конфликтующие (вы видите их при выполнении `git ls-files -u`).

Если вы начали операцию слияния, но затем вдруг передумали ее завершать, Git предоставляет простой способ отмены. Для этого до запуска итоговой команды слияния `git commit` выполните:

```
$ git merge --abort
```

Эта команда восстановит рабочий каталог и индекс до состояния, предшествующего выполнению `git merge`.

Если вы хотите отменить слияние после его завершения (то есть после того, как оно внесло новый коммит слияния), используйте следующую команду:

```
$ git reset --hard ORIG_HEAD
```

Прежде чем начинать операцию слияния, Git сохраняет `HEAD` исходной ветки в ссылке `ORIG_HEAD` конкретно для этой цели.

Но будьте внимательны. Если вы начали слияние с «грязным» рабочим каталогом и индексом, то можете столкнуться с проблемами и утратить незафиксированные изменения, находящиеся в рабочем каталоге.

Можно инициировать запрос `git merge` и с грязным рабочим каталогом, но если вы выполните `git reset --hard`, то грязное состояние до начала слияния не будет полноценно восстановлено. Сброс приведет к его утрате. С помощью `--hard` вы запрашиваете сброс до состояния `HEAD` (см. раздел «Использование `git reset`» на с. 213 в главе 9).

Стратегии слияния

До сих пор примеры были простыми, поскольку включали всего две ветки. Может показаться, что излишняя сложность используемой в Git истории в виде направленного ациклического графа и длинные трудно запоминаемые ID коммитов себя не оправдывают. Но дело в том, что они предназначены не для таких элементарных случаев, поэтому дальше разберем пример посложнее.

Представьте, что над вашим репозиторием трудится не один, а три человека. Чтобы слишком не мудрить, предположим, что каждый из них — Алиса, Боб и Кэл — может вносить изменения в виде коммитов в три отдельные одноименные ветки одного репозитория (Alice, Bob и Cal).

Разработчики трудятся над отдельными ветками, при этом Алиса управляет интеграцией этой работы. Каждый разработчик может использовать результаты других, производя непосредственное слияние с их ветками.

В итоге они создают репозиторий с историей коммитов (рис. 6.3).

Представьте, что Кэл начал проект, к которому присоединилась Алиса. Она поработала над проектом какое-то время, а затем к ним присоединился Боб. Все это время Кэл работал отдельно над собственной версией.

В итоге Алиса объединила свои изменения с изменениями Боба (то есть сделала слияние ветки Alice в ветку Bob), а Боб продолжил работать без слияния изменений Алисы со своим основным деревом. Теперь у них возникли три истории ветвления (рис. 6.4).

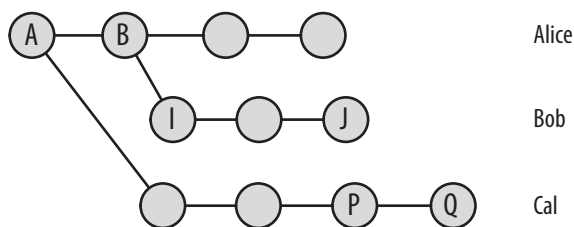


Рис. 6.3. Потенциальная конфигурация слияний

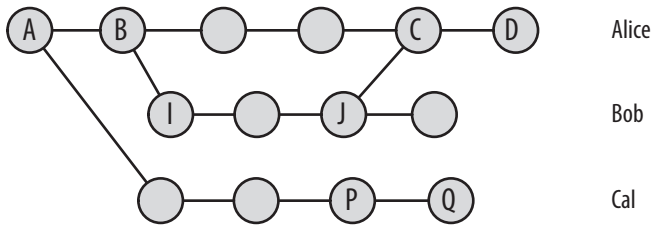


Рис. 6.4. После слияния изменений Алисы в ветку Боба

Теперь представим, что Боб хочет получить последние изменения Кэла. На этом этапе схема уже начинает усложняться. Проследите дерево, начиная от Боба, через Алису и до тех пор, пока не достигнете точки, в которой произошло первое ответвление от линии разработки Кэла. Это А, основа слияния между Бобом и Кэлом. Чтобы слить ветку Кэла, Бобу нужно взять набор изменений между основной слияния, А, и последним состоянием ветки Кэла, Q, и выполнить их трехстороннее слияние в собственное дерево, сформировав коммит К. В результате получится история, показанная на рис. 6.5.

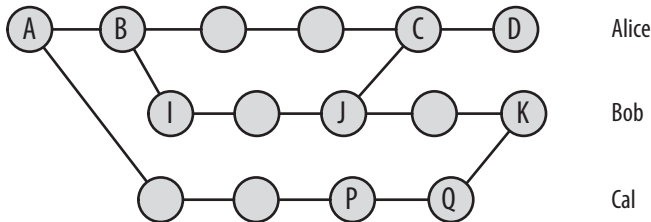


Рис. 6.5. После слияния линии Боба с линией Кэла



Вы всегда можете найти основу слияния между двух и более веток, используя `git merge-base`. При этом для множества веток может быть более одной допустимой основы.

Пока все хорошо.

Теперь Алисе тоже нужно получить последние изменения из дерева Кэла, но она не знает, что Боб уже слил его дерево в свое. Поэтому Алиса просто делает то же самое. Это еще одна простая операция, поскольку очевидно, где Алиса ответвилась от дерева Кэла. Итоговая история показана на рис. 6.6.

Далее Алиса понимает, что Боб проделал определенную работу, L, и хочет произвести слияние с внесенными им изменениями. Какой на этот раз будет основа слияния (между L и E)?

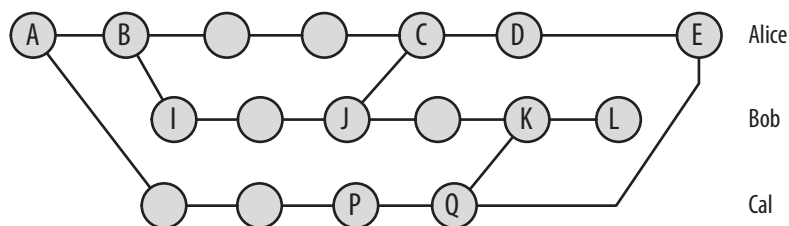


Рис. 6.6. После слияния дерева Алисы и Кэла

К сожалению, ответ неоднозначен. Если вы проследите историю по дереву, то можете решить, что удачной точкой будет исходная ревизия от Кэла. Но на самом деле это бессмысленно: сейчас и у Алисы, и у Боба есть *последняя* ревизия. Если вы запросите изменения с момента исходной ревизии Кэла и до последней ревизии Боба, то они также будут включать последние изменения Кэла, которые у Алисы уже есть и которые наверняка приведут к конфликту слияния.

А что, если использовать в качестве основы последнюю ревизию Кэла? Это уже лучше, но все же не совсем правильно: если взять отличия между последними ревизиями Кэла и Боба, то вы получите *все* изменения Боба. Но у Алисы уже есть *некоторые* из этих изменений, поэтому и здесь наверняка возникнет конфликт слияния.

А что, если использовать версию, которую Алиса слила от Боба, то есть J? Вывод отличий между ней и последней ревизией Боба будет включать только последние изменения Боба, которые нам и нужны. Но сюда же будут включены и изменения от Кэла, которые у Алисы уже есть.

Что же делать?

Такие ситуации называются *перекрестным слиянием* (criss-cross merge), поскольку изменения сливаются между ветками в обе стороны. Если бы они перемещались только в одном направлении (например, от Кэла к Алисе и Бобу, но никогда от Боба к Алисе или от Алисы к Кэлу), тогда слияние было бы простым. К сожалению, не всегда все складывается так гладко.

Разработчики Git изначально написали простой механизм для объединения двух веток с помощью коммита слияния, но сценарии вроде описанного выше вскоре сделали очевидной необходимость в более продуманном подходе. В итоге разработчики обобщили, параметризовали и ввели альтернативные, настраиваемые стратегии слияния для обработки различных сценариев.

Разберем эти стратегии и их применение.

Вырожденные слияния

Есть два типичных сценария, которые ведут к вырожденным (degenerate) слияниям, — *already up to date* и *fast-forward*. Поскольку ни в одном из них после

выполнения `git merge`¹ коммит не вносится, некоторые читатели могут решить, что это не является стратегией слияния в истинном смысле.

Already up-to-date (уже последняя версия)

Когда все коммиты из другой ветки (ее HEAD) уже есть в целевой ветке, про целевую ветку говорят, что она актуальна, то есть версия — *уже последняя*. В результате никакие коммиты в нее не добавляются.

Например, если вы выполните слияние и тут же повторите аналогичный запрос, то получите сообщение, что ветка *already up to date*.

```
# Показать, что alt готова к слиянию с main
$ git show-branch
! [alt] One world
* [main] Merge branch 'alt'
--
- [main] Merge branch 'alt'
+* [alt] One world

# Попытаться снова совместить alt с main
$ git merge alt
Already up to date.
```

На рис. 6.7, 6.8 и 6.9 показан описанный сценарий.

Fast-forward (перемотка вперед)

Слияние *fast-forward*, то есть «с перемоткой вперед» (или просто «перемотка»), происходит, когда HEAD ветки уже полностью имеется в другой ветке и представлена в ней. Этот случай противоположен ситуации *already-up-to-date*.

Поскольку HEAD уже есть в другой ветке (возможно, из-за общего предка), Git просто прикрепляет к этой HEAD новые коммиты из этой другой ветки, после чего перемещает HEAD вашей ветки на последний, новый коммит. Естественно, индекс и рабочий каталог также соответственным образом корректируются, чтобы отражать состояние последнего коммита.

Случай перемотки чаще всего встречается при отслеживании веток, поскольку они просто получают и записывают удаленные коммиты из других репозиториях. HEAD ваших веток локального отслеживания будут всегда полноценно присутствовать и представлены, потому что в этом случае HEAD ветки располагается после *последней* операции получения. Подробнее об этом в главе 11.

¹ Да, вы можете принудить Git все равно создать коммит, используя опцию `--no-ff` в случае *fast-forward* слияния. Но вы должны понимать, для чего это делаете.

Этот принцип показан на рис. 6.10 и 6.11.

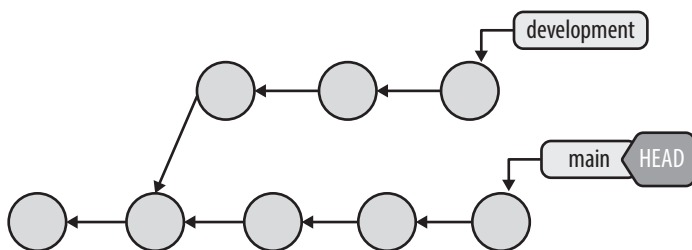


Рис. 6.7. Слияние двух веток

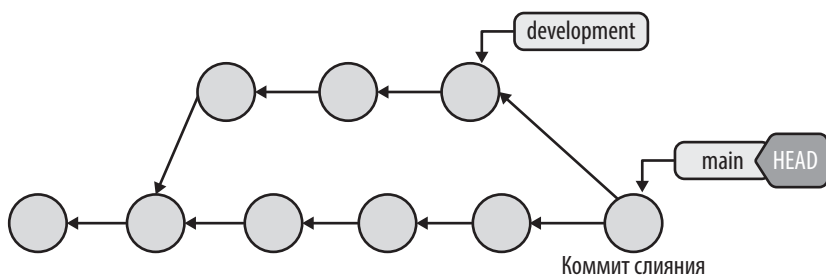


Рис. 6.8. Коммит слияния

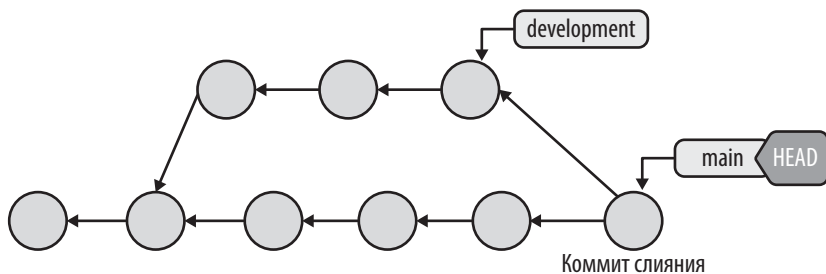


Рис. 6.9. Снова коммит слияния (уже последняя версия)

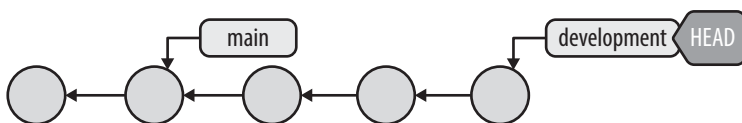


Рис. 6.10. До слияния с перемоткой

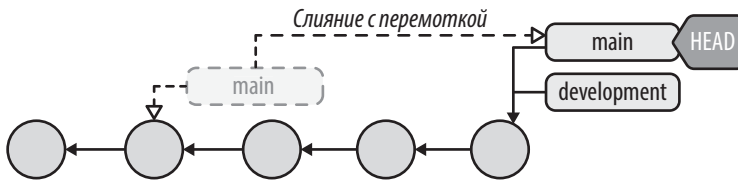


Рис. 6.11. После слияния с перемоткой

Для Git важно обрабатывать эти случаи без внесения фактических коммитов. Представьте, что произойдет в случае перемотки, если Git вдруг создаст коммит. Слияние ветки А и В сначала приведет к ситуации, показанной на рис. 6.12. Затем слияние В и А приведет к тому, что изображено на рис. 6.13, а очередное обратное слияние — к тому, что показано на рис. 6.14.

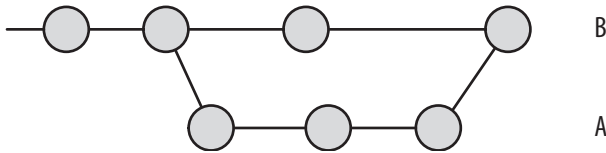


Рис. 6.12. Первое несходящееся слияние

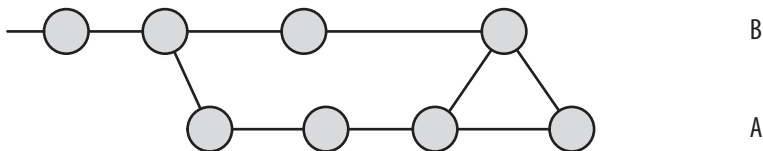


Рис. 6.13. Второе несходящееся слияние

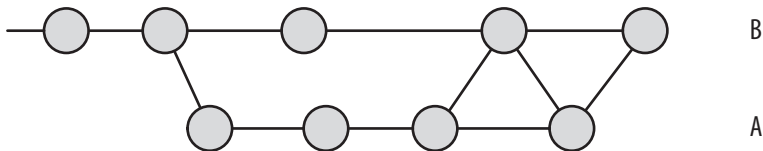


Рис. 6.14. Третье несходящееся слияние

Каждое очередное слияние — это новый коммит, поэтому такая последовательность никогда не сойдется к устойчивому состоянию и не покажет, что две ветки идентичны.

Нормальные слияния

Перечисленные ниже стратегии слияния дают итоговый коммит — *коммит слияния*, который добавляется в текущую ветку и представляет совмещенное состояние после слияния.

Разрешающая стратегия (Resolve)

Работает только для двух веток. Использует их общих предков в качестве основы слияния и выполняет прямое трехстороннее слияние через применение к текущей ветке изменений, произошедших с момента основы слияния и до вершины HEAD другой ветки.

Рекурсивная стратегия (Recursive)

Аналогична разрешающей стратегии в том, что может объединять только две ветки за раз. Предназначена для обработки сценариев, где между двумя ветками есть более одной основы слияния. В этом случае Git формирует временное слияние всех общих основ и использует полученный результат в качестве основы, из которой производит итоговое слияние двух заданных веток посредством нормального алгоритма трехстороннего слияния.

Затем временная основа слияния отбрасывается, и итоговое состояние фиксируется в целевую ветку.

Стратегия «осьминог» (Octopus)

Позволяет одновременное слияние более двух веток. Принцип довольно прост и подразумевает вызов рекурсивной стратегии множество раз, по одному для каждой совмещаемой ветки.

Тем не менее эта стратегия не может обработать слияние, требующее разрешения конфликта с участием пользователя. В таком случае вам придется выполнить серию обычных слияний, разрешая конфликты по одному за раз.

Стратегия Merge-ort

Реализует рекурсивную стратегию, но под капотом написана совершенно по-другому. Дает значительный прирост скорости и разрешает давно известные проблемы, касающихся неточных результатов слияния.

Для этого в ней задействуется кэширование и повторно используются внутренние результаты вычислений. Также позволяет специалистам, работающим в репозиториях с частичными клонами, завершать больше операций слияния, не прибегая к скачиванию всего дерева из вышестоящего репозитория. О частичных клонах речь пойдет в главе 17.

Рекурсивное слияние

На рис. 6.15 показан пример простого перекрестного слияния.

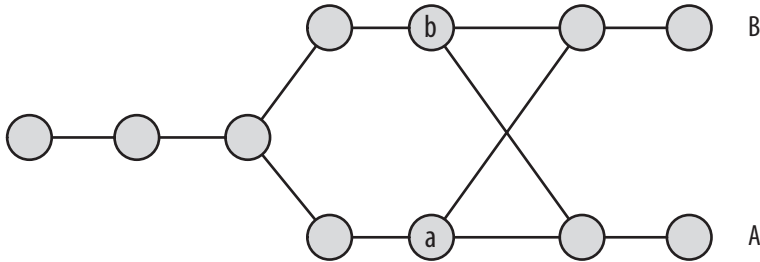


Рис. 6.15. Простое перекрестное слияние

Оба узла, *a* и *b*, являются основами слияния для *A* и *B*. В качестве основы, дающей приемлемый результат, можно использовать любой из них. В этом случае рекурсивная стратегия сливает *a* и *b* во временную основу слияния, которая будет использована как основа слияния для *A* и *B*.

Поскольку *a* и *b* могут иметь одну и ту же проблему, их слияние может потребовать еще одного слияния более старых коммитов. Именно поэтому алгоритм называется рекурсивным.

Octopus-слияния

Основные причины, по которым Git поддерживает слияние нескольких веток одновременно, — это универсальность и элегантность дизайна. В Git у коммита может вовсе не быть родителей (первый коммит); один родитель (обычный коммит); два и более (коммит слияния). «Два и более» означает, что потенциальное количество ничем не ограничено; структуры данных в Git поддерживают множество родителей¹. Стратегия «octopus», или «осьминог», стала естественным следствием дизайн-решения с произвольным числом родителей.

Octopus-слияния хорошо выглядят на диаграммах, поэтому пользователи Git любят их использовать. Только представьте, какой прилив эндорфинов получает разработчик при слиянии шести веток программы в одну. Такие слияния выглядят красиво, но больше ничего особенного они не делают. Вы можете с тем же успехом создать несколько коммитов слияния, по одному на ветку, получив тот же результат.

¹ Так работает принцип «ноль, один или бесконечное множество».



Самое масштабное octopus-слияние, известное на момент написания книги, включало в себя 66 родительских веток, которые привели к одному коммиту слияния в базе кода ядра Linux ().

Специализированные слияния

Есть две особые стратегии слияния, с которыми вам следует познакомиться, поскольку порой они могут оказаться незаменимы при решении непонятных проблем. Если вы не сталкиваетесь с подобными проблемами, то можете пропустить этот раздел. Эти две стратегии — *ours* (*наша*) и *subtree* (*поддерево*).

Каждая из них производит итоговый коммит, который добавляется в текущую ветку и представляет объединенное состояние слияния.

Ours

Стратегия *ours* приводит к слиянию с текущей веткой любого числа других веток, но фактически отбрасывает изменения из этих веток и использует только файлы из текущей. Результат оказывается идентичен текущей *HEAD*, но любые другие именованные ветки также записываются как родители коммита. Этот метод пригодится, когда вы знаете, что у вас уже есть все изменения из других веток, но все равно хотите совместить две истории. Такая стратегия позволяет записать, что вы каким-то образом выполнили слияние, возможно вручную, и что будущие операции Git не должны пытаться снова слить эти истории. Git может рассматривать это как реальное слияние, вне зависимости от того, как оно произошло.

Subtree

Стратегия поддерева подразумевает слияние с другой веткой, но все в этой ветке сливается в конкретное поддерево текущего дерева. При этом вы не указываете, какое это поддерево: Git определяет его автоматически.

Слияния *ours* и *subtree*

Эти две стратегии можно использовать совместно. Например, когда-то давно программа *gitweb* (сейчас это часть Git) разрабатывалась вне основного репозитория *git.git*. Однако в момент ревизии *0a8f4f* вся ее история была слита в *git.git* в поддерево *gitweb*. Если бы вам потребовалось проделать нечто аналогичное, то алгоритм действий был бы таким:

1. Скопировать текущие файлы из проекта *gitweb.git* в подкаталог *gitweb* вашего проекта.
2. Зафиксировать их обычным образом.
3. Выполнить *pull* из проекта *gitweb.git*, используя стратегию *ours*:

```
$ git pull -s ours gitweb.git main
```

Здесь применяется *ours*, поскольку известно, что у нас уже есть последняя версия файлов и мы поместили их в нужное место (куда их не поместила бы обычная *рекурсивная* стратегия).

4. В дальнейшем можно продолжить получать последние изменения из проекта `gitweb.git`, используя стратегию `subtree`:

```
$ git pull -s subtree gitweb.git main
```

Поскольку эти файлы уже есть в репозитории, Git автоматически знает, в какое поддерево их поместить, и выполняет обновление без каких-либо конфликтов.

Применение стратегий слияния

Как же Git понимает, какую стратегию применить? Или, если вам не нравится выбор Git, как выбрать другую?

Как правило, Git использует максимально простые и недорогие алгоритмы, поэтому сначала система пытается использовать стратегии *already up to date* и *fast-forward*, чтобы по возможности исключить тривиальные сценарии.

Если вы укажете несколько веток для слияния с вашей текущей веткой, Git придется задействовать стратегию «осьминога», поскольку только так система сможет объединить более двух веток в одном слиянии.

В случае неудачи Git использует предустановленную стратегию, которая надежно работает во всех других сценариях. Изначально такой была стратегия разрешения (*resolve*).

В ситуациях перекрестного слияния, как было описано выше, когда есть более одной возможной основы слияния, стратегия разрешения работает так: вы выбираете одну из возможных баз слияния (либо последнее слияние из ветви Боба, либо последнее слияние из ветви Кэла) и надеетесь на лучшее. И это не такой плохой вариант, каким может показаться.

Зачастую оказывается, что Алиса, Боб и Кэл работали над разными частями кода. В этом случае Git обнаруживает, что выполняет слияние некоторых изменений, которые уже находятся на своих местах, и пропускает их повторы, избегая конфликта. Либо, если все же есть небольшие изменения, вызывающие конфликт, то разработчику разрешить такие конфликты уже будет несложно.

Сейчас стратегия разрешения больше не используется в Git по умолчанию. Если бы Алиса захотела ее задействовать, ей пришлось бы сделать явный запрос:

```
$ git merge -s resolve Bob
```

В 2005 году Фредерик Куйвинен (Fredrik Kuivinen) разработал рекурсивную стратегию слияния. Она более обобщенная в сравнении со стратегией разрешения и, как было показано на примере ядра Linux, также приводит к меньшему числу

конфликтов, не вызывая ошибок. Кроме того, она неплохо обрабатывает слияния, в которых есть переименование.

В предыдущем примере, когда Алиса хотела слить в свою ветку всю работу Боба, рекурсивная стратегия сработала бы так:

1. Начинаем с последней ревизии от Кэла, которая есть у Алисы и у Боба. В данном случае это последняя ревизия Кэла, Q, которая была слита в ветки Боба и Алисы.
2. Вычисляем различия между этой ревизией и последней, которую Алиса слила от Боба, и встраиваем найденные изменения.
3. Вычисляем различия между объединенной версией и последней версией Боба, встраивая найденные изменения.

Этот метод называется *рекурсивным*, поскольку в нем могут быть дополнительные итерации в зависимости от того, сколько уровней пересечения и основ слияния встречает Git. Рекурсивный метод не только логически разумен, но и ведет к меньшему числу конфликтов в реальных ситуациях по сравнению со стратегией разрешения. Поэтому теперь по умолчанию для `git merge` используется именно он.

Естественно, какую бы стратегию ни выбрала Алиса, итоговая история будет одинаковой (рис. 6.16).

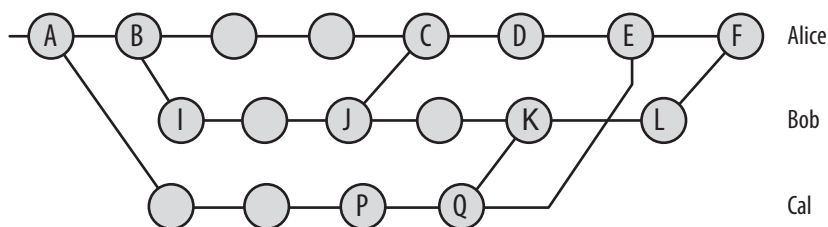


Рис. 6.16. Итоговая история перекрестного слияния

Новая стратегия `merge-ort` привела к дополнительным внутренним усовершенствованиям в Git. В последней (на момент написания книги. — *Примеч. ред.*) ревизии Git в качестве стратегии по умолчанию используется `merge-ort`.

Драйверы слияния

Каждая из описанных стратегий слияния использует внутренний драйвер слияния для разрешения и совмещения каждого отдельного файла. *Драйвер слияния* получает имена трех временных файлов, которые представляют общего предка, версию целевой ветки и версию файла в другой ветке, и изменяет версию целевой ветки, формируя из нее результат слияния.

Драйвер `text` оставляет обычные трехсторонние маркеры слияния (`<<<<<<<`, `=====` и `>>>>>>>`).

Драйвер `binary` сохраняет версию файла целевой ветки и оставляет этот файл в индексе, отмечая его как конфликтующий. По сути, это подталкивает вас к обработке двоичных файлов вручную.

Последний встроенный драйвер слияния, `union`, просто оставляет в совмещенном файле все строки из обеих версий.

Посредством внутреннего механизма присваивания Git может привязывать конкретные файлы или их паттерны к конкретным драйверам слияния. Большинство текстовых файлов обрабатываются драйвером `text`, а большинство двоичных — драйвером `binary`. Тем не менее в особых случаях, требующих применения специфичных операций слияния, вы можете создавать и указывать собственный драйвер, привязывая его к конкретным файлам.



Если вы считаете, что вам нужны собственные драйверы слияния, возможно, вам также стоит изучить пользовательские драйверы различия (`diff drivers`).

Как Git анализирует слияния

Поначалу поддержка автоматического слияния в Git кажется магией, особенно в сравнении с более сложными и уязвимыми для ошибок этапами слияния в других системах контроля версий.

Разберем, какие конкретно механизмы задействованы под капотом.

Слияния и объектная модель Git

В большинстве систем контроля версий каждый коммит имеет только одного родителя. Когда в такой системе вы вливаете `some_branch` в `my_branch`, то создаете в `my_branch` коммит с изменениями из `some_branch`. И наоборот, вливая `my_branch` в `some_branch`, вы создаете коммит в `some_branch`, содержащий изменения из `my_branch`. Вливание ветки А в ветку В и ветки В в ветку А — это две разные операции.

Но разработчики Git заметили, что в результате каждой из этих операций получается одинаковый набор файлов. Естественным способом выразить любую из них можно так: «Объединить все изменения из `some_branch` и `another_branch` в одной ветке».

В Git слияние дает новый объект дерева с объединенными файлами, а также вносит в целевую ветку объект коммита. После следующих команд:

```
$ git checkout my_branch
$ git merge some_branch
```

объектная модель будет выглядеть так, как показано на рис. 6.17.

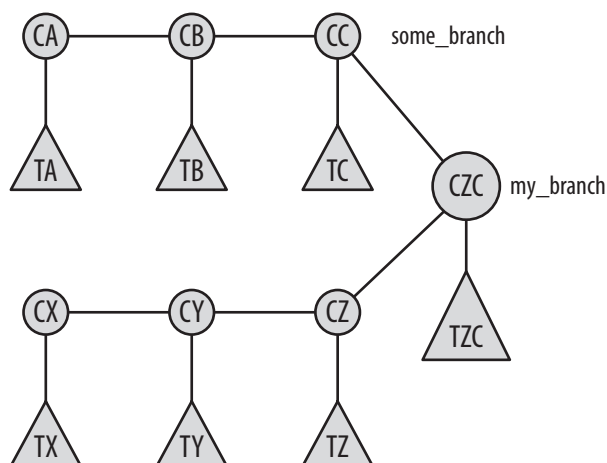


Рис. 6.17. Объектная модель после слияния

На рис. 6.17 каждый Cx — это объект коммита, а каждый Tx представляет соответствующий объект дерева. Заметьте, что здесь есть один общий совмещенный коммит (CZC), который имеет родителей CC и CZ, но в дереве TZC у него представлен всего один набор файлов. Этот объект совмещенного дерева симметрично и в равной степени представляет обе исходные ветки. Но поскольку текущей веткой, с которой происходило слияние, была *my_branch*, именно она и была обновлена новым коммитом; при этом *some_branch* осталась в прежнем состоянии.

И здесь дело не только в семантике. Этот принцип отражает внутреннюю философию Git: все ветки создаются равнозначными¹.

Сжатое слияние (squash merge)

Предположим, что в ветке *some_branch* содержится не один новый коммит, а еще 5, 10 или даже сотни коммитов. Тогда в большинстве систем слияние *some_branch* с *my_branch* включало бы определение отличий, их применение в качестве патча к *my_branch* и создание нового элемента в истории. Такой процесс называется *сжатием коммитов*, поскольку в нем все отдельные коммиты как бы сжимаются в одно большое изменение. При этом история *some_branch* оказалась бы утрачена вплоть до того момента, до которого распространяется история *my_branch*.

В Git же две ветки рассматриваются как равнозначные, поэтому будет неправильным утратить в результате сжатия одну сторону или другую. Вместо этого вся история коммитов с обеих сторон сохраняется. По рис. 6.17 видно, что

¹ Это справедливо и для клонов целых репозиториях.

пользователям за эту сложность приходится платить. Но если Git выполняет squash-коммит, вам уже не нужно видеть схему, которая расходится и затем снова сходится (или вообще думать о ней). История `my_branch` в этом случае будет выглядеть просто прямой линией.



При желании Git может выполнять сжатие коммитов. Достаточно сопроводить команду `git merge` или `git pull` опцией `-squash`. Но будьте внимательны: сжатие коммитов нарушит историю Git, усложнив последующие слияния (подробнее в главе 9).

Такая дополнительная сложность может казаться излишней, но на деле она себя оправдывает. К примеру, эта возможность означает, что команды `git blame` и `git bisect`, рассматриваемые в главе 9, гораздо мощнее своих аналогов в других системах. И как вы уже видели в случае с рекурсивной стратегией, эта дополнительная сложность и получающаяся в итоге детальная история позволяют Git автоматизировать очень сложные слияния.



Хотя в самой операции слияния оба родителя рассматриваются как равнозначные, вы можете обозначить первого из них как особого для удобства при последующем просмотре истории. Некоторые команды (например, `git log` и `gitk`) поддерживают опцию `--first-parent`, которая сопровождает только первого родителя каждого слияния. Итоговая история выглядит почти так же, как если бы вы использовали для всех слияний опцию `--squash`.

Почему бы просто не вливать изменения одно за другим?

Возникает вопрос: а можно ли делать иначе, а именно создавать простую линейную историю, в которой будет представлен каждый коммит? Git мог бы просто брать все коммиты из `some_branch` и применять их поочередно к `my_branch`. Но результат получился бы совсем другим.

Важный нюанс в истории коммитов Git — каждая ревизия в ней *реальна*. (В главе 12 подробно поговорим о рассмотрении альтернативных историй как одинаково реальных.)

Если вы примените *последовательность* чужих патчей поверх своей версии, то создадите серию абсолютно новых версий, объединяющих чужие изменения с вашими. Допустим, вы как обычно протестируете итоговую версию. Но что насчет всех новых промежуточных версий? В реальности они никогда не существовали: никто фактически не создавал эти коммиты, а значит, нельзя однозначно сказать, работали ли они когда-либо.

Git хранит подробную историю, чтобы вы могли вернуться и посмотреть, в каком состоянии находились файлы в определенный момент прошлого. Если некоторые из ваших коммитов отражают версии файлов, которые в реальности никогда не существовали, тогда теряется сам смысл наличия подробной истории.

Именно поэтому слияния Git так не работают. Если вы спросите: «В каком состоянии все находилось за пять минут до слияния?», то ответ будет неоднозначным. Вам нужно спрашивать конкретно про `my_branch` или `some_branch`, поскольку пять минут назад обе эти ветки были другими и Git может дать правдивый ответ для каждой из них.

Несмотря на то что в большинстве случаев нас интересует стандартное слияние историй, Git все же может применять последовательность патчей (подробнее в главе 13). Этот процесс называется *перебазированием* (*rebasing*) и рассматривается в главе 9. Последствия изменения историй коммитов мы разберем в разделе «Изменение публичной истории» на с. 313 (глава 12).

Итоги

Мы начали изучать слияния с очень высокого уровня. Первым делом на примерах кода мы разобрали самое простое — процесс слияния двух веток. Затем познакомились с конфликтами, которые могут при этом возникать. Разрешение таких конфликтов не должно вызывать особых сложностей и не требует профессиональных навыков использования Git. Для этого достаточно уметь обнаруживать конфликтующие файлы и выявлять в них конфликты — в частности, читать и понимать маркеры разрешения конфликтов, дополнительно используя команду `git diff` (о ней поговорим в следующей главе) для более глубокого анализа. Такие навыки — необходимый минимум для успешного разрешения любого конфликта в операциях слияния.

В конце главы мы разобрали внутренние механизмы Git по отслеживанию конфликтов, а также различные стратегии слияния, которые эта система может применять при объединении одной или более веток в репозитории.

Тема могла показаться непростой, поэтому рекомендую вернуться к ней после того, как в своих проектах вы столкнетесь с несколькими случаями слияния и разрешения конфликтов.

Команда diff

Для начала разберем принцип работы утилит `diff` Unix и Linux, поскольку команда `git diff` в Git сравнивает файлы аналогичным образом. Эта команда представляет краткую сводку различий между двумя элементами, отсюда ее название¹. Например, получая два файла, команда `diff` в Unix и Linux сравнивает их построчно, обобщая различия, как это показано далее во фрагменте кода. Точнее говоря, выводится список необходимых изменений для преобразования одного файла в другой.

В примере ниже `initial` — это одна версия некоего текста, а `rewrite` — это его последующая ревизия. Опция `-u` создает единый файл отличий (`diff`), имеющий стандартизированный формат, широко используемый для обмена изменениями.

```
$ cat initial
Now is the time
For all good men
To come to the aid
Of their country.

$ cat rewrite
Today is the time
For all good men
And women
To come to the aid
Of their country.

$ diff -u initial rewrite
--- initial      1867-01-02 11:22:33.000000000 -0500
+++ rewrite      2000-01-02 11:23:45.000000000 -0500
@@ -1,4 +1,5 @@
-Now is the time
+Today is the time
For all good men
+And women
To come to the aid
Of their country.
```

¹ `diff` — сокращение от *англ.* differences, то есть различия. — *Примеч. ред.*

Давайте детально разберем этот код. В заголовке исходный файл обозначается символами - - -, а новый — символами + + +. Строка @@ предоставляет контекст строки по ее номеру для обеих версий файла. Строку со знаком «минус» (-) нужно удалить из исходного файла для получения нового файла. И наоборот, строку со знаком «плюс» (+) нужно добавить в исходный файл, чтобы получить новый. Строка, начинающаяся с пробела, в обоих файлах одинаковая и предоставляется опцией -u как контекст.

Сам по себе результат `diff` не приводит каких-либо причин или доводов в пользу изменения, как и не подтверждает начальное или конечное состояние. Тем не менее он выдает нечто большее, нежели просто сводку по отличиям файлов. Он предоставляет формальное описание того, как преобразовать один файл в другой. (Подобные инструкции пригодятся во время применения или отката изменений.) Помимо этого, `diff` можно расширить на отображение отличий между несколькими файлами и полными иерархиями каталогов.

Команды `diff` в Unix/Linux могут вычислять различия между всеми парами файлов, находящихся в двух иерархиях каталогов. Команда `diff -r` обходит каждую эту иерархию, сопоставляет файлы по их именам (например, `original/src/main.c` и `new/src/main.c`) и обобщает отличия между каждой парой. Выполнение `diff -r -u` ведет к созданию набора унифицированных изменений, сравнивающих две иерархии.

В Git есть собственная утилита `diff`, позволяющая подобным образом создавать сводку отличий. Более того, как и `diff -r` в Unix, Git может обходить два объекта дерева, генерируя представление их различий. Но `git diff` имеет свои нюансы и широкие возможности, ориентированные на конкретные потребности пользователей. В качестве базового алгоритма сравнения Git использует разностный алгоритм Майерса (<https://oreil.ly/6rq3x>), разработанный Юджином Майерсом.

В этой главе мы изучим основы `git diff` и часть ее особых возможностей. Вы узнаете, как использовать Git для вывода правок в рабочем каталоге, а также произвольных изменений между двумя коммитами в истории проекта. Вы увидите, как `diff` в Git помогает создавать четко структурированные коммиты в ходе типичного процесса разработки, а также узнаете, как создавать патчи Git, которые подробно будут описаны в главе 13.



Теоретически объект дерева представляет только один уровень каталогов в репозитории. Он содержит информацию о файлах этого каталога и его подкаталогах, но не каталогизирует все содержимое всех подкаталогов. Но поскольку объект дерева ссылается на другие объекты дерева, соответствующие каждому подкаталогу, то объект дерева, который находится в корне проекта, по сути представляет весь проект в определенный момент времени. Можно сказать, что `git diff` как бы обходит два дерева.

Формы команды git diff

Если вы выберете для сравнения два корневых объекта дерева, `git diff` выдаст все различия между этими двумя состояниями проекта. Это впечатляющая возможность. Вы можете использовать такой `diff` для полного преобразования проекта из одного состояния в другое. Например, если вы с коллегой работаете над кодом для одного проекта, то корневой `diff` можно в любой момент времени применить для синхронизации вашего репозитория.

Есть три базовых источника для объектов дерева или им подобных, которые можно использовать с `git diff`:

- любой объект дерева в любом месте всего графа коммитов;
- рабочий каталог, который в Git в контексте `diff` называется *рабочим деревом*;
- индекс, или каталог индексирования.

Как правило, деревья, которые сравниваются командой `git diff`, именуются через коммиты, имена веток или тегов. В принципе, подойдет любое имя коммита (про имена см. раздел «Идентификация коммитов» на с. 102 в главе 4). Иерархии файлов и каталогов в рабочем каталоге, а также всю иерархию индексированных файлов можно рассматривать как деревья.

Команда `git diff`, дополненная определенными опциями, может выполнять фундаментальное сравнение. Вот типичные примеры:

`git diff`

Выводит отличия между рабочим каталогом и индексом. Показывает, что находится в рабочем каталоге, а значит, является кандидатом на индексирование для очередного коммита. Эта команда не показывает различия между тем, что находится в индексе, и тем, что постоянно хранится в репозитории (не говоря уже о возможных удаленных репозиториях; эту тему рассмотрим в главе 11).

`git diff --cached commit`

Показывает различия между индексированными изменениями в индексе и указанным коммитом с именем `commit`. Стандартным и предустановленным коммитом для сравнения, если не указано иное, выступает `HEAD`. В случае `HEAD` эта команда показывает, как очередной коммит изменит текущую ветку. Команда `git diff --staged` — это синоним для `git diff --cached`. Она выводит различия между содержимым индекса и постоянным содержимым репозитория.

`git diff commit`

Обобщает различия между рабочим каталогом и указанным коммитом `commit`. Стандартные версии этой команды в качестве `commit` указывают `HEAD` или имя конкретной ветки.

```
git diff commit1 commit2
```

Если вы укажете два произвольных коммита, эта команда выведет отличия между ними. Она игнорирует индекс и рабочий каталог, выступая типичным решением для произвольного сравнения двух деревьев, уже находящихся в объектном хранилище репозитория Git.

Количество параметров в командной строке определяет то, какая основная форма команды используется и что сравнивается. Можно сравнивать любые два коммита или дерева. Сравнимые объекты не обязательно должны иметь прямую или даже косвенную связь «родитель — потомок». Если вы не укажете объект (или оба объекта) дерева, то `git diff` сравнит предполагаемые источники: индекс или рабочий каталог.



Если нужно сравнить необработанное содержимое BLOB-объектов в объектном хранилище Git, это можно сделать с помощью команды `git diff blob1 blob2`.

Теперь проверим, как эти формы применимы к объектной модели Git.

В примере на рис. 7.1 показан каталог проекта с двумя файлами. Содержимое файла `file1`, находящегося в рабочем каталоге, было изменено с «foo» на «quux». Это изменение было индексировано с помощью `git add file1`, но еще не зафиксировано.

Версии `file1` из рабочего каталога, индекса и HEAD были идентифицированы. И хотя версия `file1` в индексе `db71363` фактически хранится в виде BLOB-объекта в объектном хранилище, на нее ведет косвенная ссылка через виртуальный объект дерева, являющийся этим индексом. Аналогично на версию HEAD, `a23bf`, также ведет косвенная ссылка через несколько шагов.

Этот пример показывает изменения в `file1`. Сплошные стрелки указывают на объекты дерева или виртуального дерева, напоминая, что сравнение фактически основывается на полных деревьях, а не только на отдельных файлах. Для интересующихся читателей мы опишем, как воссоздать этот сценарий на примерах кода в разделе «Простой пример `git diff`» на с. 183.

На рис. 7.1 мы видим, что использование `git diff` без аргументов — хорошая техника для определения готовности очередного коммита. Пока эта команда дает вывод, мы понимаем, что в рабочем каталоге есть еще не индексированные правки или изменения. Использование простой команды `git status` поможет узнать о наличии ожидающих изменений в каждом файле. Когда вы проделаете все необходимое, используйте `git add` для индексирования файла. После этого очередная команда `git diff` уже не произведет для него вывода. Так вы можете поэтапно перебирать каждый измененный файл в рабочем каталоге, пока не исчезнут все отличия. Это будет говорить о том, что все файлы и их

изменения внесены в индекс. Не забудьте также выполнить проверку на новые или удаленные файлы.

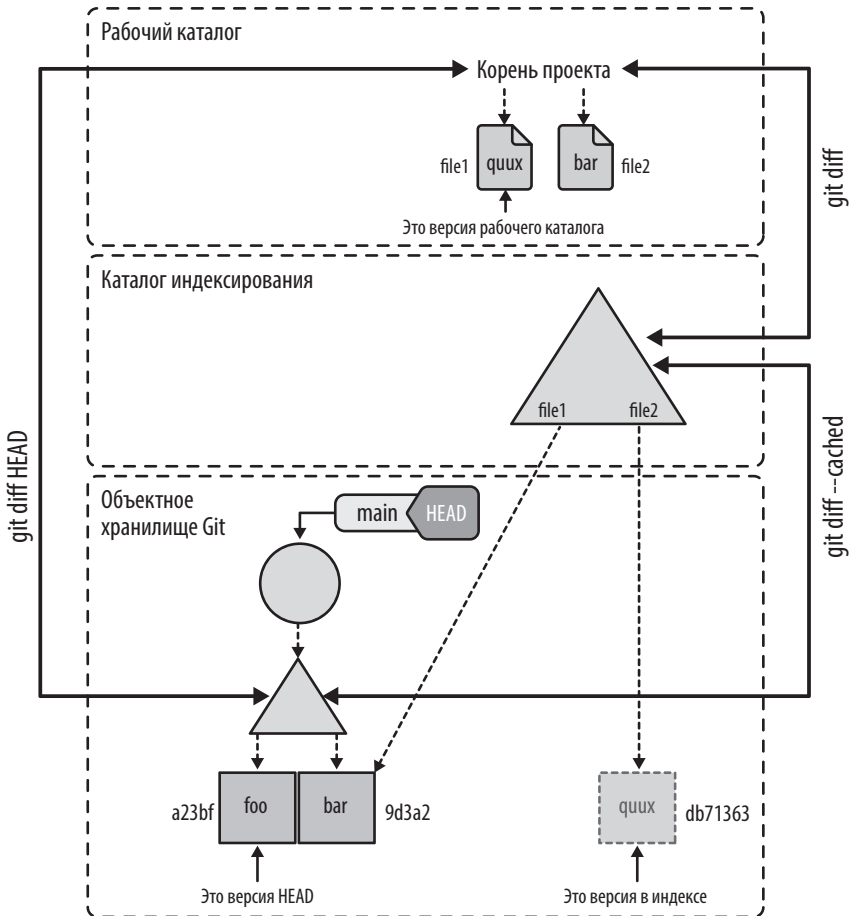


Рис. 7.1. Разные версии файла, которые можно сравнить

В любой момент индексирования команда `git diff --cached` показывает дополнительные изменения или те, которые уже были индексированы и будут в очередном коммите. Когда закончите, `git commit` отразит все эти изменения в новом коммите. Теперь вам не обязательно индексировать все изменения из рабочего каталога для одного коммита. Если вы узнаете, что у вас в рабочем каталоге есть принципиально иные изменения, которые нужно вносить разными коммитами, то можете индексировать их наборы по одному за раз, оставляя часть правок в рабочем каталоге. Коммит отражает только индексированные изменения. Повторите процесс, индексируя очередной набор файлов, подходящих для последующего коммита.

Внимательный читатель мог заметить, что, хотя мы и разобрали четыре фундаментальные формы команды `git diff`, только три из них выделены сплошными стрелками на рис. 7.1. Где же четвертая? Вашим рабочим каталогом представлен всего один объект дерева, и всего один такой объект представлен в индексе. В этом примере в объектном хранилище присутствует один коммит вместе со своим деревом. Тем не менее объектное хранилище наверняка содержит множество коммитов, названных в соответствии с разными ветками и тегами. Каждый из коммитов содержит файлы, которые можно сравнить с помощью `git diff`. Таким образом, четвертая форма `git diff` просто сравнивает любые два коммита (дерева), уже находящиеся в объектном хранилище.

Помимо базовых форм `git diff` есть и другие. Вот еще несколько, которые могут пригодиться:

-M

Опция **-M** обнаруживает переименование и генерирует упрощенный вывод, который регистрирует не удаление и последующее добавление файла, а изменение его имени. Если при переименовании произошли также некие дополнительные изменения содержимого, Git на них указывает.

-w, а также --ignore-all-space

Обе эти опции сравнивают строки, игнорируя отличия в пробелах.

--stat

Опция **--stat** добавляет статистику по отличиям между любыми двумя состояниями дерева. В компактной форме она сообщает, сколько строк было изменено, сколько добавлено, а сколько опущено.

--color

Опция **--color** раскрашивает вывод; каждый вид изменения в этом diff имеет свой уникальный цвет.

--word-diff=[*mode*]

Опция **--word-diff=[*mode*]** выделяет измененные слова. Если вы выберете режим *color*, то в выводе будут показаны измененные слова с использованием цвета для обозначения удаления и добавления. По умолчанию установлен режим *plain*, при котором слова в diff показаны как `[- удаленные слова -]` и `{+ добавленные слова +}`. Дополнительные режимы подробно описываются в руководстве по `git diff`.

--name-only

Опция **--name-only** позволяет быстро просмотреть только имена измененных файлов.

--name-status

Опция **--name-status** выводит сводку по именам и статусам измененных файлов. Статус отображается в виде **M** для измененных, **A** — для добавленных, **C** — для скопированных и **D** — для удаленных. Эти и другие статусы подробно описаны в руководстве **git diff**.

--output=file

Опция **--output=file** перенаправляет вывод вместо **stdout** в указанный файл.

Наконец, **git diff** можно ограничить до вывода различий в конкретном наборе файлов или каталогов. Делается это с помощью опции **path... diff** с тремя точками (...) используются, когда вы ищете изменения, внесенные в ветку функциональности, и хотите сравнить эту ветку с веткой **main**, из которой она была создана, но **main** с тех пор уже сама изменилась. Об ограничении пути для сравнения подробно поговорим в разделе «**git diff** с ограничением пути» на с. 191.



Опция **-a** в **git diff** означает нечто совсем иное, чем опция **-a** в **git commit**. Чтобы получить индексированные и неиндексированные изменения, используйте **git diff HEAD**.

Простой пример git diff

Давайте создадим сценарий, представленный на рис. 7.1, запустим его и понаблюдаем различные формы **git diff** в действии. Для начала настроим простой репозиторий с двумя файлами:

```
$ mkdir /tmp/diff_example
$ cd /tmp/diff_example

$ git init -b main
Initialized empty Git repository in /tmp/diff_example/.git/

$ echo "foo" > file1
$ echo "bar" > file2

$ git add file1 file2

$ git commit -m "Add file1 and file2"
[main (root-commit) 7915072] Add file1 and file2
 2 files changed, 2 insertions(+)
 create mode 100644 file1
 create mode 100644 file2
```

Далее изменим **file1**, заменив «foo» на «quux»:

```
$ echo "quux" > file1
```

Файл `file1` был изменен в рабочем каталоге, но еще не индексирован. Это состояние не соответствует ситуации на рис. 7.1, но сравнение вы все же провести можете. Вывод следует ожидать, если вы будете сравнивать рабочий каталог с версией из индекса или существующей версией `HEAD`. Однако отличий между индексом и `HEAD` быть не должно, поскольку мы ничего не индексировали (то есть у нас по-прежнему индексировано текущее дерево `HEAD`).

```
# diff между рабочим каталогом и индексом
$ git diff
diff --git a/file1 b/file1
index 257cc56..d90bda0 100644
--- a/file1
+++ b/file1 @@ -1 +1 @@
-foo
+quux

# diff между рабочим каталогом и HEAD
$ git diff HEAD
diff --git a/file1 b/file1
index 257cc56..d90bda0 100644
--- a/file1
+++ b/file1 @@ -1 +1 @@
-foo
+quux

# diff между индексом и HEAD не дает вывода,
# поскольку разницы между ними по-прежнему нет
$ git diff --cached
$
```

Применив только что описанный принцип, команда `git diff` произвела вывод, а значит, `file1` содержит свежие изменения, которые можно индексировать. Сделаем это:

```
$ git add file1

$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   file1
```

Мы получили точный повтор ситуации, отраженной на рис. 7.1. Поскольку `file1` теперь индексирован, рабочий каталог и индекс синхронизированы и отличий между ними быть не должно. Однако теперь версия `HEAD` отличается от рабочего каталога и индексированной версии:

```
# diff между рабочим каталогом и индексом
$ git diff

# diff между рабочим каталогом и HEAD
```

```
$ git diff HEAD
diff --git a/file1 b/file1
index 257cc56..d90bda0 100644
--- a/file1
+++ b/file1 @@ -1 +1 @@
-foo
+quux
```

```
# diff между индексом и HEAD
$ git diff --cached
diff --git a/file1 b/file1
index 257cc56..d90bda0 100644
--- a/file1
+++ b/file1 @@ -1 +1 @@
-foo
+quux
```

Если выполнить `git commit` сейчас, то новый коммит отразит индексированные изменения, выведенные последней командой, `git diff --cached` (которая, как говорилось ранее, имеет синоним: `git diff --staged`).

Добавим новый, более интересный сценарий. Посмотрим, что произойдет, если изменить `file1` перед коммитом:

```
$ echo "baz" > file1
```

```
# diff между рабочим каталогом и индексом
$ git diff
diff --git a/file1 b/file1
index d90bda0..7601807 100644
--- a/file1
+++ b/file1 @@ -1 +1 @@
-quux
+baz
```

```
# diff между рабочим каталогом и HEAD
$ git diff HEAD
diff --git a/file1 b/file1
index 257cc56..7601807 100644
--- a/file1
+++ b/file1 @@ -1 +1 @@
-foo
+baz
```

```
# diff между индексом и HEAD
$ git diff --cached
diff --git a/file1 b/file1
index 257cc56..d90bda0 100644
--- a/file1
+++ b/file1 @@ -1 +1 @@
-foo
+quux
```

Все три операции `diff` показывают некоторые отличия. Но какая из этих версий будет зафиксирована? Напомню, что `git commit` отражает состояние, представленное в индексе. А что у нас в индексе? Это содержимое, которое раскрывает команда `git diff --cached` или `git diff --staged`, то есть версия `file1`, содержащая слово «quux». Зафиксируем `file1`:

```
$ git commit -m "quux for the win"
[main 27f006d] quux for the win
1 file changed, 1 insertion(+), 1 deletion(-)
```

Теперь, когда в объектном хранилище есть два коммита, используем общую форму команды `git diff`:

```
# diff между прежней версией HEAD version и текущей
$ git diff HEAD^ HEAD
diff --git a/file1 b/file1
index 257cc56..d90bda0 100644
--- a/file1
+++ b/file1
@@ -1,1 @@
-foo
+quux
```

Этот вывод подтверждает, что предыдущий коммит изменил `file`, заменив «foo» на «quux». Все ли сейчас синхронизировалось? Нет. Копия `file1` в рабочем каталоге содержит слово «baz»:

```
$ git diff
diff --git a/file1 b/file1
index d90bda0..7601807 100644
--- a/file1
+++ b/file1
@@ -1,1 @@
-quux
+baz
```

Интерпретация вывода `git diff`

Мы увидели, как работают разные формы `git diff`, но пока не разобрали, как интерпретировать генерируемый вывод. Продолжим один из наших примеров: создадим и изменим новый файл, `file3`:

```
# добавляем file3 с пятью строками
$ echo -e "Line1 \nLine2 \nLine3 \nLine4 \nLine5" >> file3
$ cat file3
Line1
Line2
Line3
Line4
```

```

Line5

# добавляем file3 в индекс
$ git add file3
...

# редактируем file3, удаляем Line3, добавляем строки Line6 и Line7
...

$ cat file3
Line1
Line2
Line4
Line5
Line6
Line7

```

На рис. 7.2 показан вывод `git diff`.

```

$ Git diff file3
diff --git a/file3 b/file3
index 7e7096e..56d6bcd 100644
--- a/file3
+++ b/file3
@@ -1,5 +1,6 @@
- Line1
- Line2
- Line3
- Line4
+ Line1
+ Line2
+ Line4
  Line5
+ Line6
+ Line7

```

diff --git a/file3 b/file3 Источник сравнения diff
 index 7e7096e..56d6bcd 100644 ID внутреннего объекта Git (источника для сравнения diff)
 --- a/file3
 +++ b/file3 Маркеры источника
 @@ -1,5 +1,6 @@ Заголовки наборов отличий в выводе diff
 - Line1
 - Line2
 - Line3
 - Line4
 + Line1
 + Line2
 + Line4
 Line5
 + Line6
 + Line7 Отличия в выводе diff

Рис. 7.2. Вывод `git diff`

Ниже приведено объяснение каждой части вывода.

Источник сравнения diff

В этом блоке показаны входные файлы, которые сравниваются с помощью `diff`. Этот вывод похож на унифицированный формат, использующийся инструментом `diff` в Unix.

ID внутреннего объекта Git (источника для сравнения diff)

Здесь показываются ID сравниваемых объектов Git.

Если выполнить `git ls-files -s`, вы заметите, что `7e7096e` — это сокращенный ID SHA-1 для содержимого `file3` в индексе. При этом `56d6bcd` — это сокращенный ID SHA-1 содержимого `file3` из рабочего каталога. Если вы выполните `git hash-object file3`, то выводом будет полный ID SHA-1, который можно проверить.

Маркеры источника

В этом блоке указывается, что исходная версия файла обозначена маркерами `- - -`, а измененная — маркерами `+++`. Аналогичные маркеры будут использоваться для обозначения изменений, отображенных в блоках отличий файлов. Помните, что вывод `diff` показывает разделы файла, в которых содержатся изменения, поэтому такие маркеры полезны для идентификации этих изменений.

Заголовки наборов отличий

Блоки заголовков отличий заключаются в символы `@@` и выглядят так:

```
@@ oldfile-change-range newfile-change-range @@
```

Диапазон изменений (change range) выводится в следующем формате: сначала идет маркер источника и стартовая строка, а затем перечисляются изменения:

```
[-/+]<start-line>,<lines of change>
```

В нашем примере `-1,5 +1,6` интерпретируются как:

```
(-)oldfile<starting from line 1>,<5 lines of change>
(+)newfile<starting from line 1>,<6 lines of change>
```

Наборы отличий

В этом блоке выводятся части файла, содержащие отличия. Поскольку наш пример маленький, в выводе есть всего один набор отличий (эти наборы также включают заголовки). Заметьте, что наборы генерируются динамически по мере необходимости и в репозитории не сохраняются. Строки с изменениями, содержащиеся в старом и новом файлах, предваряются маркерами источника, а строки без изменений — пробелами. В нашем примере старая версия `file3` (в индексе) содержит диапазон от `Line1` до `Line5` (всего пять строк, начиная с первой верхней, `-1,5`), а новая версия `file3` (в рабочем каталоге) содержит строки от `Line1` до `Line7`, исключая `Line3` (всего шесть строк, начиная с первой верхней, `+1,6`).

git diff и диапазоны коммитов

Есть еще две формы `git diff`, требующие некоторого пояснения, особенно в сравнении с `git log`.

Команда `git diff` поддерживает синтаксис с использованием двух точек (..) для представления отличий между двумя коммитами. Таким образом, две приведенные ниже команды равнозначны:

```
$ git diff main bug/pr-1
$ git diff main..bug/pr-1
```

К сожалению, этот синтаксис в `git diff` означает не то же самое, что аналогичный синтаксис в `git log`, с которым мы познакомились в главе 4. Стоит сравнить `git diff` и `git log`, чтобы подчеркнуть отношение этих команд к вносимым в репозиторий изменениям.

Что следует иметь в виду при рассмотрении следующего примера:

- `git diff` не интересуется история сравниваемых ею файлов, а также информация о ветках;
- `git log`, напротив, очень хорошо знает, как определенный файл изменился, приняв свою новую форму, — например, когда две ветки разошлись и что произошло с каждой.

Команды `log` и `diff` выполняют две фундаментально разные операции. Если `log` работает с набором коммитов, то `diff` — с двумя разными конечными точками.

Представьте себе последовательность событий.

1. Кто-то создает новую ветку из ветки `main` для исправления бага `pr-1` и называет эту ветку `bug/pr-1`.
2. Тот же разработчик добавляет строку «Fix Problem Report 1» в файл из этой ветки `bug/pr-1`.
3. Тем временем другой разработчик исправляет баг `pr-3` в ветке `main`, добавляя строку «Fix Problem Report 3» в тот же файл в ветке `main`.

Выходит, что в этот файл в каждой ветке добавляется по одной строке. Если взглянуть на изменения в ветках с верхнего уровня, то можно увидеть, когда была создана `bug/pr-1` и когда было внесено каждое изменение:

```
$ git show-branch main bug/pr-1
* [main] Added a bug fix for pr-3.
! [bug/pr-1] Fix Problem Report 1
--
* [main] Added a bug fix for pr-3.
+ [bug/pr-1] Fix Problem Report 1
*+ [main^] Added Bob's fixes.
```

Если выполнить `git log -p main..bug/pr-1`, то мы увидим один коммит, поскольку синтаксис `main..bug/pr-1` представляет все коммиты в `bug/pr-1`, которых нет в `main`. Эта команда прослеживает историю назад до точки, когда `bug/pr-1` ответвилась от `main`, но не обращает внимания на то, что произошло с `main` с того момента:

```
$ git log -p main..bug/pr-1
commit 8f4cf5757a3a83b0b3dbecd26244593c5fc820ea
Author: Jon Loeliger <jdl@example.com>
Date:   Wed May 14 17:53:54 2008 -0500
```

```
Fix Problem Report 1
```

```
diff --git a/ready b/ready
index f3b6f0e..abbf9c5 100644
--- a/ready
+++ b/ready
@@ -1,3 +1,4 @@
     stup
     znull
     frot-less
+Fix Problem Report 1
```

В противоположность этому `git diff main..bug/pr-1` показывает общий набор отличий между двумя деревьями, представленными вершинами веток `main` и `bug/pr-1`. История в этом случае не важна — важно только текущее состояние файлов:

```
$ git diff main..bug/pr-1
diff --git a/ready b/ready
index f3b6f0e..abbf9c5 100644
--- a/ready
+++ b/ready
@@ -1,4 +1,4 @@
     stup
     znull
     frot-less
-Fix Problem Report 3
+Fix Problem Report 1
```

Если перефразировать вывод `git diff`, то получится, что можно изменить файл в ветке `main` на версию из ветки `bug/pr-1`, удалив из него строку «Fix Problem Report 3» и добавив строку «Fix Problem Report 1».

Как видите, этот `diff` включает коммиты из обеих веток. В небольшом примере это может показаться не столь значительным, но взгляните на рис. 7.3, где представлено больше строк разработки в двух ветках.

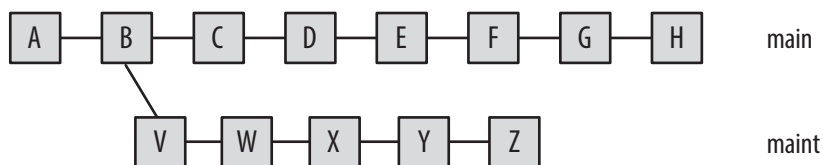


Рис. 7.3. Более обширная история `git diff`

В этом случае `git log main...maint` представляет пять отдельных коммитов, V, W, ..., Z. В то же время `git diff main...maint` отражает отличия в деревьях у N и Z, то есть 11 накопленных коммитов: C, D, ..., N и V, ..., Z.

Аналогичным образом `git log` и `git diff` получают форму `commit1...commit2`, выводя симметричные отличия. При этом `git log commit1...commit2` и `git diff commit1...commit2`, как и прежде, дают разные результаты.

Как обсуждалось в разделе «Диапазоны коммитов» на с. 117 (глава 4), команда `git log commit1... commit2` выводит коммиты, достижимые из любого из этих коммитов, но не из обоих. Таким образом, `git log main...maint` в предыдущем примере выдаст C, D, ..., N и V, ..., Z.

Симметричные отличия в `git diff` показывают разницу между коммитом, который является общим предком (или основой слияния) `commit1` и `commit2`. В случае той же генеалогии, что и на рис. 7.3, `git diff main...maint` совмещает изменения в коммитах V, W, ..., Z.

git diff с ограничением пути

По умолчанию команда `git diff` применяется ко всей структуре каталогов, корень которой находится в заданном объекте дерева. Но вы можете задействовать ту же технику ограничения пути, с помощью которой `git log` ограничивает вывод `git diff` до нужной части репозитория.

Например, в определенный момент разработки¹ собственного репозитория Git команда `git diff --stat` выдавала следующее:

```
$ git diff --stat main~5 main
Documentation/git-add.txt      | 2 +-
Documentation/git-cherry.txt   | 6 +++++
Documentation/git-commit-tree.txt | 2 +-
Documentation/git-format-patch.txt | 2 +-
Documentation/git-gc.txt       | 2 +-
Documentation/git-gui.txt      | 4 +-
Documentation/git-ls-files.txt  | 2 +-
Documentation/git-pack-objects.txt | 2 +-
Documentation/git-pack-redundant.txt | 2 +-
Documentation/git-prune-packed.txt | 2 +-
Documentation/git-prune.txt     | 2 +-
Documentation/git-read-tree.txt | 2 +-
Documentation/git-remote.txt    | 2 +-
Documentation/git-repack.txt    | 2 +-
Documentation/git-rm.txt        | 2 +-

```

¹ d2b3691b61d516a0ad2bf700a2a5d9113ceff0b1

```

Documentation/git-status.txt      | 2 +-
Documentation/git-update-index.txt | 2 +-
Documentation/git-var.txt         | 2 +-
Documentation/gitk.txt            | 2 +-
builtin-checkout.c               | 7 ++++
builtin-fetch.c                  | 6 ++-
git-bisect.sh                    | 29 ++++++-----
t/t5518-fetch-exit-status.sh     | 37 ++++++-----
23 files changed, 83 insertions(+), 40 deletions(-)

```

Чтобы ограничить этот вывод только изменениями в *документации*, нужно использовать `git diff --stat main~5 main Documentation`:

```

$ git diff --stat main~5 main Documentation
Documentation/git-add.txt      | 2 +-
Documentation/git-cherry.txt   | 6 +++++
Documentation/git-commit-tree.txt | 2 +-
Documentation/git-format-patch.txt | 2 +-
Documentation/git-gc.txt      | 2 +-
Documentation/git-gui.txt     | 4 ++-
Documentation/git-ls-files.txt | 2 +-
Documentation/git-pack-objects.txt | 2 +-
Documentation/git-pack-redundant.txt | 2 +-
Documentation/git-prune-packed.txt | 2 +-
Documentation/git-prune.txt   | 2 +-
Documentation/git-read-tree.txt | 2 +-
Documentation/git-remote.txt  | 2 +-
Documentation/git-repack.txt   | 2 +-
Documentation/git-rm.txt      | 2 +-
Documentation/git-status.txt   | 2 +-
Documentation/git-update-index.txt | 2 +-
Documentation/git-var.txt     | 2 +-
Documentation/gitk.txt        | 2 +-
19 files changed, 25 insertions(+), 19 deletions(-)

```

Естественно, также можно просматривать и отличия для одного файла:

```

$ git diff main~5 main Documentation/git-add.txt
diff --git a/Documentation/git-add.txt b/Documentation/git-add.txt
index bb4abe2..1afd0c6 100644
--- a/Documentation/git-add.txt
+++ b/Documentation/git-add.txt
@@ -246,7 +246,7 @@ characters that need C-quoting. `core.quotePath`
configuration can be
used to work this limitation around to some degree, but backslash,
double-quote and control characters will still have problems.

```

-See Also

+SEE ALSO

```

linkgit:git-status[1]
linkgit:git-rm[1]

```

В примере ниже, также взятом из репозитория Git, команда `-S"string"` просматривает последние 50 коммитов ветки `main` на предмет изменений, содержащих `string`:

```
$ git diff -S"octopus" main~50
diff --git a/Documentation/RelNotes-1.5.5.3.txt b/Documentation/RelNotes-1.5.5.3.txt
new file mode 100644
index 0000000..f22f98b
--- /dev/null
+++ b/Documentation/RelNotes-1.5.5.3.txt
@@ -0,0 +1,12 @@
+GIT v1.5.5.3 Release Notes
+=====
+
+
+Fixes since v1.5.5.2
+-----
+
+
+ * "git send-email --compose" did not notice that non-ascii contents
+   needed some MIME magic.
+
+
+ * "git fast-export" did not export octopus merges correctly.
+
+
+Also comes with various documentation updates.
```

При использовании опции `-S` (ее также называют *pickaxe*) Git выводит отличия, которые содержат изменение в количестве вхождений указанной строки `string` в `diff`. По сути, это можно рассматривать как вопрос «Где указанная `string` была внесена или удалена?». Пример использования этой опции есть в разделе «Использование инструмента Pickaxe» на с. 204 в главе 8.

Как Git выводит отличия

Большинство систем контроля версий, включая CVS и SVN, отслеживают серию ревизий и сохраняют только изменения между каждой парой файлов. Эта техника нацелена на экономию пространства хранилища и уменьшение издержек.

Подобные системы тратят много времени на анализ таких моментов, как «серия изменений между А и В». Например, когда вы обновляете файл из центрального репозитория, SVN помнит, что при последнем обновлении этого файла репозиторий находился в ревизии `r1095`, а сейчас находится в ревизии `r1123`. Таким образом, сервер должен отправить вам отличия между этими ревизиями. Как только клиент SVN получит эти отличия, он сможет встроить их в копию рабочего каталога и создать `r1123`. (Так SVN избегает отправки всего содержимого всех файлов при каждом обновлении.)

Для экономии пространства диска SVN также сохраняет собственный репозиторий в виде серии файлов отличий (diffs) на сервере. Когда вы запросите отличия

между `r1095` и `r1123`, система просмотрит все отдельные изменения в каждой версии между этими ревизиями, совместит их в один большой diff и отправит вам результат. Но Git работает иначе.

В Git каждый коммит содержит *дерево*, представляющее собой список файлов, находящихся в этом коммите. Каждое дерево независимо от всех других деревьев. Конечно, пользователи Git работают с патчами и diff в силу их невероятной полезности. Однако в этой системе diff и патч представляют собой выведенные, а не фундаментальные данные, каковыми они являются в CVS и SVN. Если вы заглянете в каталог `.git`, то не найдете ни одного diff, в отличие от репозитория SVN, который состоит преимущественно из diff.

Точно так же как SVN может вывести полный набор отличий между `r1095` и `r1123`, Git может извлечь отличия между двумя произвольными состояниями. Но SVN приходится просмотреть каждую версию между `r1095` и `r1123`, в то время как системе Git эти промежуточные шаги не требуются.

Каждая ревизия имеет собственное дерево, но Git для генерации diff они не нужны. Эта система может работать непосредственно со снимками всего состояния в каждой из двух версий. Это простое отличие в системах хранения — одна из важнейших причин, благодаря которой Git работает настолько быстрее других систем контроля версий.

Итоги

В этой главе мы увидели, как команда `git diff` служит мощным инструментом для сравнения изменений и состояния файлов в репозиториях на стадии разработки. Если вы знакомы с утилитами `diff` в Unix/Linux, то вам будет гораздо проще освоить аналогичную команду в Git. Если же она для вас нова, то рекомендуем внимательно изучить раздел «Простой пример `git diff`» на с. 183, а потом раздел «Интерпретация вывода `git diff`» на с. 186.

Часть III

Базовые навыки

В части III вы освоите базовые навыки, необходимые в работе с репозиториями Git. Мы начнем с изучения коммитов и завершим знакомством с концепцией удаленных репозиториях, а также с некоторыми лучшими практиками по управлению ими.

История репозитория состоит из коммитов, и временами могут возникнуть веские причины для ее изменения. Но чтобы изменять коммиты, нужно научиться их находить.

В главе 8 мы увидим, как искать конкретные коммиты и их метаданные.

Затем в главе 9 познакомимся с различными техниками изменения коммитов, причем некоторые из них окажутся для истории вашего репозитория деструктивными. Навыки, которые вы освоите в этой главе, не будут ограничены только операциями, связанными с изменениями коммитов. Они также помогут вам при отладке или пригодятся для понимания того, как в репозитории возникли те или иные изменения.

В главе 10 мы разберем, как можно сохранять и извлекать временные рабочие изменения, а также познакомимся с журналом ссылок (reflog), где ведется запись поддерживаемых операций для каждой вносимой ссылки или коммита.

В главе 11 узнаем, как лучше всего организовывать сотрудничество и обмениваться изменениями, работая со множеством людей, которым нужен доступ к вашему репозиторию.

Наконец, в главе 12 будет рассказано, как опубликовать свой репозиторий и правильно его структурировать для распределенной разработки.

ГЛАВА 8

Поиск коммитов

Часть хорошей системы контроля версий — это возможность изучить историю и исследовать репозиторий. В Git заложено несколько механизмов, которые позволяют находить нужные коммиты.

В этой главе вы освоите техники поиска конкретных коммитов и их метаданных. Мы сосредоточимся на трех методах, позволяющих выполнять поиск по истории коммитов. Первый метод отличается высокой надежностью и понадобится при поиске одного коммита по заданному критерию. Второй предоставляет информацию о коммитах, которые внесли изменения в файл. Третий использует определенную вариацию поиска с использованием стандартной команды `git log`.

Эта глава также послужит своеобразной подводкой к главе 9, где мы углубимся в тему изменения найденных коммитов.

Использование `git bisect`

Команда `git bisect` — это мощный инструмент для выделения конкретного коммита на основе произвольного критерия поиска. Она пригодится, когда вы обнаруживаете нечто, что «плохо влияет» на ваш репозиторий, но при этом знаете, что ранее код был в порядке.

Допустим, вы работаете с ядром Linux и у вас проваливается тест загрузки. Однако вы знаете, что незадолго до этого, возможно недель раньше или во время предыдущего релиза, все работало исправно. Это означает, что ваш репозиторий перешел из состояния «исправен» в состояние «неисправен».

Но когда именно произошел этот переход? Какой коммит привел к поломке? На этот вопрос помогает ответить команда `git bisect`.

Главное условие поиска заключается в том, чтобы, имея текущее состояние репозитория, определить, соответствует ли оно этому условию поиска. В данном случае вы должны ответить на вопрос: «Собирается ли и загружается ли текущая версия ядра?» Прежде чем начать использовать `git bisect`, нужно понять,

какая версия коммита считается «хорошей», а какая — «плохой». Иначе говоря, для эффективного поиска важно предоставить правильный диапазон коммитов.

Под капотом команда `git bisect` применяет алгоритм двоичного поиска. Она раз за разом выбирает новый коммит в постепенно уменьшающемся диапазоне, ограниченном с одной стороны точкой с исправным поведением, а с другой — точкой с неисправным поведением¹. В итоге этот сужающийся диапазон сводится к единственному коммиту — виновнику неисправности. В общем виде этот принцип показан на рис. 8.1.

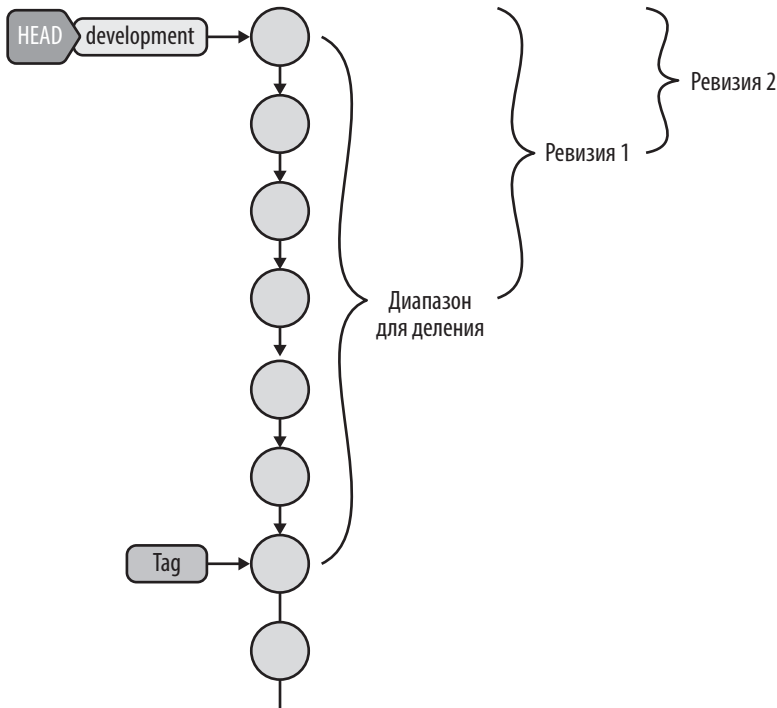


Рис. 8.1. Деление диапазона при выполнении `git bisect`

В режиме `bisect` достаточно предоставить только коммит, на момент фиксации которого все работало исправно, и второй коммит, в котором была обнаружена неисправность, после чего на каждой итерации поиска отвечать на вопрос: «Работает ли эта версия?».

Для начала определите коммит с исправным состоянием и коммит с неисправным. На практике неисправная версия зачастую представляет текущую `HEAD`,

¹ `bisect` означает «делить пополам». — *Примеч. ред.*

поскольку именно в ней вы работали, когда вдруг заметили какие-то неполадки или получили задание исправить баг.

Найти первоначальную хорошую версию может быть сложно, поскольку обычно она зарыта где-то в истории. Вы можете сделать предположение о том, что некая версия в истории репозитория работала правильно. Это может быть релиз с тегом, например `v2.6.25`, или какой-нибудь коммит 100 ревизий назад, скажем `main~100` в ветке `main`. В идеале он должен находиться сравнительно близко к коммиту с неисправностью (`main~25` лучше, чем `main~100`), а не слишком глубоко в истории. В любом случае вам нужно знать или иметь возможность убедиться, что это действительно «хороший» коммит.

При этом важно начинать процесс `git bisect` с чистым рабочим каталогом, который должен указывать на каталог верхнего уровня проекта. Процесс обязательно приведет к тому, что ваш рабочий каталог будет содержать различные версии репозитория. Начинать с «грязного» рабочего пространства в этом случае будет ошибкой. Вы можете легко потерять свой рабочий каталог. Если в нем есть какие-то рабочие изменения, то их можно временно сохранить, об этом мы подробно поговорим в главе 10.

Используя в качестве примера клон ядра Linux, попросим Git начать поиск:

```
$ cd linux-2.6
$ git bisect start
```

После инициализации двоичного поиска Git входит в соответствующий режим, устанавливая определенную информацию о состоянии. Для управления текущей версией репозитория Git использует отсоединенную HEAD — по сути, это анонимная ветка, которую можно взять для обхода репозитория и указания на различные ревизии.



Мы решили для демонстрации взять уже проверенный пример с использованием `git bisect` на устаревшей версии ядра Linux. Если вы захотите повторить этот пример, то учтите, что в качестве HEAD здесь выступает коммит `49fd6f785fd660e18a1eb4588928f47e9fa29a9a`.

Как только поиск начнется, укажите Git, какой коммит «плохой». Поскольку обычно это текущая версия, вы можете просто установить соответствующую ревизию на текущую HEAD.

```
# Сообщаем git, что текущая версия HEAD неисправна
$ git bisect bad
```

Далее сообщите Git, какая версия исправна:

```
$ git bisect good v2.6.27
Bisecting: 3857 revisions left to test after this
[cf2fa66055d718ae13e62451bb546505f63906a2] Merge branch 'for_linus'
of git://git.kernel.org/pub/scm/linux/kernel/git/mchhab/linux-2.6
```

Определяя исправную и неисправную версии, мы разграничиваем диапазон коммитов, в котором происходит переход из состояния «исправен» (good) в состояние «неисправен» (bad). Затем с каждым шагом Git будет сообщать, сколько ревизий находится в текущем диапазоне. При этом система также изменяет рабочий каталог, переключаясь на ревизию, которая находится примерно посередине между исправным и неисправным коммитами. Теперь вы отвечаете на вопрос: «Исправна ли эта версия?» При каждом ответе Git будет сужать пространство поиска вдвое, определять новую ревизию, переключаться на нее и повторять вопрос: «Исправна или неисправна?»

Например, эта версия оказалась исправной:

```
$ git bisect good
Bisecting: 1939 revisions left to test after this
[2be508d847392e431759e370d21cea9412848758] Merge git://git.infradead.org/mtd-2.6
```

Обратите внимание, что диапазон из 3857 ревизий был сужен до 1939. Прделаем еще пару шагов:

```
$ git bisect good
Bisecting: 939 revisions left to test after this
[b80de369aa5c7c8ce7ff7a691e86e1dcc89acc6] 8250: Add more OxBsemi devices
```

```
$ git bisect bad
Bisecting: 508 revisions left to test after this
[9301975ec251bab1ad7cfc84a688b26187e4e4a] Merge branch 'genirq-v28-for-linus'
of git://git.kernel.org/pub/scm/linux/kernel/git/tip/linux-2.6-tip
```

В идеальном цикле двоичного поиска команда вычисляет $\log_2$ от исходного количества ревизий, сужая их диапазон до единственного коммита.

Вот что мы получим после еще одного ответа:

```
$ git bisect good
Bisecting: 220 revisions left to test after this
[7cf5244ce4a0ab3f043f2e9593e07516b0df5715] mfd: check for
platform_get_irq() return value in sm501

$ git bisect bad
Bisecting: 104 revisions left to test after this
[e4c2ce82ca2710e17cb4df8eb2b249fa2eb5af30] ring_buffer: allocate
buffer page pointer
```

В процессе двоичного поиска Git составляет журнал ответов, внося в него также ID коммитов:

```
$ git bisect log
git bisect start
# bad: [49fd6f785fd660e18a1eb4588928f47e9fa29a9a] Merge branch
'for-linus' of git://git.kernel.dk/linux-2.6-block
git bisect bad 49fd6f785fd660e18a1eb4588928f47e9fa29a9a
# good: [3fa8749e584b55f1180411ab1b5117190bac1e5] Linux 2.6.27
```

```

git bisect good 3fa8749e584b55f1180411ab1b51117190bac1e5
# good: [cf2fa66055d718ae13e62451bb546505f63906a2] Merge branch 'for_linus'
      of git://git.kernel.org/pub/scm/linux/kernel/git/mchehab/linux-2.6
git bisect good cf2fa66055d718ae13e62451bb546505f63906a2
# good: [2be508d847392e431759e370d21cea9412848758] Merge
      git://git.infradead.org/mtd-2.6
git bisect good 2be508d847392e431759e370d21cea9412848758
# bad: [b80de369aa5c7c8ce7ff7a691e86e1dcc89acc6] 8250: Add more
      OxBsemi devices
git bisect bad b80de369aa5c7c8ce7ff7a691e86e1dcc89acc6
# good: [9301975ec251bab1ad7cfc84a688b26187e4e4a] Merge branch
      'genirq-v28-for-linus' of
      git://git.kernel.org/pub/scm/linux/kernel/git/tip/linux-2.6-tip
git bisect good 9301975ec251bab1ad7cfc84a688b26187e4e4a
# bad: [7cf5244ce4a0ab3f043f2e9593e07516b0df5715] mfd: check for
      platform_get_irq() return value in sm501
git bisect bad 7cf5244ce4a0ab3f043f2e9593e07516b0df5715

```

Если вы потерялись в этом процессе или просто хотите по какой-то причине начать заново, введите **git bisect replay**, используя в качестве ввода созданный журнал¹. При необходимости это позволит сделать шаг назад и исследовать другой путь.

Теперь сузим диапазон поиска с помощью еще пяти ответов «bad» (неисправен):

```

$ git bisect bad
Bisecting: 51 revisions left to test after this
[d3ee6d992821f471193a7ee7a00af9ebb4bf5d01] ftrace: make it
      depend on DEBUG_KERNEL

$ git bisect bad
Bisecting: 25 revisions left to test after this
[3f5a54e371ca20b119b73704f6c01b71295c1714] ftrace: dump out
      ftrace buffers to console on panic

$ git bisect bad
Bisecting: 12 revisions left to test after this
[8da3821ba5634497da63d58a69e24a97697c4a2b] ftrace: create
      _mcount_loc section

$ git bisect bad
Bisecting: 6 revisions left to test after this
[fa340d9c050e78fb21a142b617304214ae5e0c2d] tracing: disable
      tracepoints by default

$ git bisect bad
Bisecting: 2 revisions left to test after this
[4a0897526bbc5c6ac0df80b16b8c60339e717ae2] tracing: tracepoints, samples

```

Используйте команду **git bisect visualize**, чтобы просмотреть набор коммитов, все еще находящихся в диапазоне поиска. Если установлена переменная среды

¹ Сгенерировать журнал можно с помощью команды **git bisect log**.

DISPLAY, Git применяет графический инструмент `gitk`. Если нет, задействуется `git log`. В таком случае также может оказаться полезной опция `--pretty=oneline`:

```
$ git bisect visualize --pretty=oneline
```

```
fa340d9c050e78fb21a142b617304214ae5e0c2d tracing: disable tracepoints
by default
b07c3f193a8074aa4afe43cfa8ae38ec4c7ccfa9 ftrace: port to tracepoints
0a16b6075843325dc402edf80c1662838b929aff tracing, sched: LTTng
instrumentation - scheduler
4a0897526bbc5c6ac0df80b16b8c60339e717ae2 tracing: tracepoints, samples
24b8d831d56aac7907752d22d2aba5d8127db6f6 tracing: tracepoints,
documentation
97e1c18e8d17bd87e1e383b2e9d9fc740332c8e2 tracing: Kernel Tracepoints
```

Текущая рассматриваемая ревизия находится примерно в середине диапазона:

```
$ git bisect good
```

```
Bisecting: 1 revisions left to test after this
```

```
[b07c3f193a8074aa4afe43cfa8ae38ec4c7ccfa9] ftrace: port to tracepoints
```

После тестирования последней ревизии Git отделяет ту, которая внесла проблему¹, и выводит ее:

```
$ git bisect good
```

```
fa340d9c050e78fb21a142b617304214ae5e0c2d is first bad commit
```

```
commit fa340d9c050e78fb21a142b617304214ae5e0c2d
```

```
Author: Ingo Molnar <mingo@elte.hu>
```

```
Date: Wed Jul 23 13:38:00 2008 +0200
```

```
tracing: disable tracepoints by default
```

```
while it's arguably low overhead, we don't enable new features by default.
```

```
Signed-off-by: Ingo Molnar <mingo@elte.hu>
```

```
:040000 040000 4bf5c05869a67e184670315c181d76605c973931
```

```
fd15e1c4adbd37b819299a9f0d4a6ff589721f6c M init
```

Разобранный метод утомителен тем, что нужно перебирать большое число ревизий для выявления коммита, который внес неполадки. К счастью, можно поручить Git выполнить ту же операцию с помощью скрипта или команды. Это сэкономит время при поиске проблемы, если имеется большое число ревизий.

Вот как можно добиться того же результата:

```
# git bisect start <bad-commit-SHA> <good-commit-SHA>
```

```
$ git bisect start HEAD v2.6.27
```

```
$ git bisect run <command>
```

¹ Не факт, что именно этот коммит внес проблему, поскольку ответы «good» и «bad» были симитированы.

```
...
...
...
fa340d9c050e78fb21a142b617304214ae5e0c2d is first bad commit
commit fa340d9c050e78fb21a142b617304214ae5e0c2d
Author: Ingo Molnar <mingo@elte.hu>
Date:   Wed Jul 23 13:38:00 2008 +0200
```

```
tracing: disable tracepoints by default
```

while it's arguably low overhead, we don't enable new features by default.

```
Signed-off-by: Ingo Molnar <mingo@elte.hu>
```

```
:040000 040000 4bf5c05869a67e184670315c181d76605c973931
fd15e1c4adbd37b819299a9f0d4a6ff589721f6c M init
```

В предыдущем примере *command* может быть скриптом, определяющим, находится ли текущая версия кода в исправном состоянии. Если брать наш пример, то скрипт должен оценивать, можем ли мы загрузиться без сбоя.

Если текущее состояние исходного кода — *good*, то скрипт должен завершаться с кодом 0, а если *bad*, то с кодом в диапазоне от 1 до 127 (кроме 125).

Когда двоичный поиск завершен и вы получаете журнал результатов и сохраняете состояние, то важно сообщить Git, что вы закончили работу. Напомню, что весь процесс поиска выполняется для отсоединенной HEAD:

```
$ git branch
* (no branch, bisect started on main)
  main

$ git bisect reset
Previous HEAD position was <COMMIT-SHA1> ...
Switched to branch "main"

$ git branch
* main
```

Выполнение `git bisect reset` перемещает вас обратно на исходную ветку.



Можно заменить ключевые слова *bad* и *good* терминами, которые лучше отражают критерий поиска (например, при проверке снижения производительности). Для этого нужно сопроводить команду `git bisect start` опциями `--term-new` и `--term-old`:

```
git bisect start --term-old fast --term-new slow
git bisect fast
git bisect slow
```

(fast — быстрый, slow — медленный. — *Примеч. ред.*)

Использование git blame

Еще один инструмент, помогающий искать конкретный коммит, — `git blame`. Эта команда сообщает, кто последним изменял каждую отдельную строку файла и какой коммит внес это изменение:

```
$ git blame -L 35, init/version.c
```

```
4865ecf1 (Serge E. Hallyn 2006-10-02 02:18:14 -0700 35)      },
^1da177e (Linus Torvalds 2005-04-16 15:20:36 -0700 36)  });
4865ecf1 (Serge E. Hallyn 2006-10-02 02:18:14 -0700 37)  EXPORT_SYMBOL_GPL(init_uts_ns);
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 38)
c71551ad (Linus Torvalds 2007-01-11 18:18:04 -0800 39)  /* FIXED STRINGS!
Don't touch! */
c71551ad (Linus Torvalds 2007-01-11 18:18:04 -0800 40)  const char linux_banner[] =
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 41)      "Linux version "
UTS_RELEASE "
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 42)      (" LINUX_COMPILE_BY "@"
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 43)      LINUX_COMPILE_HOST ")
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 44)      (" LINUX_COMPILER ")
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 45)      " UTS_VERSION "\n";
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 46)
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 47)  const char linux_proc_banner[] =
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 48)      "%s version %s"
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 49)      " (" LINUX_COMPILE_BY
"@
LINUX_COMPILE_HOST ")
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 50)      " (" LINUX_COMPILER ")
3eb3c740 (Roman Zippel 2007-01-10 14:45:28 +0100 51)      %s\n";
```



При использовании с командой `git blame` опция `-L` позволяет указывать диапазоны. Пример: `git blame -L start-line-no,end-line-no`.

Большинство современных клиентов Git имеют расширения, которые при просмотре в окне текстового редактора по умолчанию отображают вывод `git blame` для каждой строки файла. Это помогает понять, кто внес в файл последнее изменение.

Если вам интересно узнать, кто изменял файл в течение последнего времени, то можете ограничить диапазон ревизий, сопроводив команду `git blame` опцией `<revisions range>` (диапазон ревизий):

```
$ git blame --since=1.weeks -- file
```

Использование инструмента Pickaxe

Опция `-S`, дополняющая команду `git log`, называется *pickaxe* (кирка). Это инструмент, работающий по методу грубого перебора (брутфорса).

В то время как `git blame` сообщает о текущем состоянии файла, `git log -Sstring` выполняет поиск по истории его изменений для заданной *string*. Просматривая фактические отличия между ревизиями, команда может найти коммиты, которые вносят изменения как добавлением, так и удалением:

```
$ git log -Sinclude --pretty=oneline --abbrev-commit init/version.c
cd354f1... [PATCH] remove many unneeded #includes of sched.h
4865ecf... [PATCH] namespaces: utsname: implement utsname namespaces
63104ee... kbuild: introduce utsrelease.h
1da177e... Linux-2.6.12-rc2
```

В этом коде каждый перечисленный слева коммит (`cd354f1` и т. д.) будет либо добавлять, либо удалять строки, содержащие слово `include`. Но нужно быть внимательными. Если коммит и добавляет, и удаляет одинаковое количество экземпляров строк, содержащих указанное ключевое слово, это не отобразится в выводе. Количество добавлений и удалений в коммите должно *изменяться*, чтобы они брались в расчет.



Использование опции `-g` с командой `git log` позволяет показывать коммиты, которые соответствуют указанному регулярному выражению, независимо от того, сбалансировано ли количество добавлений или удалений в этих коммитах.

Итоги

В этой короткой главе мы рассмотрели три метода для поиска коммитов, обусловивших определенные состояния проекта. Будь то баг, проблема производительности или даже ошибка компиляции, естественным следующим шагом станет выяснение того, как исправить ситуацию. Полученные в этой главе знания часто используются совместно с навыками, необходимыми для изменения коммитов. Эти навыки мы и обсудим в следующей главе.

Изменение коммитов

Коммит записывает историю вашей работы и сохраняет внесенные вами изменения. В Git есть несколько инструментов и команд, помогающих изменять и улучшать историю коммитов в вашем репозитории.

Есть множество причин, по которым нужно изменить коммит или их последовательность. Например:

- исправить проблему до того, как она перейдет в разряд легаси;
- разбить обширное изменение на несколько маленьких тематических коммитов или, наоборот, совместить несколько отдельных изменений в один большой коммит;
- внести обратную связь и предложения в процессе разработки проекта;
- изменить порядок коммитов в последовательности, которая не нарушает условия сборки;
- упорядочить коммиты более логично;
- удалить отладочный код, который был зафиксирован по ошибке;
- удалить чувствительные данные в коде, которые были зафиксированы «случайно».¹

В главе 11, где будет объясняться техника совместного использования репозитория, вы увидите, что есть и другие причины для изменения коммитов до публикации репозитория.

В этой главе мы разберем принципы изменения историй коммитов, их последствия, а также инструменты Git, с помощью которых можно изменить отдельный коммит или их серию в истории репозитория.

¹ Если вы не отправили коммит в удаленный репозиторий, то эти чувствительные данные находятся только в вашем локальном репозитории. Вы по-прежнему можете изменить коммит и исключить из локального репозитория все коммиты, на которые нет ссылок.



Все, что фиксируется в Git, остается в Git (в истории коммитов). Будьте внимательны к тому, как отслеживаются или даже контролируются версии изменений, связанных с конфиденциальными данными в коде. Это поможет избежать незначительных изменений в коммитах, приводящих к переписыванию истории репозитория.

Изменение истории коммитов. Общие принципы

Существует несколько концепций того, как следует управлять историей разработки. Первую можно описать как *реалистичную историю*: сохраняется каждый коммит, и ничто не изменяется.

Один из вариантов этой концепции называется *детальной* реалистичной историей, когда вы фиксируете каждое изменение как можно раньше, сохраняя каждый шаг. Другой вариант — это *дидактическая* реалистичная история, когда вы не спешите и фиксируете только успешные результаты работы в удобный момент.

Имея возможность корректировать историю, например вычищая из нее неудачное промежуточное решение или перестраивая коммиты в более логичном порядке, вы можете создать более «идеалистичный» вариант истории.

В реальности разработка идет в быстром темпе. В этой главе мы начнем с первого коммита и будем постепенно добавлять новые. Они будут включать новую функциональность, правки опечаток и исправления на основе результатов выполнения цикла непрерывной интеграции/развертывания (CI/CD, Continuous Integration/Continuous Delivery) (<https://oreil.ly/PThXB>), причем не обязательно в последовательном порядке. Эти действия приведут нашу историю коммитов в беспорядочное, неорганизованное состояние. Нам же нужно, чтобы репозитории имели хорошо структурированную и упорядоченную историю коммитов.

Мне нравится думать об этом как о разнице между ведением дневника и написанием книги. В дневнике вы фиксируете все как есть. Разные детали описывают, что именно происходило каждый день. Книга же, в отличие от дневника, проходит несколько этапов редактирования и правок, прежде чем публикуется ее финальная версия. Книга представляет собой структурированное повествование о неких событиях. Не забывайте об этой аналогии при чтении данной главы: так вы поймете, какой метод подходит для ваших проектов.

Как разработчик, вы можете найти ценность в детальной и реалистичной истории, поскольку она дает все необходимые подробности, касающиеся и успешных, и провальных идей. Полный рассказ покажет, как появился баг, или подскажет способ исправить ошибку. Более того, анализ истории даст представление о том, как работает программист или команда и как можно улучшить этот процесс.

Многие из деталей утратятся, если в истории не будет промежуточных шагов. Мог ли разработчик дойти до удачного решения интуитивно? Или же ему

потребовалось несколько итераций доработки? В чем первопричина бага? Если промежуточные шаги не отражены в истории коммитов, то ответить на подобные вопросы будет сложно.

Однако с более чистой историей, отражающей лишь определенные шаги и показывающей прогресс, работать приятнее. Тогда не нужно беспокоиться о последствиях, возможно, ошибочного или неоптимального шага. Другие разработчики, читая такую историю, лучше поймут стиль разработки.

Так что же лучше? Подробная реалистичная история без потери информации? Или же более чистая? Возможно, оптимальным вариантом будет промежуточное представление процесса разработки.

А что, если правильно используя ветки Git, вы сможете представить в одном репозитории и детальную реалистичную историю, и ее идеализированную форму?

Git позволяет очистить фактическую историю и превратить ее в более чистую, прежде чем публиковать или фиксировать в публично доступную запись. Будете ли вы сохранять подробную историю без изменений или выберете что-то среднее — зависит только от вас и политики вашего проекта.

Изменения истории: будьте осторожны

Как правило, вы можете свободно изменять и дорабатывать историю репозитория до тех пор, пока никто не получил его копию. Точнее говоря, вы можете изменять *конкретную ветку* до тех пор, пока ни у кого нет ее копии. Помните, что не следует переписывать или изменять какую-либо часть ветки, которую вы сделали доступной и которая может находиться в общем репозитории.



С появлением запросов `pull` и `merge` на популярных платформах хостинга Git стали доступны варианты переписывания истории конкретных веток, даже если такая ветка функциональности была опубликована. В большинстве ситуаций владельцем или автором этой ветки будет один разработчик, что позволяет ему переписывать историю ее коммитов после публикации. И если на эту ветку переключится другой разработчик, то сделает это на свой страх и риск, поскольку при согласовании изменений могут возникнуть проблемы.

Например, вы работали над своей веткой `main` и сделали коммиты от A до D, доступные другому разработчику (рис. 9.1). Как только вы делаете историю разработки доступной другому разработчику, она становится *опубликованной историей*.

Затем вы продолжаете разработку и создаете в той же ветке новые коммиты от W до Z в виде неопубликованной истории (рис. 9.2).

В этой ситуации нужно быть очень внимательными и оставлять коммиты, предшествующие W, отдельно. Но пока вы не опубликуете заново ветку `main`, причины, которая препятствовала бы изменению коммитов от W до Z, нет. Сюда может входить изменение их порядка, совмещение и удаление одного или нескольких и, конечно, добавление новых.

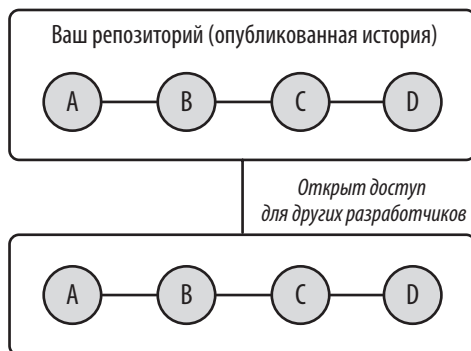


Рис. 9.1. Ваша опубликованная история

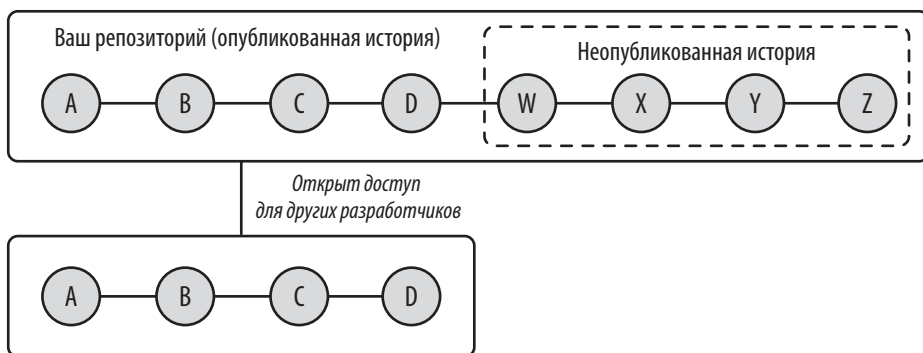


Рис. 9.2. Ваша неопубликованная история

В итоге может получиться новая улучшенная история коммитов (рис. 9.3). В этом примере коммиты X и Y были немного изменены, сформировав похожий коммит W'; коммит Z был перемещен на более ранний этап истории; был внесен новый коммит P.

Далее мы рассмотрим техники, которые помогут изменять и улучшать историю коммитов. А вот стала ли ваша история лучше, в какой момент она достаточно хороша и когда ее можно публиковать — решать вам.

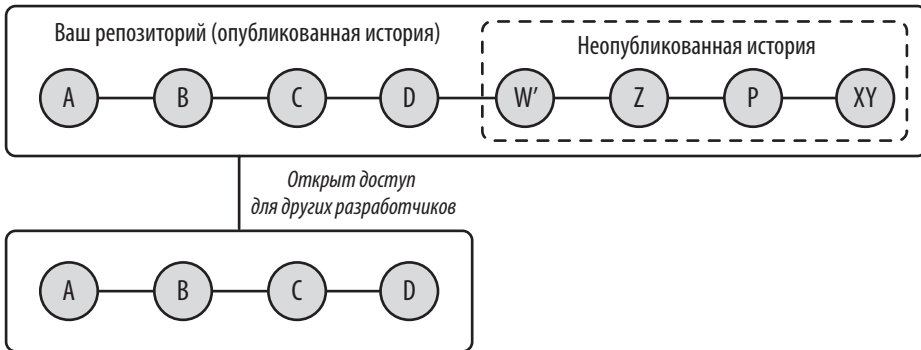


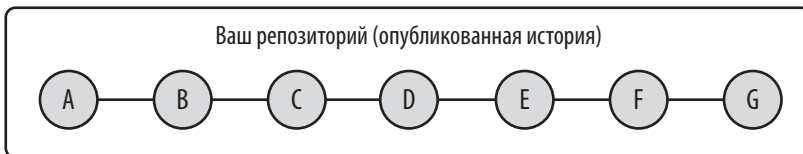
Рис. 9.3. Ваша новая история

Использование `git revert`

Команда `git revert commit` выполняет *инверсию* указанного коммита `commit` (где `commit` представляет собой снимок репозитория в определенный момент времени). Как правило, эта команда используется для внесения коммита, который отменяет изменения указанного коммита.

Действие `revert` не *изменяет* существующую историю репозитория, а добавляет в нее новый коммит. Эта команда обычно считается безопасной при изменении коммитов в общем репозитории.

Обычно ее применяют для отмены коммита, который находится глубоко в истории ветки. На рис. 9.4 история изменений была построена на основе ветки `main`. По какой-то причине, возможно в результате тестирования, коммит D был сочтен ошибочным.

Рис. 9.4. До выполнения простой команды `git revert`

Один из вариантов исправления ситуации — внести правки для отмены коммита D и затем зафиксировать эти правки напрямую. В сообщении коммита указано, что его цель — отменить (`revert`) изменения, вызванные предыдущим коммитом.

Более простое решение — выполнить `git revert`:

```
$ git revert main~3    # commit D
```

В итоге мы получим картину как на рис. 9.5, где коммит D' — это инвертированная версия коммита D.

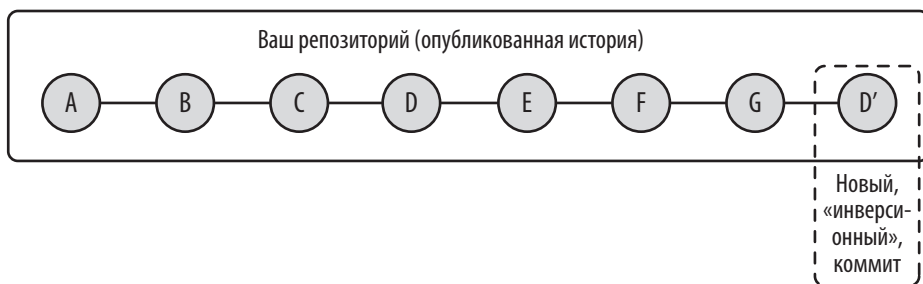


Рис. 9.5. После выполнения простой команды `git revert`

Изменение HEAD-коммита

Один из простейших способов изменить последний коммит в текущей ветке — использовать `git commit --amend`. Как правило, *исправление* (amend) подразумевает, что коммит должен сохранить основное *содержимое*, но при этом нужно исправить или подчистить какую-то его часть. При этом фактический объект коммита, который заново вносится в объектное хранилище, будет другим.

Использование `git commit --amend`, как правило, нужно для исправления опечаток сразу после коммита. Но это не единственный случай ее применения. Как и в случае любого коммита, эта команда может исправить любой файл/файлы в репозитории и добавить/удалить файл в рамках нового коммита.

По аналогии с обычной `git commit` опция `git commit --amend` переносит вас в сеанс редактора, где можно изменить сообщение коммита.

Предположим, мы работаем над речью (да, мы отслеживаем версии наших речей) и последним внесли такой коммит:

```
$ git show
commit 0ba161a94e03ab1e2b27c2e65e4cbef476d04f5d
Author: Jon Loeliger <jdl@example.com>
Date: Thu Jun 26 15:14:03 2008 -0500
```

```
Initial speech
```

```
diff --git a/speech.txt b/speech.txt
new file mode 100644
index 0000000..310bcf9
```

```

--- /dev/null
+++ b/speech.txt
@@ -0,0 +1,5 @@
+Three score and seven years ago
+our fathers brought forth on this continent,
+a new nation, conceived in Liberty,
+and dedicated to the proposition
+that all women are created equal.

```

В этот момент он хранится в репозитории объектов Git, пусть даже с некоторыми ошибками в тексте. Чтобы внести правки, можно еще раз отредактировать файл и сделать новый коммит.

Получится такая история:

```

$ git show-branch --more=5
[main] Fix timeline typo
[main^] Initial speech

```

Но если вы хотите, чтобы история была чище, то можете изменить коммит напрямую и заменить его. Для этого скорректируйте файл в рабочем каталоге. Исправьте опечатки и при необходимости добавляйте другие файлы. Как и в случае любого коммита, обновите индекс изменениями с помощью команды `git add` или `git rm`. Затем выполните `git commit --amend`:

Редактируем `speech.txt` так, как требуется

```

$ git diff
diff --git a/speech.txt b/speech.txt
index 310bcf9..7328a76 100644
--- a/speech.txt
+++ b/speech.txt
@@ -1,5 +1,5 @@
-Three score and seven years ago
+Four score and seven years ago
  our fathers brought forth on this continent,
  a new nation, conceived in Liberty,
  and dedicated to the proposition
- that all women are created equal.
+ that all women and men are created equal.

```

```
$ git add speech.txt
```

```
$ git commit --amend
```

```

# Также при желании редактируем сообщение коммита "Initial speech"
# Здесь мы его немного изменили...

```

```

$ git show-branch --more=5
[main] Initial speech that sounds familiar.

```

```

$ git show
commit 47d849c61919f05da1acf983746f205d2cdb0055
Author: Jon Loeliger <jdl@example.com>
Date: Thu Jun 26 15:14:03 2008 -0500

```

Initial speech that sounds familiar.

```
diff --git a/speech.txt b/speech.txt
new file mode 100644
index 0000000..7328a76
--- /dev/null
+++ b/speech.txt
@@ -0,0 +1,5 @@
+Four score and seven years ago
+our fathers brought forth on this continent,
+a new nation, conceived in Liberty,
+and dedicated to the proposition
+that all women and men are created equal.
```

Эта команда также может редактировать метаданные коммита. Например, передав опцию `--author`, вы можете изменить автора:

```
$ git commit --amend --author "Bob Miller <kbob@example.com>"
# ...просто закройте редактор...
```

```
$ git log
commit 0e2a14f933a3aaff9edd848a862e783d986f149f
Author: Bob Miller <kbob@example.com>
Date: Thu Jun 26 15:14:03 2008 -0500
```

Initial speech that sounds familiar.



Внимательные читатели заметили, что с каждым выполнением `git commit --amend` генерировался новый ID SHA-1 (на основе измененного содержимого), который затем был внесен в объектное хранилище Git.

Изменение HEAD, или верхнего коммита, с помощью `git commit --amend` ведет к изменению графа коммитов: рис. 9.6 становится рис. 9.7.

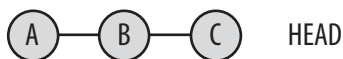


Рис. 9.6. Граф коммитов до `git commit --amend`

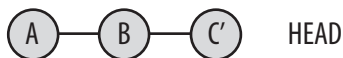


Рис. 9.7. Граф коммитов после `git commit --amend`

Суть коммита С осталась прежней, но он был изменен и стал С'. Ссылка на HEAD также изменилась и теперь указывает не на старый коммит С, а на его обновленную версию С'.

Использование git reset

Команда `git reset` изменяет репозиторий и рабочий каталог на известное состояние. В частности, она корректирует заданную ссылку `HEAD` на указанный *commit* и по умолчанию обновляет индекс, чтобы он соответствовал этому коммиту. При желании с помощью `git reset` можно изменять и рабочий каталог, чтобы он отражал ревизию проекта, представленную указанным коммитом. Эта команда также используется в составе других операций, и здесь мы разберем, как с ее помощью можно изменять историю репозитория.

Можно сказать, что `git reset` — это «деструктивная» команда, поскольку может переписывать и уничтожать изменения в рабочем каталоге. Данные действительно могут быть утрачены. И даже если у вас есть резервная копия файлов, не всегда получится восстановить работу. Тем не менее весь смысл этой команды в том, чтобы устанавливать и восстанавливать известные состояния `HEAD`, индекса и рабочего каталога.

Когда вы собираетесь использовать `git reset`, полезно представить состояние истории репозитория *до* и *после*. Иными словами, *до* выполнения `git reset` мой репозиторий находится в определенном текущем состоянии. А как я хочу, чтобы он выглядел *после* ее выполнения? Этот принцип представлен на рис. 9.8.

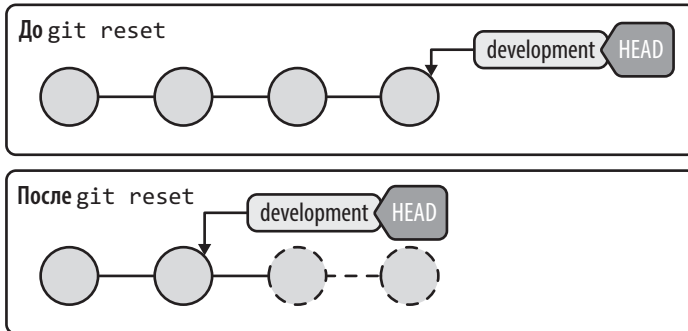


Рис. 9.8. До и после `git reset`



Еще один способ понять принцип работы `git reset` — представить историю коммитов в виде шкалы времени. Ссылка `HEAD` — это значок, указывающий на конкретную точку (конкретный момент) этой шкалы. Выполняя `git reset`, вы можете *отмотать назад* или *перемотать вперед* к нужному моменту в истории репозитория.

У команды `git reset` есть три основные опции: `--soft`, `--mixed` и `--hard`. Они влияют на то, как в итоге обновится индекс либо рабочий каталог.

Посмотрите, как на рис. 9.9–9.11 `git reset` сбрасывает ссылку HEAD, чтобы та указывала на коммит В в истории репозитория:

`git reset --soft commit`

Опция `--soft` переадресует HEAD на указанный *commit*. Содержимое индекса и рабочего каталога при этом не меняется. Эта версия команды имеет «самый слабый» эффект, изменяя только состояние символической ссылки, чтобы та указывала на новый коммит.

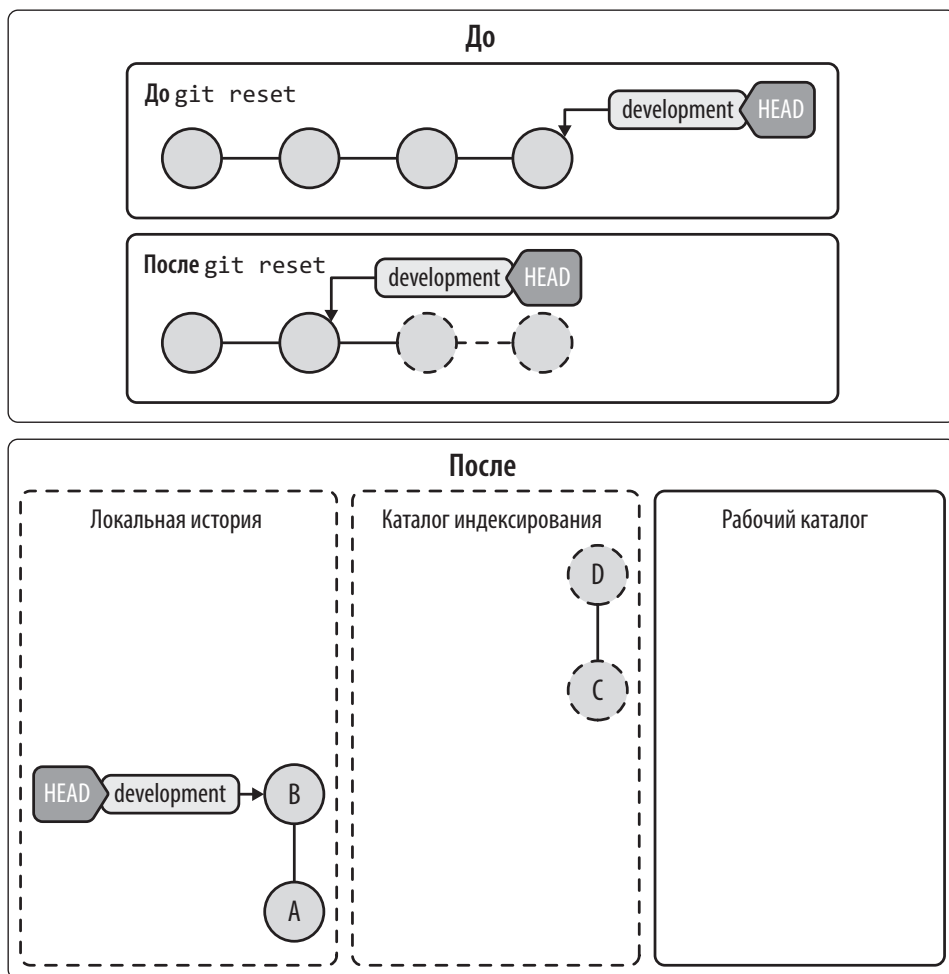


Рис. 9.9. До и после `git reset --soft` (в рабочем каталоге будут отражены изменения файлов, которые были внесены коммитами С и D)

`git reset --mixed commit`

Опция `--mixed` перенаправляет `HEAD` на указанный `commit`. Содержимое индекса также изменяется, чтобы соответствовать структуре дерева, которую указал `commit`, но содержимое рабочего каталога остается без изменений. Эта версия команды приводит индекс в такое состояние, будто вы только что индексировали все изменения, представленные `commit`, и сообщает, что конкретно изменяется в рабочем каталоге. Заметьте, что `--mixed` используется для `git reset` по умолчанию.

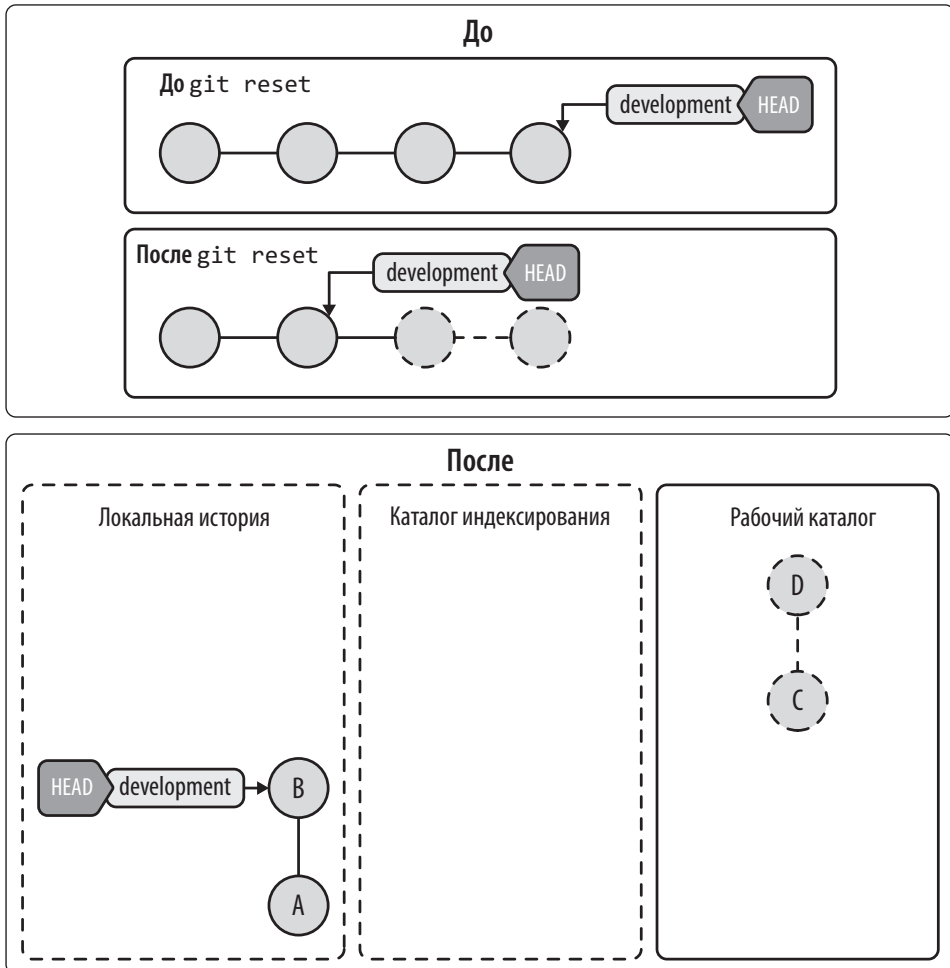


Рис. 9.10. До и после `git reset --mixed` (в рабочем каталоге будут отражены изменения файлов, которые были внесены коммитами C и D)

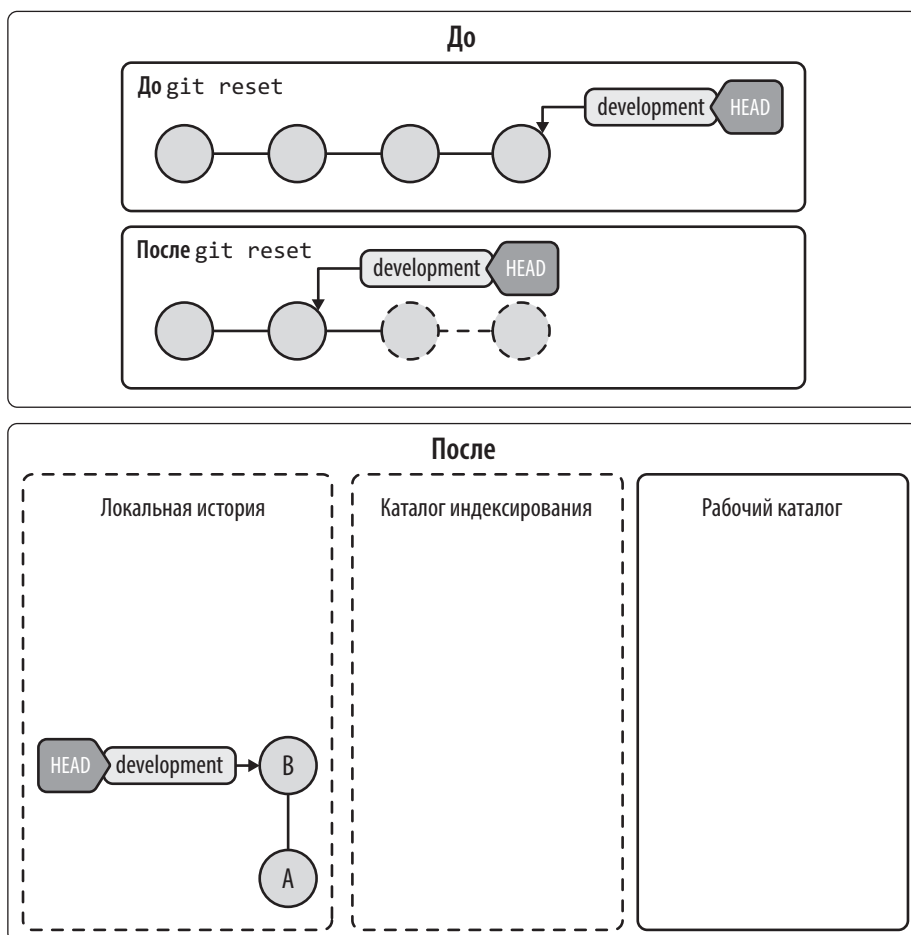


Рис. 9.11. До и после `git reset --hard` (изменения в файлах, которые были внесены коммитами C и D, будут утрачены)

`git reset --hard commit`

Опция `--hard` перенаправляет ссылку `HEAD` на указанный `commit`. Содержимое индекса также изменяется, согласуясь со структурой дерева, указанной `commit`. Изменяется и содержимое рабочего каталога, отражая состояние, представленное заданным `commit`.

При изменении рабочего каталога изменяется вся структура каталогов в соответствии с указанным `commit`. Корректировки утрачиваются, а новые файлы, которые были добавлены в репозиторий, но не отслеживаются, остаются нетронутыми. Файлы, которые находятся в указанном `commit`, но уже не существуют в рабочем каталоге, восстанавливаются.

В табл. 9.1 дана сводная информация о том, на что влияют опции `git reset`.

Таблица 9.1. Влияние опций `git reset`

Опция	HEAD	Индекс	Рабочий каталог
<code>--soft</code>	Да	Да	Нет
<code>--mixed</code>	Да	Нет	Да
<code>--hard</code>	Да	Да	Да

Команда `git reset` также сохраняет исходное значение HEAD в ссылке `ORIG_HEAD`. Это полезно, когда нужно использовать сообщение коммита исходной HEAD как основу для последующего коммита. Чтобы повторно использовать сообщение коммита, передайте аргумент `--reuse-message=ORIG_HEAD`.

С точки зрения объектной модели `git reset` перемещает текущую ветку HEAD в графе коммитов на определенный коммит. Помните: если вы укажете опцию `--hard`, то рабочий каталог тоже будет изменен.

Рассмотрим примеры работы `git reset` и начнем с настройки ветки с несколькими коммитами:

```
$ git init -b main
Initialized empty Git repository in /tmp/reset/.git/

$ echo Do >> main_file

$ git add main_file

$ git commit -m "Do"
[main (root-commit) d8789aa] Do
 1 file changed, 1 insertion(+)
 create mode 100644 main_file

$ echo Fa >> file_1
$ git add file_1
$ git commit -m "Fa"
[main cf6315c] Fa
 1 file changed, 1 insertion(+)
 create mode 100644 file_1

$ ls
file_1    main_file

$ git log --oneline
cf6315c (HEAD -> main) Fa
d8789aa Do
```

Допустим, вы добавляете `file_2` и индексируете его, а потом понимаете, что нужно совместить это индексированное изменение с предыдущим коммитом, добавившим `file_1`. Теперь вы хотите вернуться назад, чтобы сделать это.

Вы можете использовать `git reset --soft commit:`

```
$ echo Mi >> file_2
$ git add file_2

$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   file_2

$ git reset --soft HEAD^

$ git log --oneline
d8789aa (HEAD -> main) Do

# Файлы в каталоге
$ ls
file_1  file_2  main_file

# Статус Git после того, как reset soft покажет,
# что file_2 и file_1 находятся в индексе
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   file_1
    new file:   file_2

# Создание нового коммита с file_1 и file_2
$ git commit -m "Fa and Mi"
[main 0c8a4ba] Fa and Mi
 2 files changed, 2 insertions(+)
 create mode 100644 file_1
 create mode 100644 file_2

$ git log --oneline
0c8a4ba (HEAD -> main) Fa and Mi
d8789aa Do
```

Как вы помните из раздела «Идентификация коммитов» на с. 102, `HEAD^` ссылается на родителя коммита текущей вершины `main` и представляет состояние непосредственно до завершения второго коммита, который добавил `file_1`.

Продолжим:

```
$ echo Re >> file_3
$ echo So >> file_4
```

С добавлением содержимого для новых файлов может потребоваться изменить последовательность коммитов, чтобы отразить прогресс в логической последовательности.

Тогда используйте `git reset --mixed commit`:

```
# --mixed – это опция reset по умолчанию,
# поэтому передавать ее явно не нужно
$ git reset HEAD^

$ git log --oneline
d8789aa (HEAD -> main) Do

# Файлы в каталоге
$ ls
file_1    file_2    file_3    file_4    main_file

# Статус Git после того, как reset mixed показал все файлы в рабочем каталоге
$ git status
On branch main
Untracked files:
(use "git add <file>..." to include in what will be committed)
    file_1
    file_2
    file_3
    file_4

# Создание нового коммита со всеми файлами,
# чтобы отразить логическую последовательность коммитов
$ git add file_3
$ git commit -m "Re - content from file_3"
[main 61e676f] Re - content from file_3
1 file changed, 1 insertion(+)
create mode 100644 file_3

# Продолжаем добавлять и коммитить файлы в правильном порядке
# на основе их содержимого
$ git add file_2
$ git commit -m "Mi - content from file_2"
[main 4ecc185] Mi - content from file_2
1 file changed, 1 insertion(+) create mode 100644 file_2

$ git add file_1
$ git commit -m "Fa - content from file_1"
[main 4d72acb] Fa - content from file_1
1 file changed, 1 insertion(+)
create mode 100644 file_1

$ git add file_4
$ git commit -m "So - content from file_4"
[main 3ab8f23] So - content from file_4
1 file changed, 1 insertion(+)
create mode 100644 file_4
```

```
$ git log --oneline
3ab8f23 (HEAD -> main) So - content from file_4
4d72acb Fa - content from file_1
4ecc185 Mi - content from file_2
61e676f Re - content from file_3
d8789aa Do
```

Поскольку опция `--mixed` сбрасывает индекс, нужно заново индексировать все изменения, которые вы хотите внести в новом коммите. Это позволяет откорректировать существующие файлы, добавить другие или внести иные изменения перед выполнением коммита.

В некоторых случаях может потребоваться полностью избавиться от серии новых коммитов, поскольку их содержимое вам не важно. Для этого используйте `git reset --hard commit`:

```
$ git log --oneline
3ab8f23 (HEAD -> main) So - content from file_4
4d72acb Fa - content from file_1
4ecc185 Mi - content from file_2
61e676f Re - content from file_3
d8789aa Do
```

```
$ git reset --hard HEAD~4
HEAD is now at d8789aa Do
```

```
$ git log --oneline
d8789aa (HEAD -> main) Do
```

```
# Файлы в каталоге
$ ls
main_file
```

```
# Статус Git после того, как reset hard сбросил индекс и рабочий каталог,
# чтобы отразить состояние после коммита main_file
$ git status
On branch main
nothing to commit, working tree clean
```

```
# Создание нового коммита с новым содержимым и именем файла
$ echo "Do Re Mi Fa So" >> new_file
$ git add new_file
$ git commit -m "Add new_file"
[main 46623d8] Add new_file
 1 file changed, 1 insertion(+)
 create mode 100644 new_file
```

```
$ git log --oneline
46623d8 (HEAD -> main) Add new_file
d8789aa Do
```

Опция `--hard` приводит к тому, что ветка `main` возвращается к определенному состоянию. В нашем примере она изменяет рабочий каталог, чтобы тот отражал

указанное состояние (HEAD~4), в частности состояние, когда в рабочем каталоге был создан `main_file`.

Несмотря на то что во всех примерах используется некая форма HEAD, вы можете применить `git reset` к любому коммиту в репозитории. Например, чтобы убрать несколько коммитов из текущей ветки, используйте `git reset --hard HEAD~4`, `git reset --hard d8789aa` или даже `git reset --hard ORIG_HEAD`.

Но будьте внимательны. То, что вы можете именовать другие коммиты, используя имя ветки, не значит, что вы переключаетесь на эту ветку. При выполнении `git reset` вы остаетесь в той же самой ветке. Вы можете изменить рабочий каталог, чтобы он *выглядел как* вершина другой ветки, но находиться все равно будете в исходной.

Использование git cherry-pick

Команда `git cherry-pick commit` применяет изменения, внесенные именованным `commit` в текущей ветке. Она вносит новый отдельный коммит. Строго говоря, при использовании `git cherry-pick` мы не *изменяем* существующую историю репозитория, а дополняем ее.

Как и в случае с другими операциями Git, которые вносят изменения через `diff`, для полноценного внесения изменений из указанного `commit` может потребоваться разрешить конфликты.

Обычно команда `git cherry-pick` используется для внесения изменений конкретных коммитов из одной ветки репозитория в другую. Это делается в случае их переноса между веткой обслуживания (`maintenance branch`) и веткой разработки (`dev`).

На рис. 9.12 ветка `dev` содержит обычную линию разработки, в то время как в `rel_2.3` находятся коммиты для обслуживания релиза 2.3.

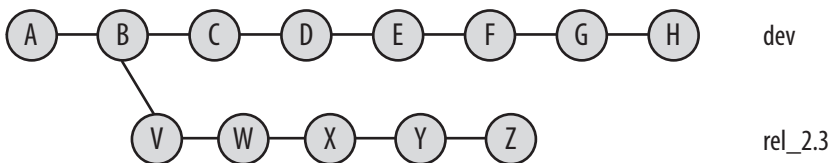


Рис. 9.12. До `git cherry-pick` одного коммита

В стандартном процессе разработки баг исправляется в линии разработки коммитом F. Если этот баг также окажется в релизе 2.3, его исправление F можно будет внести в `rel_2.3` с помощью `git cherry-pick`:

```
$ git checkout rel_2.3
$ git cherry-pick dev~2      # коммит F, выше
```

После выполнения `cherry-pick` граф примет форму, показанную на рис. 9.13.

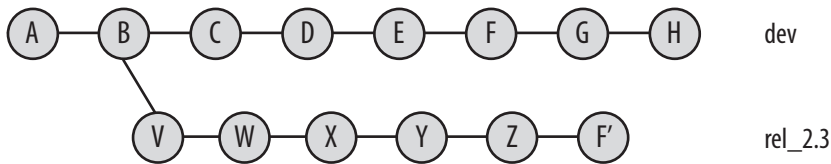


Рис. 9.13. После `git cherry-pick` одного коммита

На рис. 9.13 коммит `F'` очень похож на коммит `F`, но это новый коммит, он потребует исправления, возможно, разрешения конфликтов, поскольку применяется к коммиту `Z`, а не `E`. Ни один из коммитов, следующих за `F`, не применяется после `F'`.

Еще одно типичное использование `cherry-pick` — воссоздание серии коммитов путем их выборочного отбора из одной ветки и внесения в новую.

Предположим, в ветке разработки `my_dev` есть серия коммитов (рис. 9.14) и вы хотите внести их в ветку `main`, но в ином порядке.

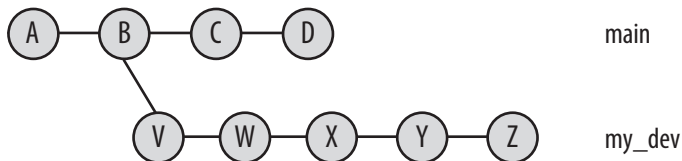


Рис. 9.14. До перемешивания с помощью `git cherry-pick`

Чтобы распределить их по ветке `main` в последовательности `Y, W, X, Z`, можно использовать следующие команды:

```

$ git checkout main
$ git cherry-pick my_dev~1 # Y
$ git cherry-pick my_dev~3 # W
$ git cherry-pick my_dev~2 # X
$ git cherry-pick my_dev   # Z

```

После этого история коммитов будет соответствовать показанной на рис. 9.15.

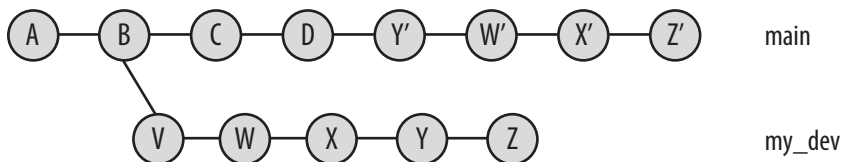


Рис. 9.15. После перемешивания с помощью `git cherry-pick`

В ситуациях, когда порядок коммитов подвергается сильным изменениям, вам наверняка придется разрешать конфликты. Это будет полностью зависеть от имеющихся между ними связей. Если коммиты сильно связаны и изменяют пересекающиеся строки, то возникнут конфликты, требующие разрешения. Если коммиты достаточно независимы, то вы сможете без проблем их перемещать.

Изначально команда `git cherry-pick` выбирала и заново применяла по одному коммиту за раз. Но в последующих версиях Git появилась возможность указывать диапазон коммитов. Например, следующая команда:

```
# в ветке main
$ git cherry-pick X..Z
```

применит новые коммиты X', Y' и Z' в ветке main. Это особенно удобно при перемещении или внесении большой последовательности коммитов из одной линии разработки в другую, поскольку не нужно использовать всю исходную ветку.

При применении этой команды можно ссылаться на несколько конкретных коммитов:

```
# В ветке rel_2.3
$ git cherry-pick dev~4 dev~2
```

Эта команда применит пятый D (dev~4) и третий F (dev~2) коммиты из ветки dev в ветке rel_2.3.

reset, revert и checkout

Команды Git `reset`, `revert` и `checkout` могут сбивать с толку, поскольку выполняют схожие операции. Еще одна причина путаницы — в других системах контроля версий ключевые слова `reset`, `revert` и `checkout` имеют другие значения.

Тем не менее есть хорошие руководства и правила того, когда использовать каждую из этих команд.

Если вы хотите переключиться на другую ветку, используйте `git checkout` или `git switch`. Ссылка HEAD вашей текущей ветки переместится на вершину указанной ветки.

Команда `git reset` не изменяет ветку. Но если вы передадите имя ветки, она изменит состояние рабочего каталога, чтобы он *выглядел* как вершина названной ветки. Иными словами, `git reset` предназначена для сброса ссылки HEAD текущей ветки.

Поскольку опция `git reset --hard` служит для отката к известному состоянию, она также может очищать неудачные или устаревшие попытки слияния. А вот `git checkout` этого не делает. Поэтому если у вас был незаконченный коммит слияния и вы попытались восстановиться с помощью `git checkout`, а не `git reset --hard`, то коммитом слияния ошибочно станет следующий коммит.

Путаница с командой `git checkout` связана с ее дополнительной способностью извлекать файл из объектного хранилища и помещать его в рабочий каталог, заменяя находящуюся там версию в зависимости от того, как она используется. Иногда версия этого файла соответствует версии текущей `HEAD`, а иногда более ранней:

```
# Переключение на file.c из индекса
$ git checkout -- path/to/file.c

# Переключение на file.c из rev v2.3
$ git checkout v2.3 -- path/to/file.c
```

В Git это называется переключением пути.

В первом случае получение текущей версии из объектного хранилища оказывается формой операции `reset`: правки файла в локальном рабочем каталоге отменяются, поскольку этот файл сбрасывается на его текущую версию `HEAD`. Это пример неудачной логики Git.

Во втором же случае более ранняя версия файла извлекается из объектного хранилища и помещается в рабочий каталог. Выглядит это как операция `revert` для файла. И это тоже пример неудачной логики Git.

В обоих случаях неправильно представлять эти операции как `reset` или `revert`. И там и там происходит переключение на файл из конкретного коммита: `HEAD` и `v2.3` соответственно.

Команда `git revert` работает для коммитов целиком, но не для отдельных файлов.

Если другой разработчик клонировал ваш репозиторий или запросил некоторые из ваших коммитов, то изменение истории коммитов будет иметь последствия. В этом случае лучше не использовать команды, изменяющие историю репозитория. Вместо этого применяйте `git revert` вместо `git reset` или `git commit --amend`, описанных в предыдущих разделах.

Перебазирование коммитов

Команда `git rebase` меняет последовательность коммитов. Для ее выполнения требуется как минимум имя другой ветки, на которую будут перемещены ваши коммиты. По умолчанию переносятся коммиты из текущей ветки, которые еще не находятся в другой.

Обычно `git rebase` используется для поддержания актуальности находящихся в разработке коммитов относительно другой ветки, как правило, ветки `main` или ветки отслеживания из другого репозитория.

На рис. 9.16 показаны две ветки. Изначально ветка `topic` начиналась от ветки `main`, когда та находилась на стадии коммита В. Впоследствии `main` параллельно с `topic` продвинулась до коммита Е.

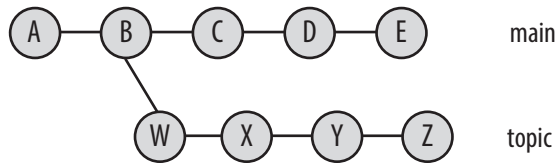


Рис. 9.16. До git rebase

Вы можете поддерживать серию коммитов в актуальном состоянии относительно ветки `main`, написав коммиты так, чтобы они основывались на коммите `E`, а не `B`. Поскольку ветка `topic` должна быть текущей, для этого можно выполнить команды:

```
$ git checkout topic
$ git rebase main
```

либо:

```
$ git rebase main topic
```

Еще одна операция перебазирувания (rebasing) завершена. Теперь граф коммитов выглядит так, как показано на рис. 9.17.

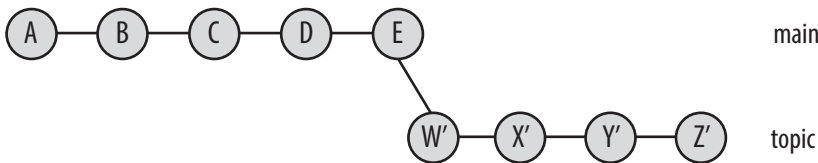


Рис. 9.17. После git rebase

Использование команды `git rebase` в ситуациях, показанных на рис. 9.16, часто называется *прямым переносом* (forward porting). В этом примере ветка `topic` была напрямую перенесена в ветку `main`.

Перебазирование может быть как прямым, так и обратным. Оба варианта реализуются с помощью `git rebase`. Интерпретация обычно зависит от понимания того, какая функциональность считается опережающей или отстающей от другой.

В контексте репозитория, который вы клонировали откуда-то еще, обычно выполняется прямой перенос ветки или веток разработки в ветку отслеживания `origin/main` с помощью операции `git rebase`. В случае же работы с удаленными репозиториями вы увидите, что эту операцию часто запрашивают мейнтейнеры репозитория, произнося что-то вроде «перебазируйте свой патч на вершину `main`». Об удаленных репозиториях поговорим в главе 11.

Команду `git rebase` также можно использовать для полного переноса линии разработки из одной ветки в другую, дополнив ее опцией `--onto`.

Например, вы разработали новую функциональность в ветке `feature`, содержащей коммиты `P` и `Q`, которые базировались в ветке `maint` (рис. 9.18). Чтобы перенести `P` и `Q` ветки `feature` от ветки `maint` к ветке `main`, нужно выполнить:

```
$ git rebase --onto main maint^ feature
```

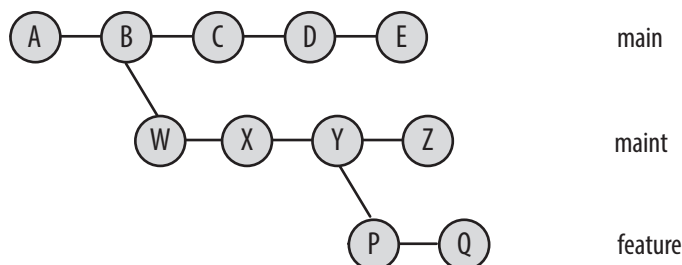


Рис. 9.18. До перебазирувания с помощью `git rebase`

Граф коммитов будет выглядеть так, как показано на рис. 9.19.

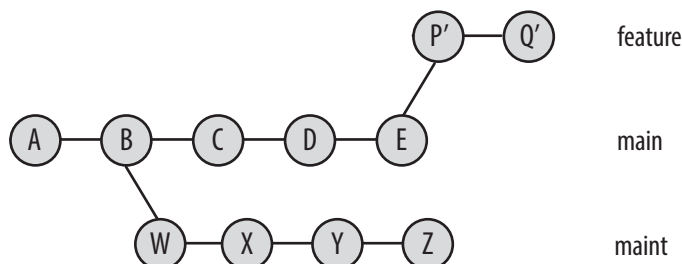


Рис. 9.19. После перебазирувания с помощью `git rebase`

Операция перебазирувания поочередно перемещает коммиты из их текущего положения на новую базу, или основание. В результате в каждом перемещенном коммите могут возникнуть конфликты, требующие разрешения.

В случае обнаружения конфликта операция `rebase` временно приостанавливает обработку, чтобы вы могли его разрешить. Любой возникший во время перебазирувания конфликт нужно обрабатывать так, как описано в разделе «Когда слияние ведет к конфликту» на с. 147 в главе 6.

Когда все конфликты разрешены и индекс обновлен, операцию перебазирувания можно возобновить командой `git rebase --continue`. Команда зафиксирует разрешенный конфликт и перейдет к следующему коммиту в серии.

Если в процессе изучения конфликта перебазирувания вы решите, что он необязательный, то можете выполнить `git rebase --skip`, чтобы команда пропустила этот коммит и перешла к следующему. Но такое решение не всегда будет правильным,

особенно если последующие коммиты зависят от изменений, вносимых текущим. В таком случае проблемы наверняка умножатся, и поэтому конфликт лучше все же разрешить.

Наконец, если операция `rebase` окажется ошибкой, то `git rebase --abort` отменит ее и восстановит репозиторий в состояние, предшествующее выполнению `git rebase`.

Использование `git rebase -i`

Предположим, вы начинаете писать хайку (традиционный японский стих). Составив две полные строчки, вы фиксируете их:

```
$ git init -b main
Initialized empty Git repository in .git/

$ cat haiku
Talk about colour
No jealous behaviour here

$ git add haiku
$ git commit -m "Start my haiku"
[main (root-commit) 1cc856c] Start my haiku
 1 file changed, 2 insertions(+)
 create mode 100644 haiku
```

Вы продолжаете сочинять, но решаете, что лучше использовать американский вариант написания *color* (цвет) вместо британского *colour*, и делаете коммит с правкой:

```
$ git diff
diff --git a/haiku b/haiku
index 088bea0..958aff0 100644
--- a/haiku
+++ b/haiku
@@ -1,2 +1,2 @@
-Talk about colour
+Talk about color
 No jealous behaviour here

$ git commit -a -m "Use color instead of colour"
[main b2c5047] Use color instead of colour
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Наконец, вы составляете последнюю строчку, которую также фиксируете:

```
$ git diff
diff --git a/haiku b/haiku
index 958aff0..cdeddf9 100644
--- a/haiku
+++ b/haiku
```

```
@@ -1,2 +1,3 @@
Talk about color
No jealous behaviour here
+I favour red wine

$ git commit -a -m "Finish my colour haiku"
[main e0d8d39] Finish my colour haiku
1 file changed, 1 insertion(+)
```

Но тут вы снова сталкиваетесь с несоответствием написания и решаете все английские «ou» заменить на американские «o»:

```
$ git diff
diff --git a/haiku b/haiku
index cdeddf9..064c1b5 100644
--- a/haiku
+++ b/haiku
@@ -1,3 +1,3 @@
Talk about color
-No jealous behaviour here
-I favour red wine
+No jealous behavior here
+I favor red wine

$ git commit -a -m "Use American spellings"
[main 0aff702] Use American spellings
1 file changed, 2 insertions(+), 2 deletions(-)
```

К этому моменту собралась история коммитов, которая выглядит так¹:

```
$ git show-branch --more=4
[main] Use American spellings
[main^] Finish my colour haiku
[main~2] Use color instead of colour
[main~3] Start my haiku
```

Оценив последовательность коммитов или получив обратную связь от рецензентов, вы решаете, что лучше сначала закончить хайку, а потом уже вносить изменения. Вы считаете, что история коммитов должна выглядеть так:

```
[main] Use American spellings
[main^] Use color instead of colour
[main~2] Finish my colour haiku
[main~3] Start my haiku
```

¹ Use American spellings — использовать американское написание.

Finish my colour haiku — закончить мое хайку о цвете.

Use color instead of colour — использовать color вместо colour.

Start my haiku — начать мое хайку. — *Примеч. пер.*

Но затем вы замечаете, что нет смысла иметь два похожих коммита, которые исправляют написание разных слов. Тогда вы сжимаете (squash) ветки `main` и `main^` в один коммит:

```
[main] Use American spellings
[main^] Finish my colour haiku
[main~2] Start my haiku
```

Переупорядочивание, редактирование, перемещение, сжатие нескольких коммитов в один и разделение одного коммита на несколько — все это легко выполняется командой `git rebase`, дополненной опцией `-i` или `--interactive`. Такая команда позволит изменить коммиты ветки и поместить их обратно в ту же ветку или в другую.

Типичный вариант использования, подходящий для этого примера, — изменение одной и той же ветки на месте. В этом случае нужно изменить три набора правок между четырьмя коммитами.

Для этого сообщаем `git rebase -i` имя коммита, который нужно скорректировать:

```
$ git rebase -i main~3
```

Команда переносит вас в редактор, где открывается файл с таким содержимым:

```
pick b2c5047 Use color instead of colour
pick e0d8d39 Finish my colour haiku
pick 0aff702 Use American spellings

# Rebase 1cc856c..0aff702 onto 1cc856c (3 commands)
#
# Команды:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .       create a merge commit using the original merge commit's
#         message (or the oneline, if no original merge commit was
#         specified); use -c <commit> to reword the commit message
#
# Порядок этих строк можно изменить; они выполняются сверху вниз.
#
# Если удалить строку здесь, ЭТОТ КОММИТ БУДЕТ УТРАЧЕН.
#
# Однако если удалить все, то перебазирование будет отменено.
```

В первых трех строках перечислены коммиты в редактируемом диапазоне, который вы указали в командной строке. Изначально они находятся в порядке от первого к последнему, каждый содержит глагол `pick`. Если выйти из редактора, то каждый коммит будет выбран (по порядку), применен к целевой ветке и зафиксирован. Символ `#` в начале строки напоминает, что это комментарий, и программа его проигнорирует.

Однако в этот момент вы можете спокойно изменить порядок коммитов, совместить их, изменить отдельный коммит или полностью его удалить. Можно также добавить коммит, если вам известен его SHA ID. Для этого просто измените в редакторе порядок коммитов, как показано ниже, и закройте его:

```
pick e0d8d39 Finish my colour haiku
pick b2c5047 Use color instead of colour
pick 0aff702 Use American spellings
```

Напомню, что первый коммит для перебаазирования — это «Start my haiku». Следующим станет «Finish my colour haiku», за ним последуют «Use color...» и «Use American...»:

```
$ git rebase -i main~3
```

```
# Изменение порядка первых двух коммитов и выход из редактора
```

```
Successfully rebased and updated refs/heads/main.
```

```
$ git show-branch --more=4
[main] Use American spellings
[main^] Use color instead of colour
[main~2] Finish my colour haiku
[main~3] Start my haiku
```

Здесь история коммитов была переписана: мы объединили два коммита, исправляющие написание слов, и два коммита, добавляющие текст.

Следующий шаг — совмещение двух коммитов с исправлением написания слов в один коммит. Выполните команду `git rebase -i main~3`, которая перебазирует список из такого:

```
pick e0d8d39 Finish my colour haiku
pick b2c5047 Use color instead of colour
pick 0aff702 Use American spellings
```

в такой:

```
pick da10180 Finish my colour haiku
pick 3dd2279 Use color instead of colour
squash d83e71c Use American spellings
```

Третий коммит будет совмещен с предыдущим, и сформируется новый шаблон сообщения журнала на основе комбинации этих коммитов.

В этом примере сообщения двух коммитов объединяются и показываются в редакторе:

```
# Это комбинация 2 коммитов.
# Это первое сообщение коммита:
```

```
Use color instead of colour
```

```
# Это второе сообщение коммита:
```

```
Use American spellings
```

Эти сообщения можно сократить до одного:

```
Use American spellings
```

Все строки с # игнорируются.

Теперь можно увидеть результаты последовательного выполнения перебаазирования:

```
$ git rebase -i main~3
```

```
# Совмещаем коммиты и переписываем их сообщения
```

```
[detached HEAD 0692bda] Use American spellings
Date: Sun Oct 3 14:36:00 2021 +0200
1 file changed, 3 insertions(+), 3 deletions(-)
Successfully rebased and updated refs/heads/main.
```

```
$ git show-branch --more=4
```

```
[main] Use American spellings
[main^] Finish my colour haiku
[main~2] Start my haiku
```

И хотя этапы переупорядочивания и сжатия здесь были выполнены двумя отдельными вызовами `git rebase -i main~3`, эти два шага можно было сделать как один. За один шаг можно также сжать несколько последовательных коммитов.

Сравнение rebase и merge

Помимо простого изменения истории, операция перебаазирования имеет и другие последствия.

Перебаазирование последовательности коммитов на вершину ветки схоже со слиянием этих двух веток. В обоих случаях новая вершина ветки будет содержать изменения от обеих представленных ей веток.

Возникает вопрос: «Как сделать лучше — совмещать или перебаазировать последовательность коммитов?» Он станет особенно актуальным в главе 11: там мы

рассмотрим ситуацию, когда в разработке задействовано много программистов, репозитория и веток.

Процесс перебазируемости последовательности коммитов заставляет Git генерировать полностью новую их последовательность. Они получают новые ID SHA-1, основываясь на новом изначальном состоянии и представляют другие различия, несмотря на то что несут в себе изменения, ведущие к тому же конечному состоянию.

В ситуации, показанной на рис. 9.16, перебазирование коммитов в формат рис. 9.17 не вызовет проблем, поскольку на перебазируемую ветку не опирается никакой другой коммит. Но даже в собственном репозитории у вас могут быть дополнительные ветки, основанные на той, которую вы хотите перебазировать. Рассмотрим граф, показанный на рис. 9.20.

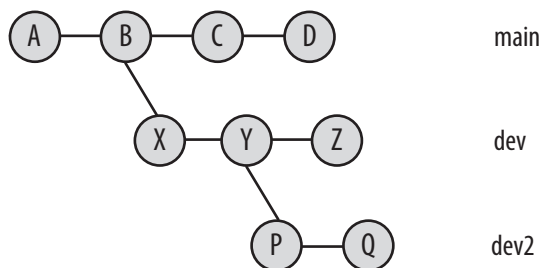


Рис. 9.20. До git rebase

Вы можете решить, что выполнение команды

```
# Перемещение на вершину main ветки dev
$ git rebase main dev
```

создаст граф, как на рис. 9.21. Но это не так.

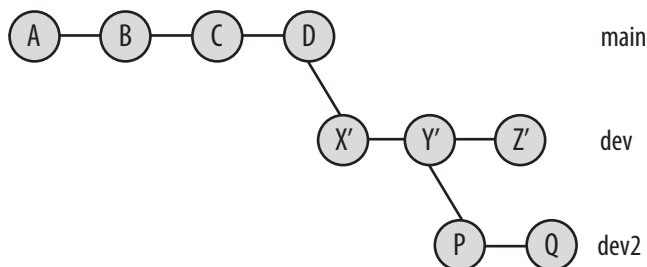


Рис. 9.21. Предполагаемый результат git rebase

На самом деле вы получите результат, представленный на рис. 9.22. Дело в том, что во время перебаазирования Git сохраняет все изменения, внесенные коммитами в текущую ветку, во временном расположении, а затем применяет их к целевой ветке один за другим, последовательно:

```
$ git rebase main dev
```

```
Successfully rebased and updated refs/heads/dev.
```

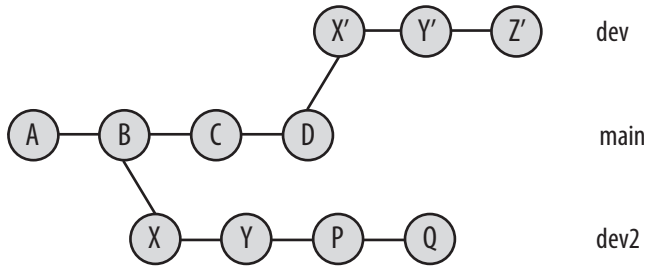


Рис. 9.22. Фактический результат git rebase

Коммиты X', Y' и Z' представляют новые версии старых коммитов, опирающихся на B. Старые коммиты X и Y по-прежнему есть в графе, потому что остаются достижимыми из ветки dev2. А вот исходный коммит Z был удален, потому что оказался недостижим. Имя ветки, которая *ранее* указывала на него, теперь указывает на новую версию этого коммита.

Итоговая история веток выглядит так, будто в ней также есть дублирующие сообщения коммитов:

```
$ git show-branch
```

```

* [dev] Z
! [dev2] Q
! [main] D
---
* [dev] Z
* [dev^] Y
* [dev~2] X
* + [main] D
* + [main^] C
+ [dev2] Q
+ [dev2^] P
+ [dev2~2] Y
+ [dev2~3] X
*++ [main~2] B

```

Но важно помнить, что это другие коммиты, которые, по сути, вносят одно и то же изменение. Если вы вольете ветку с одним из новых коммитов в новую,

содержащую один из старых коммитов, то Git не поймет, что вы применяете одни и те же изменения дважды. В результате записи в `git log` задублируются, наверняка возникнут конфликт слияния и путаница. Такую ситуацию нужно исправлять.

Если итоговый граф выглядит так, как нужно, то дело сделано. Скорее всего, вас интересует перемещение всей ветки (включая подветки). Чтобы получить такой граф, понадобится перебазировать ветку `dev2` на новый коммит `Y` в ветке `dev`:

```
$ git rebase dev^ dev2
Successfully rebased and updated refs/heads/dev2.
```

```
$ git show-branch
! [dev] Z
* [dev2] Q
! [main] D
---
* [dev2] Q
* [dev2^] P
+ [dev] Z
+* [dev2~2] Y
+* [dev2~3] X
+*+ [main] D
```

Тогда получится граф, показанный на рис. 9.21.

Еще одна ситуация, которая может запутать, возникает при перебазировании ветки, с которой выполнялось слияние. Допустим, у вас была структура веток, как на рис. 9.23.

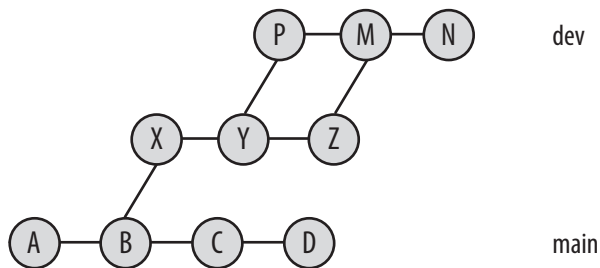


Рис. 9.23. До слияния с помощью `git rebase`

Если нужно переместить всю структуру ветки `dev`, начиная с коммита `N` и до коммита `X`, с `B` на `D`, как показано на рис. 9.24, то можете попробовать использовать команду `git rebase main dev`.

Однако результат этой команды будет неожиданным:

```
$ git rebase main dev
Successfully rebased and updated refs/heads/dev.
```

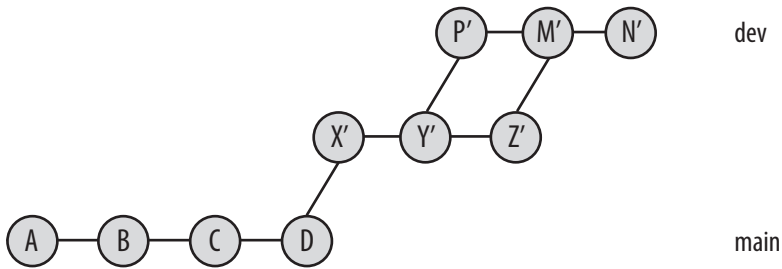


Рис. 9.24. Предполагаемый результат слияния с помощью git rebase

Выглядит так, будто все сделано правильно. Git сообщает, что применила все (невливаемые) изменения коммитов. Но действительно ли система все сделала правильно?

```
$ git show-branch
* [dev] N
! [main] D
--
* [dev] N
* [dev^] P
* [dev~2] Z
* [dev~3] Y
* [dev~4] X
*+ [main] D
```

Все эти коммиты теперь находятся в одной длинной строке. Что произошло?

Git нужно переместить часть графа, достижимую из `dev`, назад до основы слияния у `B`, поэтому система нашла коммиты в диапазоне `main..dev`. Для этого Git выполняет топологическую сортировку для соответствующей части графа, создавая линейаризованную последовательность всех коммитов в этом диапазоне. Как только последовательность была определена, Git поочередно применяет коммиты, начиная с целевого коммита, `D`. Таким образом, мы говорим, что «перебазирование линейаризовало историю исходной ветки (со слияниями) в основной ветке», как показано на рис. 9.25.

Если вы именно это и хотели либо если вас не волнует, что форма графа изменилась, то все готово. Но если вам нужно явно сохранить структуру ветвления и слияний во всей перебазированной ветке, тогда используйте опцию `--rebase-merges`. Поскольку при выполнении с этой опцией команда стремится сохранить коммиты слияния, то в случае присутствия любых неразрешенных конфликтов слияния может потребоваться разрешить их или повторно применить изменения вручную:

```
$ git rebase --rebase-merges main dev
Successfully rebased and updated refs/heads/dev.
```

С помощью команды `git log` можно увидеть, что итоговая форма графа сохраняет исходную структуру слияний:

```
$ git log --oneline --all
* 061f9fd... N
* f669404... Merge branch 'dev2' into dev
|\
| * c386cfc... Z
* | 38ab25e... P
|/
* b93ad42... Y
* 65be7f1... X
* e3b9e22... D
* f2b96c4... C
* 8619681... B
* d6fba18... A
```



В предыдущих версиях Git использовалась опция `--preserve-merges`. Позднее она была упразднена в пользу `--rebase-merges`, которая работает с интерактивными слияниями.

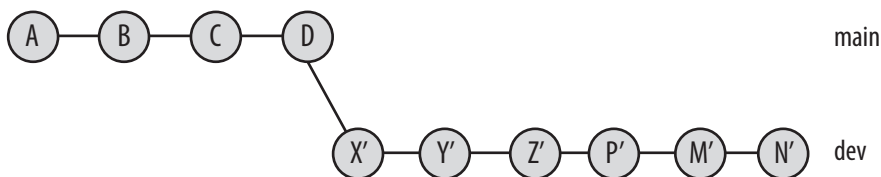


Рис. 9.25. Слияние с помощью `git rebase` после линейаризации

Получится результат, показанный на рис. 9.24.

Некоторые из принципов, на основе которых можно ответить на вопрос «Использовать перебазирующие или слияние?», применимы как к одному локальному репозиторию, так и к их распределенной системе. В главе 12 я расскажу о дополнительных последствиях для разработчиков, которые используют другие репозитории.

В зависимости от вашего стиля разработки и конечного замысла линейаризация истории развития исходной ветки при ее перебазировании может быть приемлемой или нет. Если вы уже публиковали или передавали коммиты в ветке, которую хотите перебазировать, то подумайте о негативных последствиях для других.

Если операция перебазирования неприемлема, но вам все равно нужно получить изменения ветки, тогда на выручку может прийти слияние.

Помните о следующих принципах.

- Перебазирование переписывает коммиты как новые.
- Старые коммиты, которые оказываются недостижимыми, исчезают.
- Любой пользователь одного из старых коммитов может оказаться в затруднительном положении.
- Если у вас есть ветка, использующая коммит до его перебаазирования, то может потребоваться перебаазировать и ее.
- Если в другом репозитории есть пользователь коммита до перебаазирования, то у него по-прежнему останется его копия, несмотря на то что в вашем репозитории он был перемещен. Теперь этому пользователю нужно будет исправить и свою историю коммитов.

Итоги

Вы можете свободно изменять коммиты или их последовательность, если это сделает историю более чистой и понятной.

В этой главе мы познакомились с техниками реализации нужных изменений в проектах и узнали об их последствиях, в частности, когда речь идет об истории репозитория.

В контексте разработки ПО всегда есть компромисс между чрезмерной повторяющейся доработкой и принятием удовлетворительного результата. Ключевой момент — это стремление к чистой и правильно структурированной истории коммитов, которая будет понятна вам и вашим коллегам.

Локальное хранилище и журнал ссылок

Бывает ли у вас ощущение перегруженности, когда коллеги или менеджеры постоянно дергают вас с требованиями исправить баги или просят о чем-то еще, что мешает вам заниматься *реальной* работой? Если да, то локальное хранилище вам в помощь!

В этой главе вы узнаете, как оно работает и как просматривать размещенный в нем контекст. Затем мы рассмотрим журналы ссылок, в которых записываются локальные коммиты репозитория Git. Эти инструменты позволят вам на время откладывать текущую работу, когда нужно отвлечься на другие задачи.

Локальное хранилище

Локальное хранилище, или механизм `stash`, — это способ фиксирования текущей работы. Он позволяет сохранить ее и вернуться позднее в более подходящее время.

Вы уже можете это делать с помощью механизмов существующей ветки и коммитов, но локальное хранилище позволяет быстро фиксировать весь индекс и рабочий каталог одной простой командой, оставляя репозиторий чистым и готовым к ведению альтернативного направления разработки. Другая команда восстановит индекс и рабочий каталог, что позволит вам вернуться к работе с того места, на котором вы остановились.

Рассмотрим принцип работы этого локального хранилища на примерах.

Пример: прерывание рабочего процесса

В этом сценарии вы работаете в репозитории и уже изменили несколько файлов, возможно, разместив некоторые из них в индексе. Но тут вас прерывают. Например, просят что-то срочно исправить или сосредоточиться на разработке новой функциональности.

Какими бы ни были причины, вы понимаете, что нужно сохранить текущую работу, очистить рабочее дерево и начать заново.

И здесь появляется отличная возможность для использования команды `git stash`:

```
$ cd the-git-project
# внес массу изменений, нахожусь в процессе

# Прерывание рабочего процесса на работу с высоким приоритетом
# Нужно все бросить и заняться чем-то другим

$ git stash push
Saved working directory and index state WIP on main: 1En0v80 ...

# работа с высокоприоритетным изменением
# Внесение и фиксация изменений

# Продолжение отложенной работы
$ git stash pop
```

Затем можно вернуться к тому, на чем вы остановились.

Поскольку `push` — это предустановленная опция для `git stash`, при использовании этой команды вы можете ее опустить. При сохранении в локальное хранилище Git также предоставляет заготовленное сообщение журнала, но вы можете создать и свое, которое будет лучше отражать, какую именно работу вы сохранили. Например:

```
$ git stash -m "WIP: Doing real work on my stuff"1
```

Акроним *WIP* — это общепринятое сокращение для «work in progress» (незавершенная работа).

Чтобы достигнуть того же с помощью других, более простых команд Git, нужно вручную создать новую ветку, в которую вы будете фиксировать все свои изменения, затем вернуть прежнюю ветку для продолжения работы и восстановить сохраненное состояние отложенной ветки поверх нового рабочего каталога.

Это делается примерно в такой последовательности:

```
# ... процесс разработки прерван ...

# Создание новой ветки для сохранения текущего состояния.
$ git checkout -b saved_state

# Сохранение текущей работы.
$ git add files
$ git commit -m "Saved state"
```

¹ Незавершенная работа: занимаюсь реальным делом, своей работой. — *Примеч. перев.*

```
# Возвращение к предыдущей ветке для непосредственного обновления.
$ git checkout main

# Внесение срочного исправления.
$ git add fixed-files
$ git commit -m "Fix something."

# Восстановление сохраненного состояния поверх рабочего каталога.
$ git checkout saved_state
$ git reset --mixed HEAD^

# ... Возвращение к работе с момента, на котором она была прервана.
```

В описанном процессе важны завершенность и внимание к деталям. При сохранении текущего состояния нужно отразить все изменения и не забыть переместить HEAD обратно, чтобы исключить возможное прерывание процесса восстановления.

Команда `git stash push` сохранит ваш текущий индекс (когда вы передадите флаг `--staged`) и состояние рабочего каталога, после чего очистит их так, чтобы они соответствовали вершине текущей ветки. И хотя в результате этой операции покажется, что измененные и любые обновленные файлы (что сделано, например, с помощью `git add` или `git rm`) были потеряны, это не так. Содержимое индекса и рабочего каталога фактически сохранено в виде независимых, регулярных коммитов и доступно через ссылку `.git/refs/stash`.

Чтобы проверить это утверждение, выполните такую команду:

```
$ git show-branch stash
[stash] WIP on main: 3889def Some initial files.
```

Как вы могли догадаться из использования команды `pop` для восстановления состояния, две базовые команды `stash`, `git stash push` и `git stash pop`, реализуют стек состояний локального хранилища. Это позволяет повторно прерывать уже прерванный рабочий процесс. Каждым помещенным в этот стек контекстом можно управлять независимо от регулярного процесса коммитов.

Команда `git stash pop` восстанавливает контекст, сохраненный предыдущей операцией `push`, поверх текущего рабочего каталога и индекса (чтобы хранилище выдало ранее сохраненный контекст индекса, нужно внести индексированные изменения с помощью опции `git stash push --staged`). И под восстановлением мы подразумеваем, что операция `pop` берет содержимое хранилища и *вливает* эти изменения в текущее состояние, а не просто переписывает или заменяет файлы. Здорово, да? Помните, что для успешного завершения этой операции ваш рабочий каталог должен соответствовать индексу, о чем говорится в руководстве по `git stash`.

Использование `git stash pop` может неполностью восстановить состояние работы, которое у вас было на момент сохранения. Поскольку сохраненный контекст

может оказаться применен поверх другого коммита, иногда потребуется слияние вместе с возможным разрешением возникших конфликтов вручную.

После успешной операции `pop` Git автоматически удалит сохраненное состояние из стека сохраненных состояний. Но при необходимости разрешения конфликтов Git не будет автоматически сбрасывать состояние просто на случай, если вы захотите попробовать другой подход или восстановить его на другом коммите. Как только вы разберетесь с конфликтами слияния и захотите продолжить, используйте команду `git stash drop`, чтобы вручную удалить состояние хранилища из стека. В противном случае Git будет продолжать вести разрастающийся¹ стек контекстов:

```
$ git stash drop
```

```
Dropped refs/stash@{0} (7ec158448f301a4fb4f7cb567225ddb1af135bde)
```

В предыдущем примере, если бы мы выполнили `git stash drop` без явной ссылки на запись хранилища `stash@{0}`, система удалила бы последнюю запись.

Если вы хотите просто воссоздать контекст, который сохранили в стеке хранилища, без его удаления, используйте `git stash apply`. Таким образом, команда `git stash pop` представляет собой успешную команду `git stash apply`, сопровождаемую `git stash drop`.



Вы можете использовать `git stash apply` для применения одного сохраненного в хранилище контекста поверх нескольких коммитов до его удаления из стека.

Следует внимательно оценивать, какую именно команду использовать для восстановления содержимого из локального хранилища — `git stash apply` или `git stash pop`. Понадобится ли оно вам в будущем? Если нет, применяйте `pop`, а затем удалите сохраненное содержимое и ссылку из объектного хранилища.

Этот принцип показан на рис. 10.1. Здесь репозиторий, в котором мы работаем, уже содержит стек хранилища. Команда `git stash push` помещает `file5` и `file6` на вершину стека (`stash@{0}`). Команда `git stash apply` применяет сохраненную в хранилище форму `file4` обратно к каталогу индексирования, но саму запись хранилища (`stash@{1}`) оставляет нетронутой. Команда `git stash pop` восстанавливает сохраненные `file1`, `file2` и `file3` соответственно в индексе и рабочем каталоге, удаляя из стека эту запись (`stash@{0}`), на картинке обозначенную пунктирным блоком (`.git/refs/stash`).

¹ Технически он растет неограниченно. Локальное хранилище очищается в соответствии с истечением срока действия ссылок, а также подвергается сборке мусора.

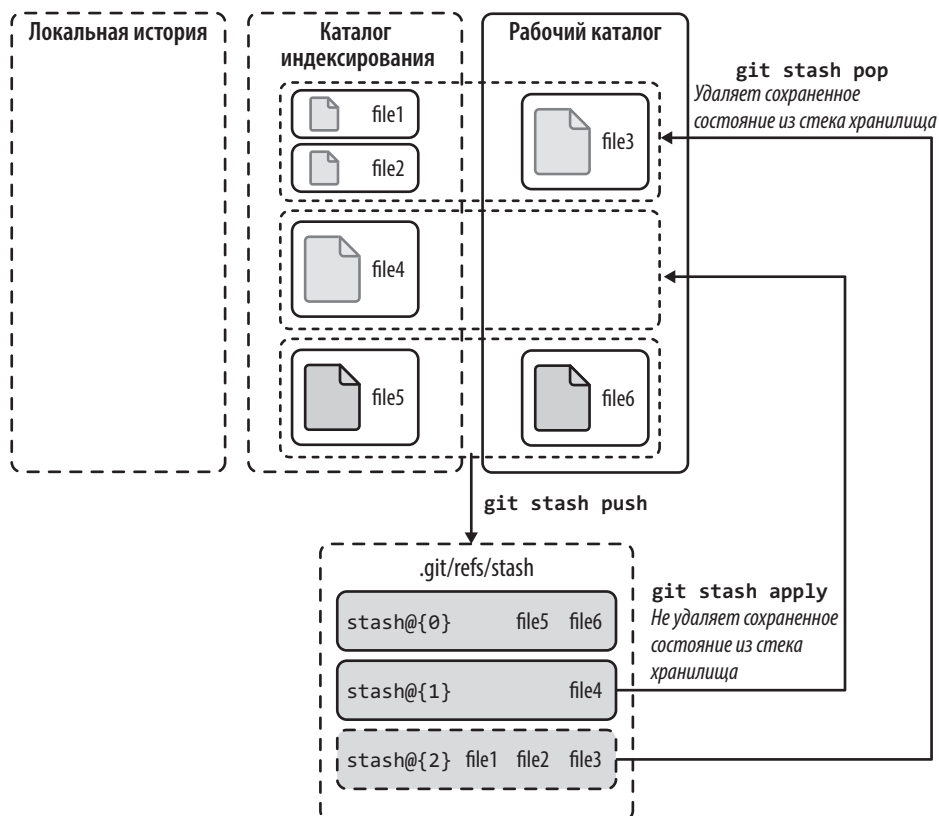


Рис. 10.1. Принципы локального хранилища

Просмотр сохраненного в локальном хранилище контекста

Команда `git stash list` перечисляет стек сохраненных контекстов от последнего сохраненного до первого.

В примере ниже мы будем работать с репозиторием Git, содержащим два файла, которые изменим и поместим в хранилище. Затем добавим в него новый файл, чтобы показать, как сохранять в хранилище новые, неотслеживаемые файлы, которые могут быть в рабочем каталоге:

```
$ mkdir my-repo
$ cd my-repo
$ git init -b main
$ cd my-repo
Initialized empty Git repository in /tmp/my-repo/.git/

$ echo "hello" >> file1
$ git add file1
```

```
$ git commit -m "Add file 1"
[main (root-commit) 9841192] add file 1
 1 file changed, 1 insertion(+)
 create mode 100644 file1

$ echo "Git" >> file2
$ git add file2
$ git commit -m "Add file 2"
[main 24d4b0a] add file 2
 1 file changed, 1 insertion(+)
 create mode 100644 file2

$ ls
file1 file2

$ echo "some foo" >> file1

$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore -- <file>..." to discard changes in working directory)

       modified:   file1

no changes added to commit (use "git add" and/or "git commit -a")

$ git stash -m "Tinkered file1"
Saved working directory and index state On main: Tinkered file1

$ git commit --dry-run
On branch main
nothing to commit, working directory clean

$ echo "some bar" >> file2

$ git stash -m "Messed with file2"
Saved working directory and index state On main: Messed with file2

$ git stash list
stash@{0}: On main: Messed with file2
stash@{1}: On main: Tinkered file1

$ echo "new" >> file3

# stash не выполнится для любого нового неотслеживаемого файла
$ git stash -m "Add new untracked file3"
No local changes to save

# Добавляем к stash опцию -u, чтобы сохранить новый неотслеживаемый файл в стеке
хранилища
$ git stash -u -m "Add new untracked file3"
Saved working directory and index state On main: Add new untracked file3
```

```
$ git stash list
stash@{0}: On main: Add new untracked file3
stash@{1}: On main: Messed with file2
stash@{2}: On main: Tinkered file1
```

Git всегда нумерует записи хранилища, присваивая последней поступившей ноль. По мере того как они становятся старше, их порядковый номер увеличивается. Поэтому будут разные имена записей хранилища: `stash@{0}` и `stash@{1}`. Об этом пойдет речь в разделе «Журнал ссылок». Команда `git stash show` показывает индекс и изменения файлов, зафиксированные для указанной записи хранилища, относительно ее родительского коммита:

```
$ git stash show stash@{1}
file2 |      1 +
1 file changed, 1 insertion(+)
```

Эта сводка может не содержать интересующую вас информацию. В таком случае добавьте к команде `git stash show` опцию `-p`, которая сделает вывод различий более информативным. По умолчанию команда `git stash show` показывает последнюю запись хранилища, `stash@{0}`.

Когда вы сохраняете изменения в состоянии локального хранилища в Git, эти изменения связаны с конкретным коммитом в вашей истории. Когда вы решаете посмотреть на содержимое локального хранилища, это больше похоже на сравнение текущего состояния (`git diff`) с состоянием конкретного коммита, чем на последовательность коммитов, как в журнале коммитов (`git log`).

Таким образом, все опции для `git diff` также могут быть применены к `git stash show`. Как мы уже видели, по умолчанию используется `-stat`, но можно применять и другие опции. Здесь `-p` используется для получения отличий в патчах указанного состояния хранилища:

```
$ git stash show -p stash@{1}
diff --git a/file2 b/file2
index 5664e30..265404f 100644
--- a/file2
+++ b/file2
@@ -1,2 @@
 git
+some bar
```

Пример: обновление локальной работы изменениями из вышестоящего репозитория

Классический случай применения `git stash` — когда у вас есть незавершенные локальные изменения, но вы хотите получить последние изменения из вышестоящего (upstream) репозитория, чтобы понять, окажутся ли они совместимы с вашими. В руководствах Git этот сценарий называется «получение изменений в грязное дерево» (pull into a dirty tree).

Разворачивается он так: вы ведете разработку в локальном репозитории и делаете несколько коммитов. После этого у вас все еще остаются незафиксированные измененные файлы. Но вы вдруг понимаете, что в вышестоящем репозитории тоже есть изменения, которые нужно внести. Если возникнет конфликт изменений, то простая операция `git pull` не сработает, отказавшись перезаписывать локальные изменения. Быстро решить эту проблему можно с помощью `git stash`:

```
$ git pull
# ... pull не проходит из-за конфликтов слияния ...

$ git stash
Saved working directory and index state WIP on main: e4896bd ...

$ git pull
$ git stash pop
Already up to date.
On branch main
...
...
Dropped refs/stash@{0} (39351f8d3bd89116df89a67119831638d6268180)
```

В этот момент может возникнуть необходимость в разрешении конфликтов, созданных `pop`.

Если у вас в локальной разработке есть новые, незафиксированные (а значит, не отслеживаемые) файлы, то есть вероятность, что команда `git pull`, которая должна внести файл с тем же именем, не сработает, отказавшись перезаписывать вашу версию этого нового файла. В этом случае сопроводите команду `git stash` опцией `--include-untracked`, чтобы она также «спрятала» (`stash`) в локальном хранилище новые, неотслеживаемые файлы вместе с остальными изменениями. Так вы обеспечите для операции `pull` полностью чистый рабочий каталог.

Опция `--all` соберет неотслеживаемые файлы вместе с игнорируемыми из `.gitignore` и *исключенными*.

Наконец, для более сложных операций временного помещения в хранилище, когда нужно выбирать, какие наборы изменений сохранить, используйте опцию `-p`, или `-patch`, как описано в руководстве (это аналогично интерактивной работе с `git add` для выбора сохраняемого набора изменений).

В похожем сценарии `git stash` применяют, когда требуется спрятать в локальное хранилище сделанные изменения для выполнения чистой `pull --rebase`. Обычно это происходит перед самым внесением локальных коммитов в вышестоящий репозиторий:

```
# ... редактирование и коммит ...
# ... дополнительные правки и работа ...

$ git commit --dry-run
# В ветке main
# Ваша ветка опережает 'origin/main' на 2 коммита.
#
```

```
# Изменен, но не обновлен:
# (use "git add <file>..." to update what will be committed)
# (use "git restore -- <file>..." to discard changes in working directory) #
#       modified:   file1.h
#       modified:   file1.c #
no changes added to commit (use "git add" and/or "git commit -a")
```

В этот момент вы можете решить, что сделанные вами коммиты нужно отправить в вышестоящий репозиторий, но нужно также оставить и измененные файлы здесь, в рабочем каталоге.

Однако Git отказывается выполнять pull:

```
$ git pull --rebase
error: cannot pull with rebase: You have unstaged changes.
error: please commit or stash them.
```

Это не совсем надуманный сценарий, как может показаться. Бывает, что в процессе работы с репозиторием нужно изменить *Makefile*¹, например, для отладки или корректировки конфигурации сборки. Мы не хотим фиксировать эти изменения, как и терять их между обновлениями из удаленного репозитория. Нужно, чтобы они какое-то время побыли в рабочем каталоге.

И здесь поможет `git stash`:

```
$ git stash
Saved working directory and index state WIP on main: 5955d14 Some commit log.

$ git pull --rebase
remote: Counting objects: 63, done.
remote: Compressing objects: 100% (43/43), done.
remote: Total 43 (delta 36), reused 0 (delta 0)
Unpacking objects: 100% (43/43), done.
From ssh://git/var/git/my_repo
   871746b..6687d58  main       -> origin/main
Successfully rebased and updated refs/heads/main.
```

После того как вы вытянете (pull) коммиты из вышестоящего репозитория и перебазируете локальные коммиты поверх них, ваш репозиторий окажется в правильном состоянии для отправки вашей работы в вышестоящий репозиторий. При желании вы можете отправить (push) свои изменения сразу же:

```
# Отправка немедленно
$ git push
```

либо после восстановления предыдущего состояния рабочего каталога:

```
$ git stash pop
Auto-merging file1.h
```

¹ Makefile — особый файл, который содержит команды оболочки, используемые утилитой сборки Make в Unix.

On branch main

Your branch is ahead of 'origin/main' by 2 commits.

Changed but not updated:

(use "git add <file>..." to update what will be committed)

(use "git restore -- <file>..." to discard changes in working directory)

modified: file1.h

modified: file1.c

no changes added to commit (use "git add" and/or "git commit -a")

Dropped refs/stash@{0} (7e2546f5808a95a2e6934fcffb5548651badf00d)

\$ git push

Если вы выполните `git push` после извлечения состояния из локального хранилища, то будет отправлена только завершенная, зафиксированная работа. Поэтому о передаче частично завершенной работы можете не беспокоиться. Также не нужно передавать спрятанное (stashed) содержимое — оно хранится только локально.

В общем виде этот принцип представлен на рис. 10.2.

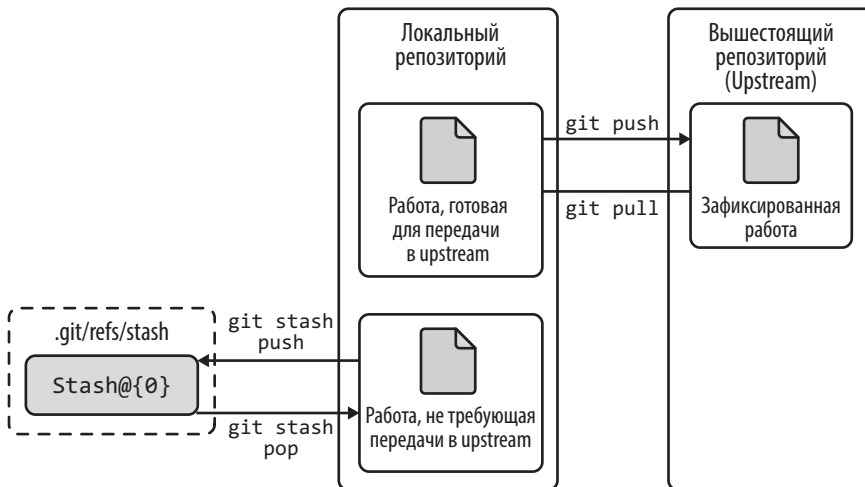


Рис. 10.2. Передача в вышестоящий репозиторий только завершенной, зафиксированной работы

Пример: преобразование изменений из локального хранилища в ветку

Бывает, что после того, как вы спрятали в локальное хранилище свои изменения, разработка в вашей ветке продолжается. Восстановление stash-состояния поверх этих изменений не имеет смысла. Кроме того, этого может повлечь за собой конфликты слияния. Однако может случиться так, что вам необходимо восстановить

сохраненную работу. В этом случае Git предлагает команду `git stash branch`. Она преобразует содержимое хранилища в новую ветку на основе коммита, который был текущим на момент создания выбранной записи хранилища.

Рассмотрим пример репозитория с историей:

```
$ git log --pretty=one --abbrev-commit
d5ef6c9 Some commit.
efe990c Initial commit.
```

Несколько файлов изменяются и индексируются:

```
$ git stash
Saved working directory and index state WIP on main: d5ef6c9 Some commit.
```

Заметьте, что запись хранилища создавалась для коммита `d5ef6c9`.

В результате продолжающейся разработки выполняется еще несколько коммитов, и ветка отклоняется от состояния `d5ef6c9`:

```
$ git log --pretty=one --abbrev-commit
2c2af13 Another mod
1d1e905 Drifting file state.
d5ef6c9 Some commit.
efe990c Initial commit.
```

```
$ git show-branch -a
[main] Another mod
```

И хотя сохраненная работа остается доступной, бесконфликтно применить ее к ветке `main` не получается:

```
$ git stash list
stash@{0}: WIP on main: d5ef6c9 Some commit.
```

```
$ git stash pop
Auto-merging foo
CONFLICT (content): Merge conflict in foo
Auto-merging bar
CONFLICT (content): Merge conflict in bar
The stash entry is kept in case you need it again.
```

По этой причине нужно сбросить определенное состояние и пойти другим путем, создав новую ветку под названием `mod`, которая будет содержать скрытые в хранилище изменения:

```
$ git reset --hard main
HEAD is now at 2c2af13 Another mod
```

```
$ git stash branch mod
Switched to a new branch 'mod'
On branch mod
```

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore -- <file>..." to discard changes in working directory)

modified: bar

modified: foo

no changes added to commit (use "git add" and/or "git commit -a")

Dropped refs/stash@{0} (96e53da61f7e5031ef04d68bf60a34bd4f13bd9f)

Нужно отметить несколько моментов. Во-первых, эта ветка основана на исходном коммите d5ef6c9, а не на текущем HEAD-коммите 2c2af13:

```
$ git show-branch -a
```

```
! [main] Another mod
```

```
* [mod] Some commit.
```

```
--
```

```
+ [main] Another mod
```

```
+ [main^] Drifting file state.
```

```
+* [mod] Some commit.
```

Во-вторых, поскольку хранилище всегда воспроизводится относительно исходного коммита, то этот коммит каждый раз будет завершаться успешно, а значит, удаляться из стека.

Наконец, восстановление состояния хранилища не ведет к автоматическому коммиту каких-либо изменений в новую ветку. Все изменения сохраненных файлов (и при желании индекса) остаются в рабочем каталоге в созданной ветке, на которую произошло переключение:

```
$ git commit --dry-run
```

```
# В ветке mod
```

```
# Изменения не индексированы для коммита:
```

```
# (используйте "git add <file>..." для обновления того, что нужно зафиксировать)
```

```
# (используйте "git checkout -- <file>..." для отмены изменений в рабочем
```

```
# каталоге)
```

```
# изменен: bar
```

```
# изменен: foo
```

```
#
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

В этот момент вы можете зафиксировать изменения в новую ветку, предположительно, в качестве заготовки для дальнейшей разработки или слияния, смотря что вас интересует:

```
$ git commit -a -m "Stuff from the stash"
```

```
[mod 42c104f] Stuff from the stash
```

```
2 files changed, 2 insertions(+), 0 deletions(-)
```

```
$ git show-branch
```

```
! [main] Another mod
```

```
* [mod] Stuff from the stash
--
* [mod] Stuff from the stash
+ [main] Another mod
+ [main^] Drifting file state.
+* [mod^] Some commit.
```

Заметьте, что это неуниверсальное средство для избежания конфликтов слияния. Если при извлечении состояния из хранилища напрямую в ветку `main` возникли конфликты, то попытка влить новую ветку в `main` приведет к тем же конфликтам:

```
$ git checkout main
Switched to branch 'main'

$ git merge mod
Auto-merging foo
CONFLICT (content): Merge conflict in foo
Auto-merging bar
CONFLICT (content): Merge conflict in bar
Automatic merge failed; fix conflicts and then commit the result.
```

Возможность создавать локальные хранилища выглядит заманчиво, но нужно быть внимательным, чтобы не наплодить их слишком много. И не превращайте их в именованные ветки, чтобы они не оставались «висеть» в проекте.

Журнал ссылок (reflog)

Случается, что Git совершает нечто загадочное, заставляя задуматься, что же произошло. Иногда нужно только получить ответ на вопрос: «Стоп, где я сейчас?» В других случаях, выполняя какую-либо операцию, вы вдруг понимаете, что делать этого не стоило. Но слишком поздно — и вот вы уже потеряли верхний коммит с недельным результатом усердной работы.

Без паники! В обоих случаях вам поможет журнал ссылок. С ним вы сможете убедиться, что операции были выполнены ожидаемым образом и в правильных ветках. Кроме того, он позволяет восстановить утраченные коммиты, если что-то пошло не так.

Журнал ссылок (reflog) хранит в себе записи изменений в вершинах веток внутри непустых репозиториях. При каждом обновлении какой-либо ссылки, включая `HEAD`, журнал также обновляется, фиксируя это изменение. Это как след из хлебных крошек, показывающий пройденный вами и вашими ссылками путь.

Вы также можете использовать журнал ссылок для отслеживания маршрута по хлебным крошкам вплоть до нужных изменений веток.

Вот базовые операции, которые регистрируются в журнале:

- клонирование;
- отправка изменений;

- создание новых коммитов;
- изменение или создание веток;
- перебазирование;
- сброс.

Регистрируется любая операция Git, изменяющая ссылку или вершину ветки.

По умолчанию журнал активен в непустых репозиториях и отключен в пустых (голых). Это состояние управляется логической опцией конфигурации `core.logAllRefUpdates`. Для каждого отдельного репозитория ее можно включить с помощью команды `git config core.logAllRefUpdates true` и отключить командой `false`.



Пустой, или голый (bare), репозиторий — это созданный на сервере репозиторий, который не имеет рабочего каталога. Можете представить его как аналог схемы. Когда разработчики в команде вносят в схему изменения, эти изменения от каждого из них отражаются в копии схемы, не затрагивая ее саму.

Вот как выглядит журнал ссылок:

```
$ git reflog show
a44d980 HEAD@{0}: reset: moving to main
79e881c HEAD@{1}: commit: last foo change
a44d980 HEAD@{2}: checkout: moving from main to fred
a44d980 HEAD@{3}: rebase -i (finish): returning to refs/heads/main
a44d980 HEAD@{4}: rebase -i (pick): Tinker bar
a777d4f HEAD@{5}: rebase -i (pick): Modify bar
e3c46b8 HEAD@{6}: rebase -i (squash): More foo and bar with additional stuff.
8a04ca4 HEAD@{7}: rebase -i (squash): updating HEAD
1a4be28 HEAD@{8}: checkout: moving from main to 1a4be28
ed6e906 HEAD@{9}: commit: Tinker bar
6195b3d HEAD@{10}: commit: Squash into 'more foo and bar'
488b893 HEAD@{11}: commit: Modify bar
1a4be28 HEAD@{12}: commit: More foo and bar
8a04ca4 HEAD@{13}: commit (initial): Initial foo and bar.
```

И хотя в нем регистрируются транзакции для всех ссылок, `git reflog show` выводит их только для одной за раз.

Пример выше показывает ссылку по умолчанию, `HEAD`. Если вы вспомните, что имена веток — это тоже ссылки, то поймете, что можете получить журнал и для любой ветки. В предыдущем примере видно, что в репозитории есть ветка `fred` — значит, мы можем вывести ее изменения с помощью очередной команды:

```
$ git reflog fred
a44d980 fred@{0}: reset: moving to main
79e881c fred@{1}: commit: last foo change
a44d980 fred@{2}: branch: Created from HEAD
```

Каждая строка отражает отдельную транзакцию из истории ссылки, начиная с последнего изменения и назад во времени. В левом столбце содержатся ID коммитов на момент внесения соответствующих изменений. Такие записи, как `HEAD@{7}` из второго столбца, предоставляют удобные имена для коммита в каждой транзакции. Таким образом, `HEAD@{0}` — это последняя запись, `HEAD@{1}` фиксирует, где `HEAD` находилась непосредственно перед этим, и т. д. Самая старая запись, `HEAD@{13}`, фактически является первым коммитом в репозитории. Остальная часть каждой строки после двоеточия описывает произошедшую транзакцию. Наконец, для каждой транзакции есть временная метка (не показана), которая регистрирует момент, когда произошло соответствующее событие.

В чем польза? У журналов ссылок есть один интересный аспект: последовательно пронумерованные имена `HEAD` (например, `HEAD@{1}`) можно использовать в качестве символических имен коммитов для любой получающей коммит команды `Git`. Например:

```
$ git show HEAD@{10}
commit 6195b3dfd30e464fffb9238d89e3d15f2c1dc35b0
Author: Jon Loeliger <jdl@example.com>
Date:   Sat Oct 29 09:57:05 2011 -0500
```

```
    Squash into 'more foo and bar'
```

```
diff --git a/foo b/foo
index 740fd05..a941931 100644
--- a/foo
+++ b/foo
@@ -1,2 +1 @@
-Foo!
-more foo
+junk
```

Это означает, что в потоке разработки, регистрируя коммиты, перемещаясь на другие ветки, выполняя перебазирование и иным образом управляя веткой, вы всегда можете использовать журнал ссылок, чтобы найти прежнее состояние этой ветки. Имя `HEAD@{1}` всегда будет указывать на предыдущий коммит ветки, `HEAD@{2}` относится к предшествующему ему `HEAD`-коммиту и т. д.

Но важно помнить, что хотя история и показывает отдельные коммиты, в ней также есть транзакции, помимо `git commit`. Каждый раз, когда вы перемещаете вершину ветки на другой коммит, эта операция логируется.

Получается, что `HEAD@{3}` не обязательно означает третью предшествующую операцию `git commit`. Говоря точнее, это имя означает третий предшествующий посещенный или адресованный коммит.

`Git` также поддерживает похожие на естественный (английский) язык классификаторы для части ссылки, заключенной в фигурные скобки. Возможно, вы точно

не знаете, сколько произошло изменений с определенного момента, но хотите узнать, как все выглядело вчера или час назад:

```
$ git log 'HEAD@{yesterday}'
commit 1a4be2804f7382b2dd399891eef097eb10ddc1eb
Author: Jon Loeliger <jdl@example.com>
Date: Sat Oct 29 09:55:52 2011 -0500
```

More foo and bar

```
commit 8a04ca4207e1cb74dd3a3e261d6be72e118ace9e
Author: Jon Loeliger <jdl@example.com>
Date: Sat Oct 29 09:55:07 2011 -0500
```

Initial foo and bar.

Git поддерживает множество различных временных квалификаторов для ссылок. К ним относятся слова вроде *yesterday*, *noon*, *midnight*, *afternoon tea*^{1,2}, названия дней недели, месяцев, индикаторы А.М. и Р.М., абсолютное время и относительные фразы вроде *last Monday*, *1 hour ago*, *10 minutes ago*, а также комбинации этих фраз: *1 day*, *2 hours ago*. И наконец, если вы опустите фактическое имя ссылки и просто используете форму *@{...}*, система предположит имя текущей ветки. Таким образом, находясь в ветке *bugfix* и используя только *@{noon}*, вы сошлетесь на *bugfix@{noon}*.



За трактовку ссылок в Git отвечает инструмент `git rev-parse`. Он детально описан в руководстве: вы во всех подробностях можете узнать, как интерпретируются ссылки.

Несмотря на свою свободную форму, эти временные квалификаторы неидеальны. Git использует для их интерпретации определенную эвристику, поэтому применять эти квалификаторы нужно осторожно. Также имейте в виду, что указанное время локальное и относится к вашему репозиторию. Эти обозначенные временем ссылки адресуют значение только в локальном репозитории. Использование той же фразы в другом репозитории наверняка выдаст другие результаты из-за отличий в журналах ссылок. Таким образом, *main@{2.days.ago}* относится к состоянию вашей локальной ветки *main* два дня назад. Если у вас нет истории журнала ссылок, охватывающей этот период, Git выдаст предупреждение:

```
$ git log HEAD@{last-monday}
warning: Log for 'HEAD' only goes back to Sat, 29 Oct 2011 09:55:07 -0500.
commit 8a04ca4207e1cb74dd3a3e261d6be72e118ace9e
```

¹ Кроме шуток. И да, это пять часов вечера.

² Вчера, полдень, полночь, послеполуденный чай (традиционно его пьют в Англии в пять вечера). — *Примеч. ред.*

```
Author: Jon Loeliger <jdl@example.com>
Date:   Sat Oct 29 09:55:07 2011 -0500
```

Initial foo and bar.

И последнее: не дайте оболочке вас обмануть. Между двумя следующими командами есть значительное отличие:

```
# Некорректно!
$ git log dev@{2 days ago}

# Наверняка корректно для вашей оболочки
$ git log 'dev@{2 days ago}'
```

В первом примере без одинарных кавычек оболочке представлено несколько аргументов командной строки. Во втором примере — передано все описание ссылки как один аргумент. Git нужно видеть эту ссылку как одно слово.

Для решения задачи с разрывом слов в Git есть несколько вариантов:

```
# Следующие команды должны быть равнозначны
$ git log 'dev@{2 days ago}'
$ git log dev@{2.days.ago}
$ git log dev@{2-days-ago}
```

Другой нюанс: если Git ведет историю транзакций каждой операции, выполненной для каждой ссылки в репозитории, не приведет ли это к чрезмерному разрастанию журнала ссылок?

К счастью, нет. Git время от времени автоматически выполняет сборку мусора. Некоторые более старые записи журнала признаются устаревшими и удаляются. По умолчанию срок действия коммита, который не имеет ссылок и недостижим из какой-либо ветки, истекает через 30 дней. При этом срок действия достижимых коммитов по умолчанию составляет 90 дней. Взгляните на рис. 10.3.

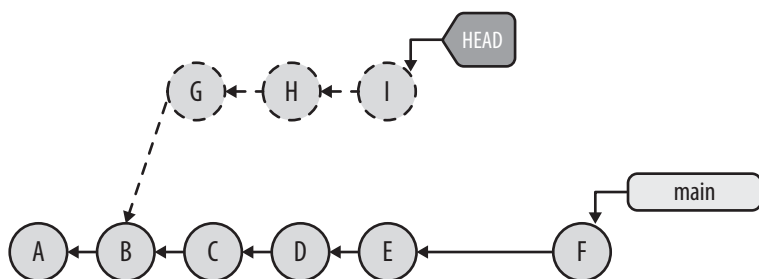


Рис. 10.3. Недостижимые коммиты

На рис. 10.3 коммиты G, H и I недостижимы, а коммиты от A до F достижимые. Ключ в том, куда указывает HEAD. Для коммитов G, H и I HEAD находится

в отсоединенном режиме, и они не связываются с именем ветки. Это результат прямого переключения на коммит В, за которым следует череда новых коммитов.

Если предустановленное расписание сборки мусора вам не подходит, установите переменные конфигурации вашего репозитория `gc.reflogExpireUnreachable` и `gc.reflogExpire` на другие значения. Для удаления отдельных записей журнала используйте команду `git reflog delete` или `git reflog expire`, чтобы напрямую настроить удаление записей старше указанного срока. Последнюю команду можно применять для принудительного обозначения журнала как устаревшего:

```
$ git reflog expire --expire=now --all
$ git gc
```

И последняя деталь: журналы ссылок сохраняются в каталоге `.git/logs`. Файл `.git/logs/HEAD` содержит историю значений `HEAD` в то время, как в подкаталоге `.git/logs/refs/` находится история всех ссылок, включая локальное хранилище. А подкаталог `.git/logs/refs/heads` содержит историю вершин ветки.

Вся информация, хранящаяся в журналах ссылок, в частности в каталоге `.git/logs`, временная и невозвратная. Отбрасывание каталога `.git/logs` или отключение журнала ссылок никак не вредит внутренней структуре данных Git. Это будет лишь означать, что ссылки вроде `main@{4}` не получится разрешить.

И напротив, активация журнала вносит ссылки на коммиты, которые иначе могут оказаться недостижимы. Если вы хотите почистить и уменьшить репозиторий, то удаление журнала ссылок поспособствует удалению коммитов, которые в этом случае станут недостижимыми.

Итоги

В этой главе мы познакомились с двумя инструментами.

Локальное хранилище, или механизм `stash`, — это умный механизм для сохранения незавершенной работы, который позволяет на время переключаться на другие задачи и возвращаться.

Журнал ссылок предоставляет аудит последовательности модификаций, которые изменили ссылку или вершину любой конкретной ветки в течение ограниченного периода времени в локальной среде разработки.

И хотя эти инструменты ориентированы на коммиты и изменения, которые вы вносите в репозиторий, их можно использовать всякий раз, когда вас прерывают для переключения на задачу, требующую неотложного решения.

ГЛАВА 11

Удаленные репозитории

До сих пор мы работали почти исключительно с одним локальным репозиторием. Теперь познакомимся с распределенными возможностями Git и научимся сотрудничать с другими разработчиками через общие репозитории.

Работа с несколькими удаленными репозиториями добавляет новые термины.

Клон (clone) — это копия репозитория. Он содержит все объекты из исходного репозитория, поэтому это независимый и автономный репозиторий и истинная симметричная пара оригинала. Клон позволяет каждому разработчику действовать локально и независимо без централизации, опросов или блокировок. Именно клонирование упрощает масштабирование в Git и дает возможность работать над одним проектом многим географически распределенным участникам.

Раздельные репозитории нужны, когда:

- разработчики работают автономно;
- ожидается значительное разветвление проекта по отдельным путям разработки. И хотя стандартные механизмы ветвления и слияния могут обработать любой объем отделенной разработки, получающаяся в итоге сложность может принести больше проблем, чем пользы. В противоположность этому отделенные пути разработки могут использовать собственные репозитории, выполняя слияние с основным по мере необходимости.

Клонирование репозитория — только первый этап в схеме обмена кодом. Нужно также связать один репозиторий с другим, чтобы установить пути для обмена данными. Git устанавливает эти связи между репозиториями через *удаленные подключения*.

В этой главе пять блоков, каждый логически продолжает предыдущий, объясняя принципы работы с удаленными репозиториями.

«Блок I: основные концепции»: здесь мы разберем понятия репозитория и работу с удаленным репозиторием.

«Блок II: пример использования удаленных репозитория»: приведем примеры и техники для обмена, отслеживания и получения данных между несколькими репозиториями.

«Блок III: цикл разработки в удаленном репозитории в картинках»: закрепим полученные знания с помощью визуализации и разберем жизненный цикл удаленного репозитория.

«Блок IV: конфигурация удаленного репозитория»: обсудим способы управления удаленными конфигурациями любого репозитория.

«Блок V: работа с ветками отслеживания»: объединим все понятия и узнаем о важности работы с ветками удаленного отслеживания, а также о лучших практиках.

Блок I: основные концепции

Рассматриваемые в этой части принципы и концепции — это основные элементы для понимания и работы с репозиторием Git в условиях удаленной и распределенной работы. Следующие разделы познакомят вас с ключевыми понятиями и терминологией, подготовив к работе с репозиторием.

Пустые репозитории и репозитории разработки

Репозиторий Git может быть голым¹ (bare) или с рабочим каталогом (nonbare).

Репозиторий с рабочим каталогом, или репозиторий разработки, используется в обычной повседневной разработке. Он отражает описание текущей ветки и предоставляет ее активную копию в рабочем каталоге. Все до сих пор упомянутые репозитории были именно такими.

Голый репозиторий не содержит рабочего каталога и не должен использоваться для обычной разработки. Такой репозиторий не несет в себе упоминания об активной ветке. Можете представить его себе просто как содержимое каталога `.git`. Делать коммиты в такой репозиторий не нужно.

Возникает вопрос: зачем он вообще нужен? Этот репозиторий играет важную роль, выступая истинной точкой координации совместной разработки. Другие разработчики выполняют `clone` и `fetch` из голого репозитория и отправляют в него обновления. Позже мы разберем пример, который покажет его принцип работы.

Для начала посмотрим, как флаг `--bare` влияет на каталог, инициализируемый при создании репозитория разработки и голого репозитория:

```
# Репозиторий разработки (nonbare)
$ cd /tmp

$ git init -b main fluff
Initialized empty Git repository in /private/tmp/fluff/.git/
```

¹ Bare repository называют также чистым или пустым репозиторием. — *Примеч. ред.*

```
$ tree fluff
fluff
```

```
0 directories, 0 files
```

```
# Голый репозиторий
```

```
$ git init --bare -b main fluff-bare
```

```
Initialized empty Git repository in /private/tmp/fluff-bare/
```

```
fatal: this operation must be run in a work tree
```

```
$ tree fluff-bare
```

```
fluff
├── HEAD
├── config
├── description
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── prepare-commit-msg.sample
│   ├── push-to-checkout.sample
│   └── update.sample
├── info
│   └── exclude
├── objects
│   ├── info
│   └── pack
└── refs
    ├── heads
    └── tags
```

```
8 directories, 17 files
```

Если выполнить команду `git clone` с опцией `--bare`, Git создаст голый репозиторий. В противном случае создается репозиторий разработки.



Обратите внимание, мы не говорили, что `git clone --bare` создает новый или пустой (empty) репозиторий. Команда создает *голый* (bare) репозиторий. В таком репозитории будет находиться копия содержимого из вышестоящего репозитория. Команда `git init` создает новый пустой репозиторий, который может стать как репозиторием разработки, так и голым репозиторием. Ранее созданные репозитории можно сравнивать с помощью `diff`:

```
# Выполните из корневого каталога обоих репозитриев
$ diff -y <(tree fluff/.git/) <(tree fluff-bare)
```

По умолчанию Git активирует журнал ссылок (о чем шла речь в разделе «Журнал ссылок» на с. 250 в главе 10) в репозиториях разработки, но не в голых репозиториях. Это указывает на то, что разработка будет идти в первом, а не во втором. По той же причине в голом репозитории не создаются удаленные подключения.

Если вы настроите репозиторий, в который разработчики будут отправлять изменения, то он должен быть голым. По сути, это частный случай более общей практики, согласно которой опубликованный репозиторий должен быть голым.

Клоны репозитория

Команда `git clone` создает новый репозиторий на основе исходного, который указывается через адрес в файловой системе или сети. Git не нужно копировать всю информацию из исходного репозитория в клонированный. Поэтому система игнорирует информацию, которая касается только исходного, например ту, которая относится к веткам удаленного отслеживания.



При стандартном использовании `git clone` локальные ветки разработки исходного репозитория, сохраненные в `.git/refs/heads/`, становятся ветками удаленного отслеживания в клоне, находящемся в `.git/refs/remotes/`. Ветки удаленного отслеживания внутри `.git/refs/remotes/` исходного репозитория не копируются. (Клону не нужно знать, что отслеживает вышестоящий репозиторий.)

Теги из исходного репозитория копируются в клон, как и объекты, достижимые из скопированных ссылок. Тем не менее собственная информация репозитория, например хуки (см. главу 14), файлы конфигурации, журнал ссылок и локальное хранилище, в клоне не воссоздается.

В разделе «Создание локальной копии репозитория» на с. 41 (глава 1) мы показали, как можно использовать `git clone` для создания копии вашего репозитория `my_website`:

```
$ git clone my_website new_website
```

Здесь `my_website` считается исходным «удаленным» репозиторием. Создаваемый из него клон — это `new_website`.

Аналогично `git clone` можно применить для создания копии репозитория из сети:

```
$ git clone git://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
```

По умолчанию каждый клон сохраняет ссылку, ведущую обратно на родительский репозиторий через удаленное подключение с именем `origin`. А вот исходный репозиторий не знает и не поддерживает ссылки на своих клонов. Здесь работает односторонняя связь¹.

¹ Двухнаправленную связь с удаленным репозиторием можно настроить позже с помощью команды `git remote`.

Имя `origin` не является каким-то особым. Если вы не хотите его использовать, то просто укажите при клонировании альтернативное имя через опцию `--origin name`.

```
$ git clone git://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git \
> --origin somename
```

Git также настраивает удаленный репозиторий по умолчанию с `refspec` для извлечения данных (`refspec` — это спецификация передачи данных в и из удаленного репозитория, о которых будет рассказано позже).

Установка `fetch` указывает, что вы хотите продолжить обновлять локальный репозиторий, получая изменения из исходного. В этом случае ветки удаленного репозитория оказываются доступны в клоне под именами с префиксом `origin/`, например `origin/main`, `origin/dev` или `origin/maint`.

Пример `refspec` из файла конфигурации репозитория выглядит так:

```
fetch = +refs/heads/*:refs/remotes/origin/*
```

Удаленные репозитории

Репозиторий, в котором вы сейчас работаете, называется *локальным*, или *текущим*, *репозиторием*. Репозиторий, с которым вы хотите обмениваться файлами, называется *удаленным репозиторием*. Но последний термин не совсем точен, поскольку репозиторий необязательно физически удален или даже находится на другой машине. Иногда им вполне может быть просто другой репозиторий в локальной файловой системе. В главе 12 мы уточним, что для обозначения удаленного репозитория, из которого в результате клонирования получился ваш локальный, обычно применяется термин *вышестоящий (upstream) репозиторий*.

Для связи с другим репозиторием и обращения к нему Git использует как удаленное подключение, так и ветки удаленного отслеживания. Удаленное подключение предоставляет для репозитория удобное имя и может использоваться вместо фактического URL репозитория. Оно также формирует часть основания имени для веток удаленного отслеживания этого репозитория.

Для создания, удаления, управления и просмотра удаленного подключения используйте команду `git remote`. Все создаваемые удаленные подключения регистрируются в файле `.git/config`, и управлять ими можно через команду `git config`.

Это показано на рис. 11.1.

Заметьте, что для операций `git clone` и `git pull` на рис. 11.1 обновление локальной истории представлено как точка.

Помимо `git clone` есть и другие распространенные команды, относящиеся к удаленным репозиториям.

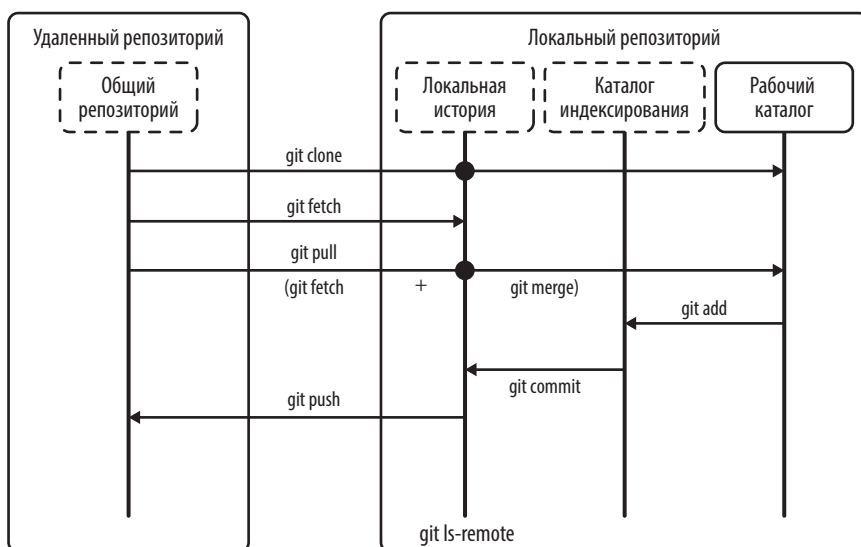


Рис. 11.1. Команда remote

git fetch

Извлекает из удаленного репозитория объекты и их метаданные.

git pull

Аналогична **git fetch**, но также вливает изменения в соответствующую локальную ветку.

git push

Переносит объекты и их метаданные в удаленный репозиторий.

git ls-remote

Показывает список ссылок вместе с SHA-1 их коммитов, находящихся в указанном удаленном репозитории (или на вышестоящем сервере). Эта команда косвенно отвечает на вопрос «Доступно ли обновление?».

Ветки отслеживания

Клонировав репозиторий, вы можете отслеживать изменения, происходящие в исходном репозитории, даже совершая локальные коммиты и создавая локальные ветки.

По мере развития Git некоторая терминология, связанная с именами веток, также развивалась и стандартизировалась. В итоге были созданы различные

пространства имен, помогающие прояснить назначение тех или иных веток. И хотя любая ветка в локальном репозитории по-прежнему считается локальной, их можно дополнительно разделить на категории:

- *Ветки удаленного отслеживания* (remote-tracking) связаны с удаленным репозиторием и предназначены для отслеживания изменений в каждой его ветке.
- *Ветка локального отслеживания* (local-tracking) совмещается с веткой удаленного отслеживания. Это вид интегрированной ветки, которая собирает изменения из локальной ветки разработки и ветки удаленного отслеживания.
- Любая локальная ветка, не предназначенная для отслеживания, обычно называется *тематической* (topic) или *веткой разработки* (development).
- Завершает список пространств имен *удаленная ветка* (remote), расположенная в нелокальном, удаленном репозитории. Как правило, она выступает вышестоящим источником для ветки удаленного отслеживания.

Во время клонирования Git создает в клоне ветку удаленного отслеживания для каждой ветки разработки в вышестоящем репозитории. Набор веток удаленного отслеживания вносится в новое отдельное пространство имен внутри локального репозитория, относящееся к клонируемому удаленному репозиторию. Локальный репозиторий использует свои ветки удаленного отслеживания для наблюдения за изменениями, происходящими в удаленном репозитории:

```
# Клонирование репозитория и создание локальной ветки
$ cd git-repo

$ git branch -a
main                <-- # ветка локального отслеживания
* mylocal-branch    <-- # ветка разработки
remotes/origin/main <-- # ветки удаленного отслеживания
```

В этом фрагменте кода «удаленная ветка» не представлена. Такой сценарий мог возникнуть, если новая ветка создавалась в удаленном репозитории после выполнения операции клонирования.



Из раздела «Ссылки и символические ссылки» на с. 104 в главе 4 вы можете помнить, что локальная ветка разработки, которую вы называете *dev*, на самом деле имеет имя `refs/heads/dev`. Аналогично ветки удаленного отслеживания сохраняются в пространстве имен `refs/remotes/`. Таким образом, ветка удаленного отслеживания `origin/main` фактически является `refs/remotes/origin/main`.

Поскольку ветки удаленного отслеживания собираются в собственном пространстве имен, есть четкое разделение между ветками, которые вы создали в репозитории (ветками разработки), и теми, которые находятся в другом, удаленном репозитории (ветками удаленного отслеживания). На заре существования Git отдельные пространства имен были лучшей практикой того, как огрaдить

пользователей от создания случайных конфликтов. Сегодня же отдельные пространства имен уже считаются гораздо более развернутой функциональностью, и это неотъемлемая часть предполагаемого процесса использования веток для взаимодействия с вышестоящими репозиториями.

Все операции, которые вы можете выполнять для стандартной ветки разработки, можно выполнить и для ветки отслеживания. Но нужно учитывать некоторые ограничения.

Поскольку ветки удаленного отслеживания используются исключительно для отслеживания изменений в другом репозитории, их нужно рассматривать как предназначенные только для чтения. Не следует выполнять слияние или делать коммиты в такие ветки. Это приведет к утрате синхронизации этой ветки с удаленным репозиторием. Что еще хуже, каждое последующее обновление из этого репозитория наверняка будет требовать слияния, все больше усложняя управление клоном. В этой главе мы еще поговорим о правильном использовании веток отслеживания.

Ссылки на другие репозитории

Для согласования вашего репозитория с другим нужно определить удаленное подключение. Это подразумевает создание именованной сущности, хранящейся в файле конфигурации. Файл будет состоять из двух частей. В первой указывается имя другого репозитория в виде URL. Во второй части, которая называется *refspec*, указывается, как ссылка (которая обычно представляет ветку) должна отображаться из пространства имен одного репозитория в пространство имен другого.

Разберем эти компоненты по очереди.

Обращение к удаленным репозиториям

Git поддерживает несколько типов универсальных указателей ресурса (Uniform Resource Locators, URL), которые можно использовать для именования удаленных репозиториях. Эти URL содержат как протокол доступа, так и адрес данных.

Технически URL в Git не являются истинными URL или URI (универсальными идентификаторами ресурса, Uniform Resource Identifiers), поскольку не отвечают в полной мере спецификациям RFC 1738 или RFC 2396. Но это удобное обозначение для расположения репозиториях. Обычно их называют *Git URL*. Более того, в файле `.git/config` также используется имя `url`.

Как вы видели, простейшая форма Git URL относится к репозиторию в локальной файловой системе, будь то физическая система или виртуальная, смонтированная локально через сетевую файловую систему (Network File System, NFS). Есть два варианта:

```
/path/to/repo.git  
file:///path/to/repo
```

И хотя эти два формата по сути идентичны, между ними есть тонкое, но важное отличие. В первом используются жесткие ссылки внутри файловой системы для прямого обмена одинаковыми объектами между текущим и удаленным репозиториями. Второй же вместо того, чтобы обмениваться объектами напрямую, копирует их. Чтобы избежать проблем, связанных с общими репозиториями, предпочтительна форма `file://`.

Далее приводятся другие формы Git URL, ссылающиеся на репозитории в удаленных системах.

Когда есть настоящий удаленный репозиторий, данные из которого нужно извлечь в сеть, самым эффективным способом их передачи будет *нативный протокол Git*. Вот примеры URL нативного протокола:

```
git://example.com/path/to/repo.git
git://example.com/~user/path/to/repo.git
```

Используя эти формы, `git-daemon` публикует репозитории для анонимного чтения. С их помощью можно как клонировать репозитории, так и получать их.

Для безопасных аутентифицированных соединений внутренний протокол Git можно туннелировать через протокол безопасной оболочки (Secure Shell, SSH), используя следующие шаблоны URL:

```
ssh://[user@]example.com[:port]/path/to/repo.git
ssh://[user@]example.com/path/to/repo.git
ssh://[user@]example.com/~user2/path/to/repo.git
ssh://[user@]example.com/~path/to/repo.git
```

Третья форма позволяет указать два имени пользователя. Первое — это пользователь, под которым аутентифицирован сеанс, а второй — это тот, к чьему домашнему каталогу происходит доступ.

Git также поддерживает форму URL с `scp`-подобным синтаксисом. Она идентична формам SSH, но не позволяет указывать параметр порта:

```
[user@]example.com:/path/to/repo.git
[user@]example.com:~user/path/to/repo.git
[user@]example.com:path/to/repo.git
```

И хотя HTTP- и HTTPS-варианты URL изначально поддерживались в Git полностью, начиная с версии 1.6.6 они претерпели серьезные изменения:

```
http://example.com/path/to/repo.git
https://example.com/path/to/repo.git
```

До версии 1.6.6 ни HTTP-, ни HTTPS-протокол не могли по своей эффективности сравниться с нативным протоколом Git. В этой же версии протокол HTTP был значительно доработан и, по сути, сравнялся по эффективности с нативным.

В литературе Git эта реализация упоминается как «умная» в противоположность предыдущей, называемой «тупой».

С возрастанием эффективности HTTP-протокола начала расти и частота употребления форм URL `http://` и `https://`. Примечательно, что большинство корпоративных файерволов разрешают оставлять открытыми HTTP-порт 80 и HTTPS-порт 443, притом что базовый порт Git 9418 обычно блокируется. Более того, эти формы URL предпочитают использовать популярные ресурсы хостинга Git вроде GitHub.

Наконец, можно указать протокол Rsync:

```
rsync://example.com/path/to/repo.git
```

Использование Rsync не приветствуется, поскольку он отстает от своих аналогов. Но если возникает крайняя необходимость, то его следует применять только для изначального клонирования, когда ссылка на удаленный репозиторий должна заменяться каким-то другим механизмом. Если продолжать использовать Rsync для последующих обновлений, то это может привести к утрате локально созданных данных.

Эти протоколы, как уже говорилось, предоставляют способ ссылаться на удаленные репозитории, но фактически также определяют метод, которым могут передаваться данные из удаленного репозитория. Каждый протокол передачи имеет собственные преимущества и недостатки. Как правило, основные вопросы связаны с безопасностью (шифрованием и необходимостью аутентифицированного доступа к передаваемым данным репозитория), простотой настройки протоколов на серверах сети и опытом работы специалистов с удаленными репозиториями.



Вам может быть интересно, какой протокол лучше подойдет при клонировании репозитория: SSH или HTTPS.

Оба этих протокола безопасны. Но когда нужно обратиться к удаленному репозиторию с нескольких машин, при использовании SSH придется генерировать ключ SSH для каждой машины и настраивать его на сервере, прежде чем вы сможете выполнить клонирование. В случае использования HTTPS этот шаг необязателен, поскольку вы можете аутентифицироваться через всплывающее окно при выполнении клонирования.

Refspec

В разделе «Ссылки и символические ссылки» на с. 104 в главе 4 мы объяснили, как ссылка (ref) именует конкретный коммит в истории репозитория. Обычно она представляет имя ветки. Спецификация ссылок (refspec) сопоставляет имена веток в удаленном репозитории с именами веток в локальном.

Поскольку refspec должна одновременно именовать ветки из локального и удаленного репозитория, в ней обычно приводятся и даже требуются полные имена

веток. В refspec имена веток разработки, как правило, указываются с префиксом `refs/heads/`, а имена веток удаленного отслеживания — с префиксом `refs/remotes/`. (В целях сокращения мы опускали `.git/`, когда указывали пути ссылок.)

Спецификации ссылок добавляются в файл `.git/config` вашего репозитория, когда вы его клонируете либо добавляете или обновляете ведущие на него удаленные подключения (добавление удаленных подключений разберем в этой главе чуть позже). Пример:

```
$ cd git-repo
$ cat .git/config
...
...
[remote "origin"]
    url = https://github.com/ppremk/git.git

    # The Refspec
    fetch = +refs/heads/*:refs/remotes/origin/*
...
...
```

А вот синтаксис refspec:

`[+]source:destination`

Эта спецификация состоит из исходной ссылки (*source*), двоеточия (`:`) и целевой ссылки (*destination*). Перед ней может идти знак «плюс» (`+`), указывающий, что во время передачи не будет выполняться стандартная проверка безопасности выполнения перемотки. Она также может предваряться звездочкой (`*`) для использования ограниченной формы подстановки в именах веток.

Бывают случаи, в которых ссылку *source* можно не указывать, как и случаи, в которых не обязательно указывать двоеточие и ссылку *destination*.

Спецификации ссылок используются и командой `git fetch`, и командой `git push`. Для такой спецификации нужно понимать определяемый ею поток данных. Сама refspec всегда представлена как *source:destination*, но роли *source* и *destination* зависят от выполняемой операции Git. Эта связь в общей форме отражена в табл. 11.1.

Таблица 11.1. Поток данных refspec

Операция	Источник	Цель
push	Отправляемая локальная ссылка	Обновляемая удаленная ссылка
fetch	Извлекаемая удаленная ссылка	Обновляемая локальная ссылка

Стандартная команда `git fetch` использует `refspec` так:

```
+refs/heads/*:refs/remotes/remote/*
```

Такую `refspec` можно перефразировать следующим образом.

Все исходные ветки удаленного репозитория в пространстве имен `refs/heads/`:

- 1) отображаются в локальный репозиторий, используя имя, состоящее из имени `remote` и
- 2) помещаются в пространство имен `refs/remotes/remote`.

Из-за звездочки эта спецификация применяется к нескольким веткам, находящимся в пространстве имен `refs/heads/` удаленного репозитория. Именно такая спецификация заставляет ветки разработки удаленного репозитория отображаться в пространство имен вашего репозитория в качестве веток удаленного отслеживания и отделяет их в подымена на основе имени удаленного репозитория.

Существует общепринятая практика, хотя и необязательная, помещать ветки конкретного `remote` в `refs/remotes/remote/*` (например, `refs/remotes/origin/*`).



Используйте `git show-ref` для вывода ссылок в текущем репозитории и `git ls-remote repository` для вывода ссылок в удаленном.

Поскольку первый шаг `git pull` — это `fetch`, используемые в этой операции спецификации ссылок в равной степени применимы к `git pull`.



Не делайте коммиты или слияния с веткой удаленного отслеживания, указанной в правой части спецификации ссылок `pull` или `fetch`. Эти ссылки будут использованы как ветки удаленного отслеживания.

Во время операции `git push` обычно нужно предоставить и опубликовать изменения, внесенные в локальных ветках разработки. Чтобы позволить другим найти ваши изменения в удаленном репозитории после их выгрузки, ваши изменения должны находиться в этом репозитории в качестве веток разработки. Таким образом, во время стандартной команды `git push` исходные ветки из вашего репозитория отправляются в удаленный репозиторий с использованием `refspec` по такому шаблону:

```
+refs/heads/*:refs/heads/*
```

Это означает следующее:

Из локального репозитория взять имя каждой ветки в пространстве имен `refs/heads/` и поместить его в соответствующую ветку с таким же именем в целевом пространстве имен `refs/heads/` удаленного репозитория.

Первое `refs/heads/` относится к локальному репозиторию (поскольку вы выполняете `push`), а второе — к удаленному. Звездочка обеспечивает дублирование всех веток.

В командах `git fetch` и `git push` можно указывать несколько `refspec`. В определении удаленного репозитория можно указывать несколько `refspec` операции `fetch`, несколько `refspec` операции `push` или комбинацию того и другого.

А что, если вообще не указать `refspec` в команде `git push`? Откуда Git узнает, что делать или куда отправлять данные?

Прежде всего без явного указания в команде удаленного репозитория Git предположит, что вы хотите использовать `origin`. Без `refspec` `git push` отправит ваши коммиты для всех веток, являющихся общими между вашим репозиторием и вышестоящим. Любая локальная ветка, которой еще нет в вышестоящем репозитории, в него отправлена не будет. Ветки должны уже существовать и совпадать по именам. Таким образом, новые ветки должны передаваться явно по имени. Позднее их можно будет задавать по умолчанию с помощью простой `git push`. В связи с этим предустановленная `refspec` делает следующие команды равнозначными:

```
$ git push origin branch
$ git push origin branch:refs/heads/branch
```

Примеры см. далее в этой главе в разделе «Добавление и удаление удаленных веток» на с. 298.

Блок II: пример использования удаленных репозитория

Теперь у вас есть основа для реализации сложного обмена данными через Git. Чтобы сохранить универсальность подачи материала и привести примеры, которые вы сможете легко воспроизвести на своих машинах, в этом разделе показаны несколько репозиториях на одной физической машине. В реальной жизни они наверняка находились бы на разных хостах в разных частях интернета. Здесь могут использоваться и другие формы удаленных URL, поскольку для репозиториях на физически удаленных машинах действуют те же механизмы.

Рассмотрим некий типичный случай. Для примера настроим репозиторий, который все разработчики будут считать достоверным (*authoritative*), хотя, по сути,

он ничем не отличается от других репозиториев. Достоверность заключается том, как все решают рассматривать репозиторий, а не в каких-то мерах безопасности или технических аспектах.

Утвержденная достоверная копия зачастую помещается в специальный каталог — *контейнер* (depot). Для его создания, как правило, есть весомые основания. Например, ваша организация сможет надежно и профессионально создавать резервные копии файловых систем крупного сервера. Или вы хотите убедить коллег проверять все содержимое основной копии в контейнере, чтобы избежать катастрофических потерь. Контейнер будет выступать удаленным исходным репозиторием для всех разработчиков.

Далее вы увидите, как поместить исходный репозиторий в контейнер, клонировать репозитории разработки из контейнера, выполнять разработку в команде и затем синхронизировать ее с контейнером.

Чтобы показать параллельную разработку в этом репозитории, второй разработчик клонирует его, работает с ним и затем отправляет изменения обратно в контейнер для общего использования.

Создание достоверного репозитория

Вы можете разместить свой достоверный контейнер в любом месте файловой системы. Для этого примера мы используем `/tmp/Depot`. В самом каталоге `/tmp/Depot` и его репозиториях не должно вестись никакой разработки. Вся индивидуальная работа должна производиться в локальном клоне.

На практике такой достоверный вышестоящий репозиторий наверняка будет размещен на каком-то сервере, возможно `git.kernel.org`, GitHub или на одной из ваших собственных машин.

Однако эти шаги подчеркивают, что именно требуется для преобразования репозитория в другой голый клон, который сможет выступить достоверным вышестоящим исходным репозиторием.

Первым шагом будет заполнение `/tmp/Depot` начальным репозиторием. Предположим, вы работаете с содержимым сайта, которое уже установлено как репозиторий Git в `~/my_website`. Сделайте копию этого репозитория и поместите ее в `/tmp/Depot/my_website.git`:

```
# Предположим, что ~/my_website – это уже репозиторий Git
$ cd /tmp/Depot/
$ git clone --bare ~/my_website my_website.git
Cloning into bare repository 'my_website.git'...
done.
```

Эта команда `clone` скопирует удаленный репозиторий Git из `~/my_website` в текущий рабочий каталог, `/tmp/Depot`. Последний аргумент присваивает этому репозиторию новое имя, `my_website.git`. По общему соглашению голые репозитории

именуются с использованием суффикса `.git`. Это не обязательное требование, но оно считается лучшей практикой.

Исходный репозиторий разработки содержит весь набор файлов проекта, доступ к которым активен на верхнем уровне, а объектное хранилище и все файлы конфигурации расположены в подкаталоге `.git`:

```
$ cd ~/my_website/
$ ls -aF
./   fuzzy.txt   index.html   techinfo.txt
../  .git/        poem.html

# Примечание: вывод в вашем терминале может отличаться
$ ls -aF .git
./      HEAD          hooks/        logs/
../     config         index         objects/
COMMIT_EDITMSG  description  info/         refs/
```

Поскольку в баре-репозитории нет рабочего каталога, его файлы имеют более простую структуру:

```
$ cd /tmp/Depot/

$ ls -aF my_website.git
./      HEAD          description   info/         packed-refs
../     config         hooks/       objects/      refs/
```

Чтобы просмотреть конфигурацию в новом голом репозитории, сделайте следующее:

```
# В /tmp/Depot/my_website.git

$ cat config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = true
    ignorecase = true
    precomposeunicode = true
[remote "origin"]
    url = ~/users/my_website
```

Поскольку во время клонирования мы использовали опцию `--bare`, Git установил `bare` как `true`. Из-за операции клонирования удаленное подключение с именем `origin` уже есть в файле конфигурации. Поскольку мы хотим, чтобы эта копия голого репозитория была достоверным источником, то удалим `origin`, выполнив команду:

```
$ git remote remove origin
$ cat config
[core]
```

```

repositoryformatversion = 0
filemode = true
bare = true
ignorecase = true
precomposeunicode = true

```

Теперь вы можете рассматривать этот голый репозиторий `/tmp/Depot/my_website.git` как достоверную версию.

Создание собственного удаленного подключения `origin`

Сейчас есть два репозитория, которые практически идентичны, за исключением того, что в исходном есть рабочий каталог, а в голом — нет.

Поскольку репозиторий `~/my_website` в каталоге `home` был создан с помощью `git init`, а не через `clone`, у него нет `origin`. Для него вообще не установлено удаленное подключение:

```

$ cd ~/my_website
$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true

```

Добавить его несложно. И оно потребуется, если наша цель — произвести дополнительную разработку в исходном репозитории и затем передать эту разработку в новый достоверный репозиторий в контейнере. В некотором смысле нужно вручную преобразовать исходный репозиторий в производный клон.

Разработчик, выполняющий клонирование из контейнера, автоматически создает удаленное подключение `origin`. Но если вы сейчас вернетесь назад и сделаете клонирование из контейнера, то получите его автоматически.

Управление удаленными подключениями происходит через команду `git remote`. Эта операция добавляет в файл `.git/config` несколько новых строк:

```

$ cd ~/my_website
$ git remote add origin /tmp/Depot/my_website

$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
[remote "origin"]
    url = /tmp/Depot/my_website
    fetch = +refs/heads/*:refs/remotes/origin/*

```

Здесь команда `git remote` добавила в нашу конфигурацию новый пункт `remote` с именем `origin`. Имя `origin` не является каким-то особенным. Можно использовать любое другое имя, но удаленное подключение, которое указывает на исходный репозиторий, принято называть `origin`.

Это подключение связывает ваш текущий репозиторий с удаленным: согласно значению `url` в данном случае это `/tmp/Depot/my_website.git`. Для удобства суффикс `.git` не требуется; при этом также сработают варианты `/tmp/Depot/my_website` и `/tmp/Depot/my_website.git`.

Теперь в этом репозитории имя `origin` стало ярлыком для удаленного репозитория, находящегося в контейнере. Заметьте, что была также добавлена предустановленная `refspec` для `fetch`, соответствующая соглашениям об отображении имен веток.

Связь между репозиторием, который содержит удаленную ссылку (рефеорирующий репозиторий, *referrer*), и этим удаленным репозиторием (рефеорируемый репозиторий, *referee*) асимметрична. Удаленное подключение всегда указывает от рефеорирующего к рефеорируемому. Рефеорируемый репозиторий не знает, что на него указывает какой-то другой репозиторий. То есть, иными словами, клон знает, где находится его вышестоящий репозиторий, но вышестоящий репозиторий не знает, где находятся его клоны.

Теперь завершив настройку удаленного подключения `origin`, установив новые ветки удаленного отслеживания в исходном репозитории, чтобы представить ветки из удаленного. Мы видим, что в нем есть лишь одна ветка, `main`, как и ожидалось:

```
# Вывести все ветки

$ git branch -a
main

Используем git remote update:

$ git remote update
Fetching origin
From /tmp/Depot/my_website
* [new branch]      main          -> origin/main

$ git branch -a
* main
remotes/origin/main
```

Git внес в репозиторий новую ветку `remotes/origin/main`. Это ветка удаленного отслеживания в удаленном репозитории `origin`, где никто не ведет разработку. Ее цель — хранить и отслеживать коммиты, совершенные в ветке `main` репозитория `origin`. Вы можете считать эту ветку посредником локального репозитория для коммитов, совершенных в удаленном репозитории. В итоге вы можете использовать ее для внесения этих коммитов в свой репозиторий.

Фраза `Fetching origin`, созданная командой `git remote update`, означает, что запись об `origin`, находящаяся в *локальном* репозитории, была обновлена на основе информации, полученной из удаленного репозитория.



Команда `git remote update` привела к обновлению всех удаленных подключений этого репозитория путем проверки и последующего получения всех новых коммитов из каждого репозитория, названного в подключении.

Вместо общего обновления всех удаленных подключений можно ограничить эту операцию получением обновлений только для одного из них, передав его имя в команде `git remote update`:

```
$ git remote update remote_name
```

Кроме того, использование опции `-f` в момент изначального добавления удаленного подключения приводит к выполнению `fetch` из соответствующего репозитория:

```
$ git remote add -f origin repository
```

На этом мы закончили связывать репозиторий с удаленным репозиторием в контейнере.

Разработка в вашем репозитории

Поработаем немного в репозитории и добавим еще один стишок, `fuzzy.txt`:

```
$ cd ~/my_website

$ git show-branch -a
* [main] add new file
! [origin/main] add new file
--
*+ [main] add new file

$ cat fuzzy.txt
Fuzzy Wuzzy was a bear
Fuzzy Wuzzy had no hair
Fuzzy Wuzzy wasn't very fuzzy,
Was he?

$ git add fuzzy.txt
$ git commit -m "Add a new poem"
[main 2daf640] Add new poem
1 files changed, 4 insertions(+)
create mode 100644 fuzzy.txt

$ git show-branch -a
* [main] Add new poem
! [origin/main] add new file
--
* [main] Add new poem
*+ [origin/main] add new file
```

Теперь в вашем репозитории на один коммит больше, чем в репозитории из `/tmp/Depot`. В вашем репозитории две ветки: одна (`main`) с новым коммитом и вторая (`origin/main`), которая отслеживает удаленный репозиторий.

Передача изменений

Любое изменение, которое вы фиксируете, локально для вашего репозитория и пока не присутствует в удаленном. Получать коммиты из ветки `main` в удаленный репозиторий `origin` удобно с помощью команды `git push`. Параметр `main` в этой команде будет зависеть от вашей версии Git:

```
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 343 bytes | 343.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To /tmp/Depot/my_website
7e876fd..2daf640 main -> main
```

Этот вывод означает, что Git получил изменения ветки `main`, объединил их и отправил в удаленный репозиторий `origin`. Git также выполнил еще один шаг: получил те же самые изменения и добавил их в ветку `origin/main` вашего репозитория. Git не повторяет изменения. В конце концов эти коммиты уже находятся в вашем репозитории. Система достаточно умна, чтобы вместо этого просто выполнить перемотку для ветки удаленного отслеживания.

Теперь обе локальные ветки, `main` и `origin/main`, отражают один и тот же коммит в репозитории:

```
$ git show-branch -a
* [main] Add a new poem
! [origin/main] Add a new poem
--
*+ [main] Add a new poem
```

Вы можете заглянуть в удаленный репозиторий и убедиться, что он тоже обновлен. Если ваш удаленный репозиторий находится в локальной файловой системе, как в нашем случае, то его можно легко проверить, перейдя в каталог контейнера:

```
$ cd /tmp/Depot/my_website.git
$ git show-branch
[main] Add a new poem
```

Когда удаленный репозиторий находится на другой физической машине, получить информацию о его ветках можно с помощью низкоуровневой команды:

```
$ cd ~/my_website
```

```
$ git ls-remote
```

```
From /tmp/Depot/my_website
2daf64034a769ee02f8cedd9fcb2e43b04fe7c17 HEAD
2daf64034a769ee02f8cedd9fcb2e43b04fe7c17 refs/heads/main
```



В нашем примере путь `From` выводит локальный каталог `Depot`. Как правило, это URL удаленного репозитория Git, если он находился на другом сервере, где размещен репозиторий.

Теперь вы можете показать, что ID этих коммитов соответствуют вашим текущим локальным веткам, используя, например, `git rev-parse HEAD` или `git show commit-id`:

```
$ git rev-parse HEAD
```

```
2daf64034a769ee02f8cedd9fcb2e43b04fe7c17
```

Добавление нового разработчика

Как только вы установили достоверный репозиторий, можно добавить в проект нового разработчика. Для этого ему достаточно создать клон вашего репозитория, в котором он сможет начать работать.

Пригласим в проект Лизу, предоставив ей собственный клонированный репозиторий.

```
$ cd /tmp/lisa
```

```
$ git clone /tmp/Depot/my_website.git
```

```
Cloning into 'my_website'...
done.
```

```
$ ls
```

```
my_website
```

```
$ cd my_website
```

```
$ ls
```

```
fuzzy.txt    index.html    poem.html     techinfo.txt
```

```
$ git branch
```

```
* main
```

```
$ git log -1
```

```
commit 6f168803f6f1b987dff55fff77531dcadf7f4b68
```

```
Author: Jon Loeliger <jdl@example.com>
```

```
Date: Sat Nov 6 18:57:37 2021 +0100
```

Add a new poem

После выполнения `ls` мы видим, что клон содержит рабочий каталог со всеми файлами, подлежащими контролю версий. То есть созданный для Лизы клон — это репозиторий разработки, а не голый репозиторий. Лиза тоже будет заниматься разработкой.

Вывод `git log` показывает, что в репозитории Лизы доступен последний коммит. Кроме того, поскольку ее репозиторий был клонирован из родительского, по умолчанию у него есть удаленное подключение `origin`. Лиза может получить дополнительную информацию об этом подключении внутри собственного репозитория:

```
$ git remote show origin
* remote origin
Fetch URL: /tmp/Depot/my_website.git
Push URL: /tmp/Depot/my_website.git
HEAD branch: main
Remote branch:
  main tracked
Local branch configured for 'git pull':
  main merges with remote main
Local ref configured for 'git push':
  main pushes to main (up to date)
```

В выводе всего содержимого файла конфигурации после стандартного клонирования мы видим удаленное подключение `origin`:

```
$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
    ignorecase = true
    precomposeunicode = true
[remote "origin"]
    url = /tmp/Depot/my_website.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = origin
    merge = refs/heads/main
```

Помимо удаленного подключения `origin` в репозитории Лизы есть несколько веток. Вывести все ветки своего репозитория можно командой

```
git branch -a:
$ git branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
```

Основная ветка разработки Лизы — это `main`, обычная локальная тематическая ветка. Она также выступает веткой локального отслеживания, связанной с веткой

удаленного отслеживания, имеющей аналогичное имя `main`. Ветка `origin/main` — это ветка удаленного отслеживания для наблюдения за коммитами из ветки `main` репозитория `origin`. Ссылка `origin/HEAD` через символическое имя указывает, какую ветку удаленное подключение считает активной. Наконец, звездочка рядом с именем `main` означает, что это текущая активная ветка репозитория.

В этом можно убедиться, выполнив команду `git branch` с опцией `-vv`, вывод которой покажет описанную связь веток локального и удаленного отслеживания:

```
$ git branch -vv
* main 2daf640 [origin/main] Add a new poem
```

Теперь Лиза сделает коммит, изменяющий новый стишок, и затем отправит этот коммит в основной репозиторий. Лиза решает, что последней строчкой должна быть *Wuzzy?*, и вносит это изменение:

```
$ git diff

diff --git a/fuzzy.txt b/fuzzy.txt
index 0d601fa..608ab5b 100644
--- a/fuzzy.txt
+++ b/fuzzy.txt
@@ -1,4 +1,4 @@
    Fuzzy Wuzzy was a bear
    Fuzzy Wuzzy had no hair
    Fuzzy Wuzzy wasn't very fuzzy,
-Was he?
+Wuzzy?

$ git commit fuzzy.txt
[main c426244] Make the name pun complete!
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Она завершает цикл разработки, передавая внесенные изменения в контейнер, как и раньше, с помощью `git push`:

```
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 330 bytes | 330.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
To /tmp/Depot/my_website.git
 2daf640..a5f8133  main -> main
```

Получение обновлений репозитория

Теперь представим, что Лиза уходит в отпуск, а вы вносите дополнительные изменения и отправляете их в репозиторий. Допустим, вы сделали это после получения последних изменений от Лизы.

Ваш коммит выглядит так:

```
$ cd ~/my_website
$ git diff
diff --git a/index.html b/index.html
index 40b00ff..063ac92 100644
--- a/index.html
+++ b/index.html
@@ -1,5 +1,7 @@
<html>
<body>
My web site is awesome!
+<br/>
+Read a <a href="fuzzy.txt">new</a> poem!
</body>
<html>

$ git commit -m "Add a new poem link." index.html
[main 89cfd9f] Add a new poem link.
1 file changed, 2 insertions(+)
```

Используя предустановленную refspec push, вы отправляете коммит в выше-стоящий репозиторий (в контейнер):

```
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 333 bytes | 333.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
To /tmp/Depot/my_website.git
e023cd1..c4e45f7  main -> main
```

Теперь, когда Лиза вернется, она захочет обновить свой клон репозитория. Основной командой для этого будет `git pull`:

```
$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 943 bytes | 235.00 KiB/s, done.
From /tmp/Depot/my_website
a5f8133..c4e45f7  main      -> origin/main
Updating a5f8133..c4e45f7
Fast-forward
 index.html | 2 ++
1 file changed, 2 insertions(+)
```

В полной форме команда `git pull` позволяет указывать и репозиторий, и несколько спецификаций ссылок: `git pull options repository refspecs`.

Если не указать репозиторий в командной строке в виде Git URL или косвенно через имя удаленного подключения, то по умолчанию используется подключение `origin`. Если вы не укажете `refspec`, то Git возьмет спецификацию для `fetch`. Если вы укажете репозиторий (напрямую или через удаленное подключение), но не пропишете `refspec`, Git извлечет `HEAD` по удаленному подключению.

Операция `git pull` состоит из двух основных шагов, каждый из которых реализуется отдельной командой Git. То есть `git pull` подразумевает `git fetch`, сопровождаемую либо `git merge`, либо `git rebase`. По умолчанию вторым шагом идет `merge`, так как почти всегда требуется именно эта операция.

Поскольку `pull` также выполняет второй шаг `merge` или `rebase`, `git push` и `git pull` не считаются противоположными. Таковыми считаются `git push` и `git fetch`. И `push`, и `fetch` отвечают за передачу данных между репозиториями, но в противоположных направлениях.

Бывает, что вы решаете выполнить `git fetch` и `git merge` как две отдельные операции (например, чтобы сначала получить обновления в свой репозиторий и просмотреть их без слияния). В этом случае можно просто выполнить для ветки удаленного отслеживания `fetch`, а затем другие операции: `git log`, `git diff` или даже `gitk`. Затем, когда понадобится, вы сможете выполнить и слияние.

Даже если вы никогда не разделяете получение и слияние, вы можете столкнуться со сложными операциями, где необходимо понимать, что происходит на каждом шаге. Поэтому разберем эти шаги подробнее.

Шаг fetch

Во время этапа `fetch` Git обнаруживает удаленный репозиторий. Поскольку в этой команде не указывается непосредственный URL репозитория или прямое имя удаленного подключения, система предполагает стандартное имя `origin`. Информация для этого подключения — в файле конфигурации:

```
[remote "origin"]
  url = /tmp/Depot/my_website.git
  fetch = +refs/heads/*:refs/remotes/origin/*
```

Теперь Git знает, что в качестве исходного репозитория нужно использовать тот, который расположен по URL-адресу `/tmp/Depot/my_website`. Поскольку в командной строке не указали `refspec`, Git использует из записи `remote` все строки `fetch =`. В результате из удаленного репозитория будет получена каждая ветка `refs/heads/*`.

Затем Git выполняет протокол согласования с исходным репозиторием, чтобы определить, какие новые коммиты есть в удаленном репозитории и отсутствуют в вашем. Система опирается на запрос о получении всех ссылок `refs/heads/*`, согласно `refspec fetch`.

Вывод команды `pull`, дополненной префиксом `remote:`, отражает этот протокол согласования, сжатия и передачи. Он показывает, что в ваш репозиторий поступают новые коммиты:

```
remote: Counting objects: 5, done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0)
```

Git помещает новые коммиты в ваш репозиторий на соответствующую ветку удаленного отслеживания и сообщает, какое сопоставление используется для определения того, к чему эти коммиты относятся:

```
From /tmp/Depot/my_website
a5f8133...c4e45f7  main    -> origin/main
```

Эти строки показывают, что Git заглянул в удаленный репозиторий `/tmp/Depot/my_website`, взял *оттуда* ветку `main`, перенес ее содержимое в ваш репозиторий и поместил в *вашу* ветку `origin/main`. Этот процесс лежит в основе *отслеживания* веток. Здесь также представлены ID соответствующих коммитов на случай, если вы захотите изучить изменения. На этом этап `fetch` завершен.

В версии Git 2.29 появилась поддержка отрицательных спецификаций ссылок. Это означает, что вы можете явно исключить нежелательные спецификации, что учитывается во время операции `push` или `fetch`. Чтобы исключить `refspec`, нужно добавить к ней префикс `^`. Однако это сработает только для стороны `source`: `(+[source]: [destination])`.

Вот пример:

```
# Fetch с отрицательной refspec

$ git fetch newdev refs/heads/*:refs/remotes/newdev/* ^refs/heads/skip-me
...
...
```

Этот код приведет к локальному получению всех веток из `remotes/origin`, исключая ветку с именем `skip-me`.

В качестве альтернативы можно отредактировать файл конфигурации, добавив отрицательную `refspec`:

```
[remote "newdev"]
  url = /tmp/Depot/my_website.git
  fetch = +refs/heads/dev:refs/remotes/origin/dev
  fetch = +refs/heads/stable:refs/remotes/origin/stable
  fetch = ^refs/heads/skip-me
```



Не обязательно запрашивать все ветки разработки из удаленного репозитория, используя форму с подстановкой `refs/heads/*`. Если нужна лишь одна или две конкретные ветки, укажите их явно:

```
[remote "newdev"]
  url = /tmp/Depot/my_website.git
  fetch = +refs/heads/dev:refs/remotes/origin/dev
  fetch = +refs/heads/stable:refs/remotes/origin/stable
```

Шаг merge или rebase

На втором шаге операции `pull` Git выполняет `merge` (по умолчанию) или `rebase`. В нашем примере Git вливает содержимое ветки удаленного отслеживания, `origin/main`, в ветку локального отслеживания, `main`, используя специальный тип слияния — *слияние с перемоткой* (`fast-forward`).

Но откуда Git понимает, что нужно влить именно эти ветки? Ответ заключается в файле конфигурации:

```
[branch "main"]
  remote = origin
  merge = refs/heads/main
```

Это можно сформулировать в виде такого сообщения для Git:

Когда `main` является текущей, активной веткой, использовать `origin` как удаленный репозиторий по умолчанию, с которого нужно получать обновления во время `fetch` (или `pull`). Затем в процессе шага `merge` операции `git pull` использовать `refs/heads/main` из удаленного репозитория в качестве ветки по умолчанию для слияния с локальной веткой `main`.

Первая часть этого объяснения — фактический механизм, посредством которого Git определяет, что во время выполнения команды `git pull` без параметров нужно использовать `origin` в качестве удаленного репозитория.

Значение поля `merge` в разделе `branch` файла конфигурации (`refs/heads/main`) рассматривается как удаленная часть `refs`спес. Оно должно соответствовать одной из `source` ссылок, только что полученных в результате выполнения `git pull`. Звучит немного запутанно, но представьте это как подсказку, которая передается из шага `fetch` в шаг `merge` команды `pull`.

Значение конфигурации `merge` применяется только во время `git pull`, поэтому если в этот момент вы вручную выполните `git merge`, то нужно указать в командной строке имя исходной ветки для слияния. Скорее всего, это будет имя ветки удаленного отслеживания:

Или в полном виде: `refs/remotes/origin/main`

```
$ git merge origin/main
Updating a5f8133..c4e45f7
```

```
Fast forward
index.html |    2 ++
1 files changed, 2 insertions(+), 0 deletions(-)
```



Поведение веток при слиянии, когда в команде указывается несколько refspec, семантически отличается от ситуации, когда они находятся в удаленной записи. В первом случае выполняется слияние по типу «осьминог», когда все ветки одновременно вливаются в ходе n-сторонней операции. Во втором же случае такого не происходит. Рекомендую внимательно почитать страницы руководства по `git pull`.

Если вы решите выполнить перебазирувание, а не слияние, Git перенесет изменения из ветки локального отслеживания в только что полученную HEAD соответствующей ветки удаленного отслеживания. Эта операция равнозначна показанной на рис. 9.16 и 9.17 в главе 9.

Команда `git pull --rebase` заставит Git перебазировать (а не вливать) вашу ветку локального отслеживания на ветку удаленного отслеживания во время только этой операции `pull`. Чтобы сделать `rebase` стандартной операцией для ветки, установите переменную конфигурации `branch.branch_name.rebase` на `true`:

```
[branch "mydev"]
  remote = origin
  merge = refs/heads/main
  rebase = true
```

На этом шаг `merge` (или `rebase`) можно считать завершенным.

Что лучше: merge или rebase?

Что лучше — вливать или перебазировать изменения во время операции `pull`? Если кратко: действуйте на свое усмотрение. Но что может подтолкнуть вас выбрать тот или иной вариант? Рассмотрим некоторые моменты, которые нужно учесть.

Используя слияние, вы потенциально будете создавать дополнительный коммит слияния при каждой `pull` для регистрации изменений, которые одновременно есть в каждой ветке. В некотором смысле это истинное отражение двух путей разработки, которые произошли независимо и затем были объединены. Во время слияния придется разрешать конфликты. Каждая последовательность коммитов в каждой ветке будет базироваться на том коммите, в котором была изначально написана. При передаче в удаленный репозиторий любые коммиты слияния остаются. Некоторые разработчики считают такие слияния избыточными и предпочитают, чтобы те не захламляли историю. Другие же стремятся их сохранять, поскольку считают, что они более точно отражают историю разработки.

Перебазирование фундаментально изменяет запись того, когда и где последовательность коммитов была создана, поэтому некоторые аспекты истории

разработки в результате утрачиваются. В частности, исходный коммит, на котором изначально базировалась вся разработка, будет изменен на вновь подставленную HEAD ветки удаленного отслеживания. В результате все будет выглядеть так, будто разработка произошла позднее (в последовательности коммитов), чем есть на самом деле. Так все происходит немного иначе и проще, чем в случае слияния истории.

При перебазирувании вам также придется разрешать возможные конфликты. Поскольку перебазируемые изменения все равно остаются строго локальными для вашего репозитория и еще не опубликованы, то и бояться изменять историю не стоит.

И при слиянии, и при перебазирувании нужно учесть, что новое, итоговое содержимое будет отличаться от того, которое независимо существовало в задействованных ветках разработки. И в новом виде оно может потребовать проверки: возможно, компиляции и тестирования перед отправкой в вышестоящий репозиторий.

Мы, авторы этой книги, стремимся к созданию более простых, линейных историй. Во время личной разработки нас обычно не особо беспокоит небольшая перестановка наших изменений по отношению к изменениям коллег, поступивших в результате получения ветки удаленного отслеживания, поэтому нам нравится использовать перебазирование.

Если вы действительно решите определить для себя один последовательный подход, то подумайте об установке опции конфигурации `branch.autosetupmerge` или `branch.autosetuprebase` на `true`, `false` или `always`. Есть и другие опции для коррекции поведения между чисто локальными ветками, а не только между локальной и удаленной.

Блок III: цикл разработки в удаленном репозитории в картинках

Интеграция локальной разработки с изменениями из вышестоящего репозитория лежит в основе цикла распределенной разработки в Git. Давайте визуализируем, что происходит с локальным репозиторием и вышестоящим origin репозиторием во время операций клонирования (`clone`) и скачивания (`pull`). В этом разделе также приведены иллюстрации, которые объясняют вызывающие путаницу случаи, когда одно и то же имя используется в разных контекстах.

В качестве основы возьмем простой репозиторий (рис. 11.2).

Как и в любом аналогичном графе, коммиты идут слева направо, и метка `main` указывает на HEAD ветки. Два последних коммита обозначены как A и B. Далее мы проследим за этими коммитами, внесем другие и посмотрим, что произойдет.

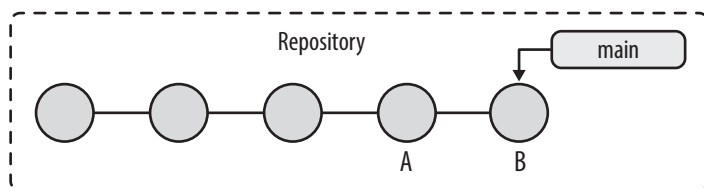


Рис. 11.2. Простой репозиторий с коммитами

Клонирование репозитория

В результате команды `git clone` получаем два отдельных репозитория (рис. 11.3).

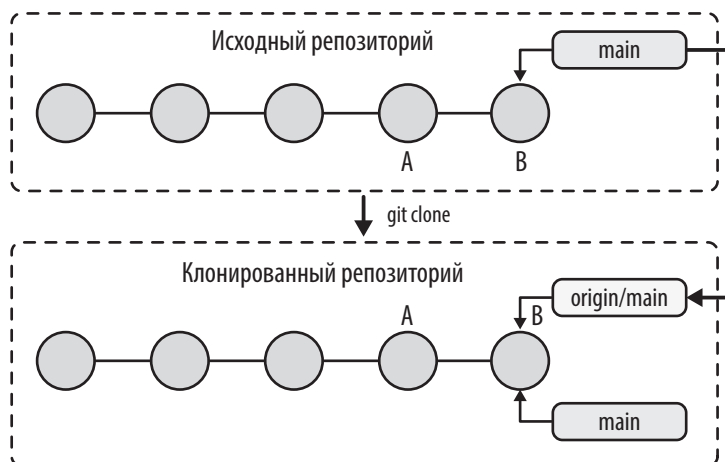


Рис. 11.3. Клонированный репозиторий

На рис. 11.3 отражены некоторые важные результаты клонирования:

- Все коммиты исходного репозитория были скопированы в его клон. Теперь можно легко извлечь более ранние этапы проекта из собственного репозитория.
- Ветка с именем `main` из исходного репозитория была внесена в созданную на клоне ветку удаленного отслеживания с именем `origin/main`.
- Внутри клонированного репозитория новая ветка `origin/main` инициализируется как указывающая на HEAD-коммит `main`, который на рисунке представлен как `B`.
- В клоне была создана ветка локального отслеживания `main`.

- Ветка `main` инициализирована как указывающая на `origin/HEAD`, `HEAD` активной ветки исходного репозитория. Ей оказывается `origin/main`, значит, она также указывает на тот же коммит, `B`.

После клонирования Git выбирает новую ветку `main` в качестве текущей и переключается на нее. Пока вы не измените ветки, все изменения, которые вносятся после `clone`, будут влиять на ветку `main`.

На всех рисунках ветки разработки в исходном репозитории и в его клоне выделены темно-серой заливкой, а ветки удаленного отслеживания — светло-серой. И ветки локального отслеживания, и ветки удаленного отслеживания — закрытые и локальные для соответствующих репозитория. При этом на языке реализации Git темно-серые метки веток относятся к пространству имен `refs/heads/`, в то время как светлые — к пространству имен `refs/remotes/`.

Альтернативные истории

После того как вы клонировали репозиторий, появляются два отдельных пути разработки. Теперь вы можете вести разработку в своем репозитории и выполнять коммиты в свою ветку `main`, как показано на рис. 11.4. На этом рисунке ваша разработка расширяет ветку `main` двумя новыми коммитами, `X` и `Y`, опирающимися на `B`.

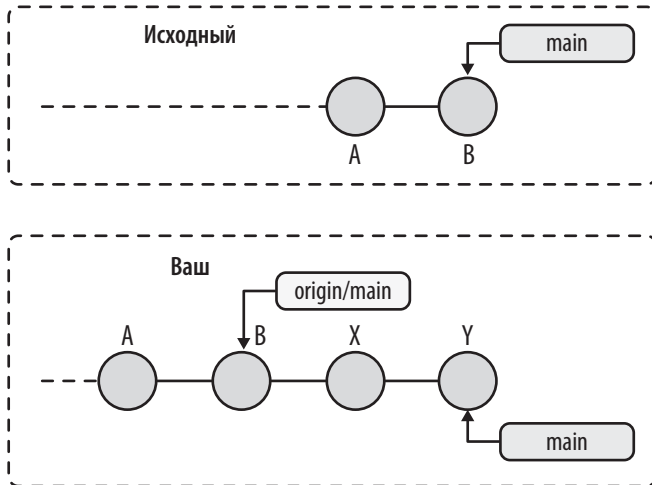


Рис. 11.4. Коммиты в вашем репозитории

Тем временем любой разработчик, имеющий доступ к исходному репозиторию, мог проделать свои шаги по разработке и передать изменения в этот репозиторий. Эти изменения представлены на рис. 11.5 в виде новых коммитов `C` и `D`.

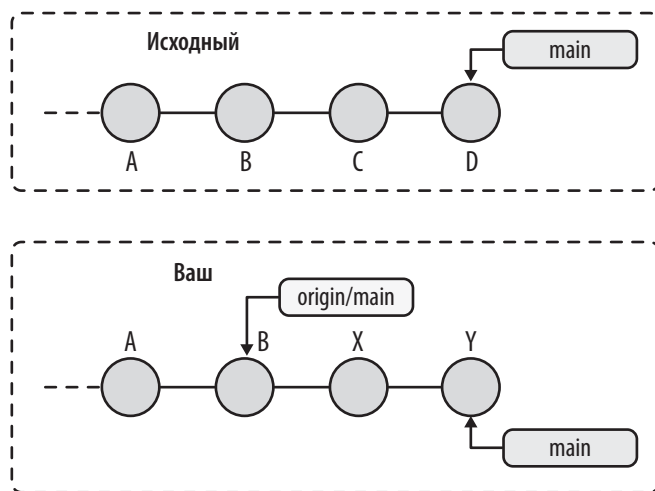


Рис. 11.5. Коммиты в исходном репозитории

В этой ситуации мы говорим, что истории репозитория *разошлись* (diverged) у коммита В. По аналогии с тем, как локальное ветвление внутри одного репозитория ведет к появлению альтернативных историй, расходящихся от некоего коммита, репозиторий и его клон могут расходиться по двум историям в результате отдельных действий, совершенных, возможно, разными людьми. Это абсолютно нормально, и все эти истории корректны.

Вся суть операции слияния в том, чтобы совместить отдельные истории. Посмотрим, как это реализует Git.

Передача изменений без перемотки

Если вы ведете разработку в модели репозитория, где есть возможность отправить изменения в репозиторий `origin`, то можно попробовать передать их в любой момент. При этом могут возникнуть проблемы, если другой разработчик ранее делал коммиты.

Такая опасность часто возникает, когда вы используете модель разработки в общем репозитории, где все разработчики могут отправлять свои коммиты и обновления в любое время.

Посмотрите еще раз на рис. 11.4, где вы внесли новые коммиты, X и Y, основанные на B.

Если бы в данный момент нужно было передать эти коммиты в вышестоящий репозиторий, то никаких проблем бы не возникло. Git перенес бы их в репозиторий `origin` и добавил в историю к коммиту B. После этого Git выполнил бы слияние с перемоткой для ветки `main`, поместив в нее правки и обновив ссылку,

чтобы та указывала на Y. Перемотка представляет простую операцию линейного продвижения истории. О ней шла речь в разделе «Вырожденные слияния» на с. 164 в главе 6.

Но допустим, другой разработчик уже отправил коммиты в репозиторий `origin`, и когда вы попытались передать в него *свою* историю, то получили такой результат, как на рис. 11.5. По сути, вы пытаетесь отправить историю в общий репозиторий, когда в нем уже есть другая история. История `origin` не просто перематывается (fast-forward) от B. Эта ситуация называется *проблемой передачи без перемотки* (non-fast-forward push problem).

Когда вы пытаетесь отправить свои изменения, Git отклоняет их, сообщая о конфликте:

```
$ git push
To /tmp/Depot/my_website
! [rejected]          main -> main (non-fast forward)
error: failed to push some refs to '/tmp/Depot/my_website'
```

Что происходит на самом деле? Вам нужно перезаписать (затереть) изменения другого разработчика или же вставить в репозиторий обе истории?



Если вы хотите перезаписать другие изменения, то такая возможность есть. Сопроводите команду `git push` опцией `-f`. Надеемся, что потом вам не понадобится *та* альтернативная история.

Чаше бывает так, что нужно не затереть существующую историю `origin`, а только внести свои изменения. В таком случае перед тем, как это сделать, следует выполнить слияние двух историй.

Получение альтернативной истории

Чтобы Git выполнил слияние двух альтернативных историй, они обе должны находиться в одном репозитории на двух разных ветках. Чисто локальные ветки разработки относятся к особому (вырожденному) случаю в том смысле, что уже находятся в одном репозитории.

Если альтернативные истории оказались в разных репозиториях из-за клонирования, тогда удаленную ветку нужно перенести в ваш репозиторий операцией `fetch`. Сделать это можно как самой командой `git fetch`, так командой `git fetch` вместе с `git pull`: оба варианта равноценны. В любом случае операция получения добавит коммиты удаленного репозитория (в нашем случае C и D) в ваш репозиторий. Результаты показаны на рис. 11.6.

Внесение альтернативной истории с коммитами C и D никак не меняет историю, представленную коммитами X и Y. Обе эти истории теперь одновременно

существуют в вашем репозитории и формируют более сложный граф. Ваша история представлена веткой `main`, а удаленная — веткой удаленного отслеживания `origin/main`.

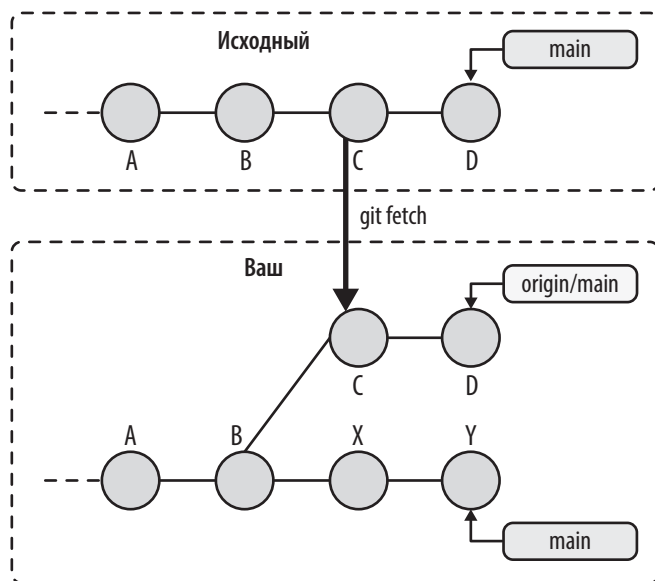


Рис. 11.6. Получение альтернативной истории

Слияние историй

Теперь, когда обе истории находятся в одном репозитории, для их объединения достаточно влить ветку `origin/main` в ветку `main`.

Операцию слияния можно инициировать либо прямой командой `git merge origin/main`, либо в качестве второго шага запроса `git pull`. Техники для операции слияния в точности такие же, как и описанные в главе 6.

На рис. 11.7 показан граф коммитов в вашем репозитории после успешного объединения двух историй коммитов D и Y в новый коммит слияния, M. Ссылка на `origin/main` по-прежнему указывает на D, так как не менялась, а `main` была обновлена коммитом слияния M, указывая, что слияние происходило именно с ней.

Конфликты слияния

Временами между альтернативными историями при слиянии будут возникать конфликты. Вне зависимости от результата такого слияния операция `fetch` все равно выполняется. Все коммиты из удаленного репозитория остаются в вашем репозитории на ветке отслеживания.

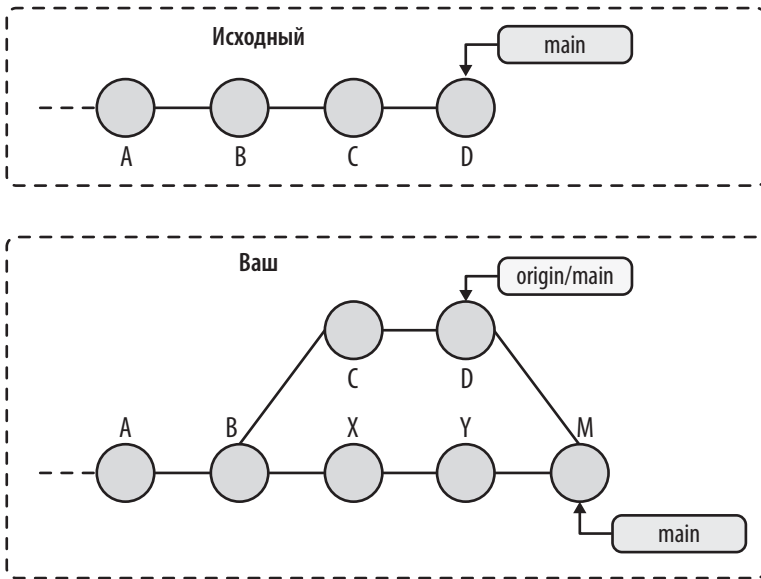


Рис. 11.7. Слияние историй

Вы можете разрешить такое слияние стандартным путем, который описывался в главе 6. Также можно отменить эту операцию и сбросить `main` к ее прежнему состоянию `ORIG_HEAD` с помощью команды `git reset --hard ORIG_HEAD`. В нашем примере это приведет к смещению `main` на *предыдущее* значение `HEAD`, Y, и соответствующему изменению рабочего каталога. Кроме того, ветка `origin/main` останется у коммита D.



Чтобы освежить в памяти значение `ORIG_HEAD`, загляните в раздел «Ссылки и символические ссылки» на с. 104 в главе 4. Кроме того, ее использование описано в разделе «Отмена или перезапуск слияния» на с. 161 в главе 6.

Передача объединенной истории

Если вы выполнили все описанные шаги, то репозиторий будет обновлен последними изменениями из репозитория `origin` вашего репозитория. Обратное при этом неверно: репозиторий `origin` по-прежнему не содержит ваших изменений.

Если ваша задача в том, чтобы просто внести последние обновления из `origin` в свой репозиторий, то после разрешения слияния она будет решена. При этом команда `git push` может вернуть унифицированную и объединенную историю из вашей ветки `main` обратно в репозиторий `origin`. На рис. 11.8 показаны результаты `git push`.

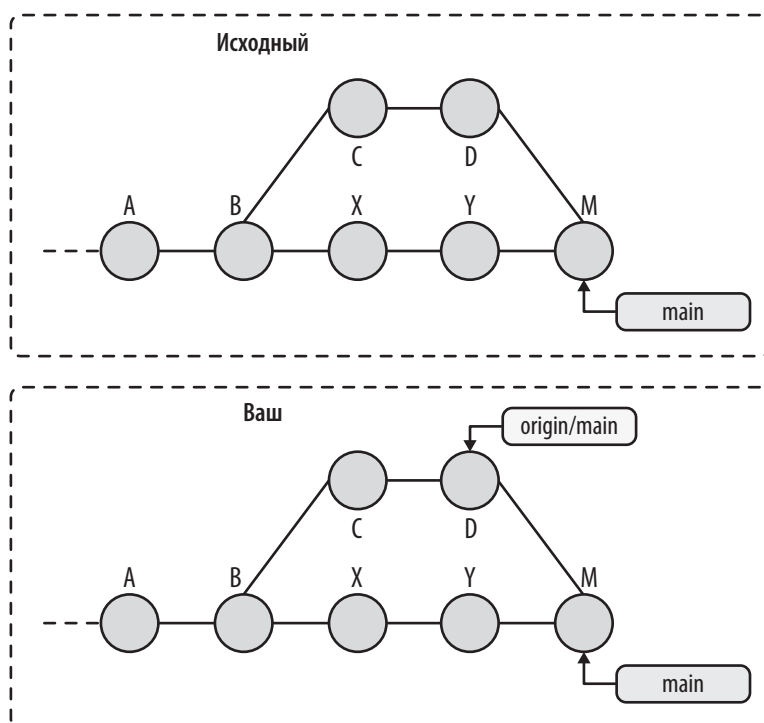


Рис. 11.8. Объединенные истории после push

Обратите внимание, что репозиторий `origin` был обновлен вашей разработкой, даже если в него были внесены другие изменения, которые должны были быть объединены в первую очередь. Ваш репозиторий и репозиторий `origin` полностью обновлены и снова синхронизированы.

Блок IV: конфигурация удаленного репозитория

Отслеживать вручную всю информацию о ссылке на удаленный репозиторий утомительно и непросто: придется запоминать полный URL этого репозитория и каждый раз, когда вы захотите получить обновления, печатать в командной строке удаленные ссылки и `refspec`; воссоздавать отображения веток и т. д. Кроме того, повторный ввод этой информации может привести к ошибкам.

Вас также может заинтересовать, как Git запоминает URL для удаленного репозитория из начального клона, чтобы использовать его в последующих операциях `fetch/push` с участием `origin`.

Git предоставляет три механизма для настройки и управления информацией об удаленных подключениях: команда `git remote`, команда `git config` и непосредственно редактирование файла `.git/config`. Все эти механизмы приводят к записи информации о конфигурации в файл `.git/config`.

Использование `git remote`

Команда `git remote` предоставляет более специализированный интерфейс для работы с удаленными подключениями, который позволяет управлять данными файла конфигурации и ссылками удаленных подключений. У нее есть несколько подкоманд с интуитивно понятными именами. Для вывода этих подкоманд выполните `git remote --help`:

```
$ git remote --help
```

```
NAME
```

```
git-remote - Manage set of tracked repositories
```

```
SYNOPSIS
```

```
git remote [-v | --verbose]
git remote add [-t <branch>] [-m <main>] [-f] [--[no-]tags]
    [--mirror=(fetch|push)] <name> <url>
git remote rename <old> <new>
git remote remove <name>
git remote set-head <name> (-a | --auto | -d | --delete | <branch>)
git remote set-branches [--add] <name> <branch>...
git remote get-url [--push] [--all] <name>
git remote set-url [--push] <name> <newurl> [<oldurl>]
git remote set-url --add [--push] <name> <newurl>
git remote set-url --delete [--push] <name> <url>
git remote [-v | --verbose] show [-n] <name>...
git remote prune [-n | --dry-run] <name>...
git remote [-v | --verbose] update [-p | --prune] [(<group> | <remote>)...]
```

О командах `git remote add` и `update` мы говорили в подразделе «Создание собственного удаленного подключения `origin`», а о команде `git remote show` — в подразделе «Добавление нового разработчика». С помощью `git remote add origin` мы добавляли новое удаленное подключение `origin` в свежий родительский репозиторий в контейнере, а `git remote show origin` выполняли для извлечения всей информации об удаленном подключении `origin`. Наконец, `git remote update` использовали для получения в локальный репозиторий всех обновлений, доступных в удаленном репозитории.

Команда `git remote rm` удаляет из локального репозитория указанное удаленное подключение и все связанные с ним ветки удаленного отслеживания. Для удаления одной такой ветки можно использовать следующую команду:

```
$ git branch -r -d origin/dev
```

Но так делать не рекомендуется, если только соответствующая удаленная ветка не была действительно удалена из вышестоящего репозитория. В противном случае следующий запрос на получение данных из удаленного репозитория, скорее всего, воссоздаст эту ветку.

Удаленный репозиторий может содержать ветки, удаленные из него другими разработчиками, хотя копии этих веток могут сохраняться в репозитории. Для удаления из вашего репозитория устаревших (относительно фактического удаленного репозитория) веток удаленного отслеживания используйте команду `git remote prune`.

Для поддержания еще большей синхронизации с удаленным репозиторием используйте команду `git remote update --prune remote`, чтобы за один шаг получить обновления из удаленного репозитория, а затем исключить устаревшие ветки отслеживания.

Чтобы переименовать удаленное подключение и все его ссылки, используйте `git remote rename old new`:

```
$ git remote rename jon jdl
```

После этой команды ссылка вроде `jon/bugfixes` будет называться `jdl/bugfixes`.

Помимо управления именем удаленного подключения и его ссылками, вы также можете обновлять URL этого подключения:

```
$ git remote set-url origin git://repos.example.com/stuff.git
```

Использование `git config`

Команду `git config` можно использовать для непосредственного управления записями в файле конфигурации. Сюда относятся несколько переменных для удаленных подключений.

Например, чтобы добавить подключение с именем `publish`, используя `push` рефсписок для всех веток, которые вы хотите опубликовать, введите такую команду:

```
$ git config remote.publish.url 'ssh://git.example.org/pub/repo.git'
$ git config remote.publish.push '+refs/heads/*:refs/heads/*'
```

Каждая из этих команд добавляет в файл `.git/config` строку. Если в удаленном подключении еще нет раздела `publish`, то первая команда создает его. В результате ваш `.git/config` содержит, помимо прочего, следующее определение удаленного подключения:

```
[remote "publish"]
  url = ssh://git.example.org/pub/repo.git
  push = +refs/heads/*:refs/heads/*
```




Для вывода всего содержимого файла конфигурации с полными именами переменных используйте команду `git config` с опцией `-l` (l в нижнем регистре):

```
# Из клона исходников git.git

$ git config -l
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
remote.origin.url=git://git.kernel.org/pub/scm/git/git.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
branch.main.remote=origin
branch.main.merge=refs/heads/main
```

Ручное редактирование

Иногда вместо использования `git remote` или `git config` проще или быстрее отредактировать файл в текстовом редакторе. В этом нет ничего плохого, но такой подход чреват ошибками и обычно делается только теми разработчиками, которые хорошо знакомы с поведением Git и конфигурационным файлом. Если вы ясно понимаете, какие части конфигурационного файла Git влияют на различные его функции и какие изменения в этом файле вызваны определенными командами, то вам, вероятно, не составит труда отредактировать этот файл.

НЕСКОЛЬКО УДАЛЕННЫХ РЕПОЗИТОРИЕВ

Операции `git remote add remote repository-URL` можно выполнять неоднократно для добавления в репозиторий нескольких удаленных подключений. В такой ситуации вы сможете затем получать коммиты из нескольких источников, совмещая их в своем репозитории. Эта возможность также позволяет установить несколько целей для отправки, которые могут получать часть вашего репозитория или его целиком.

В главе 12 вы увидите, как в процессе разработки использовать несколько репозиториях в разных сценариях.

Блок V: работа с ветками отслеживания

Создание веток отслеживания и управление ими — это существенная часть разработки в Git, поэтому важно понять, как и почему Git создает различные ветки отслеживания и как предполагает их использовать.

Создание веток отслеживания

Точно так же, как ветка `main` может рассматриваться как продолжение разработки, начатой в ветке `origin/main`, вы можете создать новую ветку на основе любой ветки удаленного отслеживания и использовать ее для продолжения этой линии разработки.

Мы уже видели, что ветки удаленного отслеживания вносятся во время операции клонирования или когда в репозиторий добавляются удаленные подключения. Git предоставляет очень простой способ создания пары из ветки локального и ветки удаленного отслеживания, используя для них согласованное ссылочное имя. Простой запрос на переключение с использованием имени ветки удаленного отслеживания приведет к созданию ветки локального отслеживания и связыванию этих веток. Тем не менее Git делает это, только если имя ветки соответствует единственному имени удаленной ветки среди всех удаленных подключений репозитория. При этом подразумевается полное имя ветки, которое указывается в `refspec` после имени удаленного подключения.

Далее мы используем репозиторий исходного кода Git для нескольких примеров. Получив данные с GitHub и `git.kernel.org`, мы создадим репозиторий с коллекцией имен веток из двух удаленных подключений, большая часть которых будет повторами:

```
# Получение репозитория Git с исходным кодом
$ git clone https://github.com/git/git.git
Cloning into 'git'...
...

$ cd git

# Добавление второго удаленного подключения для того же репозитория, но с другого сервера
$ git remote add korg git://git.kernel.org/pub/scm/git/git.git

$ git remote update
Fetching origin
remote: Enumerating objects: 931, done.
remote: Counting objects: 100% (759/759), done.
remote: Compressing objects: 100% (130/130), done.
remote: Total 931 (delta 650), reused 726 (delta 629), pack-reused 172
Receiving objects: 100% (931/931), 918.29 KiB | 6.47 MiB/s, done.
Resolving deltas: 100% (676/676), completed with 142 local objects.
From https://github.com/git/git
...
...
Fetching korg
From git://git.kernel.org/pub/scm/git/git
* [new branch]      main    -> korg/main
* [new branch]      maint   -> korg/maint
* [new branch]      master  -> korg/master
* [new branch]      next    -> korg/next
```

```
* [new branch]          seen    -> korg/seen
* [new branch]          todo    -> korg/todo
```

Предположим, что есть гипотетическая ветка, присутствующая только в удаленном подключении `korg`. Рассмотрим, как можно работать с такой веткой. Если вы повторяете примеры из книги, то приведенный ниже фрагмент кода работать не будет, поскольку для вас этой ветки не существует:

```
# Поиск ветки с уникальным именем и переключение на нее.
$ git branch -a | grep split-blob
  remotes/origin/jc/split-blob

$ git branch
* main

$ git checkout jc/split-blob
branch 'jc/split-blob' set up to track 'jc/split-blob'
Switched to a new branch 'jc/split-blob'

$ git branch
* jc/split-blob
  main
```

Заметьте, что пришлось использовать полное имя ветки `jc/split-blob`, а не просто `split-blob`.

В случаях, когда имя ветки неоднозначно, можно установить ее напрямую:

```
$ git branch -a | egrep 'maint$'
  remotes/korg/maint
  remotes/origin/maint

$ git checkout maint
hint: If you meant to check out a remote tracking branch on, e.g. 'origin',
...
...
...
fatal: 'maint' matched multiple (2) remote tracking branches

# Выберите одну из веток maint.
$ git checkout --track korg/maint
branch 'maint' set up to track 'korg/maint'.
Switched to a new branch 'maint'
```

Высока вероятность, что эти две ветки представляют один коммит, находящийся в двух разных репозиториях, и вы можете просто выбрать одну, на которой будет базироваться ваша ветка локального отслеживания.

Если по какой-то причине вы хотите использовать другое имя, то сделайте следующее:

```
$ git branch --track mynext korg/next
branch 'mynext' set up to track 'korg/next'.
```

Git автоматически добавляет в файл `.git/config` запись `branch`, указывая, что ветку удаленного отслеживания нужно влить в новую ветку локального отслеживания. Все изменения, вызванные предыдущей последовательностью команд, дадут следующий файл конфигурации:

```
$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
    ignorecase = true
    precomposeunicode = true
[remote "origin"]
    url = https://github.com/git/git.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = origin
    merge = refs/heads/main
[remote "korg"]
    url = git://git.kernel.org/pub/scm/git/git.git
    fetch = +refs/heads/*:refs/remotes/korg/*
[branch "jc/split-blob"]
    remote = origin
    merge = refs/heads/jc/split-blob
[branch "maint"]
    remote = korg
    merge = refs/heads/maint
[branch "mynext"]
    remote = korg
    merge = refs/heads/next
```

Как обычно, управлять записями `branch` в файле конфигурации вы можете с помощью `git config` или в текстовом редакторе.



Если вы запутаетесь в ветках отслеживания, используйте команду `git remote show remote`, которая поможет упорядочить все удаленные подключения и ветки.

Сейчас должно быть понятно, что, когда вы клонируете репозиторий, по умолчанию создается локальная ветка `main`, которая отслеживает удаленную ветку `origin/main`. Это делается для удобства и упрощения работы с репозиторием, как если бы вы явно переключились на главную ветку самостоятельно после клонирования.

Переключение на ветку удаленного отслеживания в более ранних версиях Git (ориентировочно до 1.6.6) приводило к отделению `HEAD`. И этот факт дополнительно подтверждает, что внесение коммитов непосредственно в ветку удаленного отслеживания оказывается неудачным вариантом.

В разделе «Отсоединенная HEAD» (с. 94, глава 3) шла речь о том, что отсоединенная HEAD — это, по сути, название анонимной ветки. В нее можно вносить коммиты, но после этого не следует обновлять HEAD ветки удаленного отслеживания какими-либо локальными коммитами, чтобы потом не мучиться при получении обновлений из этого удаленного репозитория. (Если вам понадобится разместить подобные коммиты в отдельной HEAD, используйте `git checkout -b my_branch` для создания новой локальной ветки, в которой будете затем работать над изменениями.) Хотя в целом это не самый удачный интуитивный подход.

Если у вас уже есть ветка разработки, которую вы хотите связать с веткой удаленного отслеживания вышестоящего репозитория, вы можете установить эту связь с помощью опции `--set-upstream-to`. Как правило, это делается после добавления нового удаленного подключения:

```
# Из корневого каталога существующего репозитория выполните
$ git remote add upstreamrepo git://git.example.org/upstreamrepo.git

# Существующая ветка с другим удаленным подключением: origin/dev.
# Не трогайте ее, а просто свяжите с upstreamrepo/dev.
$ git branch --set-upstream-to upstreamrepo/dev
branch dev set up to track 'upstreamrepo/dev'.
```

Опережение и отставание

После связывания веток локального и удаленного отслеживания можно провести сравнение между ними. Помимо стандартных `diff`, `log` и прочих видов сравнения, основанных на содержимом, Git может выводить сводку о числе коммитов в каждой ветке с указанием того, какая из этих веток «опережает» другую или «отстает» от нее.

Если при локальной разработке вы внесли коммиты в ветку локального отслеживания, то она будет считаться опережающей соответствующую ветку удаленного отслеживания. И наоборот, если вы получаете в ветки удаленного отслеживания новые коммиты, которых нет в ветке локального отслеживания, Git считает ее отстающей от связанной с ней ветки удаленного отслеживания.

Команда `git status` обычно выводит следующий статус:

```
$ git fetch
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 660 bytes | 220.00 KiB/s, done.
From https://github.com/ppremk/somerepo
   78625f3..cf1abb0  main      -> origin/main
```

```
$ git status
```

```
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)
```

```
nothing to commit, working tree clean
```

Чтобы увидеть, какие коммиты есть в `main`, но отсутствуют в `origin/main`, используйте такую команду:

```
$ git log main..origin/main
commit cf1abb04818c5ee5304c3cdece481442596e34da (origin/main, origin/HEAD)
Author: Prem Kumar Ponuthorai <ppremk@gmail.com>
Date: Thu Jul 14 20:40:17 2022 +0200
```

```
Update README.md
```

Код ниже покажет, как ветка может одновременно быть и отстающей, и опережающей.

```
# Make one local commit on top of previous example
$ git commit -m "Something" README.md
[main 7d28dc3] Something
1 file changed, 2 insertions(+), 1 deletion(-)
```

```
$ git status
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
(use "git pull" to merge the remote branch into yours)
```

```
nothing to commit, working tree clean
```

В этом случае, чтобы увидеть изменения, потребуется использовать симметрическую разность:

```
$ git log origin/main...main
commit 7d28dc31ca6be516a648e434fcfc04a4b14dc1b0 (HEAD -> main)
Author: Prem Kumar Ponuthorai <ppremk@gmail.com>
Date: Thu Jul 14 20:50:01 2022 +0200
```

```
Something
```

Добавление и удаление удаленных веток

Любая новая разработка, которую вы ведете в ветках локального клона, невидима в родительском репозитории, пока вы не сделаете соответствующий запрос на ее отправку. Аналогично удаление ветки в вашем репозитории останется локальным изменением и не вызовет ее удаления в родительском репозитории, пока вы не сделаете запрос.

В главе 3 вы узнали, как добавлять ветки в репозиторий и удалять из него существующие, используя команду `git branch`. Но `git branch` работает только для локального репозитория.

Чтобы выполнить аналогичные операции для удаленного репозитория, в команде `git push` нужно указывать иные формы refspec. Вспомните синтаксис refspec:

```
[+]source:destination
```

Операции отправки, в которых используется refspec только со ссылкой *source* (то есть без ссылки *destination*), создают в удаленном репозитории новую ветку.

```
$ cd ~/my_website
$ git checkout -b foo
Switched to a new branch "foo"
$ git push origin foo
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To /tmp/Depot/my_website
* [new branch]      foo -> foo
```

Такие передачи — это сокращенная форма использования одного имени для ссылок *source* и *destination*. Передачу, в которой указываются разные имена источника и цели, можно использовать для создания новой цели с именем *branch* или расширения существующего удаленного репозитория содержимым из локальной ветки. То есть `git push origin mystuff:dev` приведет к передаче локальной ветки *mystuff* в вышестоящий репозиторий и либо созданию, либо расширению ветки с именем *dev*.

Таким образом, из-за серии стандартных действий следующие команды дадут один и тот же результат:

```
$ git push upstream new_dev
$ git push upstream new_dev:new_dev
$ git push upstream new_dev:refs/heads/new_dev
```

Естественно, *upstream* будет ссылкой на соответствующий вышестоящий репозиторий, обычно *origin*.

Операции передачи, использующие refspec только со ссылкой *destination* (то есть без ссылки *source*), приведут к удалению ссылки *destination* из удаленного репозитория. Чтобы обозначить эту ссылку как *destination*, поставьте двоеточие:

```
$ git push origin :foo
To /tmp/Depot/my_website
- [deleted]      foo
```

Если форма *:branch* вам не нравится, можете использовать ее синтаксический эквивалент:

```
$ git push origin --delete foo
```

А как насчет переименования удаленной ветки? К сожалению, здесь нет простого решения. Можно создать новую вышестоящую ветку с новым именем и затем удалить старую. Это легко сделать с помощью команд `git push`, как показывалось ранее:

```
# Создание нового имени для существующего коммита old
$ git branch new origin/old
$ git push origin new

# Удаление старого имени
$ git push origin :old
```

Это легкая и очевидная часть задачи. Но каковы последствия (если вы работаете в распределенном режиме) для других разработчиков? Знаете ли вы, у кого еще есть клоны только что измененного вышестоящего репозитория? Если да, их владельцы могут просто выполнить `fetch` и `remote prune`, чтобы обновить свои репозитории. Если же нет, то у всех этих клонов появятся подвешенные ветки отслеживания. И не существует способа переименовать их в распределенном режиме.

В итоге смысл всего сказанного сводится к простой истине: «Будьте осторожны с переписыванием истории».

Голые репозитории и `git push`

Из-за одноранговой семантики репозитория в Git все они равнозначны. Вы можете отправлять изменения и получать их из репозитория разработки и голых репозитория, поскольку между ними нет фундаментальных отличий в реализации. Такой симметричный дизайн очень важен для Git, но он также ведет к неожиданному поведению, когда вы пытаетесь рассматривать голые репозитории и репозитории разработки как равные.

Напомню, что команда `git push` не переключается на файлы в получающем репозитории. Она передает объекты из исходного репозитория в получающий и затем обновляет соответствующие ссылки на стороне получения.

В голом репозитории это единственное поведение, которое можно ожидать, потому что в нем нет рабочего каталога, который можно обновить передаваемыми файлами. И это хорошо. Тем не менее это может вызвать замешательство у того, кто впоследствии будет использовать репозиторий разработки, выступающий целью передачи.

Операция `push` может обновить состояние репозитория, включая HEAD-коммит. Это означает, что даже если разработчик на удаленной стороне ничего не делал, ссылки веток и HEAD могут измениться, утратив синхронизацию с обновленными файлами и индексом.

Разработчик, который активно работает в репозитории, куда происходит асинхронная передача, не увидит последствий этой передачи. В результате последующий коммит от него произойдет в неожиданную HEAD, что создаст странную историю. Принудительная передача приведет к утрате переданных коммитов от другого разработчика. Причем у разработчика в этом репозитории также может не получиться согласовать свою историю с вышестоящим репозиторием или с нижестоящим клоном, поскольку эти истории больше не представляют простую перемотку, как это должно быть. И он не будет понимать, что произошло; а причина была в том, что репозиторий был незаметно изменен нижестоящей стороной.

Таким образом, отправлять изменения нужно только в голый репозиторий. Это не жесткое правило, но оно считается лучшей практикой и станет хорошим руководством для обычного разработчика. Есть несколько случаев, когда может потребоваться передать изменения в репозиторий разработки, но важно понимать последствия этого. Когда вы *действительно* хотите выполнить передачу в репозиторий разработки, вы можете выбрать один из двух подходов.

В первом сценарии должен быть рабочий каталог с обновленной веткой в получающем репозитории. Например, вы знаете, что никакой другой разработчик не будет вести в нем активную разработку, а значит, вы никого не введете в замешательство передачей «молчаливых» изменений в их репозитории.

В этом случае вы можете активировать в получающем репозитории хук, который будет делать активной некую ветку, возможно, только что переданную, также внося ее изменения в рабочий каталог. Чтобы подтвердить подходящее состояние получающего репозитория для выполнения автоматического переключения, хук должен убедиться, что при передаче рабочий каталог репозитория разработки не содержит правок или измененных файлов и что его индекс не имеет незафиксированного состояния. Если эти условия не выполняются, вы можете утратить свои правки или изменения, поскольку переключение приведет к их перезаписи.

Есть еще один сценарий, в котором передача в репозиторий разработки может сработать успешно. По общему соглашению любой разработчик, передающий изменения, должен отправлять их не в активную ветку, а в ветку, которая считается просто получающей. При этом то, когда и какая ветка должна быть активна, решает конкретный разработчик. Например, это тот, кто отвечает за обработку получающих веток и их слияние с основной перед переключением на нее.

Итоги

Удаленное подключение — это ссылка на другой репозиторий через файловую систему или сетевой путь. Вы используете его в качестве сокращенного имени для длинных и сложных Git URL. Вы можете определять в репозитории любое число удаленных подключений, создавая тем самым сети из общих репозиториев.

После установки удаленного подключения Git сможет передавать данные из одного репозитория в другой, используя модель `push` или `pull`. Например, типичный сценарий — периодическая передача данных коммита из исходного репозитория в его клон, чтобы поддержать их синхронность. Вы также можете создавать удаленные подключения для передачи данных из клона в его родитель или настройки их на двухсторонний обмен информацией.

Для отслеживания данных из других репозиториях Git использует ветки удаленного отслеживания. Каждая такая ветка в вашем репозитории служит посредником для конкретной ветки в удаленном репозитории. Вы можете настроить ветку локального отслеживания, которая будет формировать основу для интеграции ваших локальных изменений с удаленными из соответствующей ветки удаленного отслеживания.

В конце главы вы узнали техники, позволяющие сделать репозиторий доступным для других. В Git это называется «публикацией репозитория» (`publishing`), и сделать это можно несколькими способами.

Управление репозиторием

Из этой главы вы узнаете, как публиковать репозитории Git и какие существуют методы для управления репозиториями в случае совместной разработки. Тема достаточно дискуссионная, и мы рассмотрим различные подходы к управлению.

Начнем с объяснения настройки собственного сервера Git и предоставления доступа к опубликованным репозиториям через разные протоколы. Затем вы познакомитесь со способами структуризации репозитория для централизованного или распределенного доступа в соответствии с потребностями разработки. После этого узнаете некоторые правила, которые нужно учитывать при работе с репозиториями Git.

Затем мы расскажем о важности понимания возможных ролей разработчиков и двойной роли «разработчик-мейнтейнер». Обсудим техники, которые можно использовать, когда нужно работать с несколькими распределенными репозиториями. Закончим главу знакомством с принципом создания форка (fork) репозитория.

НЕМНОГО О СЕРВЕРАХ

Слово *сервер* используется вольно и часто означает различные вещи. И Git, и эта книга — не исключение. Поэтому проясним, чем может быть или не быть сервер, что он может делать и как Git его использует.

В принципе Git не нужен сервер. В отличие от других систем контроля версий, где часто требуется централизованный сервер, для размещения репозитория Git сервер необязателен.

Наличие сервера в контексте репозитория Git зачастую требует чуть больше, нежели установки удобного, фиксированного или известного расположения, откуда могут быть получены репозитории или где происходит обмен изменениями. Сервер Git также может предоставлять форму аутентификации или контроля доступа.

Git без проблем позволяет обмениваться файлами напрямую с репозиторием на той же машине или даже с другими машинами через различные протоколы без посредника в виде сервера.

Публикация репозитория

Создаете ли вы среду разработки с открытым исходным кодом, в которой над проектом работают множество людей по всему интернету, или же проект либо внутреннюю разработку в частной группе, механика совместной работы по сути одинакова. Основное различие между этими сценариями — в расположении репозитория и в том, кто имеет к нему доступ.



Термин *права на коммиты* (commit rights) в Git неактуален. Система не занимается управлением правами доступа, оставляя эту задачу другим инструментам (например, SSH), которые лучше для нее подходят. Вы всегда можете сделать коммит в любой репозиторий, к которому у вас есть доступ (Unix) через SSH или прямой `git`-доступ. Более адекватным будет вопрос «могу ли я обновить опубликованный репозиторий?» или, иными словами, «могу ли я отправить изменения в опубликованный репозиторий?».

В разделе «Обращение к удаленным репозиториям» (с. 263, глава 11) мы рекомендовали быть осторожными с формой URL удаленного репозитория `/path/to/repo.git`, поскольку она может вызывать проблемы, характерные для репозитория, использующих общие файлы. Тем не менее если вы будете настраивать общий контейнер с несколькими похожими репозиториями, то используйте общее внутреннее объектное хранилище. В этом случае репозитории будут равномерно увеличиваться в размере без удаления из них объектов и ссылок. Это полезно в случае общего использования объектного хранилища многими репозиториями, поскольку дает экономию огромного объема пространства на диске. Чтобы добиться такой экономии, применяйте опцию `--reference repository`, `--local` или `--shared` во время первичной настройки голого клона репозитория для публикуемых репозитория.

При любой публикации репозитория настоятельно советуем публиковать голую версию.

Репозитории с управляемым доступом

Для вашего проекта может быть достаточно публикации голого репозитория в известном месте файловой системы, к которому у всех есть доступ.

Естественно, доступ в этом контексте означает, что все разработчики могут видеть файловую систему со своих машин, имея обычные права владения Unix (или Unix-подобных систем) и разрешения на чтение/запись. В подобных сценариях достаточно URL-адреса файла вроде `/path/to/Depot/project.git` или `file:///path/to/Depot/project.git`. И хотя производительность может быть неидеальной, смонтированная NFS (Network File System, сетевая файловая система) сможет обеспечить подобный общий доступ.

Если существует несколько машин разработки, применяется чуть более сложная модель доступа. Например, IT-отдел в корпорации предоставил центральный сервер

для контейнера репозитория и поддерживает его резервное копирование. Тогда у каждого разработчика может быть настольная машина для разработки. Если прямого доступа к файловой системе вроде NFS нет, можно использовать репозитории с именами, указанными в URL-адресах SSH, но для этого все равно нужно, чтобы у каждого разработчика была учетная запись на центральном сервере.

В примере ниже к тому же репозиторию, который мы уже публиковали в `/tmp/Depot/my_website.git`, обращается разработчик, обладающий SSH-доступом к хосту:

```
desktop$ cd /tmp
desktop$ git clone ssh://example.com/tmp/Depot/my_website.git
Initialize my_website/.git
Initialized empty Git repository in /tmp/my_website/.git/
jdl@example.com's password:
remote: Counting objects: 27, done.
Receiving objects: 100% (27/27), done.objects: 3% (1/27)
Resolving deltas: 100% (7/7), done.
remote: Compressing objects: 100% (23/23), done.
remote: Total 27 (delremote: ta 7), reused 0 (delta 0)
```

После создания клона он записывает исходный репозиторий, используя следующий URL: `ssh://example.com/tmp/Depot/my_website.git`.

Аналогично команды `git fetch` и `git push` теперь можно использовать по сети:

```
desktop$ git push
jdl@example.com's password:
Counting objects: 5, done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 385 bytes, done.
Total 3 (delta 1), reused 0 (delta 0)
To ssh://example.com/tmp/Depot/my_website.git
55c15c8..451e41c main -> main
```

В этих примерах запрошенный пароль — это стандартный пароль авторизации в Unix для удаленного хоста.



Если вам нужно предоставить сетевой доступ с аутентификацией разработчиков, но при этом вы не хотите давать доступ к центральному серверу, ознакомьтесь с проектом Gitolite. Он представляет собой уровень управления доступом поверх Git. Начните здесь:

```
$ git clone https://github.com/sitaramc/gitolite.git
```

Чтобы избежать издержек, связанных с настройкой собственных серверов, в Git есть много альтернативных решений хостинга с разными ценовыми политиками, как бесплатными, так и профессиональными или корпоративными.

В зависимости от нужного масштаба, такой SSH-доступ к машинам может быть реализован внутри группы, корпорации или через интернет.

Репозитории с анонимным доступом к чтению

Чтобы поделиться кодом, вам нужно настроить центральный сервер для публикации репозитория и позволить другим их клонировать. Анонимного доступа только для чтения зачастую оказывается достаточно для того, чтобы разработчики могли клонировать эти репозитории или получать из них изменения. Стандартное и простое решение — экспорт с помощью `git-daemon` или HTTP-демона.

Фактическая среда, в которой вы можете публиковать ваш репозиторий, будет ограничена (или не ограничена) в той же степени, что и доступ к вашим HTTP-страницам или `git-daemon`. То есть если вы разместите эти команды на открытой машине, то любой сможет клонировать ваши репозитории и извлекать из них данные. Если вы поместите их за корпоративным файрволлом, то доступ будут иметь только сотрудники внутри корпорации (при условии отсутствия брешей в безопасности).

Публикация репозитория с помощью `git-daemon`

Настройка `git-daemon` (<https://oreil.ly/YxLNI>) позволит экспортировать репозитории, используя нативный протокол Git.

Для начала нужно обозначить репозитории как готовые к экспорту. Обычно это делается путем создания файла `git-daemon-export-ok` в каталоге верхнего уровня голого репозитория. Этот механизм дает детальный контроль над тем, какие репозитории демон сможет экспортировать.

Чтобы не отмечать каждый репозиторий по отдельности, можно выполнить `git-daemon` с опцией `--export-all` и опубликовать все идентифицируемые (по наличию подкаталогов `objects` и `refs`) репозитории из его списка *directories*. У `git-daemon` есть много опций, которые ограничивают и настраивают экспорт репозитория.

Один из способов — настройка `git-daemon` на сервере для активации его как службы `inetd`. В процессе также нужно будет обеспечить, чтобы в `/etc/services` была запись для Git. По умолчанию используется порт 9418, хотя вы можете установить любой. Стандартная запись может выглядеть так:

```
git      9418/tcp      # Система контроля версий Git
```

После добавления этой строки в `/etc/services` нужно настроить запись в `/etc/inetd.conf`, чтобы указать, как `git-daemon` должна вызываться.

Типичная запись может выглядеть так:

```
# Поместить одной длинной строкой в /etc/inetd.conf
```

```
git stream tcp nowait nobody /usr/bin/git-daemon
git-daemon --inetd --verbose --export-all
--base-path=/pub/git
```

Используя `xinetd` вместо `inetd`, поместите аналогичную конфигурацию в файл `/etc/xinetd.d/git-daemon`:

```
# описание: сервер git предоставляет доступ к репозиториям git service git
service git
{
    disable          = no
    type             = UNLISTED
    port             = 9418
    socket_type      = stream
    wait             = no
    user             = nobody
    server           = /usr/bin/git-daemon
    server_args      = --inetd --export-all --base-path=/pub/git
    log_on_failure   += USERID
}
```

С помощью одного из приемов `git-daemon` можно сделать так, будто репозитории расположены на отдельных хостах, хотя на самом деле они находятся в отдельных каталогах одного хоста. Пример записи ниже позволяет серверу предоставлять множество виртуально размещенных демонов Git:

Поместите одной длинной строкой в `/etc/inetd.conf`

```
git stream tcp nowait nobody /usr/bin/git-daemon
    git-daemon --inetd --verbose --export-all
    --interpolated-path=/pub/%N%D
```

В приведенной команде `git-daemon` заполнит `%N` полностью допустимым именем хоста, а `%D` — путем к каталогу репозитория. Поскольку `%N` может быть логическим именем хоста, на одном физическом сервере можно представить разные наборы репозитория.

Как правило, дополнительный уровень структуры каталогов, например `/software` или `/scm`, используется для организации публикуемых репозитория. Если совместить `--interpolated-path=/pub/%N%D` с путем каталога репозитория `/software`, публикуемые голые репозитории будут физически присутствовать на сервере в каталогах:

```
/pub/git.example.com/software/
/pub/www.example.org/software/
```

Тогда вы сможете объявить о доступности ваших репозитория по определенным URL-адресам, например:

```
git://git.example.com/software/repository.git
git://www.example.org/software/repository.git
```

Здесь `%N` заменяется хостом `git.example.com` или `www.example.org`, а `%D` — полным именем репозитория `/software/repository.git`.

Этот пример показывает, как один `git-daemon` можно использовать для поддержки и публикации нескольких отдельных коллекций репозитория Git, физически размещенных на одном сервере, но представленных на логически отдельных хостах. Репозитории, доступные с одного хоста, могут отличаться от тех, что представлены другим хостом.

Публикация репозитория с помощью HTTP-демона

Иногда проще опубликовать репозиторий с анонимным доступом к чтению при помощи HTTP-демона. Если вы также настроите `gitweb`, посетители смогут загружать URL в свои браузеры, видеть вывод индекса вашего репозитория и перемещаться с помощью кнопки «Назад» в браузере. Посетителям не нужно запускать Git, чтобы скачать файлы.

Прежде чем HTTP-демон сможет обработать ваш голый Git-репозиторий, нужно внести корректировку в конфигурацию этого репозитория, а именно — активировать опцию `hooks/post-update`:

```
$ cd /path/to/bare/repo.git
$ mv hooks/post-update.sample hooks/post-update
```

Убедитесь, что скрипт `post-update` доступен для выполнения или используйте для него `chmod 755`. Скопируйте этот репозиторий в каталог, который обслуживается вашим HTTP-демоном.

Теперь вы можете объявить, что ваш проект доступен по URL:

`http://www.example.org/software/repository.git`



Если вы видите сообщение об ошибке вроде такого:

```
...not found: did you run git update-server-info
on the server?
```

или такого:

```
Perhaps git-update-server-info needs to be run there?
```

то, скорее всего, вы неправильно выполняете команду `hooks/post-update` для сервера.

Публикация репозитория с помощью Smart HTTP

Публикация репозитория с помощью нового механизма Smart HTTP довольно проста. Однако рекомендуем почитать документацию по этому процессу на странице руководства команды `git-http-backend`.

Здесь мы приведем основную информацию.

- Эта настройка предназначена для использования с Apache. Примеры ниже показывают, как изменять файлы конфигурации Apache. В системе Ubuntu они находятся в `/etc/apache2`.

- Требуется сопоставить имена размещаемых вами репозиториев со схемой их расположения на диске, которую будет использовать Apache. Как и в документации `git-http-backend`, в результате сопоставления `http://$hostname/git/foo/bar.git` будет соответствовать формату представления файлов Apache `/var/www/git/foo/bar.git`.
- Модули Apache необходимы и должны быть включены: `mod_cgi`, `mod_alias` и `mod_env`.

Определите переменные и псевдонимы скриптов, указывающие на команду `git-http-backend`:

```
SetEnv GIT_PROJECT_ROOT /var/www/git
SetEnv GIT_HTTP_EXPORT_ALL
ScriptAlias /git/ /usr/libexec/git-core/git-http-backend/
```

Расположение вашего `git-http-backend` может отличаться. Например, Ubuntu помещает его в `/usr/lib/git-core/git-http-backend`.

У вас есть выбор: разрешить анонимный доступ для чтения, но потребовать аутентификацию для записи в репозиторий либо затребовать аутентификацию и для чтения тоже.

Для анонимного доступа к чтению настройте директиву `LocationMatch`:

```
<LocationMatch "^/git/.*/git-receive-pack$">
  AuthType Basic
  AuthName "Git Access"
  Require group committers
  ...
</LocationMatch>
```

Для аутентифицированного доступа к чтению настройте директиву `Location` для репозитория или его родительского каталога:

```
<Location /git/private>
  AuthType Basic
  AuthName "Private Git Access"
  Require group committers
  ...
</Location>
```

Дополнительные инструкции можно найти в руководстве. С их помощью вы сможете настроить скоординированный доступ `gitweb` и узнаете, как обслуживать несколько пространств имен репозиториев, а также настраивать ускоренный доступ к статическим страницам.

Публикация репозиториев через Git- и HTTP-демоны

Хотя использование сервера и браузера очень удобно, подумайте о том, сколько трафика вы планируете обрабатывать на своем сервере. Проекты разработки

могут сильно разрастаться, и HTTP будет не столь эффективным, как нативный протокол Git.

Вы можете предоставить доступ с помощью HTTP- и Git-демонов, но для этого нужно дополнительно настроить и согласовать их. В частности, это может потребовать отображения с помощью опции `--interpolated-path` для `git-daemon` и опции `Alias` для Apache, чтобы обеспечить бесшовную интеграцию двух видов одних и тех же данных. Дополнительные детали по опции `--interpolated-path` приведены на странице руководства по `git daemon`. Опция `Alias` описана в документации Apache и собственном файле конфигурации `/etc/apache2/mods-available/alias.conf`.

Репозитории с анонимным доступом к записи

В принципе вы можете использовать формы URL нативного протокола Git, чтобы разрешить анонимный доступ к записи в репозитории, обслуживаемые `git-daemon`. Это потребует активации опции `receivepack` в файле конфигурации опубликованного репозитория:

```
[daemon]
    receivepack = true
```

Можно делать это в частной локальной сети, где вы уверены в каждом разработчике, но это не слишком хорошая практика. Следует рассмотреть вариант туннелирования через SSH-подключение для отправки ваших изменений в репозиторий.

Совет по публикации репозитория

Прежде чем приступать к настройке сервера и сервисов хостинга, чтобы разместить репозитории Git, оцените реальные потребности и причины, по которым хотите предоставить доступ к этим репозиториям. Возможно, ваши задачи можно решить через уже имеющиеся компании, сайты или сервисы.

Для закрытого или публичного кода, который требует высокого качества обслуживания, рассмотрите использование коммерческой службы Git. Если вы предоставляете доступ к репозиторию с открытым исходным кодом без особых требований к обслуживанию, то можете воспользоваться множеством доступных сервисов хостинга Git. Самые популярные решения — это GitHub, GitLab и Bitbucket.

Более сложная ситуация — когда у вас есть закрытый код, который вы хотите хранить внутри компании, и поэтому вам нужно настроить и поддерживать собственный центральный контейнер для хостинга репозитория. Не забудьте и о регулярном создании бэкапов. И даже в таком случае у Git есть готовые для корпоративного использования версии платформ хостинга, обеспечивающие всю эту функциональность и доступные в виде подписки или по лицензии.

Если нужно настраивать собственные серверы, то стандартным решением является протокол SSH, чтобы все пользователи репозитория имели SSH-доступ к центральному серверу. На сервере обычно создаются общий пользовательский аккаунт и группа (например, `git` или `gituser`). Все репозитории обозначаются как принадлежащие этому групповому пользователю; обычно они находятся в определенном файловом пространстве (например, `/git`, `/opt/git` или `/var/git`), выделенном специально для этой цели. Главное, чтобы этот каталог принадлежал вашей группе `gituser`, допускал запись от имени этой группы и имел права доступа `sticky bit`.

Теперь, когда вы захотите создать на сервере новый опубликованный репозиторий с именем `newrepo.git`, подключитесь с помощью `ssh` к этому серверу и сделайте следующее:

```
$ ssh git.my-host.example.com

$ cd /git
$ mkdir newrepo.git
$ cd newrepo.git
$ git init --shared --bare
```

Четыре последние команды можно упростить до одной:

```
$ git --git-dir /git/newrepo.git init --shared
```

К этому моменту есть структура голого репозитория, но сам он еще пуст. Важно то, что репозиторий теперь является приемником при передаче изначального содержимого от любого пользователя, имеющего право подключаться к серверу:

```
# От какого-то клиента.
$ cd /path/to/existing/initial/repo.git
$ git push git+ssh://git.my-host.example.com/git/newrepo.git main
```

После выполнения команды `git init` на сервере настраиваются необходимые инструменты и права доступа для последующей работы с репозиторием.

Структура репозиториев

Хотя Git является распределенной системой, в ней можно реализовать централизованную модель работы, когда одна версия репозитория может быть источником истины для всех, кто работает над этим репозиторием. Поговорим об этом подробнее.

Структура общего репозитория

В некоторых системах контроля версий для поддержки репозитория используется централизованный сервер. В такой модели каждый разработчик выступает клиентом этого сервера, поддерживающего достоверную версию репозитория. С учетом

такой позиции сервера почти каждая операция версионирования должна связываться с ним для получения или обновления информации репозитория. Таким образом, чтобы двое разработчиков могли обмениваться данными, вся информация должна пройти через централизованный сервер. Прямой обмен данными между разработчиками невозможен.

В Git общий, достоверный и централизованный репозиторий является условным соглашением. У каждого разработчика по-прежнему есть свой клон основного репозитория контейнера, что исключает необходимость передачи любого запроса через централизованный сервер. Например, простые запросы на историю изменений могут быть сделаны каждым разработчиком в частном порядке и автономном режиме.

Одна из причин, почему некоторые операции можно выполнять локально, состоит в том, что переключение на ветку ведет к извлечению не только конкретной запрашиваемой версии, как это происходит в большинстве централизованных систем контроля версий, но всей истории. Поэтому вы можете воссоздать из локального репозитория любую версию файла.

Ничто не мешает разработчику установить альтернативный репозиторий и сделать его доступным для других разработчиков в равноправном формате или обмениваться содержимым, используя патчи и ветки.

Таким образом, представленная в Git идея модели централизованного репозитория — это условность.

Распределенная структура репозитриев

В крупных проектах часто используется распределенная модель разработки, состоящая из одного центрального, но логически сегментированного репозитория. И хотя этот репозиторий существует как одна физическая единица, его логические разделы закрепляются за разными людьми или командами, которые ведут совместную или полностью независимую работу.



Когда говорят, что Git поддерживает модель распределенных репозитриев, это не значит, что один репозиторий разбивается на отдельные части, распределяемые по множеству хостов. Распределенный репозиторий — это следствие реализуемой в Git модели распределенной разработки. У каждого разработчика есть собственный полный и самодостаточный репозиторий. При этом его репозиторий может быть распределен по сети.

То, как репозиторий разделяется или выделяется для разных разработчиков, для Git не так важно. Репозитории могут иметь как глубоко вложенную структуру каталогов, так и более плоскую. Например, разные команды разработки могут отвечать за определенные части баз кода в подмодуле, библиотеке или линиях функциональности. Каждая команда может назначить ответственного на роль

мейнтейнера своей кодовой базы и договориться пересылать все изменения через него.

По мере подключения к проекту все большего числа людей или групп структура может развиваться. Команда может формировать промежуточные репозитории, содержащие комбинации других репозиториях, и проводить в них (или не проводить) дополнительную разработку. Могут создаваться стабильные репозитории, или репозитории релизов, за каждым из которых закрепляется команда разработки и мейнтейнер.

Лучше, если репозитории и потоки данных будут разрастаться естественным образом в соответствии с оценкой и предложениями со стороны команды, а не по заданной заранее схеме. Git — достаточно гибкая система, поэтому если разработка по одной схеме не работает, ее легко сменить на более подходящую.

Принцип организации, взаимосвязи и совмещения репозиториях в крупном проекте для Git значения не имеет. Эта система поддерживает любое число моделей организации. Помните, что структура репозиториях не является жесткой. Более того, связь между двумя любыми репозиториями не предопределена. В Git все репозитории равнозначны.

Как поддерживается структура репозиториях, если никакие технические меры ее не определяют? По сути, структура представляет сеть доверия для принятия изменений. Организация репозитория и потока данных между репозиториями управляется соглашениями. Базовые возможности многих используемых Git платформ хостинга позволяют устанавливать такие соглашения в соответствии с вашими нуждами разработки. Это еще одна причина для того, чтобы воспользоваться преимуществами такой платформы.

При совместной разработке возникают два важных вопроса: примет ли мейнтейнер целевого репозитория ваши изменения и достаточно ли вы доверяете данным исходного репозитория, чтобы получать их в свой.

Практическая сторона распределенной разработки

В следующих разделах описаны лучшие практики, руководства и правила при работе с одним или более общими репозиториями Git.

Изменение публичной истории

После публикации репозитория, который смогут клонировать другие, вам следует считать его статичным и не переписывать историю какой-либо ветки. Это не строгое правило, но ему лучше следовать во избежание перемоток и изменений истории опубликованного репозитория. Это упростит жизнь всем, кто клонировал репозиторий.

Предположим, вы публикуете репозиторий, в котором есть ветки с коммитами А, В, С и D. Любой, кто его клонирует, получит эти коммиты. Допустим, это делает Алиса и приступает к разработке, взяв за основу вашу ветку.

Тем временем вы решаете исправить что-то в коммите С. Коммиты А и В остаются прежними, но, начиная с коммита С, отражаемая веткой история изменяется. Вы можете изменить С лишь немного или же сделать полностью новый коммит, X. В любом случае в результате повторной публикации репозитория коммиты А и В останутся прежними, но теперь в нем вместо С и D появятся, например, коммиты X и Y.

Все это сильно повлияет на работу Алисы. Она не сможет присылать вам патчи, делать пул-реквесты или отправлять изменения в ваш репозиторий, потому что ее разработка основывается на коммите D.

Поскольку патчи основаны на коммите D, применить их не получится. Предположим, Алиса создает пул-реквест, и вы пытаетесь получить внесенные ею изменения. Вы сможете скачать их в свой репозиторий (в зависимости от ваших веток отслеживания удаленного репозитория Алисы), но слияния наверняка приведут к конфликтам. Провал этой передачи вызван проблемой передачи без перемотки.

Если коротко, то фундамент разработки Алисы был изменен. Вы «вытанули ковер коммитов из-под ног ее разработки».

Но ситуация поправима. Git может выручить Алису, особенно если она использует команду `git rebase --onto` для переноса своих изменений в вашу новую ветку после получения этой ветки в свой репозиторий. Опция `--onto <newbase>` позволяет установить стартовую точку, от которой вы сможете создать новый коммит.

Бывает так, что наличие ветки с динамической историей вполне уместно. Например, в самом репозитории Git есть так называемые ветки предлагаемых обновлений, `pr`, которые обозначаются и позиционируются как часто перематываемые, перебазируемые или переписываемые. Как создатель клона, вы можете смело использовать такую ветку в качестве основы для своей разработки, но важно понимать назначение этой ветки и прилагать особые усилия для эффективного ее использования.

А зачем кому-то вообще публиковать ветку с динамической историей коммитов? Одна из причин в том, чтобы предупредить других разработчиков о возможных и быстро меняющихся направлениях, в которых может пойти некая ветка. Их также можно создавать исключительно для того, чтобы сделать доступным, хотя бы временно, опубликованный набор изменений, который смогут использовать другие разработчики.

Разделение коммита и публикации

Одно из очевидных преимуществ распределенных систем контроля версий — разделение этапов коммита и публикации. Коммит просто сохраняет состояние в вашем закрытом репозитории, а публикация через патчи или операции `push/pull` делает изменения публичными, фиксируя историю репозитория. В системах

CVS или SVN нет подобного разделения: чтобы сделать коммит изменений, их нужно сразу публиковать.

Благодаря разделению этапов коммита и публикации разработчику будет проще делать небольшие и логичные шаги при внесении патчей. Можно реализовать любое количество небольших изменений, не повлияв на какой-либо другой репозиторий или разработчика. Операция коммита производится в автономном режиме: это значит, что не требуется сетевого доступа для записи в ваш репозиторий положительных продвижений вперед.

В Git также есть механизмы для доработки коммитов (см. главу 9), превращающие их в чистые последовательности перед внесением в основную ветку. После того как вы все подготовите, коммиты можно делать публичными с помощью отдельной операции.

Не существует единственной истинной истории

Проекты разработки в распределенной среде имеют неочевидные особенности. Первое время это может вызывать путаницу, поскольку работа отличается от других, нераспределенных систем контроля версий. Однако Git обрабатывает эти особенности понятным и логичным образом.

Поскольку разработка происходит параллельно, каждый из участников проекта создает свою историю коммитов, которую считает истинно верной. В результате есть мой репозиторий и моя история коммитов, ваш репозиторий и ваша история коммитов, а также несколько других таких историй.

Каждый разработчик имеет свое уникальное видение истории, и каждая история верна. Нет единой истины. Нельзя указать на одну из них и сказать: «Вот это *настоящая* история».

Вероятно, эти разные истории разработки сформировались не просто так, и в итоге различные репозитории и истории коммитов будут совмещены в одном общем репозитории. Ведь речь идет о продвижении к общей цели. Когда сливаются разные ветки из разных репозиториях, присутствуют все их вариации. Результат такого слияния как бы говорит: «Объединенная история лучше любой независимой».

Git выражает эту неоднозначность историй в отношении вариаций разных веток, когда обходит направленный ациклический граф коммитов (DAG). Поэтому, если Git при попытке линеаризовать последовательность коммитов достигает коммита слияния, он должен сначала выбрать одну ветку или другую. Какой критерий он будет использовать для предпочтения одной ветки перед другой? Фамилию автора? Или метку времени коммита?

Даже если вы решите использовать временные метки и договоритесь ориентироваться на всемирное координированное время (UTC) и высокоточные значения, это не поможет. Даже такое решение оказывается совершенно ненадежным. (Часы на компьютере разработчика могут быть сбиты намеренно или случайно.)

Git не волнует, какое событие произошло первым. Единственная реальная и надежная взаимосвязь, которая может быть установлена между коммитами, — прямая родительская связь, записанная в объектах коммита. Временные метки в лучшем случае дадут вторичный ориентир, обычно в сопровождении различных эвристик, учитывающих возможные ошибки вроде неверно установленных часов.

GIT КАК P2P-СИСТЕМА РЕЗЕРВНОГО КОПИРОВАНИЯ

Сохранность исходного кода ядра Linux долгие годы поддерживалась благодаря загрузке файлов в интернет, откуда их могли скачивать другие. И это работало.

В некотором смысле Git расширяет этот принцип. Когда вы скачиваете исходный код ядра Linux с помощью Git, то получаете не только последнюю версию, но и всю предшествовавшую ей историю.

Этот принцип используется в проектах, когда системные администраторы могут управлять каталогами конфигурации */etc* через Git и даже дают возможность пользователям управлять своими домашними каталогами и делать их бэкап. Помните: то, что вы используете Git, еще не означает, что вам обязательно нужно публиковать свои репозитории; однако это позволяет с легкостью вести контроль версий ваших репозиториях прямо на сетевом накопителе (NAS, Network Attached Storage) для создания резервной копии.

Знать свое место

Участвуя в проекте распределенной разработки, важно понимать, как вы, ваш репозиторий и ваши усилия вписываются в общую картину. Помимо предоставления очевидной возможности работать в разных направлениях и требования общей координации, механика использования Git и ее инструментов сильно влияет на то, насколько гармонично ваши усилия согласуются с усилиями других участников проекта.

Это становится особенно актуально в крупномасштабных проектах, например в проектах с открытым исходным кодом. Определив свою роль в общей картине и понимая, кто потребитель изменений, а кто их создатель, вы можете легко решить многие вопросы.

Вышестоящий и нижестоящий репозитории

Между репозиториями, один из которых был клонирован из другого, нет никакой строгой иерархии. Однако родительский репозиторий принято называть *вышестоящим* (upstream). Клонированный репозиторий часто называют *нижестоящим* (downstream).

Кроме того, восходящая связь протягивается «вверх» от родительского репозитория к тому, от которого он сам был клонирован. Она также протягивается «вниз» за ваш репозиторий к любому, который был клонирован из него.

Описание вышестоящего и нижестоящего репозитория не связано напрямую с операцией клонирования. Git поддерживает полностью произвольную сеть между репозиториями. В нее можно добавлять новые удаленные подключения, и изначальное удаленное подключение вашего клона можно удалять для создания произвольно новых связей между репозиториями.

Если и есть какая-то установленная иерархия, то она условная. Боб соглашается отправлять изменения вам. Вы, в свою очередь, соглашаетесь отправлять свои изменения кому-то следующему по восходящей цепочке и т. д.

Важный аспект связей между репозиториями в том, как происходит обмен данными между ними: любой репозиторий, в который вы отправляете изменения, обычно считается для вас вышестоящим.

Все это чисто субъективное видение, но оно используется всеми. Git никак не отслеживает это условное обозначение потоков. Понятия «вышестоящий» и «нижестоящий» просто помогают наглядно представить, куда отправляются патчи.

Конечно, репозитории могут быть истинными пирами, то есть не иметь явного иерархического отношения. Если два разработчика обмениваются патчами и отправляют и получают данные из репозитория друг друга, то ни один из них не считается вышестоящим или нижестоящим.

Роли разработчика и мейнтейнера

В качестве двух типичных ролей можно выделить мейнтейнера и разработчика. Мейнтейнер выступает в основном как интегратор и модератор, в то время как разработчик генерирует изменения. Мейнтейнер собирает и координирует эти изменения от нескольких разработчиков и следит за тем, чтобы они соответствовали некоему стандарту. После этого он делает весь набор предложенных обновлений доступным. Мейнтейнер также занимается публикацией.

Его задача — собирать, модерировать, принимать или отклонять изменения, в итоге публикуя ветки, которые разработчики могут использовать. Для обеспечения плавности процесса разработки мейнтейнер не должен изменять ветку после ее публикации. Вместо этого он ожидает от разработчиков получения актуальных изменений, которые можно будет внести в опубликованные ветки.

Задача разработчика, помимо улучшения проекта, заключается в передаче изменений мейнтейнеру. Ведь правки, хранящиеся в чем-то локальном репозитории, не приносят пользы никому. Эти изменения должен принять мейнтейнер и сделать доступными для использования другими. Разработчики должны основывать свою работу на опубликованных в репозитории ветках, которые предлагает мейнтейнер.

В контексте производного клона репозитория мейнтейнер обычно считается вышестоящим относительно разработчиков.

Поскольку Git полностью симметрична, ничто не мешает разработчику рассматривать себя в роли мейнтейнера для других разработчиков, находящихся ниже по цепочке. Но важно понимать, что он находится в середине между вышестоящим и нижестоящим потоками данных, что обязывает его подчиняться условиям взаимодействия мейнтейнера и разработчика (см. следующий раздел).

Поскольку возможна такая двойная роль, то вышестоящие и нижестоящие участники не считаются строго производителями или потребителями. Вы можете создавать изменения для их передачи как выше по цепочке, так и ниже.

Взаимодействие между мейнтейнером и разработчиком

Взаимосвязь мейнтейнера и разработчика часто бывает не вполне внятной, но при этом подразумевает некий контракт. Мейнтейнер публикует ветки, которые разработчик использует в качестве основы. Однако после публикации мейнтейнер негласно обязуется не изменять ветку, поскольку это нарушит основу, от которой отталкивается разработчик.

И наоборот, разработчик, используя опубликованные ветки в качестве основы, делает так, чтобы при отправке изменений мейнтейнеру они применялись без каких-либо проблем или конфликтов, а также в соответствии с установленными мейнтейнером руководствами.

Может показаться, что этот процесс жесткий. После публикации ветки мейнтейнер ничего не может с ней делать, пока не получит изменения от разработчика. И затем, когда он применит полученные обновления, ветка неизбежно изменится, нарушив соглашение «не изменять ветку» для других разработчиков. Если бы это действительно было так, тогда по-настоящему параллельную и независимую работу невозможно было бы наладить.

К счастью, Git может обращаться к прошлой истории коммитов в измененных ветках, определять основу слияния, которая использовалась в качестве отправной точки для изменений, и применять их, несмотря на то что мейнтейнер мог внести другие изменения от других разработчиков.

В контексте, когда несколько разработчиков вносят независимые изменения, которые объединяются и сливаются в общем репозитории, могут возникать конфликты. Выявление и разрешение таких проблем ложится на плечи мейнтейнера и разработчика. Мейнтейнер может разрешать эти конфликты напрямую или отклонять изменения разработчика, если они ведут к конфликтам. Сегодня современные платформы хостинга Git предлагают возможности, которые позволяют мейнтейнерам и разработчикам систематически отслеживать и разрешать конфликты подобных изменений.

Двойные роли

Есть два базовых механизма для передачи коммитов между вышестоящим и нижестоящим репозиториями.

В первом применяются команды `git push` или `git pull`, позволяющие передавать коммиты непосредственно. Второй механизм использует `git format-patch` и `git am`, команды, которые отправляют и получают лишь представления коммитов. Выбор метода определяется соглашением внутри вашей группы разработки и в некоторой степени правами на прямой доступ (см. главу 11).

Использование `git format-patch` и `git am` для применения патчей ведет к получению такого же *содержимого* blob-объекта и объекта дерева, как и в случае внесения изменений через `git push` либо `git pull`. Но фактический объект коммита будет иным, поскольку метаданные между `push` или `pull` и соответствующим применением патча будут отличаться.

Иными словами, использование `push/pull` для передачи изменений из одного репозитория в другой ведет к точному копированию этого коммита, а при выполнении патча в точности копируются только данные файла и каталога. Более того, `push` и `pull` могут передавать между репозиториями коммиты слияния, причем такие коммиты нельзя передавать в виде патчей.

Поскольку сравнение и обработка происходят в отношении blob-объектов и объектов деревьев, Git понимает, что два разных коммита для одного изменения в двух разных репозиториях или даже в разных ветках одного репозитория представляют одно изменение. Таким образом, два разработчика могут без проблем применить к двум разным репозиториям один и тот же патч, переданный по электронной почте. До тех пор пока их итоговое содержимое будет совпадать, Git будет рассматривать эти репозитории как имеющие одинаковое содержимое.

Посмотрим, как совмещаются роли и потоки данных, формируя двойную роль между вышестоящими и нижестоящими производителями и потребителями.

Вышестоящий потребитель

Находящийся выше по цепочке разработчик, который получает ваши изменения в виде наборов патчей либо пул-реквестов. Патчи должны быть перебазированы на текущую HEAD ветки потребителя. Пул-реквесты должны предполагать прямое слияние или быть внесены вами в репозиторий. Слияние до пул-реквеста гарантирует, что конфликты будут разрешены вами, что освободит от этого бремени вышестоящего потребителя. На его месте может оказаться мейнтейнер, который, в свою очередь, публикует только что полученные изменения.

Нижестоящий потребитель

Разработчик по цепочке ниже, который использует в качестве основы для работы ваш репозиторий. Нижестоящему потребителю нужны устойчивые

опубликованные ветки разработки. Не следует перебазирувать, изменять или переписывать историю любой опубликованной ветки.

Вышестоящий производитель/публикующий

Пользователь над вами, который публикует репозитории, являющиеся основой вашей работы. Вероятнее всего, это мейнтейнер, принимающий ваши изменения. Роль вышестоящего публикующего заключается в сборе изменений и публикации веток. В опубликованных ветках не должна изменяться история, поскольку это основа для последующей нижестоящей разработки. Мейнтейнер ожидает, что присылаемые разработчиком патчи окажутся применимы, а пул-реквесты не вызовут проблем при слиянии.

Нижестоящий производитель/публикующий

Разработчик под вами, который опубликовал изменения либо в виде набора патчей, либо в виде пул-реквеста. Его цель — добиться того, чтобы вы приняли его изменения в своем репозитории. Нижестоящий производитель получает от вас ветки разработки и хочет, чтобы они оставались стабильными, не подвергались переписыванию истории или перебазированию. Он должен регулярно запрашивать обновления из удаленного репозитория, а также совмещать или перебазировать ветки разработки, чтобы те применялись к HEAD локальных вышестоящих веток. Нижестоящий производитель может перебазировать собственные локальные ветки разработки в любое время, поскольку для вышестоящего потребителя не имеет значения, что этому разработчику потребовалось несколько итераций, чтобы создать набор патчей с чистой и простой историей.

На рис. 12.1 показаны двойные роли. Связь между актерами здесь представлена в виде двунаправленного потока.

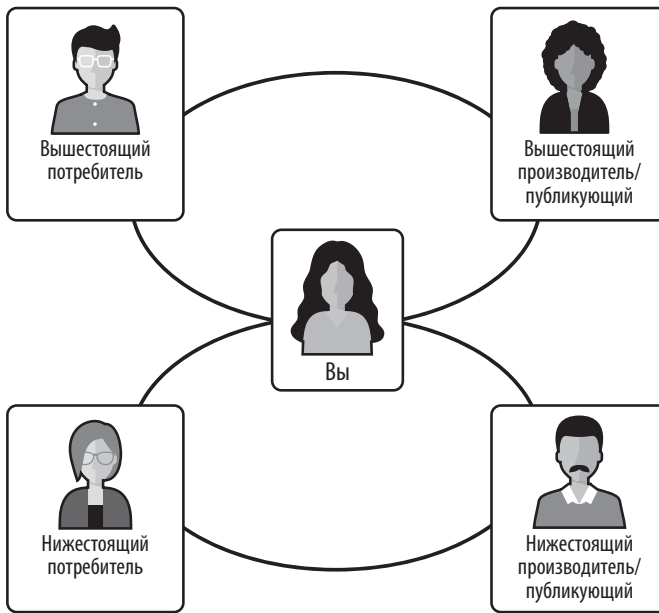
Работа с несколькими репозиториями

Далее мы разберем техники, которые вы можете использовать при работе с несколькими распределенными репозиториями.

Собственное рабочее пространство

Если вы участвуете в проекте с использованием Git, то должны создать собственную закрытую копию, или клон репозитория, где будете вести разработку. Такой репозиторий будет вашей рабочей областью, где вы сможете вносить изменения, не боясь вызвать конфликт, нарушить работу или помешать другому разработчику.

Поскольку каждый репозиторий Git содержит полную копию всего проекта, а также всю его историю, вы можете рассматривать свой репозиторий так, будто он полностью ваш. В действительности это так и есть.

**Рис. 12.1.** Двойные роли

Такая парадигма дает каждому разработчику полный контроль над рабочим каталогом. Можно вносить изменения в любую часть каталога или даже во всю систему, не беспокоясь о взаимодействии с другими разработчиками. Если нужно изменить какую-то часть, то эта часть у вас уже есть, и вы можете изменить ее в собственном репозитории, не затрагивая других разработчиков. Точно так же, если вы поймете, что ваша работа бесполезна или неактуальна, вы можете удалить ее, не влияя на другие репозитории.

Однако это не означает, что можно беспрепятственно заниматься экспериментами. Всегда учитывайте последствия своих изменений, поскольку в итоге вам может потребоваться влить их в основной репозиторий. Тогда любые внесенные вами произвольные изменения дадут о себе знать.

С чего начать свой репозиторий

Столкнувшись с рядом репозиториях, которые в итоге относятся к одному проекту, бывает сложно определить, откуда следует начать разработку. Должен ли ваш вклад основываться на главном репозитории или на репозитории, где производится работа над какой-то другой функциональностью? А может, взять за основу стабильную ветку репозитория релиза?

Без понимания способов обращения к репозиториям, а также принципов их работы и изменения можно столкнуться с дилеммой «не могу начать, потому что боюсь начать не оттуда».

Может быть, вы уже начали разработку в клоне, основанном на выбранном репозитории, но теперь понимаете, что выбрали не тот? Да, он связан с проектом и может стать хорошей отправной точкой, но, возможно, в нем не хватает функциональности, которая есть в другом репозитории. И осознание происходит уже после погружения в работу.

Другая проблема: вам требуется функциональность, которая разрабатывается в двух разных репозиториях. И ни один из них не является правильной основой для вашей работы.

Вы можете продвигаться вперед, ожидая, что и ваша работа, и работа в этих репозиториях будет унифицирована и совмещена в основном репозитории. Это приемлемый вариант. Но нужно помнить, что часть преимуществ распределенной разработки заключается в возможности ее параллельного ведения. Воспользуйтесь фактом доступности других опубликованных репозиториях с ранними версиями.

Еще одна ловушка: вы начнете работу с репозиторием, который находится на переднем крае разработки, а затем обнаружите, что он слишком нестабилен или вообще заброшен.

К счастью, Git поддерживает модель, в которой можно выбрать в качестве стартовой точки любой репозиторий проекта, даже неидеальный, а затем преобразовать, изменить или расширить его, чтобы он содержал нужную функциональность.

Если вы захотите отделить свои изменения от других вышестоящих репозиториях, то вам придется рационально и осторожно использовать отдельные ветки разработки и слияния, чтобы все согласовать.

С одной стороны, вы можете получить ветки из нескольких удаленных репозиториях и совместить их в собственный, получив правильную комбинацию функциональности, доступной в других репозиториях. С другой стороны, вы можете откатить стартовую позицию вашего репозитория к известной стабильной точке в истории разработки проекта.

Преобразование в другой вышестоящий репозиторий

Первый и самый простой вид смешивания и сопоставления репозиториях — это смена базового (обычно называемого *источником клона*) репозитория, который вы считаете исходным и с которым регулярно синхронизируетесь.

Допустим, вам нужно поработать над функцией F, и вы решаете клонировать ваш репозиторий из основной линии разработки, M (рис. 12.2).

Вы работаете, а затем выясняете, что есть другая стартовая точка, более приближенная к тому, чего вы хотите добиться, но она находится в репозитории P. Одна из причин смены источника клона — вы хотите получить поддержку функциональности, которая есть в репозитории P.

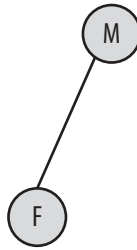


Рис. 12.2. Простое клонирование для разработки функциональности F

Другая причина кроется в долгосрочном планировании. Рано или поздно потребуется внести разработку, сделанную в репозитории Р, обратно в вышестоящий репозиторий. Примет ли мейнтейнер репозитория М ваши изменения напрямую? Может и нет. Если вы уверены, что мейнтейнер репозитория Р их примет, следует подготовить патчи, чтобы они были применимы именно к этому репозиторию.

Предположим, что Р был когда-то клонирован из М или наоборот (рис. 12.3). В итоге Р и М основываются на одном и том же репозитории одного проекта в некоторой точке, находящейся в прошлом.

Возникает вопрос: можно ли репозиторий F, изначально основанный на М, преобразовать так, чтобы он основывался на репозитории Р (рис. 12.4)? Git позволяет легко сделать это, поскольку поддерживает одноранговые отношения между репозиториями и дает удобную возможность перебазирующих веток.

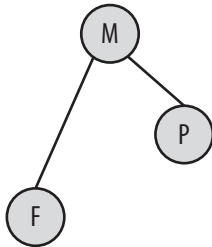


Рис. 12.3. Два клона репозитория

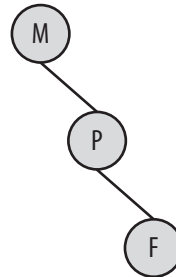


Рис. 12.4. Функциональность F реструктурирована для репозитория Р

Git знает, как выровнять различия между двумя репозиториями. Часть однорангового протокола для получения веток из другого репозитория — это обмен информацией о том, какие изменения есть или отсутствуют в каждом репозитории. Благодаря этому Git может получать только отсутствующие или только новые изменения и вносить их в ваш репозиторий.

Git может просматривать историю веток и определять, где находятся общие предки из разных веток, даже если они внесены из других репозиториях. Если у этих

веток есть общий предок, то Git найдет его и выстроит обширное представление истории коммитов, где будут представлены все изменения репозитория.

Использование нескольких вышестоящих репозиториях

Допустим, общая структура репозитория соответствует изображенной на рис. 12.5. Здесь некий основной репозиторий **M** в итоге объединит в себе всю разработку двух функций из репозиториях **F1** и **F2**.

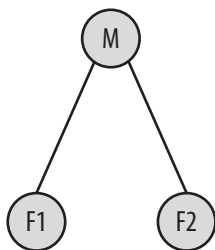


Рис. 12.5. Два репозитория функциональности

Вам нужно разработать новую функциональность **S**, которая включает использование элементов функций, присутствующих только в **F1** и **F2**. Вы можете подождать, пока произойдет слияние **F1** и **F2** с **M**. В таком случае будет доступен репозиторий с корректной общей основой для вашей работы. Но если в проекте нет строгого соблюдения жизненного цикла, требующего слияния через определенные интервалы, неизвестно, сколько времени займет этот процесс.

Вы можете начать свой репозиторий **S**, основываясь на функциональности из **F1** или из **F2** (рис. 12.6).

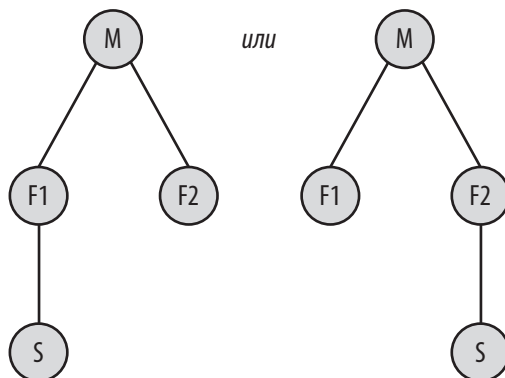


Рис. 12.6. Возможные начальные репозитории для **S**

С помощью Git можно создать репозиторий *S*, который будет содержать *F1* и *F2*. Процесс показан на рис. 12.7.

Из рис. 12.7 неясно, будет ли репозиторий *S* состоять из полных функциональностей *F1* и *F2* или только из их частей. Git поддерживает оба сценария.

Предположим, что в репозитории *F2* есть ветки *F2A* и *F2B* с функциональностью *A* и *B* соответственно (рис. 12.8). Если в разработке нужна функциональность *A*, но не *B*, вы можете получить в свой репозиторий только ветку *F2A* вместе с необходимой частью *F1*.

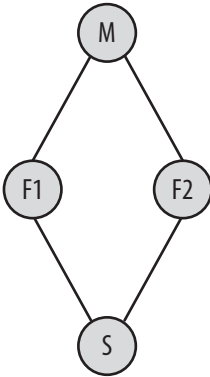


Рис. 12.7. Создание объединенной версии *S*

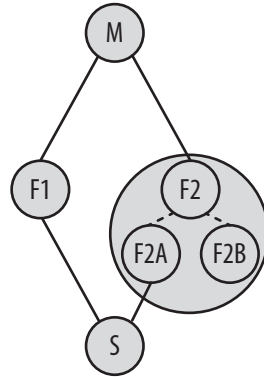


Рис. 12.8. Две ветки функциональности в *F2*

Форки проектов

Когда вы клонируете репозиторий, это действие можно рассматривать как создание *форка* (fork) проекта. Форк равнозначен ветвлению (branching) в других системах контроля версий, но в Git ветвление представляет отдельный механизм, и важно не путать их. В Git, в отличие от ветки, форк не имеет конкретного имени. Вы просто обращаетесь к нему по адресу каталога (или удаленного сервера, URL), в который выполняли клонирование.

Смысл *форка* — создание развилки на дороге, или двух путей, по которым пойдет разработка. Создание *ветки* в Git — это аналогия с веткой дерева. «Ветвление» и «создание форка» отражают два разных намерения. Концептуальное отличие в том, что ветвление обычно происходит внутри одного репозитория, а форки формируются на уровне всех репозиториях.

И хотя Git позволяет делать форки, это может быть скорее социальным или политическим выбором, чем техническим. В открытых или опенсорсных проектах наличие доступа к копии или клону всего репозитория вместе с его историей — это фактор, который одновременно способствует и сдерживает создание форков.

Делать форк или нет?

Исторически термин «развилка проекта» (forking) имел негативный оттенок, например это происходило при захвате контроля над проектом, нежелании сотрудничать или отказе от проекта. Трудный человек, оказавшись у руля сложного проекта, может застопорить его. Раскол возникает между теми, кто во главе проекта, и остальными. Часто единственным очевидным решением становится создание форка для нового проекта. В таком случае сложно получить копию истории проекта и начать заново.

Создание форка — привычный термин, описывающий ситуацию, когда один разработчик опенсорс-проекта оказывается недоволен основной линией разработки, делает копию исходного кода и начинает работу над своей версией.

В этом смысле создание форка считалось отрицательным. Это означало, что недовольный разработчик не смог найти способ получить желаемое от основного проекта и в итоге решил сделать все сам. В итоге образовались *два* почти одинаковых проекта. Ни один из них не будет достаточно хорош для всех, и в конце концов какой-то из них окажется заброшен. Поэтому большинство опенсорсных проектов стремятся избегать форков.

Создание форков может быть как удачным, так и неудачным решением. С одной стороны, альтернативное видение и новое руководство могут оживить проект. С другой — принести разногласия и путаницу в общий процесс разработки.

Примирение с форками

Git стремится снять клеймо с форков. Когда разработчик скачивает или клонирует проект и начинает его, он всего лишь создает альтернативный путь разработки, хотя бы временно.

Работая над ядром Linux, Линус Торвальдс понял, что создание форков становится проблемой только тогда, когда они в итоге не совмещаются с основной линией разработки. Поэтому Git проектировался с иным подходом к форкам: система *поощряет* их создание. При этом Git позволяет при желании легко совмещать два форка.

В Git соединение ответвленного в форк проекта с основной линией упрощается за счет поддержки масштабного внесения и импорта одного репозитория в другой и простого процесса слияния веток.

Хотя многие социальные вопросы могут оставаться, полностью распределенные репозитории снижают напряженность, уменьшая воспринимаемую важность человека, стоящего в центре проекта. Любой амбициозный разработчик может легко скопировать весь проект и его историю, а это означает, что при необходимости человека «во главе» можно заменить и продолжить разработку.

Сегодня, когда опенсорс становится образом жизни разработчиков, форки применяются для внедрения структурированного подхода на популярных платформах Git-хостинга, чтобы сообщество могло активно вносить вклад в популярные проекты.

Форки позволяют основным мейнтейнерам работать чисто и устанавливать безопасный доступ к своим проектам при сохранении стабильно действующей кодовой базы для их репозитория.

Форк проектов на GitHub

Многие участники сообщества не любят создания форков. Все потому, что в результате обычно получаются непрерывно расходящиеся копии ПО. Следует делать акцент не на неприятии концепции форков, а на количестве расхождений до того, как две линии разработки сойдутся снова.

В GitHub создание форков рассматривается более позитивно. Значительная часть GitHub построена вокруг краткосрочных форков. Любой активный разработчик может сделать копию (форк) публичного репозитория, внести в код нужные, на его взгляд, изменения, а затем предложить владельцу проекта их внедрение в основной проект. Мейнтейнеры репозитория могут просматривать все форки их репозитория в централизованном виде.

Форки, которые предлагаются обратно в основной проект, называются *пул-реквестами* (pull request). Пул-реквесты обеспечивают видимость форков и упрощают управление расходящимися ветками. К пул-реквесту можно добавить обсуждение, предоставив контекст, объясняющий, почему он был принят или отправлен создателю на доработку.

Хорошо поддерживаемые проекты отличаются частым обращением к очереди пул-реквестов. Контрибьюторы проекта должны обрабатывать всю очередь пул-реквестов, принимая, комментируя или отклоняя их. Это говорит о внимании и активной поддержке кодовой базы, а также о том, что проект имеет обширное сообщество разработчиков.

Несмотря на то что GitHub создан с целью упрощения использования форков, злоупотребление этой функцией все еще встречается. Например, изменения в кодовой базе проекта в отрыве от основной линии разработки приводят к возникновению конфликтов и затрудняют интеграцию изменений обратно в основной проект.

Однако подобное встречается редко. Это происходит из-за того, что форки видны всем участникам и они могут отклоняться от основной кодовой базы в графе коммитов, что делает процесс слияния изменений обратно в основной проект более сложным.

Итоги

Выбор правильного решения для проекта будет зависеть от ваших требований и принципов. Какой бы подход вы ни выбрали, Git реализует распределенную модель разработки. Например, даже если ваша команда централизует репозиторий, у каждого разработчика есть полноценная копия, с которой он может работать независимо. Работа происходит распределенным образом, но при этом координируется общим центральным репозиторием. Модель репозитория и модель разработки независимы от индивидуальных характеристик друг друга. В реальности вы наверняка будете работать с репозиториями на платформе хостинга Git.

Разделы этой главы, где разбирались структура репозитория, рекомендации по его публикации, работа в распределенной среде и осознание своего места в общем процессе, дадут вам достаточно знаний для принятия правильных решений.

Часть IV

Продвинутые навыки

Далее мы разберем темы, объясняющие альтернативные методы передачи изменений между репозиториями, с которыми ведется распределенная работа. Рассмотрим простой, но эффективный механизм передачи изменений в виде патчей, которые можно отправлять по электронной почте.

Вы также познакомитесь с возможностью расширения стандартного потока выполнения нескольких операций Git. Это пригодится, когда помимо стандартных операций Git в повседневной работе потребуется реализовать пользовательский поток выполнения. Хотя популярные платформы хостинга поддерживают современную функциональность потока разработки, знание этой техники позволит вам воспользоваться дополнительными приемами, когда понадобится функциональность, недоступная из коробки.

В главе 15 рассказано, как разбить проект на модули и управлять ими в отдельных репозиториях Git. Мы разберем два распространенных метода на примерах, показывающих техническую реализацию обращения к разным репозиториям Git.

В главе 16 мы покажем элегантные техники, которые помогут создавать или группировать наборы изменений в виде атомарного коммита, а также находить потерянные коммиты. Кроме того, мы объясним, как использовать команду `git rev-list`. Примером из этой главы станет инструмент `git-filter-repo`. Это универсальное средство, которое потребуется для исправления коммитов и управления ими и их историей. Рекомендуем внимательно изучить этот раздел.

ГЛАВА 13

Патчи

Git позволяет передавать результаты разработки напрямую из одного репозитория в другой, используя модели `push` и `pull`. Это происходит через различные протоколы, о которых мы говорили в разделе «Обращение к удаленным репозиториям» на с. 263 в главе 11.

Git реализует собственный протокол для обмена данными между репозиториями. Для экономии времени и места протокол передачи Git выполняет короткое рукопожатие, определяет, каких коммитов в исходном репозитории не хватает в сравнении с целевым, и наконец, передает двоичную, сжатую форму этих коммитов. Получающий репозиторий внедряет эти новые коммиты в свою локальную историю, дополняет граф коммитов и при необходимости обновляет его ветки и теги.

Для обмена результатами разработки можно использовать и HTTP. Хотя этот протокол значительно проигрывает в эффективности нативному протоколу Git, он подходит для перемещения коммитов. Оба этих протокола гарантируют неизменность передаваемых коммитов как в исходном, так и в целевом репозиториях.

На платформах хостинга Git систематическое внедрение изменений из нескольких рабочих копий репозитория в централизованный источник истины происходит практически так же, как при использовании HTTP. Новые рабочие потоки и возможности вносятся поверх внутренних базовых слоев модели передачи-получения (`push-and-pull`). В зависимости от выбранной платформы хостинга их называют *пул-реквесты* или *запросы на слияние*. О пул-реквестах подробно поговорим в главе 18.

Эти возможности и протоколы — не единственные механизмы для обмена коммитами и поддержания синхронности распределенных репозиториях. Иногда их использование оказывается нецелесообразным. Опираясь на методы из ранней эпохи разработки Unix, Git также поддерживает схему, подразумевающую создание патча и его последующее применение, при которой обмен данными обычно происходит по электронной почте.

В Git есть три специальные команды, упрощающие обмен патчами:

- `git format-patch`: генерирует патч в виде электронного письма;
- `git send-email`: отправляет патч Git по протоколу Simple Mail Transfer Protocol (SMTP);
- `git am`: применяет патч, полученный в электронном письме.

Предположим, что вы и другой или другие разработчики создаете клон общего репозитория и начинаете совместную разработку. После выполнения некоторой работы вы делаете несколько коммитов в своей копии репозитория и решаете, что пора передать эти изменения коллегам. Вы выбираете коммиты, которыми хотели бы поделиться, и коллег, с кем нужно поделиться. Поскольку патчи отправляются по почте, каждый получатель может выбирать, какие из полученных патчей применить: никакие, некоторые или все.

В этой главе рассмотрим случаи, когда могут использоваться патчи, а затем приведем примеры генерации, отправки и (для получателей) применения патча. По своей сути патч представляет собой внесение изменений из одного репозитория в другой. Поэтому мы также разберем, чем этот процесс отличается от слияния.

Зачем использовать патчи?

Несмотря на то что протокол Git намного эффективнее HTTP, есть как минимум две причины для создания патчей: техническая и социальная.

- Иногда нативный протокол Git или HTTP нельзя использовать для обмена данными между репозиториями для передачи и/или получения.

Например, корпоративный файрволл может запрещать открытие соединений с внешним сервером через протокол или порт Git. При этом SSH может быть недоступен в качестве альтернативы. Но даже если HTTP разрешен, что бывает часто, и вы можете скачивать репозитории и получать обновления, может отсутствовать возможность отправлять изменения вовне. В подобных ситуациях электронная почта оказывается удобным способом передачи патчей.

- Одно из важных преимуществ одноранговой модели разработки, как в случае с Git, — это коллаборация. Патчи, особенно те, которые отправляются в публичный список рассылки, представляют способ открытого распространения предлагаемых изменений для код-ревью.

Прежде чем применять патчи к репозиторию, другие разработчики могут обсудить, переработать, протестировать, подтвердить или отклонить присланные патчи. Поскольку патчи представляют конкретные изменения, подходящие можно сразу применить к репозиторию.

Даже если ваша среда разработки позволяет прямой обмен, вы все равно можете использовать схему «создание патча, рассылка по почте, ревью и применение», чтобы получить все преимущества коллективного ревью.

Вы можете даже применить политику разработки, при которой изменения каждого разработчика до применения через `git pull` или `git push` должны рассылаться по почте для прохождения ревью. Такой подход даст все преимущества коллективного ревью и сохранит простоту прямого получения изменений.

Но есть и другие причины использовать патчи.

По аналогии с тем, как вы можете выбрать коммит одной из ваших собственных веток и применить его к другой ветке, использование патчей позволяет выбирать коммиты из репозитория другого разработчика без необходимости полностью получать и объединять содержимое этого репозитория.

Конечно, вы можете попросить другого разработчика разместить нужные коммиты в отдельной ветке, а затем получить и объединить только эту ветку. Или получить весь репозиторий, а затем выбрать нужные коммиты из отслеживаемых веток. Но не исключены причины, по которым получать весь репозиторий нежелательно.

Если вам нужен один определенный коммит — например, отдельное исправление бага или конкретная функция, — тогда применение соответствующего патча будет разумным решением.



Собственный исходный код Git поддерживается через внесение патчей. Он постоянно обновляется через список рассылки Git, в котором состоят мейнтейнеры и контрибьюторы. Если вы хотите участвовать в рассылке, не подписываясь на нее, то отправьте вопрос или комментарий по адресу `git@vger.kernel.org`.

Исходный код Git можно найти на GitHub, хотя там лежит только зеркальная копия.

Генерация патчей

Команда `git format-patch` генерирует патч в виде сообщения электронной почты. Она создает по одному элементу для каждого указанного коммита. Вы можете указать коммиты, используя любую из техник, описанных в разделе «Идентификация коммитов» на с. 102 в главе 4.

Распространенные сценарии использования:

- Указанное число коммитов, например: `-2`.
- Диапазон коммитов, например: `main~4..main~2`.
- Один коммит, часто имя ветки, например: `origin/main`.

И хотя в основе команды `git format-patch` лежит механизм `diff`, он отличается от `git diff`.

- В то время как `git diff` генерирует один патч, совмещающий отличия всех выбранных коммитов, `git format-patch` генерирует одно сообщение электронной почты для каждого выбранного коммита.
- `git diff` не генерирует заголовки писем. Помимо фактического содержимого `diff`, `git format-patch` генерирует сообщение почты с заголовками, указывающими автора коммита, дату фиксации и сообщение журнала, описывающее изменения.



`git format-patch` и `git log` выглядят похожими. В качестве эксперимента сравните вывод двух следующих команд: `git format-patch -1` и `git log -p -1--pretty=email`.

Допустим, у вас есть репозиторий с одним файлом `file`. Содержимое этого файла представляет серию одиночных заглавных букв — от `A` до `D`. Каждая буква вносились в файл на отдельную строку и сопровождалась коммитом, содержащим соответствующее сообщение журнала:

```
$ mkdir patch-demo ; cd patch-demo
$ git init -b main
$ echo A > file
$ git add file
$ git commit -m "Add A"
[main (root-commit) 11200ff] Add A
 1 file changed, 1 insertion(+)
 create mode 100644 file
$ echo B >> file ; git commit -m "Add B" file
[main ace6f32] Add B
 1 file changed, 1 insertion(+)
$ echo C >> file ; git commit -m "Add C" file
[main b621372] Add C
 1 file changed, 1 insertion(+)
$ echo D >> file ; git commit -m "Add D" file
[main d578a1c] Add D
 1 file changed, 1 insertion(+)
```

Таким образом, в истории есть четыре коммита:

```
$ git show-branch --more=4 main
[main] Add D
[main^] Add C
[main~2] Add B
[main~3] Add A
```

Простой способ сгенерировать патчи для последних n коммитов — это использовать опцию `-n`:

```
$ git format-patch -1
0001-Add-D.patch
```

```
$ git format-patch -2
0001-Add-C.patch
0002-Add-D.patch
```

```
$ git format-patch -3
0001-Add-B.patch
0002-Add-C.patch
0003-Add-D.patch
```

По умолчанию Git генерирует каждый патч в отдельном файле с порядковым номером, который формируется на основе сообщения коммита. Команда выводит имена файлов по мере выполнения.

Можно указать, какие коммиты нужно сформатировать как патчи, используя диапазон коммитов. Предположим, вы ожидаете, что другие разработчики имеют репозитории, основанные на коммите **B** вашего репозитория. Вы хотите применить к их репозиториям все изменения, внесенные между коммитами **B** и **D**.

На основе предыдущего вывода `git show-branch` мы видим, что версия **B** именуется `main~2`, а **D** — `main`. Значит, в команде `git format-patch` вы указали бы эти имена в качестве диапазона коммитов.

Несмотря на то что вы включаете три коммита в диапазоне (**B**, **C** и **D**), в итоге получаются два сообщения электронной почты, представляющие два коммита: первый содержит отличия между **B** и **C**, а второй — между **C** и **D**. Это показано на рис. 13.1.

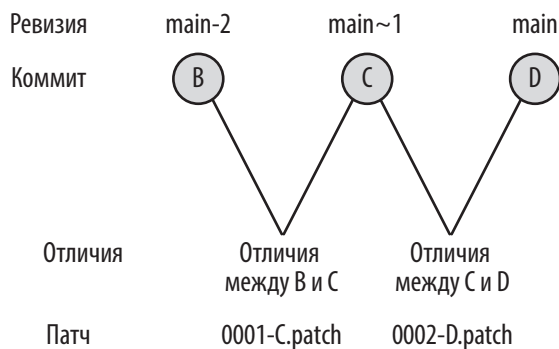


Рис. 13.1. `git format-patch` с диапазоном коммитов

Вот вывод этой команды:

```
$ git format-patch main~2..main
0001-Add-C.patch 0002-Add-D.patch
```

Каждый файл представляет отдельное письмо. При этом все письма пронумерованы в том порядке, в каком должны быть применены.

Первый патч:

```
$ cat 0001-Add-C.patch
From b621372d83c034eb8b962574d5ad01449b93f6b7 Mon Sep 17 00:00:00 2001
From: Prem Kumar Ponuthorai <ppremk@gmail.com>
Date: Wed, 5 Jan 2022 12:48:32 +0100
Subject: [PATCH 1/2] Add C

---
file | 1 +
1 file changed, 1 insertion(+)

diff --git a/file b/file
index 35d242b..b1e6722 100644
--- a/file
+++ b/file
@@ -1,2 +1,3 @@
A
B
+C
--
2.37.0
```

Второй патч:

```
$ cat 0002-Add-D.patch
From d578a1c3031e995f5234d1a25f36e470842484eb Mon Sep 17 00:00:00 2001
From: Prem Kumar Ponuthorai <ppremk@gmail.com>
Date: Wed, 5 Jan 2022 12:48:46 +0100
Subject: [PATCH] Add D

---
file | 1 +
1 file changed, 1 insertion(+)

diff --git a/file b/file
index b1e6722..8422d40 100644
--- a/file
+++ b/file
@@ -1,3 +1,4 @@
A
B C
+D
--
2.37.0
```

Усложним наш пример, добавив еще одну ветку `alt`, основанную на коммите `B`.

В то время как основной разработчик добавил в ветку `main` отдельные коммиты со строками `C` и `D`, разработчик `alt` добавил в ту же ветку коммиты (и строки) `X`, `Y` и `Z`.

```
# Создание ветки alt у коммита B
# (извлечение SHA коммита B через выполнение git log --oneline.
# Ваш SHA ID может отличаться от нашего.)
```

```
$ git checkout -b alt ace6f32
# Переключение на новую ветку 'alt'.
$ echo X >> file ; git commit -m "Add X" file
[alt 43250ac] Add X
  1 file changed, 1 insertion(+)
$ echo Y >> file ; git commit -m "Add Y" file
[alt fd1ca23] Add Y
  1 file changed, 1 insertion(+)
$ echo Z >> file ; git commit -m "Add Z" file
[alt cc9993c] Add Z
  1 file changed, 1 insertion(+)
```

Итоговый граф коммитов представлен на рис. 13.2.

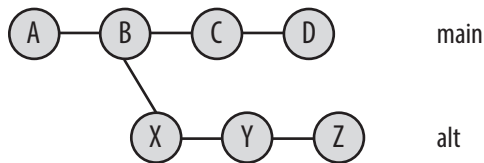


Рис. 13.2. Граф патчей с веткой alt



Вы можете нарисовать ASCII-граф со всеми своими ссылками, используя опцию `--all`:

```
$ git log --graph --pretty=oneline --abbrev-commit --all
* cc9993c (HEAD -> alt) Add Z
* fd1ca23 Add Y
* 43250ac Add X
| * d578a1c (main) Add D
| * b621372 Add C
|/
* ace6f32 Add B
* 11200ff Add A
```

Затем разработчик `main` влил ветку `alt`, начиная с коммита Z, в ветку `main` у коммита D, сформировав коммит слияния E. В завершение он внес еще одно изменение, которое добавило в ветку `main` коммит F:

```
$ git checkout main
$ git merge alt

# Разрешите конфликты удобным вам способом.
# Мы использовали последовательность: A, B, C, D, X, Y, Z

$ git add file
$ git commit -m "Add all lines"
[main 5727615] Add all lines

$ echo F >> file ; git commit -m "Add F" file
[main 9ab717f] Add F
  1 file changed, 1 insertion(+)
```

Итоговый граф коммитов показан на рис. 13.3.

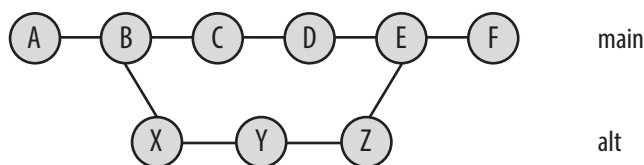


Рис. 13.3. История двух веток

Представление истории коммитов выглядит так:

```

$ git show-branch --more=10
! [alt] Add Z
* [main] Add F
--
* [main] Add F
+* [alt] Add Z
+* [alt^] Add Y
+* [alt~2] Add X
* [main~2] Add D
* [main~3] Add C
+* [main~4] Add B
+* [main~5] Add A
  
```

Если у вас сложное дерево ревизий, то внесение патчей может происходить на удивление гибко. Рассмотрим пример.

Будьте осторожны при указании диапазона коммитов, особенно когда он включает слияния. В рассмотренном примере ожидается, что диапазон D..F охватит два коммита, E и F. Так оно и есть. Но коммит E несет все содержимое, включенное в него из *всех* вливаемых веток:

```

# Форматирование патчей D..F
$ git format-patch main~2..main
0001-Add-X.patch
0002-Add-Y.patch
0003-Add-Z.patch
0004-Add-F.patch
  
```

Диапазон коммитов определяется как включающий все коммиты, ведущие к *конечной точке* диапазона, и исключающий все коммиты, ведущие к *стартовой* точке диапазона включительно. В случае D..F это означает, что включаются все коммиты, относящиеся к F (каждый коммит в примере графа), но все коммиты, ведущие к D и включающие его (A, B, C и D), исключаются. Сам коммит слияния патч не генерирует.



Перед генерированием патчей выполните `git rev-list --no-merges -v от..до`, чтобы проверить набор коммитов, для которых это нужно сделать.

ПОДРОБНОЕ ОБЪЯСНЕНИЕ РАЗРЕШЕНИЯ ДИАПАЗОНА

Чтобы определить диапазон, выполните следующие действия. Начните с коммита конечной точки и включите его. Двигайтесь назад через каждый родительский коммит, относящийся к нему, и также включайте их. Рекурсивно включите родителей всех уже включенных коммитов. Закончив с коммитами, связанными с конечной точкой, вернитесь и начните со стартовой точки. Удалите стартовую точку. Затем двигайтесь назад через каждый родительский коммит, относящийся к ней, и также удаляйте их. Рекурсивно удалите родителей всех удаленных коммитов.

На рис. 13.3 отражен случай с нашим диапазоном D..F. Начните с коммита F и включите его. Вернитесь к родительскому коммиту E и тоже включите его. Затем посмотрите на E и добавьте его родителей D и Z. Теперь рекурсивно включайте родительские коммиты D, а именно C, B и A. Возвращаясь по линии Z, рекурсивно включите Y, X, а затем снова B и, наконец, снова A. (На самом деле B и A не добавляются повторно, рекурсия может прекращаться, когда сталкивается с уже включенным узлом.) Теперь включены все коммиты. Вернитесь и начните со стартовой точки D, удалите ее. Удалите также ее родителя C, затем рекурсивно удалите его родителя B, а затем и его родителя — A.

Останется набор F E Z Y X. Однако E — это коммит слияния, а значит, его нужно удалить. В итоге остаются F Z Y X, представляющие зеркальный вариант сгенерированного набора.

Можно обращаться к одному коммиту как к варианту диапазона `git format-patch`. Но Git интерпретирует такую команду контринтуитивно.

Обычно Git рассматривает аргумент в виде одного коммита как «все коммиты, ведущие и относящиеся к этому коммиту». В противоположность этому, `git format-patch` рассматривает параметр с одним коммитом так, будто вы указали диапазон `commit..HEAD`. Команда использует ваш коммит в качестве стартовой точки и получает HEAD в виде конечной точки. Таким образом, сгенерированная серия патчей неявно находится в контексте активной на данный момент ветки.

В этом примере ветка `main` активная, и патч формируется с указанием коммита A. Создаются все семь патчей:

```
$ git branch
  alt
* main

# Из коммита A.
$ git format-patch main~5
0001-Add-B.patch
0002-Add-C.patch
```

```
0003-Add-D.patch
0004-Add-X.patch
0005-Add-Y.patch
0006-Add-Z.patch
0007-Add-F.patch
```

Но когда происходит переключение на ветку **alt** и в команде указывается тот же коммит A, используются только те патчи, которые относятся к вершине этой ветки:

```
$ git checkout alt
Switched to branch "alt"
```

```
$ git branch
* alt
  main
```

```
$ git format-patch main~5
0001-Add-B.patch
0002-Add-X.patch
0003-Add-Y.patch
0004-Add-Z.patch
```

Несмотря на то что указан коммит A, патч для него не создается. Корневой коммит особенный: для него нет зафиксированного состояния, относительно которого можно было бы вычислить разницу. И патч для него представляет добавление всего исходного содержимого.

Если вы хотите генерировать патчи для каждого коммита вплоть до именованного *end-commit*, включая исходный корневой коммит, используйте опцию **--root**:

```
$ git format-patch --root <end-commit-SHA>
```

Исходный коммит генерирует патч так, будто каждый файл был добавлен на основе **/dev/null** (так обозначается добавленный или удаленный файл):

```
$ git format-patch --root main~5
$ cat 0001-Add-A.patch
From 11200ff80b78bd777d8832b112eaddcdd8b899db Mon Sep 17 00:00:00 2001
From: Prem Kumar Ponuthurai
<ppremk@gmail.com>
Date: Wed, 5 Jan 2022 12:47:17 +0100
Subject: [PATCH] add file A

---
file | 1 +
1 file changed, 1 insertion(+)
create mode 100644 file

diff --git a/file b/file
new file mode 100644
index 0000000..f70f10e
--- /dev/null
```

```
+++ b/file
@@ -0,0 +1 @@
+A
--
2.37.0
```

Обращение к одному коммиту так, будто вы указали `commit..HEAD`, может показаться необычным, но такой подход ценен в определенном сценарии. Когда вы указываете `commit` в ветке, отличной от текущей активной ветки, команда выдает патчи, относящиеся к текущей ветке, но не к названной. Она генерирует набор патчей, которые могут синхронизировать другую ветку с текущей.

Допустим, что вы переключились на ветку `main`:

```
$ git branch
  alt
* main
```

Теперь вы указываете ветку `alt` в виде параметра `commit`:

```
$ git format-patch alt
0001-Add-C.patch
0002-Add-D.patch
0003-Add-F.patch
```

Патчи для коммитов C, D и F — это точно такой же набор патчей в ветке `main`, но не в `alt`.

Сила этой команды с единственным параметром коммита становится очевидна, когда этот коммит — `HEAD`-ссылка ветки отслеживания из чужого репозитория.

Например, если вы клонируете репозиторий Алисы и ваша ветка `main` основана на ветке `main` Алисы, то вы получите ветку отслеживания с именем `alice/main`.

После того как вы внесете несколько коммитов в свою ветку `main`, выполнение команды `git format-patch alice/main` сгенерирует набор патчей, которые нужно будет отправить, чтобы репозиторий Алисы гарантированно включал все содержимое вашей `main`. Ее репозиторий может содержать и *другие* изменения из других источников, но в данном случае это неважно. Вы выделили набор изменений из своего репозитория (ветки `main`), который явно отсутствует в репозитории Алисы.

Таким образом, `git format-patch` разработана для создания патчей из коммитов, находящихся в ветке разработки вашего репозитория, но отсутствующих в вышестоящем репозитории.

Патчи и топологическая сортировка

Патчи, генерируемые командой `git format-patch`, выпускаются в топологическом порядке. Для конкретного коммита патчи от всех родительских коммитов

генерируются и выпускаются до патча самого коммита. Так обеспечивается правильный порядок создаваемых патчей, но он не обязательно уникален — для конкретного графа может быть несколько корректных порядков патчей.

Рассмотрим некоторые из возможных порядков генерации патчей, которые в случае их применения по очереди привели бы к получению корректного репозитория. В примере 13.1 показано несколько возможных топологических порядков сортировки для коммитов нашего образца графа.

Пример 13.1. Некоторые топологические порядки сортировки

A B C D X Y Z E F

A B X Y Z C D E F

A B C X Y Z D E F

A B X C Y Z D E F

A B X C Y D Z E F

Несмотря на то что создание патчей регулируется топологической сортировкой выбранных узлов в графе коммитов, они будут созданы на основе только части этих узлов.

Первый порядок из примера 13.1 — тот, который Git выбрал для `git format-patch main~5`. Поскольку A — это первый коммит в этом диапазоне и опция `--root` не используется, патч для него не создается. Коммит E представляет слияние, поэтому и для него не создается патч. Таким образом, патчи генерируются в последовательности B C D X Y Z F.

Какой бы порядок применения патчей Git ни выбрал, важно понимать, что система *линеаризует* все выбранные коммиты вне зависимости от степени сложности или разветвленности исходного графа.

Отправка патчей по электронной почте

После генерации патча или их серии следующий шаг — отправка их другому разработчику на ревью. Конечная цель процесса — выбор патча разработчиком или вышестоящим мейнтейнером и его применение к другому репозиторию.

Отформатированные патчи обычно предназначены для отправки по почте путем их прямого импорта в Mail User Agent (MUA, почтовый агент) или с помощью команды `Git git send-email`. Использовать `git send-email` необязательно — это лишь один из вариантов. В следующем разделе вы увидите, что есть другие инструменты, которые используют файл патча напрямую.

Чтобы отправить сгенерированный патч, можно выполнить `git send-email` и указать почтовой программе непосредственно на патчи или вложить их в письмо.

Рассмотрим использование `git send-email`. Патч `0001-Add-A.patch` отправляется в список рассылки `devlist@example.org`:

```
$ git send-email --to devlist@example.org 0001-Add-A.patch
0001-Add-A.patch
(mbox) Adding cc: Prem Kumar Ponuthorai <ppremk@gmail.com> from line 'From: Prem
Kumar Ponuthorai
<ppremk@gmail.com>'
```

```
From: Prem Kumar Ponuthorai <ppremk@gmail.com>
To: devlist@example.org
Cc: Prem Kumar Ponuthorai <ppremk@gmail.com>
Subject: [PATCH] Add A
Date: Wed, 5 Jan 2022 15:13:38 +0100
Message-Id: <20220105141338.79674-1-ppremk@gmail.com>
X-Mailer: git-send-email 2.33.0
MIME-Version: 1.0
Content-Transfer-Encoding: 8bit
```

The Cc list above has been expanded by additional addresses found in the patch commit message. By default `send-email` prompts before sending whenever this occurs. This behavior is controlled by the `sendemail.confirm` configuration setting.

For additional information, run `'git send-email --help'`. To retain the current behavior, but squelch this message, run `'git config --global sendemail.confirm auto'`.

Send this email? ([y]es|[n]o|[e]dit|[q]uit|[a]ll): y

```
OK. Log says:
Sendmail: /usr/sbin/sendmail -i devlist@example.org ppremk@gmail.com
From: Prem Kumar Ponuthorai <ppremk@gmail.com>
To: devlist@example.org
Cc: Prem Kumar Ponuthorai <ppremk@gmail.com>
Subject: [PATCH] add file A
Date: Wed, 5 Jan 2022 15:13:38 +0100
Message-Id: <20220105141338.79674-1-ppremk@gmail.com>
X-Mailer: git-send-email 2.33.0
MIME-Version: 1.0
Content-Transfer-Encoding: 8bit
```

Result: OK

Есть множество вариантов использования или обхода различных проблем и особенностей SMTP. При этом важно точно знать SMTP-сервер и порт. Как правило, это традиционная программа `sendmail` или действительный исходящий SMTP-хост, например `smtp.my-isp.com`.



Не настраивайте открытые SMTP-сервера ретрансляции только для того, чтобы отправить письмо Git. Это создаст проблемы со спамом.

У команды `git send-email` есть много опций, описанных на соответствующей странице руководства.

Возможно, вам будет удобнее записывать SMTP-информацию в глобальном файле конфигурации, установив, например, значение `sendemail.smtpserver` и `sendemail.smtpserverport` и используя такие команды:

```
$ git config --global sendemail.smtpserver smtp.my-isp.com
$ git config --global sendemail.smtpserverport 465
```

Если почтовый агент позволяет, вы можете напрямую импортировать весь файл или каталог патчей в каталог почты. Это упростит отправку больших или сложных серий патчей.

В примере ниже традиционный каталог почты в стиле `mbox` создается с помощью `format-patch`, а затем напрямую импортируется в `mutt`, где сообщение может быть адресовано и отправлено. `mbox` — это формат хранения почты в системах Unix, `mutt` — текстовый почтовый клиент для этих систем.

```
$ git format-patch --stdout main~2..main > mbox
```

```
$ mutt -f mbox
```

```
q:Quit d:Del u:Undel s:Save m:Mail r:Reply g:Group ?:Help
 1 N   Dec 29 Jon Loeliger   ( 15) [PATCH] X
 2 N   Dec 29 Jon Loeliger   ( 16) [PATCH] Y
 3 N   Dec 29 Jon Loeliger   ( 16) [PATCH] Z
 4 N   Dec 29 Jon Loeliger   ( 15) [PATCH] F
```

Последние два механизма, использование `send-email` и прямой импорт каталога почты, являются предпочтительными техниками отправки писем, поскольку оба надежны и не повреждают отформатированное содержимое патчей. При использовании одной из этих техник вы гораздо реже услышите от разработчика жалобы на разрывы строк.

С другой стороны, вы можете столкнуться с необходимостью напрямую включить сгенерированный патч в новое письмо в популярных почтовых агентах вроде Gmail, Thunderbird или Microsoft Outlook. В таких случаях риск нарушить форматирование патча уже намного выше. Обязательно отключите все виды HTML-форматирования и отправьте простой текст в формате ASCII.

В зависимости от возможностей получателя по обработке почты или от политик списка рассылки вы можете или не можете использовать вложение файлов в письмо. Как правило, вложение — это более простой и правильный подход, что упрощает дальнейшее ревью. Но если патч вложен, то некоторые заголовки, генерируемые командой `git format-patch`, придется обрезать, оставив в теле письма только `From:` и `Subject:`.



Если вы часто вкладываете патчи в виде текстовых файлов в письма и вас раздражает, что надо удалять лишние поля, то используйте команду `git format-patch --pretty=format:%s%n%n%b commit`. Ее можно настроить в виде глобального псевдонима Git, как описано в разделе «Файлы конфигурации» на с. 42 в главе 1.

Независимо от способа отправки при получении патчи должны быть идентичны исходным файлам, несмотря на дополнительные поля в письме.

Идентичность патча до и после отправки неслучайна. Ключом к успешной работе будет использование простого текста, чтобы почтовый агент не мог изменить формат патча при разрыве строки. Если вы можете исключить подобные вмешательства, то патч будет корректным независимо от того, сколько агентов пересылки почты передадут его данные.



Если ваш почтовый агент переносит строки в исходящих письмах, используйте `git send-email`.

Есть много настроек, позволяющих контролировать генерацию полей в письмах с патчами. В вашем проекте наверняка действуют соглашения, которым вы должны следовать. Например, если вы регулярно добавляете поля в письмо с патчем после его генерации, то обратите внимание на опцию `format.headers`.

Серию патчей можете передать в общий каталог, дополнив команду `git format-patch` опцией `-o имя каталога`. После этого можно использовать `git send-email имя каталога` для их одновременной отправки. Чтобы написать сопроводительное письмо к серии патчей, используйте `git format-patch --cover-letter` или `git send-email --compose`.

Есть также опции, позволяющие реализовать разные социальные аспекты списков рассылки. Например, используйте `--cc` для добавления альтернативных получателей, добавления или исключения каждого адреса `Signed-off-by:` в качестве получателя `Cc:` или для выбора того, как серия патчей должна быть упорядочена в списке.

Подробнее об этих опциях читайте в документации `git send-email`.

Применение патчей

В Git есть две команды для применения патчей. Более высокоуровневая команда `git am` частично реализуется через низкоуровневую `git apply`.

Команда `git apply` — это «рабочая лошадка» для процедуры применения патча. Она получает вывод в стиле `git diff` или `diff` и применяет их к файлам в вашем

рабочем каталоге. Несмотря на некоторые отличия, она выполняет ту же роль, что и команда `patch` Ларри Уолла (Larry Wall) (программа `patch` для систем Unix).

Поскольку `diff` содержит только построчные правки без какой-либо дополнительной информации (автора, даты или сообщения журнала), эта команда не может выполнить коммит и записать в журнал изменение в вашем репозитории. Таким образом, когда `git apply` завершается, файлы в рабочем каталоге остаются измененными. (В некоторых случаях она может использовать или изменять индекс.)

А вот патчи, отформатированные командой `git format-patch` до либо после их отправки, содержат дополнительную информацию для внесения и регистрации нужного коммита в вашем репозитории. И хотя `git am` настроена на получение патчей, генерируемых `git format-patch`, она также обрабатывает и другие патчи, если они следуют тем же простым правилам форматирования¹. Обратите внимание: `git am` создает коммиты в текущей ветке.

Завершим пример, используя репозиторий из раздела «Генерация патчей» на с. 332 (создание патча, отправка по почте, ревью и применение). Допустим, один разработчик создал полноценный набор патчей от `0001-Add-B.patch` до `0007-Add-F.patch` и отправил или иным образом сделал его доступным для другого разработчика. Тот, в свою очередь, имеет более раннюю версию репозитория и хочет применить полученные патчи.

Сначала рассмотрим наивный подход, показывающий типичные проблемы, которые невозможно разрешить. Затем разберем более грамотный подход.

Патчи из исходного репозитория:

```
$ git format-patch -o /tmp/patches main~5
/tmp/patches/0001-Add-B.patch
/tmp/patches/0002-Add-C.patch
/tmp/patches/0003-Add-D.patch
/tmp/patches/0004-Add-X.patch
/tmp/patches/0005-Add-Y.patch
/tmp/patches/0006-Add-Z.patch
/tmp/patches/0007-Add-F.patch
```

Второй разработчик мог получить эти патчи по электронной почте и сохранить на своем диске. Также их могли поместить напрямую в общую файловую систему.

Построим начальный репозиторий в качестве целевого для этой серии патчей. (Как именно он будет сделан — не так важно. Например, его могут клонировать из начального репозитория.) Для долгосрочного успеха важно, чтобы оба репозитория содержали одинаковые файлы.

¹ К моменту, когда вы начнете следовать правилам со страницы инструкции `git am` (разграничение From:, Subject:, Date: и содержимого патча), то наверняка будете называть это сообщением почты.

Клонируем имеющийся репозиторий и откатим его до содержания того же файла `file` с изначальным содержимым `A`. Так мы получим такое же содержимое, какое было в исходном репозитории:

```
$ cd /tmp
$ git clone ~/patch-demo am
Cloning into 'am'...
done.
$ cd am

# Получаем SHA первого коммита с помощью git log --
# затем откатываем репозиторий до этого SHA.
$ git reset --hard 11200ff

# Отключаемся от вышестоящего репозитория для создания патчей
# (это необязательно, но почему бы и нет)
$ git remote remove origin
$ git log --oneline
11200ff (HEAD -> main) Add A
```

Прямое использование `git am` вскрывает проблемы:

```
$ git am /tmp/patches/*
Applying: Add B
Applying: Add C
Applying: Add D
Applying: Add X
error: patch failed: file:1
error: file: patch does not apply
Patch failed at 0004 Add X
hint: Use 'git am --show-current-patch=diff' to see the failed patch
When you have resolved this problem, run "git am --continue".
If you prefer to skip this patch, run "git am --skip" instead.
To restore the original branch and stop patching, run "git am --abort".
```

Это сложный вид отказа, и вы можете не понять, что делать дальше. Хорошее решение — внимательно изучить происходящее:

```
$ git show-branch --more=10
[main] Add D
[main^] Add C
[main~2] Add B
[main~3] Add A
```

Вполне ожидаемо. В рабочем каталоге не осталось ни одного грязного файла, и Git успешно применил патчи вплоть до `D`.

Изучение патча и файлов, на которые он влияет, часто помогает прояснить проблему. Воспользуемся подсказкой, которую нам дал Git:

```
$ git am --show-current-patch=diff
---
file | 1 +
```

```
1 file changed, 1 insertion(+)
```

```
diff --git a/file b/file
index 35d242b..7f9826a 100644
--- a/file
+++ b/file
@@ -1,2 +1,3 @@
 A
 B
+X
--
2.37.0
```

```
$ cat file
A
B
C
D
```



При выполнении команды `git am` создается каталог `.git/rebase-apply`. Он содержит различную информацию для всей серии патчей и отдельных частей (автора, сообщения журнала и т. д.) каждого из них.

Здесь сложное место. В файле четыре строки, но патч применяется к той версии этого файла, где их всего две. Как показал вывод команды `git am`, патч не применяется:

```
error: patch failed: file:1
error: file: patch does not apply
Patch failed at 0004 Add X.
```

Вы-то знаете, что целью является создание файла, где все буквы расположены по порядку. Но Git не может определить это автоматически. Системе не хватает контекста для правильного разрешения конфликта.

Как и в случае других конфликтов файла, `git am` предлагает несколько вариантов:

```
When you have resolved this problem run "git am --continue".
If you would prefer to skip this patch, run "git am --skip" instead.
To restore the original branch and stop patching run "git am --abort".1
```

К сожалению, в этом случае нет даже конфликта содержимого файла, который можно было бы разрешить.

¹ Если вы разрешили проблему, выполните `git am --continue`.

Если вы предпочитаете пропустить этот патч, выполните `git am --skip`.

Чтобы восстановить исходную ветку и прекратить применение патча, выполните `git am --abort`. — *Примеч. пер.*

Тогда можно попробовать пропустить патч X, как предлагает система:

```
$ git am --skip
Applying: Add Y
error: patch failed: file:1
error: file: patch does not apply
Patch failed at 0005 Add Y
hint: Use 'git am --show-current-patch=diff' to see the failed patch1
When you have resolved this problem, run "git am --continue".
If you prefer to skip this patch, run "git am --skip" instead.
To restore the original branch and stop patching, run "git am --abort".
```

Но как и в случае патча Y, все последующие тоже не пройдут. Очевидно, что текущий подход не позволит применить эту серию патчей.

Можно попробовать восстановить состояние, но это нелегко без знания исходного описания ветвления, которое привело к попытке выполнить серию патчей с помощью `git am`. Коммит X был применен к новой ветке, происходящей из коммита B. Это означает, что патч X сработал бы корректно, если применялся бы повторно к этому состоянию коммита. Это можно проверить: откатить репозиторий к моменту присутствия только коммита A, очистить каталог `rebase-apply`, применить коммит B, используя `git am /tmp/patches/0001-B.patch`, и убедиться, что коммит X применится тоже.

```
# Откат до коммита A
$ git reset --hard main~3
HEAD is now at 11200ff Add A

# Удаление каталога .git/rebase-apply.
$ rm -rf .git/rebase-apply/

$ git am /tmp/patches/0001-Add-B.patch
Applying: Add B

$ git am /tmp/patches/0004-Add-X.patch
Applying: Add X
```



Упрощение процесса восстановления после неудачного, испорченного или безнадежного применения `git am` и восстановление исходной ветки можно свести к команде `git am --abort`.

Успешное применение `0004-Add-X.patch` к коммиту B подскажет, что делать дальше. Однако вы не сможете применить патчи X, Y и Z, поскольку последующие патчи C, D и F не применятся. Не переживайте по поводу создания точной структуры исходной ветки. Даже если вы захотите ее воссоздать, то как узнаете, какой была ее изначальная структура?

¹ Подсказка: используйте `"git am --show-current..."`, чтобы увидеть непрошедший патч.

Определение базового файла, к которому можно применить `diff`, — сложная проблема, для которой Git предлагает простое техническое решение. Если вы внимательно изучите патч или `diff`, сгенерированный Git, то увидите дополнительную информацию, которая не входит в традиционную сводку `diff` в Unix. Дополнительная информация, которую Git предоставляет для патча `0004-Add-X.patch`, показана в примере 13.2.

Пример 13.2. Новый контекст патча в файле `0004-Add-X.patch`

```
diff --git a/file b/file index 35d242b..7f9826a 100644
--- a/file
+++ b/file
```

После строки `diff --git a/file b/file` Git добавляет новую строку `index 35d242b..7f9826a 100644`. Эта информация позволяет уверенно ответить на вопрос, каково было исходное состояние, к которому применяется патч.

Первое число в строке `index` — `35d242b` — представляет хеш SHA1 blob-объекта в объектном хранилище Git, к которому применяется патч. `35d242b` — это файл в своем текущем виде с двумя строками:

```
$ git show 35d242b
A
B
```

Это именно та версия `file`, к которой применяется эта часть патча `X`. Если данная версия находится в репозитории, тогда Git может применить к ней патч.

Такой механизм — наличие текущей версии файла, его альтернативной версии и обнаружение оригинальной версии файла, к которой применяется патч, — называется *трехсторонним слиянием* (*three-way-merge*). Git может воссоздать этот сценарий, используя команду `git am` с опцией `-3` или `--3way`.

Устраним последствия нашей неудачной попытки. Откатим репозиторий к состоянию с одним коммитом `A` и заново применим серию патчей:

```
# При необходимости избавляемся от временного контекста "git am".
$ rm -rf .git/rebase-apply/

# Используем "git log" для обнаружения коммита A, это был SHA1 573956b.
# Откатываемся к моменту присутствия только коммита A. Ваш SHA1 будет другим.
$ git reset --hard 11200ff
HEAD is now at 11200ff Add A

$ git show-branch --more=10
[main] Add A
```

Используя опцию `-3`, применяем серию патчей:

```
$ git am -3 /tmp/patches/*
Applying: Add B
Applying: Add C
Applying: Add D
```

```
Applying: Add X
Using index info to reconstruct a base tree...
M   file
Falling back to patching base and 3-way merge...
Auto-merging file
CONFLICT (content): Merge conflict in file
error: Failed to merge in the changes.
Patch failed at 0004 Add X
hint: Use 'git am --show-current-patch=diff' to see the failed patch
When you have resolved this problem, run "git am --continue".
If you prefer to skip this patch, run "git am --skip" instead.
To restore the original branch and stop patching, run "git am --abort".
```

Уже лучше!

Как и прежде, простая попытка пропатчить файл провалилась, но вместо выхода система Git применила трехстороннее слияние. Система поняла, что может выполнить слияние, но конфликт остается, поскольку перекрывающиеся строки были изменены двумя разными способами.

Git не может корректно разрешить этот конфликт, поэтому команда `git am -3` приостанавливается. Вам нужно его разрешить, прежде чем продолжить.

Стратегия оценки происходящего поможет понять, что делать:

```
$ git status
On branch main
You are in the middle of an am session.
  (fix conflicts and then run "git am --continue")
  (use "git am --skip" to skip this patch)
  (use "git am --abort" to restore the original branch)

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   file

no changes added to commit (use "git add" and/or "git commit -a")
```

Как уже было указано, файл `file` все еще требует разрешения конфликта.

Содержимое показывает маркеры конфликта при слиянии, который нужно разрешить с помощью редактора:

```
$ cat file
A
B
<<<<<< HEAD
C
D
=====
X
>>>>>> Add X
# Исправить конфликты в "file"
$ vi file
```

```
$ cat file
A
B
C
D
X
```

После разрешения конфликта и очистки возобновляем команду `git am -3`:

```
$ git am -3 --continue
Applying: Add X
You still have unmerged paths in your index.
You should 'git add' each file with resolved conflicts to mark them as such.
You might run `git rm` on a file to accept "deleted by them" for it.
When you have resolved this problem, run "git am --continue".
If you prefer to skip this patch, run "git am --skip" instead.
To restore the original branch and stop patching, run "git am --abort".
```

Вы не забыли применить `git add`? Мы забыли!

```
$ git add file
$ git am -3 --continue
Applying: Add X
Applying: Add Y
Using index info to reconstruct a base tree...
M   file
Falling back to patching base and 3-way merge...
Auto-merging file
Applying: Add Z
Using index info to reconstruct a base tree...
M   file
Falling back to patching base and 3-way merge...
Auto-merging file
Applying: Add F
```

Наконец успех!

```
$ cat file
A
B
C
D
X
Y
Z
F
```

```
$ git show-branch --more=10
[main] F
[main^] Z
[main~2] Y
[main~3] X
[main~4] D
[main~5] C
```

```
[main~6] B  
[main~7] A
```

Применение этих патчей не привело к созданию копии структуры ветки из исходного репозитория. Все патчи выполнялись в линейной последовательности, что отразилось в истории коммитов ветки `main`.

Чтобы убедиться в этом, выполним команду `git log --graph --oneline`:

```
$ git log --graph --oneline  
* 37e3f43 (HEAD -> main) Add F  
* 3a2d1df Add Z  
* 3484b10 Add Y  
* f18328e Add X  
* 68cf338 Add D  
* a9e1a1b Add C  
* 06d2fad Add B  
* 11200ff Add A
```

Обратите внимание: `Author` и `AuthorDate` патча будут относиться к исходному коммиту и патчу. А данные получателя патча будут отражать действия по применению этого патча и его фиксации в соответствующей ветке и репозитории.

```
# Демопатч был применен на следующий день  
# после создания исходного примера.  
$ git log --pretty=fuller -1 a9e1a1b  
commit a9e1a1bad95082f597b781bb4b22694eaacd9698  
Author: Prem Kumar Ponuthorai <ppremk@gmail.com>  
AuthorDate: Wed Jan 5 12:48:32 2022 +0100  
Commit: Prem Kumar Ponuthorai <ppremk@gmail.com>  
CommitDate: Thu Jan 6 16:30:03 2022 +0100
```

Add C

Плохие патчи

Создание надежного и идентичного контента в многочисленных распределенных репозиториях по всему миру — непростая задача. Не удивительно, что идеальный патч может быть испорчен сбоями в работе почтовых сервисов. Поскольку электронная почта и другие механизмы передачи данных бывают ненадежными, именно Git гарантирует, что процесс отправки патча, его ревью и применение пройдут без потерь и воспроизведут идентичное содержимое на всех этапах.

Сбои в патчах происходят по разным причинам, включая несовпадающие инструменты и подходы. Пожалуй, самая распространенная причина — несоблюдение точных характеристик обработки строк оригинального содержимого. Обычно это проявляется в виде переносов строк из-за обработки текста почтовым агентом отправителя или получателя либо промежуточными агентами передачи писем. К счастью, формат патча имеет внутренние проверки согласованности, которые не позволяют такому сбою повредить репозиторий.

Если вы работаете с платформами хостинга Git, проблем быть не должно. Такие платформы предоставляют множество возможностей, позволяющих систематически и легко интегрировать изменения, вносимые в распределенные репозитории и между командами разработчиков, в централизованный единый источник истины для проекта — вышестоящий репозиторий.

Патчи и слияния

Git может обрабатывать ситуации, когда в одном репозитории были применены патчи и получены изменения. Несмотря на то что коммит в получающем репозитории отличается от коммита в исходном, из которого создавался патч, Git может сопоставить их содержимое для решения проблемы.

Позднее, к примеру, последующие `diff` не покажут каких-либо изменений в содержимом. Сообщение журнала и информация об авторе останутся такими же, что указаны в письме с патчем, но данные вроде даты и SHA1 будут отличаться.

Прямое получение и вливание ветки со сложной историей приведут к созданию другой истории в репозитории-получателе. Она будет отличаться от истории, полученной в результате последовательности патчей. Один из эффектов создания последовательности патчей для сложной ветки заключается в топологической сортировке графа в виде линеаризованной истории. Таким образом, применение ее к другому репозиторию даст линеаризованную историю, которой не было изначально. Также при создании патчей *коммиты слияния* не генерируются в составе операции внесения изменений.

В зависимости от вашего стиля разработки и намерений наличие исходной истории разработки, линеаризованной внутри получающего репозитория, может стать проблемой или нет. В худшем случае вы утрачиваете всю историю ветвления, которая привела к этой последовательности патчей. В лучшем случае — не имеет значения, как вы получили такую последовательность.

Итоги

Глава началась с обсуждения причин использования патчей. Создание патчей весьма полезно, когда вы хотите поделиться изменениями с коллегами, которые не имеют прямого доступа к платформам хостинга Git, но работают с единой кодовой базой общей версии репозитория вашего проекта. С помощью описанных выше техник для генерации и применения патчей вы сможете обмениваться коммитами между репозиториями. Если вы используете патчи вместо слияний, мы рекомендуем разобраться, как это отразится на вашей команде в целом, поскольку такое решение может повлиять на запись истории репозитория.

ГЛАВА 14

Хуки

Хуки (hook) позволяют настраивать стандартные операции Git. С их помощью можно организовать выполнение одного или более скриптов в момент возникновения в репозитории конкретного события, например коммита или патча. Событие разбивается на несколько этапов, к каждому из которых можно привязать пользовательский скрипт. Впоследствии при возникновении события в начале каждого этапа вызывается соответствующий скрипт.

Хуки относятся к конкретному репозиторию и действуют только в его контексте. При клонировании они не копируются. Хуки, настроенные в частном репозитории, не распространяются на клон этого репозитория и не изменяют его поведение. Если ваш рабочий процесс предписывает использование хуков в личном репозитории каждого разработчика, нужно организовать копирование каталога `.git/hooks` другим методом (не клонирования).

Не путайте этот процесс с инициализацией нового репозитория. Когда указывается команда `git init`, Git копирует доступные хуки в новый репозиторий. К ним могут относиться стандартные хуки, предоставляемые Git, или иные пользовательские хуки, указанные в файле конфигурации через механизм *каталога шаблонов*.

Мы начнем с разбора доступных в Git типов хуков и их установки, после чего познакомимся с созданием простого хука. Вы также узнаете, как и когда использовать хуки для изменения операций Git.

Типы хуков

Хук выполняется в контексте вашего текущего, локального репозитория либо в контексте удаленного репозитория. Например, получение данных в ваш репозиторий из удаленного и выполнение локального коммита могут запускать выполнение локальных хуков. Передача изменений в удаленный репозиторий может запускать выполнение хуков в нем. Соответственно, их называют хуками *клиентской* и *серверной* стороны (где слово «клиент» относится к локальному

репозиторию, а «сервер» — к удаленному). Обычно серверные хуки — это те, которые активируются в централизованной или вышестоящей копии репозитория.

Большинство хуков делятся на две категории:

- *прехук (pre-hook)*: выполняется до завершения действия и используется для утверждения, отклонения или корректировки изменения, прежде чем оно будет применено;
- *постхук (post-hook)*: выполняется после завершения действия и используется для активации уведомлений (например, почтовых) или запуска дополнительной обработки, например сборки или закрытия бага.

Согласно общему правилу, если прехук завершается с ненулевым статусом (условное обозначение сбоя), действие отменяется. При этом выходной статус постхука обычно игнорируется, поскольку он уже не может влиять на результат или завершение действия.

Об использовании хуков

Разработчики рекомендуют использовать хуки с осторожностью. Их нужно задействовать только в крайнем случае, когда другие способы не приносят результат. Например, если нужно указать опцию при каждом коммите, переключении на файл или создании ветки, хук необязателен. Это реализуется с помощью псевдонима (см. раздел «Файлы конфигурации» на с. 42 в главе 1) или скриптов оболочки, позволяющих дополнить команды `git commit`, `git checkout` и `git branch` соответственно¹.

На первый взгляд хук может показаться простым решением, но оно имеет последствия.

- Хук изменяет поведение Git. Если хук выполняет нестандартную операцию, другие знакомые с Git разработчики могут столкнуться с неожиданностями при использовании вашего репозитория.
- Хук может замедлять операции, которые без него выполнились бы быстро. Часто разработчиков привлекает идея использовать хуки для юнит-тестов до отправки коммита, что замедляет этот процесс. В Git коммит должен быть быстрой операцией, чтобы ее делали часто для предотвращения потери данных. Если замедлить коммиты, то работать с Git станет не так удобно.
- Скрипт хука, содержащий ошибку, может нарушать вашу работу и снижать продуктивность. Единственный способ обойти это — отключить хук. Если вы

¹ Так сложилось, что выполнение хука при коммите — это настолько распространенное требование, что для этого есть даже хук `pre-commit`, хотя он и не строго обязательный.

используете вместо хука псевдоним или скрипт оболочки, то всегда можете вернуться к обычной команде `Git` там, где это имеет смысл.

- Коллекция хуков репозитория не дублируется автоматически. Если вы установите хук коммита в свой репозиторий, он не повлияет на коммиты другого разработчика. Отчасти это сделано в целях безопасности — вредоносный скрипт можно незаметно внести в безопасно выглядящий репозиторий. Отчасти из-за того, что в `Git` нет механизма для дублирования чего-либо, кроме `blob`-объектов, деревьев и коммитов.

ОБЗОР ХУКОВ ОТ ДЖУНИО

Джунио Хамано (Junio Hamano) изложил в списке рассылки `Git` свое видение хуков.

Пять причин создавать хуки для команд/операций в `Git`.

- Для отмены решения, принятого основной командой. Для этого используются `update` и `pre-commit`.
- Для управления данными, сгенерированными после запуска команды. Пример — изменение сообщения коммита с помощью хука `commit-msg`.
- Для выполнения действия на удаленной стороне подключения, к которой есть доступ только через протокол `Git`. Для этого есть специальный хук `post-update`, который выполняет `git update-server-info`.
- Для получения блокировки в целях взаимного исключения. Это происходит редко, но для реализации такой задачи есть много различных хуков.
- Для выполнения одной из нескольких возможных операций в зависимости от результата команды. Пример: хук `post-checkout`.

В каждом из пяти случаев требуется не менее одного хука. Достичь аналогичного результата, используя внешние средства или инструменты, не связанные с самим `Git`, невозможно.

В то же время, если нужно, чтобы какое-либо действие всегда происходило до или после выполнения операции `Git` локально, хук необязателен. Например, если ваша постобработка зависит от результатов выполнения команды (пункт 5 в списке), но при этом результаты очевидны, то хук не нужен.

Ознакомившись с этими «предостережениями», мы можем утверждать, что хуки существуют по веским причинам и их использование может принести пользу.

Установка хуков

Каждый хук — это скрипт. Коллекцию хуков для конкретного репозитория можно найти в каталоге `.git/hooks`. Git не дублирует хуки между репозиториями. Если вы выполните `git clone` или `git fetch` из другого репозитория, то не получите его хуки — их придется копировать вручную.

Каждый скрипт хука именуется согласно событию, с которым связывается. Например, хук, выполняющийся непосредственно перед операцией `git commit`, имеет имя `.git/hooks/pre-commit`.

Хук должен следовать стандартным правилам скриптов Unix: быть исполняемым (`chmod a+x .git/hooks/pre-commit`) и начинаться со строки, обозначающей язык программирования, на котором он написан (например, `#!/bin/bash` или `#!/usr/bin/perl`).

Если хук имеет корректное имя и разрешение файла, Git использует его автоматически.

Шаблоны хуков

В зависимости от версии Git вы можете обнаружить некоторые хуки при создании репозитория. Они автоматически копируются из каталога шаблонов (*template directory*). Например, в Debian и Ubuntu это каталог `/usr/share/git-core/templates/hooks`. Большинство версий Git включают доступные для использования базовые хуки, предустановленные в каталоге `templates`.

Познакомимся с хуками, инициализировав новый репозиторий:

```
$ mkdir hooks-sample && cd hooks-sample
$ git init -b main
$ tree .git
.git
├── HEAD
├── config
├── description
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   └── prepare-commit-msg.sample
```

```

|   |   | push-to-checkout.sample
|   |   | update.sample
|   |   |
|   |   | info
|   |   |   |   | exclude
|   |   |   |   |
|   |   |   |   | objects
|   |   |   |   |   |   | info
|   |   |   |   |   |   | pack
|   |   |   |   |   |   |
|   |   |   |   |   |   | refs
|   |   |   |   |   |   |   | heads
|   |   |   |   |   |   |   | tags

```

8 directories, 17 files

Что нужно знать о шаблонах хуков?

- Шаблоны не всегда делают то, что вам нужно. Их можно читать, редактировать или использовать для обучения, но сами они применяются редко.
- Хотя шаблоны создаются по умолчанию, изначально все хуки отключены. В зависимости от версии Git и вашей операционной системы хуки отключаются снятием бита исполнения (execution bit) или путем добавления к имени файла `.sample`. В последних версиях Git имена исполняемых хуков дополняются суффиксом `.sample`.
- Чтобы активировать шаблон хука, удалите суффикс `.sample` из имени (`mv .git/hooks/pre-commit.sample .git/hooks/pre-commit`).

Изначально каждый стандартный хук копировался из каталога `template` в каталог `.git/hooks/` с удаленным разрешением на выполнение. Затем его можно было включить, установив как исполняемый.

Этот метод хорошо работал в системах Unix и Linux, но не подходил для Windows, где разрешения файлов реализованы иначе и файлы являются исполняемыми по умолчанию. Шаблоны хуков изначально тоже были исполняемыми, что вызывало много путаницы среди пользователей Git: все хуки постоянно выполнялись, даже когда этого не должно было происходить.

Из-за проблемы с Windows в поздних версиях Git имена хуков стали дополняться суффиксом `.sample`, чтобы предотвращать выполнение. Для активации хуков требуется переименовать соответствующие скрипты.

Если шаблоны хуков вам не нужны, их можно удалить из репозитория, используя `rm .git/hooks/*`. Хуки можно вернуть в любой момент, скопировав из каталога `template`.



Помимо хуков в каталоге `template`, Git предоставляет и другие шаблоны. Они находятся в каталоге `contrib` и относятся к исходному коду платформы. Эти вспомогательные файлы можно установить вместе с Git. Например, в Debian и Ubuntu такие хуки устанавливаются в `/usr/share/doc/git-core/contrib/hooks`. В Fedora отправляются в `/usr/share/git-core/contrib/$$hooks`.

Создание первого хука

Разберем принцип работы хука на примере его установки. Для начала создадим репозиторий и добавим туда несколько файлов:

```
$ mkdir hooktest && cd hooktest
$ git init -b main
Initialized empty Git repository in /somepath/.git/

$ touch a b c

$ git add a b c

$ git commit -m 'Added a, b, c'
[main (root-commit) 10f32fe] Added a, b, c
3 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 a
create mode 100644 b
create mode 100644 c
```

Создадим хук `pre-commit`, который будет исключать внесение изменений, содержащих слово *broken* (для примера предположим, что в репозитории нет пред-установленного одноименного хука):

```
# Создание хука.
$ touch .git/hooks/pre-commit
```

С помощью текстового редактора поместите следующий код в созданный файл `.git/hooks/pre-commit`:

```
#!/bin/bash
echo "Hello, I'm a pre-commit script!" >&2
if git diff --cached | grep '^+' | grep -q 'broken'; then
    echo "ERROR: Can't commit the word 'broken'" >&2
    exit 1 # reject
fi
exit 0 # accept
```

Этот скрипт генерирует список из всех внесенных отличий, извлекает строки, которые нужно *добавить* (начинающиеся с символа +), и проверяет их на наличие слова *broken*.

Есть много способов для проверки наличия слова *broken*, но самые очевидные приведут к возникновению малозаметных проблем. Речь не о том, как «проверить» слово *broken*, а о том, как найти текст, который нужно просканировать на наличие этого слова.

Например, вы могли бы использовать такую проверку и попытаться найти слово *broken* во всех файлах репозитория:

```
if git ls-files | xargs grep -q 'broken'; then
```

Но у этого подхода есть две проблемы. Если кто-то другой уже зафиксировал файл, содержащий искомое слово, то скрипт заблокирует все последующие коммиты (пока вы его не исправите), даже если коммиты с ним не связаны. Команда Unix `grep` никак не может знать, какие файлы были фактически зафиксированы. Если вы добавите *broken* в файл *b*, внесете несвязанное изменение в *a*, а затем выполните `git commit a`, то коммит окажется полностью валиден, поскольку вы коммитите не *b*. Однако скрипт с этим тестом все равно отклонит коммит.



Если вы напишете скрипт `pre-commit`, ограничивающий допустимые для добавления элементы, то однажды его потребуется обойти. Для этого нужно сопроводить команду `git commit` опцией `--no-verify` или на время отключить сам хук.

Теперь у нас есть хук `pre-commit`, сделаем его исполняемым:

```
# Установка бита исполнения.  
$ chmod a+x .git/hooks/pre-commit
```

Протестируем его работу:

```
$ echo "perfectly fine" >a
```

```
$ echo "broken" >b
```

```
# Попытка зафиксировать все файлы, даже содержащие слово 'broken'.  
$ git commit -m "test commit -a" -a  
Hello, I'm a pre-commit script!  
ERROR: Can't commit the word 'broken'
```

```
# Выборочный коммит файлов без слова 'broken' работает.  
$ git commit -m "test only file a" a  
Hello, I'm a pre-commit script!  
[main 5468656] test only file a  
1 file changed, 1 insertion(+)
```

```
# Коммит файлов с 'broken' не проходит.  
$ git commit -m "test only file b" b  
Hello, I'm a pre-commit script!  
ERROR: Can't commit the word 'broken'
```

Даже когда коммит работает, скрипт `pre-commit` все равно выдает "Hello". В реальном скрипте такое поведение нежелательно, поэтому такое сообщение нужно использовать только при отладке скрипта. При отклонении скрипта команда `git commit` также не выводит сообщение об ошибке: есть только то сообщение, которое создает скрипт. Чтобы не вводить пользователя в заблуждение, всегда выводите из предварительного скрипта сообщение об ошибке, если он должен вернуть ненулевой код выхода (`reject`).

Теперь рассмотрим различные хуки, которые вы можете создавать.

Доступные хуки

Чтобы узнать, какие хуки доступны в вашей версии Git, выполните `git help hooks`. Также изучите документацию, где описаны все параметры командной строки вместе с вводом и выводом каждого хука.

Хуки для коммитов

При выполнении `git commit`, Git запускает процесс, представленный на рис. 14.1.

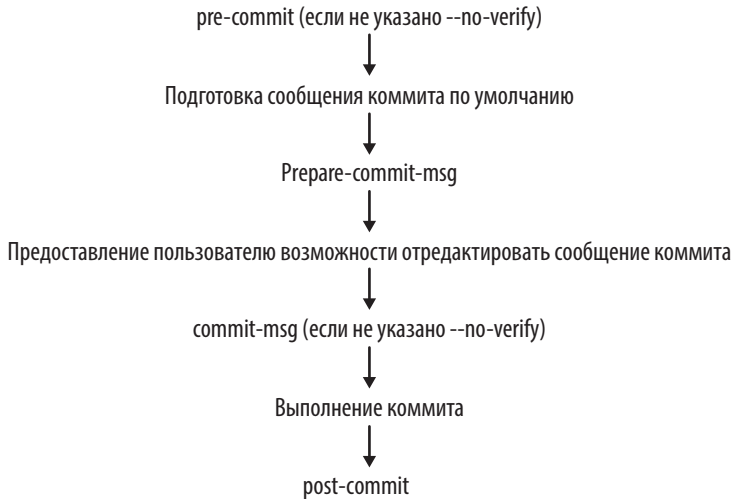


Рис. 14.1. Обработка хука коммита



Любой хук коммита выполняется исключительно через `git commit`. Например, `git rebase`, `git merge` и `git am` по умолчанию не выполняют хуки коммитов. А вот `git commit --amend` хуки коммитов *выполняет*.

У каждого хука своя задача.

- **pre-commit** отменяет коммит, если с его содержимым что-то не так. Он выполняется до того, как у пользователя появляется возможность отредактировать сообщение коммита, поэтому пользователь не увидит сообщение, если коммит будет отменен. Хук можно использовать для автоматического изменения содержимого коммита.
- **prepare-commit-msg** изменяет стандартное сообщение Git до его показа пользователю. Вы можете изменить шаблон предустановленного сообщения коммита.

- `commit-msg` проверяет или изменяет сообщение коммита после его редактирования пользователем. Вы можете проверять сообщение на опечатки или отклонять сообщения, содержащие строки, превышающие максимально допустимую длину.
- `post-commit` выполняется после завершения коммита. В этот момент можно обновить файл журнала, отправить почту или активировать автоматический сборщик. Некоторые разработчики используют этот хук для автоматического обозначения багов как исправленных, если номер бага упомянут в сообщении коммита. На практике этот хук используется редко, поскольку репозиториум, для которого выполняется `git commit`, редко делятся с другими (более подходящим будет хук `update`).

Хуки для патчей

При выполнении `git am` запускается процесс, представленный на рис. 14.2.

- `applypatch-msg` проверяет прикрепленное к патчу сообщение коммита и определяет его корректность. Например, можно отклонить патч, если там нет заголовка `Signed-off-by:`. В этот момент вы можете изменить сообщение коммита.
- Название хука `pre-applypatch` не совсем верно отражает его суть. Скрипт запускается *после* применения патча, но до коммита результатов. Это делает его аналогичным скрипту `pre-commit` при выполнении `git commit`, несмотря на его название. Разработчики часто выбирают создание скрипта `pre-applypatch`, выполняющего `pre-commit`.
- `post-applypatch` аналогичен `post-commit`.



Рис. 14.2. Обработка хука патча



Несмотря на названия хуков, показанных на рис. 14.2, `git apply` не выполняет хуки `applypatch`; это делает только `git am`. Дело в том, что `git apply` ничего не фиксирует, поэтому причины выполнять хуки нет.

Хуки для передачи

При выполнении `git push` *получающая сторона* Git запускает процесс, представленный на рис. 14.3.

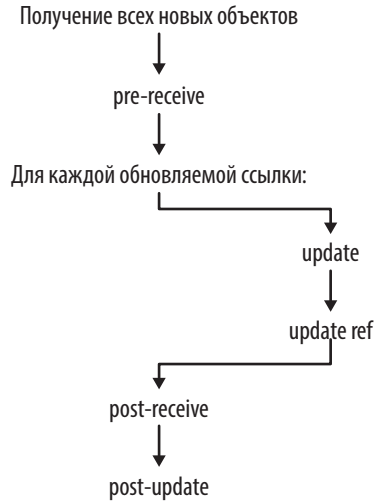


Рис. 14.3. Обработка хуков, связанных с получением



Все хуки, связанные с передачей, выполняются на стороне получателя, а не отправителя. Поэтому все скрипты находятся в каталоге `.git/hooks` получающего, а не отправляющего репозитория. При этом вывод, создаваемый удаленными хуками, все равно показывается пользователю, выполняющему `git push`.

Как видно из схемы, первым шагом `git push` идет передача всех недостающих объектов (blob-объектов, деревьев и коммитов) из локального репозитория в удаленный. В этом процессе хук не нужен, поскольку все объекты Git определяются по их уникальному хешу SHA1. Ваш хук не может изменить объект, так как в противном случае это изменило бы хеш. Кроме того, нет причины отклонять объект: `git gc` в любом случае выполнит очистку, если объект не нужен.

Хуки, связанные с операцией `push`, не манипулируют самими объектами, а вызываются в момент обновления ссылок (веток и тегов):

- **pre-receive** получает список всех обновляемых ссылок, включая новый и старый указатели объектов. Единственное, что может сделать этот хук, — принять или отклонить все изменения разом, что не особо полезно. Рассматривайте его как функциональную возможность, поскольку он обеспечивает целостность передачи данных между ветками. Если такое поведение вам не подходит, используйте хук **update**.

- `update` вызывается один раз для каждой обновляемой ссылки. Он может принимать или отклонять обновления конкретных веток, не влияя на процесс обновления других. Для каждого обновления можно активировать действие: закрытие бага или отправку подтверждения по электронной почте. Обычно лучше обрабатывать такие уведомления в хуке `update`, чем в `post-commit`, поскольку коммит не считается «окончательным», пока не передан в общий репозиторий.
- Как и `pre-receive`, хук `post-receive` получает список всех только что обновленных ссылок. Все, что может сделать `post-receive`, делает и `update`, но иногда `post-receive` оказывается удобнее. Например, если вы хотите отправить по почте уведомление, `post-receive` может отправить всего одно сообщение обо всех изменениях. Хук `post-receive` получает для всех измененных ссылок и старое, и новое значения, чем превосходит `post-update`.
- `post-update` предназначен только для отправки уведомлений и не изменяет результат операции `git push`.

Другие хуки локального репозитория

Есть еще ряд хуков, и к моменту выхода этой книги их число может увеличиться. Список доступных хуков выводит команда `git help hooks`.

- `pre-rebase` (<https://oreil.ly/44zID>) выполняется при попытке перебазировать ветку. Полезен тем, что может остановить вас от случайного выполнения `git rebase` для ветки, которую не следует перебазировать, поскольку она уже опубликована.
- `post-checkout` выполняется после переключения на ветку или отдельный файл. Позволяет автоматически создавать пустые каталоги (Git не отслеживает пустые каталоги), устанавливать разрешения файлов или Access Control Lists (ACL, списки управления доступом) для этих файлов (Git не отслеживает ACL). Вы можете попытаться использовать этот хук для изменения файлов после переключения на них, например для подстановки переменных в стиле RCS, но это плохая идея, поскольку Git решит, что файлы были изменены локально.
- `post-merge` выполняется после операции слияния, используется редко. Если ваш хук `pre-commit` вносит в репозиторий изменения, то скрипт `post-merge` может потребоваться для выполнения чего-то похожего.
- `pre-auto-gc` помогает `git gc --auto` определить, нужно ли проводить очистку. Если в хуке возвращается ненулевое значение, то команда `git gc --auto` пропускает свою задачу по очистке. Однако в большинстве случаев этого не требуется.

Создавать хук или нет?

Дополняя стандартные операции Git, хуки позволяют расширить процесс разработки. В разделе «Об использовании хуков» на с. 355 мы указали, что их следует рассматривать как крайнюю меру. Если вы решаете их применять, вам нужно обеспечить механизм, гарантирующий обновление и согласованность хуков для всех разработчиков, которые от них зависят (при клонировании хуки не передаются). Иногда затраты на создание, настройку и поддержку хуков могут быть больше, чем выгода, которую они приносят в виде автоматизации или улучшения согласованности работы.

Платформы хостинга Git предоставляют возможности для достижения тех же результатов без внесения дополнительной сложности или затраты лишних усилий на поддержание пользовательских хуков. GitHub или GitLab обладают функциональностью, которая позволяет расширять их стандартные возможности и повышать надежность разработки в команде. Например: соответствие установленным стандартам код-ревью, успешная компиляция автотестов, линтинг. При наличии четко определенных требований вы сможете объективно оценить, нужно ли использовать хук или тех же результатов можно добиться стандартными средствами платформы.

Итоги

Мы познакомились с различными видами хуков, а затем на простом примере научились их устанавливать и создавать. Рассмотрели доступные хуки, начиная с тех, что используются с коммитами и патчами, и заканчивая теми, что активируются перед стандартными операциями Git. Каким бы привлекательным ни казалось использование хуков, оно связано с дополнительными издержками, и стоит решить, идти на такой компромисс или нет.

ГЛАВА 15

Подмодули

Распространенная ситуация: имеется много приложений, опирающихся на одну вспомогательную библиотеку или набор библиотек. В таком случае нужно, чтобы каждое из них разрабатывалось, разветвлялось и сливалось в собственном репозитории Git. Это важно как с точки зрения логического разделения функциональности, так и из-за проблем с владением кодом, когда разные команды ответственны за разные приложения или части кода.

Однако такое разделение приложений создает проблему. Каждое из них использует конкретную версию общей библиотеки, и нужно отслеживать эти версии. Если кто-то случайно обновит библиотеку до версии, которая еще не тестировалась, это приведет к сбою приложения. Кроме того, библиотеки утилит разрабатываются для того, чтобы делиться с другими. Обычно люди расширяют их, добавляя новую функциональность, необходимую им в приложениях, а в итоге эта функциональность оказывается доступной для других разработчиков, создающих свои приложения.

Для решения указанной проблемы есть несколько стратегий, включая выполнение частичного переключения, импорт зависимого кода напрямую в проект и копирование зависимого проекта в целевой проект в качестве подкаталога. Но все эти решения нельзя назвать элегантными, скорее это «костыли».

Git решает проблему с помощью подмодулей (*submodule*). *Подмодуль* — это проект, являющийся частью вашего репозитория Git, при этом существующий независимо в собственном репозитории системы контроля версий. Механизм Git для указания прямой ссылки на другой репозиторий Git называется *gitlink*.

Эту главу мы начнем с объяснения *gitlink*, после чего познакомим вас с несколькими техниками для работы с подмодулями. В конце расскажем о команде *subtree*, которая используется в качестве альтернативы для подмодулей.

Gitlink

Gitlink — это ссылка из объекта дерева на объект коммита. Из главы 2 вы знаете, что каждый объект коммита указывает на объект дерева, а каждый объект

дерева — на множество blob-объектов и деревьев, которые соответствуют строкам и подкаталогам. Объект дерева коммитов уникально идентифицирует точный набор файлов, имен файлов и разрешений, прикрепленных к этому коммиту. В разделе «Графы коммитов» на с. 112 в главе 4 мы говорили, что сами коммиты соединены друг с другом в направленном ациклическом графе (DAG). Каждый объект коммита указывает на ноль или большее число родительских коммитов, и все вместе они описывают историю проекта.

Теперь узнаем, как объекты деревьев могут указывать на объекты коммитов через gitlink.

Создадим репозиторий `superproject`, в который будем импортировать исходный код Git:

```
$ cd /tmp
$ mkdir myapp
$ cd myapp

# Начинаем новый superproject.
$ git init -b main
Initialized empty Git repository in /tmp/myapp/.git/

$ echo hello >hello.txt

$ git add hello.txt

$ git commit -m 'first commit'
[main (root-commit) 2f8e120] first commit
1 file changed, 1 insertion(+)
create mode 100644 hello.txt
```

Импортируем проект git напрямую:

```
$ ls
hello.txt

# Клонирование в репозиторий в виде подмодуля.
$ git submodule add https://github.com/git/git.git git
Cloning into 'git'...
remote: Enumerating objects: 329719, done.
remote: Counting objects: 100% (338/338), done.
remote: Compressing objects: 100% (156/156), done.
remote: Total 329719 (delta 214), reused 271 (delta 182), pack-reused 329381
Receiving objects: 100% (329719/329719), 194.04 MiB | 27.39 MiB/s, done.
Resolving deltas: 100% (246495/246495), done.
Updating files: 100% (4167/4167), done.

$ ls
git hello.txt

$ git add git
```

```
$ git commit -m 'import git'
[main 62079a5] import git
files changed, 4 insertions(+)
create mode 100644 .gitmodules
create mode 160000 git
```



Обычно команды `git add git` и `git add git/` (с POSIX-совместимым слешем в конце, указывающим, что `git` должен быть каталогом) равнозначны. Но в случае `gitlink` это не так. Выше мы показали, что добавление слеша в команду `git add git/` не приведет к созданию `gitlink`. Это только добавит все файлы в каталог `git`, что в данном случае не нужно.

Просматриваем полученное дерево суперпроекта (superproject):

```
$ git ls-tree HEAD
100644 blob fc7f9429d5ca844b0a71bd9cc9062318eaea08bd .gitmodules
160000 commit 4e2a4d1dd44367d7783f33b169698f2930ff13c0 git
100644 blob ce013625030ba8dba906f756967f9e9ca394464a hello.txt
```

Подкаталог `git` имеет тип `commit` и режим `160000`. Это делает его ссылкой `gitlink`.

Git обычно рассматривает `gitlink` как простые значения указателя ссылок на другие репозитории. Большинство операций Git, например `clone`, не разыменовывают `gitlink`, а затем выполняют действия в репозитории подпроекта.

Например, если вы отправите свой проект в другой репозиторий, это не приведет к загрузке коммитов, деревьев и `blob`-объектов подпроекта. Если вы клонируете репозиторий вашего суперпроекта, каталоги в его репозитории будут пустыми.

В коде ниже каталог `git` подпроекта после выполнения команды `clone` остается пуст:

```
$ cd /tmp

$ git clone myapp app2
Cloning into 'app2'...
done.

$ cd app2

$ ls
git/  hello.txt

$ ls git

$ du git
0      git
```

Важной особенностью ссылок `gitlink` является то, что они указывают на объекты, которых *может не быть* в репозитории.

Поскольку gitlink ссылаются на отсутствующие объекты, это позволяет выполнять частичное переключение. Вам не нужно переключаться на каждый подпроект, а только на те, которые требуются.

Подмодули

Подмодули — это мощный элемент инструментария Git, но иногда понять их непросто. На самом высоком уровне подмодули представляют собой вспомогательный инструмент для компоновки репозитория Git (рис. 15.1).

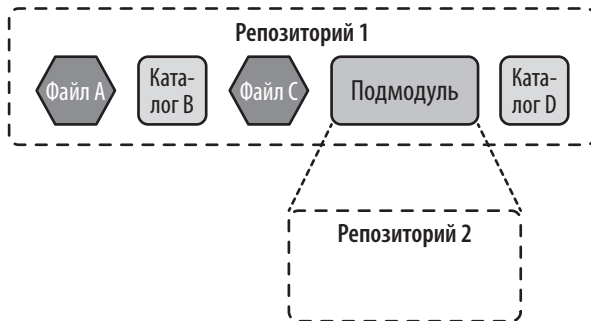


Рис. 15.1. Вложенные репозитории

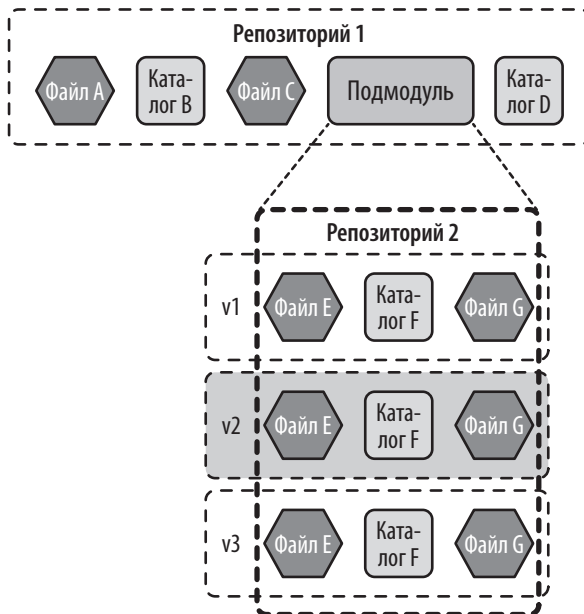


Рис. 15.2. Вложенные репозитории, указывающие на точную ревизию

Подмодули обеспечивают бóльшую точность, указывая не только на сетевой адрес вложенного репозитория, но также и на хеш коммита этого репозитория (рис. 15.2).

Поскольку каждая ссылка коммита содержит внутри репозитория уникальный идентификатор конкретной точки в графе и все родительские состояния, которые привели к этой точке, то указание на ссылку другого репозитория регистрирует это точное состояние в истории коммитов родительского проекта.

Зачем нужны подмодули

Наиболее распространенный фактор, побуждающий к их использованию, — это модульность. Подмодули позволяют получить компонентное представление кодовой базы при отсутствии модульности на двоичном уровне (DLL, JAR, SO). Есть хорошо известные решения Java — Maven Multimodule Projects (<https://oreil.ly/yLgCm>) и Gradle Multiproject Builds (<https://oreil.ly/woFw3>), которые позволяют управлять двоичными или полудвоичными, разбитыми на компоненты зависимостями, не требуя переключения всего исходного кода в монолитный каталог. Аналогично для .NET есть сборки (<https://oreil.ly/irYSR>), которые позволяют потреблять в двоичном виде подкомпоненты и плагины. В экосистеме Objective-C подмодули обычно используются по причине недостатка решений для реализации модульности и включения скомпилированных двоичных файлов.

Подмодули Git сохраняют существующую структуру каталогов подкомпонентов при условии, что разделение компонентов (подмодулей) осуществляется по границам каталогов в структуре проекта. Это позволяет точно управлять изменениями в каждом компоненте, выполнять откат к предыдущим версиям и поддерживать версию совместимости между различными компонентами проекта.

Работа с подмодулями

Команда `git submodule` представляет собой скрипт оболочки Unix под названием `git submodule.sh` (<https://oreil.ly/A1uH4>), состоящий из 755 строк. Его задача проста: отслеживать `gitlinks` и выполнять за вас переключение на соответствующие репозитории.

Прежде чем что-либо делать, команда `git submodule` должна понимать, где найти репозиторий для вашего подмодуля. Эта информация содержится в файле `.gitmodules`, который выглядит так:

```
# git - имя подмодуля репозитория
[submodule "git"]
    path = git
    url = https://github.com/git/git.git
```

Инициализируем новый репозиторий с файлом, а затем выполним несколько операций Git для работы с репозиторием подмодуля:

```
# Создание во временном каталоге.
$ mkdir superproject && cd superproject
$ git init -b main
Initialized empty Git repository in /tmp/superproject/.git/

$ echo "hello from super project" > sp-readme.md
$ git add sp-readme.md
$ git commit -m "Initial commit"
[main (root-commit) 3689ee7] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 sp-readme.md

$ git status
On branch main
nothing to commit, working tree clean
```

Далее мы кратко обсудим некоторые операции Git для работы с репозиторием подмодуля.

Добавление подмодуля

При добавлении подмодуля в существующий репозиторий командой `git submodule add` Git создает файл `.gitmodules` вместе с подкаталогом, содержащим все файлы для добавленного подмодуля. Эти файлы автоматически добавляются в индекс:

```
# Для продолжения создайте новый репозиторий в своем аккаунте GitHub на основе
этого шаблона:
# https://github.com/ppremk/example-submodule-template
# Кликните по зеленой кнопке Use this template для создания копии репозитория.

# Добавьте существующий подмодуль из URL клона вашего репозитория на GitHub.

$ git submodule add https://github.com/ppremk/example-submodule.git
Cloning into '/tmp/superproject/example-submodule'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (4/4), done.

$ ls
example-submodule sp-readme.md

$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitmodules
    new file:   example-submodule

# Выполните коммит подмодуля.
```

```
$ git commit -m "Add submodule"
[main ecc6142] Add submodule
 2 files changed, 4 insertions(+)
 create mode 100644 .gitmodules
 create mode 160000 example-submodule
```



История проекта добавленного подмодуля останется отдельной и не станет частью истории суперпроекта. Также обратите внимание на режим файла 160000 для каталога example-submodule, что делает его gitlink.

Клонирование репозитория

Когда вы клонируете репозиторий, который уже содержит подмодуль, учитывайте несколько моментов. Посмотрите на этот пример, который пытается клонировать репозиторий с подмодулями:

```
# Создание bare-версии для имеющегося репозитория.
# Это позволит передать изменения из клона репозитория.
$ cd ..
$ git clone --bare /tmp/superproject /tmp/superproject-upstream
Cloning into bare repository '/tmp/superproject-upstream'...
done.

# Клонирование нового вышестоящего репозитория с примером подмодуля.
$ git clone /tmp/superproject-upstream superproject-clone-1
Cloning into 'superproject-clone -1'...
done.

# Создание еще одного клона для последующей демонстрации.
$ git clone /tmp/superproject-upstream superproject-clone-2
Cloning into 'superproject-clone -2'...
Done.

$ cd superproject-clone-1
$ ls
example-submodule sp-readme.md

$ cd example-submodule && ls
$
```

Подкаталог для подмодулей пуст, поскольку во время операции клонирования superproject подмодули не инициализируются. Для инициализации подмодуля выполните команду:

```
$ cd ..
$ git submodule update --init
Submodule 'example-submodule' (https://github.com/ppremk/example-submodule.git)
registered for path
  'example-submodule'
Cloning into '/tmp/superproject-clone/example-submodule'...
```



```
Submodule path 'example-submodule': checked out '3b294591ab02a849bdc29f545658cd74f6832bcb'
```

```
$ cd example-submodule && ls
LICENSE      README.md
```

Если подмодули вашего `superproject` содержат другие подмодули, сопроводите команду опцией `-recursive`, чтобы инициализировать их все сразу:

```
$ git submodule update --init --recursive
```

Однако более удачным способом инициализации каталога подмодулей и файлов при клонировании `superproject` будет использование опции `--recurse-submodules` с URL клона:

```
$ git clone --recurse-submodules /tmp/superproject-upstream superproject-clone
```

Сравнение команд `submodule add` и `submodule init`

Команда `git submodule add` добавляет в файл `.gitmodules` запись и заполняет новый репозиторий клоном добавленного, а `git submodule init` копирует настройки из файла `.gitmodules` в ваш файл `.git/config`:

```
$ pwd
/tmp/app2

$ git submodule init
Submodule 'git' (https://github.com/git/git.git) registered for path 'git'

$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
[remote "origin"]
    url = /tmp/myapp
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = origin
    merge = refs/heads/main
[submodule "git"]
    active = true
    url = https://github.com/git/git.git
```

Команда `git submodule init` добавляет только последние две строки.

Этап `git submodule init` позволяет перенастроить локальные подмодули, чтобы они указывали на другой репозиторий, а не тот, что упомянут в официальном файле `.gitmodules`.

Рассмотрим такой сценарий.

Когда вы клонируете проект, где используются подмодули, то бывает необходимым сохранить собственную копию этих подмодулей и указать на нее локальному клону. В таком случае не нужно менять официальное расположение модуля в `.gitmodules`, но требуется, чтобы `git submodule` указывал на предпочтительное расположение. Таким образом, `git submodule init` копирует всю недостающую информацию о подмодуле из `.gitmodules` в `.git/config`, где ее можно безопасно отредактировать. Найдите раздел `[submodule]`, относящийся к подмодулю, который хотите изменить, и исправьте URL.

Затем выполните команду `git submodule update`, чтобы обновить файлы из своей копии подмодуля или создать полностью новый клон начального репозитория подпроекта, удалив существующие файлы:

```
# Принудительно выполните полное новое клонирование, удалив существующие данные,
# если это необходимо, когда указываете на собственную версию подмодуля.
$ pwd
/tmp/app2

$ rm -rf git
$ git submodule update
Cloning into '/tmp/app2/git'...
Submodule path 'git': checked out '4e2a4d1dd44367d7783f33b169698f2930ff13c0'
```

Изменение подмодулей внутри суперпроекта

Бывает, что при работе над основным суперпроектом требуется внести одно-временные изменения в файлы зависимого подмодуля. Делается это так же, как и в любом репозитории Git. Но есть один нюанс.

При клонировании суперпроекта с использованием `--recurse-submodules` или получении обновлений с помощью команды `git submodule update` файлы в каталоге подмодуля обновляются и будут находиться в *состоянии отсоединенной HEAD*.

Это означает, что любые изменения, которые вы хотите внести, нужно вносить в локальной ветке отслеживания, связанной с файлами подмодуля.

Рассмотрим пример:

```
$ pwd
/tmp/superproject-clone-1

$ cd example-submodule
$ git status
HEAD detached at 3b29459
nothing to commit, working tree clean

$ git branch -a
* (HEAD detached at 3b29459)
  main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
```



Мы не будем усложнять этот пример, поскольку тема и так непростая. Поэтому в примере напрямую обновим основную ветку. На практике вам потребуются придерживаясь рабочего процесса и вносить новую ветку функциональности, прежде чем вливать изменения в основную стабильную ветку подмодуля.

Внесем изменения напрямую в подмодуль, переключившись на ветку локального отслеживания `main`:

```
$ pwd
/tmp/superproject-clone-1/example-submodule

$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ echo "Add new submodule file" > new-submodule-file.md
$ git add new-submodule-file.md
$ git commit -m "Add new submodule file"
[main 03c93dc] Add new submodule file
1 file changed, 1 insertion(+)
create mode 100644 new-submodule-file.md

$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
```

Продолжим передачу изменений в удаленный вышестоящий URL подмодуля:

```
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 355 bytes | 355.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/ppremk/example-submodule.git
3b29459..03c93dc main -> main
```



Если вы предпочитаете передавать изменения, зафиксированные в проекте подмодуля, из каталога суперпроекта, то сделать это можно командой `git push --recurse-submodules=on-demand`.

В предыдущем примере мы передали изменения подмодуля из подкаталога напрямую в удаленный вышестоящий URL. И хотя это явно обновило удаленный URL подмодуля, суперпроект оказался в устаревшем состоянии. Он по-прежнему

указывает на старый SHA коммита, соответствующий моменту изначального добавления подмодуля:

```
$ cd ..
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   example-submodule (new commits)
```

no changes added to commit (use "git add" and/or "git commit -a")

Выполнение команды `git status` в каталоге `superproject` даст краткую информацию о том, что в каталоге подмодуля есть новые коммиты. Если вы хотите получить подробную сводку об изменениях, то установите в конфигурации Git флаг `status.submoduleSummary` на `true`:

```
$ git config --local status.submoduleSummary true
$ git status
On branch main
Your branch is up to date with 'origin/main'.
```

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   example-submodule (new commits)
```

Submodules changed but not updated:

```
* example-submodule 3b29459...03c93dc (1):
  > Add new submodule file
```

no changes added to commit (use "git add" and/or "git commit -a")

```
$ git add example-submodule
$ git commit -m "Add new submodule changes"
[main e044d76] Add new submodule changes
 1 file changed, 1 insertion(+), 1 deletion(-)
```

```
$ git push
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 330 bytes | 330.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0
To /tmp/superproject-upstream
ecc6142..e044d76  main -> main
```



Очень важно, чтобы суперпроект регистрировал SHA последнего коммита подмодуля. Если этого не сделать, то после выполнения очередной операции клонирования или обновления подмодуля суперпроект продолжит ссылаться на устаревший коммит подмодуля.

Получение обновлений подмодуля

Есть два способа обновления проекта подмодуля в суперпроекте. Первый — использовать стандартную операцию `Git fetch` с операцией `merge`. Это делается из подкаталога вашего подмодуля (как и в случае любого стандартного репозитория `Git`). Так вы сможете выборочно указать, какую именно ветку подмодуля обновить.

Второй способ — использовать команду `git submodule update --remote`. По умолчанию эта операция внесет последние изменения из ветки `main` подмодуля в ваш локальный подмодуль:

```
$ pwd
/tmp/superproject

$ git submodule update --remote
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 335 bytes | 111.00 KiB/s, done.
From https://github.com/ppremk/example-submodule
   3b29459..03c93dc  main       -> origin/main
Submodule path 'example-submodule': checked out '03c93dc48b60829baed35f8f78959a41b89dd832'

$ cd example-submodule && ls
LICENSE      README.md    new-submodule-file.md
```



В случае совместной разработки при работе с подмодулем рекомендуется указывать вашу ссылку на его готовую продакшен-версию (или стабильную версию). Для этого мейнтейнеры подмодуля обычно включают именованную *стабильную* ветку. Чтобы настроить ваш подмодуль на отслеживание конкретной ветки, установите следующую конфигурацию:

```
git config -f .gitmodules submodule.example-submodule.branch
<stable-branch-name>
```

Опция `-f .gitmodules` отвечает за то, чтобы файл `.gitmodules` обновлялся, сохраняя согласованность со всеми участниками через отслеживание конкретной ветки.

Получение обновлений суперпроекта, использующего подмодуль

При получении обновлений для суперпроекта с помощью команды `git pull` `Git` получает изменения в репозитории ссылающегося подмодуля, но не обновляет их.

Для обновления суперпроекта выполните команду `git submodule update`:

```
# Переключаемся на второй клонированный репозиторий.
$ pwd
/tmp/superproject-clone-2

$ git pull
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 2 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (2/2), 310 bytes | 310.00 KiB/s, done.
From /tmp/superproject-upstream
    ecc6142..e044d76  main    -> origin/main
Updating ecc6142..e044d76
Fast-forward
 example-submodule | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

$ cd example-submodule && ls
$

$ cd ..
$ git submodule update --init --recursive
Submodule 'example-submodule' (https://github.com/ppremk/example-submodule.git)
registered for...
Cloning into '/tmp/superproject-clone-2/example-submodule'...
Submodule path 'example-submodule': checked out '03c93dc48b60829baed35f8f78959a41
b89dd832'

$ cd example-submodule && ls
LICENSE                                README.md                             new-submodule-file.md
```

При обновлении суперпроекта помните, что в него могли быть добавлены новые подмодули или же вышестоящий URL подмодуля мог измениться. Советуем выполнять команду `git submodule update --init`, чтобы при получении изменений для `superproject` инициализировался каждый подмодуль.



Чтобы объединить шаги `git pull` и `git submodule update --init`, используйте команду `git pull --recurse-submodules`.

Когда в вышестоящем URL подмодуля есть изменения, выполнение команды `git submodule sync` вместе с `git submodule update --init` обеспечит получение вашим суперпроектом последних изменений:

```
# Сначала обновите локальный файл конфигурации новым вышестоящим URL подмодуля
$ git submodule sync --recursive
$ git submodule update --init --recursive
```

Переключение веток при работе с суперпроектом, где есть подмодули

Работа с несколькими ветками в суперпроекте, ссылающимися на подмодули, может сбивать с толку. Дело в том, что состояние подмодуля может не соответствовать ожидаемому состоянию и могут возникнуть проблемы с согласованностью версий между суперпроектом и его подмодулями.

Каждый раз, когда вы работаете с суперпроектом, содержащим подмодули, рекомендуется использовать команду `git checkout` или `git switch` с опцией `--recurse-submodules`. Опция `--recurse-submodules` обеспечит синхронизацию корректного состояния подмодулей с веткой, на которую вы переключаетесь:

```
$ git checkout --recurse-submodules <branch-name>
```

Вывод предыдущей команды будет таким:

```
$ git checkout <branch-name>
$ git submodule update --recursive
```

Подмодули и переиспользование учетных данных

Традиционный адрес `git clone user@hostname:pathtorepo` подходит для автономного репозитория Git. Однако для команды `git submodule add URL` такой адрес менее желателен, поскольку имя пользователя сохранится в метаданных подмодуля на уровне репозитория суперпроекта. Имя будет ненамеренно использоваться всеми клонирующими его разработчиками.

В организации, где управление доступом к репозиториям определяется на уровне пользователей, нежелательно сохранять конкретное имя пользователя как часть записываемого в `.gitmodules` адреса подмодуля. Хорошая практика, если имя пользователя суперпроекта, используемое при его клонировании, будет передаваться в операцию клонирования подмодуля.

```
$ git clone --recurse-submodules https://github.com/user1/myproject
Username for 'https://github.com': user1
Password for 'https://user1@github.com':
```

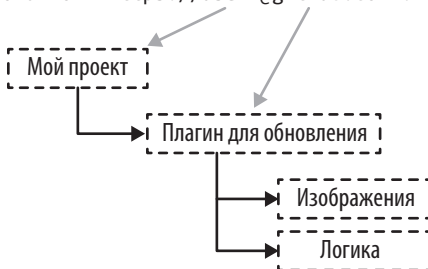


Рис. 15.3. Переиспользование учетных данных в подмодулях

Команды Git для подмодулей получают учетные данные во время клонирования суперпроекта и далее передают их (рис. 15.3) в любые действия, которые активируются через опцию `--recurse-submodules`. В результате адрес `.gitmodules` не содержит никаких имен пользователей и может быть использован любым разработчиком, имеющим право клонировать проект.

Поддеревья Git

В Git поддеревья считаются альтернативой подмодулям. С их помощью можно добавить подпроект в качестве подкаталога текущего проекта и включить всю его историю или объединить коммиты в один. В отличие от подмодулей, поддеревья не требуют дополнительной конфигурации в виде файла `.gitmodules` или ссылок `gitlink`. Для большинства пакетных менеджеров `git subtree` устанавливается автоматически вместе с Git. Если нужно установить это расширение вручную, то все этапы описаны в документации Git (<https://oreil.ly/4viZZ>).



Не путайте поддеревья со стратегией *слияния поддеревьев*. Git также позволяет извлекать историю конкретного подкаталога проекта через команду `git submodule split`.

В примере ниже мы еще раз инициализируем репозиторий с файлом. А в следующих подразделах приведем операции Git для работы с поддеревьями:

```
# Создание во временном каталоге.
$ mkdir superproject && cd superproject
$ git init -b main
Initialized empty Git repository in /tmp/superproject/.git/

$ echo "hello from super project" > sp-readme.md
$ git add sp-readme.md
$ git commit -m "Initial commit"
[main (root-commit) a5b6866] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 sp-readme.md

$ git status
On branch main
nothing to commit, working tree clean
```

Добавление подпроекта

При добавлении подпроекта с помощью поддеревьев команда `git subtree add` также добавляет в ваш суперпроект новый подкаталог со всеми связанными файлами. Но при использовании команды `subtree` вы совместите историю подпроекта с историей вашего суперпроекта:


```
# Для продолжения создайте в аккаунте репозиторий на основе шаблона
# https://github.com/ppremk/example-subtree-template
# Для создания копии репозитория щелкните по зеленой кнопке Use this template.
```

```
# Создайте в копии шаблона example-subtree несколько фиктивных коммитов,
# используя GitHub WebUI.
```

```
$ ls
sp-readme.md
```

```
$ git log --oneline --graph
* a5b6866 (HEAD -> main) Initial commit
```

```
# Внесите существующее поддерево через URL клона вашего репозитория на GitHub.
# С помощью команды subtree add укажите, на какую ветку подпроекта хотите сослаться.
```

```
# Убедитесь, что установили расширение git subtree (на всякий случай).
$ git subtree add --prefix=example-subtree https://github.com/ppremk
/example-subtree.git main
```

```
git fetch https://github.com/ppremk/example-subtree.git main
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 15 (delta 5), reused 4 (delta 0), pack-reused 0
Unpacking objects: 100% (15/15), 2.87 KiB | 367.00 KiB/s, done.
From https://github.com/ppremk/example-subtree
 * branch          main          -> FETCH_HEAD
Added dir 'example-subtree'
```

```
$ ls
example-subtree sp-readme.md
```

```
# История коммитов подпроекта была совмещена
# с историей git, относящегося к суперпроекту.
```

```
$ git log --oneline --graph
* 78d0604 (HEAD -> main) Add 'example-subtree/' from commit '08d39b...'
|\
| * 08d39b1 Final update to README.md
| * 87c0990 One more update to README.md
| * 473e4f9 Update README.md
| * e5d123a Initial commit
* a5b6866 Initial commit
```

В примере выше команда `subtree` добавила рабочее дерево подпроекта к существующему дереву суперпроекта. Если история подпроекта неважна и без нее можно обойтись, то укажите опцию `--squash`, чтобы сохранить чистую объединенную историю:

```
$ git reset --hard a5b6866
HEAD is now at a5b6866 Initial commit
```

```
$ git log --oneline --graph
* a5b6866 (HEAD -> main) Initial commit

# Совмещение истории коммитов подпроекта.

$ git subtree add --prefix=example-subtree \
> https://github.com/ppremk/example-subtree.git main --squash
git fetch https://github.com/ppremk/example-subtree.git main
From https://github.com/ppremk/example-subtree
 * branch          main          -> FETCH_HEAD
Added dir 'example-subtree'

$ ls
example-subtree sp-README.md

$ git log --oneline --graph
* 525f638 (HEAD -> main) Merge commit '5d3ce7...' as 'example-subtree'
|\
| * 5d3ce7a Squashed 'example-subtree/' content from commit 08d39b1
* a5b6866 Initial commit
```



В этом примере URL поддерева сокращается до `subremote-origin`, что упрощает идентификацию удаленного репозитория в дальнейших операциях:

```
$ git remote add subremote-origin
https://github.com/ppremk/example-subtree.git
```

Получение обновлений подпроекта

Когда для подпроекта появляется обновление, то получить его из суперпроекта можно так:

```
$ cd example-subtree && ls
LICENSE                README.md              another-new-file.md new-file.md

# Добавление нового файла для вашего репозитория example-subtree
# непосредственно через GitHub WebUI.

$ cd ..

$ git subtree pull --prefix=example-subtree subremote-origin main --squash
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 18 (delta 6), reused 4 (delta 0), pack-reused 0
Unpacking objects: 100% (18/18), 3.50 KiB | 511.00 KiB/s, done.
From https://github.com/ppremk/example-subtree
 * branch          main          -> FETCH_HEAD
 * [new branch]    main          -> subremote-origin/main
Merge made by the 'ort' strategy.
 example-subtree/sparkling-file.md | 1 +
1 file changed, 1 insertion(+)
```

```
create mode 100644 example-subtree/sparkling-file.md

# Совмещенные обновления подпроекта добавляются в суперпроект в виде коммита
слияния.
$ git log --oneline --graph
* c468ae8 (HEAD -> main) Merge commit '8bc211...'
|\
| * 8bc211a Squashed 'example-subtree/' changes from 08d39b1..ff9255a
* | 525f638 Merge commit '5d3ce7a...' as 'example-subtree'
|\|
| * 5d3ce7a Squashed 'example-subtree/' content from commit 08d39b1
* a5b6866 Initial commit

$ cd example-subtree && ls
LICENSE      README.md    another-new-file.md new-file.md    parking-file.md
```

Изменение подпроекта из суперпроекта

При внесении изменений в подпроект помните, что коммиты, записывающие его изменения, будут частью истории коммитов суперпроекта в той ветке, где он активен. Это делает процесс более прямолинейным в сравнении с подмодулями, когда нужно вносить изменения в вышестоящий URL подпроекта с помощью поддерева.

Передать изменения в подпроект можно так:

```
# Внесение изменений в каталог подпроекта
$ cd example-subtree

$ touch just-file.md

$ git add just-file.md
$ git commit -m "Add just file from superproject"
[main 0408fed] Add just file from superproject
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 example-subtree/just-file.md

$ git log --oneline --graph
* 0408fed (HEAD -> main) Add just file from superproject
* c468ae8 (HEAD -> main) Merge commit '8bc211...'
|\
| * 8bc211a Squashed 'example-subtree/' changes from 08d39b1..ff9255a
* | 525f638 Merge commit '5d3ce7a...' as 'example-subtree'
|\|
| * 5d3ce7a Squashed 'example-subtree/' content from commit 08d39b1
* a5b6866 Initial commit

# Эту команду нужно выполнять из каталога суперпроекта.
$ cd ..

$ git subtree push --prefix=example-subtree subremote-origin main
```

```
git push using: subremote-origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 251 bytes | 251.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ppremk/example-subtree.git
    ff9255a..3b0c49a  3b0c49a978fe67f811943b778aaa5b3f867a0cf1 -> main
```

Другие операции Git, такие как клонирование репозитория, получение обновлений суперпроекта и переключение веток при работе с суперпроектами, в которых добавлены подпроекты с помощью `git subtree`, не требуют специальных команд Git. Вы можете вносить изменения по мере необходимости, используя стандартные команды.

Визуальное сравнение подмодулей и поддеревьев Git

Выше мы показали примеры включения подмодулей и поддеревьев в ваши проекты Git. На рис. 15.4 и 15.5 показаны концептуальные отличия этих двух инструментов.

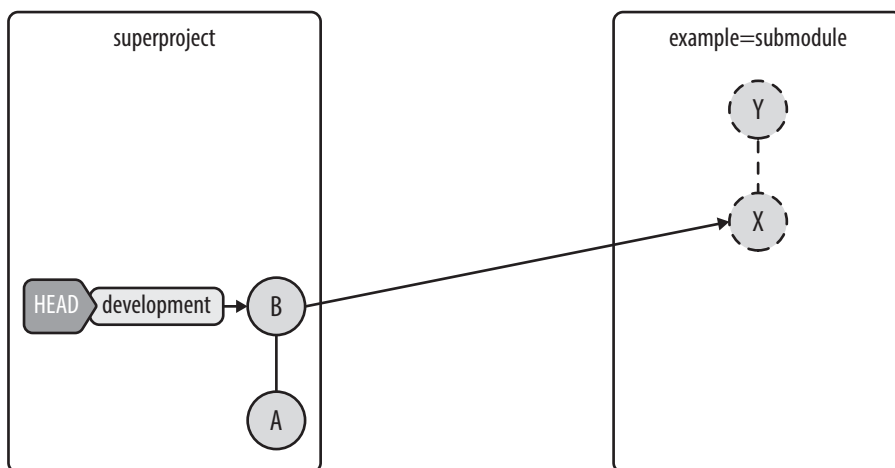


Рис. 15.4. Принцип построения истории коммитов подмодулей

На рис. 15.4 история коммитов (HEAD) суперпроекта содержит явную ссылку на SHA коммита репозитория `example-submodule`. История обоих репозиториев остается раздельной.

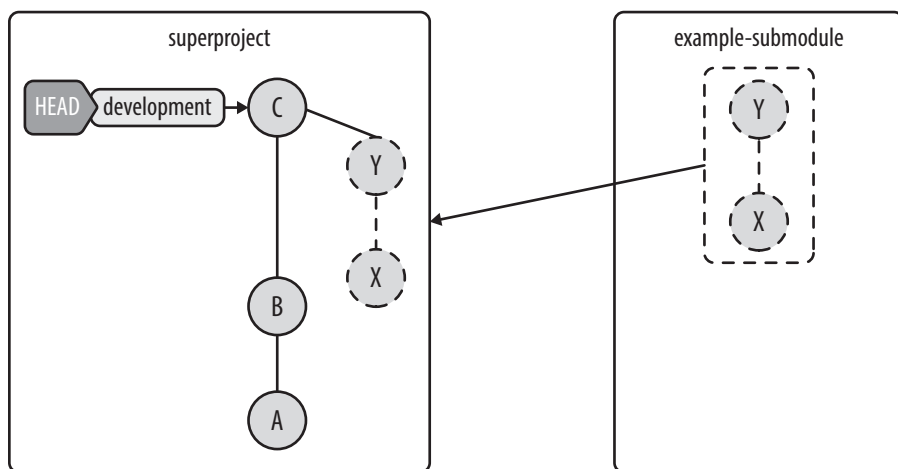


Рис. 15.5. Принцип построения истории коммитов поддеревьев

На рис. 15.5 в историю коммитов (HEAD) репозитория суперпроекта записывается SHA нового коммита, объединя историю репозитория `example-subtree`. История коммитов репозитория `example-subtree` переносится как часть истории суперпроекта.

Итоги

В этой главе мы показали принцип использования ссылок `gitlink`, подмодулей и поддеревьев для совмещения репозитория или проектов. Примеры с подмодулями были упрощены, чтобы вы поняли основные принципы работы со стандартными операциями Git.

По сравнению с поддеревьями использовать подмодули сложнее. Рекомендуем тщательно оценивать, насколько эта сложность оправдана в той или иной ситуации. Если в вашем проекте возможны альтернативы, то лучше обратиться к инструментам управления зависимостями, которые решают эти задачи без издержек, связанных с ведением подмодулей или поддеревьев. Подробную информацию ищите в руководстве `git-scm` (<https://oreil.ly/vwnzl>).

Продвинутые способы управления

В этой главе обсудим продвинутые команды для управления репозиториями Git. Они варьируются от управления файлами до поиска коммитов, а также поддержки, анализа и изменения репозитория под определенные требования. Управление репозиториями в Git имеет свои последствия, в частности при выполнении операций, изменяющих историю коммитов. Поэтому мы рекомендуем соблюдать осторожность при использовании этих команд.

Интерактивное индексирование фрагментов

Несмотря на загадочное название, интерактивное индексирование фрагментов (interactive hunk staging) — это невероятно мощный инструмент, позволяющий упростить и разделить процесс разработки на четкие и понятные коммиты. Если вас когда-либо просили разбивать изменения в коде на отдельные патчи, каждый из которых представляет собой логически связанное изменение или функциональность, то этот раздел будет вам полезен.

Если только вы не суперпрограммист, который мыслит в понятиях четких патчей, то, скорее всего, ваш обычный процесс разработки примерно такой же, как у большинства (включая нас): не всегда последовательный, возможно, перегруженный и включающий в себя несколько идей или концепций, которые взаимно связаны или переплетаются друг с другом. Одна мысль ведет к другой, вы исправляете один баг, затем натываетесь на другой, исправляете и его тоже, а заодно добавляете какую-то функциональность, раз уж здесь оказались. Ах да, еще поправляете пару опечаток.

Если вы, как и мы, просите сделать кого-то ревью ваших изменений в важном коде, прежде чем принять их в основную версию, то велика вероятность, что не связанные между собой изменения, представленные в одном патче, покажутся нелогичными или труднопонижаемыми для других участников проекта. Более того, некоторые проекты с открытым исходным кодом требуют, чтобы патчи содержали отдельные самостоятельные исправления: то есть один патч не должен решать сразу несколько задач. Каждая идея должна быть самостоятельной и быть

представлена в виде определенного и простого патча, достаточно большого для решения конкретной проблемы, но не более того.

Если в вышестоящий репозиторий нужно передать более одной идеи, то нужно несколько, возможно, поочередных патчей. Как показывает опыт, такие патчи и их последовательности повышают надежность ревью и скорость обратной связи, а также упрощают принятие изменений в основную линию разработки.

Как реализуются последовательности патчей? Хотя мы и стремимся к стилю разработки, способствующему созданию простых патчей, это получается не всегда. Однако Git предоставляет инструменты, помогающие составлять хорошие патчи. Один из этих инструментов позволяет интерактивно выбирать и фиксировать элементы, или «фрагменты» (hunks), в текущем патче, оставляя другие части для коммита в следующем. В итоге мы создаем новую последовательность мелких коммитов, объединяющих всю исходную работу.

При этом Git не решит за вас, какие фрагменты должны быть объединены, а какие нет. Вы должны уметь определить их назначение и логически сгруппировать. Иногда фрагменты находятся в одном файле, но чаще оказываются в разных. Все концептуально связанные фрагменты должны выбираться и индексироваться в одном коммите.

Фрагменты нужно выбирать так, чтобы этот набор соответствовал всем внешним требованиям. Например, если вы пишете код, который нужно скомпилировать, то важно, чтобы кодовая база после каждого коммита оставалась компилируемой. Таким образом, ваш патч при повторной сборке в виде более мелких частей должен компилироваться при каждом коммите в новой последовательности. Здесь придется подумать: Git сделать это за вас не сможет.

Чтобы интерактивно индексировать фрагменты патчей, добавьте к команде `git add` опцию `-p`.

```
$ git add -p file.c
```

Интерактивное индексирование выглядит довольно просто, и так оно и есть. Но важно держать в уме, что именно Git делает с патчами. В главе 5 мы рассказывали, как Git поддерживает индекс в виде области, накапливающей изменения до их коммита. Здесь все происходит так же. Но вместо единовременного сбора изменений в одном файле Git выбирает те, которые вы сделали в рабочей копии файла, и позволяет вам указать отдельную часть или части для индексации при подготовке к коммиту.

Например, мы пишем программу для вывода гистограммы разделенных пробелами слов из файла. Ее первая версия Hello, World! демонстрирует, что начальная стадия компиляции проходит успешно. Вот ее файл `main.c`:

```
#include <stdio.h>

int main(int argc, char **argv)
```

```
{
    /*
     * Выводит гистограмму содержащихся в файле слов.
     * "Слова" – это любые символы, разделенные пробелом.
     * Слова перечислены в произвольном порядке.
     * ФИХМЕ: все еще требуется реализация!
     */
    printf("Histogram of words\n");
}
```

Добавим Makefile и .gitignore, поместив их в новый репозиторий:

```
$ mkdir /tmp/histogram
# cd /tmp/histogram
$ git init -b main
Initialized empty Git repository in /tmp/histogram/.git/
$ git add main.c Makefile .gitignore

$ git commit -m "Initial histogram program."
[main (root-commit) 42300e7] Initial histogram program.
3 files changed, 18 insertions(+), 0 deletions(-)
create mode 100644 .gitignore
create mode 100644 Makefile
create mode 100644 main.c
```

Сделаем несколько разных изменений, пока файл `main.c` не станет выглядеть так:

```
#include <stdio.h>
#include <stdlib.h>

struct hentry {
    char *item;
    int count;
    struct hentry *next;
};

struct hentry ht_table[256];

void ht_init(void)
{
    /* ФИХМЕ: подробности */
}

int main(int argc, char **argv)
{
    FILE *f;

    f = fopen(argv[1], "r");
    if (f == 0)
        exit(-1);

    /*
     * Выводит гистограмму слов, содержащихся в файле.
     */
}
```



```

    * "Слова" – это любые символы, разделенные пробелом.
    * Слова перечисляются в произвольном порядке.
    * FIXME: по-прежнему требуется реализация!
    */
    printf("Histogram of words\n");

    ht_init();
}

```

Обратите внимание, что были внесены два концептуально различных изменения: структура хеш-таблицы и хранилище, а также начало операции чтения файла. В идеале эти концепции вносились бы в программу в виде двух отдельных патчей. Для этого потребуется сделать пару шагов, но Git поможет правильно разделить эти изменения.

Git, как и большая часть свободного ПО, считает фрагментом любую серию строк из команды `diff`, разграниченных строкой, которая выглядит так:

```
@@ -1,7 +1,27 @@
```

Или так:

```
@@ -9,4 +29,6 @@ int main(int argc, char **argv)
```

В этом случае `git diff` показывает два фрагмента:

```

$ git diff
diff --git a/main.c b/main.c
index 9243ccf..b07f5dd 100644
--- a/main.c
+++ b/main.c
@@ -1,7 +1,27 @@
#include <stdio.h>
#include <stdlib.h>
+
+struct hentry {
+    char *item;
+    int count;
+    struct hentry *next;
+};
+
+struct hentry ht_table[256];
+
+void ht_init(void)
+{
+    /*     FIXME: details */
+}

int main(int argc, char **argv)
{
+    FILE *f;
+

```

```

+         f = fopen(argv[1], "r");
+         if (f == 0)
+             exit(-1);
+
+
+     /*
+      * Выводит гистограмму слов, содержащихся в файле.
+      * "Слова" – это любые символы, разделенные пробелом.
+@@ -9,4 +29,6 @@ int main(int argc, char **argv)
+      * FIXME: по-прежнему требуется реализация!
+      */
+     printf("Histogram of words\n");
+
+
+     ht_init();
+ }

```

Первый фрагмент начинается со строки @@ -1,7 +1,27 @@ и заканчивается в начале второго фрагмента: @@ -9,4 +29,6 @@ int main(int argc, char **argv).

При интерактивном индексировании с помощью `git add -p` Git поочередно предлагает выбор для каждого фрагмента: индексировать его или нет? (Stage this hunk?)

Рассмотрим патч повнимательнее и подумаем, нужно ли разделять его, чтобы концептуально связанные элементы оказались собраны вместе и были индексированы одновременно. Это означает, что в первом патче нужно индексировать все элементы хеш-таблиц, а во втором — все операции с файлами. Но похоже, что первый фрагмент содержит и хеш-таблицу, и операции с файлами. Выходит, что для выполнения первого коммита (то есть элементов с хеш-таблицами) его одновременно и нужно индексировать, и нет. Иными словами, нужно индексировать *часть* этого фрагмента. Если Git спрашивает нас только о первом и втором фрагментах, возникнут проблемы.

Но беспокоиться не стоит. Индексирование фрагментов поможет *разделить* их. В любом месте, где непрерывная последовательность добавленных и удаленных строк, обозначенная знаком «плюс» или «минус» в первой колонке, разрывается текстом исходного контекста, можно выполнить операцию разбиения.

Начнем с команды `git add -p main.c`:

```

$ git add -p
diff --git a/main.c b/main.c
index 4809266..c60b800 100644
--- a/main.c
+++ b/main.c
@@ -1,7 +1,27 @@
+ #include <stdio.h>
+ #include <stdlib.h>
+
+ struct hentry {
+     char *item;

```

```

+   int count;
+   struct htentry *next;
+};
+
+struct htentry ht_table[256];
+
+void ht_init(void)
+{
+   /* FIXME: details */
+}

int main(int argc, char **argv)
{
+   FILE *f;
+
+   f = fopen(argv[1], "r");
+   if (f == 0)
+       exit(-1);
+
+   /*
+    * Выводит гистограмму содержащихся в файле слов.
+    * "Слова" - это любые символы, разделенные пробелом.
+    Stage this hunk [y,n,q,a,d,/,j,J,g,s,e,?]?

```

Просмотрев этот фрагмент и обнаружив изменения, связанные с хеш-таблицей и операциями файлов, вы понимаете, что индексировать его нужно не полностью. Для этого вы указываете *s* (split, разделить) для разделения фрагмента на две части:

```

Stage this hunk [y,n,q,a,d,/,j,J,g,s,e,?]? s
Split into 2 hunks.
@@ -1,4 +1,18 @@
 #include <stdio.h>
+#include <stdlib.h>
+
+struct htentry {
+   char *item;
+   int count;
+   struct htentry *next;
+};
+
+struct htentry ht_table[256];
+
+void ht_init(void)
+{
+   /* FIXME: details */
+}

int main(int argc, char **argv)
{
Stage this hunk [y,n,q,a,d,/,j,J,g,e,?]?

```

Далее нужно ввести `y` (`yes`), указав тем самым, что вы хотите индексировать первую часть:

```
Stage this hunk [y,n,q,a,d,/,j,J,g,s,e,?]? y
```

Вы получите следующий код, который спрашивает, хотите ли вы проиндексировать вторую часть:

```
@@ -2,6 +16,12 @@
```

```
int main(int argc, char **argv)
{
+   FILE *f;
+
+   f = fopen(argv[1], "r");
+   if (f == 0)
+       exit(-1);
+
+   /*
+    * Выводит гистограмму содержащихся в файле слов.
+    * "Слова" – это любые символы, разделенные пробелом.
```

```
Stage this hunk [y,n,q,a,d,/,K,j,J,g,e,?]?
```

Вы не хотите это делать, поэтому вводите `n` (`no`):

```
Stage this hunk [y,n,q,a,d,/,j,J,g,s,e,?]? n
```

Наконец, Git предлагает индексировать второй фрагмент. Вы соглашаетесь и вводите `y`:

```
@@ -9,4 +29,6 @@ int main(int argc, char **argv)
+   * FIXME: Implementation needed still!
+   */
+   printf("Histogram of words\n");
+
+   ht_init();
+ }
```

```
Stage this hunk [y,n,q,a,d,/,j,J,g,s,e,?]? y
```

Разберемся, что произошло.

Изначально было два фрагмента, но вам требовалось индексировать только часть первого и весь второй. Поэтому, когда Git предложил индексировать первый фрагмент, его пришлось разделить на две части. После этого вы индексировали первую часть, но не вторую. В конце вы добавили в индекс весь второй фрагмент.

Вот как можно проверить корректность индексированных элементов:

```
$ git diff --staged
diff --git a/main.c b/main.c
index 4809266..8a95bb0 100644
--- a/main.c
```

```

+++ b/main.c
@@ -1,4 +1,18 @@
#include <stdio.h>
+#include <stdlib.h>
+
+struct htentry {
+    char *item;
+    int count;
+    struct htentry *next;
+};
+
+struct htentry ht_table[256];
+
+void ht_init(void)
+{
+    /* FIXME: details */
+}

int main(int argc, char **argv)
{
@@ -9,4 +23,6 @@ int main(int argc, char **argv)
    * FIXME: Implementation needed still!
    */
    printf("Histogram of words\n");
+
+    ht_init();
}

```

Все выглядит правильно, значит, можно делать коммит. Не беспокойтесь о том, что в файле `main.c` остались неиндексированные изменения. Это сделано намеренно, и они войдут в *следующий* патч. Не используйте то же имя файла в следующей команде `git commit`, поскольку так вы задействуете весь файл, а не индексированные части:

```

$ git commit -m "Introduce a Hash Table."
[main 66a212c] Introduce a Hash Table.
1 files changed, 16 insertions(+), 0 deletions(-)

```

```

$ git diff
diff --git a/main.c b/main.c
index 8a95bb0..c60b800 100644
--- a/main.c
+++ b/main.c
@@ -16,6 +16,12 @@ void ht_init(void)

int main(int argc, char **argv)
{
+    FILE *f;
+
+    f = fopen(argv[1], "r");
+    if (f == 0)
+        exit(-1);
}

```

```
+
/*
 * Выводит гистограмму содержащихся в файле слов.
 * "Слова" – это любые символы, разделенные пробелом.
```

Теперь добавьте и выполните коммит оставшихся изменений, поскольку они содержат все необходимые модификации для реализации патча операций с файлами:

```
$ git add main.c
$ git commit -m "Open the word source file."
[main e649d27] Open the word source file.
1 files changed, 6 insertions(+), 0 deletions(-)
```

Изучив историю коммитов, мы увидим два новых:

```
$ git log --graph --oneline
* e649d27 Open the word source file.
* 66a212c Introduce a Hash Table.
* 3ba81f7 Initial histogram program.
```

И это удачная последовательность патчей.

Как обычно, есть несколько оговорок и обстоятельств, о которых стоит упомянуть. Например, есть такая коварная строка:

```
#include <stdlib.h>
```

Не относится ли эта строка к патчу с файловыми операциями, а не к патчу хеш-таблиц? Да, именно так.

Обработать ее будет чуть сложнее. Для этого потребуется опция `e` (edit, редактировать). Сначала сбросьте первый коммит и оставьте все его изменения в рабочем дереве, чтобы можно было сделать все заново:

```
$ git reset 3ba81f7
Unstaged changes after reset:
M   main.c
```

Снова выполните `git add -p` и разделите первый патч, как и раньше. Но на этот раз на запрос об индексировании первой части ответьте `e`, чтобы войти в режим редактирования патча:

```
$ git add -p
diff --git a/main.c b/main.c
index 4809266..c60b800 100644
--- a/main.c
+++ b/main.c
@@ -1,7 +1,27 @@
 #include <stdio.h>
+#include <stdlib.h>
+
+struct hentry {
```

```

+   char *item;
+   int count;
+   struct htentry *next;
+};
+
+struct htentry ht_table[256];
+
+void ht_init(void)
+{
+   /* FIXME: details */
+}

int main(int argc, char **argv)
{
+   FILE *f;
+
+   f = fopen(argv[1], "r");
+   if (f == 0)
+       exit(-1);
+
+   /*
+    * Выводит гистограмму содержащихся в файле слов.
+    * "Слова" – это любые символы, разделенные пробелом.

```

Stage this hunk [y,n,q,a,d,/,j,J,g,s,e,?]? s

Split into 2 hunks.

@@ -1,4 +1,18 @@

```

#include <stdio.h>
+include <stdlib.h>
+
+struct htentry {
+   char *item;
+   int count;
+   struct htentry *next;
+};
+
+struct htentry ht_table[256];
+
+void ht_init(void)
+{
+   /* FIXME: details */
+}

int main(int argc, char **argv)
{

```

```

Stage this hunk [y,n,q,a,d,/,j,J,g,e,?]? e

```

Откроется выбранный вами редактор¹, и вы сможете вручную отредактировать патч. Прочтите комментарий в нижней части буфера редактора. Осторожно удалите строку `#include <stdlib.h>`, не меняя остальные строки или их количество.

¹ Это же Емакс, правда?

Git и большинство программ для работы с патчами не смогут обработать патч, если вы измените контекстные строки.

Поскольку строка `#include` была удалена, она будет включена в следующие патчи. Это позволяет ее вставить в правильное место в патче с другими изменениями операций с файлами.

Здесь немного запутанно, но теперь Git предполагает, что, когда вы выйдете из редактора, нужно применить оставшийся в нем патч и индексировать его изменения. Поэтому система предложит *приведенный ниже* фрагмент, позволив выбрать его размещение. Будьте внимательны.

Поскольку Git перешел к изменениям операций с файлами, не индексируйте эти изменения, но примите последнее изменение хеш-таблицы:

```
@@ -2,6 +16,12 @@

    int main(int argc, char **argv)
    {
+     FILE *f;
+
+     f = fopen(argv[1], "r");
+     if (f == 0)
+         exit(-1);
+
+     /*
+      * Выводит гистограмму содержащихся в файле слов.
+      * "Слова" – это любые символы, разделенные пробелом.
Stage this hunk [y,n,q,a,d,/,K,j,J,g,e,?]? n
@@ -9,4 +29,6 @@ int main(int argc, char **argv)
+     * FIXME: Implementation needed still!
+     */
+     printf("Histogram of words\n");
+
+     ht_init();
+ }
Stage this hunk [y,n,q,a,d,/,K,g,e,?]? y
```

Разделение можно проверить: обратите внимание, что строка `#include <stdlib.h>` теперь корректно связана с файловыми операциями:

```
$ git diff
diff --git a/main.c b/main.c
index 3e77315..c60b800 100644
--- a/main.c
+++ b/main.c
@@ -1,4 +1,5 @@
+ #include <stdio.h>
+ #include <stdlib.h>

struct hentry {
    char *item;
```



```
@@ -15,6 +16,12 @@ void ht_init(void)

int main(int argc, char **argv)
{
+   FILE *f;
+
+   f = fopen(argv[1], "r");
+   if (f == 0)
+       exit(-1);
+
+   /*
+    * Выводит гистограмму содержащихся в файле слов.
+    * "Слова" – это любые символы, разделенные пробелом.
```

Как и ранее, завершаем этап командой `git commit` для патча хеш-таблиц, после чего индексируем и фиксируем оставшуюся часть с файловыми операциями.

Мы лишь кратко затронули основные варианты ответов на вопрос: «Индексировать этот фрагмент?» Помимо операций, перечисленных в запросе на ввод (то есть `[y,n,q,a,d,/,k,g,e,?]`), доступны и другие опции. Например, возможность отложить решение о том, индексировать ли фрагмент, и вернуться к нему, когда система вновь о нем спросит.

Приведенный пример рассматривал всего два фрагмента в одном файле, но операция индексирования обобщает огромное множество фрагментов, разделенных на множество файлов. Совмещение изменений, распределенных по нескольким файлам, может заключаться в простом применении `git add -p` к каждому файлу, содержащему фрагмент, требующий индексирования.

Есть еще один, более общий уровень процесса интерактивного индексирования, доступный через команду `git add -i`. Он может показаться запутанным, но его задача — дать вам выбрать опцию `patch` для нужных путей. Это запускает описанный ранее механизм индексирования для каждого файла.

Полезная команда `git rev-list`

Однажды Джону пришло письмо:

Джон, привет!

Пытаюсь понять, как переключиться из репозитория Git на пустой рабочий каталог на основе даты. Тщательно пролистал документацию, но понятнее не стало.

Эрик

Так бывает. Попробуем разобраться в проблемных местах документации.

Переключение на основе даты

На первый взгляд кажется, что должна сработать команда вроде `git checkout main@{Jan 1, 2011}`. Но эта команда для разрешения обращения к ссылке `main` на основе даты использует `reflog` (см. раздел «Локальное хранилище» на с. 238 в главе 10). Есть много случаев, когда такая, казалось бы, безобидная конструкция не сработает. Например, в вашем репозитории отключен журнал ссылок (`reflog`), или вы не можете задействовать ссылку `main` в течение указанного периода, или журнал ссылок содержит устаревшие ссылки из этого периода. Эта конструкция также может не дать вам ожидаемого ответа. Команда запрашивает журнал ссылок для определения местоположения вашей основной ветки на момент, когда вы что-то делали с веткой, а не согласно временной линии коммитов ветки. Эти моменты могут быть связаны (но не всегда), особенно если вы разрабатывали и фиксировали историю ветки в этом репозитории.

Такой подход может завести в тупик. Использование журнала ссылок *может* дать искомую информацию, а может и завершиться неудачей, поэтому надежным его считать нельзя.

Используйте команду `git rev-list`. Это универсальное рабочее решение, позволяющее объединить множество опций, разобраться в сложной истории большого числа веток, предвидеть потенциально неточные указания пользователей, ограничить пространство поиска и, наконец, найти выбранные коммиты в истории репозитория. Команда выдает один или более SHA-1 ID для использования другими инструментами. Можете рассматривать `git rev-list` со всем ее множеством опций как фронтенд-инструмент для обращения к базе данных коммитов репозитория.

В этом случае цель проста: найти один коммит, который существовал в указанной ветке сразу же после указанной даты, и переключиться на него.

Далее используем репозиторий исходного кода Git, поскольку он имеет обширную и доступную для исследования историю. Для начала применим `rev-list`, чтобы найти SHA-1. Опция `-n 1` ограничит вывод команды одним ID коммита.

Найдем последний коммит в основной ветке от 2011 года:

```
$ https://github.com/git/git.git
Cloning into 'git'...
remote: Counting objects: 126850, done.
remote: Compressing objects: 100% (41033/41033), done.
remote: Total 126850 (delta 93115), reused 117003 (delta 84141)
Receiving objects: 100% (126850/126850), 27.56 MiB | 1.03 MiB/s, done.
Resolving deltas: 100% (93115/93115), done.

$ cd git
$ git rev-list -n 1 --before="Jan 1, 2012 00:00:00" main
0eddcbf1612ed044de586777b233caf8016c6e70
```

Обнаружив нужный коммит, можно его использовать, снабдить тегом, сослаться или даже переключиться на него. Но примечание при переключении сообщает, что вы находитесь в отсоединенной HEAD:

```
$ git checkout 0eddcb
Note: switching to '0eddcb'.
```

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may do so (now or later) by using `-c` with the switch command. Example:

```
git switch -c <new-branch-name>
```

Or undo this operation with:

```
git switch -
```

Turn off this advice by setting config variable `advice.detachedHead` to `false`

HEAD is now at 0eddcbf161 Add MYMETA.json to perl/.gitignore

А действительно ли это нужный коммит?

```
$ git log -1 --pretty=fuller
commit 0eddcbf1612ed044de586777b233caf8016c6e70
Author: Jack Nagel <jacknagel@gmail.com>
AuthorDate: Wed Dec 28 22:42:05 2011 -0600
Commit: Junio C Hamano <gitster@pobox.com>
CommitDate: Thu Dec 29 13:08:47 2011 -0800
```

```
Add MYMETA.json to perl/.gitignore
...
```

При выборе даты с помощью `rev-list` используется поле `CommitDate`, а не `AuthorDate`. И похоже, что последний коммит 2011 года в репозитории Git был 29 декабря.

Предостережения при переключении на основе даты

Рассмотрим некоторые нюансы. Работа с датами в Git реализуется с помощью функции `approximate()`. Это не значит, что даты приблизительные. Это значит, что Git делает приблизительную интерпретацию вашего запроса, обычно из-за недостаточной детализации или точности:

```
$ git rev-list -n 1 --before="Jan 1, 2012 00:00:00" main
0eddcbf1612ed044de586777b233caf8016c6e70
```

```
$ git rev-list -n 1 --before="Jan 1, 2012" main
5c951ef47bf2e34dbde58bda88d430937657d2aa
```

Мы ввели эти команды в 11:05 по местному времени. Из-за отсутствия указанного *времени* во второй команде Git предположил, что мы подразумевали «в это время 1 января 2012 года». В результате диапазон, в котором могли оказаться соответствующие коммиты, расширился на 11 дополнительных часов:

```
$ git log -1 --pretty=fuller 5c951ef
commit 5c951ef47bf2e34dbde58bda88d430937657d2aa
Author:      Clemens Buchacher <drizzd@aon.at>
AuthorDate:  Sat Dec 31 12:50:56 2011 +0100
Commit:     Junio C Hamano <gitster@pobox.com>
CommitDate: Sun Jan 1 01:18:53 2012 -0800
```

Documentation: read-tree --prefix works with existing subtrees

...

Коммит произошел через 1 час 18 минут после наступления нового года — значительно позднее, чем 11 часов после полуночи, которые мы ошибочно указали во второй команде.

ИНТЕРПРЕТАЦИЯ ДАТ В GIT

Имеет ли смысл такое поведение при интерпретации дат в Git? Возможно.

Git пытается угадать, что подразумевается в нечетко прописанных запросах по времени. Например, как трактовать **yesterday** (вчера)? Как предыдущий промежуток из 24 часов? Или как период абсолютного времени от полуночи до полуночи предыдущего календарного дня? Может, под этим подразумевались рабочие часы вчерашнего дня? Git использует первый вариант интерпретации, то есть рассматривает 24 часа, предшествующих текущему моменту. Получается, что любая дата, которая используется в качестве стартовой или конечной точки в Git, сопоставляется с текущим временем. Если дата указана без времени, то подразумевается именно текущее время. Если бы вы хотели более точно выразить свой запрос в отношении вчера, то могли бы указать что-то вроде **noon yesterday** (вчера в полдень) или **5pm yesterday** (вчера в пять часов вечера).

Другой нюанс о переключении на основе даты: хотя вы можете получить действительный ответ на ваш запрос о конкретном коммите, тот же вопрос в последующий момент времени может дать другой ответ.

Рассмотрим репозиторий, где на разных ветках ведется несколько линий разработки. Когда вы запрашиваете коммит в указанной ветке **--before дата**, то вы получаете ответ для ветки в ее текущем состоянии. Позже в нее могут быть внесены новые коммиты из других веток, и для поиска того же коммита понадобится уже другой запрос. Например, в предыдущем примере с 1 января 2012 года

кто-то мог внести в 13:08:47 коммит из другой ветки, который оказался бы ближе к полуночи 31 декабря 2011 года, чем к 29 декабря 2011-го.

Извлечение старой версии файла

Иногда при исследовании старых программных компонентов может потребоваться просто извлечь из истории репозитория старую версию файла. При этом использование техник переключения на основе даты, описанных в разделе «Переключение на основе даты», покажется избыточным, поскольку это вызовет полное изменение состояния рабочего каталога для каждого подкаталога и файла. Скорее всего, вы хотите сохранить текущее состояние каталога, но заменить имеющуюся версию всего одного файла более старой версией.

Первым шагом будет определение коммита, содержащего нужную версию файла. Прямолинейный подход подразумевает использование конкретной ветки, тега или ссылки, в которой уже содержится корректная версия. Если же такой информации нет, придется прибегнуть к поиску. При поиске истории коммитов стоит обратить внимание на техники `rev-list` для обнаружения коммитов, содержащих нужный файл. Для выбора интересующих коммитов можно использовать даты. Git также позволяет ограничивать поиск конкретным файлом или их набором. Такой подход называется *ограничением пути* (path limiting). Он предоставляет полноценный путеводитель к возможным предыдущим коммитам, которые могут содержать разные версии файла или, как их называет Git, *пути*.

Обратимся к репозиторию исходного кода Git, чтобы найти доступные версии, например, файла `date.c`:

```
$ https://github.com/git/git.git
Cloning into 'git'...
remote: Counting objects: 126850, done.
remote: Compressing objects: 100% (41033/41033), done.
remote: Total 126850 (delta 93115), reused 117003 (delta 84141)
Receiving objects: 100% (126850/126850), 27.56 MiB | 1.03 MiB/s, done.
Resolving deltas: 100% (93115/93115), done.

# Мы заменили предустановленное имя ветки на main
# с помощью команды git branch -M.
$ git rev-list main -- date.c
974c919d36d944e9005def346fb363d8a83399f7
f1e9c548ce45005521892af0299696204ece286b
...
89967023da94c0d874713284869e1924797d30bb
ec9e9d9e793c7573cf3730fb9746527a0a7e94e7
```

Неплохо. Где-то 120 с лишним строк ID коммитов. Но что это значит? И как это использовать?

Поскольку мы не указали опцию `-n 1`, система сгенерировала и вывела ID всех соответствующих запросу коммитов. По умолчанию они выводятся в обратном

хронологическом порядке. То есть коммит `ee646e` содержит самую свежую версию файла `date.c`, а коммит `ec99d9` — самую старую. Фактически при рассмотрении `ec99d9` мы видим, что файл вносится в репозиторий впервые.

```
$ git show --stat ec99d9 --pretty=short
commit ec99d9e793c7573cf3730fb9746527a0a7e94e7
Author: Edgar Toernig <froese@gmx.de>
```

```
[PATCH] Do date parsing by hand...
```

```
Makefile      |    4 +-
cache.h       |    3 +
commit-tree.c |   27 +-----
date.c        |  184 +++++++++++++++++++++++++++++++++++++++++++++++++++++
4 files changed, 191 insertions(+), 27 deletions(-)
```

Определить дальнейшие действия с этой точки в поиске нужного коммита — непростая задача. Можно выполнять операции `git log` для произвольно выбранных значений SHA-1 из вывода `rev-list list`. Также можно использовать двоичный поиск по временным меткам коммитов из этого списка. До выбора последнего коммита мы использовали опцию `-n 1`. Сложно сказать, какой прием сработает в вашем случае для точного обнаружения коммита, содержащего нужную версию файла.

Но как затем использовать один из таких коммитов, когда вы его найдете? Как выглядит искомая версия `date.c`? Что, если нужно извлечь ее сразу по месту?

Есть три разных подхода к получению этой версии файла. При первом мы напрямую переключаемся на названную версию и переписываем ею версию, существующую в рабочем каталоге:

```
$ git checkout ec99d9 date.c
Updated 1 path from 0cd8a2506a
```

При выполнении двух других похожих команд Git получает выражение `commit:path`, именующее нужный файл (то есть путь) в той форме, в какой он существовал на момент коммита, и записывает указанную версию в `stdout`. Что делать с этим выводом, зависит от вас. Можете направить вывод на другие команды или создать файлы:

```
$ git show ec99d9:date.c > date.c-oldest
```

Или:

```
$ git cat-file -p 89967023:date.c > date.c-first-change
```

Разница между этими двумя формами немного загадочна. Первая прогоняет файл через любые применимые фильтры преобразования текста, а вторая выступает более базовой командой без фильтрации. Различия могут проявиться при управлении двоичными файлами, когда настроены фильтры `textconv`, или во

время изменения обработки переноса строки. Если вам нужны сырые данные, используйте форму `cat -p`. Если нужна преобразованная версия в том виде, в каком она будет находиться после переключения или добавления в новый репозиторий, используйте форму `show`.

Восстановление утраченного коммита

Иногда несвоевременное выполнение команды `git reset` или случайное удаление ветки заставляет пожалеть об утраченной проделанной работе, и вы задумываетесь, как ее восстановить. Типичным подходом будет изучение журнала ссылок (см. главу 10). Но иногда журнал оказывается недоступен, поскольку отключен (например, установкой `core.logAllRefUpdates = false`), устарел или в нем не сохраняется история, так как вы работаете с голым репозиторием. Журнал ссылок не всегда поможет восстановить утраченный коммит.

Команда `git fsck`

Git предоставляет команду `git fsck`, с помощью которой можно найти утраченные данные. Слово `fsck` — старая аббревиатура для «file system check» (проверка файловой системы). И хотя эта команда на самом деле не выполняет проверку файловой системы, у нее есть много характеристик и алгоритмов, которые напоминают такую проверку и дают на выходе некоторые аналогичные результаты.

Чтобы понять работу `git fsck`, важно хорошо разбираться со структурой данных Git, описанных в главе 2. Как правило, каждый объект в репозитории Git, будь то blob-объект, дерево, коммит или тег, связан с другим объектом и прикреплен к имени ветки, имени тега или некой другой символической ссылке, такой как имя в журнале ссылок.

Однако различные команды и действия могут оставлять в объектном хранилище объекты, по какой-то причине не связанные с целостной структурой данных. Эти объекты называются *недостижимыми* или *висячими* (dangling). Они недостижимы, потому что обход полной структуры данных, начинающийся с каждой именованной ссылки и следующий за каждым тегом, коммитом, родителем коммита и ссылкой на объект дерева, никогда не столкнется с потерянным объектом. Буквально объект «подвешен в пустом пространстве».

Но обход основанного на ссылках графа коммитов — не единственный способ обойти каждый объект базы данных. Можно вывести список объектов из их хранилища с помощью `ls`:

```
$ cd path/to/some/repo
$ ls -R .git/objects/
.git/objects/:
25 3b 73 82 info pack
```

```
.git/objects/25:
7cc5642cb1a054f08cc83f2d943e56fd3ebe99
```

```
.git/objects/3b:
d1f0e29744a1f32b08d5650e62e2e62afb177c
```

```
.git/objects/73:
8d05ac5663972e2dcf4b473e04b3d1f19ba674
```

```
.git/objects/82:
b5fee28277349b6d46beff5fd6a7152347ba0
```

```
.git/objects/info:
```

```
.git/objects/pack:
```

В этом примере множество объектов репозитория перечислены без обхода ссылок и коммитов.

Тщательно сравнивая общее множество объектов с теми, которые достижимы через обход основанного на ссылках графа коммитов, вы можете определить все объекты, на которые не ведут ссылки. В предыдущем примере второй объект из перечисленных оказался blob-объектом (то есть файлом), на который нет ссылки:

```
$ git fsck
Checking object directories: 100% (256/256), done.
dangling blob 3bd1f0e29744a1f32b08d5650e62e2e62afb177c
```

Разберем на примере, как может возникнуть потерянный коммит и как его восстановить с помощью `git fsck`. Построим новый простой репозиторий, содержащий один файл:

```
$ mkdir /tmp/lost
$ cd /tmp/lost
$ git init -b main
Initialized empty Git repository in /tmp/lost/.git/
$ echo "foo" >> file
$ git add file
$ git commit -m "Add some foo"
[main (root-commit) 1adf46e] Add some foo
 1 files changed, 1 insertions(+), 0 deletions(-)
 create mode 100644 file
```

```
$ git fsck
Checking object directories: 100% (256/256), done.
```

```
$ ls -R .git/objects
.git/objects/:
25 2d 4a info pack
```

```
.git/objects/25:
7cc5642cb1a054f08cc83f2d943e56fd3ebe99
```



```
.git/objects/2d:
491f5bbeed7c7e28290e93b5695ce4dd7401f1
```

```
.git/objects/4a:
1c03029e7407c0afe9fc0320b3258e188b115e
```

```
.git/objects/info:
```

```
.git/objects/pack:
```

Здесь всего три объекта, но ни один из них не висячий. Начиная со ссылки `main`, которой является объект коммита `2d491f5`, обход указывает на объект дерева `4a1c0302`, а затем на blob-объект `257cc564`.



Определить тип объекта можно командой `git cat-file -t SHA1 ID`.

Сделаем второй коммит и произведем принудительный откат к первому:

```
$ echo bar >> file
$ git add file
$ git commit -m "Add some bar"
[main 66919fa] Add some bar
1 files changed, 1 insertions(+), 0 deletions(-)
```

А вот и «случайность», приводящая к потере коммита:

```
$ git reset --hard HEAD^
HEAD is now at 2d491f5 Add some foo
```

```
$ git fsck
Checking object directories: 100% (256/256), done.
```

Подождите-ка! `git fsck` не сообщает ни о каком висячем объекте. Похоже, он не потерялся. Именно для этого и задуман журнал ссылок — предотвращать случайную потерю коммитов (см. раздел «Журнал ссылок (reflog)» на с. 250 в главе 10). Повторим попытку после грубого удаления журнала:

```
# Так делать не нужно. Этот пример — только для учебных целей
$ rm -rf .git/logs
$ git fsck
Checking object directories: 100% (256/256), done.
dangling commit 66919fae5a5f62afc0a349cc216bb0c59ffe37d0
```

```
$ ls -R .git/objects
.git/objects/:
25  2d  3b  41  4a  66  info pack

.git/objects/25:
7cc5642cb1a054f08cc83f2d943e56fd3ebe99
```

```
.git/objects/2d:
491f5bbeed7c7e28290e93b5695ce4dd7401f1
```

```
.git/objects/3b:
d1f0e29744a1f32b08d5650e62e2e62afb177c
```

```
.git/objects/41:
31fe4d33cd85da805ac9a6697c2251c913881c
```

```
.git/objects/4a:
1c03029e7407c0afe9fc0320b3258e188b115e
```

```
.git/objects/66:
919fae5a5f62afc0a349cc216bb0c59ffe37d0
```

```
.git/objects/info:
```

```
.git/objects/pack:
```



Находить всякие объекты можно с помощью команды `git fsck --no-reflog` так, будто журнал ссылок недоступен. Это приведет к тому, что объекты, достижимые только из журнала ссылок, будут считаться недостижимыми.

Теперь мы видим, что со вторым коммитом `66919fae`, в котором добавлялось содержимое `bar`, был связан только журнал ссылок.

Но как понять, что такое всякий коммит?

```
$ git show 66919fae
commit 66919fae5a5f62afc0a349cc216bb0c59ffe37d0
Author: Jon Loeliger <jdl@example.com>
Date: Sun Jul 10 17:16:28 2022 +0200
```

```
Add some bar
```

```
diff --git a/file b/file
index 257cc56..3bd1f0e 100644
--- a/file
+++ b/file
@@ -1,2 @@ foo
+bar
```

В строке "index" выше указывается blob-объект `3bd1f0e`.

```
$ git show 3bd1f0e
foo bar
```

Заметьте, что blob-объект `3bd1f0e` не считается всячим, поскольку на него ссылается коммит `66919fae`, хотя на сам этот коммит ссылок нет.

Иногда `git fsck` будет находить blob-объекты, на которые не ведут ссылки. Запомните: при каждом `git add` файла в индекс в объектное хранилище добавляется его blob-объект. Если вы позже измените содержимое и снова его добавите, никакой коммит не отразит промежуточный blob-объект, который добавлен в хранилище, и тот в итоге окажется висячим:

```
$ echo baz >> file
$ git add file
$ git fsck
Checking object directories: 100% (256/256), done.
dangling commit 66919fae5a5f62afc0a349cc216bb0c59ffe37d0

$ echo quux >> file
$ git add file
$ git fsck
Checking object directories: 100% (256/256), done.
dangling blob 0c071e1d07528f124e31f1b6c71348ec13f21a7a
dangling commit 66919fae5a5f62afc0a349cc216bb0c59ffe37d0
```

Команда `git fsck` не показала висячий blob-объект, потому что на него все еще вела прямая ссылка из индекса. Только после того, как содержимое, связанное с `file`, было снова изменено и повторно добавлено, этот blob-объект оказался висячим:

```
$ git show 0c071e1d
foo
baz
```

Если вы получили отчет `git fsck`, перегруженный blob-объектами и коммитами, и захотите почистить его, то подумайте о сборке мусора, описанной в разделе «Сборка мусора» на с. 419 в главе 17.

Восстановление связи с потерянным коммитом

Команда `git fsck` предоставляет удобный способ обнаружения SHA-1 потерянных коммитов и blob-объектов. Ранее в качестве другого механизма мы упоминали также журнал ссылок. Фактически вы можете скопировать его из какой-то оставшейся строки вывода, находящейся в нижней части вывода. В итоге неважно, как вы найдете SHA-1 потерянного blob-объекта или коммита. Остается вопрос: когда вы узнали SHA-1, как можно восстановить связь с коммитом в репозитории Git и интегрировать его обратно в проект?

По определению blob-объекты — это безымянное содержимое файлов. Все, что вам нужно сделать для восстановления такого объекта, — поместить его содержимое в файл и заново выполнить `git add`. Как мы показали в предыдущем разделе, чтобы получить все содержимое blob-объекта, можно применить к его SHA-1 команду `git show`. Перенаправьте его на нужный файл:

```
$ git show 0c071e1d > file2
```

Выбор способа воссоединения с коммитом зависит от того, что вы планируете с ним делать. Простой пример в предыдущем разделе рассматривает лишь один коммит. Но этот коммит мог вполне оказаться лишь первым в целой цепочке потерянных. Возможно, случайно была потеряна даже вся ветка. Поэтому общепринятой практикой является повторное внесение всяческого коммита в виде ветки.

Здесь ранее потерянный коммит, 11e0dc9c, вносивший содержимое “bar”, заново включается в новую ветку `recovered`:

```
$ git branch recovered 66919fae
$ git show-branch
* [main] Add some foo
! [recovered] Add some bar
--
+ [recovered] Add some bar
*+ [main] Add some foo
```

Оттуда им уже можно манипулировать на ваше усмотрение.

Использование `git filter-repo`

Команду `git filter-repo` (<https://oreil.ly/tW4qJ>) придумал и разработал Элайджа Ньюрен (Elijah Newren). Ее исходный код размещен в публичном репозитории на GitHub. Команда позволяет переписывать всю историю коммитов репозитория, используя широкий спектр доступных опций фильтрации. Часть этих фильтров работает для коммитов, часть — для blob-объектов, объектов деревьев, структур каталогов, а часть позволяет определять функцию в виде универсальных обратных вызовов.

До появления команды `git filter-repo` общей командой для обработки веток была `git filter-branch`. Она позволяла произвольно переписывать коммиты ветки, используя пользовательские команды, действующие для разных объектов в репозитории. Тем не менее в руководстве по `git filter-branch` *предупреждающий* раздел (<https://oreil.ly/H7vGj>) описывает ее как имеющую множество нюансов, которые могут привести к путанице при явном переписывании истории репозитория. Руководство рекомендует использовать `git filter-repo` лишь в качестве альтернативного инструмента фильтрации истории.



Если вам все-таки нужно использовать команду `git filter-branch`, то делайте это с учетом ее негативного влияния на производительность. Рекомендуем прочитать раздел руководства, посвященный *безопасности* при работе с этой командой.

Команда `git filter-repo` полезна и опасна одновременно.

Как вы могли предположить, с большой силой приходит и большая ответственность¹. Мощность и назначение `git filter-repo` также выступают причиной для другого предупреждения. Поскольку команда переписывает всю историю коммитов репозитория, ее выполнение в отношении репозитория, уже опубликованного для клонирования и использования другими, наверняка создаст владельцам клонов неудобства. Как и в случае любых операций перебазирования, изменится история коммитов. После выполнения этой команды для репозитория следует рассматривать все имеющиеся на тот момент клоны как устаревшие.

С учетом этого предупреждения выясним, на что способна эта команда, когда и почему может быть полезна и как использовать ее ответственно.

Примеры использования `git filter-repo`

Разберем несколько случаев, в которых использование `git filter-repo` будет продуктивным. Одна из наиболее актуальных ситуаций — когда у вас есть закрытый репозиторий, который вы хотите почистить или провести масштабное изменение, прежде чем делать его доступным для клонирования и общего использования.

Объяснения строятся на основе примеров, приведенных в руководстве по использованию команды. В нем четко описаны назначение и порядок применения кода, поэтому мы решили реализовать пример, который познакомит вас с разными сценариями.



Сценарии опубликованы с разрешения автора инструмента Элайджи Нью-рена.

Рекомендуем клонировать указанный ниже репозиторий, чтобы вы могли повторить приведенные примеры:

```
$ git clone https://github.com/ppremk/analyze-this.git
Cloning into 'analyze-this'...
remote: Enumerating objects: 22, done.
remote: Counting objects: 100% (22/22), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 22 (delta 7), reused 22 (delta 7), pack-reused 0
Receiving objects: 100% (22/22), 36.25 KiB | 337.00 KiB/s, done.
Resolving deltas: 100% (7/7), done.
```

```
$ cd analyze-this
```

¹ Конечно же, Франсуа-Мари Аруэ!

Установка git-filter-repo

Поскольку `git-filter-repo` — это скрипт на Python, состоящий из одного файла, проще всего установить его с помощью пакетного менеджера для вашей операционной системы:

```
$ [PACKAGE_MANAGER] install git-filter-repo
```



Инструкции по установке скрипта `git-filter-repo` вместе с другими сопутствующими опциями доступны онлайн (<https://oreil.ly/ey8Jd>).

Анализ репозитория

При передаче с опцией `--analyze` команда `git filter-repo` проанализирует историю репозитория и выдаст отчет, на который вы сможете сослаться, прежде чем решить, какие шаги предпринять для изменения репозитория. Команду можно использовать и после изменения, чтобы убедиться в получении желаемого результата.

Выполнение этой команды не приведет к обновлению репозитория:

```
$ git filter-repo --analyze
Processed 9 blob sizes
Processed 5 commits
Writing reports to .git/filter-repo/analysis...done.
```

```
$ tree .git/filter-repo/analysis
.git/filter-repo/analysis
├── README
├── blob-shas-and-paths.txt
├── directories-all-sizes.txt
├── directories-deleted-sizes.txt
├── extensions-all-sizes.txt
├── extensions-deleted-sizes.txt
├── path-all-sizes.txt
├── path-deleted-sizes.txt
└── renames.txt
```

`README` объясняет, как трактовать содержимое этих файлов. В нашем случае мы изменим клон репозитория, удалим часть объемных и ненужных файлов и переименуем некоторые каталоги.

Сначала рассмотрим разные файлы и их размеры:

```
$ cat .git/filter-repo/analysis/blob-shas-and-paths.txt
=== Files by sha and associated pathnames in reverse size ===
Format: sha, unpacked size, packed size, filename(s) object stored as
04b334ff842a8ce96abd1d16b7904eacb649839f 35000000 34042 bigfile3.exe
6f2bb1c9e846f0546416e8ec33bc007f353aefff 30000000 797 bigfile2.exe
```

```

d38148a863256f6af5aee0edb6020c4e87c2c24b 25000000 667 bigfile1.exe
31204afb3d72e8c0f95fde7add90e3893421f422 2000000 77 file6.md
7c2624a6b9687e88178638cd95b609c329177ade 1000000 50 [file4.md, file5.md]
08869d950c7543382974f936596e462c1dcc0eaa 24 34 README-v0.md
861761f28976d55103acf61e8423cd32779d7174 20 30 README.md
99cf872931885fa266f3dc4d998092f087dc120a 14 23 scripts/env.config
e69de29bb2d1d6434b8b29ae775ad8c2e48c5391 0 9 [logs/client.log,
                                           logs/server....

```

В репозитории есть большие файлы, которые определенно не должны там находиться. Есть еще несколько мелочей, которые мы исправим в последующих разделах.

Фильтрация на основе пути

Для примера удалим файл и каталог, а также переименуем два каталога. Но сначала посмотрим на историю коммитов этого репозитория и его содержимое:

```

$ git log --oneline
* e2a20a2 (HEAD -> main, origin/main, origin/HEAD) add folders
* 19263cb add remaining files
* 9ea727a add bigfiles
* 1d07224 add updated readme
* cd654e0 initial commit

$ ls -al
total 7896
drwxr-xr-x 14 ppremk stax      448 Mar 19 22:10 .
drwxr-xr-x  5 ppremk stax      160 Mar 19 22:10 ..
drwxr-xr-x 13 ppremk stax      416 Mar 19 22:19 .git
-rw-r--r--  1 ppremk stax       24 Mar 19 22:10 README-v0.md
-rw-r--r--  1 ppremk stax       20 Mar 19 22:10 README.md
-rw-r--r--  1 ppremk stax 25000000 Mar 19 22:10 bigfile1.exe
-rw-r--r--  1 ppremk stax 30000000 Mar 19 22:10 bigfile2.exe
-rw-r--r--  1 ppremk stax 35000000 Mar 19 22:10 bigfile3.exe
-rw-r--r--  1 ppremk stax 1000000 Mar 19 22:10 file4.md
-rw-r--r--  1 ppremk stax 1000000 Mar 19 22:10 file5.md
-rw-r--r--  1 ppremk stax 2000000 Mar 19 22:10 file6.md
drwxr-xr-x  4 ppremk stax      128 Mar 19 22:10 logs
drwxr-xr-x  3 ppremk stax       96 Mar 19 22:10 scripts
drwxr-xr-x  4 ppremk stax      128 Mar 19 22:10 toolkit

```

Удалим нежелательный файл README-v0.md:

```

$ git filter-repo --path README-v0.md --invert-paths
Parsed 5 commits
New history written in 0.20 seconds; now repacking/cleaning...
Repacking your repo and cleaning out old unneeded objects
HEAD is now at afd2aa7 add folders
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.

```

```
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (19/19), done.
Total 19 (delta 6), reused 11 (delta 4), pack-reused 0
Completely finished after 0.40 seconds.
```

Обратите внимание, как выглядит история коммитов после удаления этого файла.

```
$ git log --oneline
* afd2aa7 (HEAD -> main) add folders
* bb4ce63 add remaining files
* 2f3721a add bigfiles
* 3f9bd4a add updated readme
```

При использовании фильтра `-path` вы можете удалить один или больше файлов либо каталогов. Если при этом не указать опцию `--invert-paths`, то команда `git filter-repo` удалит все файлы, кроме указанного файла или каталога.

При внимательном рассмотрении вывода `git log` можно заметить, что информация удаленного отслеживания репозитория после изменения удалена. Это делается для того, чтобы предотвратить случайную передачу изменений репозитория в вышестоящий репозиторий и избежать нежелательных последствий в случае принудительного переписывания старой версии репозитория новой версией его истории коммитов.

Удалим ненужный каталог `logs`:

```
$ git filter-repo --path logs/ --invert-paths
Parsed 4 commits
New history written in 0.08 seconds; now repacking/cleaning...
Repacking your repo and cleaning out old unneeded objects
HEAD is now at ceacc4b add folders
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (18/18), done.
Total 18 (delta 6), reused 14 (delta 4), pack-reused 0
Completely finished after 0.31 seconds.
```

Обратите внимание, как изменилась история после удаления каталога.

```
$ git log --oneline
* ceacc4b (HEAD -> main) add folders
* bb4ce63 add remaining files
* 2f3721a add bigfiles
* 3f9bd4a add updated readme
```

Переименуем каталог `scripts` в `configs`, чтобы его название лучше отражало назначение. Переименуем также каталог `toolkit` в `scripts`: для этого используем фильтр `--path-rename`:


```
$ git filter-repo --path-rename scripts:configs --path-rename toolkit:scripts
Parsed 4 commits
New history written in 0.08 seconds; now repacking/cleaning...
Repacking your repo and cleaning out old unneeded objects
HEAD is now at ba656b4 add folders
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (18/18), done.
Total 18 (delta 6), reused 14 (delta 4), pack-reused 0
Completely finished after 0.27 seconds.
```

Обратите внимание, как выглядит история после переименования каталогов.

```
$ git log --oneline
* ba656b4 (HEAD -> main) add folders
* bb4ce63 add remaining files
* 2f3721a add bigfiles
* 3f9bd4a add updated readme
```

```
$ ls -l
total 7888
-rw-r--r-- 1 ppremk stax 20B Mar 19 22:10 README.md
-rw-r--r-- 1 ppremk stax 24M Mar 19 22:10 bigfile1.exe
-rw-r--r-- 1 ppremk stax 29M Mar 19 22:10 bigfile2.exe
-rw-r--r-- 1 ppremk stax 33M Mar 19 22:10 bigfile3.exe
drwxr-xr-x 3 ppremk stax 96B Mar 19 22:30 configs
-rw-r--r-- 1 ppremk stax 977K Mar 19 22:10 file4.md
-rw-r--r-- 1 ppremk stax 977K Mar 19 22:10 file5.md
-rw-r--r-- 1 ppremk stax 1.9M Mar 19 22:10 file6.md
drwxr-xr-x 4 ppremk stax 128B Mar 19 22:30 scripts
```

Отлично! Изменение принимает отчетливые очертания. Но похоже, нужно удалить еще некоторые большие файлы, которые не должны находиться в репозитории. Для точного выполнения этой задачи сопроводим команду `git filter-repo` опцией `--strip-blobs-bigger-than`.

Начнем с удаления файлов, превышающих определенный размер:

Удаляем файлы больше 5 Мбайт.

```
$ git filter-repo --strip-blobs-bigger-than 5M
Processed 8 blob sizes
Parsed 4 commits
New history written in 0.10 seconds; now repacking/cleaning...
Repacking your repo and cleaning out old unneeded objects
HEAD is now at eea637d add folders
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
```

```
Writing objects: 100% (13/13), done.
Total 13 (delta 2), reused 8 (delta 1), pack-reused 0
Completely finished after 0.37 seconds.
```

Обратите внимание, как выглядит история коммитов после переименования каталогов.

```
$ git log --oneline
* eea637d (HEAD -> main) add folders
* 21e1028 add remaining files
* 3f9bd4a add updated readme

# Verify the big files are removed
$ ls -l
total 7840
-rw-r--r--  1 ppremk  stax   20B Mar 19 22:10 README.md
drwxr-xr-x  3 ppremk  stax   96B Mar 19 22:30 configs
-rw-r--r--  1 ppremk  stax  977K Mar 19 22:10 file4.md
-rw-r--r--  1 ppremk  stax  977K Mar 19 22:10 file5.md
-rw-r--r--  1 ppremk  stax  1.9M Mar 19 22:10 file6.md
drwxr-xr-x  4 ppremk  stax  128B Mar 19 22:30 scripts
```

До сих пор мы показывали операции, которые работают с файлами и каталогами, используя команду `git filter-repo`. Рекомендуем изучить руководство по этой команде, чтобы узнать все возможные вариации того, как можно применять фильтры путей, учитывая их гибкость и мощь. В следующем разделе вы узнаете, как изменять содержимое файла.

Фильтрация на основе содержимого

Допустим, при изучении содержимого каталога `config` вы узнаете, что в `env.config` хранится чувствительная информация, которую нельзя фиксировать. Обработать эту информацию можно так:

```
$ cat configs/env.config
PAT=S0m3T0k3n
```

Для удаления чувствительных данных команда `git filter-repo` при передаче с фильтром `--replace-text` использует список выражений, указанных вами в сопроводительном файле:

```
# Создаем файл выражений, содержащий чувствительные данные, которые нужно
удалить.
$ echo "S0m3T0k3n" >> sensitive-data.txt

$ git filter-repo --replace-text sensitive-data.txt
Parsed 3 commits
New history written in 0.10 seconds; now repacking/cleaning...
Repacking your repo and cleaning out old unneeded objects
HEAD is now at 7a67e7e add folders
Enumerating objects: 13, done.
```

```
Counting objects: 100% (13/13), done.  
Delta compression using up to 8 threads  
Compressing objects: 100% (7/7), done.  
Writing objects: 100% (13/13), done.  
Total 13 (delta 2), reused 8 (delta 1), pack-reused 0  
Completely finished after 0.30 seconds.
```

```
$ git log --oneline  
* 7a67e7e (HEAD -> main) add folders  
* 21e1028 add remaining files  
* 3f9bd4a add updated readme
```

```
$ cat configs/env.config  
PAT=***REMOVED***
```

Поскольку мы явно не указали заменяющий текст, по умолчанию `git-filter-repo` заменяет совпадающие результаты на `REMOVED`. Вы можете использовать строковые литералы, глобальное сопоставление шаблонов или регулярные выражения для поиска и сопоставления слов, которые хотите заменить при создании файла выражений.



Заменить существующий текст новым можно так: `old==>new`. Это выражение заменит слово *old* словом *new*.

Фильтрация сообщений коммита

Мы почти закончили изменение репозитория. Теперь сделаем так, чтобы сообщения коммитов, представляющих новую историю, обновились в соответствии с новой версией.

Изучая обновленную историю репозитория, вы можете заметить, что первый коммит в серии содержит опечатку (`udated` вместо `updated`), а еще не делает то, что нужно: не выбирает последнюю версию файла `README.md`.

Исправим это:

```
$ git log --oneline  
* 7a67e7e (HEAD -> main) add folders  
* 21e1028 add remaining files  
* 3f9bd4a add updated readme
```

Чтобы изменить сообщение коммита при выполнении команды `git filter-repo` с фильтром `--replace-message`, нужно передать файл со списком выражений:

```
# Создаем файл, содержащий новое сообщение коммита.  
# Добавляем старое сообщение коммита, как указано в подсказке выше.
```

```
$ echo "add updated readme==>add latest readme" >> fix-commit-msgs.txt
```

```
$ git filter-repo --replace-message fix-commit-msgs.txt
Parsed 3 commits
New history written in 0.08 seconds; now repacking/cleaning...
Repacking your repo and cleaning out old unneeded objects
HEAD is now at fbf574b add folders
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (13/13), done.
Total 13 (delta 2), reused 10 (delta 2), pack-reused 0
Completely finished after 0.28 seconds.
```

```
$ git log --oneline
* fbf574b (HEAD -> main) add folders
* c13ca61 add remaining files
* 06f04cd add latest readme
```

Имея измененную версию репозитория, вы, наконец, можете передать ее в новый вышестоящий репозиторий или выполнить серию шагов, чтобы убедиться в замене старой версии новой. Как и при любой совместной разработке, важно, чтобы каждый участник работал с последней копией репозитория.

Опции фильтрации для `git-filter-repo` — лишь вершина айсберга. Есть множество случаев применения и продвинутых техник, которые можно использовать для пересмотра и изменения репозитория. Вы можете менять информацию о пользователе и электронную почту, переименовывать имена тегов, заменять родителя объекта коммита и даже частично переписывать историю, используя в качестве цели конкретную ветку или серию коммитов. Если возникнет такая необходимость, то обязательно ознакомьтесь с документацией.



Чтобы больше узнать о внутреннем устройстве `git-filter-repo`, изучите раздел *Internals* документации команды. В нем дается высокоуровневое описание принципа действия `git-filter-repo`:

```
$ git filter-repo --help
```

Итоги

В этой главе мы расширили свои знания о принципах управления коммитами. Мы разобрали очень мощную технику — команду `git add -p`, позволяющую интерактивно индексировать фрагменты несвязанных изменений при составлении коммитов. Затем познакомились с командой `git rev-list`, а также узнали способы извлечения более старых версий файлов. Если вам нужно выбрать один из методов построения собственного инструментария работы с Git, то изучите команды `git-filter-repo` и `git fsck`. При необходимости они станут универсальным инструментом.

Часть V

Советы и приемы

Заключительная часть книги состоит из двух глав, охватывающих практические стороны повседневной работы с репозиториями Git.

Глава 17 объясняет концепции интерактивного перебазирования и предлагает техники миграции проектов, управляемых версиями (как в легаси-системах, так и в существующих репозиториях, готовых к Git), на новую платформу, работающую на Git. Важно понимать эти принципы и техники, поскольку, научившись успешно выполнять миграцию в одной системе контроля версий, вы сможете повторить все эти шаги (а некоторые пропустить) при миграции других проектов. Также поговорим о правильном версионировании крупных или двоичных файлов, чтобы не раздувать репозиторий, что может иметь нежелательные последствия при работе в общей среде.

В главе 18 поговорим о GitHub — популярной платформе хостинга Git, которую используют миллионы разработчиков по всему миру. Разберем возможности, которые позволяют выполнять стандартные операции Git за пределами командной строки. Также рассмотрим клиентские инструменты GitHub. Завершим главу изучением способов расширения платформы под свои потребности.

ГЛАВА 17

Советы, приемы и техники

Имея множество команд и параметров, Git представляет собой богатый ресурс для выполнения разнообразных и существенных изменений в репозитории. Однако иногда фактические шаги для выполнения конкретной задачи оказываются не вполне ясными — назначение конкретной команды и опции может быть запутанным, а техническое описание сложным.

В этой главе мы приведем советы, приемы и техники, которые продемонстрируют возможности Git выполнять интересные трансформации.

Интерактивное перебазирувание при грязном рабочем каталоге

Зачастую при реализации изменений в виде последовательности коммитов в локальной ветке мы понимаем, что в уже сделанный коммит нужно внести корректировки. Чтобы не делать пометку для дальнейших правок, мы можем сразу же отредактировать и внести нужное изменение прямо в новый коммит, указав, что его нужно объединить с предыдущим коммитом.

Но когда мы приступаем к очистке последовательности коммитов и используем `git rebase -i`, то можем обнаружить грязный рабочий каталог. В этом случае Git не будет выполнять перебазирование:

```
$ git show-branch --more=10
[main] Tinker bar
[main^] Squash into 'More foo and bar'
[main~2] Modify bar
[main~3] More foo and bar
[main~4] Initial foo and bar.

$ git rebase -i main~4
error: cannot rebase: You have unstaged changes.
error: Please commit or stash them.
```

Если это происходит, очистите рабочий каталог командой `git stash`.

```
$ git stash
```

```
Saved working directory and index state WIP on main: ed6e906 Tinker bar
```

```
$ git rebase -i main~4
```

```
# Используя редактор, поместите main^ рядом  
# с main~3 и отметьте, что она подлежит совмещению.
```

```
pick 1a4be28 More foo and bar  
squash 6195b3d Squash into 'more foo and bar'  
pick 488b893 Modify bar  
pick ed6e906 Tinker bar
```

```
# Следуйте инструкциям текстового редактора.
```

```
[detached HEAD e3c46b8] More foo and bar with additional stuff.  
Date: Sun Mar 27 16:18:02 2022 +0200  
2 files changed, 0 insertions(+), 0 deletions(-)  
create mode 100644 file2  
create mode 100644 file4  
Successfully rebased and updated refs/heads/main.
```

Конечно, теперь вы захотите восстановить изменения рабочего каталога:

```
$ git stash pop
```

```
# В ветке main
```

```
# Изменения не индексированы для коммита:
```

```
# (используйте "git add <file>..." для обновления того, что нужно зафиксировать)
```

```
# (используйте "git checkout -- <file>..." для отмены изменений в рабочем
```

```
# каталоге)
```

```
# изменено: foo #
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

```
Dropped refs/stash@{0} (71b4655668e49ce88686fc9eda8432430b276470)
```

Сборка мусора

В подразделе «Команда `git fsck`» на с. 403 (глава 16) вы узнали о принципе достижимости. Мы рассказали, как объектное хранилище Git и его граф коммитов могут оставлять висячие объекты, а также привели примеры, как некоторые команды могут оставлять недостижимые объекты в репозитории.

Наличие висячих коммитов или недостижимых объектов не всегда плохо. Вы могли намеренно уйти от конкретного коммита или добавить blob-объект и затем изменить его, прежде чем зафиксировать. Плохо то, что при работе с репозиторием в объектном хранилище может оставаться много висячих объектов.

Исторически в IT-индустрии такие объекты традиционно удалялись во время *сборки мусора* при помощи команды `git gc`. Задача этой команды заключается в периодической сборке мусора и поддержании порядка в объектном хранилище.

В Git сборка мусора имеет еще одну важную задачу: оптимизацию размера репозитория через обнаружение нераспакованных объектов (незакрепленных объектов) и создание для них pack-файлов. Такие файлы мы кратко рассмотрели в разделе «Файлы packfile» на с. 53 (глава 2).

ЭВРИСТИКА РАСК-ФАЙЛОВ

К сожалению, документация не описывает подробностей эвристики упаковки объектов в Git отдельно от реализации исходного кода. Тем не менее некоторый свет на этот процесс проливает небольшая выдержка из технической документации Git (<https://oreil.ly/uRXwS>). Ниже приведена процедура сжатия объектов в Git.

Когда Git создает раск-файл, то сначала находит файлы с очень похожим содержимым и сохраняет все содержимое в одном из них. Затем Git вычисляет отличия, или *дельты*, между похожими файлами и сохраняет только эти отличия. Например, если вы будете менять или добавлять в файл всего одну строку, то Git сохранит полную обновленную версию этого файла, а затем обозначит измененную строку как дельту и также сохранит ее в пакете.

Git реализует упаковку файлов весьма умно. Поскольку Git основан на *содержимом*, системе не важно, если вычисленные дельты файлов будут принадлежать двум версиям одного и того же файла. Git может взять любые два файла из любой части репозитория и вычислить между ними дельту, если решит, что они достаточно схожи для достижения высокой степени сжатия данных. Получается, что в Git используется сложный алгоритм для обнаружения по всему репозиторию и сопоставления потенциальных кандидатов на вычисление дельты. Система также может выстраивать серию дельт, отражающих отличия между поочередными версиями файла.

Git также сохраняет содержимое исходного SHA-1 blob-объекта для каждого полного файла (все содержимое или в виде реконструкции после применения дельт) в упакованном представлении. Это обеспечивает механизму индексирования основу для обнаружения объектов в пакете.

Когда и как часто происходит сборка мусора? Автоматически или ее нужно запускать вручную? Удаляется и упаковывается все что можно?

Как обычно, ответ таков: это зависит от разных факторов.

Git запускает сборку мусора автоматически в стратегические моменты:

- если в репозитории оказывается слишком много непривязанных объектов;
- при передаче в удаленный репозиторий;

- после выполнения команд, которые могут привести к появлению непривязанных объектов;
- когда команды, например `git reflog expire`, явно запрашивают сборку мусора.

Можно попросить Git выполнить сборку мусора с помощью команды `git gc`, что будет актуальным в нескольких ситуациях:

- если вы только что завершили `git filter-repo`. Напомним, что `filter-repo` переписывает множество коммитов, вносит новые и оставляет старые на ссылке, которую следует удалить, когда вы закончите работу. Недоступные объекты, на которые больше нет ссылок, нужно удалить через сборку мусора;
- после команд, которые могут привести к появлению множества непривязанных объектов. Например, при масштабном перебазировании.

Следует избегать сборки мусора в следующих сценариях:

- есть потерянные ссылки, которые вы могли бы решить восстановить;
- вы использовали `git rerere`¹ и не хотите сохранять результаты разрешения конфликтов навсегда;
- для сохранения коммита навсегда достаточно только тегов и веток;
- если вы использовали `git fetch` для извлечения изменений из удаленного репозитория и они были сохранены в `FETCH_HEAD`, то такие объекты могут быть удалены при сборке мусора, поскольку не имеют прямых ссылок на них в вашем репозитории.

Git не осуществляет сборку мусора по своей воле и даже в автоматическом режиме. Определенные команды, которые вы выполняете, заставляют систему рассмотреть возможность сборки мусора и упаковки. Однако когда вы запускаете эти команды и Git выполняет `git gc`, это не означает, что система действует по этому сигналу. Git анализирует все параметры конфигурации, которые определяют, каким образом происходит удаление непривязанных объектов и создание `pack`-файлов.

Некоторые наиболее важные параметры `git config`:

`gc.auto`

Сколько непривязанных объектов может находиться в репозитории, прежде чем сборка мусора приведет к их упаковке. По умолчанию 6700.

`gc.autopacklimit`

Сколько `pack`-файлов может быть в репозитории, прежде чем они будут упакованы в более крупные и эффективные `pack`-файлы. По умолчанию 50.

¹ Это не опечатка. См. раздел «Автоматизация повторений» на с. 430.

gc.pruneexpire

Как долго недостижимые объекты могут висеть в объектном хранилище. По умолчанию две недели.

gc.reflogexpire

Время, по истечении которого команда `git reflog expire` начнет удалять записи `reflog`. По умолчанию 90 дней.

gc.reflogexpireunreachable

Время, по истечении которого команда `git reflog expire` начнет удалять записи `reflog`, но только если они окажутся недостижимы из текущей ветки. По умолчанию 30 дней.

Большинство параметров сборки мусора имеют значение, которое указывает либо «сделать сейчас», либо «никогда не делать».

Советы по восстановлению коммитов

Время — враг потерянных коммитов. В итоге сборка мусора все же случится и очистит любые висячие коммиты и `blob`-объекты. Сборка мусора очистит и ссылки из журнала. В этот момент потерянные коммиты окажутся окончательно утрачены, и команда `git fsck` не сможет их найти. Если вы знаете, что склонны поздно замечать потерю коммита, настройте таймауты для срока действия журнала ссылок и удаления висячих коммитов при сборке мусора:

```
# По умолчанию 90 дней.
```

```
$ git config --global gc.reflogExpire "6 months"
```

```
# По умолчанию 30 дней.
```

```
$ git config --global gc.reflogExpireUnreachable "60 days"
```

```
# По умолчанию 2 недели.
```

```
$ git config --global gc.pruneexpire="1 month"
```

Восстановление после перебазирувания вышестоящего репозитория

В распределенной среде разработки, когда вы работаете с удаленным репозиторием, над которым у вас нет полного контроля, может возникнуть ситуация, когда версия основной ветки, на которой вы разрабатывали свои изменения, изменена другими участниками через передачу без перемотки (`non-fast-forward`) или перебазирувание. Такие изменения приводят к тому, что ваша ветка теряет связь с исходной точкой и не может быть просто слита с удаленным репозиторием.

К сожалению, у мейнтейнера вышестоящего репозитория в Git нет способа определить, как должны рассматриваться эти ветки. Нет флага, который гласил бы: «Эта ветка будет перебазирована по желанию» или «Не ожидайте, что эта ветка будет перемотана». Вам, как нижестоящему разработчику, нужно просто интуитивно понимать поведение ветки или напрямую спрашивать вышестоящего мейнтейнера. Но, как правило, ветки обычно подвергаются простой перемотке, а не перебазируются.

Это может быть плохо: мы уже объясняли, что изменение опубликованной истории ведет к печальным последствиям. Но иногда это происходит. И есть даже очень хорошие модели разработки, которые поощряют периодическое перебазирование ветки при разработке.

Что в этом случае предпринять? Как сделать так, чтобы вашу работу снова можно было отправить в вышестоящий репозиторий?

Для начала выясните, действительно ли перебазированная ветка является той, на которой вы должны основывать свою работу. Часто ветки предназначены только для чтения. Например, собирается коллекция веток и в целях тестирования совмещается в ветку только для чтения. Но в остальном эти ветки доступны по отдельности и выступают основой для разработки. В таком случае вряд ли нужно вести разработку на ветке, представляющей объединенную коллекцию. (Так обычно работают ветки `next` в Linux.)

В зависимости от масштабов перебазирования, произошедшего в вышестоящем репозитории, вы можете восстановить все простой командой `git pull --rebase`. Если это сработает, вам повезло. Однако на такое везение лучше не рассчитывать. Будьте готовы к исправлению этой запутанной ситуации с помощью `reflog`.

Правильный и более надежный подход заключается в методичном переносе вашей последовательности коммитов с устаревшей ветки на новую ветку вышестоящего репозитория. Последовательность действий следующая.

1. Переименуйте старую вышестоящую ветку. Важно сделать это до этапа получения изменений из вышестоящего репозитория, поскольку тогда вы сможете получить новую вышестоящую историю в чистом виде. Используйте `git branch save-origin-main origin/main`.
2. Получите изменения из вышестоящего репозитория, чтобы восстановить его текущее содержимое. Для этого хватит простой команды `git fetch`.
3. Перебазируйте свои коммиты из переименованной ветки в новую вышестоящую ветку, используя команду `cherry-pick` или `rebase`. Это должна быть команда `git rebase --onto origin/main save-origin-main main`.
4. Почистите и удалите временную ветку. Используйте `git branch -D save-origin-main`.

Кажется, просто, но ключ в том, чтобы найти в истории вышестоящей ветки точку, где исходная и новая истории начали расходиться. Может оказаться, что все,

что находится между этой точкой и вашим первым коммитом, не нужно. То есть переписанная история коммитов не меняет ничего, что пересекалось бы с вашей работой. В этом случае вам повезло, поскольку перебазирование должно пройти без проблем. В то же время может быть и так, что переписанная история затрагивает ту же самую область, которую вы разрабатывали. В таком случае вас ждет непростой путь перебазирования: вам нужно будет полностью понять семантику первоначальной и измененной истории, чтобы разрешить желаемые изменения в разработке.

Быстрый обзор изменений

Если вам требуется поддерживать репозиторий в актуальном состоянии, регулярно получая обновления из вышестоящего источника, то часто будет возникать вопрос: какие изменения произошли за последнюю неделю?

Ответ можно получить с помощью команды `git whatchanged`. Как и многие другие команды, она поддерживает различные опции. Часть из них сосредоточена на `git rev-parse` и предназначена для выбора коммитов, а часть служит для их форматирования: например, типичная для `git log` опция `--pretty=`.

Также часто используется опция `--since=`:

```
# Репозиторий исходного кода Git.
$ cd ~/Repos/git
$ git whatchanged --since="three days ago" --oneline
745950c p4000: use -3000 when promising -3000
:100755 100755 d6e505c... 7e00c9d... M t/perf/p4000-diff-algorithms.sh
42e52e3 Update draft release notes to 1.7.10
:100644 100644 ae446e0... a8fd0ac... M Documentation/RelNotes/1.7.10.txt
561ae06 perf: export some important test-lib variables
:100755 100755 f8dd536... cf8e1ef... M t/perf/p0000-perf-lib-sanity.sh
:100644 100644 bcc0131... 5580c22... M t/perf/perf-lib.sh
1cbc324 perf: load test-lib-functions from the correct directory
:100755 100755 2ca4aac... f8dd536... M t/perf/p0000-perf-lib-sanity.sh
:100644 100644 2a5e1f3... bcc0131... M t/perf/perf-lib.sh
```

Вывод перегруженный. Поскольку мы запросили `-oneline`, журнал коммитов был обобщен в одиночные строки:

```
561ae06 perf: export some important test-lib variables
```

И каждую из них сопровождает список файлов, изменявшихся с каждым коммитом:

```
:100755 100755 f8dd536... cf8e1ef... M t/perf/p0000-perf-lib-sanity.sh
:100644 100644 bcc0131... 5580c22... M t/perf/perf-lib.sh
```

Предыдущий код включает биты режимов файлов до и после коммита, SHA-1 каждого blob-объекта до и после коммита, букву статуса (M означает *измененное*

содержимое — `modified content` или *биты режима* — `mode bits`) и, наконец, путь измененного blob-объекта.

Хотя предыдущий пример по умолчанию указывал на ветку `main`, можно выбрать любую или явно запросить набор изменений, которые были только что получены:

```
$ git whatchanged ORIG_HEAD..HEAD
```

Также можно ограничить вывод набором изменений, затронувших названный файл:

```
$ cd /usr/src/linux
$ git pull

$ git whatchanged ORIG_HEAD..HEAD --oneline Makefile
fde7d90 Linux 3.3-rc7
:100644 100644 66d13c9... 56d4817... M Makefile
192cfd5 Linux 3.3-rc6
:100644 100644 b61a963... 66d13c9... M Makefile
```

Основную работу выполняет `git diff-tree`. Использование этой команды требует ее внимательного изучения из-за сложности и разнообразия опций. Поэтому заварите себе кофе, прежде чем приниматься за чтение страницы руководства.

Очистка

Всем нравится чистая и аккуратная структура каталогов. Достичь такого состояния каталогов репозитория поможет команда `git clean`, с помощью которой можно удалять из рабочего дерева неотслеживаемые файлы.

Зачем это делать? Возможно, очистка является частью итеративного процесса сборки, который повторно использует один каталог для повторяющихся операций сборки, но требует, чтобы сгенерированные файлы каждый раз удалялись (подумайте о `make clean`).

По умолчанию `git clean` просто удаляет из текущего и нижестоящих каталогов все файлы, которые не версионированы. Неотслеживаемые каталоги считаются чуть более ценными, чем простые файлы, и остаются нетронутыми, пока вы не дополните команду опцией `-d`.

Более того, в рамках этой команды Git оперирует немного более консервативной формой понятия «версионирование». На страницах руководства используется фраза «файлы, неизвестные Git». На то есть веская причина: фактически Git известны даже те файлы, которые упоминаются в `.gitignore` и `.git/info/exclude`. Это файлы, которые не версионированы, но Git о них *знает*. И поскольку они упоминаются в `.gitignore`, то должны иметь известное (вам) поведение, которое не должно быть изменено Git. Поэтому система не удаляет игнорируемые файлы, если вы явно не запросите это с помощью опции `-x`.

Опция `-x` вызывает обратное поведение: удаляются только файлы, явно игнорируемые Git. Будьте внимательны при выборе важных файлов. Если сомневаетесь, то сначала выполняйте `--dry-run`.

Использование `git-grep` для поиска по репозиторию

В разделе «Использование инструмента `Pickaxe`» на с. 204 (глава 8) мы ввели опцию *pickaxe* (кирка) (указывается как *-Строка*) для команды `git log`, после чего в разделе «`git diff` с ограничением пути» (с. 191, глава 7) показали ее применение вместе с командой `git diff`. Эта опция ищет в истории ветки коммиты, которые вносят или удаляют указанную строку или регулярное выражение.

Для поиска по репозиторию можно использовать другую команду — `git grep`. Команда `git grep` просматривает содержимое файлов репозитория, а не ищет, какие изменения в ветку внес каждый коммит. Поскольку `git grep` — это универсальный инструмент со множеством опций, будет точнее сказать, что команда ищет текстовые шаблоны в отслеживаемых blob-объектах (то есть файлах) рабочего дерева, blob-объектах, кэшированных в индексе, или blob-объектах в указанных деревьях. По умолчанию команда выполняет поиск по отслеживаемым файлам рабочего дерева.

Таким образом, инструмент `pickaxe` можно использовать для поиска по серии коммитов, а `git grep` — для поиска по дереву репозитория в конкретный момент этой истории.

Посмотрим, как работает `git grep` на примере репозитория исходного кода Git¹.

```
$ cd /tmp
$ https://github.com/git/git.git

Cloning into 'git'...
remote: Counting objects: 129630, done.
remote: Compressing objects: 100% (42078/42078), done.
Receiving objects: 100% (129630/129630), 28.51 MiB | 1.20 MiB/s, done.
remote: Total 129630 (delta 95231), reused 119366 (delta 85847)
Resolving deltas: 100% (95231/95231), done.

$ cd git

$ git grep -i loeliger
.mailmap:Jon Loeliger <jdl@jdl.com> <jdl@freescale.com>
.mailmap:Jon Loeliger <jdl@jdl.com> <jdl@freescale.org>
Documentation/gitcore-tutorial.txt:Here is an ASCII art by Jon Loeliger that
illustrates how
Documentation/revisions.txt:Here is an illustration, by Jon Loeliger. Both commit
nodes B
```

¹ Мы опустили устаревшую ссылку на имя и сократили строки вывода.

Documentation/revisions.txt:Here are a handful of examples using the Loeliger illustration above,

```
$ git grep jdl
```

```
.mailmap:Jon Loeliger <jdl@jdl.com> <jdl@freescale.com>
```

```
.mailmap:Jon Loeliger <jdl@jdl.com> <jdl@freescale.org>
```

```
Documentation/technical/pack-heuristics.txt:    <jdl> What is a "thin" pack?
```

Вы можете спросить, где находится документация для команды `git-grep`? В каких файлах `git.git` упоминается `git-grep`? Это можно выяснить так:

```
# По-прежнему находимся в репозитории /tmp/git.
```

```
$ git grep -l git-grep
```

```
.gitignore
```

```
Documentation/RelNotes/1.5.3.6.txt
```

```
Documentation/RelNotes/1.5.3.8.txt
```

```
Documentation/RelNotes/1.6.3.txt
```

```
Documentation/config/grep.txt
```

```
Documentation/git-grep.txt
```

```
Documentation/gitweb.conf.txt
```

```
command-list.txt
```

```
gitweb/gitweb.perl
```

```
grep.c
```

```
t/README
```

```
t/perf/p4220-log-grep-engines.sh
```

```
t/perf/p4221-log-grep-engines-fixed.sh
```

```
t/perf/p7810-grep.sh
```

```
t/perf/p7820-grep-engines.sh
```

```
t/perf/p7821-grep-engines-fixed.sh
```

Обратите внимание на несколько вещей. Во-первых, `git-grep` поддерживает многие из стандартных опций командной строки, относящихся к инструменту `grep`, например `-i` для поиска без учета регистра, `-l` для вывода списка только соответствующих имен файлов и `-w` для сопоставления по словам. С помощью опции разделения `--` можно ограничить пути или каталоги, по которым Git будет искать. Чтобы ограничить поиск каталогом `Documentation/`, сделайте следующее:

```
# По-прежнему находимся в репозитории /tmp/git.
```

```
$ git grep -l git-grep -- Documentation
```

```
Documentation/RelNotes/1.5.3.6.txt
```

```
Documentation/RelNotes/1.5.3.8.txt
```

```
Documentation/RelNotes/1.6.3.txt
```

```
Documentation/config/grep.txt
```

```
Documentation/git-grep.txt
```

```
Documentation/gitweb.conf.txt
```

Используя опцию `--untracked`, можно выполнить поиск шаблонов в неотслеживаемых (но не игнорируемых) файлах, которые не были ни добавлены в кэш, ни

зафиксированы как часть истории репозитория. Эта опция пригодится, если при разработке функциональности вы начали добавлять новые файлы, но еще их не зафиксировали. По умолчанию `git grep` не будет выполнять поиск в этой области, несмотря на то что ваш опыт работы с традиционной `grep` может говорить о другом: что поиск будет выполнен по всем файлам рабочего каталога (и, возможно, его подкаталогах).

Зачем тогда вообще добавлять `git grep`? Разве недостаточно стандартного инструмента оболочки? И да и нет.

Встраивание `git grep` непосредственно в инструментарий Git дает несколько преимуществ. Первое — скорость и простота. Git не нужно полностью переключаться на ветку, чтобы произвести поиск. Она может работать напрямую с объектами из объектного хранилища. Это означает, что вам не нужно писать скрипты для переключения на коммит из далекого прошлого, последующего поиска нужных файлов и итогового возвращения к исходному состоянию. Во-вторых, с помощью интегрированного инструмента Git может предложить расширенные возможности и опции. В частности, поиск можно ограничить отслеживаемыми и неотслеживаемыми файлами, файлами, кэшированными в индексе, игнорируемыми или исключенными файлами, вариантами поиска по снимкам из истории репозитория и характерным для репозитория ограничителям `pathspec`. Чтобы узнать подробности о команде `git grep`, введите `git grep --help`.

Обновление и удаление ссылок

В разделе «Ссылки и символические ссылки» на с. 104 в главе 4 мы ввели понятие ссылки и отметили, что в Git также используется несколько видов символических ссылок. К настоящему времени вы должны быть знакомы с ветками как ссылками, с тем, как они поддерживаются в каталоге `.git`, и с тем, что в нем также поддерживаются символические ссылки. Где-то там в нем есть множество значений SHA-1, которые обновляются, перемещаются, удаляются и на которые ссылаются другие ссылки.

Иногда уместно и даже необходимо изменить либо удалить ссылку напрямую. Если вы знаете, что делаете, то можете управлять этими файлами вручную. Но если сделать это неправильно, можно легко все испортить.

Чтобы гарантировать правильное выполнение основных действий со ссылками, Git предоставляет команду `git update-ref`. Команда понимает все нюансы ссылок, символических ссылок, веток значений SHA-1, логирования изменений, журнала ссылок и т. д. Если вам нужно напрямую изменить значение ссылки, то используйте подобную команду:

```
$ git update-ref someref SHA1
```


где *someref* — это имя ветки или ссылки, обновляемой новым значением, *SHA1*. Если вы хотите удалить ссылку, то правильно это сделать так:

```
$ git update-ref -d someref
```

Конечно, обычные операции с ветками могут быть более уместны. Однако при изменении ссылки напрямую использование `git update-ref` гарантирует, что обновление соответствующих записей в системном журнале изменений и другие аспекты, необходимые для нормальной работы Git, будут происходить корректно.

Отслеживание перемещаемых файлов

Если на протяжении своей истории файл перемещается из одного места структуры каталогов репозитория в другое, то Git будет отслеживать его историю только на основе текущего имени.

Чтобы увидеть полную историю файла, включая его перемещения, используйте команду `--follow`. Например, команда ниже выведет журнал коммитов для файла, который сейчас имеет имя *file*, но также включит журнал, описывающий историю его предыдущих имен:

```
$ git log --follow file
```

Добавьте опцию `--name-only`, чтобы Git также указывал имя файла при его изменении:

```
$ git log --follow --name-only file
```

В примере ниже файл *a* сначала был добавлен в каталог *foo*, а затем перемещен в каталог *bar*:

```
$ git init -b main
Initialized empty Git repository in /tmp/tips-tricks/moved-file/.git/

$ mkdir foo
$ touch foo/a
$ git add foo/a
$ git commit -m "First a in foo"
[main (root-commit) 1b4fbff] First a in foo
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 foo/a

$ mkdir bar
$ git mv foo/a bar/a
$ git commit -m "Move foo/a to bar/a"
[main 00229c3] Move foo/a to bar/a
1 file changed, 0 insertions(+), 0 deletions(-)
rename {foo => bar}/a (100%)
```

В этот момент выполнение команды `git log bar/a` покажет только коммит, создавший файл `bar/a`, но с опцией `--follow` эта команда проследит историю файла, включая изменения его имени:

```
$ git log --oneline bar/a
00229c3 (HEAD -> main) Move foo/a to bar/a
```

```
$ git log --oneline --follow bar/a
00229c3 (HEAD -> main) Move foo/a to bar/a
1b4fbff First a in foo
```

Если вы хотите использовать исходное имя, придется потрудиться, поскольку стандартным образом можно сослаться только на текущее имя файла, то есть `bar/a`. Сработает добавление опции `--`, сопровождаемой любым из его имен, будь то текущее или одно из предыдущих. При этом добавление `--all` приведет к выполнению обширного поиска, как если бы он включал и все ссылки тоже:

```
$ git log --oneline foo/a
fatal: ambiguous argument 'foo/a': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
```

```
$ git log --oneline -- foo/a
00229c3 (HEAD -> main) Move foo/a to bar/a
1b4fbff First a in foo
```

Автоматизация повторений

Вы сталкивались с ситуацией, когда приходилось многократно выполнять сложное слияние или перебазирование? Это утомляет. Но есть ли способ автоматизировать этот процесс?

О этом позаботились разработчики Git.

В Git есть функциональность под названием `rerere`, которая автоматизирует сложную повторяющуюся работу по разрешению конфликтов при слиянии или перебазировании. Название этой функциональности — это сокращение от *reuse recorded resolution* (повторное использование записанного процесса разрешения). Иногда возникают долгие циклы разработки, когда ветка используется для сохранения работы, подвергающейся серии итераций перед итоговым слиянием с основной линией. Эту ветку приходится многократно перебазировать или перемещать с разрешением одного и того же набора конфликтов.

Чтобы активировать и использовать команду `git rerere`, установите логическую опцию `rerere.enabled` на `true`:

```
$ git config --global rerere.enabled true
```

После включения функциональность записывает правую и левую стороны конфликта слияния в каталоге `.git/rr-cache` и в случае его разрешения также записывает ручной вариант разрешения конфликта. Если тот же конфликт встречается снова, срабатывает функция его автоматического разрешения.

Когда опция `rerere` включена и участвует в слиянии, то предотвращает автоматический коммит этого слияния. Выполняющему слияние разработчику нужно проанализировать автоматическое разрешение конфликта, прежде чем делать его частью истории коммитов.

У `rerere` есть один явный недостаток: непереносимость каталога `.rr-cache`. Запись конфликтов и их разрешение происходит на основе каждого клонирования и не передается при операциях `push/pull`.

Миграция в Git

При переносе существующего проекта в систему версионирования Git рекомендуем спланировать весь импорт, преобразование и очистку до публикации первой Git-версии вашего репозитория. Есть несколько этапов для хорошо спланированного переноса, которые вы должны выполнить до того, как у кого-либо появится возможность клонировать первую версию вашего репозитория Git. Например, провести любые глобальные изменения — переименование каталогов, удаление информации об авторе и электронной почте, удаление больших файлов, манипуляции с ветками и построение тегов — будет намного труднее как для вас, так и для нижестоящих потребителей, если эти изменения произойдут после клонирования преобразованного репозитория.

При переносе проекта в Git возможны два сценария. Первый — когда вы переходите с одной платформы Git на другую. Второй — переход в Git из другой системы контроля версий. Первый сценарий довольно прост, особенно когда вы работаете с готовой к работе в Git кодовой базой, которая переносится между популярными платформами хостинга Git. А во втором случае для переноса потребуется предварительная подготовка (преобразование базы кода, находящейся не в формате Git, в репозиторий Git).

Перенос из системы Git

Допустим, вы разместили репозитории проекта на локальном сервере Git и хотите переместить их на одну из популярных платформ хостинга Git. Перенос может заключаться в простом добавлении пути удаленного источника (`origin`) и удалении его старого значения. Это позволит объединить все последние изменения каждого контрибьютора репозитория перед переносом. В техническом плане нижестоящие участники могут обновить свои копии репозитория, указав

в них новый путь удаленного источника, но с точки зрения администрирования рекомендуется координировать перенос централизованно.

В примере ниже мы перенесем один из наших локальных репозиториев на GitHub. Приведенный код показывает, как можно перенести на эту платформу локальный репозиторий:

```
# Путь локального репозитория.
$ pwd
/tmp/active-projects/my-super-project

# Получение пути удаленного сервера.
$ git remote -v
origin      /tmp/bare-repositories/my-super-project (fetch)
origin      /tmp/bare-repositories/my-super-project (push)
```



Если у вас нет локального репозитория, вы можете смоделировать процесс переноса, создав и инициализировав новый репозиторий на локальной машине. Разница в том, что у вас не будет значения удаленного источника.

Создайте пустой временный репозиторий-заполнитель в учетной записи GitHub (<https://github.com>). Щелкните по кнопке **New button** на странице **Dashboard** (рис. 17.1).

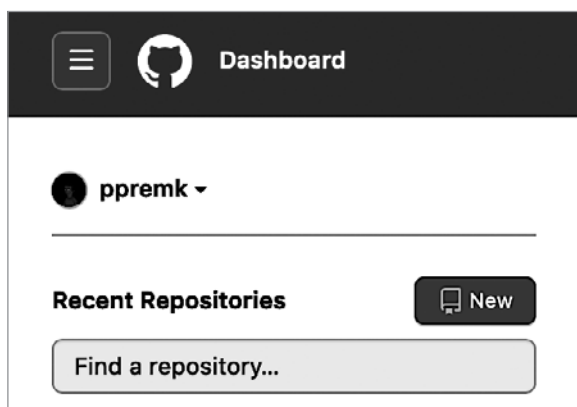


Рис. 17.1. Кнопка New на странице Dashboard

На странице **Create a new repository** укажите название вашего репозитория (рис. 17.2). На этом этапе инициализировать репозиторий *не нужно*.

На следующей странице выберите инструкцию: **...or push an existing repository from the command line** («...или передать существующий репозиторий через командную строку»), рис. 17.3.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? Import a repository.

Repository template

Start your repository with a template repository's contents.

No template ▾

Owner *

ppremk ▾

Repository name *

my-super-project ✓

Great repository name! my-super-project is available. Need inspiration? How about super-duper-broccoli?

Description (optional)

☒ Public

Anyone on the internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☐ Add a README file

This is where you can write a long description for your project. [Learn more.](#)

Add .gitignore

Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: None ▾

Choose a license

A license tells others what they can and can't do with your code. [Learn more.](#)

License: None ▾

Grant your Marketplace apps access to this repository

You are subscribed to 2 Marketplace apps

☐ Azure Pipelines

Continuously build, test, and deploy to any platform and cloud

☒ Azure Boards

Connects Azure Boards with GitHub to plan, track, and discuss work across your teams

 You are creating a public repository in your personal account.

Create repository

Рис. 17.2. Создание репозитория-заполнителя

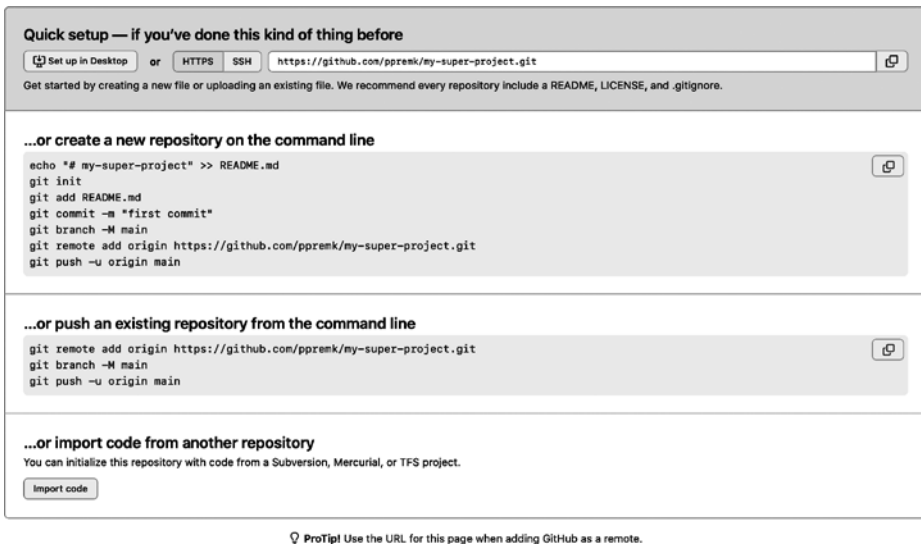


Рис. 17.3. Обновление нового удаленного источника и передача репозитория

Поскольку мы работаем с существующим репозиторием, нужно либо удалить старое значение удаленного источника, либо указать команду `git remote set-url` для обновления пути URL. Мы удалили старый удаленный источник, поскольку хотим опубликовать репозиторий в новом:

```
$ git remote remove origin

# Возвращает пустой результат.
$ git remote -v

# Добавляем url нового удаленного источника.
$ git remote add origin https://github.com/ppremk/my-super-project.git

# Передаем локальный репозиторий в новый вышестоящий репозиторий
# Можно пропустить 'git branch -M main',
# поскольку наша предустановленная ветка уже именуется main.
$ git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (8/8), 917 bytes | 917.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/ppremk/my-super-project.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

Теперь ваш локальный репозиторий доступен в новом удаленном источнике, размещенном на GitHub. Это позволяет использовать возможности платформы,

которые помогут вести совместную разработку с вашей командой и поддерживать проект. В главе 18 подробно поговорим о GitHub.

Вы можете выполнить те же простые шаги при переносе проекта с одной платформы хостинга Git на другую. Если вы просто переносите кодовую базу, версиями которой управляет Git, то вам нужно удалить старый удаленный источник и добавить новый, поскольку история репозитория останется нетронутой. (Переход на использование метаданных или возможностей новой платформы — это отдельная тема, которая выходит за рамки книги.)

Перенос из другой системы контроля версий

Чтобы перенести кодовую базу из системы, отличной от Git, потребуется сделать дополнительные шаги, прежде чем публиковать репозиторий для использования другими. Объем работы в этом случае будет зависеть от исходной системы, но все типичные задачи можно разделить на четыре шага.

Первый шаг — создание файла сопоставления данных автора (может потребоваться в зависимости от используемого инструмента переноса, подробности ниже). Это обеспечит корректное сопоставление текущей авторской информации в нужный формат Git при успешном преобразовании проекта в Git-репозиторий. Например, инструмент `git svn` может получать для этого файл авторских данных, содержащий список имен пользователей SVN¹, подлежащих отображению в формат Git. Помните, что создание файла сопоставлений может потребовать написания скриптов для извлечения нужных деталей об авторе коммита из исходного проекта.

Второй шаг — использование инструмента для преобразования исходного проекта в подготовленный для Git репозиторий. Есть несколько инструментов, предназначенных для конкретных систем контроля версий исходного проекта. Среди них можно выделить `git svn` (<https://oreil.ly/N8cr5>) и `git-p4` (<https://oreil.ly/pvoM1>). Для преобразования вам нужно знать размер проекта, количество его веток, способ сохранения двоичных файлов, за какой период времени нужно сохранить историю версий и есть ли у используемого инструмента переноса какие-либо ограничения. Только тогда можно планировать этапы преобразования и корректно уведомить об этом переходе своих коллег.



В зависимости от используемого инструмента первые два шага можно объединить, предоставив файл сопоставления данных автора в качестве опции при выполнении преобразования с помощью поддерживаемых команд.

Третий шаг — провести предварительную подготовку репозитория перед его публикацией для коллег. Это может быть операция очистки, включающая удаление ненужных файлов или каталогов, выбор способа управления крупными объектами

¹ SVN — свободная централизованная система управления версиями ПО. — *Примеч. ред.*

или двоичными файлами, а также добавление релевантных файлов, таких как `.gitignore`, и любые другие задачи для подготовки переносимого репозитория.

Четвертый этап — публикация репозитория через создание URL нового удаленного источника, который смогут использовать участники проекта. Вам нужно создать временный репозиторий и передать в него тот, который переносится на платформу Git.

В примере ниже мы преобразуем репозиторий SVN в репозиторий Git. В качестве источника используется <https://svnbook.red-bean.com>.

Извлечем информацию об авторе и создадим файл сопоставления:

```
# Переключение на репозиторий svn.
$ svn checkout https://svn.code.sf.net/p/svnbook/source/trunk/
A   trunk/en
A   trunk/en/book
A   trunk/en/book/sample-repositories
...
...

# Для получения информации об авторе выполните в каталоге "trunk":
$ svn log --xml --quiet | grep author | sort -u | perl -pe 's/.*>(.*)<.*$1 = /'
> authors.txt

# Отредактируйте содержимое authors.txt, чтобы оно соответствовало
# следующему формату, указав корректные Author Name и Email.

$ cat authors.txt
FLamY = FLamY <FLamY@email.com>
Imaged = Imaged <Imaged@email.com>
```



Ссылка на скрипт для генерации информации об авторе в SVN взята из онлайн-руководства git-scm (<https://oreil.ly/NPkh>). Ссылка также содержит примеры и советы по переходу из систем контроля версий, отличных от Git.

Если вы не заполнили файл `authors.txt`, то можете сэкономить время, скачав для него готовое содержимое (<https://oreil.ly/1nDT5>).

Мы используем инструмент переноса `git svn`, поэтому файл сопоставления данных автора можно предоставить в качестве опции при клонировании источника. У `svnbook` есть 6055 ревизий — его клонирование может занять немало времени. Поэтому мы ограничим число ревизий с помощью опции `-r`. Обратите внимание, если в среде разработки не установлен инструмент `git svn`, потребуется добавить его через поддерживаемый пакетный менеджер:

```
# Ограничение числа копируемых ревизий.
$ git svn clone -r1:100 https://svn.code.sf.net/p/svnbook/source/trunk/
--authors-file=authors.txt
...
...
```



```
$ cd trunk
$ pwd
/tmp/migrations/trunk

$ git log -2
commit c0e1e3844c8e54d1d5887b0c0adc9b761de7fd8e (HEAD -> master, git-svn)
Author: fitz <fitz@email.com>
Date:   Fri Nov 29 15:27:48 2002 +0000

    * ch03.xml:  port broken-book changes to trunk.

git-svn-id: https://svn.code.sf.net/p/svnbook/source/trunk@100
b70f5e92-ccc6-4167-9ab2-d027528d294b

commit 82f925a9b5a165c4abbcbce4381b4e6fb50950f3a
Author: cmpilato <cmpilato@email.com>
Date:   Wed Nov 27 18:31:51 2002 +0000

    * doc/book/book/ch05.xml
      Little spelling and grammar fixes and such. Thanks to my beautiful
      wife Amy for trudging through a full reading of this chapter to
      help me find these things. Sweetie, you wanna be a repository
      administrator now?

git-svn-id: https://svn.code.sf.net/p/svnbook/source/trunk@99
b70f5e92-ccc6-4167-9ab2-d027528d294b
```



При клонировании репозитория команда `git svn` также импортирует метаданные платформы Subversion. Это помогает сохранить исходный `git-svn-id`, который внес эту ревизию. Если вам не нужно ссылаться на эти метаданные, укажите опцию `--no-metadata`, чтобы пропустить импорт.

Выполним все необходимые операции и запишем произошедшие изменения в виде новых коммитов в конвертированном Git-репозитории проекта.

Добавим в корень каталога файл `README.md`, чтобы дать репозиторию контекст:

```
# Выполните в /tmp/migrations/trunk.
$ echo "This repo was converted from SVN" >> README.md

# Добавьте и зафиксируйте изменения в репозитории.
$ git status
[main cee31bf] Add README.md in project root
1 file changed, 1 insertion(+)
create mode 100644 README.md
```

Теперь добавим для преобразованного репозитория новый удаленный источник и передадим его в вышестоящий репозиторий, чтобы ваши коллеги могли работать с ним в уже новом версионизируемом формате. Делается это с помощью шагов, описанных в разделе «Перенос из системы Git» на с. 431.



Поскольку `git svn` создает название ветки, используя предустановленный вариант Git, вы можете переименовать ее из `master` в `main` с помощью команды `git branch -M master main`.

Теперь передадим преобразованный репозиторий в репозиторий-заполнитель с тем же названием:

```
# Создаем на GitHub репозиторий-заполнитель
# Выполните в /tmp/migrations/trunk.
# Добавьте новый удаленный исходный репозиторий.
$ git remote add origin https://github.com/ppremk/svnbook.git

$ git push -u origin main
Enumerating objects: 561, done.
Counting objects: 100% (561/561), done.
Delta compression using up to 8 threads
Compressing objects: 100% (510/510), done.
Writing objects: 100% (561/561), 871.80 KiB | 7.45 MiB/s, done.
Total 561 (delta 255), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (255/255), done.
To https://github.com/ppremk/svnbook.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Теперь перейдем на платформу Git и посмотрим на перенесенный репозиторий (рис. 17.4).

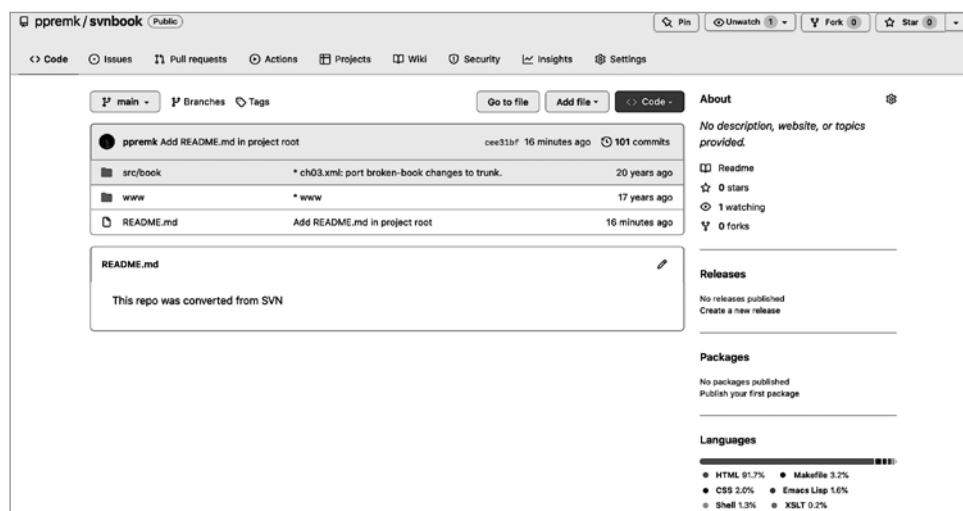


Рис. 17.4. Перенесенный репозиторий svnbook на GitHub

Если у вас нет сложных требований к переносу и вам не нужна старая история коммитов, поскольку она будет заархивирована, то можете выполнить только чистый перенос.

Чистый перенос (clean cut migration) — это процесс преобразования последней версии проекта в новый репозиторий Git. Он включает в себя инициализацию нового репозитория на основе исходного проекта, добавление всех файлов и от-правку их в виде начального коммита, а затем передачу этого репозитория на новый удаленный сервер. Обязательно убедитесь, что провели в исходной копии все необходимые внутренние процедуры, прежде чем делать ее доступной для ваших коллег:

Существующий исходный проект после внутренней подготовки.

```
$ pwd
/tmp/migrations/svn-repo/trunk

$ git init -b main
Initialized empty Git repository in /tmp/migrations/svn-repo/trunk/.git/

$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .svn/
    en/
    tools/

nothing added to commit but untracked files present (use "git add" to track)

$ git add .
warning: CRLF will be replaced by LF in .svn/pristine/6d/6dab3fba2...
The file will have its original line endings in your working directory
warning: CRLF will be replaced by LF in en/book/sample-repositories
/ch04-sample-repos.dump.
The file will have its original line endings in your working directory

$ git commit -m "Initial commit"
[main (root-commit) 7c09122] Initial commit
182 files changed, 1573706 insertions(+)
create mode 100644 .svn/entries
create mode 100644 .svn/format
...
...

# Добавление нового удаленного сервера и перемещение в вышестоящий репозиторий
# для завершения миграции.
```

Примечание по работе с крупными репозиториями

Крупный репозиторий можно определить как репозиторий с историей коммитов, охватывающей годы разработки, или как репозиторий, который имеет огромный размер из-за того, что хранит большие двоичные и другие файлы. Он может быть комбинацией и того и другого, что часто является результатом переноса легаси-проекта. Такой репозиторий может быть либо монолитом (<https://oreil.ly/sKxsv>), либо монорепозиторием (<https://oreil.ly/izQUT>) в зависимости от типа разрабатываемого проекта.

Описанные техники работы с такими репозиториями касаются управления различными объектами Git, которые потребуются в локальной разработке. Каждая из этих техник имеет собственные плюсы и минусы, начиная с производительности и заканчивая поведением некоторых команд Git при выводе. Поэтому следует обдумать, стоят ли эти компромиссы затраченных усилий, когда вы будете использовать описанные техники при работе с крупными репозиториями.

Краткое описание техник

Частичное клонирование (partial clone)

Оптимизирует производительность клонированного репозитория. Позволяет клонировать репозиторий, не требуя передачи копии всего объектного хранилища. Объекты Git, которых будет не хватать в клоне, можно получить позднее при необходимости.

Чтобы запустить частичное клонирование, сопроводите команду `clone` опцией `--filter=<filter-spec>`.

К примеру, команда `git clone --filter=blob:none repo_url` клонирует только объекты дерева и достижимые коммиты. Будут получены лишь blob-объекты, достижимые из верхнего коммита.

Команда `git clone --filter=tree:0 repo_url` клонирует только достижимые коммиты. Получены будут только объекты дерева и blob-объекты, достижимые из верхнего коммита.

Поверхностное клонирование (shallow clone)

Позволяет клонировать репозиторий, сократив историю коммитов до определенного количества. Как правило, это последний коммит или последняя ревизия репозитория.

Для поверхностного клонирования сопроводите команду `clone` опцией `--depth <depth>`.

Например, при указании команды `git clone --depth=1 repo_url` клон получит только верхний коммит вместе с достижимыми от него объектами дерева и blob-объектами. Поверхностное клонирование можно выполнить на основе даты. Команда `git clone --shallow-since=[date] repo_url` клонирует репозиторий, урезав историю коммитов после указанной даты.

Выборочное получение (sparse-checkout)

Команда `sparse-checkout` появилась в версии Git 2.25.0. Она упрощает процесс инициализации и настройки работы с ограниченным набором каталогов репозитория. Команда также предназначена для значительного повышения производительности при работе с крупным репозиторием.

Используя `sparse-checkout` для имеющегося репозитория, вам надо инициализировать нужную опцию конфигурации Git, а затем указать, с какими каталогами вы планируете работать в локальном репозитории:

```
# Инициализируем конфигурации Git для sparse-checkout.
$ git sparse-checkout init --cone

# Указываем, с какими каталогами будем работать с sparse-checkout.
$ git sparse-checkout set <dir1> <di2> <dir3>
```

Внимательно изучите размещенные на GitHub статьи *Get Up to Speed with Partial Clone and Shallow Clone* (<https://oreil.ly/sI1B9>) и *Bring Your Monorepo Down to Size with Sparse-Checkout* (<https://oreil.ly/EHvxv>), чтобы узнать, как использовать `sparse-checkout` при работе с монорепозиторием. Статьи написал Деррик Стоули (Derrick Stolee), контрибьютор Git с 2017 года, работающий над повышением производительности платформы. Он также работал над повышением быстродействия команд `git log --graph` и `git push` для больших репозиториях.

В следующем разделе познакомимся с еще одной техникой, ориентированной на контроль версий больших файлов или объектов вашего репозитория Git.

Git LFS

Большое файловое хранилище Git, или Git LFS (Large File Storage), — это open-source-расширение платформы, созданное для отслеживания крупных файлов или объектов в репозиториях. Обычно Git сам справляется с версионированием больших файлов, но когда размер репозитория начинает быстро расти, это становится проблемой и приводит к падению быстродействия стандартных операций.

Если вы в основном работаете с крупными текстовыми файлами, то это нестрашно, поскольку используемая в Git библиотека сжатия (<https://www.zlib.net>) эффективно сжимает такие файлы, оптимизируя их для передачи и хранения в репозитории Git. Для файлов, которые сжимаются плохо и требуют контроля версий, например видео, игровой графики и датасетов, Git LFS может стать полезным инструментом.

Использовать Git LFS несложно. Все потому, что это расширение и основная часть низкоуровневой работы скрыта от вас уже на начальном этапе. После установки и базовой настройки вы можете продолжить отслеживать объемные файлы аналогично любым другим, используя стандартные операции и команды. Основные платформы хостинга Git предоставляют поддержку для работы с Git LFS на их серверах, позволяя использовать расширение без каких-либо проблем.

Репозиторий до и после Git LFS

Рассмотрим, как репозиторий увеличивается при версионировании крупного файла.

Для примера используем видео, так как подобные файлы сжимаются плохо. Созданный репозиторий будет содержать текстовый файл размером 1 Мбайт и видеофайл 100 Мбайт. На рис. 17.5 показано, как можно отредактировать исходный видеофайл и отслеживать изменения в виде новых коммитов для каждого вносимого изменения. Это могут быть, скажем, изменения настройки насыщенности или добавление анимации.

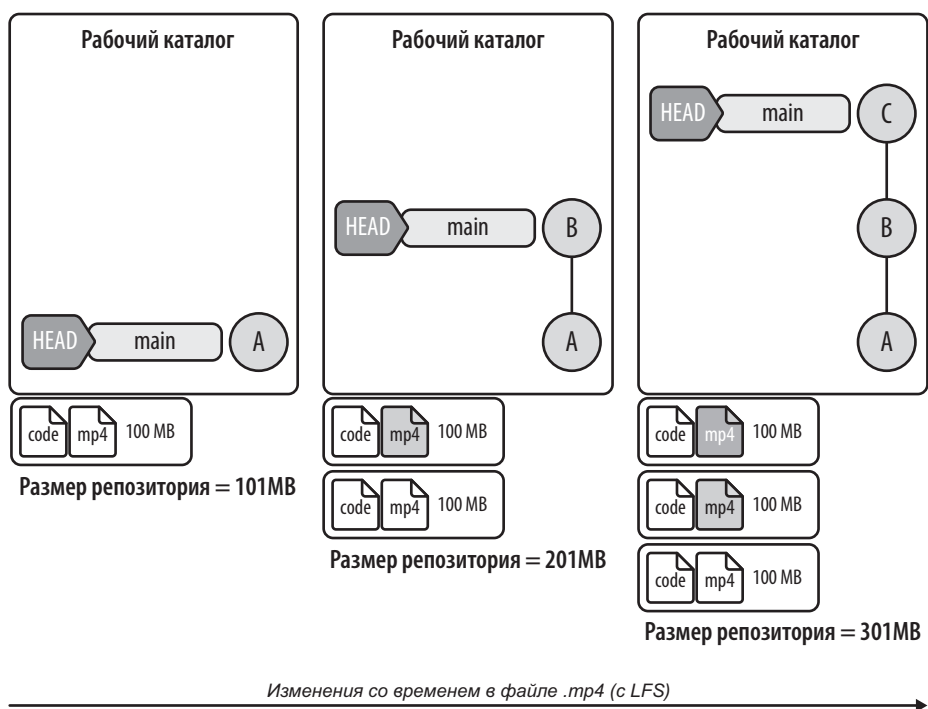


Рис. 17.5. Изменение размера репозитория без использования Git LFS

Обратите внимание, что размер репозитория увеличивается с каждым коммитом. И именно такой размер будет у репозитория при его клонировании с удаленного сервера другим разработчиком.

Размер нашего репозитория составляет 300 Мбайт, но представьте, что проект отслеживает файлы видеоролики или датасеты, которые могут увеличить размер репозитория до нескольких гигабайтов. Это значительно повлияет на пропускную способность удаленного сервера, когда несколько разработчиков клонируют, передают и получают изменения, снижая быстродействие сети.

Теперь оценим тот же репозиторий, но используя Git LFS (рис. 17.6).

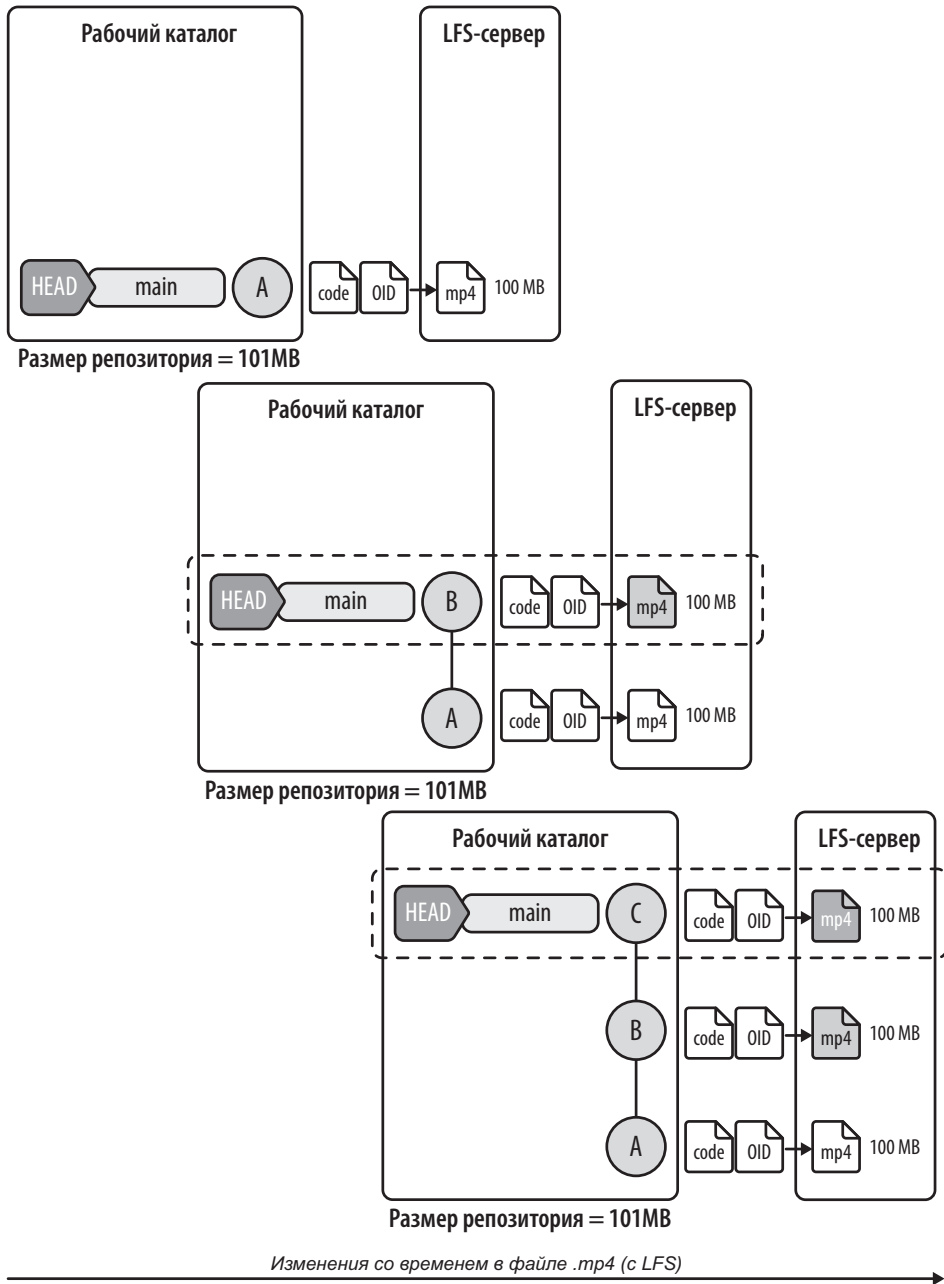


Рис. 17.6. Изменение размера репозитория во времени при использовании Git LFS

Главное различие заключается в том, что при использовании Git LFS большие файлы хранятся не в самом объектном-хранилище, а на сервере Git LFS. Вместо того чтобы хранить большой отслеживаемый файл напрямую, система хранит файл-указатель, что позволяет сохранить размер репозитория небольшим даже при наличии объемных файлов. Просто и эффективно!



На рис. 17.6 пунктирной линией выделены состояние репозитория и его размер сразу после клонирования и переключения на него.

Обратите внимание, что на рис. 17.6 размер репозитория остается небольшим, поскольку вы переключаетесь на текущую версию конкретного коммита. Это также означает, что при клонировании репозитория и переключении на предустановленную ветку вы, по сути, скачиваете репозиторий размером 101 Мбайт, представляющий версию с последними изменениями в видеофайле. Разработчики, которые хотят поработать с более старой версией этого файла, всегда могут получить ее при переключении на известный коммит или ветку. Это позволяет Git получать видеофайл с LFS-сервера, если его еще нет на локальной машине разработчика.

Установка Git LFS

Чтобы установить расширение командной строки Git LFS, воспользуйтесь популярными пакетными менеджерами или сделайте это напрямую с GitHub (<https://oreil.ly/OUpOT>):

```
# Системы на базе Linux.  
$ sudo apt-get install git-lfs=3.1.4  
  
# MacOS.  
$ brew install git-lfs
```

Установка Git LFS добавит в файлы конфигурации два фильтра. Эти фильтры (вместе с настроенным файлом `.gitattributes`) отвечают за перехват стандартных операций Git и за то, чтобы крупные отслеживаемые файлы не сохранялись напрямую в объектном хранилище репозитория. Они будут помещаться в локальный каталог по пути `.git/lfs/objects/`:

```
$ cat ~/.gitconfig  
[filter "lfs"]  
    smudge = git-lfs smudge -- %f  
    process = git-lfs filter-process  
    required = true  
    clean = git-lfs clean -- %f
```

Когда вы добавляете отслеживаемый с помощью LFS файл, фильтр `clean` перехватывает команду `git add`. Он генерирует хеш SHA-256 на основе содержимого

этого файла, сохраняет его в локальном каталоге `.git/lfs/objects/`, генерирует файл указателей, содержащий OID, и сохраняет его в объектном хранилище репозитория.

Файл-указатель — это текстовый файл с кодировкой UTF-8. Он соответствует определенному формату и содержит три поля: версию, обозначающую спецификацию файла-указателя; OID, представляющий ID объекта отслеживаемого файла; и размер файла в байтах.

Выполнение следующей команды приведет к генерации файла-указателя для заданного файла в вашем локальном каталоге:

```
$ git lfs pointer --file=path/to/file
```

```
Git LFS pointer for file
```

```
version https://git-lfs.github.com/spec/v1
```

```
oid sha256:91104678a2b7598a4df9fd42e16baac8e8f695208e5990d47449b9c458ce7a47
```

```
size 35000000
```

Для передачи файла, отслеживаемого с помощью LFS, вам по-прежнему нужно использовать команду `git push`. Хук `pre-push` обеспечит передачу LFS-файла на удаленный сервер LFS.

Вместе с тем, когда вы перемещаете файлы, отслеживаемые в LFS, из своего репозитория в рабочий каталог, срабатывает фильтр `smudge`. При выполнении фильтр перехватывает процесс, записывающий содержимое репозитория в ваш рабочий каталог. В случае Git LFS `smudge` перехватит запись файла-указателя, обработает данные файла в памяти и считает Git LFS OID, используя его для просмотра пути `.git/lfs/objects/{OID}`. Если файл есть, он прочитает объект LFS. В противном случае фильтр запишет содержимое LFS в рабочий каталог.



Подробную спецификацию расширения Git LFS см. по ссылке <https://oreil.ly/xJkTe>.

Отслеживание с помощью Git LFS крупных объектов

После установки Git LFS вы можете использовать его в своих репозиториях.

Для начала потребуется настроить Git LFS:

```
# Инициализация репозитория.
```

```
$ pwd
```

```
/tmp/git-lfs
```

```
# Инспектирование каталога .git перед настройкой git lfs.
```

```
$ tree .git
```

```
.git
```

```
├── HEAD
```

```
└── config
```

```
├── description
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── prepare-commit-msg.sample
│   ├── push-to-checkout.sample
│   └── update.sample
├── info
│   └── exclude
├── objects
│   ├── info
│   └── pack
└── refs
    ├── heads
    └── tags
```

8 directories, 17 files

Настройка git lfs для этого репозитория.

```
$ git lfs install
```

Updated git hooks.

Git LFS initialized.

Инспектирование каталога .git после настройки git lfs.

```
$ tree .git
```

```
.git
├── HEAD
├── config
├── description
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-checkout
│   ├── post-commit
│   ├── post-merge
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── pre-push
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── prepare-commit-msg.sample
│   └── push-to-checkout.sample
```

```

├── update.sample
├── info
│   └── exclude
├── lfs
│   └── tmp
├── objects
│   ├── info
│   └── pack
├── refs
│   ├── heads
│   └── tags

```

10 directories, 21 files

Git LFS устанавливает дополнительный каталог, `.git/lfs/tmp`, вместе со следующими хуками коммитов: `post-checkout`, `post-commit`, `post-merge` и `pre-push`. Это напрямую связано с фильтрами `clean` и `smudge`, которые Git LFS использует при перехвате стандартных команд `git add`, `git push` и `git checkout`.

Теперь можно добавить крупные файлы и отслеживать их в виде файла Git LFS:

```
# Создание крупных файлов.
```

```
$ truncate -s 100M video1.mp4
```

```
$ truncate -s 100M video2.mp4
```

```
$ truncate -s 300M texture1.png
```

```
$ truncate -s 350M texture2.png
```

```
$ ls -al
```

```
total 0
```

```

drwxr-xr-x  7 ppremk stax      224 Apr 24 16:36 .
drwxr-xr-x  6 ppremk stax      192 Apr 24 16:15 ..
drwxr-xr-x 10 ppremk stax      320 Apr 24 16:36 .git
-rw-r--r--  1 ppremk stax 314572800 Apr 24 16:36 texture1.png
-rw-r--r--  1 ppremk stax 367001600 Apr 24 16:36 texture2.png
-rw-r--r--  1 ppremk stax 104857600 Apr 24 16:35 video1.mp4
-rw-r--r--  1 ppremk stax 104857600 Apr 24 16:35 video2.mp4

```

```
$ git status
```

```
On branch main
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
texture1.png
```

```
texture2.png
```

```
video1.mp4
```

```
video2.mp4
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

Поскольку в примере есть и видеофайлы, и изображения, для их отслеживания потребуются указать типы файлов, которыми будет оперировать Git LFS. Выполнив приведенную ниже команду, мы создадим файл `.gitattributes`, который

тоже нужно будет отслеживать. Обратите внимание, что вы можете в любой момент настроить отслеживание новых типов файлов, выполнив ту же команду с другими параметрами, указывающими на тип файла, который собираетесь добавить:

```
$ git lfs track "*.mp4" "*.png"
Tracking "*.mp4"
Tracking "*.png"

$ cat .gitattributes
*.mp4 filter=lfs diff=lfs merge=lfs -text
*.png filter=lfs diff=lfs merge=lfs -text

# Важно отслеживать и фиксировать файл .gitattributes.
$ git add .gitattributes
...

# Добавляем оставшиеся файлы привычным образом.
$ git add video1.mp4 video2.mp4 texture1.png texture2.png
$ git commit -m "Add video and texture files"

$ git log --oneline
0c88d26 (HEAD -> main) Add video and texture files
f134596 Add lfs tracking file

# Вывод информации о файлах Git LFS в индексе и рабочем дереве.
$ git lfs ls-files
17a88af837 * texture1.png
37e130c167 * texture2.png
20492a4d0d * video1.mp4
20492a4d0d * video2.mp4

$ tree .git/lfs
.git/lfs
├── objects
│   ├── 17
│   │   └── a8
│   │       └── 17a88af83717f68b8bd97873ffc022c8aed703416fe9b08e0fa9e3287692bf0
│   ├── 20
│   │   └── 49
│   │       └── 20492a4d0d84f8beb1767f6616229f85d44c2827b64bdbfb260ee12fa1109e0e
│   ├── 37
│   │   └── e1
│   │       └── 37e130c1679c21acfe85c2c2f30aba984c910d389c8df9340b5bc4455f809220
│   └── tmp
└── 8 directories, 3 files
```

Описанные шаги работы с Git LFS очень просты. Большую часть работы расширение выполняет за вас. Остается только установить Git LFS на машину, настроить репозитории, которые будут отслеживать крупные файлы, и продолжить мониторить эти файлы привычным образом.

Полезные техники в работе с Git LFS

Ниже перечислены рекомендуемые техники для отслеживания файлов через Git LFS.

Корректное отслеживание типов файлов

При передаче глобального шаблона для сопоставления типов файлов, как в этой команде:

```
git lfs track "*.mp4"
```

убедитесь, что указываете кавычки, поскольку иначе оболочка расширит охват этого шаблона. В результате Git LFS будет отслеживать все отдельные файлы, соответствующие ему, а новые файлы `.mp4` отслеживаться не будут, потому что в файле `.gitattributes` будут указаны только конкретные имена файлов, а не шаблон.

Сопоставление по глобальному шаблону

Git LFS поддерживает шаблоны отслеживания, аналогичные тем, что поддерживает файл `.gitignore`. Это позволяет указывать их для отслеживания каталогов или сложных имен файлов. Рекомендуется выполнять команду `git lfs track` из каталога верхнего уровня репозитория, поскольку шаблоны сопоставляются относительно каталога, в котором выполняются. Например:

- `git lfs track images/` отслеживает все файлы в каталоге `images` и его подкаталогах;
- `git lfs track *-lfs*.png` отслеживает все файлы `.png`, в названии которых есть `-lfs`.

Сопоставление файлов на чувствительных к регистру платформах

При работе в кроссплатформенной среде различия в регистре символов в именах файлов могут влиять на процессы разработки. Например, когда вы указываете следующий шаблон в Windows или macOS:

```
git lfs track "*.png"
```

сопоставляться будут и файлы `image.PNG`, и `image.png`. А вот на машине с Linux сопоставятся только файлы `image.png`. Рассмотрите такую технику:

```
git lfs track ".*[pP][nN][gG]"
```

Это хороший способ обеспечить правильное сопоставление нечувствительных к регистру файлов с глобальными шаблонами.

Отмена отслеживания файла в LFS

Если вы решите отменить отслеживание определенного типа файлов, используйте команду `git lfs untrack`. После ее выполнения нужно добавить

обновленный файл `.gitattributes` и выполнить его коммит, чтобы зафиксировать изменения:

```
$ git lfs untrack "*.png"
Untracking "*.png"
```

`git add` и `commit` обновленный файл `.gitattributes`.

Разрешение конфликтов слияния двоичных файлов

Разрешать конфликты слияния в текстовых файлах легко, но при работе с крупными двоичными файлами Git не может должным образом обработать конфликты. Лучший способ разрешить такой конфликт — избежать его. С помощью команды `lock` вы можете зарегистрировать файл как заблокированный на удаленном сервере. Так вы добьетесь того, что Git LFS создаст на вашей локальной машине специальный файл *только для чтения*. Тем самым это предотвратит любые попытки редактирования файла пользователями без предварительной его разблокировки. Вы или другой разработчик сможете единолично вносить изменения в крупный отслеживаемый двоичный файл и избежать лишней работы в случае разрешения конфликта слияния, возникшего из-за одновременного изменения этого файла.

Для блокировки отслеживаемого файла LFS укажите, какие типы файлов поддерживают блокировку:

```
$ git lfs track "*.png" --lockable
```

Затем, когда вы соберетесь редактировать файл, укажите команду `lock`:

```
$ git lfs lock assets/image.png
Locked assets/image.png
```

Закончив работу с файлом, снимите с него блокировку командой `unlock`. При передаче изменений из локального репозитория Git LFS будет проверять, не заблокирован ли файл другим разработчиком:

```
$ git lfs unlock assets/image.png
Unlocked assets/image.png
```

Чтобы получить список заблокированных файлов, используйте команду `git lfs locks`:

```
$ git lfs locks
assets/image.png      ppremk      ID:1401911
```



Чтобы разблокировать существующий файл, сопроводите команду опцией `-force`. Например, так: `git lfs unlock assets/image.png --force`.

Преобразование репозиторий для использования Git LFS

Если ваш репозиторий уже содержит крупные файлы и не настроен на использование Git LFS, вы все равно можете установить Git LFS и начать отслеживать известные типы файлов. При этом, когда вы начнете это делать, Git LFS не будет автоматически преобразовывать старые версии этих файлов в объекты Git LFS.

Старые версии — уже часть истории коммитов. Если вам нужно преобразовать их для использования LFS, историю придется переписать. Если вы готовы пожертвовать историей коммитов ради миграции существующего репозитория на использование Git LFS, применяйте команду `git lfs migrate`.

Попробуем:

```
# Клонирование репозитория.
$ git clone https://github.com/ppremk/analyze-this.git
Cloning into 'analyze-this'...
remote: Enumerating objects: 22, done.
remote: Counting objects: 100% (22/22), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 22 (delta 7), reused 22 (delta 7), pack-reused 0
Receiving objects: 100% (22/22), 36.25 KiB | 331.00 KiB/s, done.
Resolving deltas: 100% (7/7), done.

# Инспектируем файлы.
$ ls -al
total 7896
-rw-r--r--  1 ppremk  stax      24 Apr 28 00:47 README-v0.md
-rw-r--r--  1 ppremk  stax      20 Apr 28 00:47 README.md
-rw-r--r--  1 ppremk  stax 25000000 Apr 28 00:47 bigfile1.exe
-rw-r--r--  1 ppremk  stax 30000000 Apr 28 00:47 bigfile2.exe
-rw-r--r--  1 ppremk  stax 35000000 Apr 28 00:47 bigfile3.exe
-rw-r--r--  1 ppremk  stax 10000000 Apr 28 00:47 file4.md
-rw-r--r--  1 ppremk  stax 10000000 Apr 28 00:47 file5.md
-rw-r--r--  1 ppremk  stax 20000000 Apr 28 00:47 file6.md
drwxr-xr-x  4 ppremk  stax    128 Apr 28 00:47 logs
drwxr-xr-x  3 ppremk  stax    96 Apr 28 00:47 scripts
drwxr-xr-x  4 ppremk  stax   128 Apr 28 00:47 toolkit

$ git log --oneline
e2a20a2 (HEAD -> main, origin/main, origin/HEAD) add folders
19263cb add remaining files
9ea727a add bigfiles
1d07224 add updated readme
cd654e0 initial commit
```

По умолчанию команда `git lfs migrate` будет работать только в отношении текущей рабочей ветки. Используйте опцию `-everything`, чтобы включить все ветки репозитория.

Теперь проверим, какие типы файлов занимают в репозитории много места:

```
# Определяем крупные файлы в клонированном репозитории.
$ git lfs migrate info --everything
```

```

migrate: Sorting commits: ..., done.
migrate: Examining commits: 100% (5/5), done.
*.exe      90 MB    3/3 files(s) 100%
*.md       4.0 MB   5/5 files(s) 100%
*.config   14 B     1/1 files(s) 100%

LFS Objects 0 B      0/4 files(s) 0%

```

Переведем эти файлы на использование Git LFS. Помните, что команда `git lfs migrate` выполнит это преобразование только в локальном репозитории. После выполнения преобразования вам нужно принудительно передать преобразованный репозиторий в существующий или создать для него новый удаленный:

```

# Файл ".exe" подходит для перевода на использование Git LFS.
$ git lfs migrate import --everything --include="*.exe"
migrate: Sorting commits: ..., done.
migrate: Rewriting commits: 100% (5/5), done.
    main     e2a20a2db7916f1efc5430a9a15bc5c0a8c41860 -> 7e08e949e76653c8f008a65
              aa42cea9d48b8b473
migrate: Updating refs: ..., done.
migrate: checkout: ..., done.

# Проверяем новую историю коммитов.
$ git log --oneline
7e08e94 (HEAD -> main) add folders
293ff49 add remaining files
9fab33c add bigfiles
bdd78fa add updated readme
b682340 initial commit

# Передайте преобразованный под работу с LFS репозиторий
# в существующий удаленный репозиторий или создайте для него новый.

```



С помощью команды `git lfs migrate` можно проверять, создавать и изменять файлы `.gitattributes`.

Итоги

Приведенные в этой главе советы помогут быстро разобраться с рутинными задачами в Git, особенно когда нужно узнать, что изменилось в вашем репозитории за последнее время, или нужно выполнить прямой поиск объектов в объектном хранилище Git с помощью команды `git grep`. Мы также обсудили способы, как успешно запланировать и выполнить перенос вашего репозитория в полноценную среду Git. В конце главы мы показали технику версионирования крупных файлов с помощью Git LFS: она пригодится при работе с файлами, которые Git не может сжать. Так вы избежите раздувания репозитория.

Git и GitHub

С 2005 года, когда появилась система Git, наблюдается рост все большего числа инструментов для этой платформы. Сегодня их уже более сотни — начиная от десктопных GUI (<https://oreil.ly/f9gaX>) и расширений Git до популярных IDE. Но самый широко известный инструмент среди разработчиков и других специалистов — GitHub (<https://github.com>).

Многие рассматривают GitHub как площадку для совместного программирования (social coding). Такой подход впервые был использован в открытых проектах, но со временем идея кодовой базы как единой точки распределенного сотрудничества прижилась даже в коммерческих компаниях, где пишут проприетарный код. Неудивительно, что эта тенденция привела к изменениям, из-за которых современные разработчики активно используют внутренние ресурсы для поддержки и создания программного обеспечения.

В этой главе мы рассмотрим, как GitHub использует функциональность Git, а также узнаем, чего можно ожидать в случае размещения своего репозитория на этой платформе. Начнем с обзора GitHub как платформы для хостинга личных и корпоративных аккаунтов. Затем обсудим, как GitHub вписывается в экосистему Git. Разберем специфику работы с репозиториями в GitHub и использование доступных возможностей. Также рассмотрим применение простой стратегии ветвления и разрешение конфликтов слияния. В заключение обсудим доступные методы для расширения и интеграции с платформой GitHub, а также разберем, как применять современные методы разработки ПО с помощью этой платформы.

0 GitHub

GitHub — это одна из самых популярных платформ хостинга Git. Она предлагает широкие возможности, которые облегчают совместное программирование, автоматизацию и CI/CD, заботится о безопасности, предоставляет средства для управления проектами, администрирования команд и клиентские приложения. Эта функциональность стала катализатором развития сообщества разработчиков, стремящихся создавать и выпускать программные продукты через эту платформу, будь то для личного или коммерческого использования.

На момент написания книги сообщество GitHub насчитывало 83+ миллиона разработчиков, 4+ миллиона организаций и 200+ миллионов репозиториев. При этом платформа представлена примерно в 90 % компаний из списка Fortune 100, что делает ее самой крупной и продвинутой площадкой для разработки в мире (<https://oreil.ly/B6tLd>) (рис. 18.1).



Рис. 18.1. Домашняя страница GitHub

Виды учетных записей GitHub

Для использования GitHub нужно создать учетную запись (аккаунт) на сайте платформы GitHub.com. Аккаунт — это ваш идентификатор, который представляет вас как личность. Наличие активного аккаунта позволяет участвовать в написании размещенного на GitHub кода. Это также позволяет организовывать и контролировать доступ к расположенному на этой площадке коду или репозиторию.

На GitHub есть три вида аккаунтов.

Личные учетные записи (personal)

У любого личного аккаунта есть имя пользователя и профиль. На рис. 18.2 показан пример профиля пользователя. Имея активный аккаунт, вы можете создавать репозитории, пакеты GitHub и проекты. Все действия, которые вы совершаете на платформе, будут ассоциироваться с вашим личным аккаунтом.

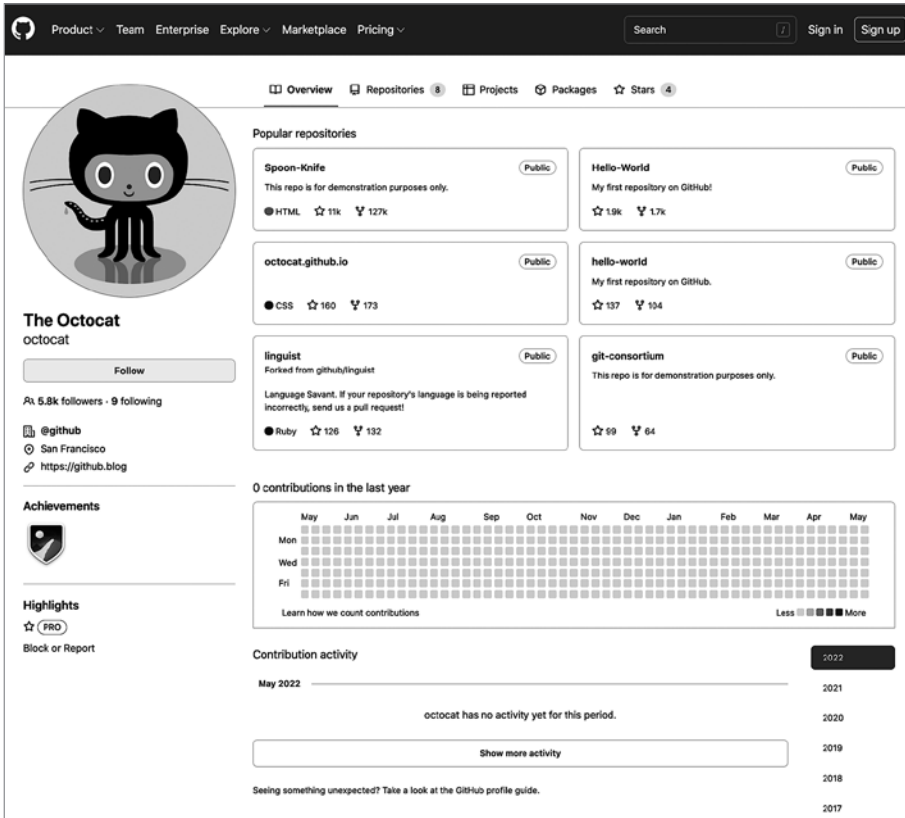


Рис. 18.2. Профиль Octocat

Личные аккаунты бывают двух видов: GitHub Free и GitHub Pro. Ни один из них не имеет ограничений на количество публичных или частных репозиториев, которыми может владеть аккаунт. Разница в том, что с аккаунтом GitHub Free у вас будет доступ только к ограниченному набору функций для частных репозиториев.

Учетные записи организации (organization)

Такие аккаунты — общие, они позволяют неограниченному количеству участников совместно работать над ресурсами, принадлежащими организации.

Такая учетная запись позволяет заводить собственные репозитории и проекты, а также иметь пакеты GitHub. На рис. 18.3 показана возможность отображения содержимого файла README.md на странице профиля.

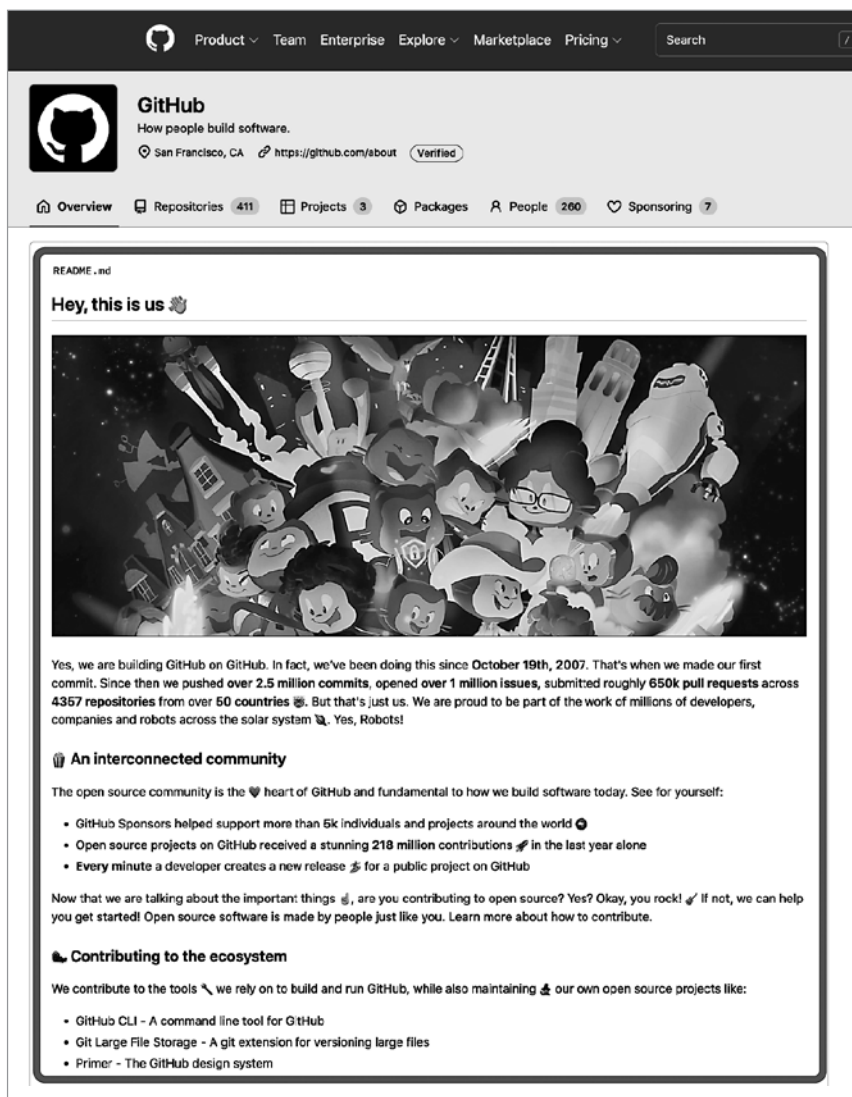


Рис. 18.3. Профиль аккаунта организации на GitHub

Если вы получили доступ к работе с организацией на GitHub, то войти напрямую в учетную запись этой организации не можете. Для этого нужно войти в свою личную учетную запись. Когда вы работаете с ресурсами (например,

репозиториями) внутри организации, то ваши действия будут ассоциированы с личным аккаунтом, а не с организацией. При этом аккаунт может быть связан с несколькими организациями.

Всем личным аккаунтам, имеющим доступ к аккаунту организации, присваивается одна из трех ролей: *владелец* (*owner*), *участник* (*member*) и *внешний участник* (*outside collaborator*). Владелец также является администратором организации. Роли предоставляют разные уровни доступа к самой организации и ее ресурсам. Этот принцип показан на рис. 18.4, который взят из официальной документации GitHub (<https://oreil.ly/QD7xC>).

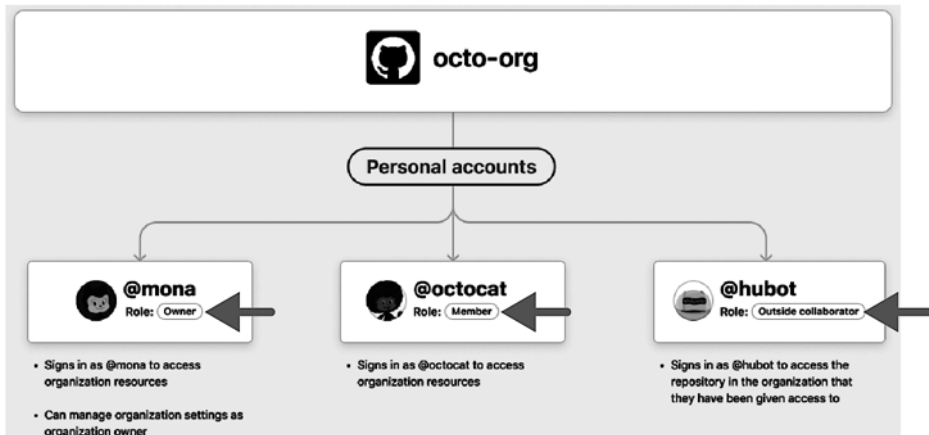


Рис. 18.4. Роли в аккаунте организации

Учетные записи предприятия (*enterprise*)

Учетная запись предприятия похожа на аккаунт организации. Отличие в том, что с таким аккаунтом вы можете управлять несколькими организациями.

GitHub Enterprise Cloud и GitHub Enterprise Server — это варианты аккаунтов предприятия. С их помощью можно внедрять и централизованно управлять любыми политиками или платежной информацией одной или нескольких организаций на аккаунте предприятия. Они также позволяют использовать различные опции внедрения этих политик, делегировать их настройку и право на внедрение владельцам организаций, повышая гибкость администрирования аккаунта.

Есть два варианта развертывания: облачный дата-центр GitHub Enterprise Cloud и развертывание GitHub Enterprise Server на собственном дата-центре либо на платформах поддерживаемых провайдеров.

При использовании GitHub вы авторизуетесь через личный аккаунт. Аккаунт организации помогает расширить сотрудничество между личными аккаунтами,

а аккаунт предприятия упрощает централизованное управление несколькими организациями.



Посмотреть стоимость услуг и общий обзор возможностей, доступных для каждого вида аккаунтов, можно на сайте GitHub (<https://oreil.ly/YiH5i>). Там же доступно расширенное сравнение этих возможностей (<https://oreil.ly/m4rvL>).

GitHub в экосистеме Git

В разделе «Компоненты Git» на с. 24 (глава 1) мы привели обзор компонентов Git. На рис. 18.5 та же схема представлена в контексте GitHub.

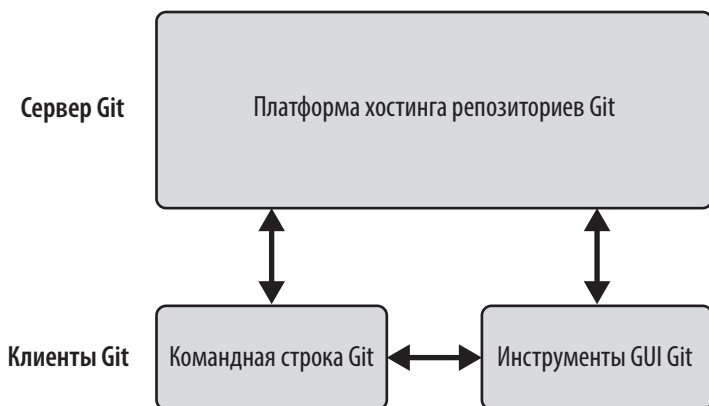


Рис. 18.5. Обзор компонентов Git

Согласно рис. 18.5, GitHub — это платформа хостинга репозитория Git, что с технической точки зрения делает ее сервером Git. Помимо поддержки функциональности самого Git, GitHub предлагает возможности, позволяющие реализовать совместное программирование. Примеры таких возможностей: создание тикетов (issues), пул-реквесты, защищенные ветки, управление правами доступа для владельцев кода, возможности организаций и команд, а также проекты для организации и управления задачами.

GitHub — это клиент Git, и со временем платформа выработала и сделала свободно доступными некоторые инструменты, которые способствуют ее популяризации.

К ним относятся:

GitHub Desktop

Позволяет работать в привычном потоке Git, предоставляя визуальный интерфейс. Приложение GitHub на основе Electron дает возможность интерактивно

создавать и исправлять коммиты, переключаться на ветки из пул-реквестов, создавать новые ветки, с легкостью менять репозитории и даже добавлять к коммитам соавторов. На рис. 18.6 показано, что Git Desktop — это GUI для *Git Client*.

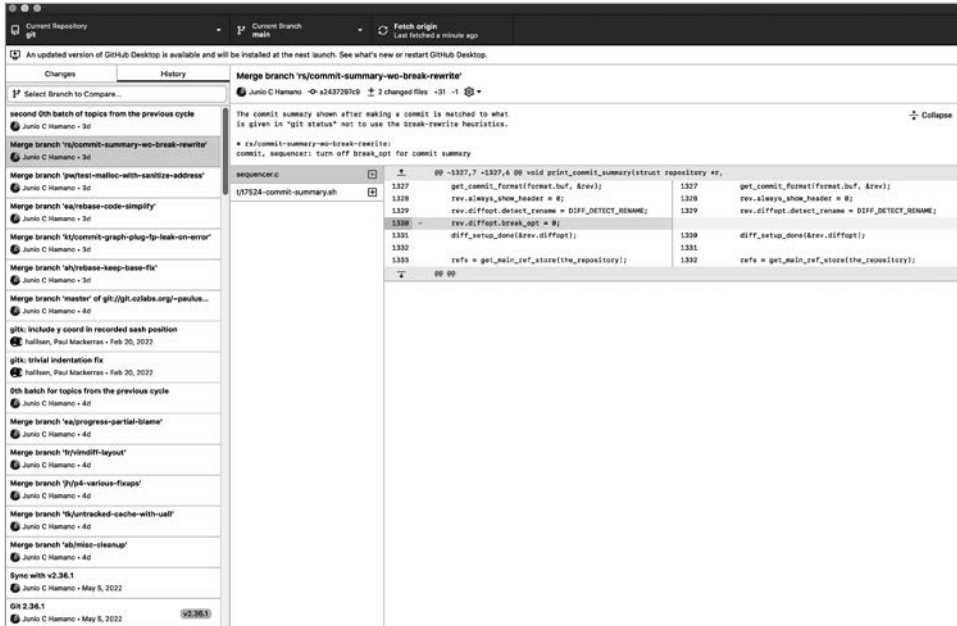


Рис. 18.6. GitHub Desktop



Если вы хотите поучаствовать в разработке GitHub Desktop, то можете сделать форк его репозитория (<https://oreil.ly/5pOEm>).

GitHub CLI

Инструмент командной строки, привносящий функциональность GitHub в вашу среду разработки (выполняется либо как системный процесс без подключения к терминалу, либо как эмулятор терминала). Позволяет взаимодействовать с тикетами, пул-реквестами и различными функциями GitHub прямо из среды разработки. Создан с целью сократить число переключений контекста между средой разработки и GitHub UI при выполнении типовых задач.

GitHub CLI можно установить для поддерживаемых операционных систем с помощью пакетных менеджеров. Инструмент бесплатен и может быть расширен дополнительной функциональностью.

После установки используйте команду `gh` для запроса и работы с любым репозиторием GitHub непосредственно из терминала:

```
# Установка GitHub CLI.
```

```
$ gh repo
```

```
Work with GitHub repositories
```

```
USAGE
```

```
gh repo <command> [flags]
```

```
CORE COMMANDS
```

```
clone:      Clone a repository locally
create:     Create a new repository
fork:       Create a fork of a repository
list:       List repositories owned by user or organization
sync:       Sync a repository
view:       View a repository
```

```
INHERITED FLAGS
```

```
--help      Show help for command
```

```
ARGUMENTS
```

```
A repository can be supplied as an argument in any of the following formats:
```

- "OWNER/REPO"
- by URL, e.g. "https://github.com/OWNER/REPO"

```
EXAMPLES
```

```
$ gh repo create
```

```
$ gh repo clone cli/cli
```

```
$ gh repo view --web
```

```
LEARN MORE
```

```
Use 'gh <command> <subcommand> --help' for more information about a command.
```

```
Read the manual at https://cli.github.com/manual
```



GitHub CLI дополняет, но не заменяет собой нативные команды Git. Тем не менее он добавляет функциональность GitHub в командную строку. Подробнее об установке и участии в разработке см. на сайте (<https://oreil.ly/XB9bG>).

GitHub Mobile

Клиентское приложение, позволяющее управлять работой на GitHub через мобильные устройства и смартфоны. Однако через мобильное приложение вам доступен ограниченный набор возможностей платформы. Вы можете приоритезировать уведомления и управлять ими, а также участвовать в обсуждении тикетов и пул-реквестов. В приложении также можно настроить двухфакторную аутентификацию.

GitHub Codespaces

Облачная среда разработки. Работает на базе Visual Studio Code, доступ к которой можно получить через браузер.

Новое рабочее пространство можно создать из любой ветки или коммита репозитория GitHub. При этом в процессе разработки используются облачные вычислительные ресурсы. Для запуска рабочего пространства можно выбрать различные конфигурации вычислительных ресурсов на базе виртуальных машин, размещенных на GitHub и имеющих от 2 до 32 вычислительных ядер.

Преимущество использования рабочего пространства — в предоставлении согласованной среды разработки. Достаточно зафиксировать файл конфигурации пространства в репозиторий, и его сможет использовать любой участник, имеющий доступ к репозиторию.

Подробности о Codespaces см. в документации GitHub (<https://oreil.ly/uTvwd>).

На рис. 18.7 показано, как легко запустить новую среду из браузера. Это приближает нас еще на один шаг к избавлению от проблемы «а на моем компьютере работает», с которой разработчики сталкиваются уже много лет.

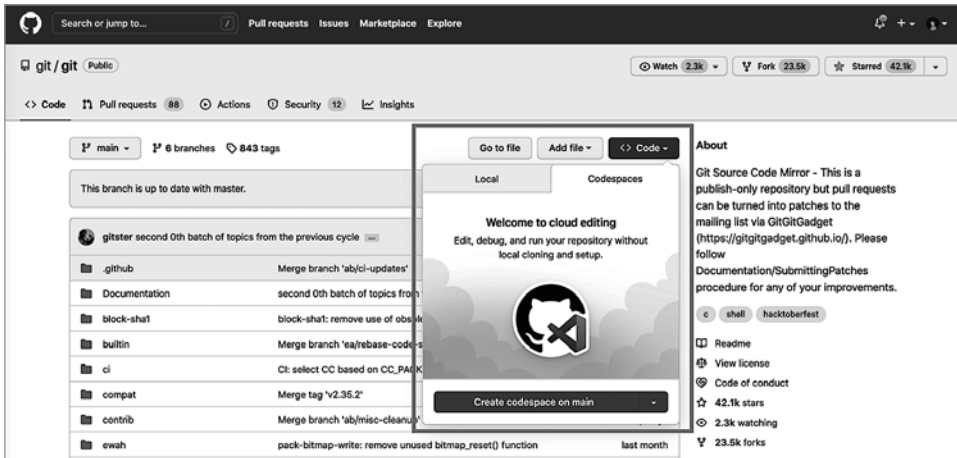


Рис. 18.7. Запуск рабочего пространства



Администратор организации может менять систему оплаты при использовании рабочего пространства, устанавливая лимит затрат, чтобы избежать чрезмерных расходов.

Размещение репозитория на GitHub

Зарегистрировав учетную запись, вы можете создать репозиторий и начать совместную работу. После авторизации на странице **Sign In** система перенаправит вас на **GitHub Dashboard** (рис. 18.8). Здесь вы увидите различную полезную информацию. Вы можете быстро переключаться на репозитории, с которыми недавно

работали, отслеживать последнюю активность, получать от GitHub информацию о последних изменениях, а также узнавать о новых проектах или действиях людей, на которых подписались.

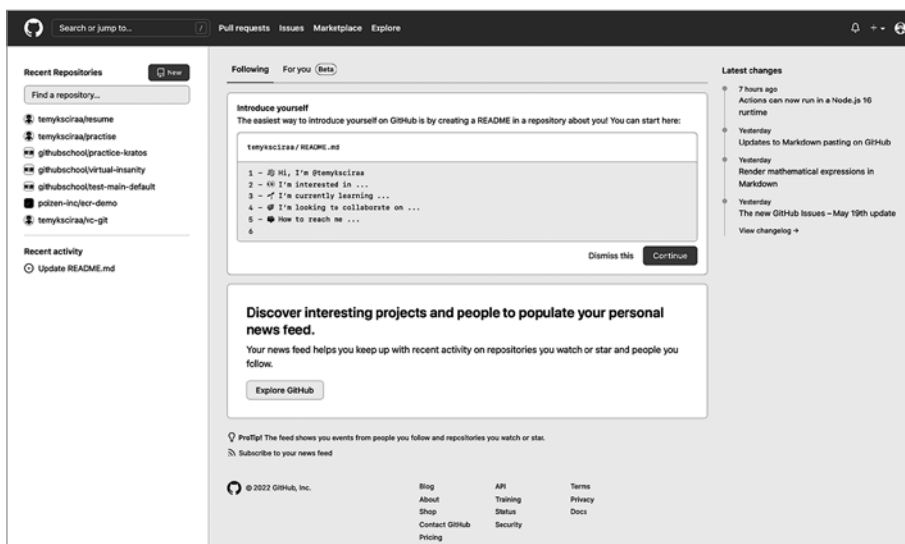


Рис. 18.8. GitHub Dashboard

Чтобы создать новый репозиторий, кликните на кнопке **New** в верхнем левом углу страницы. Откроется страница **Create New Repository** (рис. 18.9). Укажите название нового репозитория, который будет закреплен за вашим личным аккаунтом. Если вы член организации, то можете выбрать аккаунт организации в выпадающем меню **Owner**.



Создать репозиторий можно также на странице **New Repository** (<https://oreil.ly/LBZIr>).

На этой странице можно добавить описание репозитория и выбрать тип репозитория — открытый (**public**) либо закрытый, или приватный (**private**). В зависимости от типа аккаунта возможен третий вариант — **Internal**. Открытый репозиторий доступен всем посетителям GitHub.com. Найти приватный репозиторий сложнее — для этого нужно, чтобы владелец пригласил участников присоединиться. Владелец репозитория может в любое время изменить настройки видимости. Важно учитывать последствия таких изменений, особенно если у проекта уже сформировалось растущее сообщество активных участников. Если вы планируете сделать проект доступным бесплатно, то советуем сразу обозначить его как открытый.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? Import a repository.

Owner * **Repository name ***

temyksiraa / vc-git-3rd ✓

Great repository names are short and memorable. Need inspiration? How about **fantastic-goggles**?

Description (optional)

Demo repo for Version Control with Git 3rd Edition

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☒ **Add a README file**
This is where you can write a long description for your project. [Learn more.](#)

Add .gitignore
Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: None

Choose a license
A license tells others what they can and can't do with your code. [Learn more.](#)

License: MIT License

This will set **main** as the default branch. Change the default name in your settings.

ⓘ You are creating a public repository in your personal account.

Create repository

© 2022 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Docs](#) [Contact GitHub](#) [Pricing](#) [API](#) [Training](#) [Blog](#) [About](#)

Рис. 18.9. Создание репозитория

Определившись с видимостью репозитория, инициализируйте его. Это означает создать голый репозиторий с файлом `README.md`. Вы также можете добавить дополнительные файлы: `.gitignore` на основе списка шаблонов популярных языков программирования и файл *лицензий*, предлагающий шаблоны на выбор.



Если вы сомневаетесь, какой тип лицензии выбрать для вашего проекта, то изучите виды лицензий на сайте (<https://oreil.ly/sI7U7>).

Если у вас уже есть репозиторий, который нужно перенести на GitHub, не выбирайте опцию **Add a README file**. Так вы создадите вышестоящий репозиторий-заполнитель с нужным названием. Эта тема подробно была рассмотрена в разделе «Перенос из системы Git» на с. 431 главы 17. Щелкнув на кнопке **Create repository**, вы создадите репозиторий с указанными настройками.

Представление репозитория

На рис. 18.10 показана страница репозитория, где представлены вкладки и ссылки на часто используемые при разработке функции.

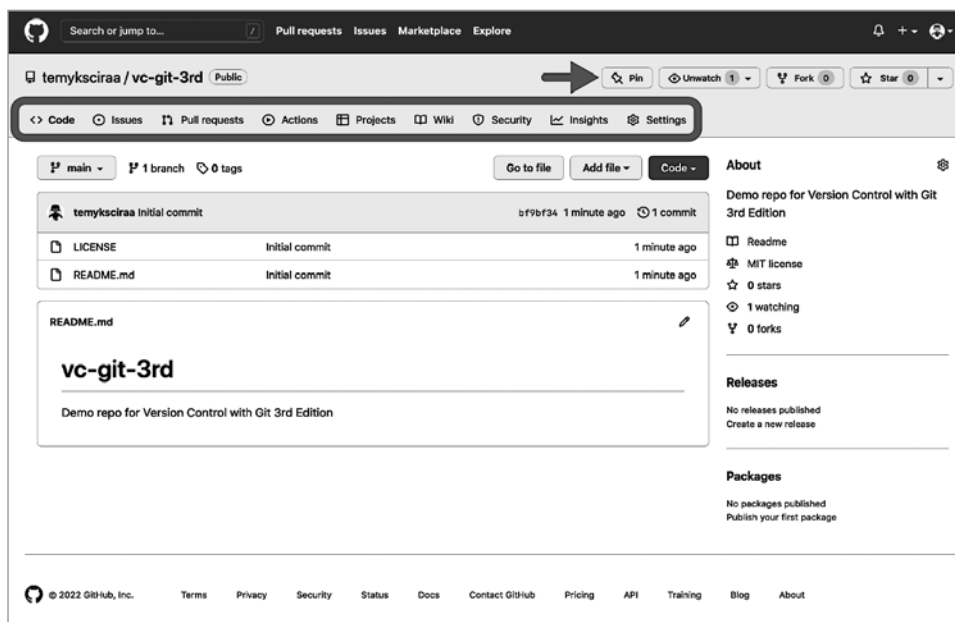


Рис. 18.10. Представление репозитория

Страницу репозитория вы видите по умолчанию при его открытии на GitHub. Здесь показаны имя владельца и название репозитория. На рис. 18.10 представлен открытый репозиторий `vc-git-3`, его владелец — `temyksciraa`. Выше расположены кнопки **Pin**, **Watch**, **Fork** и **Star**.

Pin закрепляет репозиторий на странице вашего профиля (рис. 18.11).

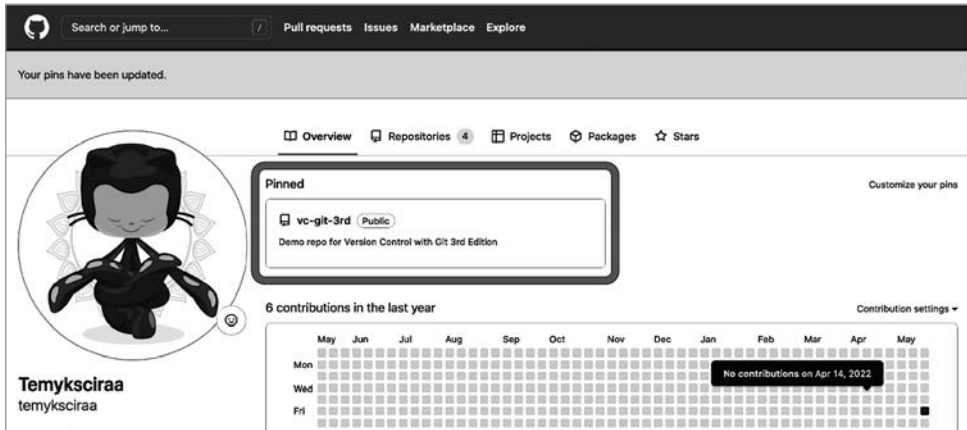


Рис. 18.11. Закрепленный репозиторий

Как показано на рис. 18.12, вы можете настраивать уведомления о действиях в репозитории. Эти опции позволяют выбирать интересующие вас действия. В открытом проекте число наблюдателей часто становится показателем его полезности или популярности.

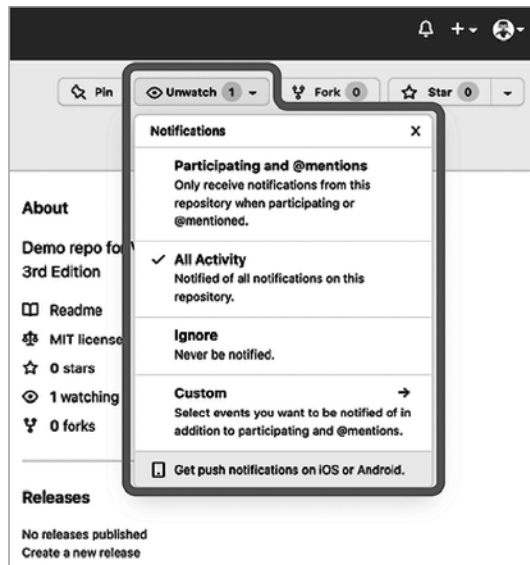


Рис. 18.12. Опции наблюдения



Опция `@mention` представляет удобный способ уведомить либо включить участника в обсуждение тикетов или пул-реквестов. Применяется к аккаунту пользователя или команды.

Опция **Fork** позволяет создавать копию репозитория в аккаунте, к которому у вас есть доступ.

Раньше термин *forking* (создание форка) имел негативную коннотацию. Еще недавно создание форка означало агрессивное разделение проекта, чтобы увести его развитие в иное направление.

Однако на GitHub идея форков воспринимается в позитивном смысле и позволяет большему числу контрибьюторов внести свой вклад в управляемом и прозрачном контексте. Это не представляет риска для основного проекта, поскольку изменения происходят в ответвленном репозитории.

Основное преимущество такой модели — в прозрачности и публичной видимости вкладов со стороны участников. Прежде чем изменения будут обсуждаться и включаться в основной проект, сообщество может посмотреть их и оценить. На рис. 18.13 показана зеркальная копия исходного кода Git на GitHub, которая представляет отличный пример форка.

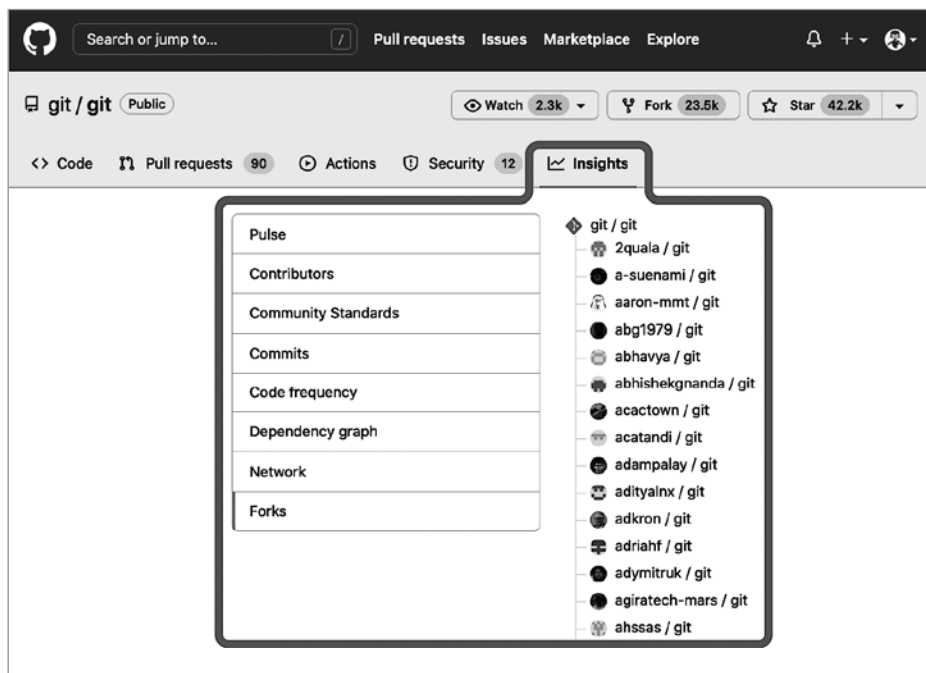


Рис. 18.13. Создание форков

Кнопка **Star** позволяет создавать список избранных репозиториях, которые вас интересуют или на которые вы подписаны. С ее помощью вы можете отслеживать разные категории репозиториях (рис. 18.14).

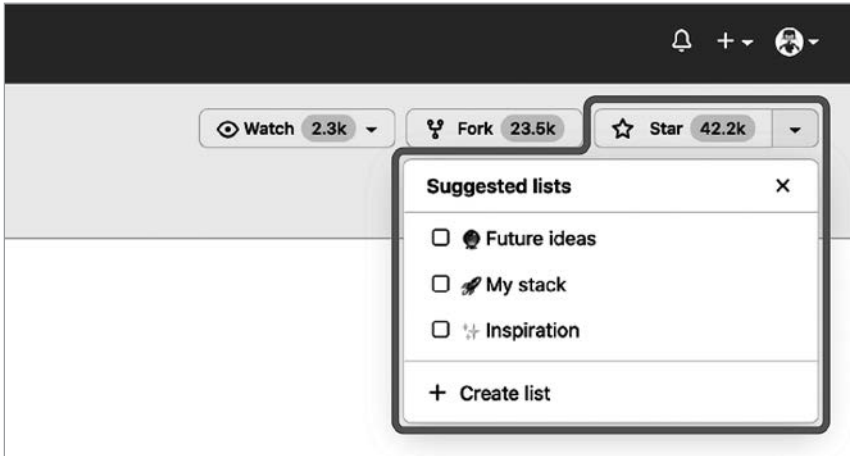


Рис. 18.14. Создание списков Star

Code

На рис. 18.15 показано корневое представление репозитория. Вы можете увидеть общее число веток и тегов, а также раздел **About** с описанием репозитория и ссылками на **README**, **License**, **Stars**, **Code of Conduct** (если доступен), **Watching** и **Forks**. Если репозиторий содержит **Releases** или **GitHub Packages**, они также будут отображены здесь. В этом окне представлены иконки профилей участников проекта и используемые в репозитории языки программирования с указанием их процентного соотношения.

Вы можете найти конкретный файл в репозитории, используя кнопку **Go to file**. Чтобы добавить в репозиторий файл, не прибегая к клонированию, используйте **Add File**. На рис. 18.16 показано, как получить удаленный URL репозитория для клонирования через поддерживаемые протоколы. На этой странице можно запустить выделенную рабочую среду (при возможности).

На рис. 18.17 показаны заголовок и информация о файлах репозитория. Заголовок представляет собой краткое изложение сообщения коммита вместе с его коротким SHA и временной меткой. В нем также отражается общее число коммитов репозитория.

Если щелкнуть по деталям коммита в заголовке, то вы окажетесь на странице с подробной информацией о конкретном коммите. Чтобы открыть визуальное представление истории коммитов выбранной ветки, щелкните по общему числу коммитов в заголовке (рис. 18.18).

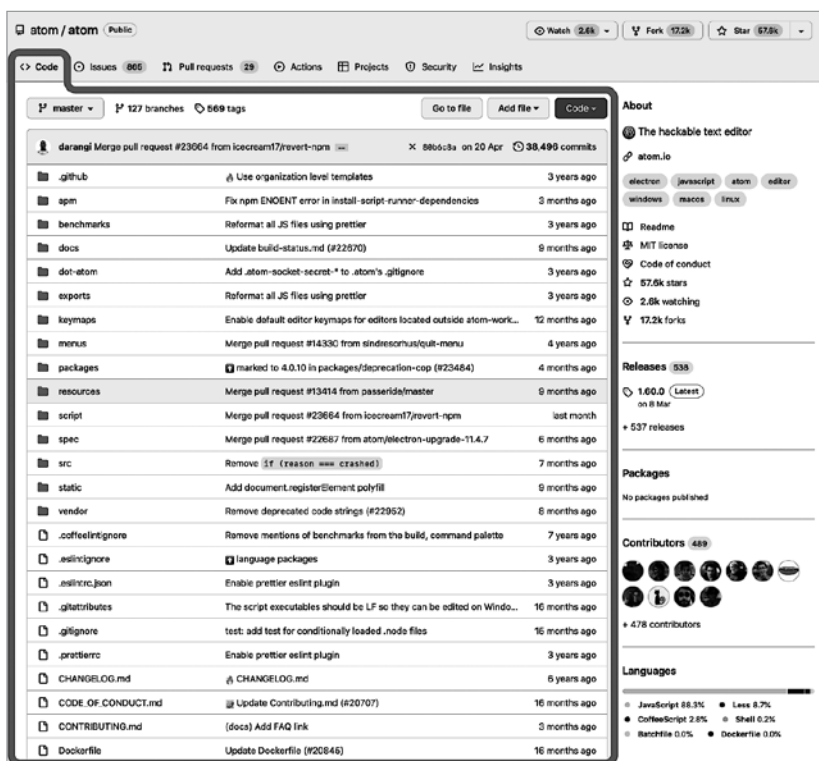


Рис. 18.15. Представление кода

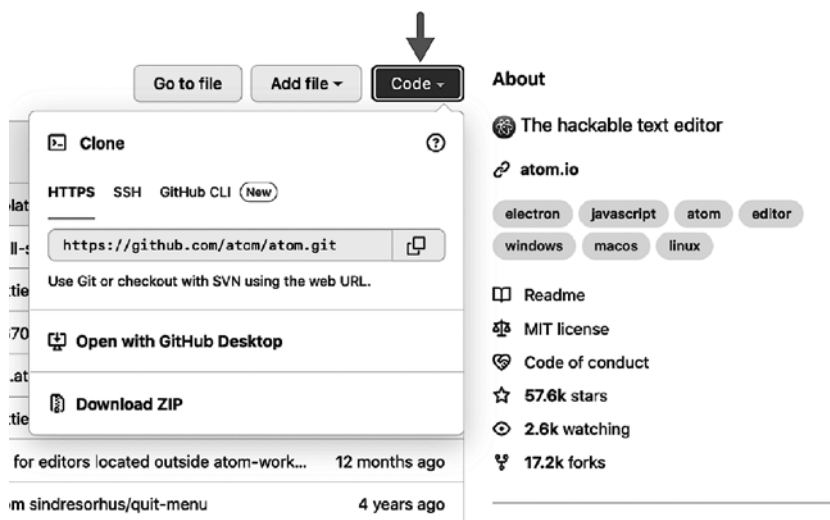


Рис. 18.16. Кнопка Code



Рис. 18.17. Список файлов репозитория

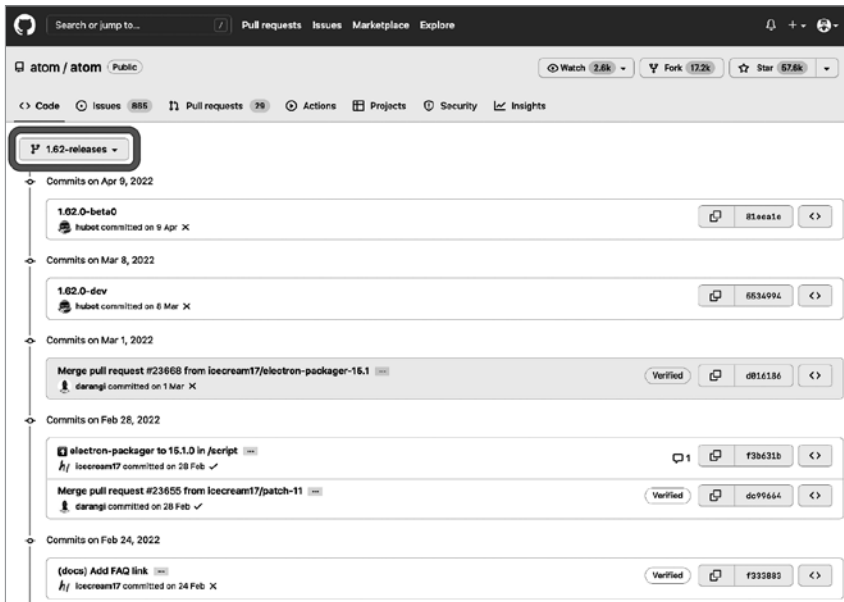


Рис. 18.18. История коммитов

Представление списка файлов репозитория точно соответствует файлам и структуре вашего проекта. Другими словами, если вы посмотрите на список файлов в клонированном репозитории на GitHub, то увидите точно такие же файлы и структуру, какие имеются в вашем проекте. Такое представление хорошо тем, что, если у вас в каталоге есть файл `README.md` с разметкой Markdown, интерфейс автоматически отобразит его содержимое в представлении Code. Файлы `README` можно добавлять в определенные каталоги, чтобы улучшить асинхронное взаимодействие и повысить эффективность опенсорсной разработки.

Тикеты в GitHub

Тикеты (в терминологии GitHub — Issues) помогают организовывать и управлять рабочими процессами на GitHub. И хотя тикеты не являются функциональностью Git, они позволяют отслеживать обратную связь, предложения идей, задачи и даже баги. При создании тикета вы можете перекрестно ссылаться на связанные с ним другие тикеты и пул-реквесты, а также — через **@mention** — на команды или пользователей, причастных к репозиторию. На рис. 18.19 показаны все связанные с конкретным репозиторием тикеты (в закрытом или открытом состоянии). Это позволяет отражать исторический контекст: как или почему были приняты те или иные решения в ходе разработки проекта. В этом же представлении вы можете закреплять наиболее популярные тикеты.

В списке Issues можно упорядочивать ключевые точки (Milestone) и метки истории репозитория, отфильтровывать их и управлять ими. Кнопка **New Issue** создает новые тикеты.

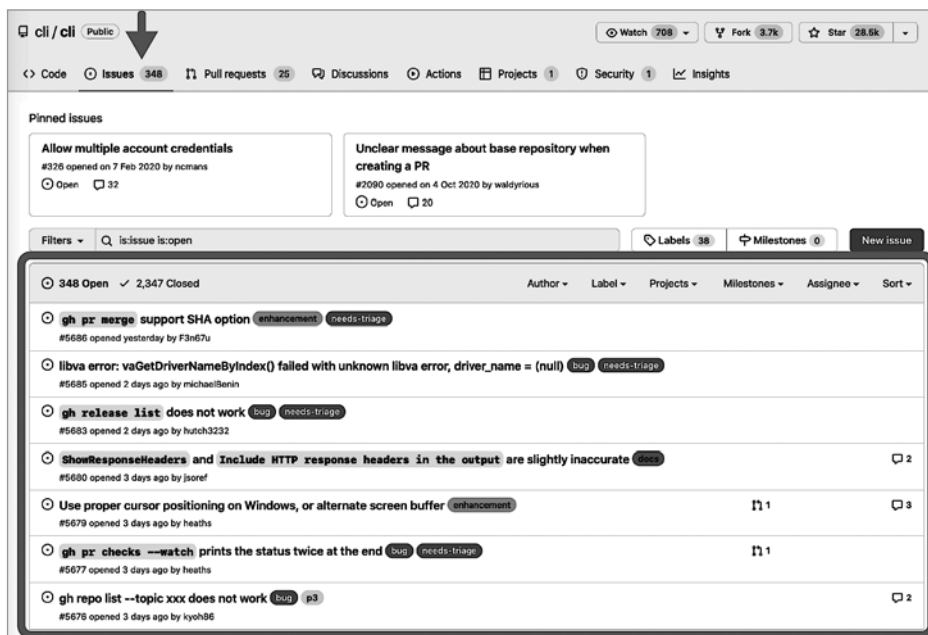


Рис. 18.19. Список тикетов

На рис. 18.20 показана основная ветка для тикета, за которой следуют ответы на комментарии к нему. В поле **Comment** можно писать и форматировать ответы, используя синтаксис *GitHub Flavored Markdown*. Правильно использовать этот синтаксис поможет соответствующая панель инструментов. Все действия, которые вы совершаете в тикете, отражаются на его временной линии.

Сюда относятся добавление участников, создание меток и внесение тикетов в раздел **Milestone** или **Projects**. Для конкретного тикета можно создать новую ветку, где будет вестись работа по добавлению функциональности или исправлению бага, либо связывать этот тикет с пул-реквестом, содержащим изменения для решения задачи.

Отслеживать прогресс тикетов можно через подписку. В настройках вы можете указать, о каких событиях хотите получать уведомления (рис. 18.21).

При наличии необходимых прав на управление репозиторием вы можете блокировать обсуждение тикета, закреплять его, переносить и даже удалять (рекомендуется закрывать тикет, а не удалять: это позволит зафиксировать историю развития проекта).

Использование тикетов для организации и отслеживания работы на GitHub описано в документации платформы ([https:// oreil.ly/5iuAU](https://oreil.ly/5iuAU)).

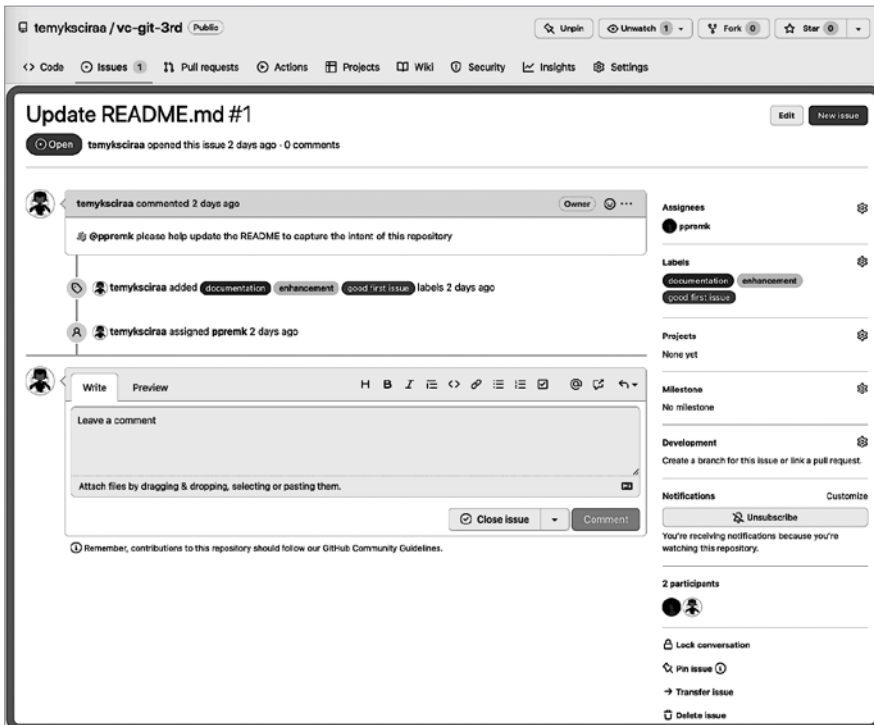


Рис. 18.20. Представление тикетов



Спецификация для синтаксиса GitHub Flavored Markdown доступна на сайте (<https://oreil.ly/ZXE7c>).

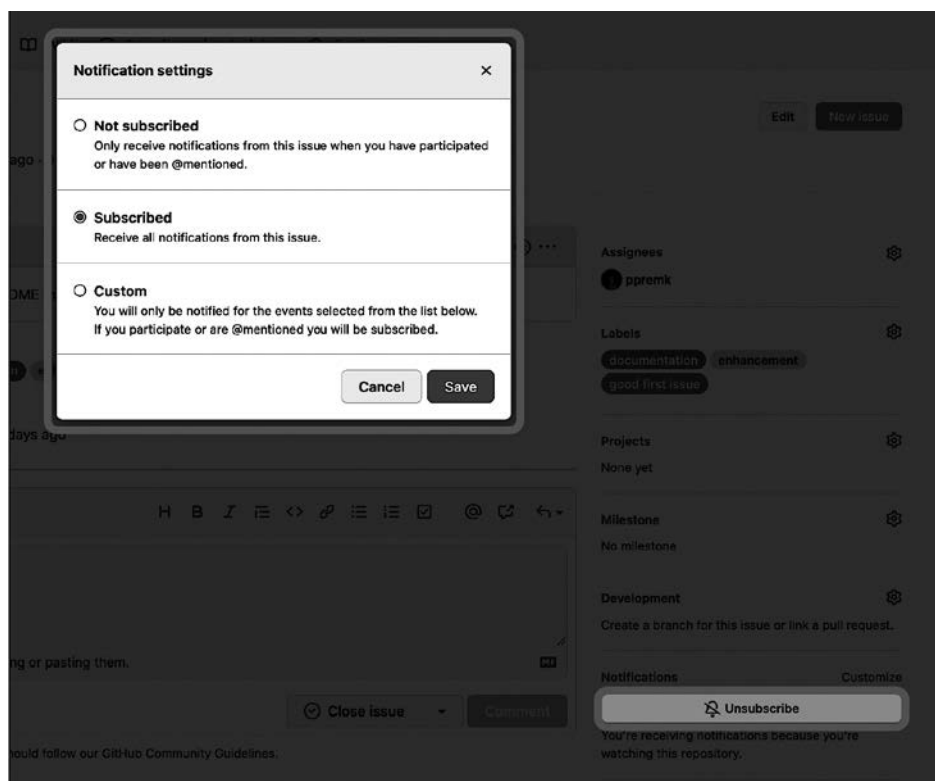


Рис. 18.21. Настройки уведомлений о прогрессе тикета

Пул-реквесты

Пул-реквесты сообщают об изменениях, которые вы предлагаете внести в ветку конкретного репозитория. Вы можете инициировать код-ревью и обсуждение потенциальных изменений с другими участниками репозитория, что позволит внести необходимые доработки или коммиты на основе полученной обратной связи. Как только все требования будут выполнены, можно перейти к слиянию вашего пул-реквеста с целевой веткой.

Как и в случае с тикетами (см. рис. 18.19), пул-реквесты перечисляются в виде списка в соответствующей вкладке (рис. 18.22) и имеют похожую функциональность.

Кнопка **New pull request** создает новый пул-реквест.

Рисунок 18.23 во многом схож с рис. 18.20, за исключением некоторых моментов. Например, здесь есть дополнительная кнопка **Reviewers** для назначения ревьюеров на рассмотрение любого открытого пул-реквеста. Все действия, которые вы совершаете в окне пул-реквестов, отражаются на временной линии пул-реквеста, включая коммиты, которые вы добавляете через основную ветку.

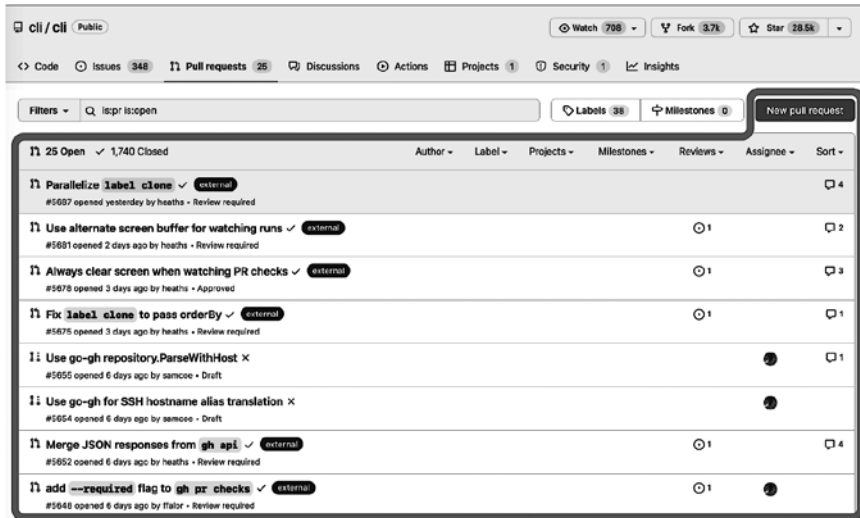


Рис. 18.22. Список пул-реквестов

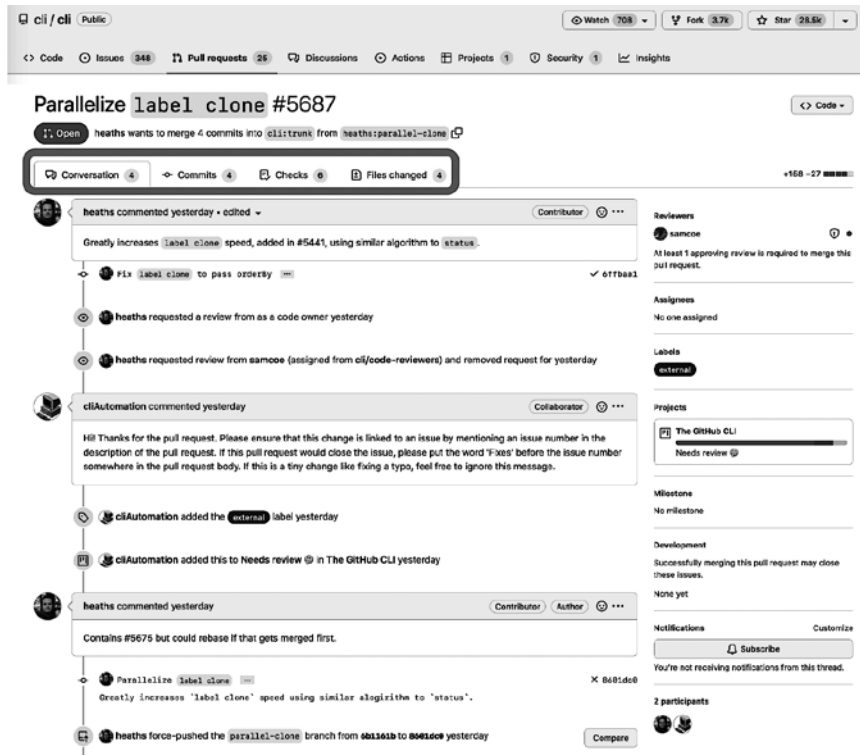


Рис. 18.23. Окно пул-реквестов



Ревьюеров можно назначать автоматически, если в ветке репозитория определен файл CODEOWNERS (<https://oreil.ly/dHqJs>).

Представление пул-реквестов, показанное на рис. 18.23, содержит несколько вкладок. **Conversations** используется для организации обсуждения ревью изменений, которые вы вносите через ветку функциональности. Эти обсуждения могут отражаться в виде комментария к тикету или встраиваться в строку кода, которую предполагается фиксировать.

На рис. 18.24 показан пример добавления обсуждения в код.

Open Parallelize **label clone** #5687
heaths wants to merge 4 commits into cli:trunk from heaths:parallel-clone

heaths reviewed 20 hours ago View changes

pkg/cmd/label/clone.go Outdated

```

149 +     doneCh := make(chan struct{})
150 +     go func() {
151 +         for range created {
131 152             successCount++
132 153         }
154 +         close(doneCh)
155 +     }()

```

heaths 20 hours ago Contributor Author ...

@samcoe this is the same pattern you added to `status` but I think there's a potential deadlock issue here. In trying to figure out why the mac test failed (it's still not clear since that's the only API that would return an error in the `errgroup.Group`), I found this would occasionally deadlock in an error case. I think we also need to select `ctx.Done()` and/or check if the channel was closed (might actually be redundant). I'm pushing another commit that shows this. See what you think, and I'd be happy to add another commit to fix `status` or open another PR.

heaths 19 hours ago Contributor Author ...

I appreciate the Go axiom "share by communicating", but for just incrementing a counter perhaps this is a bit convoluted? I still think the new pattern might be good to use for `status` but what about just using a mutex here or `atomic.AddUint32()` from `sync/atomic`? The `errgroup` support would still be valuable - just thinking about eliminating the separate goroutine to handling incrementing the counter.

Reply...

heaths added 2 commits 20 hours ago

- Fix deadlock issue X e9323ac
- Fix intermittent error ✓ 6cbe33e

Рис. 18.24. Обсуждение в коде

Во вкладке **Conversation** внизу страницы есть раздел, содержащий список одной из функций, связанных с качеством кода, в запросе на объединение изменений (еще одна функция находится во вкладке **Checks**). Представление на рис. 18.25

информирует, соответствуют ли ваши коммиты всем установленным владельцем репозитория стандартам и условиям.

Parallelize label clone #5687
heaths wants to merge 4 commits into cli:trunk from heaths:parallel-clone

heaths added 2 commits 20 hours ago

- Fix deadlock issue ✖ a9323ac
- Fix intermittent error ✓ 6cbe33e

Review required Hide all reviewers
At least 1 approving review is required by reviewers with write access. [Learn more.](#)

1 pending reviewer ⬆

samcoe was requested for review

All checks have passed Hide all checks
6 successful checks

✓	Code Scanning / CodeQL-Build (pull_request)	Successful in 4m	Details
✓	Lint / lint (pull_request)	Successful in 1m	Details
✓	Tests / build (ubuntu-latest) (pull_request)	Successful in 2m	Required Details
✓	Tests / build (windows-latest) (pull_request)	Successful in 8m	Required Details
✓	Tests / build (macos-latest) (pull_request)	Successful in 4m	Required Details
✓	Code scanning results / CodeQL	Successful in 5s — No new or fixed alerts	Details

Merging is blocked
Merging can be performed automatically with 1 approving review.

Write **Preview** H B I ≡ <> 🔗 ≡ ≡ ☑ @ ↺ ↻

Leave a comment

Attach files by dragging & dropping, selecting or pasting them.

Comment

Remember, contributions to this repository should follow its contributing guidelines and code of conduct.

ProTip! Add .patch or .diff to the end of URLs for Git's plaintext views.

Рис. 18.25. Проверка статуса пул-реквеста

Проверки статуса в основном ассоциируются с внешними процессами, чаще всего с вашими сборками для непрерывной интеграции и конвейерами развертывания, линтингом кода, а также проверками его качества, которые выполняются при каждой передаче пул-реквеста в ветку. Владельцы репозитория и его администраторы

могут настраивать нужные проверки статуса с помощью правил защиты веток из раздела **Code and automation** на странице конфигурации.

На вкладке **Commits** представлена история коммитов (рис. 18.26), связанных с вашим пул-реквестом. Отсюда можно переходить к конкретным коммитам, щелкая по их SHA, сообщению коммита или кнопке просмотра кода — **<>**. В виде галочек здесь отображаются уведомления о проверке статуса каждого коммита.

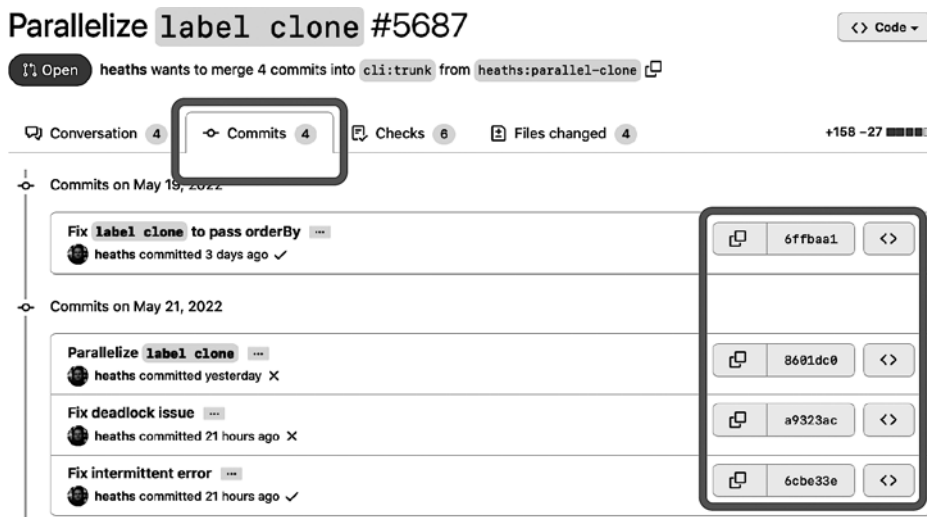


Рис. 18.26. История коммитов ветки

В пул-реквесте есть два типа проверок статуса. Первая показана на рис. 18.25. Вторая находится на странице **Checks** (рис. 18.27). Проверки на вкладке **Checks** предоставляют подробные аннотации строк и сообщения и доступны только для использования с приложением GitHub.



Как указано в официальной документации GitHub (<https://oreil.ly/0jzxv>), вкладка **Checks** будет заполняться только в том случае, если вы настроили проверки.

При ревью пул-реквеста любые участники с доступом на чтение репозитория могут просматривать и комментировать предлагаемые изменения. Предложения могут касаться конкретных строк кода в файлах или всей связки коммитов в ветке пул-реквеста. Во вкладке **Files changed** мейнтейнер репозитория может просматривать входящие изменения и применять их напрямую из пул-реквеста. На рис. 18.28 показано представление после вкладки **Files changed**.

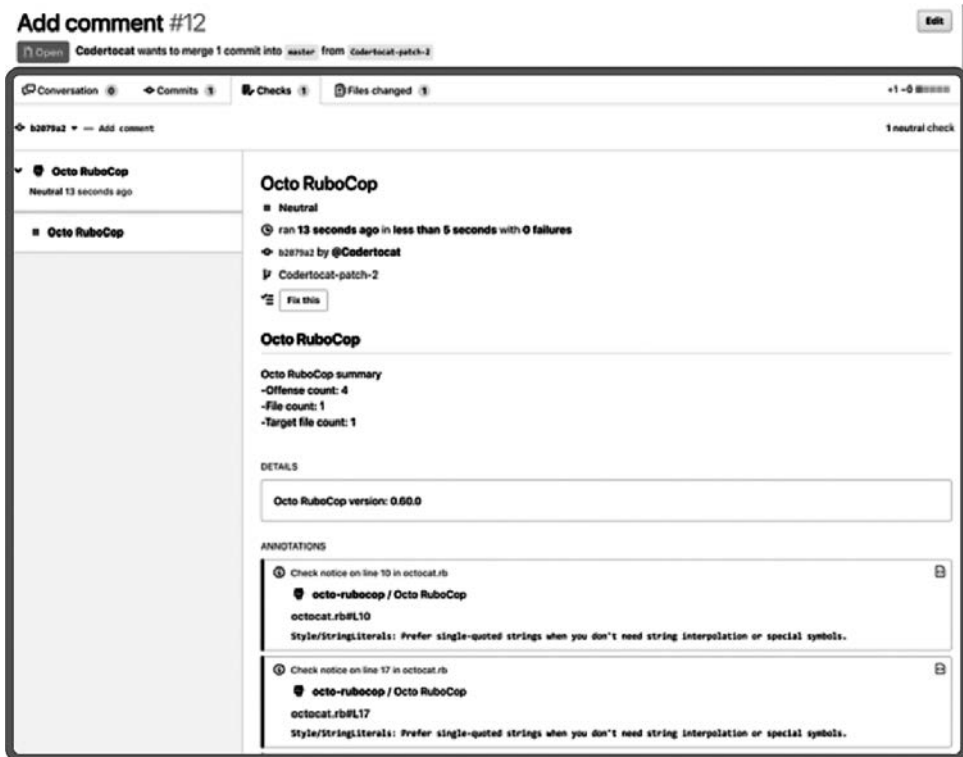


Рис. 18.27. Вкладка Checks

Как ревьюер, вы можете предложить изменения, нажав на синий плюсик, появляющийся при наведении курсора на строки файла (рис. 18.29).

Представление дерева файлов обеспечивает удобство навигации между файлами, которые редактируются в пул-реквесте. Оно позволяет сравнивать изменения из выбранных коммитов или всех коммитов этого пул-реквеста (рис. 18.30).

Просматривая внесенные в файл изменения, вы можете выбрать вариант представления diff, щелкнув по значку шестеренки. Можно выбрать единое или раздельное представление, а также скрыть при необходимости пробелы (рис. 18.31).

В качестве мейнтейнера или владельца репозитория вы можете утверждать пул-реквесты (рис. 18.32) через меню **Review changes**. Вы можете оставить комментарий, утвердить пул-реквест либо попросить внести в него изменения. Вы можете утверждать пул-реквесты или запрашивать изменения только от других участников репозитория.

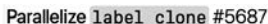


Рис. 18.28. Измененные файлы



Рис. 18.29. Предложение изменений

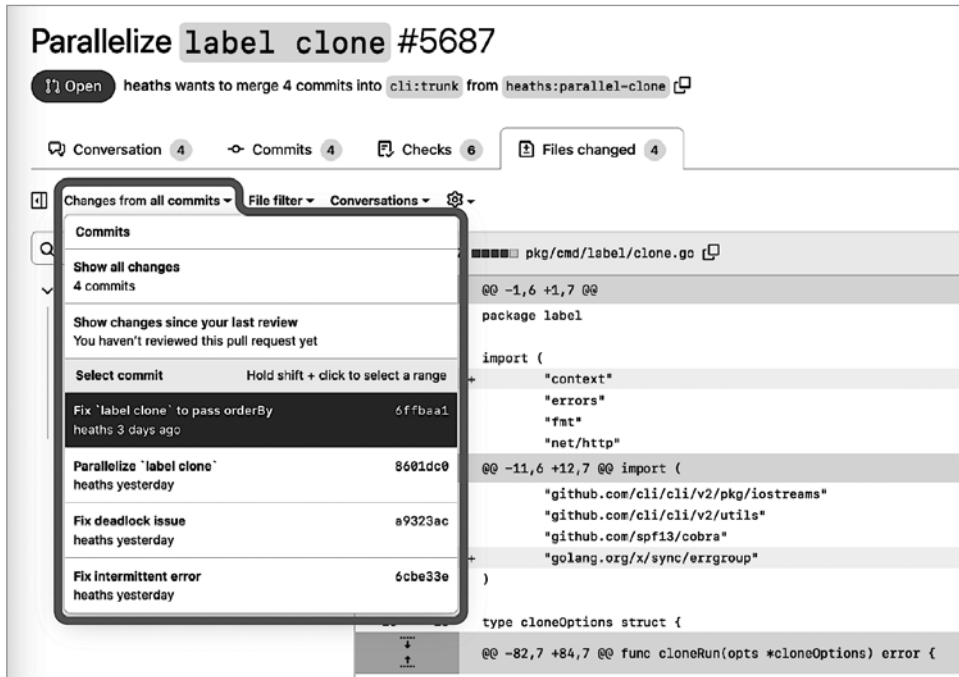


Рис. 18.30. Сравнение изменений

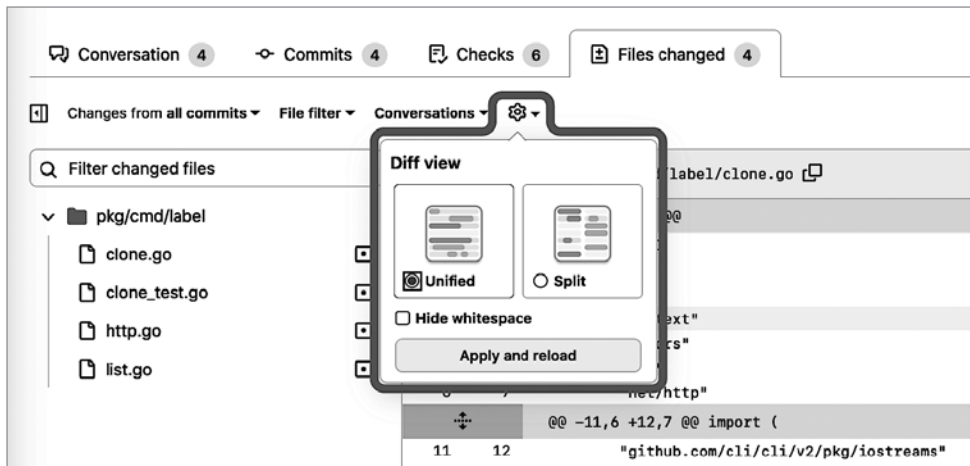


Рис. 18.31. Представление diff

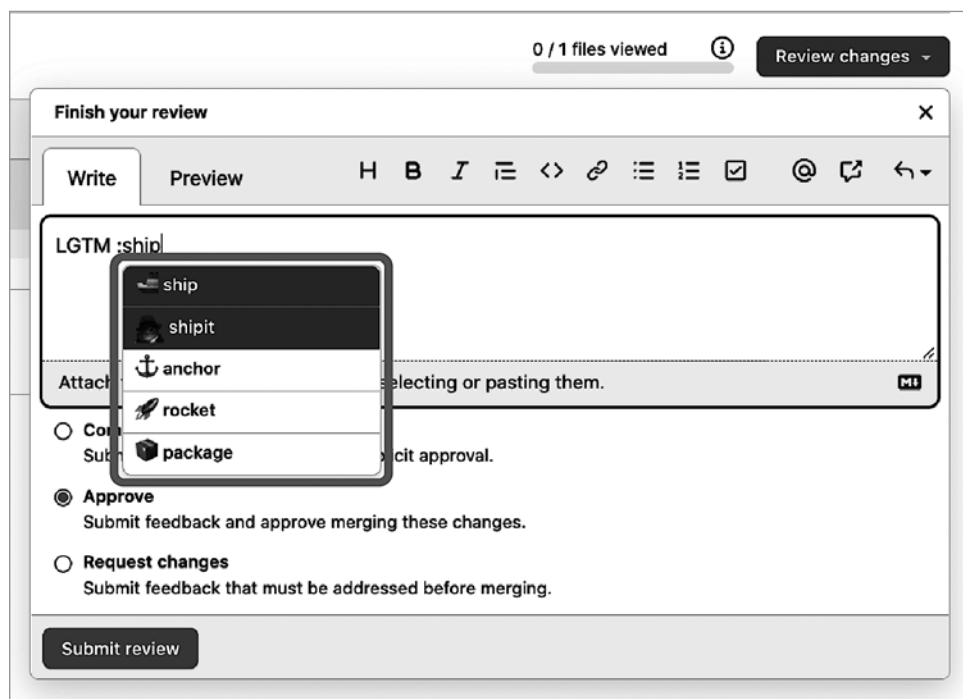


Рис. 18.32. Утверждение пул-реквеста



При утверждении пул-реквеста используется акроним LGTM (Looks Good To Me — «мне нравится»), сопровождаемый командой shipit.

На рис. 18.33 показан результат принятия пул-реквеста. Если все проверки статуса проходят и нужные утверждения получены, пул-реквест считается готовым к слиянию с целевой веткой. GitHub предоставляет три варианта слияния (рис. 18.34). Опции говорят сами за себя и происходят из нативных стратегий слияния Git, которые мы разбирали в главе 17 (Create a merge commit — создать коммит слияния, Squash and merge — Сжать и выполнить слияние, Rebase and merge — Перебазировать и выполнить слияние).

После слияния пул-реквеста желательно удалять ветку, то есть выбирать опцию Delete branch (рис. 18.35), — так вы сохраните порядок в репозитории. Удаленную ветку при желании можно восстановить позднее, но имейте в виду, что в таком случае могут возникнуть конфликты, поскольку проект за это время изменится.

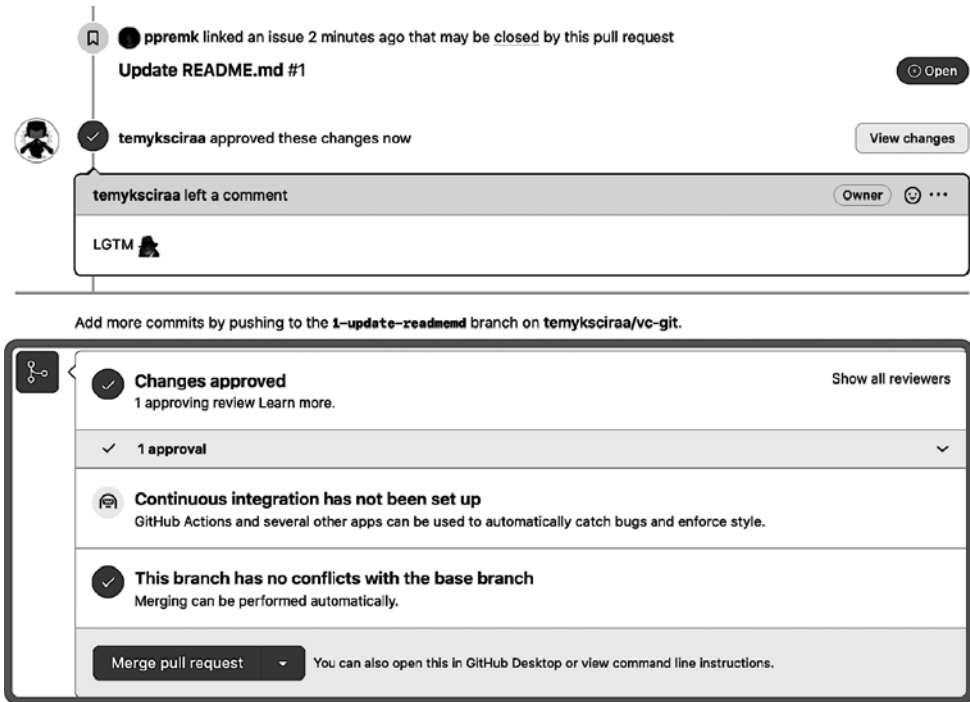


Рис. 18.33. Утвержденный пул-реквест в обсуждении

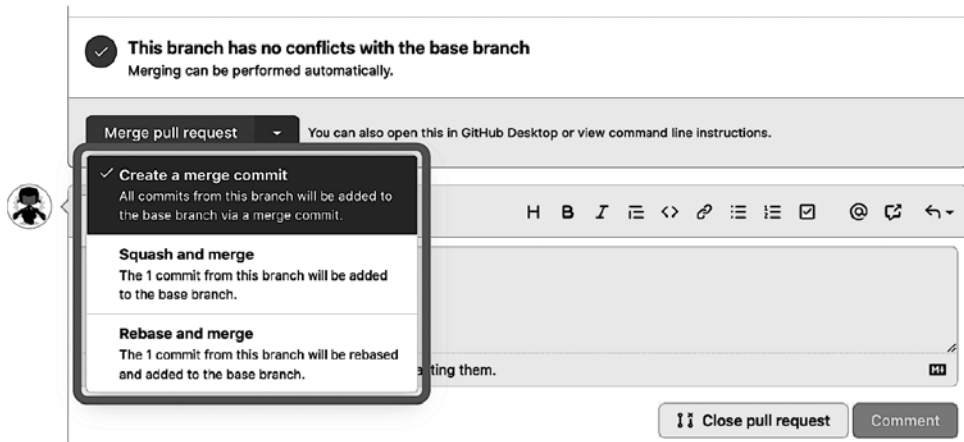


Рис. 18.34. Три опции слияния

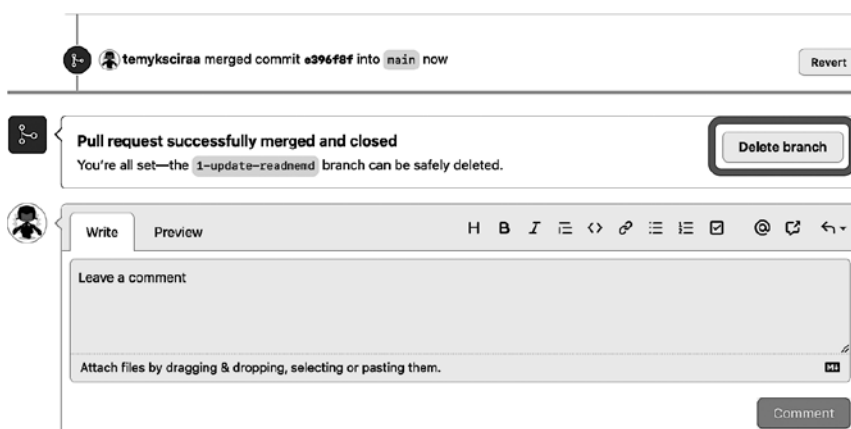


Рис. 18.35. Удаление ветки

GitHub Flow

GitHub Flow представляет собой обычный рабочий поток на основе веток. Стратегия ветвления в GitHub Flow строится из одной долгоживущей ветки и краткосрочных веток, через которые в нее вносятся изменения.

Долгоживущая ветка является веткой *по умолчанию* и называется *main* (основная). Краткосрочные — это ветки *функциональности*, представляющие вносимые изменения.

Краткосрочные ветки используются для внесения в репозиторий изменений. При этом изменения вливаются в основную ветку только после того, как содержащие их ветки выполняют все установленные требования. Эти изменения могут включать и исправления багов, и добавление разной функциональности.

Если вы собираетесь работать по схеме GitHub Flow, то можете сделать это напрямую через веб-интерфейс GitHub либо использовать стандартную командную строку Git, GitHub CLI, GitHub Desktop или любую IDE с поддержкой Git.

Шаг 1: создание ветки

Для начала нужно создать в репозитории ветку (рис. 18.36). Выберите содержательное название для ветки: это поможет вашим коллегам быстро понять, какие изменения через нее вносятся. При работе в команде лучше придерживаться согласованной схемы именования. Это упростит процесс разработки.

Внесение изменений через второстепенную ветку даст возможность экспериментировать и править изменения отдельно от ветки *main*, не нарушая работу основной кодовой базы.

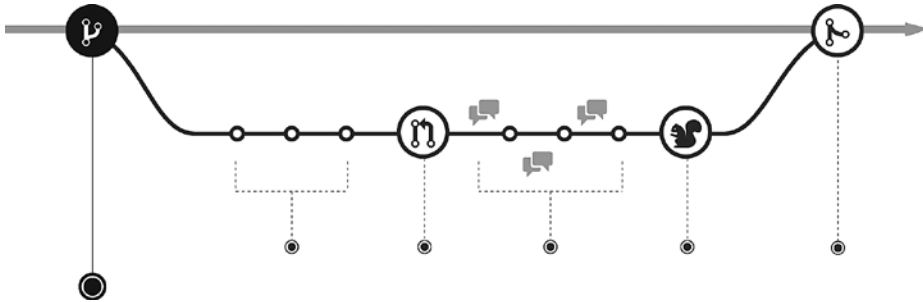


Рис. 18.36. Создание ветки

Шаг 2: внесение изменений

Следующий шаг — внесение изменений. Это делается с помощью коммитов (рис. 18.37). Коммиты должны быть атомарными, то есть каждый коммит должен захватывать и группировать изменения в логическую единицу. Если вам понадобится отменить какое-либо изменение, достаточно будет отменить один коммит, а другие изменения останутся нетронутыми.

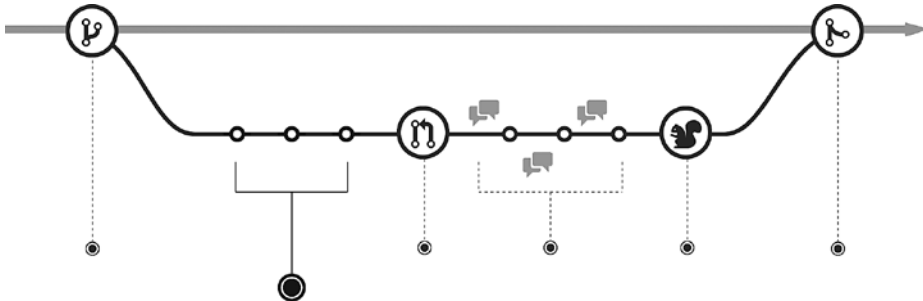


Рис. 18.37. Внесение изменений

Короткое и четкое сообщение коммита облегчит вам и вашим коллегам дальнейшую работу.

Шаг 3: создание пул-реквеста

Когда вы будете готовы представить изменения коллегам, откройте пул-реквест для вашей ветки (рис. 18.38). Помимо контекста вносимых изменений вам нужно указать другую важную информацию, которая пригодится ревьюерам. Это позволит участникам проводить обоснованные обсуждения вокруг конкретных изменений в асинхронном режиме и сделает процесс ревью более плавным.

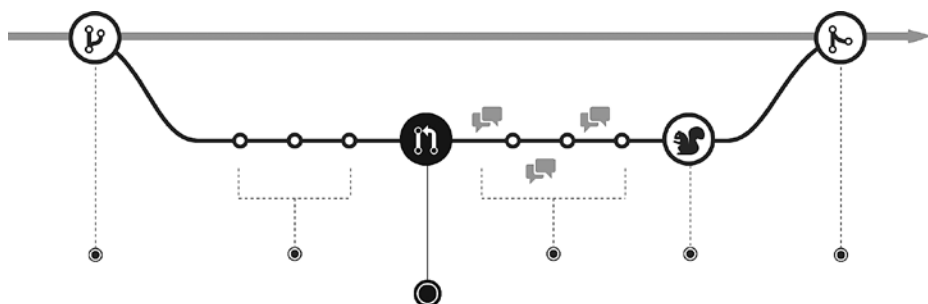


Рис. 18.38. Создание пул-реквеста

Пул-реквест, который вы открываете, находится на GitHub. Для этого нужно передать изменения на удаленный репозиторий, размещенный там. То есть если вы работали на локальной машине, то вам нужно передать изменения в удаленный вышестоящий репозиторий на GitHub.com.

Шаг 4: работа с комментариями

После открытия пул-реквеста вы с коллегами можете обсудить содержащийся в нем код и изменения (рис. 18.39). Если в репозитории настроены проверки качества кода, убедитесь, что все установленные критерии были соблюдены.

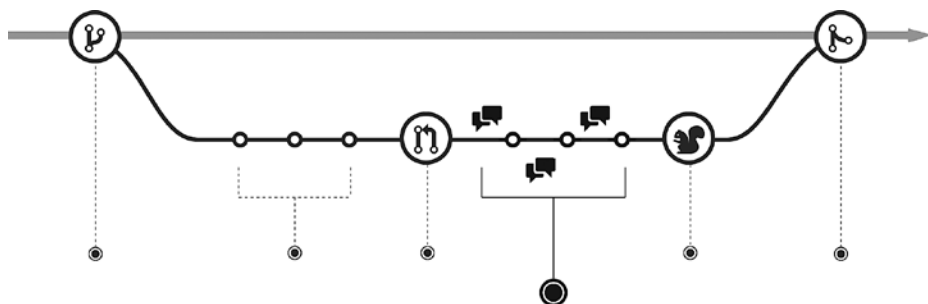


Рис. 18.39. Работа с комментариями

Ревьюеры могут запрашивать изменения, оставлять комментарии и утверждать ваши изменения. Доступ ко всему этому вы можете получить на вкладке Conversation.

Шаг 5: развертывание кода

После утверждения пул-реквеста вы можете развернуть код из вашей ветки функциональности в промежуточную или продакшен-среду (рис. 18.40). Этот шаг во многом зависит от вашего рабочего потока, в частности от автоматизации, сборок непрерывной интеграции и процессов развертывания, выбранных командами в качестве практики разработки.

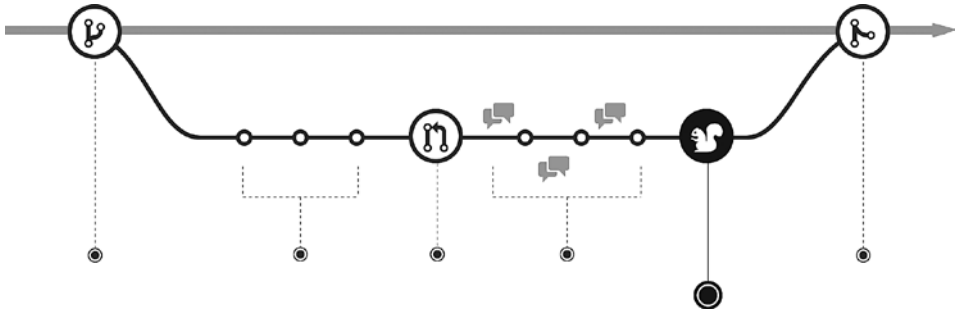


Рис. 18.40. Развертывание кода

Функциональность *ChatOps* позволяет развертывать пул-реквест в продакшен-среде, делая его доступным для любого участника репозитория. В качестве одной из лучших практик при реализации подобных рабочих потоков развертывание может производиться в промежуточную (stage) среду, а не в продакшен.

Шаг 6: слияние пул-реквеста

Последний этап в GitHub Flow — слияние пул-реквеста с веткой `main` (рис. 18.41). Здесь также требуется обеспечить соблюдение всех требований, установленных правилами защиты веток в репозитории. Это может включать прохождение всех проверок, получение нужных статусов и утверждений.

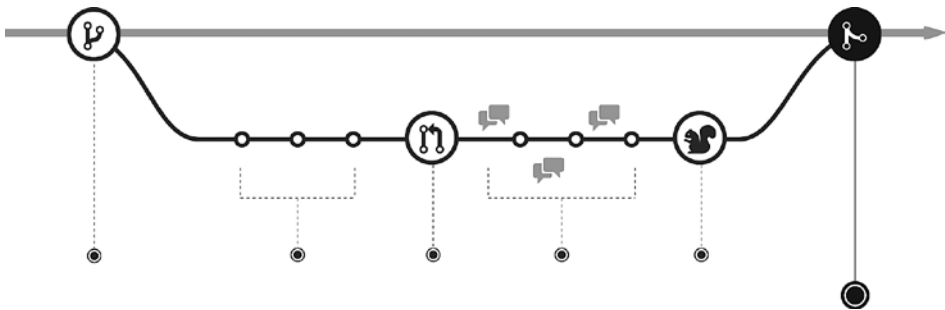


Рис. 18.41. Слияние пул-реквеста

После успешного слияния ветки функциональности с основной рекомендуется удалить эту ветку, поскольку все ее изменения уже будут частью основной линии. Это исключит вероятность того, что ваши коллеги по ошибке начнут работать с устаревшей веткой.

Полное руководство по GitHub Flow, включая промежуточные шаги, доступно в официальной документации платформы (<https://oreil.ly/VKZc9>).

Помимо GitHub Flow есть и другие рабочие потоки, которые вы можете применять в соответствии с вашими потребностями при разработке. См. документацию Git (<https://oreil.ly/siP25>).

Другой популярный поток — *Git Flow*. Изучите пост Винсента Дризена (Vincent Driessen) (<https://oreil.ly/Y6prx>), где он подробно рассказывает о модели ветвления Git Flow.

Разрешение конфликтов слияния в GitHub

Конфликты слияния — это естественный побочный продукт при работе в распределенных системах контроля версий, и они не всегда означают что-то плохое. Конфликты слияния возникают, когда ветки содержат конфликтующие изменения.

При работе с конфликтами через веб-интерфейс GitHub вы можете визуально сравнивать изменения, которые хотите внести из вашей ветки, с целевой веткой. Вы можете открывать список конфликтующих изменений или файлов кнопкой **Resolve conflicts** (рис. 18.42) на странице **Pull request**.



Рис. 18.42. Разрешение конфликта слияния

Смоделируем конфликт слияния и рассмотрим, как его разрешить через веб-интерфейс GitHub.

Для внесения изменения в файл `README.md`, находящийся в нашем примере репозитория, используем GitHub Flow. Для простоты изложения пропустим описанные выше шаги 4 и 5, чтобы сфокусироваться на ключевых моментах.

Шаг 1: создание ветки

Создайте новую ветку из веб-интерфейса GitHub от имени пользователя `temyksciraa`, щелкнув по меню выбора веток. Придумайте для ветки осмысленное название и выберите опцию **Create branch** (рис. 18.43).

Шаг 2: внесение изменений

Внесите изменения в файл `README.md`. Можно отредактировать его напрямую, щелкнув по значку карандаша. Внесите в него содержательное сообщение коммита и сделайте коммит в ранее созданную ветку (рис. 18.44).

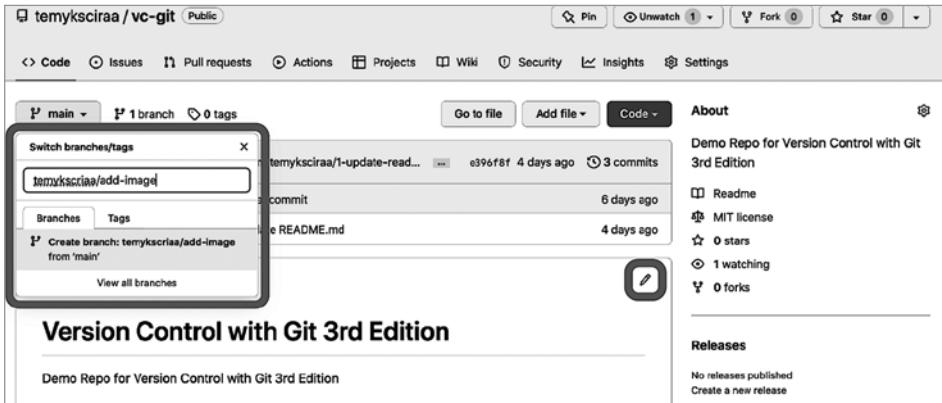


Рис. 18.43. Добавление новой ветки функциональности

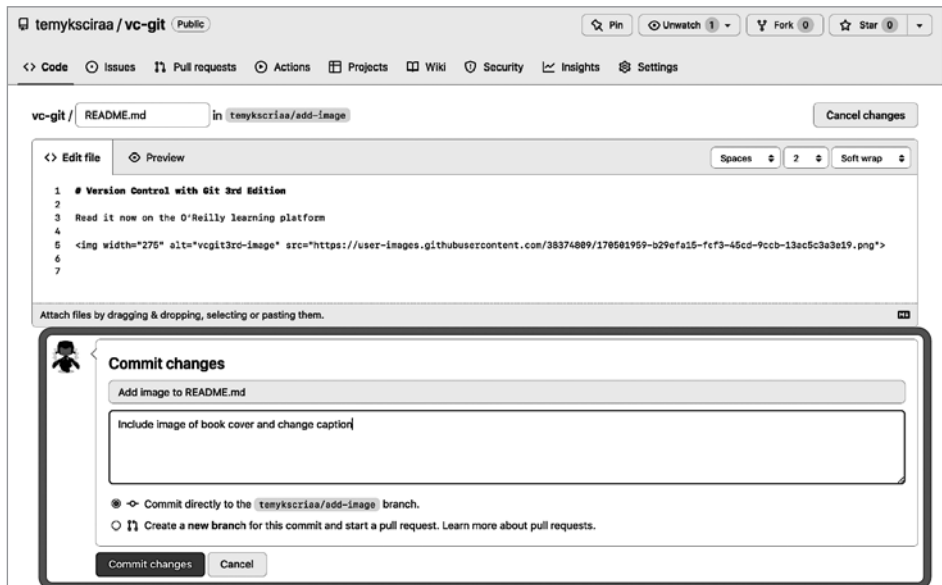


Рис. 18.44. Внесение изменений в файл README.md

Шаг 3: создание пул-реквеста

На вкладке **Code** можно быстро создать пул-реквест из веток, в которые недавно были внесены изменения и которые были отправлены в репозиторий. GitHub автоматически обнаруживает такие изменения и предлагает название ветки. Щелкните по **Compare & pull request**, и система перенаправит вас на страницу **Create a pull request**, где вы можете проверить вносимые изменения и открыть новый пул-реквест (рис. 18.45).



Чтобы этот пример сработал, не вливайте созданный пул-реквест сразу в основную ветку.

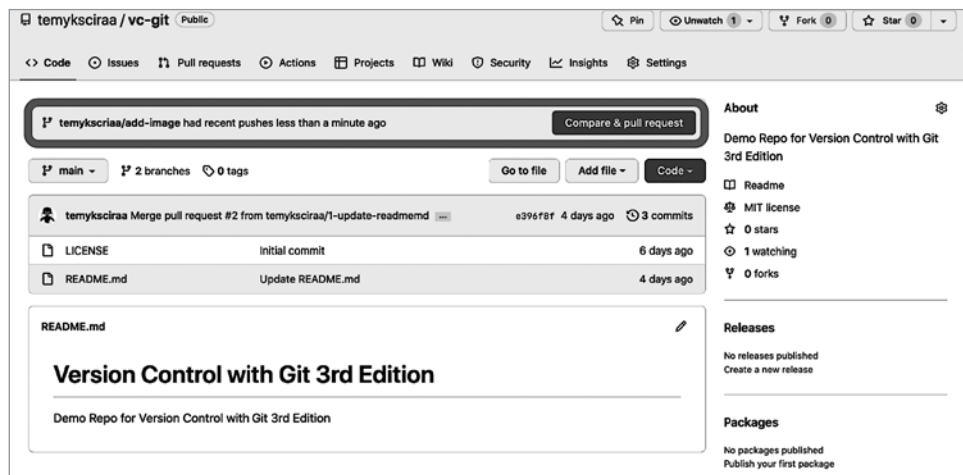


Рис. 18.45. Создание пул-реквеста

Шаг 4: работа с комментариями

После открытия пул-реквеста вы с коллегами можете обсудить содержащийся в нем код и изменения. Если в репозитории настроены проверки качества кода, убедитесь, что все установленные критерии были соблюдены. Это нужно сделать, пока пул-реквест находится на ревью.

Ревьюеры могут запрашивать изменения, оставлять комментарии и утверждать ваши изменения, доступ к которым вы можете получить на вкладке Conversation.

Шаг 5: развертывание кода

После утверждения пул-реквеста вы можете выполнить развертывание из вашей ветки функциональности в промежуточную или продакшен-среду. Этот шаг во многом определяется вашим рабочим потоком, в частности наложенными в нем процессами автоматизации, сборками для непрерывного развертывания, а также конвейерами развертывания, используемыми командой в ходе разработки.

Функциональность ChatOps позволяет развертывать пул-реквест в продакшен-среде, делая его доступным для любого участника репозитория. В качестве одной из лучших практик в индустрии также используются аналогичные рабочие потоки, но с развертыванием в промежуточной среде, а не продакшене.

Шаг 6: слияние пул-реквеста

Когда пул-реквест готов, вы можете влить его в основную ветку, щелкнув по кнопке **Merge pull request** (рис. 18.46).

Подготовив первую ветку, внесем конфликтующее изменение через другую, созданную из основной. Прodelайте описанные шаги GitHub Flow, чтобы создать другую ветку и внести в нее изменения, затем откройте пул-реквест, но не делайте слияние.



Для этого примера смоделируем создание ветки `ppremk/update-readme` пользователем `ppremk`.

Чтобы вызвать конфликт слияния, нужно влить изменения из предыдущего пул-реквеста, то есть изменения, внесенные пользователем `temyksciraa` через ветку `temyksciraa/add-image` (рис. 18.46).

Рис. 18.46. Вливание изменений

После слияния вы увидите кнопку **Resolve conflicts** в разделе пул-реквеста, созданного вторым пользователем, ppremk (рис. 18.47).

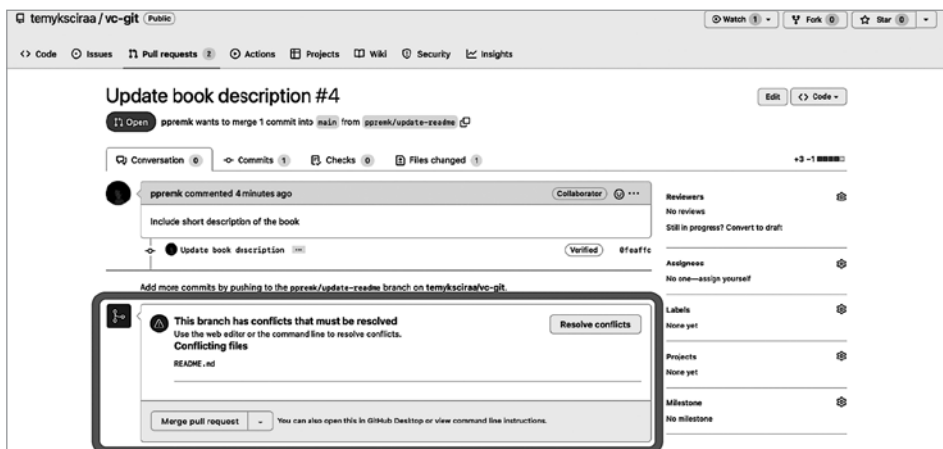


Рис. 18.47. Разрешение конфликтов

На странице **Conflict resolution**, показанной на рис. 18.48, вы увидите маркеры разрешения конфликта вместе с информацией о конфликтующих ветках. Это представлено в виде короткого описания между заголовком пул-реквеста и списком файлов. Разрешите конфликт так, как считаете нужным. Инспектирование и разрешение конфликтов подробно обсуждалось в разделе «Проверка конфликтов» на с. 152 в главе 6.

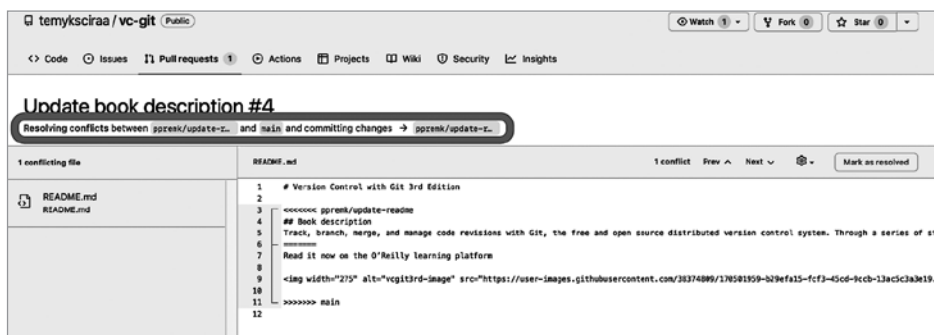


Рис. 18.48. Страница разрешения конфликтов



Когда в файле есть более одного конфликтующего изменения, вы можете переключаться между ними с помощью кнопок **Prev** и **Next**, расположенных в заголовке представления файла.

В нашем примере мы объединяем оба конфликтующих изменения, внесенные через отдельные ветки. Также мы очистили маркеры разрешения конфликтов. В качестве последнего шага щелкните по кнопке **Mark as resolved** (рис. 18.49).



Рис. 18.49. Обозначение конфликта как разрешенного

Когда вы отметили конфликт как разрешенный, у относящихся к нему файлов появится зеленая галочка, указывающая на то, что все конфликты были успешно разрешены. Щелкнув по кнопке **Commit merge** в верхней части страницы, вы перейдете к следующим шагам слияния пул-реквеста (рис. 18.50).



Рис. 18.50. Коммит разрешения конфликта

На странице **Pull request** вы увидите, что кнопка **Merge pull request** снова стала активной (рис. 18.51) и можно продолжить процесс вливания изменений в целевую ветку. Имейте в виду, что разрешение конфликта будет представлено в виде дополнительного коммита со стандартным сообщением **Merge branch main into...**

После слияния перейдите на вкладку **Code**, чтобы просмотреть новое содержимое файла **README.md**, в котором объединены изменения пользователей **temyksciraa** и **ppremk** (рис. 18.52).

В этом примере мы разрешали простые конфликты напрямую через веб-интерфейс GitHub. Но иногда встречаются сложные сценарии, когда конфликт слияния не

может быть разрешен таким образом. В таких случаях используйте командную строку (рис. 18.53).

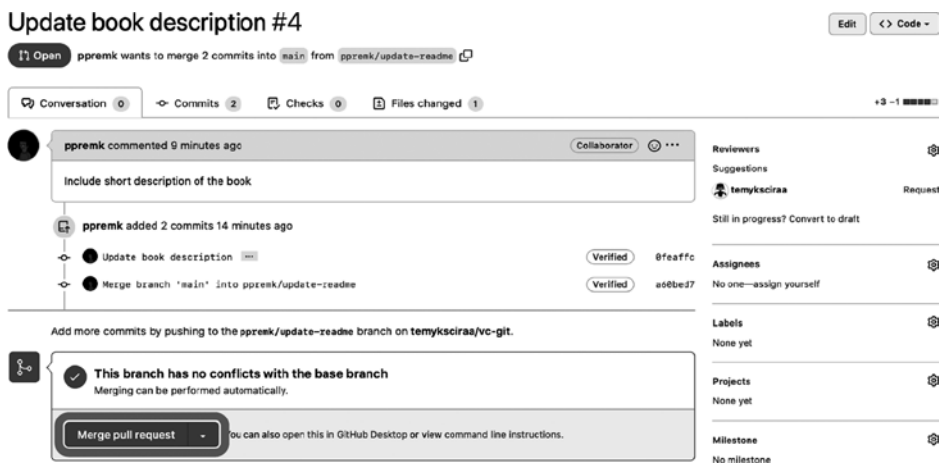


Рис. 18.51. Вливание разрешенного конфликта

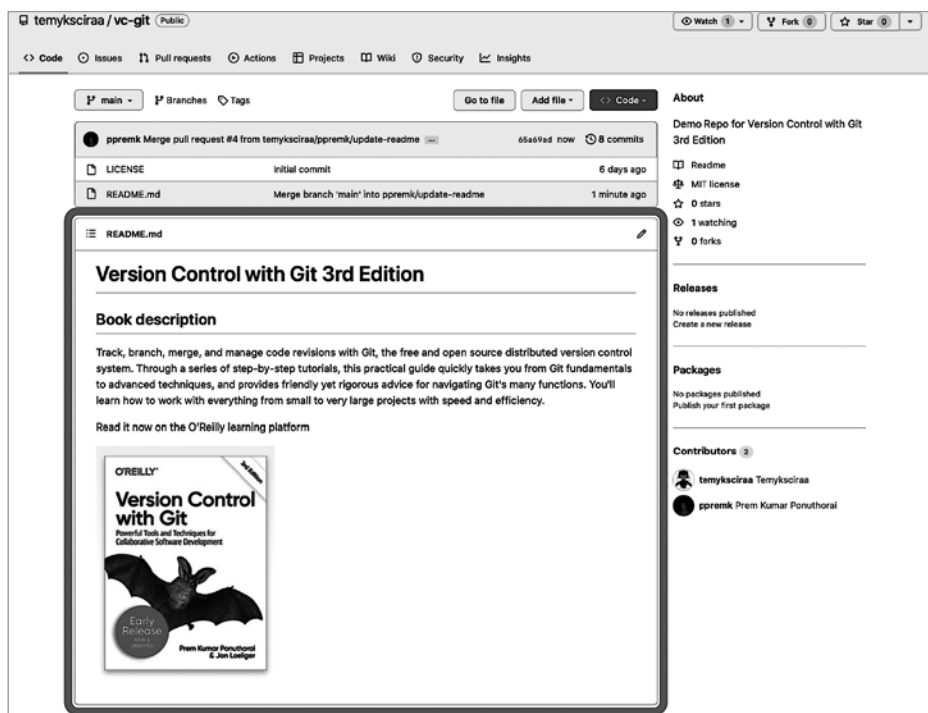


Рис. 18.52. Обновленный файл README.md



Рис. 18.53. Разрешение конфликта из командной строки

GitHub предоставляет несколько советов, как это правильно реализовать. Щелкните на ссылке **command line instructions**, чтобы увидеть описание шагов для разрешения конфликта с локальной машины. После разрешения передайте изменения обратно в вышестоящий репозиторий, чтобы продолжить процесс вливания.

Потоки разработки

Выбор Git в качестве системы контроля версий для командной разработки, а особенно выбор GitHub в качестве хостинга репозитория открывает десятки уникальных сценариев использования. В этом кратком обзоре мы рассмотрим три из них.

Централизованная модель, представленная на рис. 18.54, хотя и предлагает локальное внедрение коммитов, недоступное в по-настоящему централизованных системах вроде SVN, является простейшей и наименее интересной. Ее можно рассматривать в качестве простого первого шага, поскольку разработчики часто отправляют коммиты, имитируя модель «все находится на центральном сервере», которая привычна для них при работе с предыдущим инструментом контроля версий. Хотя это может быть приемлемым подходом для начала работы с Git, это всего лишь один шаг на пути к использованию ценностей распределенной и совместной модели, которую предлагают Git и GitHub.

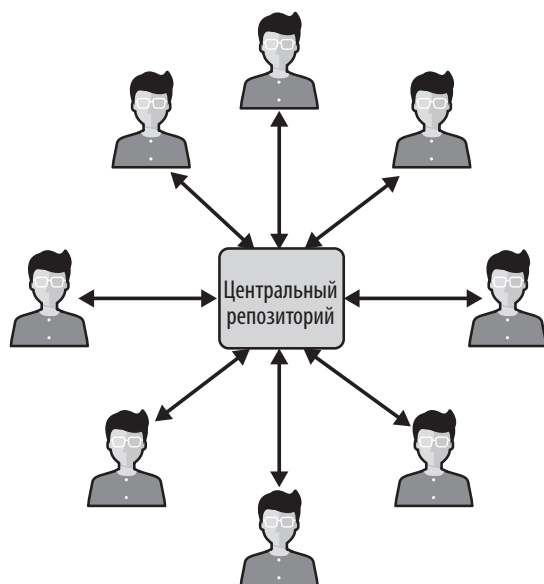


Рис. 18.54. Централизованная модель

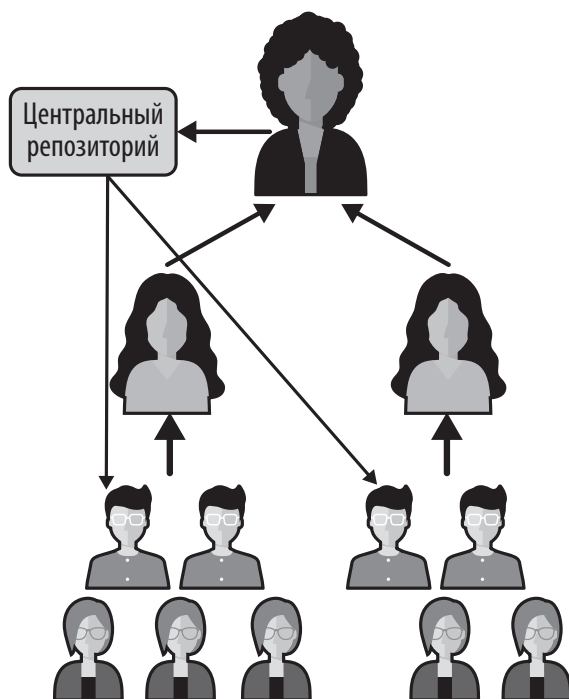


Рис. 18.55. Модель «лейтенант и командир» в Linux

Модель «лейтенант и командир», показанная на рис. 18.55, во многом напоминает ту, которая реализуется с помощью пул-реквестов в GitHub. Следует отметить, что в проектах Git без использования GitHub эту модель можно реализовать через обмен электронными письмами и передачу ссылок в виде патчей, однако это сопряжено с большими трудностями и формальностями по сравнению с реальными пул-реквестами.

Наконец, для компаний, которые ведут опенсорс-разработку и хотят получать исправления багов, но при этом оставлять свои инновации внутри компании, доступен вариант установки для двух репозиториев системы «арбитр». Как показано на рис. 18.56, арбитр выбирает и определяет, какие коммиты нужно отправить обратно в публичный домен открытой версии проекта. Однако по мере того, как все большее число компаний приобретает опыт опенсорсной разработки, популярность такого подхода постепенно падает. Организации либо нанимают разработчиков открытого исходного кода в качестве внештатных сотрудников, либо создают особые команды, где устанавливают политики и правила, определяющие, как разработчик должен вносить вклад в проект, избегая нежелательных юридических последствий.

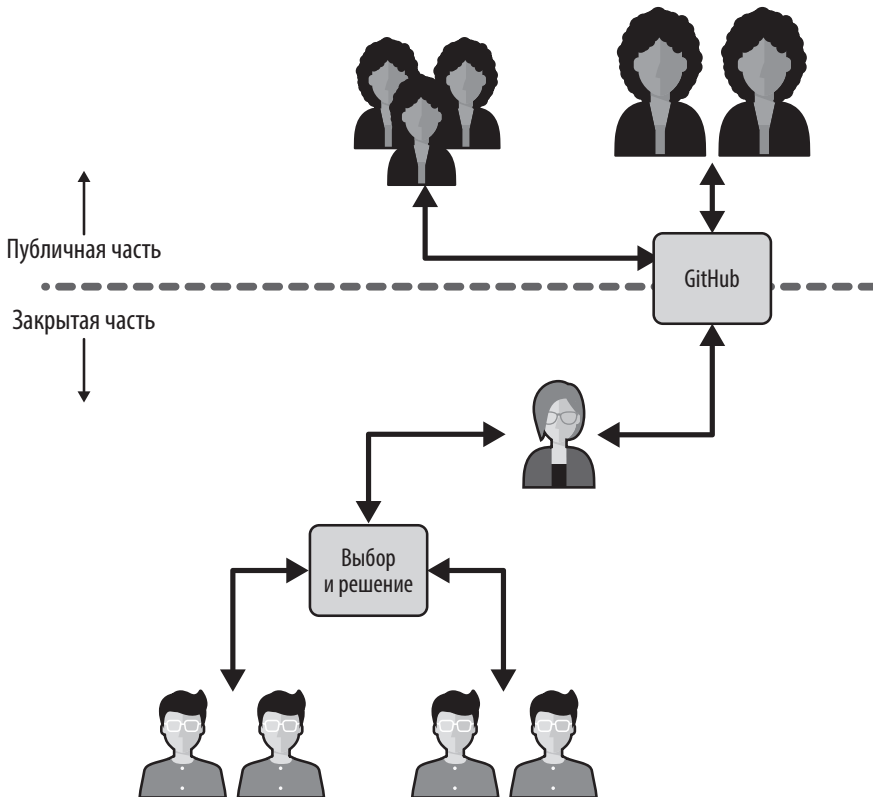


Рис. 18.56. Модель частичного опенсорса

Интеграция с GitHub

GitHub как веб-приложение для хостинга Git — отличная отправная точка. С растущим сообществом разработчиков, которые стремятся использовать реальные сервисы, а не только функции, доступные через веб-интерфейс, GitHub может быть легко интегрирован на различных уровнях.

Разработчики GitHub создали полноценный API, доступный в виде двух стабильных версий: *REST API* и *GraphQL API*. Версия REST API прошла уже три основных релиза, и сейчас REST API v3 (<https://oreil.ly/J6UZg>) предоставляет практически все функции, доступные через пользовательский интерфейс, в форме API. В некоторых случаях продвинутые сервисы предоставляются через API, пока не являющийся частью UI GitHub. При этом GraphQL API (<https://oreil.ly/nQRXP>), напротив, предоставляет различную функциональность, обеспечивая более точный и гибкий способ запроса конкретных данных через поддерживаемых клиентов.

Веб-хуки (<https://oreil.ly/1ISTX>) — еще один вариант интеграции, который вы можете использовать для расширения функциональности GitHub. С их помощью можно создать приложение GitHub или OAuth, которое может быть запущено для выполнения определенной бизнес-логики. Для этого в нем устанавливается прослушивание определенных событий, выбранных на уровне организации или репозитория. Вы можете выбирать конкретные события, поскольку GitHub устанавливает лимит на число передаваемых серверу запросов. Например, применение метки к тикету или его закрытие считается событием. На странице «Webhook events and payloads» (<https://oreil.ly/TH2uC>) приводится список всех доступных веб-хуков и их полезных нагрузок.

На рис. 18.57 показаны примеры кода вызова API GitHub и его ответа.



Ответ на вызов API будет содержать все тикеты, относящиеся к аутентифицированному пользователю во всех видимых репозиториях.

На рис. 18.58 показан пример кода вызова GraphQL API.



Запрос на рис. 18.58 возвращает 20 последних закрытых тикетов (их названия и URL), а также первые пять меток в репозитории `octocat/Hello-World`.

Приложение GitHub (<https://oreil.ly/sVXBc>) имеет прямой доступ к ресурсам и функциональности платформы GitHub и может выполнять широкий спектр действий без необходимости использования промежуточных средств или авторизации от имени конкретного пользователя. Приложение OAuth (<https://oreil.ly/>

TEFHI) использует GitHub для проверки личности пользователей, чтобы аутентифицировать их и предоставить доступ к ресурсам, запрашиваемым приложением. Пользователь должен предоставить доступ к своему аккаунту на GitHub, чтобы приложение могло работать от его имени.

Примеры кода

GET /issues

cURL JavaScript GitHub CLI

```
curl \
  -H "Accept: application/vnd.github.v3+json" \
  https://api.github.com/issues
```

Ответ

Прмер ответа	Схема ответа
Status: 200	<pre>[{ "id": 1, "node_id": "MDU6SXNzdWUx", "url": "https://api.github.com/repos/octocat/Hello-World/issues/1347", "repository_url": "https://api.github.com/repos/octocat/Hello-World", "labels_url": "https://api.github.com/repos/octocat/Hello-World/issues", "comments_url": "https://api.github.com/repos/octocat/Hello-World/issu", "events_url": "https://api.github.com/repos/octocat/Hello-World/issues", "html_url": "https://github.com/octocat/Hello-World/issues/1347", "number": 1347, "state": "open", "title": "Found a bug", "body": "I'm having a problem with this.", "user": { "login": "octocat", "id": 1, "node_id": "MDQ6VXNlcjE=",</pre>

Рис. 18.57. Пример кода REST API

```
query {  
  repository(owner:"octocat", name:"Hello-World") {  
    issues(last:20, states:CLOSED) {  
      edges {  
        node {  
          title  
          url  
          labels(first:5) {  
            edges {  
              node {  
                name  
              }  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

Рис. 18.58. Пример кода GraphQL API

Если вы планируете создавать интеграции с помощью этой модели, изучите раздел документации «Differences Between GitHub Apps and OAuth Apps» (<https://oreil.ly/Wrkso>). Однако есть вероятность, что к моменту выхода книги легаси-приложения OAuth на GitHub не будут поддерживаться.

Итоги

GitHub пошатнул основы многих централизованных систем контроля версий. Эта платформа показала, что высокопроизводительный, коллаборативный и распределенный контроль версий встречается как среди опенсорсных решений, так и в корпоративных средах, использующих аналогичные принципы.

Содержание этой главы лишь поверхностно описывает все возможности при использовании GitHub. Мы не затрагивали GitHub Actions, GitHub Packages, GitHub Pages, GitHub Repository Releases и GitHub Project Board и другие полезные функции, которые вы можете использовать для решения современных задач разработки. Для полноценного знакомства с Git и GitHub в таких практиках разработки, как GitOps или IssueOps, потребовалась бы целая книга.

GitHub показал, что эффективное веб-приложение, выступающее как платформа хостинга Git, может упростить работу с инструментарием, исправление багов, повысить число участников проекта и, что самое главное, превратить написание кода в по-настоящему совместный процесс, сохраняя при этом высокое качество.

История Git

Ни один осмотрительный человек в наши дни не начинает проект без стратегии резервного копирования. Поскольку данные могут быть легко утрачены, например из-за ошибочного изменения кода или катастрофического сбоя диска, то разумно хранить архив проделанной работы.

Стратегия резервного копирования для текстовых и кодовых проектов обычно включает контроль версий, который позволяет отслеживать и управлять изменениями. Каждый разработчик может создавать по несколько ревизий в день, и постоянно растущий объем проекта одновременно выступает в качестве репозитория, описания проекта, средства коммуникации, а также инструмента для управления командой и продуктом. Учитывая его значительную роль, контроль версий оказывается наиболее эффективен, когда используется с учетом рабочего потока и целей команды проекта.

Инструмент, который управляет и отслеживает различные версии ПО или другого содержимого, может называться системой контроля версий (Version Control System, VCS), менеджером исходного кода (Source Code Manager, SCM), системой контроля ревизий (Revision Control System, RCS). Он также может описываться другими комбинациями слов *ревизия*, *версия*, *код*, *содержимое*, *управление* и *система*. Несмотря на различия в названиях, все эти системы решают одинаковые задачи — разработка и обслуживание репозитория содержимого, предоставление доступа к историческим редакциям каждой единицы информации и регистрация всех изменений в журнале. В этой книге термин *система контроля версий* используется в отношении любой формы системы контроля ревизий.

Git — это мощный и гибкий инструмент с низкими издержками, который превращает совместную разработку в удовольствие. Система была создана Линусом Торвальдсом для поддержки разработки ядра Linux¹, но с тех пор стала ценным инструментом для множества проектов.

¹ Linux® является зарегистрированным товарным знаком Линуса Торвальдса, действующим в США и других странах.

Рождение Git

Когда возникает несоответствие между инструментом и проектом, разработчики нередко просто создают новый инструмент. При наличии множества уже существующих систем контроля версий не следует бездумно принимать решение о разработке еще одной. Однако если возникает критическая потребность, а также имеется перспективная идея и здоровая порция мотивации, то создание нового инструмента может стать именно тем, что нужно.

Git, нежно названный своим создателем «информационным менеджером из ада» (Линус известен своим непримиримым характером и сарказмом), как раз такой инструмент. И хотя точные обстоятельства и его время рождения окутаны тайнами сообщества Linux, нет сомнения в том, что в итоге родилась хорошо спроектированная система контроля версий, способная поддерживать масштабную разработку по всему миру.

До Git разработка ядра Linux велась с помощью коммерческой системы BitKeeper, которая предоставляла сложные операции, недоступные в существовавших тогда альтернативах вроде RCS и CVS (Concurrent Versions System, система параллельных версий). Когда компания, владевшая BitKeeper, весной 2005 года установила дополнительные ограничения на ее версию «free as in beer»¹, сообщество Linux осознало, что этот инструмент больше не подходит.

Линус начал искать альтернативы. Отказавшись от коммерческих решений, он изучал свободные программные пакеты, но находил в них те же ограничения и недостатки, которые вынуждали его отказываться от них ранее. Что было не так с существовавшими тогда VCS? Каких возможностей или характеристик, которые искал Линус, им не хватало?

Облегчение распределенной разработки

У распределенной разработки есть множество нюансов, и Линусу требовалась новая VCS, которая охватывала бы большинство из них. Она должна была поддерживать параллельную, а также независимую и одновременную разработку в закрытых репозиториях, без необходимости постоянной синхронизации с центральным репозиторием, который мог стать узким местом. Система также должна была позволять участвовать в разработке множеству географически удаленных участников, даже когда некоторые из них находились офлайн.

Масштабирование для участия тысяч разработчиков

Просто иметь модель распределенной разработки недостаточно. Линус понимал, что в каждом релизе Linux участвуют тысячи разработчиков. Поэтому

¹ Есть два значения слова «free»: свободное (от слова freedom, то есть свобода) и бесплатное (как в словосочетании free beer, то есть бесплатное пиво). — *Примеч. ред.*

любая новая VCS должна обрабатывать очень большое число участников независимо от того, работают ли они над одной или разными частями общего проекта. При этом такая VCS должна надежно интегрировать все результаты их работы.

Быстрая и эффективная работа

Линус хотел, чтобы новая VCS была быстрой и эффективной. Чтобы поддерживать огромный объем операций обновления только для ядра Linux, он понимал, что и отдельные операции обновления, и операции передачи по сети должны быть очень быстрыми. Для экономии места и времени передачи потребуются методы сжатия и дельты. Использование распределенной модели вместо централизованной также позволило бы избежать задержек.

Сохранение целостности и доверия

Поскольку Git — это распределенная система контроля ревизий, она должна обеспечивать сохранение целостности данных, исключая их нежелательное изменение. Откуда вы знаете, что данные не были изменены при передаче от одного разработчика к другому? Или из одного репозитория в другой? И как вообще понять, что эти данные являются тем, чем заявлено?

Для именования и идентификации объектов в базе данных Git использует криптографическую хеш-функцию Secure Hash Function (SHA1). Хотя это не обеспечивает абсолютной гарантии, на практике функция доказала свою надежность в обеспечении целостности и доверия ко всем распределенным репозиториям Git.

Подотчетность

Одним из ключевых аспектов систем контроля версий является знание о том, кто изменил файлы и, если это возможно, почему. Git ведет журнал изменений каждого коммита, изменяющего файл. Информация, сохраняющаяся в этом журнале, определяется разработчиком, требованиями проекта, менеджерами, соглашениями и т. д. Git гарантирует, что файлы, версии которых контролируются, не окажутся загадочным образом изменены, поскольку все изменения фиксируются вместе с их авторами.

Неизменяемость

База данных репозитория Git содержит объекты данных, являющиеся *неизменяемыми*. Это значит, что после создания и размещения в базе данных эти объекты изменять нельзя. Конечно, их можно заново создать в ином виде, но исходные данные поменять без последствий не получится. База данных Git устроена так, что вся история, хранящаяся в базе контроля версий, также неизменяемая. Использование неизменяемых объектов имеет свои преимущества, включая быстрое сравнение на равенство.

Атомарные транзакции

В случае атомарных транзакций различные, но связанные изменения применяются все вместе или не применяются совсем. Это свойство гарантирует, что база данных контроля версий в процессе обновления или внесения коммита не останется в частично измененном или поврежденном состоянии. Git реализует атомарные транзакции, записывая завершенные, дискретные состояния репозитория, которые нельзя разбить на отдельные или меньшие изменения этого состояния.

Поддержка и поощрение ветвления разработки

Практически все системы контроля версий позволяют создавать внутри одного проекта линии разработки, имеющие различное назначение и название. Например, одна последовательность изменений кода может быть названа «разработка», а другая — «тестирование». Каждая система также может разделять отдельные линии разработки на несколько второстепенных потоков, а затем объединять их. Как и в случае с большинством VCS, Git называет линию разработки веткой и присваивает каждой ветке имя.

Наряду с ветвлением в этом процессе есть и слияние. Линус хотел не только обеспечить простоту ветвления, чтобы побудить участников к созданию альтернативных линий разработки, но и простоту последующего объединения ответвленных линий. Поскольку слияние веток в системах версионирования часто было сложной и трудоемкой операцией, новый инструмент должен был обеспечить чистое и простое слияние.

Полноценные репозитории

Чтобы разработчикам не нужно было обращаться к централизованному серверу для получения исторической информации, каждый репозиторий должен содержать полную копию всех прошлых ревизий каждого файла.

Чистый внутренний дизайн

Конечные пользователи могут и не обращать внимания на чистоту внутреннего дизайна, но она важна для Линуса и других разработчиков Git. Используемая в этой системе объектная модель содержит простые структуры, которые отражают фундаментальные принципы сырых данных, структуры каталогов, записи изменений и т. д. Связывание объектной модели с техникой глобально-уникальных идентификаторов позволило создать очень чистую модель данных, которая может быть управляема в распределенной среде разработки.

Be free, as in freedom¹

Этим все сказано.

¹ Есть два значения слова «free»: свободное (от слова freedom, то есть свобода), и бесплатное (как в словосочетании free beer, то есть бесплатное пиво). — *Примеч. ред.*

Получив возможность создать новую VCS с чистого листа, многие талантливые программисты объединили усилия, в результате чего появился Git. Необходимость снова стала матерью изобретения!

Эволюция систем контроля версий

Полная история систем контроля версий выходит за рамки этой книги. Однако есть несколько ключевых инновационных систем, которые заложили основу для развития Git или прямо привели к его созданию. (В этом разделе приведена выборочная информация, отражающая моменты внедрения новых функций или их популяризации в сообществе свободного ПО.)

Source Code Control System (SCCS, система управления исходным кодом) была одной из первых систем в Unix¹, которую разработал М. Дж. Рочкинд (M. J. Rochkind) в начале 1970-х². SCCS стала, пожалуй, первой VCS, доступной в любой системе Unix.

Центральное хранилище в ней называлось репозиторием, и этот фундаментальный принцип сохранился по сей день. SCCS также предоставляла простую модель блокировки для упорядочивания процесса разработки. Если разработчику требовались файлы для выполнения и тестирования программы, он использовал их без блокировки. Если ему требовалось внести в файл правки, он получал его версию для редактирования и блокировал (по соглашению, соблюдение которого обеспечивалось через файловую систему Unix). Закончив вносить изменения, разработчик возвращал файл в репозиторий и разблокировал его.

В начале 1980-х³ Уолтер Ф. Тичи (Walter F. Tichy) запустил Revision Control System (RCS, система контроля ревизий). RCS вводила принципы прямого и обратного дельта-кодирования для эффективного хранения ревизий файлов.

Concurrent Versions System (CVS), спроектированная и изначально реализованная Диком Груном (Dick Grune) в 1986 году, а затем в 1990-м доработанная Берлинером и его коллегами, успешно расширила и изменила модель RCS. CVS завоевала популярность и на многие годы стала стандартом в опенсорсном сообществе (<https://oreil.ly/rK76m>). Она имела несколько преимуществ в сравнении с RCS, включая распределенную разработку и наборы изменений репозитория для целых «модулей».

¹ UNIX® является зарегистрированной торговой маркой The Open Group, действующей в США и других странах.

² Раздел The Source Code Control System, *IEEE Transactions on Software Engineering* 1 (4) (1975): 364–370.

³ Раздел RCS: A System for Version Control, *Software Practice and Experience* 15 (7) (1985): 637–654.

Более того, CVS внесла новую парадигму блокировки. Если в предыдущих системах разработчику требовалось блокировать каждый файл, прежде чем изменять его, в связи с чем возникала очередь из ожидающих, то CVS предоставляла каждому разработчику разрешение на запись в его закрытой рабочей копии. Система могла автоматически совмещать изменения, внесенные разными разработчиками, если только они не относились к изменению одной строки. В этом случае обозначался конфликт, и разработчикам нужно было разрешить его. Новые правила блокировки позволили писать код многим разработчикам одновременно.

Как это часто бывает, недоработки CVS привели к созданию следующей системы — Subversion (SVN). SVN появилась в 2001 году и быстро стала популярной в сообществе бесплатного ПО. В отличие от CVS, SVN коммитила изменения автоматически и предлагала значительно улучшенную схему ветвления.

BitKeeper и Mercurial стали радикальным отходом от всех упомянутых выше решений. Разработчики отказались от идеи центрального репозитория, реализовав распределенное хранилище. Теперь каждому предоставлялась собственная копия, которой можно было поделиться с другими. На основе этой одноранговой модели и была построена Git.

Наконец, в Mercurial and Monotone начал использоваться хеш-отпечаток для идентификации содержимого файла. В этом случае присваиваемое файлу имя выступает только как псевдоним для удобства пользователя. Git также использует этот принцип, где идентификатор основывается на содержимом файла. Это называется *файловым хранилищем с адресацией содержимого*¹. Концепцию Линус позаимствовал из Monotone². Mercurial же реализовала ее параллельно с Git.

Хронология

После подготовки, внешнего толчка и нарастающего кризиса в системах контроля версий в апреле 2005 года был создан Git. Первый самостоятельный коммит Git совершил 7 апреля:

```
commit e83c5163316f89bfbde7d9ab23ca2e25604af29
Author: Linus Torvalds <torvalds@ppc970.osdl.org>
Date: Thu Apr 7 15:13:13 2005 -0700
```

Initial revision of "git", the information manager from hell

Вскоре после этого был сделан первый коммит Linux:

```
commit 1da177e4c3f41524e886b7f1b8a0c1fc7321cac2
Author: Linus Torvalds <torvalds@ppc970.osdl.org>
```

¹ См.: «Venti: A New Approach to Archival Storage» (<https://oreil.ly/A5ljM>), Plan 9, Bell Labs.

² В личной переписке.

Date: Sat Apr 16 15:20:36 2005 -0700

Linux-2.6.12-rc2

Initial git repository build. I'm not bothering with the full history, even though we have it. We can create a separate "historical" git archive of that later if we want to, and in the meantime it's about 3.2GB when imported into git - space that would just make the early git days unnecessarily complicated, when we don't have a lot of good infrastructure for it.

Let it rip!

Этот коммит вносил всю огромную систему ядра Linux в репозиторий Git¹ и состоял из:

17291 files changed, 6718755 insertions(+), 0 deletions(-)

Да, он нес в себе 6,7 миллиона строк кода!

Спустя всего три минуты к ядру с помощью Git уже был применен первый патч. Убедившись в работоспособности системы, 20 апреля 2005 года Линус объявил о ее запуске через список рассылки ядра Linux.

Линус в первую очередь хотел вернуться к разработке ядра, поэтому 25 июля 2005 года передал обслуживание исходного кода Git доверенному разработчику Джунио Хамано, заявив, что «выбор Джунио оказался вполне очевидным».

Примерно через два месяца вышла версия ядра 2.6.12, уже с использованием Git.

Что означает название Git

Сам Линус про название «Git» говорит так: «Я эгоистичный @\$%& и называю все свои проекты в честь себя. Сначала Linux, теперь git». Название «Linux» действительно стало смесью слов Linus и Minix. Во втором же случае прослеживается ирония в использовании английского слова, означающего «глупый» или «никчемный» человек².

С тех пор были предложены другие, более приемлемые интерпретации этого названия, наиболее популярной среди которых стала *Global Information Tracker* (глобальная система отслеживания информации).

¹ Вот статья об истории Linux (<https://oreil.ly/fj0nf>), где говорится о том, что старые логи Linux (до версии 2.5) импортировались из BitKeeper в репозиторий Git.

² Сленговая трактовка слова git. — *Примеч. пер.*

ПРИЛОЖЕНИЕ Б

Установка Git

Чтобы начать использовать систему Git, ее нужно установить. Этапы установки будут зависеть от поставщика и версии вашей ОС. В этом приложении описан процесс установки Git в Linux, macOS и Microsoft Windows. Вы также узнаете, как получить исходный код Git, чтобы выполнить сборку и установку на его основе.

Использование бинарных дистрибутивов Linux

Многие поставщики Linux предоставляют скомпилированные бинарные пакеты, упрощающие установку приложений, инструментов и утилит. В каждом пакете указываются его зависимости, и пакетный менеджер устанавливает все необходимые компоненты вместе с самим пакетом в один шаг.

Debian/Ubuntu

В большинстве систем Debian/Ubuntu Git предоставляется в виде коллекции пакетов, которые можно установить независимо исходя из ваших потребностей. До релиза 12.04 основной пакет Git назывался `git-core`. В следующих версиях название сократилось до `git`. К примерам пакетов Git относятся `git-gui`, `gitk`, `gitweb` и `git-svn`.

Поскольку дистрибутивы существенно различаются, полный список относящихся к Git пакетов лучше всего искать в хранилище пакетов вашего дистрибутива.

Эта команда установит последнюю стабильную версию Git в ОС Debian/Ubuntu:

```
$ apt-get install git
```

Другие бинарные дистрибутивы

Для установки Git на других дистрибутивах Linux найдите соответствующий пакет или пакеты и используйте менеджер пакетов дистрибутива для установки:

Gentoo

```
$ emerge --ask --verbose dev-vcs/git
```

Arch Linux

```
$ pacman -S git
```

Fedora

```
# Включая версию Fedora 21.
$ yum install git

# Для версий Fedora 22 и старше.
$ dnf install git
```

В других дистрибутивах

Для других популярных дистрибутивов все необходимые команды ищите в документации (<https://oreil.ly/eods1>).

В некоторых дистрибутивах релиз Git может быть распределен по нескольким пакетам. Если в вашей системе нет какой-то команды Git, то нужно установить дополнительный пакет.

Убедитесь, что пакеты Git в вашем дистрибутиве актуальные. После установки Git выполните `git --version`, чтобы узнать свою версию. На момент написания книги последней версией была Git 2.37.0¹.

Установка Git в macOS

В macOS вам доступны различные варианты установки Git с помощью пакетного менеджера или установщиков двоичных дистрибутивов:

Homebrew

```
$ brew install git
```

MacPorts

```
# Для установки Git в системе должен быть установлен MacPorts.
$ sudo port install git
```

Другие установщики

Популярные установщики приводятся в документации `git-scm` (<https://oreil.ly/8QlHq>).

¹ На момент подготовки русскоязычного издания это версия 2.45.2. — *Примеч. ред.*

Установка Git в Windows

В Windows установить Git можно с помощью отдельного установщика, портативного установщика, а также через пакетный менеджер Windows winget (<https://oreil.ly/OfzUG>) либо через поддерживаемый сообществом менеджер chocolatey (<https://oreil.ly/h5NLj>):

Git для Windows

Скачайте установщик (<https://oreil.ly/RsWSY>) и следуйте инструкциям для настройки и выбора опций.

winget

```
# Выполните из Powershell или командной строки.  
PS C:\>winget install --id Git.Git -e --source winget
```

Chocolatey

```
# Выполните из Powershell или командной строки.  
PS C:\>choco install git.install
```

Получение исходного кода релиза

При желании вы можете скачать исходный код Git (<https://mirrors.edge.kernel.org/pub/software/scm/git>) с официального источника или выбрать определенную версию в виде tar-архива. В этой книге использовалась версия Git 2.37.0:

```
$ wget https://mirrors.edge.kernel.org/pub/software/scm/git/git-2.37.0.tar.gz  
$ tar xzf git-2.37.0.tar.gz  
$ cd git-2.37.0
```

Сборка и установка на основе исходного релиза

Git похож на другие программные продукты с открытым исходным кодом. Настройте его, выполните команду `make` и установите. Проще простого, правда? Но есть свои нюансы.

Если в вашей системе есть нужные библиотеки и надежная среда сборки и если вам не требуется настраивать Git, то сборка кода будет мгновенной. Если же на вашей машине нет компилятора или набора серверных библиотек и библиотек для программной разработки или вы никогда не собирали сложное приложение из исходного кода, тогда лучше рассматривать сборку Git с нуля только в крайнем случае. Git предоставляет огромные возможности конфигурации, и к сборке нужно относиться серьезно.

Для продолжения сборки ознакомьтесь с файлом `INSTALL` (<https://oreil.ly/dCEy3>) в наборе исходного кода Git. В нем приводится список нескольких внешних зависимостей, включая библиотеки `zlib`, `ssh` и `libcurl`.

Некоторые необходимые библиотеки и пакеты могут быть частью более крупных пакетов.

Как только ваша система и параметры сборки готовы, дело за малым. Следующие команды выполнят сборку и установят Git в домашний каталог (мы добавили флаг `--prefix=/usr/local`, чтобы выполнить установку для всех пользователей):

```
$ cd git-2.37.0
$ ./configure --prefix=/usr/local
$ sudo make all
$ sudo make install
```

```
# Обновление оболочки.
$ exec bash
```

```
$ git --version
git version 2.37.0
```

```
$ which git
/usr/local/bin/git
```

Для установки документации Git добавьте к командам `make` и `make install` цели `doc` и `install-doc` соответственно:

```
$ cd git-2.37.0
$ make all doc
$ sudo make install install-doc
```

Сборка из исходного кода включает все подпакеты и команды Git. Отдельно собирать или устанавливать эти утилиты не нужно.

Об авторах

Прем Кумар Понуторай отвечает за разработку стратегии и внедрение предложений Expert Services Delivery на GitHub. Обладая опытом в области программной инженерии и являясь приверженцем Git, проводит семинары по Git на конференциях и обучает работе в Git корпоративных клиентов из различных отраслей.

Джон Лолигер (Jon Loeliger) — программист-фрилансер, участвующий в разработке открытых проектов Linux, U-Boot и Git. Неоднократно выступал с докладами по Git на масштабных конференциях, включая Linux World. Написал по теме Git несколько работ для *Linux Magazine*.

Иллюстрация на обложке

На обложке книги «Git: контроль версий» изображен ушан. Это вид летучих мышей, распространенный на территории Великобритании и Ирландии, их также можно встретить в Японии. Собираясь в колонии, включающие от 50 до 100 и более особей, ушаны живут на открытых лесистых местностях, в парках, садах, а также в помещениях под крышами жилых домов и церквей. Зимуют в пещерах, где ведут обособленный образ жизни.

Ушан — летучая мышь среднего размера с размахом крыльев около 25 см. Уши длинные и имеют характерную складку. Внутренние края сходятся на макушке, а внешние заканчиваются за изгибом рта. На время сна ушан сворачивает уши под крылья, а в полете направляет их вперед. Шерсть длинная, пушистая, шелковистая. Окрас темно-коричневый сверху и светло-коричневый снизу. Молодые особи имеют светло-серую окраску. Питаются ушаны мухами, мотыльками и жуками. Во время охоты ушан парит среди листвы, часто зависая в воздухе в поиске насекомых.

Период размножения — осенью и весной. Беременные самки собираются в колонии по 100 и более особей ранним летом, после чего в июне-июле рожают одного или двух детенышей. Летучие мыши — единственные истинные млекопитающие среди летающих видов животных. Вопреки популярному заблуждению, они не слепы, и многие из них видят достаточно хорошо. Все летучие мыши, обитающие на территории Великобритании, в ночное время ориентируются в пространстве с помощью эхолокации. Они испускают ультразвуковые волны, частота которых превышает слышимый человеком диапазон, после чего прослушивают и интерпретируют отражения этих волн от находящихся вокруг объектов (включая добычу). Это позволяет им выстраивать «звуковую картину» окружающего пространства.

Как и другие летучие мыши, этот вид подвержен множеству угроз, включая потерю мест обитания вследствие того, что старые полые деревья часто спиливают в целях безопасности. Огромный вред для ушанов несет использование пестицидов: это снижает число насекомых и приводит к отравлению пищи потенциально смертельными токсинами. Особую опасность представляют инсектициды, которыми обрабатывают древесные строительные материалы в зданиях, где мыши организуют места обитания. Обработка может вести к истреблению целых колоний мышей (такая обработка незаконна), но самое страшное то, что химикаты могут сохранять отравляющее воздействие вплоть до 20 лет. В Великобритании

в соответствии с Законом о дикой природе и сельской местности (Wildlife and Countryside Act) запрещается намеренно убивать, калечить, ловить или продавать летучих мышей, владеть живой летучей мышью или частями ее тела, а также намеренно повреждать или разрушать места обитания летучих мышей. Нарушители могут быть подвергнуты штрафу в размере до 5000 фунтов за каждую летучую мышь и тюремному заключению на срок до шести месяцев.

Многие из животных с обложек книг O'Reilly находятся под угрозой вымирания. Все они очень важны для нашего мира.

Иллюстрацию обложки этой книги выполнила Карен Монтгомери (Karen Montgomery), взяв за основу гравюру из книги *Dover Animals*.